队伍编号	MCB2600843
题号	B

基于多源观测的吉隆口岸冰崩—泥石流链关联与条件预测

摘 要

2026 年 8 月 26 日 10:30，吉隆口岸遭受高位冰崩引发的碎屑流—泥石流—山洪链式冲击。本文利用题给多源观测、灾前地形与公开气象产品，建立“关联排序—起动判别—固定发布时刻条件预报—情景传播”的分析框架，并区分可核验的条件结果与尚不可识别的真实触发。

针对问题一，本文从 MET、RADM、SOIL、FLUX 四类逐时观测固定 31 项唯一候选，在 93 个同季参照窗上计算事件前 168 小时稳健异常，并以 Spearman 相关簇去冗余。正式前五依次为前期降水指数、净辐射、24 小时升温、0.1 米土壤含水率和潜热通量，A 分别为 3.829043、2.343641、2.234448、1.487331 和 1.133226，方向均为事件窗偏高。该结果是单事件环境关联，2025 年同类窗口最大 $A = 3.336817$，不足以证明普适前兆或因果效应。

针对问题二，本文在床面受力中加入床体沿坡自重，区分 78 条水流控制与 6 条重力失稳沟道。产流采用完整降雨事件的累计 SCS-CN 扣损，逐时径流取累计曲线差；事件边界由连续六个零雨小时确认，场初曲线数由前一小时 API 确定。正式中心线单宽与断面临界流量分别为 $q_c = 67.588\ \mathrm{m^2/s}$ 和 $Q_c = 2.893667 \times 10^3\ \mathrm{m^3/s}$；事件窗水源峰值 $202.650\ \mathrm{m^3/s}$，最大触发比 0.070032，10:00 为 0.018814，六小时均未越阈。335 个厚冰陡坡像元仅是源区候选；冰体强度、裂隙和位移缺失，冰力学状态尚不可识别。

针对问题三，本文固定 2026 年 8 月 26 日 08:00 当地时发布，只采用 06、12、18 UTC 三次 IFS 单次起报及其到达发布端的六小时上界，生成 3000 个配对成员。短波和降水按原始结束标签对齐到 $[t, t+1)$，温湿取两端时点值的梯形均值。源区融水代理的 95% 条件区间为 09:03:22–10:57:51，即时释放假设下的口岸条件到达区间为 09:35:26–11:31:59。真实释放、形成和实际到达尚不能估计，真实预警与覆盖率也未给出；10:30 仅落入辅助即时释放区间（1/1），不代表真实覆盖，第三问的真实触发预报仍未完成。08:00 融水提示相对锚点有 2.5 小时信息时差，辅助到达区间相对锚点的提前量为 -1.0332 至 $+0.9094$ 小时；同链沿沟距离为 22.924 千米。

针对问题四，混合入沟体积为 $2.635835 \times 10^6\ \mathrm{m^3}$。D8 基线 32.766603 分钟到达口岸代理，口岸历史峰深 10.085302 米，质量相对残差 2.075820×10^{-15}、最大 Courant 数 0.188281。16 个配置全部终止于 26045.494670 米分析域末端的右删失，并非自然最大运程；一维基线则在 9.336769 千米处自然能量停止。数值闭合说明给定离散方案内守恒一致，不构成地形、流变或现场过程验证。

外部气象产品同日降水差异明显，故并列对照而不互相替代。真实泥石流形成真值、现场流深流速及沿程可动库存仍缺失，因此本文给出条件预报及其适用范围，不提供真实灾害 95% 覆盖保证。

关键词：统一候选关联；床体自重；前向预报；右删失；D8 路由

目录

一、 问题重述与研究设计 . . . 1
1.1 任务与需要分开的时间对象 . . . 1
1.2 资料、来源与用途 . . . 1
1.3 坐标与位置的来源说明 . . . 1
1.4 质量控制和空间一致化 . . . 2
1.5 外部气象对照 . . . 3
二、 问题分析与建模思路 . . . 3
三、 基本假设与符号约定 . . . 4
3.1 公共假设与适用范围 . . . 4
四、 问题一：近地层结构、天气过程与统一关联排序 . . . 5
4.1 问题口径与分析范围 . . . 5
4.2 五层静力订正与稳定度诊断 . . . 5
4.3 辐射收支、天气过程与极端事件 . . . 6
4.4 31 项统一候选关联模型 . . . 8
4.5 正式前五与解释 . . . 9
4.6 构造性对照与适用限制 . . . 10
五、 问题二：床面临界条件与多尺度触发评估 . . . 10
5.1 问题定义与适用边界 . . . 10
5.2 完整受力、临界链条与三个分支 . . . 10
5.3 对象级状态元组与冰崩链状态识别 . . . 11
5.4 水源流量与事件前处理 . . . 12
5.5 事件窗求解与水力状态 . . . 13
5.6 历史规模关联与双语义降雨包络 . . . 14
5.7 时间与空间可靠性 . . . 14
5.8 综合判读 . . . 15
六、 问题三：固定发布时刻的条件预报与真实触发识别 . . . 16
6.1 任务目标与可识别范围 . . . 16
6.2 变量时间对齐与历史能量关系 . . . 16
6.3 条件区间与支持包络 . . . 17
6.4 单事件包含、提示时差与真实预警 . . . 19
6.5 空间距离与右删失运程 . . . 19
6.6 真实释放的识别条件 . . . 19
6.7 一致性检查与适用限制 . . . 20
七、 问题四：双引擎情景演算、右删失处理与有限库存机制 . . . 21
7.1 任务目标与模型定位 . . . 21
7.2 公共几何、源区体积与注入边界 . . . 21
7.3 一维诊断引擎与自然停止 . . . 22
7.4 D8 屈服门槛—Manning 路由与临界量 . . . 23
7.5 D8 过程、峰值与域末端右删失 . . . 24
7.6 质量收支与数值闭合 . . . 25
7.7 有限库存、静止混合与分段动力对照 . . . 25
7.8 综合判读与适用范围 . . . 27
八、 综合检验、适用范围与防灾策略 . . . 27
8.1 结果解释与适用范围 . . . 27
8.2 不确定性的来源 . . . 28
8.3 分层预警与监测建议 . . . 28
8.4 模型评价与改进顺序 . . . 29
九、 结论 . . . 29
参考文献 . . . 29
附录 1： 支撑材料与程序说明 . . . 31
附录 2： 补充诊断图 . . . 31
附录 3： AI 使用声明 . . . 34
附录 4： 软件环境与代码 . . . 35

一、问题重述与研究设计

1.1 任务与需要分开的时间对象

2026 年 8 月 26 日 10:30，高位冰崩引发的碎屑流、泥石流与山洪链式过程冲击吉隆口岸。此类高位冰崩—碎屑流—泥石流链在喜马拉雅山区多次成灾，其运动性受冰水含量、体积与沿程地形共同影响[1;2;3]。题给时间是口岸冲击锚点，并非源区冰体最初失稳的直接观测。研究需回答四个相连的问题：气象与近地层环境如何关联灾害，何种水源和床面条件达到触发门槛，何时可能触发及如何给出区间与预警，过程在既定地形和参数下如何演化。

令 t_m 为源区融水起始代理，t_{trig} 为真实泥石流形成时刻，$t_a^{(0)}$ 为同一成员在即时释放假设下传播到口岸代理的条件到达时刻，t_a 为实际到达，t_p 为固定发布时刻，t_{hit} 为题给口岸冲击锚点。冰体失稳可能发生在融水阈值之前，t_m 与 t_{trig} 并不等同。为避免同一次事件参与自身验证，本文固定 t_p =2026-08-26 08:00 当地时，先产生源区与口岸配对成员，再以 10:30 作事后单点核对。四个问题分别回答环境异常、力平衡状态、条件时间区间和传播情景，辅助包含只作单点核对。

地形、物源和水源分别对应能量、物质与触发条件。本文以水源供给与床面起动比值作为主状态，避免把资料条件不同的过程压缩为单一概率；空间模型用于环境对照，运动模型给出条件传播，各问共用同一条公共中心线。

1.2 资料、来源与用途

题给八类资料、其发布机构与本文用途见表1。来源以机构名与产品名注明，请求参数与产品版本随成果保存。

融水面积权重取附件 3 的 RGI6 编目多边形与集水区的面积交比，属于编目年代边界，不是 2026 年实时冰川或积雪范围；高程带代理并列报告但不进入融水。补充的 ERA5 与 MERRA2 用于同一 UTC+8 自然日的产品对照；IFS 采用按单次起报保存的 06、12、18 UTC 档案，分别记录起报时刻、可用上界、有效小时和产品版本。起报可用性表示信息时次满足发布要求，源区冰面实况仍需独立观测确认。

站表共 49545 行，物理有效 46755 小时；五层温湿共同有效 45577 小时，可计算整体理查森数的共同样本 44018 小时。历史 84 处灾点中降雨型 41、冰水混合型 25、冰雪融水型 16、冰湖溃决型 2，大及特大规模合计 9 处。该目录每个对象都已发生过灾害，因此无法据此构造完整的发生/未发生样本；DEM 相交灾点只是其中一部分，像元样本量不等于独立灾害数量。人口“合计”字段为 17536 人，只给县级背景，不按均匀密度分摊到狭窄演算走廊。

1.3 坐标与位置的来源说明

题面只给“吉隆口岸”，未给实际受灾断面坐标。本文以 (85.3826°E,28.2826°N) 为工作点，以 (85.3777°E,28.2797°N) 为参考候选，两者平面相距约 571 米，均按 DEM 近似位置处理。固定源区、网格、体积和动力参数，仅改变目标点重演：两候选的路线吸附偏移为 5.186582 和 27.40 米，D8 到达为 32.766603 和 33.90 分钟，峰流深 10.085302

表 1 题给资料及用途

附件	资料范围	来源产品	用途与限制
1	2021–2026 逐时 MET/RADM/SOIL/FLUX	吉隆灾害专题（国家青藏高原科学数据中心）	温湿风廓线、辐射、土壤及湍流诊断；单站代表性有限
2	2005–2025 周边县日均温（至 2025-08）	ERA5-Land 逐日统计产品，同上	长期气候背景和同口径积温对照，不代替源区温度观测
3	2000–2020 四期高程变化	全球冰川高程变化产品，同上	有效像元加权区域背景，均值误差不视为区域均值标准误
4	灾前冰川 DEM、冰厚、速度	冰川地形与冰厚遥感产品，同上	原生源区积分、源几何与速度重叠旁证
5	2020–2023 土地覆盖	土地覆被遥感产品，同上	原类码作为分类变量；未确认类码不解释为建设用地
6	84 处历史灾点面及属性	历史灾点调查数据，同上	沟道临界域、规模排序、历史等效摩擦和面内像元对照
7	事件日期研究域 DEM	事件区数字高程模型，同上	公共水文地形与路由载体；日期不证明灾前或灾后采集
8	第七次县级人口普查	县级普查汇总，同上	县级承灾背景，不空间化为精确暴露人口

和 7.42 米，峰速 6.601474 和 8.82 米每秒。5.19 米是路线吸附偏移，不代表口岸定位精度；两候选间距另行报告。

站点坐标 (85.30°E, 28.40°N) 是吉隆沟高寒山区的假定代表点：原始 MET、RADM、SOIL 与 FLUX 档案均不含站点元数据。海拔取气压反演值 3978.85 m（标准大气推断），不是实测安装高度；辐射与触发计算取整使用 3980 m，与反演值相差 1.15 m，远小于反演不确定度。太阳能时角计算使用该经度，经度假定误差会线性传播至辐射诊断。

1.4 质量控制和空间一致化

四类站点资料先按时间戳拼接、去重并重建连续小时轴；哨兵值及不合理物理值置空，质量码释义未充分提供时保守屏蔽非零码。滚动三小时要求三小时均有效，三日小时正积温要求 72 小时完整；普通自然日聚合至少 20 个有效小时。日光反照率和晴空指数只在符合日光条件的时段考察，采用有效比例要求而非要求不存在的 20 小时日照。无观测小时不补零降水或融化；跨缺测的 API 先重启并保留暖启动期，雨事件分割只把完整观测的干期当间隔，不完整过程的水源状态记为未知。该处理减少可用样本，但避免把缺测误作无雨无融；插补、分位与标准化只在训练集拟合，事件日全日资料仅作描述，不作为事前特征。

空间资料统一以像元中心定位。经纬网格的米制尺寸近似为

$$d_y = 110574|\Delta\phi|, \qquad d_x = 111320\cos\phi_0|\Delta\lambda|, \quad \phi_0 = 28.4^\circ. \tag{1}$$

主网格南北约 30.71 米、东西约 27.20 米，面积约 835.27 平方米，与 900 平方米的名义值有明显差异。冰川原生 30 米 UTM 网格上的体积先积分再投影，防止多个原生像元落到同一演算格后重复积分；粗化网格原点位于整个块的中心而非左上第一个细格中心。D8 选向按真实链接长度比较最大坡降，与后续流量、时间和 CFL 所用几何一致。

1.5 外部气象对照

请求明确指定 ERA5、UTC 与关闭海拔自动降尺度；NASA POWER 请求明确指定 UTC。两套逐时响应先转为 UTC+8，再按完整 24 小时汇总事件自然日。两产品事件日

表 2 同日界外部气象对照

产品	响应代表高程/m	日均/最高/最低温度/°C	日降水/mm
ERA5（Open-Meteo 接口）	4322	9.41/13.10/6.40	7.50
MERRA2（NASA POWER 接口）	4836.18	6.09/11.06/2.43	1.22

降水差异明显，日均温相差约 3.32 K。格点代表高程相差约 514 m，但温度差与高程差不同向，无法据此反推直减率；粗格点、下垫面与模式偏差未分离，两个接口也不保证统计独立，因此只作并列对照，不替代站点观测。

第三类资料是 IFS 单次起报。主起报为 2026 年 8 月 25 日 18 UTC，并以同日 06、12 UTC 作模式演化对照；按六小时分发上界，三者最迟分别于 25 日 12 UTC、18 UTC 和 26 日 00 UTC 可用，均不晚于固定发布时刻。每次取发布前最后完成小时和其后 24 个有效小时，源区与口岸配对生成 1000 个成员。普通短波与降水按原始结束标签对齐到 $[t, t+1)$，温湿取两端时点值的梯形均值；任一端点缺失则该小时不可评估，不用下一条记录跨过时间缺口。18 UTC 口岸格点序列只作独立上下文，不进入主传播链。该流程保证信息时次不晚于发布；模式输出与冰面实况的一致性需另行检验。

站点对照按 UTC+8 转 UTC 日，仅保留温降水均有 24 个有效小时的共同日期；2021–2025 分别保留 247、363、361、363、214 日。2025 同窗口站点降水 102.00 毫米、POWER 为 742.41 毫米；站点年总量不完整，二者不作同口径比较。站点位置、时区与小时标签仍是假设，故只是代表点产品差异。

原始响应、请求参数与产品版本随成果保存；公开站点元数据缺失，真实安装高度及时区仍未知，海拔按反演值处理，不视为实测信息。

二、问题分析与建模思路

四个问题共享同一套资料、时间轴和公共几何，但回答对象不同：问题一识别事件前环境异常，问题二判断水源与床面是否达到起动条件，问题三在固定发布时刻给出条件时间产品，问题四在给定体积与地形下演算传播。图1给出全文技术路线。

四问共享时间基准、坐标和注入断面，箭头只表示计算依赖；本文将结果分为计算闭合、模型内的条件分布与真实灾害外推三层，结论限于前两层。

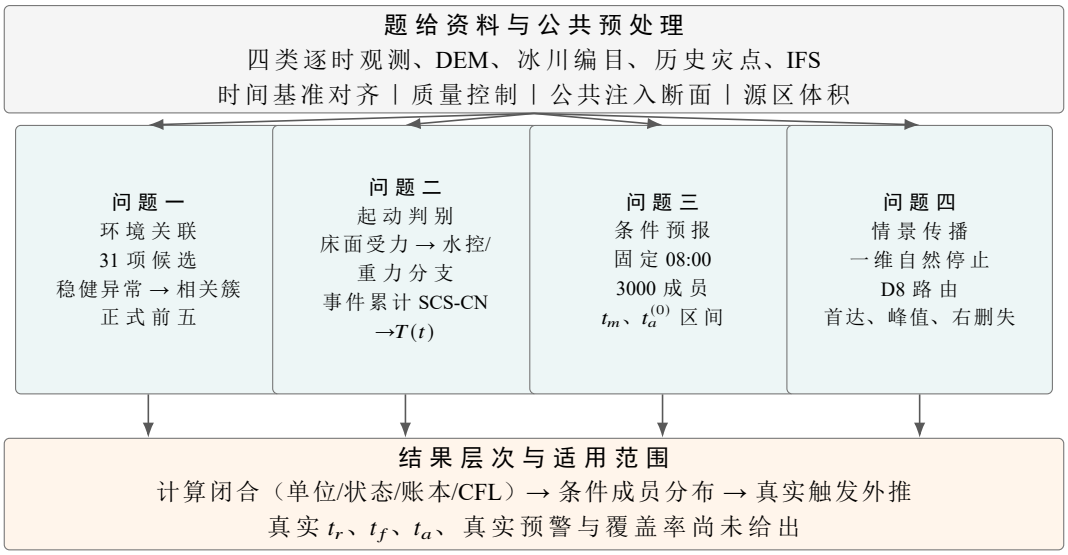


图 1 全文建模思路与结果层次

三、基本假设与符号约定

3.1 公共假设与适用范围

跨问题主要符号见表3；公共假设逐条列于其后，编号 H0-1 至 H0-9。

表 3 主要符号

符号	意义	单位
θ_v, Ri_b	虚位温、整体理查森数	K、无量纲
R_n, H, LE, G	净辐射、感热、潜热、土壤热通量	W/m^2
e_m, m_{src}, m_{cat}	小时能量、源区/流域等效融水	MJ/m^2、mm
Q_w, Q_c, q_c	水源流量、起动流量、单宽起动流量	m^3/s、m^2/s
T, T_w, P	触发指数、双通道候选指示、单调相对评分	无量纲
S, n, τ_y, μ_H	比降、糙率、屈服门槛、等效摩擦	1、s/m$^{1/3}$、Pa、1
V_{src}, V_{rel}, V_{df}	源冰总量、释放冰量、混合入沟体积	m^3
h, v, I	等效泥深、速度、单位宽动量通量	m、m/s、N/m
$t_m, t_{\mathrm{trig}}, t_a^{(0)}, t_a$	融水起始、真实形成、条件到达、实际到达	h
t_p, t_{hit}	固定发布、题给冲击锚点	h
$V_{inj}, V_{dom}, V_{out}$	累计注入、域内、累计出域体积	m^3
μ_{dyn}, κ, v_0; $\Delta V_{a,j}, B_{a,j}$	动力与节点库存	1、m^{-1}、m/s、m^3

1. 题给冲击时刻与附件逐时记录按同一 UTC+8 标度对齐；IFS 原始有效时刻按 UTC 保存，再转换到当地发布窗。时区元数据缺失，时间对齐仍是主要不确定性。

2. 单站代表同一高程带的大尺度强迫，源区气温按 0.5–0.7 K/(100 m) 直减率做外推敏感性；不假定局地风和冰体温度已知。

3. 能量余量表示潜在融化能量，闭合缺口不全部归因于实际消融。冰厚、源区比例与物源放大均作为明示场景参数。

4. 模型成员概率、条件分位、确定配置包络和真实事件频率分别命名，避免混合成单一的真实灾害置信区间。

5. 前向链的 t_m 是融水起始代理，t_{trig} 是真实泥石流形成时刻；$t_a^{(0)}$ 由同一成员经固定传播配置得到，是即时释放假设下的条件到达，t_a 是实际到达。真实释放时刻 t_r、形成时刻 t_{trig} 和实际到达 t_a 缺少独立观测，本文暂不给出估计。

6. 过程引擎采用固定地形和集中有限历时注入；侵蚀补给、颗粒分选和堰塞湖溃坝未显式求解。D8 终止于分析域时记为右删失；已有漫溢背景作为后续外加源风险，不计入当前释放边界。

7. 历史沟长和扇长按水平投影解释；DEM 语义不明时并列冰面与冰下高程假设，不用高分位消除语义歧义。

8. 有限库存对照中的沿程节点与库存均为合成机制情景，不代表已确认的物源点；其速度是团块沿固定路线的条件速度，不等于真实前锋速度或流深。

9. 口岸工作坐标为 DEM 量图近似位置（28.2826°N, 85.3826°E），与参考候选相差约 0.6 km；源区至注入断面水平距离 2041.449 m，公共注入至口岸代理中心线 20882.237 m。站点坐标为假定代表点，海拔取气压反演值 3978.85 m。上述位置均按 DEM 近似，不是实测安装或受灾断面数据。

这些假设由现有资料条件决定；若与观测冲突，本文保留原设定并报告偏差，不通过调整起点、剔除成员或放宽阈值追求表面一致。阈值用于比较，进一步率定仍需独立灾害目录和现场观测。

全文小时步上的 mm 是该小时水当量深，不额外再除一小时；流量转换统一使用 $1\,\mathrm{mm}\times1\,\mathrm{km}^2/1\,\mathrm{h} = 1/3.6\,\mathrm{m}^3/\mathrm{s}$。同一符号在日累计时附日下标，避免把全期累计能量与单日门槛相比较。

四、问题一：近地层结构、天气过程与统一关联排序

4.1 问题口径与分析范围

题面要求依据 MET、RADM、SOIL 与 FLUX 四类逐时观测，分析高寒山区近地层结构、天气过程和极端事件，并给出与本次泥石流关联最高的五个特征。本文将诊断与排序分开：前者描述边界层和能量过程的结构，后者在固定候选池和同季匹配背景下判断本次冲击前哪个环境量最异常。目前只有一次确证事件，且没有完整非灾目录，因此结果限于环境关联排序，不涉及灾害发生概率、总体因果效应或分类 AUC。正式前五指：在统一候选池中，按事件前环境窗相对同季历史窗的稳健异常、经相关簇去冗余后的五个代表量。

4.2 五层静力订正与稳定度诊断

梯度观测高度为 1、2、4、10、20 米。MET 表头只给一个 Pressure 字段，未给气压传感器参考高度；FLUX 的 PA 又缺少安装与校准信息。若把整层气压视为常数，层间

压降会进入位温、虚位温和整体理查森数，使稳定度混入压力假设。本文以 2 米为参考高度的显式工作假设，保持五层温湿与 MET 气压共同有效，按

$$\frac{\mathrm{d}\log p}{\mathrm{d}z} = -\frac{g}{R_d T_v}, \qquad T_v = (T_C + 273.15)(1 + 0.61q) \tag{2}$$

作逆虚温分段梯形积分，更新层压三次。每次用当前层压重算比湿、位温和虚位温，再以第三次层压统一重算全部诊断量；第二、三次更新差的中位约 9.1×10^{-12} hPa，最大约 6.5×10^{-11} hPa，说明迭代已收敛，但不构成传感器误差界。

五层共同有效 45577 小时，占全部 49545 小时的 91.99%；仅 3 个可用小时因五层要求被新增剔除。可计算整体 Ri_b 的共同样本为 44018 小时。位温、虚位温与整体理查森数取

$$\theta_z = T_{K,z}\left(\frac{1000}{p_z}\right)^{0.2857}, \quad \theta_{v,z} = \theta_z(1 + 0.61 q_z), \quad Ri_b = \frac{g}{\bar{\theta}_v}\frac{(\theta_{v,20} - \theta_{v,2})(20 - 2)}{(u_{20} - u_2)^2 + (v_{20} - v_2)^2}, \tag{3}$$

其中饱和水汽压采用 Bolton 式，温度入口为摄氏、位温入口为开尔文，两个入口不混换。切变平方小于 $10^{-6}\ \mathrm{m^2/s^2}$ 时屏蔽并报告比例，不强行赋有限值。

静力订正会实质改变稳定度分类。与“全层同一气压”的对照相比，44018 个共同小时中有 9016 个分类改变，占 20.48%；同压对照的 Ri_b 中位为 −0.0636，订正后为 −0.0294，$p05$–$p95$ 为 −4.343–19.647。订正后的五级频率依次为强不稳定 0.2441、不稳定 0.3856、中性 0.0531、稳定 0.0852、强稳定 0.2320。两组使用同一批小时，只改变层压处理，故差异不是由样本变化制造的。

逆温按实际温度梯度而非位温梯度定义，操作阈值为 0.05 K/(100 m)。总逆温频率为 0.4111，夜间 0.6007、白天 0.2079，呈现夜间辐射降温、白天地表受热增强混合的相对日变化。有效厚度以 2 米为底层，依次检查 2–4、2–10、2–20 米的首个 $Ri_b = 0.25$ 交点；已出现交点的中位为 4 米，交点层顶以 4 米最多（7698 小时），10 米 2392 小时、20 米 1437 小时；31116 小时在 20 米内仍未达到 0.25，按右删失记录；20 米是观测上限，不是真实边界层高度。

图中季节能量为代理量，事件全日曲线仅用于描述。这些频率受质量码、夜间弱湍流缺测和 20 米观测上限限制。图2的日变化是多年统计，不是单个事件的独立控制实验；夜间逆温常见，也不等于每次泥石流都由逆温触发。图3比较同一批小时上“整层同一气压”与 1、2、10、20 米参考高度订正的结果，分类改变集中在中性与稳定边界附近。参考高度是工作假设，差异不反映真实传感器高度。

4.3 辐射收支、天气过程与极端事件

用四分量辐射构造 $R_n = R_{sd} - R_{su} + R_{ld} - R_{lu}$，反照率只在短波足够大时使用。未调整的小时潜在融化能量为

$$e_m = 0.0036[R_n - G - (H + LE)]^+ \quad [\mathrm{MJ/m^2\,h^{-1}}], \qquad [x]^+ = \max(x, 0). \tag{4}$$

正部约定排除夜间净冷却被误当作负融水。取 2021–2025 年、$R_n - G > 20\ \mathrm{W/m^2}$ 且四项共同有效的 12125 小时，回归斜率 $\beta_{EC} = 0.3343$，而闭合比

$$C = \frac{\sum(H + LE)}{\sum(R_n - G)} = 0.4515 \tag{5}$$

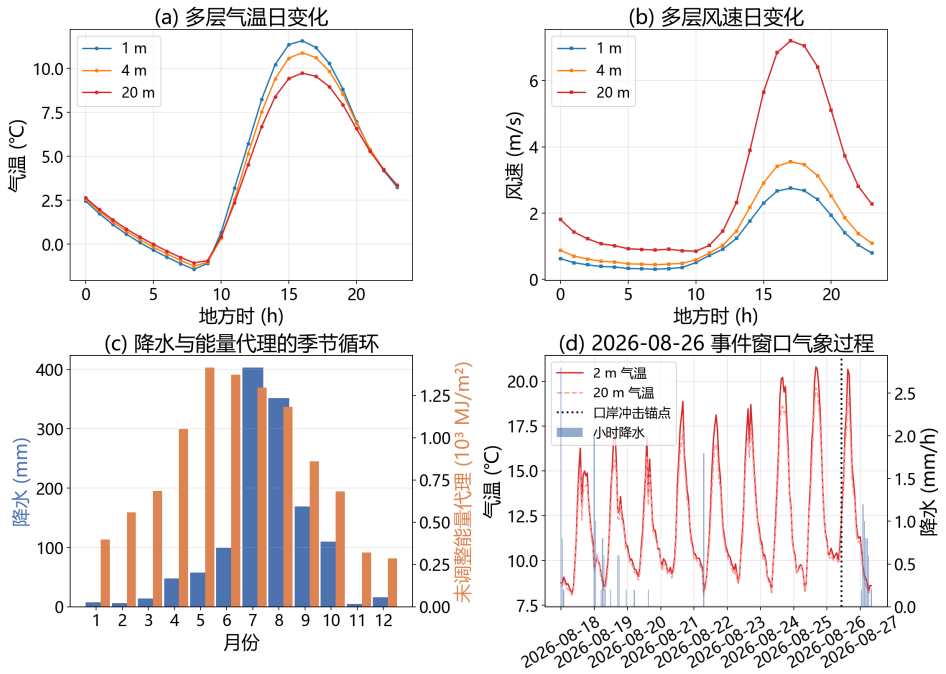


图 2　近地层日变化与事件窗口

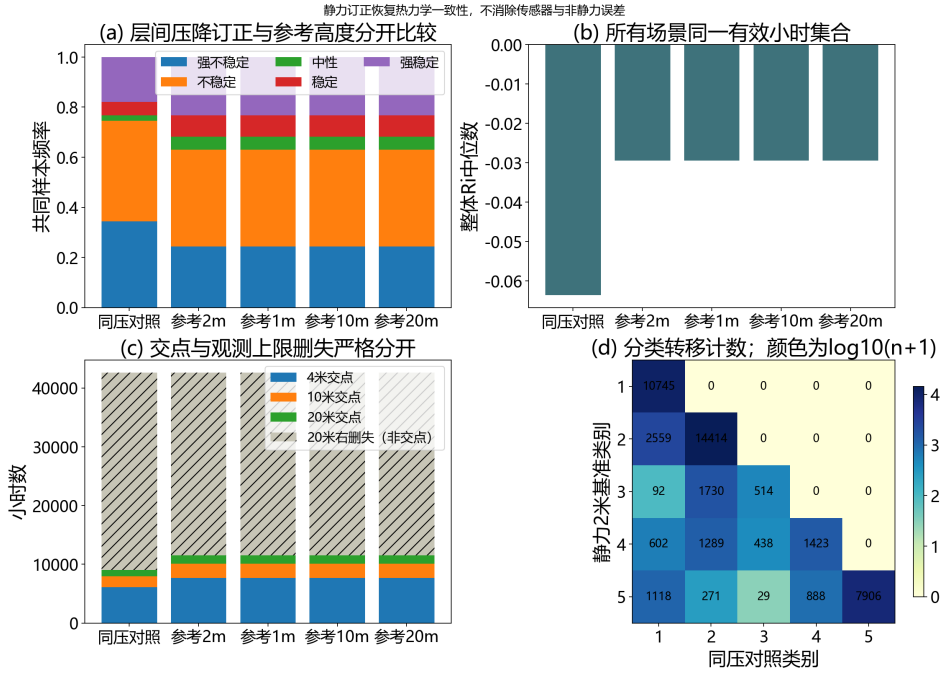


图 3　静力订正敏感性

该比值描述能量账本比例，其不确定度由块重采样估计，回归斜率标准误不适用于此。以完整训练年为块重采样 500 次，C 的 95% 区间为 $[0.4441, 0.4619]$，无效重采样比例为 0。事件相关三日未调整能量合计 32.19 MJ/m^2；点闭合比调整后为 9.35 MJ/m^2，两端比值情景约 8.85–10.06 MJ/m^2。变化远大于数值舍入误差，故基准保留未调整值，调整值只作结构敏感性，不把两者平均为“更真实的消融量”。

将五层气温、湿度、风速和气压组成 16 维向量，在 2021–2025 年标准化后作 SVD。前三主成分解释方差为 53.997%、30.716%、11.173%，达到 85% 需要 3 个模态。第一

主成分与融化能量相关 0.4304，与 24 小时降水相关 −0.0964，故不宜简单命名为降雨模态。另以训练期 90% 分位划分强降水、强消融、二者同超和平缓四类；34782 个有效小时的比例依次为 9.16%、8.82%、1.43%、80.58%，缺测小时保持未知，不归入平缓型。

极端量采用 12 小时游程去丛集后的广义帕累托分布。令 $Y = X - u > 0$，则

$$F_Y(y) = 1 - (1 + \xi y/\sigma)^{-1/\xi}, \qquad x_T = u + \frac{\sigma}{\xi}\left[(T\lambda_y)^{\xi} - 1\right], \tag{6}$$

其中 λ_y 为独立过程的年次数，暴露年数按有效小时折算；$\xi = 0$ 取连续极限，$\xi < 0$ 时返回水平还须低于有限上界 $u - \sigma/\xi$。小时降水 97% 分位落在 0，正雨小时全部进入尾部；短记录下 2 年、5 年水平只是拟合描述。复合极端主要集中在 6–9 月，但因缺少多事件起动目录，本文不估计因果风险倍数。

4.4 31 项统一候选关联模型

正式候选池在求值前固定为 31 个唯一变量键，家族构成为 MET 19 项、RADM 5 项、SOIL 3 项、FLUX 4 项。候选覆盖原始气象、温湿与风廓线派生量、辐射能量、土壤状态和湍流通量。小时降水 P 与 3 小时、24 小时累积量是不同时间尺度，不按别名删除；$P_{\max,1h}$ 与 P 的数值及缺测掩膜逐行相同，只保留 P。晴空指数和反照率仅白昼有效，固定后分别只有 84 与 82 个目标小时，低于 135 小时门槛；冻融量则因参照窗不足或稳健尺度为零，三项均保留在候选字典中并标为不可排名，其余 28 项进入正式评分。

题一关联截止为 2026-08-26 00:00（非题三 08:00 业务发布），最后可用标签为前一日 22:00。主窗为

$$\mathcal{W}_{168} = [\text{2026-08-18 23:00}, \text{2026-08-25 22:00}], \tag{7}$$

按小时标签 $[t, t+1)$ 计共 168 小时；72 小时敏感性窗取最后 72 个标签。目标窗和每个候选的参照窗均要求至少达到 80% 有效，即主窗不少于 135 小时，缺测保持为空，不插值或补零。主参照为 2021、2023、2024 年同积日前后 15 日内的 93 个发布窗；2022 年因 FLUX 摩擦速度和观测湍流动能缺测，没有窗口通过共同覆盖门槛，故该年进入逐时相关计算但不进入窗口参照。2025 年同历日窗作为无灾害真值的同类日期对照。

对候选 j 和窗长 L，令目标窗均值为 $m_{j,L}$，参照窗均值的中位数和稳健尺度分别为

$$b_{j,L} = \operatorname{median}_r\, m_{j,L}^{(r)}, \qquad s_{j,L} = 1.4826\, \operatorname{median}_r \left| m_{j,L}^{(r)} - b_{j,L} \right|, \tag{8}$$

并定义 $\Delta_{j,L} = m_{j,L} - b_{j,L}$、$Z_{j,L} = \Delta_{j,L}/s_{j,L}$、$A_{j,L} = |Z_{j,L}|$。MAD 为零时采用预先规定的 $IQR/1.349$ 回退；回退值仍为零则不排名。同时报告原始单位差、有符号方向和经验尾秩

$$p_{j,L}^{\text{tail}} = \frac{1 + \sum_r 1[A_{j,L}^{(r)} \ge A_{j,L}]}{N_{ref} + 1}, \tag{9}$$

该尾秩只是已选参照中的描述量，不是独立检验 p 值；主排序只按 A 降序。

去冗余在 2021–2024 年 6–9 月逐对计算 Spearman 相关。一对候选至少共享 200 个有效小时才可判定边，$|\rho_s| > 0.8$ 时连边，再取完整连通簇；全部 465 对均达到共同有效门槛，共同有效小时数的中位为 11208 小时。31 项候选合并为 16 个连通簇，其中 13 个

含可排名代表。每簇在 168 小时主评分中只保留 A 最大的可排名候选进入展示，其他成员和完整统计量继续保存。相关簇只解决展示重复，强相关只表示统计关联，不表示机制相同。

4.5 正式前五与解释

去冗余后的正式前五见表4，所有方向均为事件窗偏高。前五直接构成题面所问的统一关联排序；代理标签榜和 CRITIC 榜不再占用“本题答案”。

表 4　事件前 168 小时统一关联榜前五

次序	变量	家族	Δ（原单位）	A	经验尾秩
1	API（前期降水指数）	MET	14.484464	3.829043	0.0106
2	R_n（净辐射）	RADM	19.928307	2.343641	0.0745
3	24 小时升温	MET	0.312310	2.234448	0.0957
4	VWC（0.1 米土壤含水率）	SOIL	0.035295	1.487331	0.1809
5	LE（潜热通量）	FLUX	12.883223	1.133226	0.1724

表中 A 的前三项未舍入值为 3.8290425800714973、2.3436406435384014 和 2.2344476050143953；后两项为 1.4873309113872661 和 1.1332257049823602。

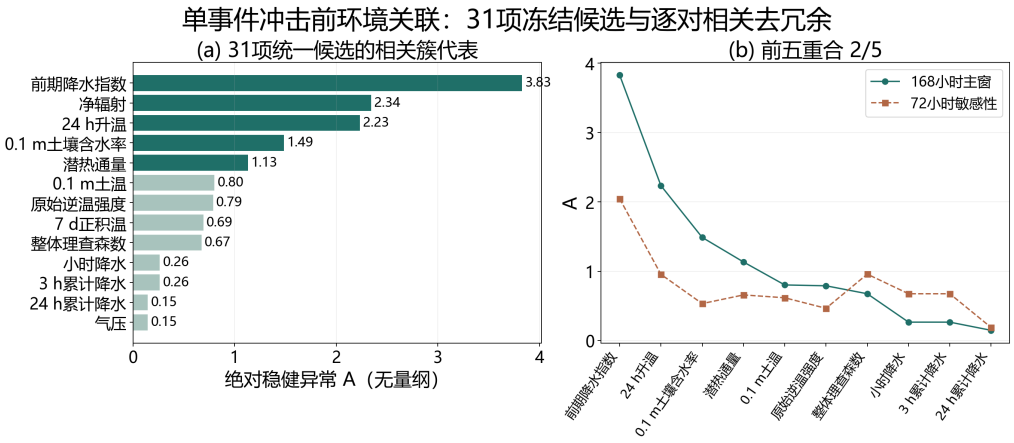


图 4　31 项候选关联排序与去冗余前五

图4同时给出完整排序、相关簇和头部去冗余结果；颜色只区分观测家族，尾秩是参照窗内的描述量，不表示机制独立性。

API 的事件窗均值为 23.492 mm，参照中位为 9.008 mm，绝对差 14.484 mm，是唯一 $A > 3$ 的候选，说明本次过程前存在较持续的降水记忆。R_n 的事件窗均值为 158.479 W/m^2，参照中位为 138.551 W/m^2；其所在相关簇同时含 2 米温湿、风切变、短波、消融能量及部分湍流通量，应把它理解为近地层能量状态的代表，而不是独立机制。24 小时升温均值为 0.294 °C，参照中位为 −0.019 °C，反映事件窗内快速增温。0.1 米土壤含水率均值 0.1271，高于参照中位 0.0918，表示浅层土壤水分偏湿；LE 均值 54.917 W/m^2，高于参照中位 42.033 W/m^2。后四项的 A 均小于 1.5，异常幅度明显小于 API。

窗长敏感性保留了“降水记忆和升温”这一信号：72 小时去冗余前五为小时消融有效能量、API、Ri_b、24 小时升温和降水，与 168 小时前五重合 API 和 24 小时升温两项。

留一年检查亦不完全稳定：省略 2021、2023、2024 年时前五重合数分别为 4、3、3 项；省略不同年可能使 API 与 7 日前期降水互换代表，但相关簇成员关系仍在。2025 年同历日对照的最大 $A = 3.336817142622748$，低于本次正式头项 3.8290425800714973；这表明本次头项不低于该同类日期，但单个 2025 日期不足以构成确证非灾样本。

4.6 构造性对照与适用限制

旧的 26 项代理榜以水源代理 Q_w 的 90% 分位为标签，前五为净辐射、饱和水汽压差、2 米气温、API 和 24 小时累积降水，2021–2024 训练、2025 留出的 AUC 为 0.9902。该高分首先说明水文代理可由其驱动项重建；净辐射、温度、降水和 API 又直接参与标签或供水构造，其排序不构成独立发现。与代理榜前五相比保留 4 项，全榜 $\tau = 0.7169$；CRITIC 三列反映事件异常、强水源日判别与触发耦合之间的冲突，三列共享观测，只用于方法比较，不构成独立数据验证。因此两榜定位为构造性对照，不是本文正式前五。

正式榜的适用范围如下。第一，它是单事件环境关联；没有非灾目录，A 不用于转换发生概率、风险比或普适排序。第二，93 个参照窗来自三个有效年份，年际样本有限；尾秩受窗口重叠影响，不作为独立 p 值。第三，API、R_n、消融能量和 LE 与问题二水源模型共享观测或物理构成，榜上位置不表示独立预测能力。第四，站点代表局地近地层，不代表源区冰温和县域空间梯度。第五，相关簇去冗余只影响展示；被合并变量仍有物理意义，完整 31 项排序和相关边随成果保存。正式结论是：在固定候选池和同季匹配方案下，API 最异常，净辐射、24 小时升温、浅层土壤含水率和潜热通量随后；这五项用于解释本次事件前的环境组合，不足以单独证明因果或预测未来泥石流。

五、问题二：床面临界条件与多尺度触发评估

5.1 问题定义与适用边界

水源异常与床面能否起动不是同一命题。本文把连续水力型沟道与冰崩链分开：前者用无黏聚、无超孔压的饱和松散床层给出门槛；后者在现有资料下输出条件状态，不用一次事件的时间窗反向标定系数。附件 6 的 84 个对象均为已发生灾点，不含确证非灾沟道，因此本文比较已发生对象之间的门槛和规模，不估计发生概率、真实空间 AUC 或误报率。

床层竖直厚度记为 z_b，水深垂直床面，应力和门槛统一按单位坡面面积报告。取水密度 $\rho_w = 1000\ \mathrm{kg/m^3}$、固相密度 $\rho_s = 2650\ \mathrm{kg/m^3}$ 和床体固相体积分数 $C_v^{(b)}$，有

$$\rho_{\mathrm{sat}} = C_v^{(b)}\rho_s + (1 - C_v^{(b)})\rho_w, \qquad \Delta\rho = \rho_{\mathrm{sat}} - \rho_w = 1650 C_v^{(b)}. \tag{10}$$

若 $C_v^{(b)} \notin (0, 1]$、$\Delta\rho \le 0$、$z_b \le 0$ 或比降无效，力学分支记 not_evaluable，不用零密度差继续产生正门槛。

5.2 完整受力、临界链条与三个分支

令 $S = \tan\theta$。水流在床面产生的拖曳应力、床体沿坡自重和有效法向摩擦抗力分别为

$$\tau_w = \rho_w g h \sin\theta, \qquad \tau_{g,\parallel} = \Delta\rho\, g z_b \sin\theta\cos\theta, \qquad \tau_r = \Delta\rho\, g z_b \cos^2\theta \tan\varphi. \tag{11}$$

最低无黏聚、无超孔压模型的起动条件为

$$\tau_w + \tau_{g,\parallel} \geq \tau_r . \tag{12}$$

控制水流状态时，水流所需承担的净抗力为

$$\tau_{wc} = \Delta\rho\, g z_b \cos^2\theta\,(\tan\varphi - \tan\theta)\,. \tag{13}$$

取无量纲容差 $\varepsilon_S = 10^{-6}$，按 S 与 $\tan\varphi$ 的关系划分：

$$\begin{cases} \text{水流控制}, & S < \tan\varphi - \varepsilon_S, \\ \text{摩擦平衡}, & |S - \tan\varphi| \leq \varepsilon_S, \\ \text{重力失稳}, & S > \tan\varphi + \varepsilon_S. \end{cases} \tag{14}$$

因此 $S > \tan\varphi$ 为重力失稳，零水流门槛不等于安全；$S = \tan\varphi$ 为摩擦平衡，表示净水流抗力为零，同样不属于“稳定”或“已触发”状态。水流控制时，临界水深、单宽临界流量和断面临界流量依次为

$$h_c = \frac{\tau_{wc}}{\rho_w g \sin\theta}, \qquad q_c = \frac{h_c^{5/3} S^{1/2}}{n_m}, \qquad Q_c = q_c \frac{B}{\kappa_B}, \tag{15}$$

其中 n_m 为 Manning 糙率，B 为全沟宽，κ_B 为全沟宽与有效过流宽度之比。零门槛分支令 $h_c = q_c = Q_c = 0$，并保留其机械状态；只有水流控制且 $Q_c > 0$ 时才定义有限触发比 $T = Q_w/Q_c$，避免零除和无意义的无穷大。

基准参数取 $C_v^{(b)} = 1050/1650 = 0.636364$、$z_b = 1.0\,\mathrm{m}$、$\tan\varphi = 0.55$、$n_m = 0.06\,\mathrm{s/m^{1/3}}$、$\kappa_B = 6$。中心线沟宽按 $B = \min\{9\sqrt{A_{km^2}}, 400\}$ m 取为 256.881 m。正式中心线比降为 0.10185，代入上式得 $h_c \approx 4.596\,\mathrm{m}$、$q_c = 67.587745\,\mathrm{m^2/s}$、$Q_c \approx 2.893667\times10^3\,\mathrm{m^3/s}$。这些是带明示工程参数的模型门槛，不是现场设防标准。由式15可见，q_c 随 z_b 和 $\Delta\rho$ 按约 5/3 次幂放大，随 n_m 近似反比变化；一次事件不能分别识别床厚、摩擦角、宽度和糙率。

对 84 条历史沟道，正式状态为 78 条水流控制、6 条重力失稳，无摩擦平衡和不可评估对象。78 条正门槛沟道的总起动流量中位为 63.261 $\mathrm{m^3/s}$，四分位区间为 12.614–228.441 $\mathrm{m^3/s}$。床体固相分数取 0.15、0.636364、0.80 时，正门槛中位分别约 5.690、63.261、92.635 $\mathrm{m^3/s}$，说明床体密度假设本身可改变门槛量级。图5给出完整受力分支、正门槛分布和参数敏感性。

5.3 对象级状态元组与冰崩链状态识别

84 个对象以源数据唯一标识 object_id 存储，不使用可重复的显示名称作为主键。每个对象和时刻保存六元组

$$\mathcal{R}_j = (G_j, H_j, \Theta_j, F_{\mathrm{ice},j}, U_{\mathrm{ice},j}, C_{\mathrm{ice},j}), \tag{16}$$

其中 G 为水流控制、摩擦平衡、重力失稳或不可评估；H 为水力候选、低于门槛、不适用或不可评估；Θ 为热力前兆成立、低于热力参照或不可评估。F_{ice} 表示附件 4 中存

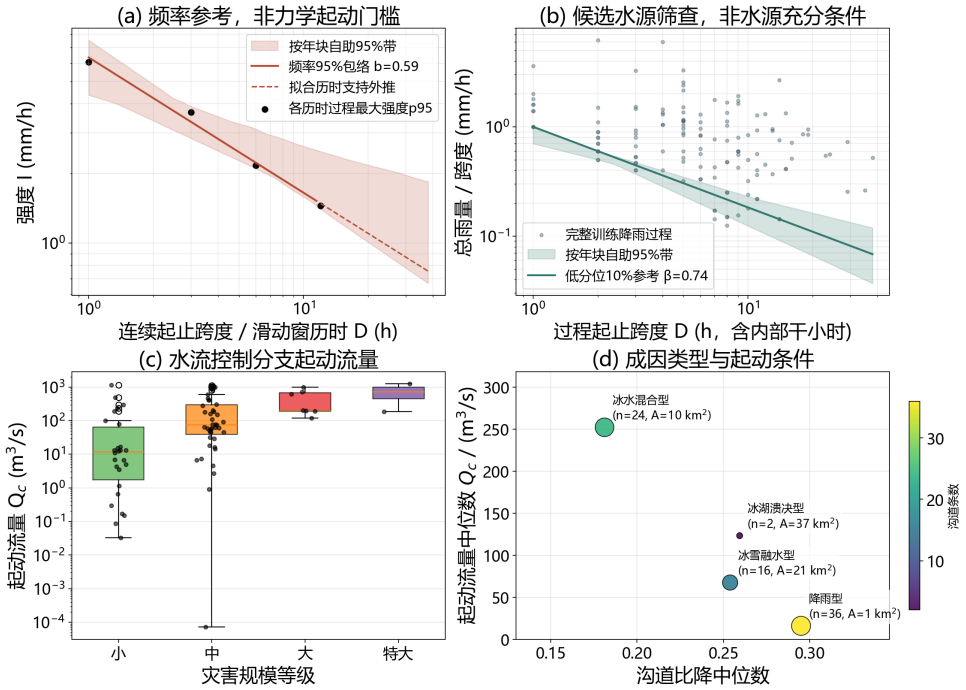


图 5　临界条件与参数敏感性

在冰厚与坡度同时达到源区阈值的编目像元，本数据中阈值为 50 m 与 25°，共 335 个像元；U_{ice} 记录冰体强度、裂隙、位移或监测资料缺失；C_{ice} 只在热力前兆成立且 $F_{\mathrm{ice}} = 1$ 时为 1。

事件时刻将同一口岸集水区水源流量施加于 84 个历史沟道几何作比较，H 的计数为 44 条低于门槛、34 条水力候选、6 条不适用。该统计是同一流量口径下的条件对照，不是 84 条沟道在现场同时获得相同水源的观测事实。事件日 Θ 为低于热力参照，因而对象层 $C_{\mathrm{ice}} = 0$；但 $U_{\mathrm{ice}} = 1$ 表明冰体力学状态仍不可识别，该保留项始终存在。热力未达参考、有限水力比未越 1，均不能证明冰体没有失稳，也不能把状态元组压成未经校准的 0–1 概率。

5.4 水源流量与事件前处理

SCS-CN 是单场降雨的累计损失模型，初损按完整事件累计，不逐小时重置[4]。对完整小时轴，连续六个已观测、严格零降雨小时确认事件边界；短于六小时的干隙保持同一事件，微量正降雨仍属于降雨，不套用 I–D 筛查的湿小时阈值。设事件起始小时为 s_e，事件累计雨量为 $P_e(t) = \sum_{u=s_e}^{t} P(u)$，累计直接径流曲线与逐时增量为

$$F_e(P) = \begin{cases} \dfrac{(P - I_{a,e})^2}{P - I_{a,e} + S_e}, & P > I_{a,e}, \\ 0, & P \le I_{a,e}, \end{cases} \qquad R_{\mathrm{rain}}(t) = F_e(P_e(t)) - F_e(P_e(t-1)), \tag{17}$$

其中 $P_e(s_e - 1) = 0$，$S_e = 25400/C_{N,e} - 254$，$I_{a,e} = 0.2S_e$。单场内初损只扣一次；事件起始前的最后一小时记为 $s_e - 1$。基准曲线数 $C_{N,0} = 85$ 是独立设定的工程情景，不由土地覆盖类码推导。API 分位只用 2021–2025 年 6–9 月的 14640 个有效小时计算，固定为 $q_1 = 3.574888$、$q_3 = 10.194567$；不使用全季节回退，也不用 2026 年重估。场初曲线数

由事件起始前一小时的 API 确定：

$$C_{N,e}=\operatorname{clip}\begin{cases}0.75C_{N,0}, & \mathrm{API}(s_e-1)<q_1,\\ C_{N,0}, & q_1\le \mathrm{API}(s_e-1)\le q_3,\\ 1.15C_{N,0}, & \mathrm{API}(s_e-1)>q_3,\end{cases}\quad [30,98]. \tag{18}$$

事件内不再随 API 改变而重新套用累计公式作差。初始边界、时间缺口或缺测降雨使累计状态未知，须重新观测到六个连续严格零雨小时才恢复已知边界；负雨量和非有限值视为无效，不将负值裁零来制造干期。若场初前一小时 API 不可用，整个事件的正雨产流记不可评估；已知场初且记录末端未闭合的事件，其已观测前缀仍可计算。汇流继续按原核传播缺测，缺测保持为空。该分级与六小时边界近似前期含水量和事件分隔，缺少本区水文率定；三、六、十二小时边界的函数行为已由单元测试核对。

源区温度由站点温度按 0.006 K/m 直减率外推至有效冰厚像元平均高程 5109.653 m，站点高程取 3980 m，温差订正为 −6.7779 °C。融水权重取 Logistic 温度因子 $w_T(t)$[5]，RGI6 编目边界与集水区面积交比 $f_{gl}=0.23602$。于是

$$m_{\mathrm{src}}(t)=\frac{w_T(t)e_m(t)}{0.334},\qquad m_{\mathrm{cat}}(t)=f_{gl}m_{\mathrm{src}}(t), \tag{19}$$

其中 e_m 以 MJ/(m²·h) 计，0.334 MJ/m² 对应 1 mm 水当量。雨与融水分别作三小时汇流后，

$$Q_w(t)=\frac{A_{km^2}}{3.6}\left[\sum_{i=0}^{2}w_iR_{\mathrm{rain}}(t-i)+\frac{1}{3}\sum_{i=0}^{2}m_{\mathrm{cat}}(t-i)\right],\quad (w_0,w_1,w_2)=(0.6,0.3,0.1). \tag{20}$$

f_{gl} 是编目年代面积权重，不是 2026 年实时冰雪范围；任一必需小时缺测时，Q_w 保持缺测。

5.5 事件窗求解与水力状态

正式事件窗为 2026 年 8 月 26 日 08:00–13:00 的六个整点标签。水源窗峰值为 202.650179 m³/s，出现在 13:00；相对正式中心线门槛，触发比在该时为

$$T_{\mathrm{peak}}=\frac{202.650179}{2.893667\times10^{3}}=0.07003232034381977, \tag{21}$$

10:00 的 $T=0.018814$，窗口内 $T\ge1$ 的小时数为 0，未出现越阈时刻。该结果说明当前无黏聚、无超孔压水控链条没有把此次冰崩链识别为水力越阈事件；它不说明冰体未失稳，反而要求保留 $U_{\mathrm{ice}}=1$ 的冰力学未识别状态。旧事件窗标定系数 $k=11.19431$ 仅作对照，正式模型不使用事件窗 k，也不将窗口水源峰值归一为 1。

图6把事件窗与更长时间序列放在同一条件门槛下展示；图中 T 是条件强度比，不是发生概率，事件窗内未出现 $T\ge1$。未越阈是当前模型的结果，不是对冰体失稳或链式过程发生的否定。

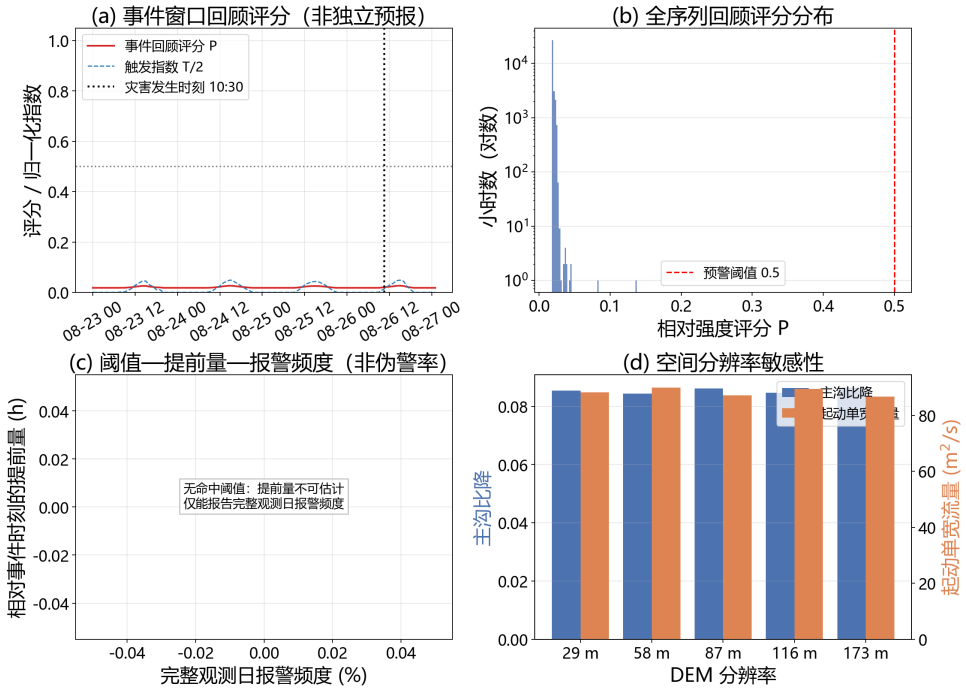


图 6　水源流量与触发比

5.6 历史规模关联与双语义降雨包络

在 78 条水流控制正门槛对象上，分别检验总起动流量 $Q_{\mathrm{crit}} = q_{\mathrm{crit}} B/\kappa_B$ 和单宽起动流量 q_{crit} 与已发生对象规模排序的关系，二者名称与量纲需分开。总流量对规模的 Kendall 检验为 $\tau = 0.408386$、$p = 5.471436410144295 \times 10^{-6}$，Spearman 检验为 $\rho = 0.498676$、$p = 3.3745699520687622 \times 10^{-6}$；单宽流量对规模的 Kendall 检验为 $\tau = 0.318602$、$p = 3.904720246143714 \times 10^{-4}$，Spearman 检验为 $\rho = 0.400620$、$p = 2.784782801479891 \times 10^{-4}$。四组 p 值均按双精度和科学计数法保留，不静默舍入为零。正相关表明在本已发生对象中，较大规模沟道并不具有更低的流量门槛；它不支持发生概率方向，也不能外推到未发生沟道。

降雨阈值保持两种语义[6]。138 场完整降雨过程给出的 95% 频率包络为 $I_{95}(D) = 6.3507 D^{-0.5853}$，事件平均强度的 10% 分位参考为 $I_{10}(D) = 1.0000 D^{-0.7374}$，$D$ 以小时计、I 以 mm/h 计。前者描述历史稀有度，后者只是未作灾害标注的低分位候选筛查线；二者含义不同，不取平均；“超过包络”只表示历史稀有，不表示真实起动。完整样本支持到 12 小时，更长时间的幂律仅作外推提示。

5.7 时间与空间可靠性

时间聚合按同一有效窗比较。3、6 小时和日尺度的有效窗口数分别为 10583、4980、1190，平均/峰值保持率为 0.722655、0.537989、0.230808；三种聚合下均无 $T \geq 1$ 窗口。2021–2026 年最大 T 分别为 0.136392、0.195132、0.104129、0.540807、0.397600、0.235953，均明显低于 1。全序列触发评分 P 的最大值为 0.137433，出现在 2024-09-28 18:00，也远低于 0.5 的报文门槛。平均会削弱短脉冲，但本数据中即使取窗内峰值也未越阈；该负值反映当前参数与分辨率下的模型行为，不表示现场未发生链式过程。

继承像元对照在 30、90、270 米名义尺度上的 LR AUC 分别为 0.9814、0.9269、0.8790，

组块自举区间分别约为 [0.9757, 0.9872]、[0.8984, 0.9571]、[0.8355, 0.9195]。这些数值主要反映灾点面内正像元与面外 500 米背景的采样环境差异；背景没有确证非灾真值，不代表发生概率或危险区划。共同候选域统一 DEM、覆被、最小地面面积和 500 米隔离后，三尺度均只形成 2 个正组和 188 个背景组，无法形成至少两折同时含两类的训练/测试划分，因此不报告 AUC。图7展示共同域、物理块和空间隔离的诊断结果。

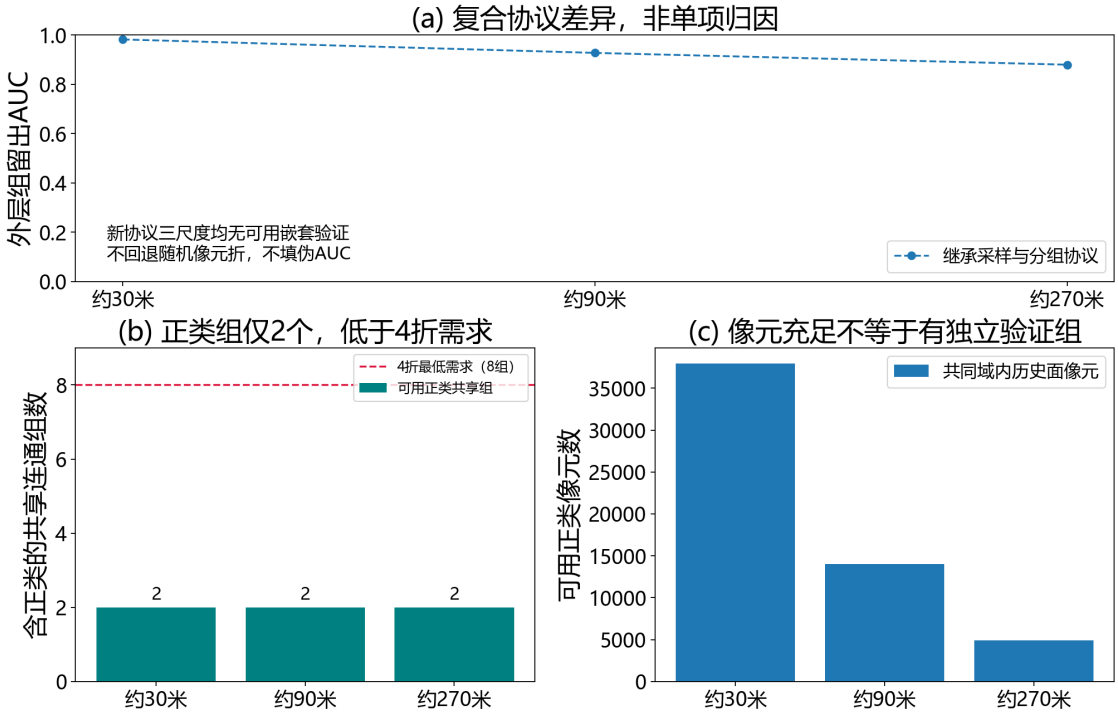


图 7　共同域空间隔离诊断

分辨率和路径口径也会改变门槛。用于 D8 追迹敏感性检验的主沟比降在 2–6 倍粗化下为 0.084354–0.086502，单宽临界流量约 86.699–89.999 m^2/s，集水区面积相对偏差绝对值不超过 0.4814%，但口岸投影偏移最大约 83.4 m。上述范围与正式中心线门槛不混作同一口径：前者检验路由和粗化稳定性，后者使用公共注入至口岸中心线。中心线平均比降不覆盖每个局部起动段；跨尺度移植时宽度、汇流和床面材料可能同时变化。

5.8 综合判读

正式链条是“床体沿坡自重＋水流拖曳与大有效法向摩擦”的条件力平衡，再经 h_c、q_c、Manning 和 Q_c 转换到断面尺度。84 条历史沟道中 78 条仍需正水流门槛、6 条在给定模型参数下重力失稳。事件窗中心线水源峰值 202.650179 m^3/s 对应 T_{max} = 0.070032，10:00 仅 0.018814，当前水控模型未越阈；冰体源区虽有 335 个达到 50 m/25° 阈值的像元，但强度、裂隙和位移资料缺失，冰力学状态仍未识别。空间上仅能确认继承采样环境的可分性，严格共同域因正组不足而不可估计；时间上仅能确认水控门槛未越，而水控未越不等于冰崩链未发生。模型适用于有有效几何且床面假设落在给定范围内的条件评估；真实发生概率、无资料沟道安全认证和未经校准的危险区划不在本文范围。

六、问题三：固定发布时刻的条件预报与真实触发识别

6.1 任务目标与可识别范围

题目要求给出触发时间的 95% 置信区间、预警时间、区间覆盖率和空间距离。现有资料只有一个口岸冲击时刻，没有源区冰体失稳、物料释放或泥石流形成的独立时间观测；附件 4 的表面速度是 2014–2024 气候态栅格，没有 2026 年灾前时间维。因而现有资料不足以识别真实释放时刻 t_r、形成时刻 t_f、实际到达 t_a 及其预警和覆盖率。

本文把时间对象分开。t_m 为源区融水条件首次满足的代理时刻，$t_a^{(0)}$ 为“融水出现即释放既定物料”情景下的口岸条件到达，t_a 为实际到达。三者满足

$$t_r \neq t_m, \qquad t_f = t_r + \Delta_{r\to f}, \qquad t_a = t_r + \Delta_{r\to \mathrm{inj}} + \Delta_{\mathrm{inj}\to a}, \tag{22}$$

其中 $\Delta_{r\to f}$ 为释放到流态转变的延迟，$\Delta_{r\to \mathrm{inj}}$ 为释放到公共注入断面的传播时间，$\Delta_{\mathrm{inj}\to a}$ 为注入到口岸的运行时间。没有释放观测时，本文不把 t_m 当作 t_r，也不把 $t_a^{(0)}$ 写成实际到达。表5列出各目标的处理。

表 5 第三问目标与可识别性

目标	可识别性	本文输出
t_m：源区融水起始代理	辅助气象目标	条件 95% 区间
t_r：冰岩物料实际释放	不可识别	空值
t_f：泥石流形成	不可识别	空值
t_a：口岸实际到达	不可识别	空值
$t_a^{(0)}$：即时释放条件到达	辅助情景	条件 95% 区间
10:30 口岸冲击	题面单一锚点	事后单点包含

因此，本文给出的产品是 t_m 与 $t_a^{(0)}$ 的条件区间、状态计数和空间距离。真实 t_r、t_f、t_a、真实预警和覆盖率尚不能给出；当前结果属于辅助条件产品，不作为真实触发预报。

主发布时刻固定为 2026 年 8 月 26 日 08:00 当地时（00 UTC），预报窗口为

$$\mathcal{H} = [\text{2026-08-26 08:00}, \text{2026-08-27 08:00}). \tag{23}$$

主起报为 8 月 25 日 18 UTC，另取同日 06、12 UTC 两次单次起报构造模式演化差异。三者按六小时分发上界分别于 25 日 12 UTC、18 UTC 和 26 日 00 UTC 可用，均不晚于发布时刻；每次起报生成 1000 个成员，合计 $N = 3000$。题面 10:30 只用于事后单点包含，不进入成员生成、系数拟合或配置选择。

6.2 变量时间对齐与历史能量关系

普通小时短波是前一小时均值，降水是前一小时累计，温湿度是对应时点的瞬时值。若按原始标签把三者放在同一行，会把前一小时间接辐射归入后一小时支持。本文按变量真实支持对齐到统一半开区间 $[t, t+1)$：

$$SW_t = SW_{t+1}^{\mathrm{raw}}, \qquad P_t = P_{t+1}^{\mathrm{raw}}, \qquad \bar{T}_t = \frac{T_t^{\mathrm{raw}} + T_{t+1}^{\mathrm{raw}}}{2}, \qquad \overline{RH}_t = \frac{RH_t^{\mathrm{raw}} + RH_{t+1}^{\mathrm{raw}}}{2}. \tag{24}$$

前两者是过去小时均值与累计量，后两者是时点值的梯形均值近似。先取小时均值，再施加正部与门控，不交换非线性操作；任一端点缺失则该小时不可评估，不用下一条记录跨过时间缺口。发布前最后完成支持 [07:00, 08:00) 取原始标签 08:00 的短波与降水，以及两个温湿端点；完整轨迹的末支持起点为原始末时点减一小时。若按起始标签对齐，会把前一小时间接辐射归入后一小时支持，故本文不采用该口径。

历史站点四类小时标签缺少起止或瞬时元数据。主假设为 MET 时点端点梯形均值、RADM/SOIL/FLUX 起点标签小时均值；对 MET 三种处理与其余三族起点/终点组合共 24 种情形重建净辐射与消融能量目标、重拟合回归和 AR(1)，各情形使用同一流程，未按事件结果挑选。24 种情形给出相同的成员首达支持集合，说明辅助区间对时间标签假设不敏感；真实释放仍需独立观测。

在 2021–2025 年 6–9 月共同有效小时上拟合

$$\hat{e}_m = \beta_0 + \beta_T T^+ + \beta_R R^+ + \beta_H RH + \varepsilon, \qquad T^+ = \max(T, 0), \quad R^+ = \max(SW, 0), \tag{25}$$

其中 $\hat{e}_m$ 与 R^+ 的单位为 W/m^2，T 为 °C，RH 为百分数。训练集共 11986 行，估计得

$$\beta = (-119.0252,\ -5.8526,\ 0.4706,\ 1.7154), \tag{26}$$

残差一阶自回归参数 $\phi = 0.400328$、创新标准差 $\sigma_\varepsilon = 41.015695$ W/m^2，边际标准差 44.715577 W/m^2，连续小时对 11772 个。预报时仅当 $T > 0$、$SW > 20$ W/m^2 且 $\hat{e}_m > 0$ 时判定潜在融水；任一必需输入缺失记不可评估，不以零值填补。小时水当量按 $m_s = \hat{e}_m / 334000 \times 3600$ 换算。

发布前最后完成支持 [07:00, 08:00) 没有成员满足活动判据。发布后 24 小时窗内，3000 个源区成员和 3000 个传播成员均判定为有效，有效比例 $p_{\mathrm{fin}} = 1$；无事件、发布时已活跃、右删失、不可评估、未到达及口岸左删失的成员计数均为 0，状态合计一致。可行动质量

$$p_{\mathrm{actionable}} = p_{\mathrm{act}} + p_{\mathrm{fin}} = 0 + 1 = 1 \tag{27}$$

只表示给定模型集合中需要发出源区融水提示的成员比例，不是物理发生概率。

6.3 条件区间与支持包络

每个成员的小时支持原样保留为 $[L_i, U_i)$，宽度为 1 h。为给出事件时刻分位，在固定随机流中为每个有限成员在支持内均匀抽取代表时刻

$$\tilde{t}_i = L_i + \tau_i (U_i - L_i), \qquad \tau_i \sim U[0, 1), \tag{28}$$

再取 2.5% 与 97.5% 分位。t_m 与 $t_a^{(0)}$ 由同一成员、同一传播配置配对生成：

$$\tilde{t}_{a,i}^{(0)} = \tilde{t}_{m,i} + \Delta t_{\mathrm{fall},i} + \Delta t_{\mathrm{run}}^{(c_i)}, \tag{29}$$

其中 Δt_{fall} 为源区质心至公共注入断面的无摩擦首波近似，两种高程语义下为 29.50–29.67 s；$\Delta t_{\mathrm{run}}^{(c_i)}$ 取自 16 个去重 D8 配置之一。配置等权抽样只表示模型集合设计，不是现实配置概率。

表 6　条件事件时刻区间

对象	2.5% 分位	中位	97.5% 分位	宽度/h
源区 t_m	09:03:22.847	10:13:44.990	10:57:51.047	1.907833
即时释放 $t_a^{(0)}$	09:35:26.068	10:46:31.992	11:31:59.410	1.942595

条件区间见表6，均为 2026 年 8 月 26 日 UTC+8 时刻，保留到毫秒。它们是有限成员分布内的名义 95% 分位，不是经真实多事件频率校准的信赖区间。

完整支持包络与支持端点分位不是同一对象。前者为有限成员全部半开支持的凸包

$$E_{\text{support}} = [\min_i L_i, \max_i U_i), \tag{30}$$

可能含空档，不是成员支持的精确保集；后者为

$$E_{0.95,\text{endpoint}} = [q_{0.025}(L_i), q_{0.975}(U_i)], \tag{31}$$

只是端点边际分位的收缩诊断。两者均不替代表6的代表时刻事件区间，具体数值见表7。

表 7　支持包络

对象	包络类型	起点	终点	宽度/h
t_m	完整支持	[09:00:00.000	11:00:00.000)	2.000000
t_m	端点 2.5%/97.5%	[09:00:00.000	11:00:00.000]	2.000000
$t_a^{(0)}$	完整支持	[09:18:03.728	11:45:13.334)	2.452668
$t_a^{(0)}$	端点 2.5%/97.5%	[09:26:42.679	11:45:13.170]	2.308470

三类时间对象的分工见图8；图中只比较代表事件分位、完整支持包络和支持端点包络，不把不同定义混成一个“95% 触发区间”。

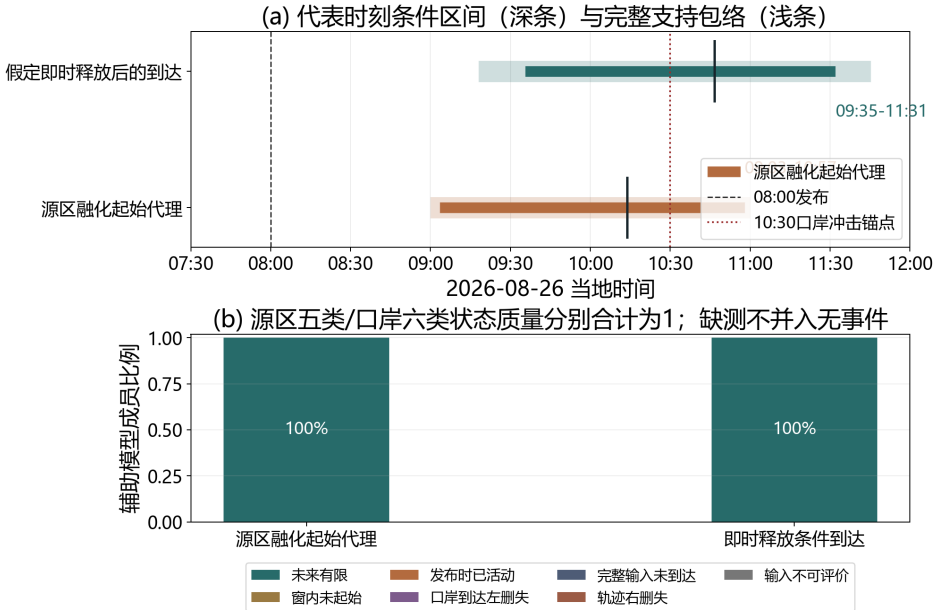


图 8　三类时间对象

6.4 单事件包含、提示时差与真实预警

真实泥石流形成没有独立真值，真实触发覆盖率不可估计，本文不给出数值。题面只给出一个口岸冲击点 y_h = 10:30。在即时释放辅助情景下，条件区间 09:35:26.068 至 11:31:59.410 包含该点，报告为 1/1。该 1/1 表明辅助情景与题面锚点不矛盾，不代表实际到达覆盖率。实际到达覆盖记为不可估计（释放模型未识别），区间、确定覆盖和可能覆盖尚未给出；长期频率需独立资料。

融水提示规则在发布时刻确定：若 $p_{\mathrm{actionable}} \geq 0.5$，则在 08:00 发布源区融水提示。本次 $p_{\mathrm{actionable}} = 1$，故提示时刻为 08:00。相对 10:30 锚点的信息时差为 2.5 h，这是“融水提示”相对既有冲击锚点的时间差，不是真实灾害预警提前量。真实灾害预警的首次报警、可执行报警和提前量尚不能给出。

辅助到达区间相对 10:30 的提前量为

$$\Delta_{a,\mathrm{range}} = [10{:}30 - U_{t_a^{(0)}},\ 10{:}30 - L_{t_a^{(0)}}] = [-1.033169,\ +0.909426]\ \mathrm{h}, \tag{32}$$

区间跨过锚点，故 2.5 h 信息时差不构成命中提前量；源区代理中位距发布 2.229 h，也不是真实释放提前量。图9给出辅助单点包含和提示延迟对照；真实释放、形成、实际到达和真实灾害预警均标注为不可估计。

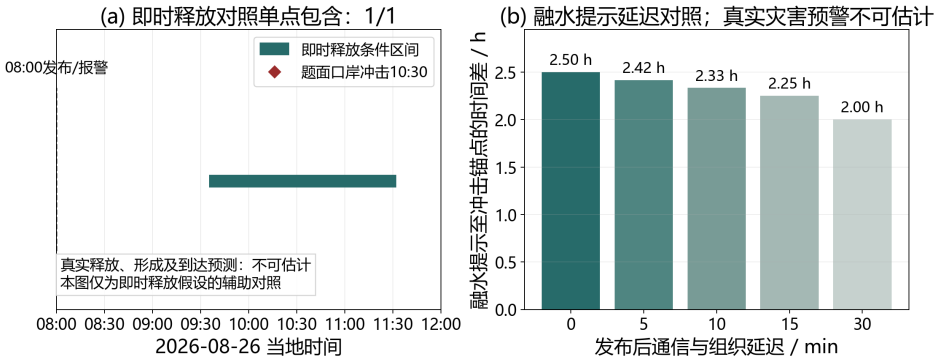


图 9 辅助单点包含与提示时差

6.5 空间距离与右删失运程

按同一源区、公共注入断面和口岸代理断面分别计算距离。源区至注入断面的水平距离为 2041.449 m；注入断面至口岸代理的中心线沿沟距离为 20882.237 m；组合沿沟路线为 22923.686 m。源区至口岸代理的平面直线距离为 14771.715 m。沿沟里程与平面直线距离不可相加成一个“距离范围”。

16 个去重 D8 配置的口岸首达时间范围为 1054.226–2683.668 s，即 17.57–44.73 min。全部配置的终态分析域里程均为 26045.494670 m，且右删失标志为真；该值是演算分析域末端的右删失前缘，不是自然最大运程，也不是实测危险区边界。空间对象与状态见图10。

6.6 真实释放的识别条件

式 (22) 式中的 t_r 不能由融水代理替代。可检验的观测约束路径是渐进破坏的逆速度模型。若取得发布前各时点 u_j 的独立位移 d_j、相关误差与可用时刻，则对加速段构

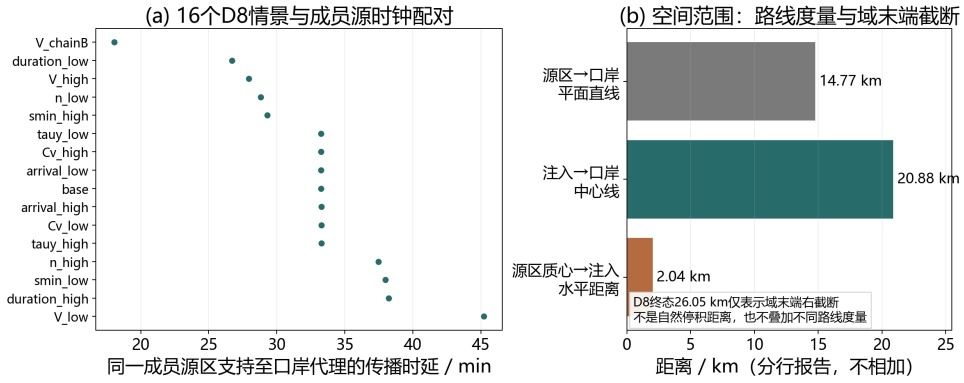


图 10 沿沟距离与 D8 到达

造

$$v_j = \frac{d_j - d_{j-1}}{u_j - u_{j-1}}, \qquad \dot{v} = Av^{\alpha}, \tag{33}$$

当 $\alpha = 2$ 时

$$v^{-1}(u) = a + bu, \qquad t_r = -a/b \quad (b < 0). \tag{34}$$

它把可观测加速连接到渐进破坏的预测外推，但仍需检验稳定运动基线、窗口敏感性、测量误差、速度求差相关性和外推失效。附件 4 的速度场没有 2026 年时间维，无法构造 $v_j(u_j)$；一张气候态速度图或空间四分位带也无法给出 a, b, α 或释放时间区间。用 10:30 口岸冲击反推 a, b 属于事后标定，本文不采用。

在取得合格序列前，释放状态记为“未知”，不记为“未释放”；真实 t_r、$\Delta_{r\to f}$、t_a 及其预警尚不能估计。所需资料包括：发布前源区位移、裂隙开度、地震或声学释放信号，可核查的释放时刻和泥石流形成时刻，以及测量误差与缺测记录。取得这些资料后，才能把真实触发区间、真实预警和独立覆盖率作为主产品重新计算；本文不将当前结果表述为真实触发预报。

6.7 一致性检查与适用限制

本节的一致性检查包括：成员总数为 3000，源区与口岸状态合计均为 3000，支持宽度均为 1 h，$p_{\text{actionable}}$ 与状态质量一致，16 个 D8 配置均提供有限首达，24 种站点小时支持组合给出相同辅助区间。这些检查说明成员生成、支持传播和状态记录一致；融水代理不等同于真实释放或泥石流形成，区间也不具备真实频率覆盖。

主要不确定性如下。第一，IFS 格点温度、短波和湿度是模式输出，跨高程与地表类型迁移可能产生系统偏差。第二，残差自回归、回归系数扰动和 D8 配置抽样组成模型集合不确定性，各分量未由多事件联合校准。第三，D8 配置等权不是物理概率，改变配置集合会改变条件分位。第四，附件 4 速度场是气候态，只提供长期背景，缺少灾前加速序列；释放机制仍不可识别。第五，题面仅有一个口岸冲击时刻，无法检验长期覆盖率或误报率。因此，本节给出固定时刻的条件辅助产品及其适用范围，不将单事件结果升级为真实灾害确定性预测。

七、问题四：双引擎情景演算、右删失处理与有限库存机制

7.1 任务目标与模型定位

问题四要求在给定灾前地形、源区体积与传播参数下，估计过程沿沟演进、口岸到达时间、运程和影响强度。现有资料缺少真实释放过程、沿程侵蚀物源、实测流深流速和现场流变，故本文建立参数透明的条件情景模型，而不把结果称为已率定的灾害重建。两个引擎承担不同任务：一维纵剖面用于检验准恒定断面下的自然停止与沿程尺度；D8 网络用于显式连续性推进，输出首达场、历史峰值、终态场和质量收支。D8 每格只向一个下游格路由，不含横向动量、漫滩扩散和完整浅水方程[7]。

两个引擎使用同一公共注入断面和同一口岸代理断面，并以注入断面统一计时。口岸代理是题面地点在 DEM 上的吸附断面，不是实测受灾断面；全域出口是另一对象，不能代替口岸断面。两引擎结果差异用于揭示结构不确定性，不是两次独立观测，也不因 D8 到达口岸而删除一维自然停止的结果。

7.2 公共几何、源区体积与注入边界

在附件 4 原生冰川网格上，以冰厚至少 50 m 且坡度至少 25° 筛选源区，共 335 个原生像元。先在原生网格积分，再映射到演算网格；335 次投影全部落域、去重为 323 格，体积不在映射阶段重复计数。链 A 的体积链为

$$V_{\mathrm{src}} = \sum_{k} h_{\mathrm{ice},k} a_k, \qquad V_{\mathrm{rel}} = \kappa_{\mathrm{rel}} V_{\mathrm{src}}, \tag{35}$$

$$V_{\mathrm{df}}^{(A)} = \gamma \frac{\rho_{\mathrm{ice}}}{\rho_{\mathrm{df}}} V_{\mathrm{rel}}, \qquad \rho_{\mathrm{df}} = C_v \rho_s + (1 - C_v)\rho_w. \tag{36}$$

取 $\kappa_{\mathrm{rel}} = 0.10$、$\gamma = 3$、$\rho_{\mathrm{ice}} = 900\ \mathrm{kg/m^3}$、$\rho_s = 2650\ \mathrm{kg/m^3}$、$\rho_w = 1000\ \mathrm{kg/m^3}$、$C_v = 0.45$，则 $\rho_{\mathrm{df}} = 1742.5\ \mathrm{kg/m^3}$。源冰总量为 $1.70109 \times 10^7\ \mathrm{m^3}$，释放冰量为 $1.70109 \times 10^6\ \mathrm{m^3}$，链 A 混合入沟体积为

$$V_{\mathrm{target}}^{(A)} = 2635835.294118\ \mathrm{m^3}. \tag{37}$$

链 B 以厚冰筛选区的实际平均厚度、面积和可动比例形成 $1.123497 \times 10^7\ \mathrm{m^3}$ 的大体量对照。它不经过与链 A 相同的密度与物源换算，只表示本情景族的高体积端点，不是现实严格上界。比较两链前需统一对象是源冰、释放冰还是混合入沟体积，避免把不同对象混为同一规模。

链 A 落域格中汇流累积最大者作为公共注入点，口岸代理取该下行路线距口岸工作坐标最近的有效格，吸附偏移 5.186582 m。中心线水平长度 20.882237 km、总落差 2126.790527 m，重采样为 209 个约 100 m 节点。原始纵剖面含 137 个上升步，可能来自真实局部地形、填洼或重采样；本文仅以单调不增回归构造演算剖面，最大调整量 39.517428 m。该处理便于 Manning 局部坡度计算，也会抹平部分蓄滞，不应视为仅修正地形。

7.3 一维诊断引擎与自然停止

以特征流量 $Q_{\mathrm{char}} = V_0/T_{\mathrm{pass}}$ 驱动准恒定 Manning 断面[8]：

$$B(s) = B_0(1 + 0.8s/L). \tag{38}$$

$$h(s) = \left[\frac{nQ_{\mathrm{char}}}{B(s)\sqrt{S_{\mathrm{eff}}(s)}}\right]^{3/5}, \qquad v(s) = \frac{Q_{\mathrm{char}}}{B(s)h(s)}. \tag{39}$$

基线取 $V_0 = 2635835.294118\ \mathrm{m}^3$、$B_0 = 256.880875\ \mathrm{m}$、$n = 0.10\ \mathrm{s/m}^{1/3}$、$T_{\mathrm{pass}} = 900\ \mathrm{s}$、$\mu_H = 0.145259$、$S_{\mathrm{eff}} \geq 0.01$。经验能量线为

$$E(s) = E_0 + 2g\left[(z_0 - z(s)) - \mu_H(s - s_0)\right], \qquad E_0 = 200\ \mathrm{m}^2/\mathrm{s}^2. \tag{40}$$

连续 5 个节点满足 $E \leq 0$ 时判自然停止。E 具有速度平方量纲，是准稳断面可达性的诊断量，不是完整瞬态能量守恒解；停止规则本身也属于结构假设。

基线在 9.336769 km 处自然停止，停止时刻 1783.858032 s（29.73 min），未达 20.882237 km 口岸，状态为自然能量停止。17 个去重一维配置中 16 个未达口岸，仅 $\mu_H = 0.1017$ 的条件配置于 83.3575 min 触及口岸末端；后者是口岸右截断下界，不是自然停止距离。该负结果原样保留，不通过删除未达配置或反向挑选摩擦系数制造统一到达。基线沿程剖面见图11；能量线在上游判停，图中线段不延伸为虚假的口岸到达时间。

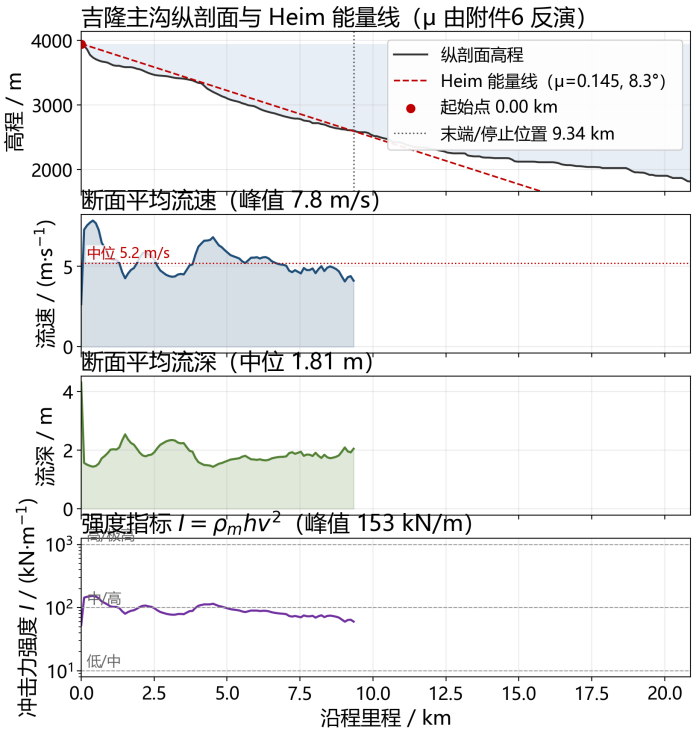


图 11 一维沿程演进

强度采用

$$I = \rho_{\mathrm{df}} h v^2 \quad [\mathrm{N/m}], \tag{41}$$

其中 $\rho_{\mathrm{df}} = 1742.5\ \mathrm{kg/m}^3$。该量表示单位宽度动量通量尺度，不是压强。基线沿程最大强度为 153.092248517 kN/m，只作情景分级，不作为本地已验证的建筑破坏阈值。基线

未到口岸，故不在口岸生成物理解释上的堆积扇体；任何几何扇形只能作为额外条件包络。

7.4 D8 屈服门槛—Manning 路由与临界量

在 Priority-Flood 填洼地形上，每格按米制最大下坡选择唯一接收者[9]。记格面积 a_i、等效过流宽度 W_i 和源项 q_i^{src}，连续方程与代数流量为

$$\frac{dh_i}{dt} = \frac{\sum_{j:r(j)=i} Q_j - Q_i}{a_i} + q_i^{\mathrm{src}}, \tag{42}$$

$$Q_i = \begin{cases} n^{-1} h_i^{5/3} \sqrt{S_{\mathrm{eff},i}}\, W_i, & \rho_{\mathrm{df}} g h_i S_{\mathrm{eff},i} \geq \tau_y, \\ 0, & \text{其他}, \end{cases} \qquad S_{\mathrm{eff},i} = \max(S_i, S_{\min}). \tag{43}$$

这是屈服开关加 Manning 运动波，不含独立库仑摩擦和宾汉塑性黏度，不是完整的库仑—宾汉流变。

由屈服条件可写

$$h_{y,i} S_{\mathrm{eff},i} = \frac{\tau_y}{\rho_{\mathrm{df}} g}. \tag{44}$$

右端在固定屈服门槛与密度下具有长度量纲，但它是“屈服水深 × 坡度”的乘积，不是可直接解释为水深的 h_y。记该乘积为 Π_c，基线值为

$$\Pi_c = h_y S_{\mathrm{eff}} = 0.04680025213635838\ \mathrm{m}. \tag{45}$$

坡度无量纲，故单位仍为 m。按口岸代理的有效坡度 $S_{\mathrm{eff}} = 0.02$ 换算，得到口岸坡度的等效屈服水深

$$h_{y,\mathrm{port}} = \frac{\Pi_c}{0.02} = 2.3400126068179192\ \mathrm{m}. \tag{46}$$

前者只对应局部临界水深，后者只对应口岸坡度，均不作为全线统一门槛。

网络基准参数见表8。参数为给定情景值，不是独立率定后验。

表 8 D8 网络路由基准参数

参数	基准值	单位	含义
V_{target}	2635835.294118	m^3	链 A 混合入沟体积
n	0.10	$\mathrm{s/m}^{1/3}$	等效 Manning 糙率
τ_y	800	Pa	屈服门槛
C_v	0.45	1	岩相体积分数情景
$S_{\min}$	0.02	1	缓坡驱动下限
τ_{src}	1800	s	三角源项历时
$h_{\min}$	0.10	m	首次到达分析阈值
C_{CFL}	0.2	1	设定 Courant 上限
T_{end}	21600	s	最长演算时限

源项取

$$Q_{\mathrm{src}}(t)=\begin{cases}\dfrac{2V_{\mathrm{target}}}{\tau_{\mathrm{src}}}\left(1-\dfrac{t}{\tau_{\mathrm{src}}}\right), & 0\le t\le\tau_{\mathrm{src}},\\ 0, & \text{其他},\end{cases}\tag{47}$$

并在每步直接使用累计函数

$$F(t)=V_{\mathrm{target}}(2u-u^2),\qquad u=\min\{\max(t/\tau_{\mathrm{src}},0),1\}\tag{48}$$

之差注入，避免跨越源项结束点时多计体积。

同一通量从供体扣除、向受体加入，域外通量单独累计。固定宽度下特征波速 $c=5v/3$，实际 Courant 数及步长约束为

$$Co_i=\frac{c_iW_i\Delta t}{a_i}\le C_{\mathrm{CFL}},\qquad \Delta t\le C_{\mathrm{CFL}}\min_i\frac{a_i}{c_iW_i}.\tag{49}$$

候选步先计入解析源量并估算源后波速，若超限则缩步重算。正性限制与 CFL 通过都不替代空间分辨率和物理参数验证。

7.5 D8 过程、峰值与域末端右删失

首次到达定义为

$$t_{\mathrm{arr},i}=\inf\{t:h_i(t)\ge h_{\min}\},\tag{50}$$

历史峰值逐步独立累积为

$$h_{\mathrm{peak},i}=\max_t h_i(t),\quad v_{\mathrm{peak},i}=\max_t v_i(t),\quad I_{\mathrm{peak},i}=\max_t\left[\rho_{\mathrm{df}}h_i(t)v_i(t)^2\right],\tag{51}$$

峰值需由同一时刻的 h,v 计算，不用终态水深代替，也不拼接不同时刻的 $\max h$ 与 $\max v$。

基线口岸代理首达 1965.996174 s，即 32.7666029 min。16 个去重 D8 配置全部到达口岸代理，首达范围为 17.5704376–44.7278067 min；最快端由链 B 高体积情景给出，最慢端由 V_{low} 体积情景给出。全部配置均在注入结束后转为静止，并因物质前锋到达或越过分析域末端而终止。两项需联合解释：注入后不再产生有效变化，不等于发生了物理自然停积；前锋抵达域边界，只说明演算被计算域截断。

全部配置的分析域末端里程均为 26045.4946703 m。这只是计算域的右删失边界，不是自然最大运程、现场停积边界或危险区外缘，也不用于估计分析域之外的真实影响范围。

基线口岸历史峰深 10.08530186 m、终态深度 2.298002229 m，全域历史最大深 13.31272084 m，口岸峰速 6.601474228 m/s。终态口岸深度虽接近按口岸坡度换算的 2.3400126068179192 m 临界量，但低于该量并不表示永久稳定，未跨域演算也无法恢复真实停积位置。过程量和全部配置的终止条件见表9，到达时间场与终态泥深场见图12。终态仅为分析截止时的像元状态，图幅不表示自然最大运程或实测漫滩范围。

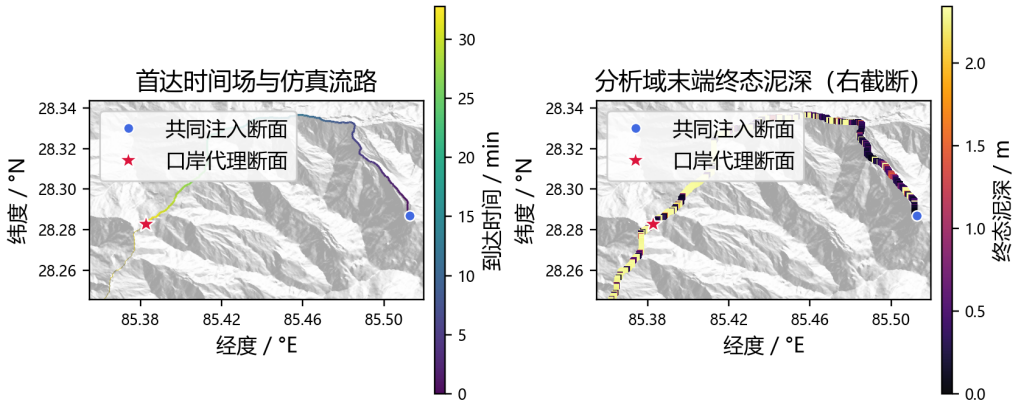


图 12　首达时间与终态泥深

表 9　D8 基线结果

指标	当前结果
口岸代理首达时间	1965.996174 s，即 32.7666029 min
16 配置首达范围	17.5704376–44.7278067 min
口岸历史峰深	10.08530186 m
口岸终态深度	2.298002229 m
全域历史最大深	13.31272084 m
口岸峰值速度	6.601474228 m/s
质量相对残差	$2.075820191829099\times10^{-15}$
最大实际 Courant 数	0.188281359
终止条件	注入结束后演算转为静止（全部配置）
停积性质	域末端右删失，非自然停积（全部配置）
域末端里程	26045.4946703 m

7.6 质量收支与数值闭合

逐步记录累计注入、域内、出域和损失体积，质量账本为

$$V_{\mathrm{inj}}(t) - V_{\mathrm{dom}}(t) - V_{\mathrm{out}}(t) = V_{\mathrm{loss}}(t). \tag{52}$$

以实际注入体积为分母，基线最大相对残差为 $2.075820191829099\times10^{-15}$；源项相对误差为 0，全域最小深度非负，最大实际 Courant 数为 0.188281359，未发生供体体积截断步骤。质量收支曲线见图A4。

这些数值说明源项积分、离散转移和状态记录在机器精度内闭合，CFL 实现未超过给定上限；这些数值只说明数值实现一致；固定地形、单流向、屈服门槛、等效糙率和集中源项是否等于真实物理过程，仍需现场检验。

7.7 有限库存、静止混合与分段动力对照

一维和 D8 均以集中注入和固定流变处理物源；沿程静止物料加入后团块会加速还是减速，需要另设模型。为此增加固定路径代表团块模型，仅保留动摩擦 μ_{dyn}、二次阻

力 κ 和入口速度 v_0 三个全局工程量。它不替代前述引擎，其速度也不等于真实前锋速度或流深。

对固定水平路线，令相邻支持点三维段长为 L_k、下坡角为 α_k，$w=v^2$，则

$$v\frac{dv}{d\ell}=g(\sin\alpha_k-\mu_{\mathrm{dyn}}\cos\alpha_k)-\kappa v^2,\qquad \frac{dw}{d\ell}=2a_k-2\kappa w. \tag{53}$$

左侧 v 因子不可省略。常参数段在 $\kappa>0$ 时满足

$$w(x)=w_{\mathrm{in}}e^{-2\kappa x}+\frac{a}{\kappa}(1-e^{-2\kappa x}),\qquad a=g(\sin\alpha-\mu_{\mathrm{dyn}}\cos\alpha). \tag{54}$$

当 $a<0$ 时首次停止距离为

$$x_{\mathrm{stop}}=\frac{\log\left(1+\kappa w_{\mathrm{in}}/|a|\right)}{2\kappa}, \tag{55}$$

$\kappa=0$ 时取连续极限 $x_{\mathrm{stop}}=w_{\mathrm{in}}/(2|a|)$。段内停止后不自动重启；未达目标是无后续输入、无再启动假设下的条件结论。

真实 GIS 未确认沿程可接触物源节点及库存，故真实路线最低基线为无新增节点。原始、填洼和单调三种高程剖面在固定基准 $\mu_{\mathrm{dyn}}=0.10$、$\kappa=10^{-5}\ \mathrm{m^{-1}}$、$v_0=30\ \mathrm{m/s}$ 下，分别于 18.360547、18.226423 和 18.535479 km 首次停止，对应 273.057496、281.660354 和 287.613654 s，均未到口岸。无节点工程扫描共 144 个配置，其中每个剖面 48 个配置；三种剖面合计 48 个配置正速通过、96 个首次停止。该比例只描述已检验工程域，不是发生概率。

合成节点固定于路线水平里程的 1/3 和 2/3 处，仅作机制示范。节点库存及静止加入的动量、混合损失为

$$B_{r,j}=q_rV_{r,0}\pi_j,\qquad B_{w,j}=q_wV_{w,0}\pi_j,\qquad \sum_j\pi_j=1, \tag{56}$$

$$m^+v^+=m^-v^-,\qquad v^+=\frac{v^-}{1+\lambda},\qquad E_{\mathrm{mix}}=\frac{1}{2}m^-(v^-)^2\frac{\lambda}{1+\lambda}\geq 0, \tag{57}$$

其中 $\lambda=\Delta m/m^-$。库存非负、有限且每节点至多支取一次；同一加入量不在入口预富集和沿程库存中重复计量。

机制对照 M0 关闭新增水但保留岩屑，M1 加入同一有限水库存并计入静止混合，M2 与 M1 的水岩库存完全相同，仅额外把累计新增水严格大于零后的下游动摩擦乘以 0.6。M2 的系数 0.6 是预先设定的假设映射，不是浓度率定律，也不依据是否到达口岸进行选择。三种对照的运程差见表10；表中只比较同一剖面、同一事件定义下的正速到达结果，未达组不参与时间差计算。

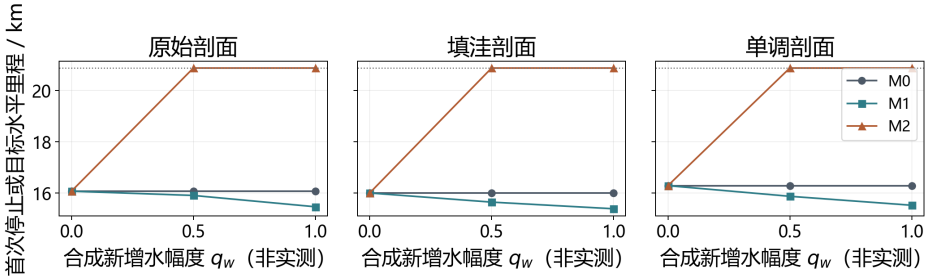
仅加入静止水会增加质量并按动量守恒降低速度，故 M1 相对 M0 缩短运程；M2 的额外距离来自唯一改变的下游减阻映射。它不证明“水越多越远”，也不说明真实沿程侵蚀会按此发生。M0、M1 和 M2 对照见图13。

有限库存系列的数值检查包含 23 项合成测试，全部通过。当前最大质量相对残差为 $2.076390489509933\times10^{-16}$，全程能量相对残差为 $6.18040483706205\times10^{-15}$，段能量相对残差为 $2.451294653897242\times10^{-12}$，节点动量和节点能量相对残差分别为 $4.429633044287857\times10^{-16}$ 和 $2.8349651483442284\times10^{-15}$。这些结果表明固定路径积分、库存扣减和解析停止规则与设定一致；物源可动性、真实流变或 M2 减阻映射仍需现场验证。

表 10　有限库存机制对照

q_r	q_w	$\Delta L_{\mathrm{M1-M0}}$/m	$\Delta L_{\mathrm{M2-M1}}$/m
0.5	0.5	−166.797	+4980.217
0.5	1.0	−608.791	+5422.211
1.0	1.0	−935.331	+6366.604

人工1/3、2/3节点，固定$q_r = 0.5$：加水混合与唯一假设减阻分离



M2仅假设μ乘0.6；三组均不证明实际补水、实际减阻或真实灾害到达

图 13　库存机制运程对照

7.8 综合判读与适用范围

问题四的主要结论是：在给定体积、公共断面和参数下，一维基线于 9.336769 km 自然能量停止，未达口岸；D8 的 16 个配置在 17.5704376–44.7278067 min 内到达口岸代理，但全部于 26045.4946703 m 分析域末端右删失，自然最大运程尚不能确定。两引擎差异来自停止规则、有效断面宽度、缓坡驱动与路由结构，不是同一观测量的两次测量。

D8 质量账本说明离散转移守恒；地形时效、源区体积、屈服门槛、糙率和单流向假设是否正确，仍需现场检验。有限库存模型表明静止加水通常因质量稀释缩短运程，只有预先设定的减阻假设才显著延长，而该减阻缺少独立观测。未取得可核查原文和观测定义的公开报道量值不进入验证。因此，D8 到达时间是条件路由结果，右删失末端是计算范围，M2 是机制假设；升级为口岸风险预报仍需源区释放、沿程流深流速、可核实物源库存、现场流变和独立多事件观测。

八、综合检验、适用范围与防灾策略

8.1 结果解释与适用范围

各模型回答的对象不同，主要检验及其不能推出的结论见表11。统计判别、数值闭合和条件成员分位不能自动跨越到真实灾害发生概率或现场预测技巧。

这些负结果分别限定了前兆、水力门槛、真实预警和自然运程的外推：问题一前五在留一年检验中仅重合 3–4 项，问题二峰值触发比 0.070032 未识别冰崩链，问题三真实释放与到达不可估计，问题四 D8 全部右删失而一维自然停止。辅助区间包含 10:30，不代表真实覆盖。

表 11 检验内容与适用范围

检验	支持内容	不能推出的结论
五层静力订正与同压对照	44018 个共同小时中 9016 个稳定度分类改变	参考高度就是真实安装高度
31 项统一候选、相关簇与留年检查	固定候选与同季匹配下的事件前环境异常排序	真实灾害因果、发生概率或普适前兆
完整床面受力与 78/6 状态分支	给定床层参数下的水流门槛和重力失稳分类	所有沟道的现场稳定性或安全认证
三组 IFS 起报的 3000 配对成员	固定 08:00 发布时可形成条件源区与即时释放到达分位	真实泥石流形成区间或频率校准
10:30 单点与融水提示核验	辅助即时释放区间包含 1/1、提示时差 2.5 小时	真实到达覆盖、真实灾害预警或长期覆盖率
确定性与结果一致性检查	单位、状态、支持与来源一致	自然过程被物理验证
D8 质量收支、CFL 与终止状态	固定地形和给定参数下的离散守恒与右删失状态	自然最大运程或实测危险区
一维自然停止与 D8 到达	两种结构在同一事件下的显著分歧	可从两引擎拼接最优时间或距离

8.2 不确定性的来源

观测与语义不确定度包括站点代表性、质量码释义、附件 4 冰面/冰下语义、附件 7 采集时刻、口岸断面吸附和历史长度定义。参数不确定度包括曲线数、RGI6 编目面积权重、释放比例、物源放大、床厚与摩擦角、Manning 糙率、屈服门槛、D8 配置和有限库存幅度。结构不确定度包括静力积分假设、集总产流、融水代理、AR(1) 残差、相关簇阈值、单流向路由和右删失分析域。

IFS 成员内部同时扰动回归系数、连续小时残差和传播配置，但三者不是由多事件联合估计，抽样概率只属于给定模型集合[10]。D8 十六个配置等权也不表示现实配置概率；改变集合会使条件分位移动。质量残差 2.075820×10^{-15} 和最大 Courant 数 0.188281 说明数值实现满足给定约束，不说明地形时效、源区体积、屈服门槛或流变正确。

8.3 分层预警与监测建议

第一层是固定时次的环境值守。可在每日固定时次计算 31 项关联指标、近地层稳定度和水源比，但只有单事件关联的头部特征应作为背景解释，不能直接升级为灾害概率。

第二层是源区监测与前向预报。优先补充冰体位移、裂缝、地震或声学释放信号以及可核查的泥石流形成时刻；将真实 t_{trig} 事件目录用于重新训练、区间校准和独立覆盖，而不是用 t_m 或口岸冲击替代。发布端应固定数据截止规则，记录模式起报、实际可用延迟和航线；本次六小时分发上界与 10:30 锚点只构成 2.5 小时融水提示时差，不足以保证真实灾害预警获得正提前量。

第三层是传播与现场处置。D8 到达时间只适用于同一体积、断面、地形和参数设定；若物质前锋进入分析域末端，应继续扩展域或标记右删失，不把 26045.494670 米当

作自然边界。沿程物源节点需有库存和可动性依据才能进入主情景；堰塞湖漫溢作为背景风险，溃决流量需另建坝体、库容和出流模型。

8.4 模型评价与改进顺序

本文的改进在于结果可追溯，而非追求表面精度：前五不排除 API 与净辐射，床面门槛含沿坡自重并区分重力失稳，问题三固定发布时次，问题四区分自然停止与域删失；代价是保留多处不利结论，并维护结果与数据来源的对应关系。

后续改进按观测收益排序：先获取真实形成时刻、源区释放和现场流深流速；其次补充床厚、材料强度、孔压及沿程库存；再以独立多事件目录训练和检验时间模型；最后才考虑二维横向扩散或复杂流变。扩大候选池、挑选有利端点或放宽区间都不能替代这些观测。

九、结论

本文形成共享候选、几何、门槛、发布时刻和传播参数的条件分析框架。

1. 五层静力订正使 9016/44018 个可算小时的稳定度分类改变，占 20.48%；订正后 Ri_b 中位 −0.0294，夜间逆温频率 60.07%。31 项统一候选去冗余后的前五为 API、净辐射、24 小时升温、0.1 米土壤含水率和潜热通量，A 为 3.829043 至 1.133226；该结果限定为本次事件前环境关联。

2. 床面受力纳入床体沿坡自重后，84 条历史沟道分为 78 条水流控制和 6 条重力失稳。正式中心线 q_c = 67.588 m^2/s、Q_c = 2.893667 × 10^3 m^3/s；事件窗水源峰值 202.650 m^3/s，最大 T = 0.070032，未出现 $T \geq 1$。335 个源区候选像元不构成冰体失稳识别，严格共同域空间验证不可估计。

3. 固定 08:00 发布的三组 IFS 起报生成 3000 个有限配对成员。源区融水代理区间为 09:03:22.847–10:57:51.047，即时释放条件到达区间为 09:35:26.068–11:31:59.410，$p_{\text{actionable}}$ = 1；真实释放、形成和实际到达覆盖不可估计，真实灾害预警尚不能给出。10:30 落入辅助即时释放区间（1/1），但辅助到达区间提前量为 −1.033169 至 +0.909426 小时，信息时差 2.5 小时不构成命中提前量；同链沿沟距离 22.924 千米，16 配置首达 17.570–44.728 分钟。

4. 问题四的混合入沟体积为 2.635835 × 10^6 m^3。D8 基线 32.766603 分钟到达口岸代理，质量相对残差 2.075820 × 10^{-15}、最大 Courant 数 0.188281；16 个配置均在 26045.494670 米分析域末端右删失，不足以外推自然运程。一维基线 9.336769 千米自然停止；144 个无节点配置中 48 个正速通过、96 个首次停止。

5. ERA5 与 MERRA2 同日降水为 7.50 和 1.22 毫米；IFS 起报时刻与产品版本已分类保存。真实形成时刻、现场传播观测和沿程库存仍缺失，故本文给出条件预报及其适用范围，不提供真实灾害 95% 覆盖或工程设防保证。

参考文献

[1] Schneider D, Huggel C, Haeberli W, Kaitna R. Unraveling driving factors for large rock–ice avalanche mobility. Earth Surface Processes and Landforms, 2011, 36(14): 1948–1966.

[2] Iverson R M. The physics of debris flows. Reviews of Geophysics, 1997, 35(3): 245–296.

[3] Richardson S D, Reynolds J M. An overview of glacial hazards in the Himalayas. Quaternary International, 2000, 65–66: 31–47.

[4] Hawkins R H, Ward T J, Woodward D E, Van Mullem J A. Curve Number Hydrology: State of the Practice. Reston: American Society of Civil Engineers, 2009: 6–20.

[5] Hock R. Glacier melt: a review of processes and their modelling. Progress in Physical Geography, 2005, 29(3): 362–391.

[6] Guzzetti F, Peruccacci S, Rossi M, Stark C P. The rainfall intensity–duration control of shallow landslides and debris flows: an update. Landslides, 2008, 5(1): 3–17.

[7] O'Brien J S, Julien P Y, Fullerton W T. Two-dimensional water flood and mudflow simulation. Journal of Hydraulic Engineering, 1993, 119(2): 244–261.

[8] Chow V T. Open-Channel Hydraulics. New York: McGraw-Hill, 1959: 89–127.

[9] Tarboton D G. A new method for the determination of flow directions and upslope areas in grid digital elevation models. Water Resources Research, 1997, 33(2): 309–319.

[10] Beven K, Binley A. The future of distributed models: model calibration and uncertainty prediction. Hydrological Processes, 1992, 6(3): 279–298.

[11] DeepSeek，deepseek-v4.1-flash，深度求索（DeepSeek），2026-10-30。

附　录

附录 1：支撑材料与程序说明

支撑材料的全部文件按类别列于表12，源码位于源码目录，论文取值所用数据位于数据目录。数据目录共 88 个文件，包括公共统计、四问结果、外部对照以及论文取值汇总与逐值溯源；每个数值可按溯源文件查到源文件和字段定位。全部参数、状态记录和逐时输出随计算成果保存，文件清单与目录结构对应，便于复现。

表 12　支撑材料文件清单

类别	文件
主流程	run_all.py、prepare_inputs.py、README.md、requirements.txt、复现.ps1
公共库	mmlib/__init__.py、finite_inventory.py、geodesy.py、glacier_masks.py、hydro.py、interval_metrics.py、io_data.py、meteo.py、ranking.py、terrain.py、terrain_prep.py、trigger.py
问题一	step01_boundary_layer.py、step01b_preimpact.py、step01c_unified_association.py
问题二	step02_trigger_threshold.py、step02b_spatial.py、step02c_spatial_matched.py、step02_figures.py
问题三	step03_forecast.py、step03_time_support_sensitivity.py、step03_figures.py
问题四	step04_simulation.py、step04b_sim2d.py、step04b_map.py、step04c_dynamics.py、step04_figures.py、step04b_figures.py
问题四仿真	问题四仿真过程与可视化.pdf（支撑材料根目录）
数据	数据目录：公共 6、问题一 22、问题二 27、问题三 11、问题四 15、外部对照 5、论文取值 2，共 88 个文件；文件名前缀对应问题与步骤
数据整理与导出	step00_data_inventory.py、step06_data_audit.py、archive_external.py、select_paper_values.py、validate_v50.py、export_summary.py
测试	test01b_hydrostatic.py、test01b_preimpact.py、test02c03_protocols.py、test04c_dynamics.py、test_scs_event_runoff.py、test_time_support.py、test_v32_data_fixes.py
AI 使用详情	AI 使用详情.pdf（支撑材料根目录）

问题四的完整仿真流程、参数、逐时快照、到达/峰值/终态图和情景敏感性见《问题四仿真过程与可视化.pdf》，正文图 10 至图 13 为其中核心结果。

附录 2：补充诊断图

以下四图为补充诊断，正文结论以统计与物理结果为准。稳定度图并列回归斜率与闭合比；共同域三尺度均因正组不足判为不可估计；起报图给出三次 IFS 起报、可用上界与 08:00 发布；质量收支图给出机器精度闭合。

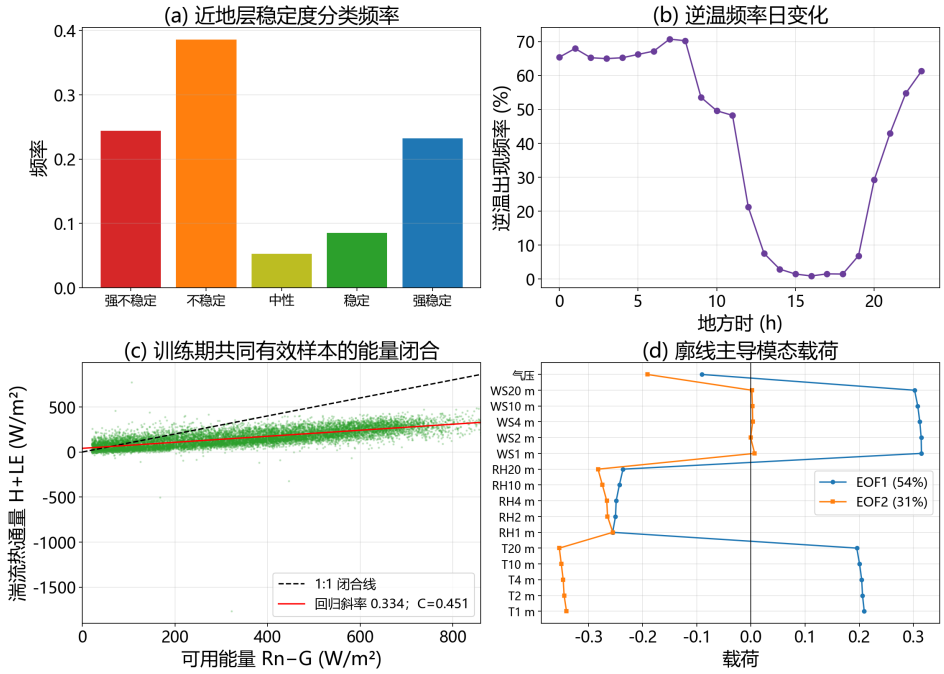


图 A1　稳定度与能量诊断

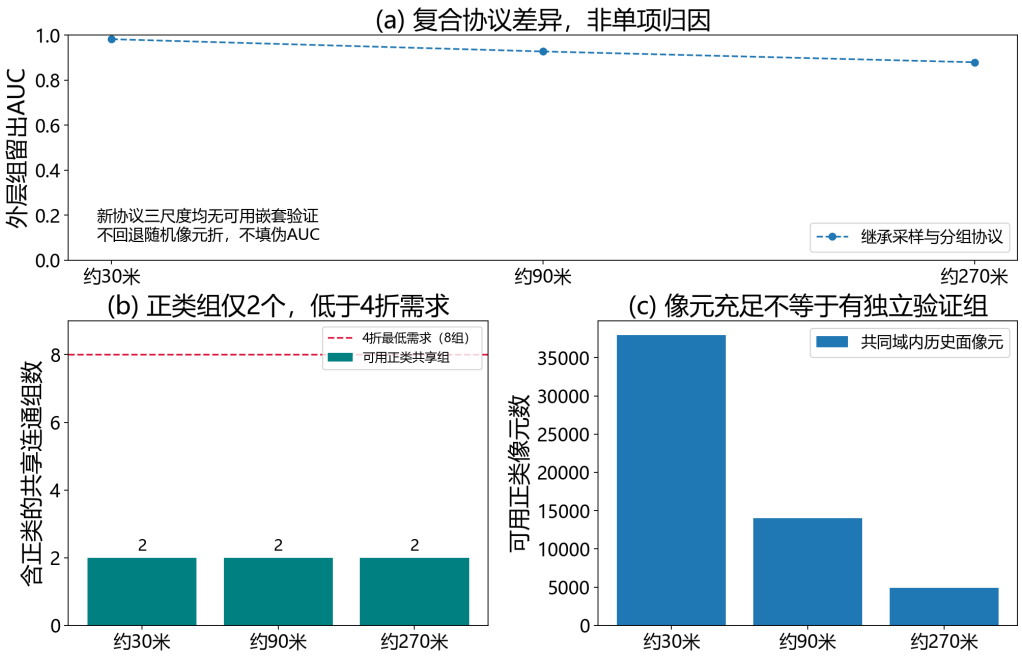


图 A2　共同域空间隔离

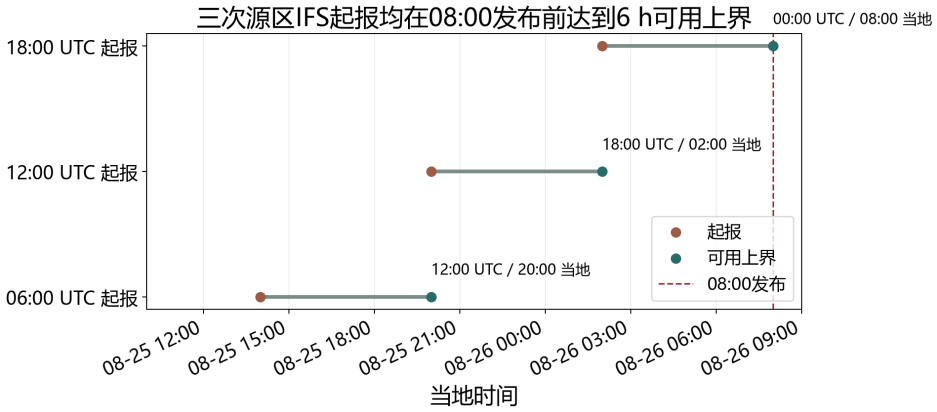


图 A3　IFS 起报可用性

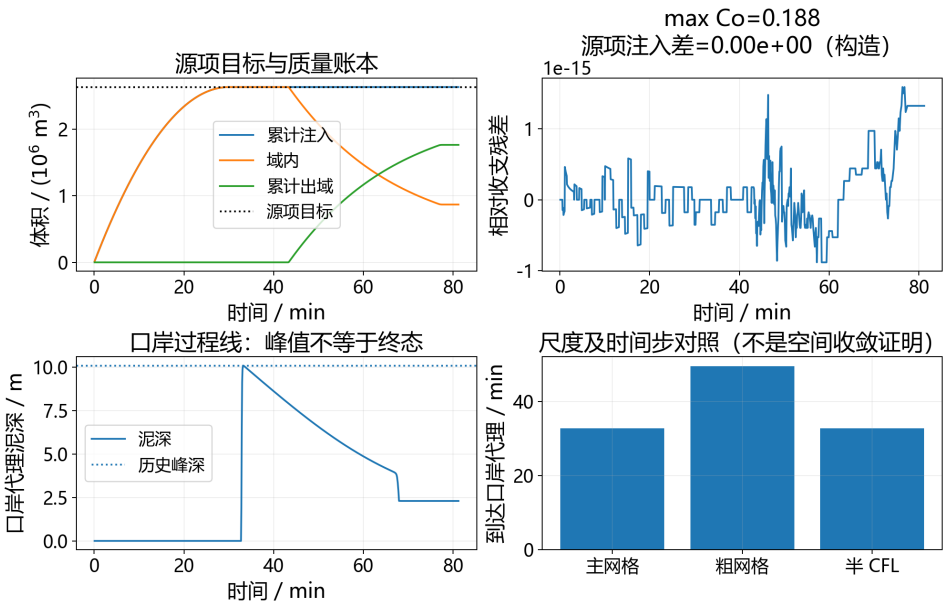


图 A4　质量收支诊断

附录 3：AI 使用声明

本文使用了 deepseek-v4.1-flash 模型（参考文献 [11]），用于资料查找、代码修改和论文语言润色；模型输出仅作辅助，研究设计、计算与结论由作者负责。

附录 4：软件环境与代码

本文的代码编辑、调试与运行环境为 Visual Studio Code（VS Code），解释器为 Python 3.13.5，依赖版本见 `requirements.txt`。以下按主流程、公共库、四问计算与绘图、数据整理与校验、测试的顺序，全文列出支撑材料源码目录中的 43 个 Python 文件。代码按提交内容列出，仅将当前等宽字体缺少字形的温度符号、分数斜线与平行号分别显示为 °C、/、‖，其余字符未改动。

主流程

`run_all.py` 全流程入口、步骤顺序与断点续跑

```python
"""一键复现：按依赖顺序运行全部计算与图件脚本。

用法
----
   python run_all.py     # 全流程
   python run_all.py --skip-fig # 只跑计算
   python run_all.py --only 04 # 只跑问题4，需已准备依赖

依赖顺序
--------
step00 → step04 → step04b → step01 → step01b → step01c → step02 → step02_figures → step04_figures
     → step03 → step03_figures
问题1/2 的起动流量 q_c 以问题4 的中心线比降为唯一口径（建模成果/问题2.md §2.4.2），
故 step04 必须先于 step01 运行——它产出 04_中心线校验.json 与 04_仿真基线.json。
step04b（二维过程引擎）消费 step04 的 04_初始体积.json（链A 源区与体积族），
产出 04b_二维引擎.json（到达口岸样本 + 质量账本），供 step03 的同断面情景对照消费；
两者都以口岸像元（85.3826°E, 28.2826°N）为同一到达目标。
问题3 依赖问题2（触发序列）与问题4/04b（运行段到达时间），排在 02 之后。
"""
from __future__ import annotations

import argparse
import hashlib
import json
import os
import platform
import subprocess
import sys
import time
from datetime import datetime
from pathlib import Path

HERE = Path(__file__).resolve().parent
ROOT = HERE.parent.parent
PY = sys.executable
LOG_PATH = ROOT / "求解结果" / "运行记录" / "实验日志.jsonl"


def log_event(record: dict) -> None:
    """把一条运行事实追加到 JSONL 实验日志。

    日志按行独立成 JSON，便于逐行解析与追加，不需要读入全文件。
    写入失败不阻断主流程——复现性记录不应让计算失败。
    """
    try:
        LOG_PATH.parent.mkdir(parents=True, exist_ok=True)
        record = {"time": datetime.now().astimezone().isoformat(timespec="seconds"), **record}
        with LOG_PATH.open("a", encoding="utf-8") as fh:
            fh.write(json.dumps(record, ensure_ascii=False) + "\n")
    except OSError as exc:                          # pragma: no cover
        print(f"  警告：实验日志写入失败（{exc}）")

STEPS = [
    ("prep", "prepare_inputs.py", "原始附件清点与解压"),
    ("00", "step00_data_inventory.py", "数据清单与基础统计"),
    ("04", "step04_simulation.py", "问题4 仿真环境与参数（产出中心线口径）"),
    ("04b", "step04b_sim2d.py", "问题4 栅格路由过程引擎（数值验收与情景对照）"),
    ("04c", "step04c_dynamics.py", "问题4 有限库存分段动力对照与解析验收"),
    ("01", "step01_boundary_layer.py", "问题1 边界层结构与关联性排序"),
    ("01b", "step01b_preimpact.py", "问题1 静力修正与标签外冲击前旁证"),
    ("01c", "step01c_unified_association.py", "问题1 31项统一候选与环境关联前五"),
    ("02", "step02_trigger_threshold.py", "问题2 临界条件与触发评估模型"),
    ("02b", "step02b_spatial.py", "问题2 像元环境对照与分组验证"),
    ("02c", "step02c_spatial_matched.py", "问题2 共同沟谷域与内外层空间隔离"),
    ("02f", "step02_figures.py", "问题2 图件"),
    ("04f", "step04_figures.py", "问题4 图件"),
    ("04bf", "step04b_figures.py", "问题4 栅格路由过程图件"),
```

```
    ("04bmf", "step04b_map.py", "问题4 首达时间场与终态停积双联地图"),
    ("03", "step03_forecast.py", "问题3 预报模型与区间"),
    ("03s", "step03_time_support_sensitivity.py", "问题3 历史小时支持组合敏感性"),
    ("03f", "step03_figures.py", "问题3 图件"),
    ("06", "step06_data_audit.py", "同日界外部核验与目标坐标敏感性"),
    ("07", "archive_external.py", "外部响应哈希归档（离线复核）"),
    ("05s", "select_paper_values.py", "从当前结果生成本版论文精选值"),
    ("v50", "validate_v50.py", "v5.0 全链确定性验收"),
    ("05", "export_summary.py", "当前结果与逐值来源导出"),
]


def main() -> int:
    for stream in (sys.stdout, sys.stderr):
        if hasattr(stream, "reconfigure"):
            stream.reconfigure(errors="replace")
    ap = argparse.ArgumentParser()
    ap.add_argument("--skip-fig", action="store_true", help="跳过图件脚本")
    ap.add_argument("--only", default=None, help="只运行以该前缀开头的步骤")
    ap.add_argument("--from-step", default=None, help="resume from an existing dependency checkpoint")
    args = ap.parse_args()

    steps = STEPS
    if args.from_step:
        index=next(i for i,s in enumerate(steps) if s[0]==args.from_step)
        steps=steps[index:]
    if args.skip_fig:
        steps = [s for s in steps if not s[0].endswith("f")]
    if args.only:
        k = args.only
        steps = [s for s in steps if s[0].startswith(k)]

    if not steps:
        print("没有匹配的步骤")
        return 1

    env_note = f"python = {PY}\n工作目录 = {ROOT}"
    print(env_note)
    print("=" * 72)

    log_event({
        "event": "run_start",
        "python": sys.version.split()[0],
        "executable": PY,
        "platform": platform.platform(),
        "cwd": str(ROOT),
        "steps": [{"tag": t, "script": s, "desc": d} for t, s, d in steps],
        "argv": sys.argv[1:],
    })

    t_all = time.time()
    batch_id = datetime.now().strftime("resume-%Y%m%d-%H%M%S")
    def fingerprints():
        found = {}
        for folder in (HERE, ROOT / "求解结果" / "数据", ROOT / "求解结果" / "图表"):
            for p in folder.rglob("*"):
                if p.is_file() and p.suffix not in (".pyc",):
                    stat = p.stat()
                    found[p.relative_to(ROOT).as_posix()] = {
                        "path": p.relative_to(ROOT).as_posix(),
                        "sha256": hashlib.sha256(p.read_bytes()).hexdigest(),
                        "bytes": stat.st_size,
                        "mtime_ns": stat.st_mtime_ns,
                    }
        return found
    results = []
    output_owners = {}
    child_env = os.environ.copy()
    child_env["PYTHONIOENCODING"] = "utf-8"
    child_env["PYTHONUTF8"] = "1"
    for tag, script, desc in steps:
        path = HERE / script
        if not path.exists():
            print(f"[{tag}] 缺失 {script}，跳过")
            results.append((tag, script, "MISSING", 0.0))
            continue
        print(f"\n[{tag}] {desc}")
        print(f"  {script}")
        before = fingerprints()
        started_at = datetime.now().astimezone().isoformat()
        t0 = time.time()
        r = subprocess.run([PY, str(path)], cwd=str(HERE),
                           capture_output=True, text=True, encoding="utf-8",
                           errors="replace", env=child_env)
        dt = time.time() - t0
        ok = r.returncode == 0
        run_dir = ROOT / "求解结果" / "运行记录" / f"{batch_id}-{tag}"
        run_dir.mkdir(parents=True, exist_ok=False)
        (run_dir / "stdout.txt").write_text(r.stdout or "", encoding="utf-8")
        (run_dir / "stderr.txt").write_text(r.stderr or "", encoding="utf-8")
        after = fingerprints()
        outputs = [v for k, v in after.items() if before.get(k) != v and k.startswith("求解结果/")]
        for record in outputs:
            output_owners.setdefault(record["path"], []).append(tag)
```

```
        manifest = {
            "schema_version": 2, "run_id": run_dir.name, "return_code": r.returncode,
            "started_at": started_at, "ended_at": datetime.now().astimezone().isoformat(),
            "command": [PY, str(path)], "seed": 20260826, "python": PY,
            "script": path.relative_to(ROOT).as_posix(), "script_sha256": hashlib.sha256(path.read_bytes()).hexdigest
                (),
            "inputs": list(before.values()),
            "raw_archive_manifest": str(ROOT / "临时产物" / "原始附件清单.json"),
            "outputs": outputs, "duration_seconds": round(dt, 3),
            "goal": desc, "baseline": "B1/B2 inherited methods plus explicit repaired theory",
            "success_criterion": "process success; scientific assertions inside steps; review still required",
            "failure_interpretation": "failed steps block downstream execution; no failure is a passed result",
            "stdout": str((run_dir / "stdout.txt").relative_to(ROOT)),
            "stderr": str((run_dir / "stderr.txt").relative_to(ROOT))}
        (run_dir / "run_manifest.json").write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf
            -8")
        for line in (r.stdout or "").splitlines()[-8:]:
            print(f"  {line}")
        if not ok:
            print(f"  失败（exit={r.returncode}）")
            for line in (r.stderr or "").splitlines()[-25:]:
                print(f"  {line}")
        print(f"  {'完成' if ok else '失败'}, 耗时 {dt:.1f} s")
        results.append((tag, script, "OK" if ok else "FAIL", dt))
        log_event({
            "event": "step_end",
            "tag": tag,
            "script": script,
            "desc": desc,
            "return_code": r.returncode,
            "seconds": round(dt, 1),
            "status": "OK" if ok else "FAIL",
            "stdout_tail": (r.stdout or "").splitlines()[-5:],
            "stderr_tail": (r.stderr or "").splitlines()[-5:] if not ok else [],
        })
        if not ok:
            break

    print("\n" + "=" * 72)
    print(f"{'步骤':<5}{'状态':<8}{'耗时/s':>9} 脚本")
    for tag, script, st, dt in results:
        print(f"{tag:<5}{st:<8}{dt:>9.1f} {script}")
    print(f"\n总耗时 {time.time() - t_all:.1f} s")

    bad = [r for r in results if r[2] != "OK"]
    duplicate_owners = {
        path: owners for path, owners in output_owners.items() if len(owners) > 1
    }
    if duplicate_owners:
        print("\n输出所有权冲突（同一求解结果被多个步骤写入）：")
        for path, owners in sorted(duplicate_owners.items()):
            print(f"  {path}: {', '.join(owners)}")
    # Bind the settled result tree only after downstream steps have finished.
    if not bad and not duplicate_owners:
        records = fingerprints()
        aggregate = ROOT / "求解结果" / "运行记录" / f"{batch_id}-aggregate"
        aggregate.mkdir()
        source_records = [v for k, v in records.items() if k.startswith("支撑材料/")]
        result_records = [v for k, v in records.items() if k.startswith("求解结果/")]
        production_prefixes = ("求解结果/数据/", "求解结果/图表/")
        production_outputs = {
            path for path in records
            if path.startswith(production_prefixes)
            and not path.startswith("求解结果/数据/cache/")
        }
        internal_cache_outputs = {
            path for path in records
            if path.startswith("求解结果/数据/cache/")
        }
        unowned_outputs = sorted(production_outputs - set(output_owners))
        untracked_internal_cache = sorted(internal_cache_outputs - set(output_owners))
        owner_check_ok = not unowned_outputs
        raw_manifest = ROOT / "临时产物" / "原始附件清单.json"
        source_records.append({"path": raw_manifest.relative_to(ROOT).as_posix(),
                               "sha256": hashlib.sha256(raw_manifest.read_bytes()).hexdigest(),
                               "bytes": raw_manifest.stat().st_size})
        receipt = {"run_id": aggregate.name, "return_code": 0, "seed": 20260826,
                   "command": [PY, str(Path(__file__).resolve()), *sys.argv[1:]],
                   "inputs": source_records, "outputs": result_records,
                   "step_runs": [f"{batch_id}-{t}" for t, _, _, _ in results],
                   "goal": "settled complete pipeline result binding",
                   "all_steps_ok": True,
                   "output_owner_check": {
                       "status": "pass" if owner_check_ok else "fail",
                       "tracked_outputs": len(production_outputs),
                       "production_outputs": len(production_outputs),
                       "unowned_outputs": unowned_outputs,
                      "duplicate_owners": {},
                       "all_result_output_records": len(result_records),
                       "internal_cache": {
                          "count": len(internal_cache_outputs),
                          "untracked_reused_count": len(untracked_internal_cache),
                          "scope": (
                             "internal deterministic caches excluded from deliverable ownership; "
                             "their current hashes remain bound in outputs"
```

```
                    ),
                },
            }}
    (aggregate / "run_manifest.json").write_text(json.dumps(receipt, ensure_ascii=False, indent=2), encoding="utf
        -8")
    if not owner_check_ok:
        bad.append(("aggregate", "run_all.py", "FAIL", 0.0))
        print("\n输出归属覆盖不完整：")
        for path in unowned_outputs:
            print(f" {path}")
log_event({
    "event": "run_end",
    "total_seconds": round(time.time() - t_all, 1),
    "steps": [{"tag": t, "script": s, "status": st, "seconds": round(dt, 1)}
              for t, s, st, dt in results],
    "all_ok": not bad,
})
return 0 if not bad and not duplicate_owners else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

`prepare_inputs.py` 附件清点、解压与输入准备

```
"""Prepare local supplied archives without modifying the originals."""
from __future__ import annotations

import hashlib
import json
from pathlib import Path
import sys
import zipfile

from mmlib import io_data as io


def main() -> int:
    candidates = list(io.ROOT.parent.glob('*/附件'))
    if len(candidates) != 1:
        raise RuntimeError('Expected one sibling problem/attachment directory')
    archives = candidates[0]
    manifest = []
    for p in sorted(archives.iterdir()):
        if p.suffix.lower() not in ('.zip', '.rar'):
            continue
        prefix = p.name[:3]
        target = io.RAW / (p.stem if p.suffix == '.zip' else '（3）冰川高程变化')
        target.mkdir(parents=True, exist_ok=True)
        root = target.resolve()
        count = 0
        if p.suffix == '.zip':
            with zipfile.ZipFile(p) as ar:
                for member in ar.infolist():
                    dest = (root / member.filename).resolve()
                    if not dest.is_relative_to(root):
                        raise RuntimeError('Archive path escapes target')
                    if member.is_dir():
                        dest.mkdir(parents=True, exist_ok=True)
                    elif not dest.exists() or dest.stat().st_size != member.file_size:
                        dest.parent.mkdir(parents=True, exist_ok=True)
                        with ar.open(member) as src, dest.open('wb') as dst:
                            import shutil
                            shutil.copyfileobj(src, dst)
                    count += 1
        else:
            import libarchive
            with libarchive.file_reader(str(p)) as ar:
                for member in ar:
                    dest = (root / member.pathname).resolve()
                    if not dest.is_relative_to(root):
                        raise RuntimeError('Archive path escapes target')
                    if member.isdir:
                        dest.mkdir(parents=True, exist_ok=True)
                    elif member.isfile:
                        dest.parent.mkdir(parents=True, exist_ok=True)
                        if not dest.exists() or dest.stat().st_size != member.size:
                            with dest.open('wb') as out:
                                for block in member.get_blocks():
                                    out.write(block)
                    count += 1
        manifest.append({'archive': p.name, 'sha256': hashlib.file_digest(p.open('rb'), 'sha256').hexdigest(),
                         'bytes': p.stat().st_size, 'members': count, 'target': str(target.relative_to(io.ROOT))})
        print(prefix, count, flush=True)
    io.write_json(io.ROOT / '临时产物/原始附件清单.json', {'archives': manifest, 'python': sys.version})
    return 0


if __name__ == '__main__':
    raise SystemExit(main())
```

公共库

mmlib/__init__.py　公共库入口

```python
"""吉隆泥石流建模工具箱。"""
from . import io_data # noqa: F401

__all__ = ["io_data"]
```

mmlib/finite_inventory.py　有限库存团块动力

```python
"""SI fixed-path, finite stationary-inventory dynamics; not a depth solver."""
from __future__ import annotations

import math

import numpy as np
from scipy.integrate import quad
from scipy.optimize import brentq

G = 9.80665
RHO_R, RHO_W, RHO_I = 2650.0, 1000.0, 900.0


def _phi_psi(u: float) -> tuple[float, float]:
    if abs(u) < 1e-4:
        return (1 - u/2 + u*u/6 - u**3/24 + u**4/120,
                .5 - u/6 + u*u/24 - u**3/120 + u**4/720)
    phi = -math.expm1(-u)/u
    return phi, (1-phi)/u


def speed_squared(w0: float, a: float, kappa: float, x: float) -> float:
    """Analytic segment speed squared, valid only before the first stop."""
    if kappa == 0:
        return w0 + 2*a*x
    phi, _ = _phi_psi(2*kappa*x)
    return w0 + 2*(a-kappa*w0)*x*phi


def stopping_distance(w0: float, a: float, kappa: float) -> float:
    if w0 < 0 or kappa < 0:
        raise ValueError("negative speed squared or drag")
    if w0 == 0:
        return 0.0
    if a >= 0:
        return math.inf
    if kappa == 0:
        return w0/(2*abs(a))
    return math.log1p(kappa*w0/abs(a))/(2*kappa)


def travel_time(w0: float, a: float, kappa: float, x: float,
                w1: float | None = None) -> float:
    if x == 0:
        return 0.0
    if w0 <= 0 or x < 0 or kappa < 0:
        raise ValueError("positive entrance velocity and nonnegative distance/drag required")
    if w1 is None:
        w1 = speed_squared(w0, a, kappa, x)
    if w1 < -1e-9*max(w0, 1):
        raise ValueError("time requested beyond first stop")
    v0, v1 = math.sqrt(w0), math.sqrt(max(w1, 0.0))
    if kappa == 0:
        return 2*x/(v0+v1)
    if a == 0:
        return math.expm1(kappa*x)/(kappa*v0)
    phi, _ = _phi_psi(2*kappa*x)
    dv = 2*(a-kappa*w0)*x*phi/(v0+v1)
    if a < 0:
        b = -a
        angle = math.atan2(-dv*math.sqrt(kappa*b), b+kappa*v0*v1)
        return angle/math.sqrt(b*kappa)
    veq = math.sqrt(a/kappa)
    logpart = math.log1p(dv/(v0+veq))
    numerator = kappa*x + logpart
    if numerator > 1e-10*(abs(kappa*x)+abs(logpart)):
        return numerator/math.sqrt(a*kappa)
    # The analytic antiderivative can subtract close terms above terminal speed.
    return x*quad(lambda u: 1/math.sqrt(speed_squared(w0, a, kappa, x*u)),
                  0, 1, epsabs=1e-11, epsrel=2e-12)[0]


def segment(w0: float, a: float, kappa: float, length: float) -> dict:
    if not all(math.isfinite(x) for x in (w0, a, kappa, length)):
        raise ValueError("finite SI segment inputs required")
    if w0 <= 0 or kappa < 0 or length <= 0:
        raise ValueError("positive entrance speed/length and nonnegative drag required")
    stop = stopping_distance(w0, a, kappa)
    stopped = stop <= length + 8*np.finfo(float).eps*max(length, 1)
    x = min(stop, length) if stopped else length
    w1 = 0.0 if stopped else speed_squared(w0, a, kappa, x)
    if w1 <= 0 and not stopped:
        raise FloatingPointError("unresolved nonpositive velocity")
```

```
   phi, psi = _phi_psi(2*kappa*x)
   iw = w0*x*phi + 2*a*x*x*psi
   if iw < -1e-9*max(w0*x, 1):
      raise FloatingPointError("negative squared-speed integral")
   return {"distance_m": x, "w_out_m2_s2": w1,
         "time_s": travel_time(w0, a, kappa, x, w1),
         "stopped": bool(stopped), "integral_w_dell_m3_s2": max(0.0, iw)}


def mass(volumes: np.ndarray) -> float:
   return float(np.dot(volumes, [RHO_R, RHO_I, RHO_W]))


def stationary_mix(volumes, speed: float, added) -> dict:
   before, added = np.asarray(volumes, float), np.asarray(added, float)
   if before.shape != (3,) or added.shape != (3,) or np.any(before < 0) or np.any(added < 0):
      raise ValueError("nonnegative rock, ice, water vectors required")
   m0, dm = mass(before), mass(added)
   if m0 <= 0 or speed < 0:
      raise ValueError("positive mass and nonnegative velocity required")
   after = before + added
   v1 = m0*speed/(m0+dm)
   loss = .5*m0*speed*speed*dm/(m0+dm)
   return {"volumes_after_m3": after, "mass_before_kg": m0,
         "added_mass_kg": dm, "mass_after_kg": m0+dm,
         "velocity_after_m_s": v1, "lambda": dm/m0,
         "mix_loss_J": loss,
         "momentum_residual_kg_m_s": (m0+dm)*v1-m0*speed,
         "kinetic_residual_J": .5*m0*speed**2-.5*(m0+dm)*v1**2-loss}


def resolve_mode(mode, q_w):
   """Map a requested mechanism label to (added water, effective dynamics mode).

   M0 always closes the added-water channel; M2 without added water cannot trigger
   the pre-registered friction mapping and is therefore the same run as M1 at q_w=0.
   """
   if mode not in ("M0", "M1", "M2"):
      raise ValueError("requested mode must be M0, M1 or M2")
   if not np.isfinite(q_w) or q_w < 0:
      raise ValueError("requested water inventory must be finite and nonnegative")
   water = 0.0 if mode == "M0" else float(q_w)
   effective = "M2" if mode == "M2" and water > 0 else "M1"
   return water, effective


def prepare(s_h, z, initial_volumes, nodes=(), target_sh=None, subdivisions=1,
         mu=.1, kappa=1e-5, mode="M1") -> dict:
   """Split geometry at fixed physical nodes/target; never create new stocks."""
   sh, zz = np.asarray(s_h, float), np.asarray(z, float)
   iv = np.asarray(initial_volumes, float)
   if (sh.ndim != 1 or sh.shape != zz.shape or len(sh) < 2 or
         np.any(np.diff(sh) <= 0) or not np.all(np.isfinite(sh)) or
         not np.all(np.isfinite(zz)) or sh[0] != 0):
      raise ValueError("finite increasing horizontal route starting at zero required")
   if iv.shape != (3,) or np.any(iv < 0) or mass(iv) <= 0:
      raise ValueError("invalid initial components")
   if mode not in ("M0", "M1", "M2") or mu < 0 or kappa < 0 or subdivisions < 1:
      raise ValueError("invalid dynamics configuration")
   target = float(sh[-1] if target_sh is None else target_sh)
   if not 0 < target <= sh[-1]:
      raise ValueError("target must lie on route")
   stock = []
   for j, n in enumerate(nodes):
      where = float(n["s_h_m"])
      vv = np.array([n.get("rock_m3", 0), n.get("ice_m3", 0), n.get("water_m3", 0)], float)
      if not 0 < where < target or np.any(vv < 0) or not np.all(np.isfinite(vv)):
         raise ValueError("inventory nodes must be interior, finite, nonnegative")
      if mode == "M0":
         vv[2] = 0
      stock.append({"id": str(n.get("id", j)), "s_h_m": where, "volumes_m3": vv})
   if len({n["id"] for n in stock}) != len(stock):
      raise ValueError("inventory identifiers must be unique")
   points = sorted(set([0.0, target] + list(sh[(sh > 0) & (sh < target)]) +
                  [n["s_h_m"] for n in stock]))
   split = [points[0]]
   for left, right in zip(points[:-1], points[1:]):
      split.extend(np.linspace(left, right, int(subdivisions)+1)[1:])
   split = np.asarray(split)
   elev = np.interp(split, sh, zz)
   dh, dz = np.diff(split), -np.diff(elev)
   ll = np.hypot(dh, dz)
   volumes, added_water = iv.copy(), 0.0
   segments, actions = [], []
   for i in range(len(dh)):
      for n in stock:
         if n["s_h_m"] == split[i]:
            m0 = mass(volumes)
            volumes += n["volumes_m3"]
            added_water += n["volumes_m3"][2]
            actions.append({**n, "before_segment": i,
                        "lambda": (mass(volumes)-m0)/m0})
      muk = mu*(.6 if mode == "M2" and added_water > 0 else 1)
      segments.append({"i": i, "s0_m": float(split[i]), "s1_m": float(split[i+1]),
                  "z0_m": float(elev[i]), "z1_m": float(elev[i+1]),
                  "dh_m": float(dh[i]), "length_m": float(ll[i]),
                  "sin_alpha": float(dz[i]/ll[i]), "cos_alpha": float(dh[i]/ll[i]),
```

```
                    "mu": muk, "kappa_m_inv": kappa,
                    "a_m_s2": G*(dz[i]-muk*dh[i])/ll[i]})
    return {"segments": segments, "actions": actions, "nodes": stock,
            "initial_volumes_m3": iv, "target_sh_m": target, "mode": mode}


def simulate(s_h, z, initial_volumes, *, nodes=(), v0=30.0, mu=.1,
             kappa=1e-5, mode="M1", target_sh=None, subdivisions=1,
             time_limit_s=None, vreq=1.0) -> dict:
    if not math.isfinite(v0) or v0 <= 0:
        raise ValueError("positive conditional entrance velocity required; no restart model")
    if time_limit_s is not None and time_limit_s <= 0:
        raise ValueError("positive time limit required")
    plan = prepare(s_h, z, initial_volumes, nodes, target_sh, subdivisions, mu, kappa, mode)
    volumes = plan["initial_volumes_m3"].copy()
    initial_m = mass(volumes)
    initial_inventory = sum(mass(n["volumes_m3"]) for n in plan["nodes"])
    initial_energy = initial_m*(.5*v0*v0+G*float(z[0]))
    w, t, ell = v0*v0, 0.0, 0.0
    loss_c, loss_d, loss_mix, stock_pe, added_mass = 0., 0., 0., 0., 0.
    max_energy, max_segment, max_node_p, max_node_e = 0., 0., 0., 0.
    trajectory, ledger, segments = [], [], []
    consumed = set()

    def point(sh, elev, event):
        m = mass(volumes)
        total = initial_energy+stock_pe-loss_c-loss_d-loss_mix
        residual = total-m*(.5*w+G*elev)
        return {"s_h_m": sh, "ell_m": ell, "z_m": elev, "time_s": t,
                "velocity_m_s": math.sqrt(max(w, 0)), "mass_kg": m,
                "rock_m3": float(volumes[0]), "ice_m3": float(volumes[1]),
                "water_m3": float(volumes[2]), "rock_volume_fraction": float(volumes[0]/volumes.sum()),
                "energy_residual_J": residual, "event": event}

    trajectory.append(point(0., float(z[0]), "injection"))
    status = "positive_target_passage"
    for seg in plan["segments"]:
        for action in plan["actions"]:
            if action["before_segment"] != seg["i"]:
                continue
            if action["id"] in consumed:
                raise AssertionError("duplicate inventory consumption")
            m_before = mass(volumes)
            result = stationary_mix(volumes, math.sqrt(w), action["volumes_m3"])
            volumes = result["volumes_after_m3"]
            w = result["velocity_after_m_s"]**2
            dm = result["added_mass_kg"]
            stock_pe += dm*G*seg["z0_m"]
            added_mass += dm
            loss_mix += result["mix_loss_J"]
            consumed.add(action["id"])
            max_node_p = max(max_node_p, abs(result["momentum_residual_kg_m_s"])/max(m_before*v0, 1))
            max_node_e = max(max_node_e, abs(result["kinetic_residual_J"])/max(.5*m_before*v0*v0, 1))
            ledger.append({"id": action["id"], "s_h_m": action["s_h_m"],
                           "consumed_volumes_m3": action["volumes_m3"].tolist(),
                           "remaining_volumes_m3": [0., 0., 0.],
                           **{k: v for k, v in result.items() if k != "volumes_after_m3"},
                           "stock_potential_transferred_J": dm*G*seg["z0_m"]})
            trajectory.append(point(seg["s0_m"], seg["z0_m"], "stationary_mixing"))
        oldw = w
        out = segment(w, seg["a_m_s2"], seg["kappa_m_inv"], seg["length_m"])
        partial_timeout = time_limit_s is not None and t+out["time_s"] > time_limit_s
        if partial_timeout:
            remaining = time_limit_s-t
            distance = brentq(lambda x: travel_time(oldw, seg["a_m_s2"], kappa, x)-remaining,
                              0, out["distance_m"], xtol=1e-10)
            out = segment(oldw, seg["a_m_s2"], kappa, distance)
        frac = out["distance_m"]/seg["length_m"]
        dh_run = seg["dh_m"]*frac
        znow = seg["z0_m"]+(seg["z1_m"]-seg["z0_m"])*frac
        shnow = seg["s0_m"]+dh_run
        m = mass(volumes)
        wc = seg["mu"]*m*G*dh_run
        wd = m*kappa*out["integral_w_dell_m3_s2"]
        wg = m*G*(seg["z0_m"]-znow)
        residual = .5*m*(out["w_out_m2_s2"]-oldw)-(wg-wc-wd)
        max_segment = max(max_segment, abs(residual)/max(abs(wg)+wc+wd+.5*m*oldw, 1))
        loss_c += wc
        loss_d += wd
        w, t, ell = out["w_out_m2_s2"], t+out["time_s"], ell+out["distance_m"]
        if partial_timeout:
            status = "time_limit_unknown"
        elif out["stopped"]:
            at_target = abs(shnow-plan["target_sh_m"]) <= 1e-9
            status = "zero_speed_target_contact" if at_target else "first_stop_before_target"
        p = point(shnow, znow, status if partial_timeout or out["stopped"] else "segment_end")
        trajectory.append(p)
        max_energy = max(max_energy, abs(p["energy_residual_J"])/max(abs(initial_energy)+abs(stock_pe), 1))
        segments.append({**seg, **out, "gravity_work_J": wg, "coulomb_loss_J": wc,
                         "drag_loss_J": wd, "energy_residual_J": residual})
        if partial_timeout or out["stopped"]:
            break
    reached = status == "positive_target_passage"
    final = trajectory[-1]
    remaining_inventory = initial_inventory-added_mass
    inventory_mass_residual = mass(volumes)+remaining_inventory-initial_m-initial_inventory
    return {"status": status, "reached_target_positive": reached,
```

```
        "target_passage_class": ("meets_vreq" if reached and math.sqrt(w) >= vreq else
                                 "positive_below_vreq" if reached else status),
        "arrival_s": t if reached else None,
        "stop_time_s": t if status in ("first_stop_before_target", "zero_speed_target_contact") else None,
        "final_sh_m": final["s_h_m"], "final_ell_m": ell, "final_time_s": t,
        "final_velocity_m_s": math.sqrt(max(w, 0)),
        "initial_mass_kg": initial_m, "final_mass_kg": mass(volumes),
        "declared_inventory_mass_kg": initial_inventory, "consumed_inventory_mass_kg": added_mass,
        "remaining_inventory_mass_kg": remaining_inventory,
        "mass_residual_kg": inventory_mass_residual,
        "initial_mechanical_energy_J": initial_energy,
        "stock_potential_transferred_J": stock_pe, "coulomb_loss_J": loss_c,
        "drag_loss_J": loss_d, "mix_loss_J": loss_mix,
        "final_mechanical_energy_J": mass(volumes)*(.5*w+G*final["z_m"]),
        "energy_relative_residual_max": max_energy,
        "segment_energy_relative_residual_max": max_segment,
        "node_momentum_relative_residual_max": max_node_p,
        "node_energy_relative_residual_max": max_node_e,
        "trajectory": trajectory, "node_ledger": ledger, "segment_ledger": segments,
        "unvisited_node_ids": [n["id"] for n in plan["nodes"] if n["id"] not in consumed],
        "parameters": {"mu_dyn": mu, "kappa_m_inv": kappa, "v0_m_s": v0,
                       "mode": mode, "vreq_m_s": vreq, "subdivisions": subdivisions},
        "clock": "common_injection_to_port_proxy", "restart": "disabled_conditional"}


def inverse_threshold(s_h, z, initial_volumes, *, nodes=(), mu=.1, kappa=1e-5,
                      mode="M1", target_sh=None, vreq=1., subdivisions=1) -> dict:
   if vreq <= 0:
      raise ValueError("positive target convention required")
   plan = prepare(s_h, z, initial_volumes, nodes, target_sh, subdivisions, mu, kappa, mode)
   r = vreq*vreq
   records = []
   for seg in reversed(plan["segments"]):
      u = 2*kappa*seg["length_m"]
      if u > 700:
         return {"status": "required_speed_overflow_outside_engineering_domain",
                 "required_v0_m_s": None, "records": records}
      phi, _ = _phi_psi(u)
      A = math.exp(-u)
      B = 2*seg["a_m_s2"]*seg["length_m"]*phi
      after = r
      r = max(0., (r-B)/A)
      for node in reversed(plan["actions"]):
         if node["before_segment"] == seg["i"]:
            r *= (1+node["lambda"])**2
      records.append({"s_h_m": seg["s0_m"], "required_pre_w_m2_s2": r,
                      "required_out_w_m2_s2": after})
   # Track which intermediate positivity or final-speed constraint controls w0.
   AA, BB, max_required, bottleneck = 1., 0., 0., 0.
   for seg in plan["segments"]:
      for node in plan["actions"]:
         if node["before_segment"] == seg["i"]:
            factor = (1+node["lambda"])**2
            AA, BB = AA/factor, BB/factor
      u = 2*kappa*seg["length_m"]
      phi, _ = _phi_psi(u)
      A = math.exp(-u)
      AA, BB = A*AA, A*BB+2*seg["a_m_s2"]*seg["length_m"]*phi
      need = max(0., -BB/AA)
      if need > max_required:
         max_required, bottleneck = need, seg["s1_m"]
   terminal_requirement = max(0., (vreq*vreq-BB)/AA)
   if terminal_requirement >= max_required:
      bottleneck = plan["target_sh_m"]
   return {"status": "conditional_threshold_requires_forward_check",
         "required_v0_m_s": math.sqrt(r), "required_w0_m2_s2": r,
         "threshold_is_infimum": True, "target_vreq_m_s": vreq,
         "controlling_boundary_sh_m": bottleneck,
         "intermediate_positive_w0_infimum": max_required,
         "terminal_vreq_w0_requirement": terminal_requirement,
         "records": list(reversed(records))}
```

`mmlib/geodesy.py`　坐标与距离换算

```
"""坐标转换：UTM 45N ↔ WGS84 经纬度（精确横轴墨卡托反算）。"""
from __future__ import annotations

import math

A = 6378137.0
F = 1 / 298.257223563
K0 = 0.9996
E2 = F * (2 - F)
EP2 = E2 / (1 - E2)


def _meridian_arc(phi: float) -> float:
   e4, e6 = E2 * E2, E2 * E2 * E2
   return A * ((1 - E2 / 4 - 3 * e4 / 64 - 5 * e6 / 256) * phi
             - (3 * E2 / 8 + 3 * e4 / 32 + 45 * e6 / 1024) * math.sin(2 * phi)
             + (15 * e4 / 256 + 45 * e6 / 1024) * math.sin(4 * phi)
             - (35 * e6 / 3072) * math.sin(6 * phi))


def utm_to_lonlat(easting: float, northing: float, zone: int = 45, south: bool = False):
   x = easting - 500000.0
```

```
    y = northing - (10000000.0 if south else 0.0)
    mu = y / K0 / (A * (1 - E2 / 4 - 3 * E2 ** 2 / 64 - 5 * E2 ** 3 / 256))
    e1 = (1 - math.sqrt(1 - E2)) / (1 + math.sqrt(1 - E2))
    phi1 = (mu + (3 * e1 / 2 - 27 * e1 ** 3 / 32) * math.sin(2 * mu)
           + (21 * e1 ** 2 / 16 - 55 * e1 ** 4 / 32) * math.sin(4 * mu)
           + (151 * e1 ** 3 / 96) * math.sin(6 * mu)
           + (1097 * e1 ** 4 / 512) * math.sin(8 * mu))
    sin1, cos1, tan1 = math.sin(phi1), math.cos(phi1), math.tan(phi1)
    c1 = EP2 * cos1 ** 2
    t1 = tan1 ** 2
    n1 = A / math.sqrt(1 - E2 * sin1 ** 2)
    r1 = A * (1 - E2) / (1 - E2 * sin1 ** 2) ** 1.5
    d = x / (n1 * K0)
    lat = phi1 - (n1 * tan1 / r1) * (
       d ** 2 / 2 - (5 + 3 * t1 + 10 * c1 - 4 * c1 ** 2 - 9 * EP2) * d ** 4 / 24
       + (61 + 90 * t1 + 298 * c1 + 45 * t1 ** 2 - 252 * EP2 - 3 * c1 ** 2) * d ** 6 / 720)
    lon = (d - (1 + 2 * t1 + c1) * d ** 3 / 6
          + (5 - 2 * c1 + 28 * t1 - 3 * c1 ** 2 + 8 * EP2 + 24 * t1 ** 2) * d ** 5 / 120) / cos1
    return math.degrees(lon) + (zone * 6 - 183), math.degrees(lat)


def lonlat_to_utm(lon: float, lat: float, zone: int = 45):
    phi, lam = math.radians(lat), math.radians(lon)
    lam0 = math.radians(zone * 6 - 183)
    n = A / math.sqrt(1 - E2 * math.sin(phi) ** 2)
    t = math.tan(phi) ** 2
    c = EP2 * math.cos(phi) ** 2
    aa = (lam - lam0) * math.cos(phi)
    m = _meridian_arc(phi)
    east = K0 * n * (aa + (1 - t + c) * aa ** 3 / 6
                     + (5 - 18 * t + t ** 2 + 72 * c - 58 * EP2) * aa ** 5 / 120) + 500000.0
    north = K0 * (m + n * math.tan(phi) * (
       aa ** 2 / 2 + (5 - t + 9 * c + 4 * c ** 2) * aa ** 4 / 24
       + (61 - 58 * t + t ** 2 + 600 * c - 330 * EP2) * aa ** 6 / 720))
    return east, north
```

`mmlib/glacier_masks.py`　冰川编目掩膜与面积交比

```
"""Rasterize supplied RGI6 boundaries on DEM pixel centres, retaining holes."""
import hashlib
import numpy as np
from matplotlib.path import Path as PolygonPath
from . import io_data as io

def rgi6_mask(grid, shape):
    source = io.GLACIER_DHDT_DIR / 'RGI_V6_JilongFlood_Buffer_200km.shp'
    if not source.exists():
        raise FileNotFoundError(f'RGI6 supplied boundary required; run prepare_inputs.py: {source}')
    prj = source.with_suffix('.prj')
    wkt = prj.read_text(encoding='utf-8-sig')
    if 'GEOGCS' not in wkt.upper() or 'PROJCS' in wkt.upper():
        raise ValueError('RGI6 must use a geographic CRS; no silent projected-coordinate interpretation')
    digest = hashlib.sha256(source.read_bytes()).hexdigest()
    mask_path, meta_path = io.CACHE/'rgi6_dem_mask.npy', io.CACHE/'rgi6_dem_mask.json'
    import json
    signature = {'boundary_sha256': digest, 'shape': list(shape), 'grid': grid, 'version': 'rgi6-centre-xor-v1'}
    if mask_path.exists() and meta_path.exists():
        old = json.loads(meta_path.read_text(encoding='utf-8'))
        if old.get('signature') == signature:
            return np.load(mask_path), old
    mask = np.zeros(shape, dtype=bool)
    nx, ny = shape[1], shape[0]
    xmin, xmax = sorted([grid['lon0'], grid['lon0']+(nx-1)*grid['dlon']])
    ymin, ymax = sorted([grid['lat0'], grid['lat0']+(ny-1)*grid['dlat']])
    used = 0
    for feature in io.read_shp_polygons(source):
        bx0, by0, bx1, by1 = feature['bbox']
        if bx1 < xmin or bx0 > xmax or by1 < ymin or by0 > ymax: continue
        rf = [(by0-grid['lat0'])/grid['dlat'], (by1-grid['lat0'])/grid['dlat']]
        cf = [(bx0-grid['lon0'])/grid['dlon'], (bx1-grid['lon0'])/grid['dlon']]
        r0,r1=max(0,int(np.floor(min(rf)))-1),min(ny,int(np.ceil(max(rf)))+2)
        c0,c1=max(0,int(np.floor(min(cf)))-1),min(nx,int(np.ceil(max(cf)))+2)
        rr,cc=np.mgrid[r0:r1,c0:c1]
        points=np.column_stack([(grid['lon0']+cc*grid['dlon']).ravel(),(grid['lat0']+rr*grid['dlat']).ravel()])
        inside=np.zeros(len(points),dtype=bool)
        ends=feature['parts']+[len(feature['pts'])]
        for start,end in zip(ends[:-1],ends[1:]):
            ring=feature['pts'][start:end]
            if len(ring)>=3: inside ^= PolygonPath(ring).contains_points(points)
        mask[r0:r1,c0:c1] |= inside.reshape(rr.shape)
        used+=1
    if not mask.any(): raise ValueError('No supplied RGI6 polygons intersect the DEM; do not substitute elevation')
    meta={'signature':signature,'source':str(source.relative_to(io.ROOT)), 'crs_wkt':wkt,
          'features_intersecting_dem':used, 'pixels':int(mask.sum()),
          'scope':'RGI6 inventory epoch, not observed 2026 glacier/snow extent; centre rasterization with ring parity'
          }
    np.save(mask_path,mask); io.write_json(meta_path,meta)
    return mask,meta
```

`mmlib/hydro.py`　水文过程与 SCS-CN 径流

```
"""一维准恒定断面和经验能量线诊断；不解守恒输运。"""
from __future__ import annotations

import numpy as np
```

```python

G = 9.80665


def _slope_profile(s: np.ndarray, z: np.ndarray, win: int) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """河段尺度比降。

    D8 沿程采样会给出"台阶—平台"式纵剖面，平台段瞬时坡度为 0 并不代表
    河段坡度为 0。故对纵剖面先做窗口平滑，再取比降，并设下限 slope_min
    以避免下游平原段除零。
    """
    ds = float(np.median(np.diff(s)))
    if win > 1 and len(z) > win:
        k = np.ones(win) / win
        zp = np.pad(z, (win // 2, win - 1 - win // 2), mode="edge")
        zs = np.convolve(zp, k, mode="valid")[: len(z)]
    else:
        zs = z
    slope = np.clip(-np.diff(zs, prepend=zs[0]) / ds, 1e-4, 1.5)
    sin_t = slope / np.sqrt(1.0 + slope ** 2)
    cos_t = 1.0 / np.sqrt(1.0 + slope ** 2)
    return slope, sin_t, cos_t


def solve_flow(s_m: np.ndarray, z_m: np.ndarray, p: dict) -> dict:
    """准恒定 Manning 流 + Fahrböschung 停止判据。

    Parameters
    ----------
    s_m, z_m : 中心线沿程水平里程与高程（m）
    p : 参数字典
        V0_m3    初始释放体积 (m³)
        B0_m     沟道初始宽度 (m)
        n_man    泥石流曼宁糙率 (s·m^{-1/3})
        mu       等效摩擦系数 (= tan 等效摩擦角)
        T_pass_s 过流历时 (s)，用于由体积折算特征流量
        slope_window 平滑窗口（节点数）
    """
    s = np.asarray(s_m, dtype="float64")
    z = np.asarray(z_m, dtype="float64")
    n = len(s)
    ds = float(np.median(np.diff(s)))

    V0 = float(p["V0_m3"]); B0 = float(p["B0_m"])
    nman = float(p["n_man"]); mu = float(p["mu"])
    T_pass = float(p.get("T_pass_s", 900.0))
    slope, sin_t, cos_t = _slope_profile(s, z, int(p.get("slope_window", 7)))

    # Common injection section is the zero of both propagation clocks.
    i0 = int(p.get("start_index", 0))
    if not 0 <= i0 < n - 1:
        raise ValueError("invalid common injection index")
    E0 = float(p.get("E0_m2s2", 0.0))

    # 沟宽沿程放大（下游汇流增大）
    B = B0 * (1.0 + float(p.get("beta_B", 0.8)) * s / max(s[-1], 1.0))
    # 特征流量：V0 在 T_pass 内通过任一断面
    Q_pk = V0 / max(T_pass, 1.0)

    # 断面流深与流速（Manning）。陡坎处 √S 极大、水深会被压到亚米级，
    # 平原段 √S→0 又会使水深发散；这两种情形都不代表真实断面，
    # 故对坡度取工程下限 slope_floor（默认 0.01，约 0.6°）并对水深设上限。
    s_eff = np.maximum(slope, float(p.get("slope_floor", 0.01)))
    h = (nman * Q_pk / (B * np.sqrt(s_eff))) ** 0.6
    h = np.minimum(h, float(p.get("h_cap", 15.0)))
    v = Q_pk / (B * h)
    v = np.minimum(v, float(p.get("v_cap", 30.0)))
    continuity = (B * h * v - Q_pk) / max(Q_pk, 1e-12)
    F = v / np.sqrt(G * h)

    # 停止位置（Fahrböschung / Heim 原理）：
    #   E(x) = E0 + 2g[ (z_i0 − z(x)) − μ·(s(x) − s_i0) ]
    # 该式几何上表示自释放点按倾角 arctan(μ) 作直线，与纵剖面的交点即
    # 堆积前锋的理论停止位置（Heim 1932；Rickenmann 2005）。E0 为冰崩
    # 在起始点注入的比能，使平坦的粒雪盆段不至于被误判为终点。
    # 停止判据需连续 sustain 个节点跌破 0，以容忍局部小凸起（等高线台阶）。
    sustain = int(p.get("sustain_nodes", 5))
    E = E0 + 2.0 * G * ((z[i0] - z) - mu * (s - s[i0]))
    E[:i0] = -np.inf
    E[i0] = max(E0, 0.0) + 1e-6
    below = E <= 0.0
    stop_idx = n - 1
    for i in range(i0 + 1, n):
        if i + sustain <= n and np.all(below[i:i + sustain]):
            stop_idx = i
            break
    if stop_idx <= i0 and i0 < n - 1:
        stop_idx = min(i0 + 2, n - 1)

    wet = (np.arange(n) >= i0) & (np.arange(n) <= stop_idx)
    h = np.where(wet, h, 0.0)
    v = np.where(wet, v, 0.0)

```

```
    # 到达时刻（梯形积分，起点之前不计时）
    t = np.full(n, np.nan)
    t[i0] = 0.0
    for i in range(i0 + 1, stop_idx + 1):
        t[i] = t[i - 1] + (s[i] - s[i-1]) / max(0.5 * (v[i - 1] + v[i]), 1e-9)

    # 几何强度 h·v²（m²·s⁻²）。强度指标 I = ρ_m h v² 在 step04_simulation
    # 中由 C_v 换算，这样密度参数有唯一的进入点，不会两处各算一套。
    inten = h * v ** 2
    # 体积诊断（修正版）。Q_pk = V0/T_pass 是定义式，用它反算 V0 必然恒等
    # （相对误差 0），不构成检验。真正有信息量的量是：
    #  沟槽储水体积 S(x) = ∫_{i0}^{x} B·h ds
    # 即按 Manning 流深把整条过流段同时充满所需的物质体积。
    #
    # 需要注意其正确解读：Heim 停止判据 E=V0 无关，只含 μ 与地形，故
    # **停止位置恒由 μ 控制**，与 V0 无关。V0 决定的是"同时有多少沟道在
    # 过流"，即波列长度 L_surge = V0 / mean(B·h)。S/V0 因此是"波列占运移
    # 路程的比例的倒数"，用于判断这是脉冲型（surge）还是满槽型流动，而
    # 不是判定物质是否充足的判据。
    dseg = np.zeros(n)
    dseg[i0:] = ds
    S_cum = np.cumsum(B * h * dseg)
    S_stop = float(S_cum[stop_idx]) if stop_idx >= i0 else 0.0
    bh = np.where(wet, B * h, 0.0)
    mean_bh = float(np.mean(bh[wet])) if np.any(wet) else 0.0
    L_surge_m = V0 / mean_bh if mean_bh > 0 else 0.0
    run_m = float(s[stop_idx] - s[i0]) if stop_idx >= i0 else 0.0
    return {
        "s_km": s / 1000.0, "z_m": z, "slope": slope, "B_m": B,
        "h_m": h, "v_m_s": v, "Q_m3s": np.where(wet, Q_pk, 0.0),
        "Fr": F, "E": E, "t_s": t, "intensity": inten,
        # 波列长度与占运移路程的比例
        "波列长度_m": float(L_surge_m),
        "波列占路程比": float(min(L_surge_m / run_m, 1.0)) if run_m > 0 else np.nan,
        "液型": ("脉冲型（surge）" if L_surge_m < 0.6 * run_m else "近满槽型"),
        # 满槽储水体积（诊断量，非守恒校验）
        "沟槽储水体积_m3": S_stop,
        "物质充裕度": float(S_stop / V0) if V0 > 0 else np.nan,
        "运移控制因素": "停止位置由 μ 控制；V0 决定波列长度",
        "定义恒等式_非守恒检验": float(abs(V0 - Q_pk * T_pass) / V0),
        "stop_idx": stop_idx,
        "runout_km": float(s[stop_idx] / 1000.0),
        "total_km": float(s[-1] / 1000.0),
        "reached_outlet": bool(stop_idx >= n - 1),
        "arrival_outlet_s": (float(t[-1]) if stop_idx == n - 1
                             and np.isfinite(t[-1]) else None),
        "stop_time_s": float(t[stop_idx]) if np.isfinite(t[stop_idx]) else None,
        "termination_reason": ("right_censored_at_port" if stop_idx == n - 1
                               else "natural_energy_stop"),
        "runout_right_censored": bool(stop_idx == n - 1),
        "start_index": i0,
        "common_clock": "common_injection_section",
        "continuity_relative_residual_max": float(np.max(np.abs(continuity[wet]))),
        "slope_floor_nodes": int(np.sum((slope < p.get("slope_floor", 0.01)) & wet)),
        "params": dict(p),
        "v_max_m_s": float(np.max(v)), "v_median_m_s": float(np.median(v[wet])),
        "h_max_m": float(np.max(h)), "h_median_m": float(np.median(h[wet])),
        "I_max": float(np.max(inten)),
        "Fr_median": float(np.median(F[wet])),
        "drop_m": float(z[0] - z[-1]),
        "stopped_drop_m": float(z[0] - z[stop_idx]),
        "mu_crit": float((z[0] - z[-1]) / s[-1]),
        "mu": mu, "n_man": nman, "V0": V0, "Q_pk": Q_pk,
        "等效摩擦角_deg": float(np.degrees(np.arctan(mu))),
    }


def calibrate_mu(s_m, z_m, target_km: float, params: dict,
                 lo: float = 0.02, hi: float = 0.5) -> dict:
    """二分反演使运动距离等于目标值的 μ（L_max ≈ Δz/μ 单调，收敛稳定）。"""
    def runout(m):
        q = dict(params); q["mu"] = float(m)
        return solve_flow(s_m, z_m, q)["runout_km"]

    a, b = lo, hi
    for _ in range(70):
        mid = 0.5 * (a + b)
        if runout(mid) > target_km:
            a = mid
        else:
            b = mid
        if abs(b - a) < 1e-6:
            break
    mu = 0.5 * (a + b)
    q = dict(params); q["mu"] = mu
    r = solve_flow(s_m, z_m, q)
    return {"mu": float(mu), "runout_km": r["runout_km"], "目标_km": float(target_km),
            "残差_km": float(r["runout_km"] - target_km),
            "等效摩擦角_deg": r["等效摩擦角_deg"]}


def runout_curve(s_m, z_m, mus, params: dict) -> dict:
```

```
    rows = []
    for m in mus:
        q = dict(params); q["mu"] = float(m)
        r = solve_flow(s_m, z_m, q)
        rows.append({"mu": float(m), "等效摩擦角_deg": r["等效摩擦角_deg"],
                     "runout_km": r["runout_km"], "reached_outlet": r["reached_outlet"],
                     "arrival_outlet_s": r["arrival_outlet_s"],
                     "v_max_m_s": r["v_max_m_s"], "I_max": r["I_max"]})
    return {"曲线": rows}


def lateral_spread(cl: dict, reach_km: float, half_angle_deg: float, I0: float,
                   dem: np.ndarray, grid: dict) -> dict:
    """出山口以下的横向扩散影响区（扇形 + 沿程指数衰减）。"""
    h, w = dem.shape
    cell_x = abs(grid["dlon"]) * 111320.0 * np.cos(np.radians(28.4))
    cell_y = abs(grid["dlat"]) * 110574.0
    apex = cl["end_rc"]
    k = max(2, len(cl["s_m"]) // 20)
    ang = float(np.arctan2(cl["y_m"][-1] - cl["y_m"][-k], cl["x_m"][-1] - cl["x_m"][-k]))
    rmax = reach_km * 1000.0
    phi = np.radians(half_angle_deg)

    rr, cc = np.mgrid[0:h, 0:w]
    px = (cc - apex[1]) * cell_x
    py = (rr - apex[0]) * cell_y
    r = np.hypot(px, py)
    theta = np.arctan2(py, px)
    dtheta = np.abs((theta - ang + np.pi) % (2 * np.pi) - np.pi)
    inside = (r <= rmax) & (dtheta <= phi) & (r > 0)
    inten = np.zeros((h, w), dtype="float32")
    inten[inside] = (I0 * np.exp(-r[inside] / max(rmax, 1.0)) *
                     np.cos(dtheta[inside] / max(phi, 1e-6) * np.pi / 2)).astype("float32")
    cell_area = cell_x * cell_y
    return {"intensity": inten, "affected_cells": int(inside.sum()),
            "affected_area_km2": float(inside.sum() * cell_area / 1e6),
            "apex_rc": [int(apex[0]), int(apex[1])],
            "main_direction_deg": float(np.degrees(ang)),
            "half_angle_deg": float(half_angle_deg), "reach_km": float(reach_km),
            "I0": float(I0), "max_intensity": float(np.max(inten))}


def intensity_classes(inten, mask, cell_area: float, edges=None) -> dict:
    """按单位宽度冲击力 I = ρ_m·h·v² 分级（默认阈值 10 / 100 / 1000 kN·m⁻¹）。

    泥石流的破坏力由流深、流速与密度共同决定。指标取单位宽度冲击力：
        I = ρ_m·h·v²，ρ_m = C_v·ρ_s + (1−C_v)·ρ_w
    量纲 kg·m⁻³ × m × m²·s⁻² = N/m（不是压强 Pa，二者相差一个流深因子）。
    乘上 ρ_m 是关键：若写成 h·v²（m²·s⁻²），含沙量 15% 与 80% 的流动会给出
    相同破坏力，与题面把固相含量作为定义性特征的设定冲突。

    默认分级界 10 / 100 / 1000 kN·m⁻¹（ρ_m≈1742 kg·m⁻³）：
      10 kN/m ≈ h=0.5 m、v=3.4 m/s —— 可推动沟床松散物质；
      100 kN/m ≈ h=1.5 m、v=6.2 m/s —— 可破坏浆砌石护岸；
      1000 kN/m ≈ h=3.0 m、v=13.8 m/s —— 可摧毁钢筋混凝土结构。
    edges 若给出则按 edges 分级（N/m），供调用方与参数表保持同一口径。
    """
    edges = tuple(edges) if edges is not None else (0.0, 1.0e4, 1.0e5, 1.0e6, np.inf)
    names = ("低", "中", "高", "极高")
    out = {}
    for i, name in enumerate(names):
        lo, hi = edges[i], edges[i + 1]
        m = (inten >= lo) & (inten < hi) & mask
        out[name] = {"像元数": int(m.sum()), "面积_km2": float(m.sum() * cell_area / 1e6)}
    out["公式"] = "I = ρ_m·h·v²（单位宽度冲击力，N·m⁻¹）"
    out["分级阈值_N_m"] = [float(v) for v in edges[:-1]] + ["inf"]
    out["最大强度_N_m"] = float(np.nanmax(inten[mask])) if mask.any() else 0.0
    return out
```

mmlib/interval_metrics.py　区间与覆盖指标

```
"""Interval diagnostics with explicit numerical and label-resolution semantics."""
import numpy as np

TOLERANCE_H = 1e-9

def canonical_endpoint(value):
    a = np.asarray(value, dtype=float)
    nearest = np.rint(a)
    return np.where(np.abs(a-nearest) <= TOLERANCE_H, nearest, a)

def covered(y, lower, upper):
    y, lower, upper = map(lambda a: np.asarray(a, dtype=float), (y, lower, upper))
    return (y >= lower-TOLERANCE_H) & (y <= upper+TOLERANCE_H)

def interval_score(y, lower, upper, alpha=.05):
    y, lower, upper = map(lambda a: np.asarray(a, dtype=float), (y, lower, upper))
    left = np.maximum(lower-y-TOLERANCE_H, 0)
    right = np.maximum(y-upper-TOLERANCE_H, 0)
    return upper-lower + 2/alpha*(left+right)
```

mmlib/io_data.py　数据读写与校验记录

```python
"""吉隆泥石流建模：公共数据读写与预处理模块。

所有路径均以项目根目录为基准，可通过环境变量 JILONG_ROOT 覆盖。
原始附件解压目录默认为 <root>/临时产物/原始数据。
"""
from __future__ import annotations

import json
import os
import struct
import zlib
from pathlib import Path

import numpy as np
import pandas as pd

# ---------------------------------------------------------------- 路径

def project_root() -> Path:
    """定位工程根目录（含 求解结果/ 与 临时产物/ 的目录）。

    优先取环境变量 JILONG_ROOT；否则自本文件向上逐级搜索，命中第一个同时含有
    `求解结果` 与 `临时产物` 的目录。**不能用固定的 parents[N]**：源码位于
    `支撑材料/源码/mmlib/`，parents[2] 是 `支撑材料` 而不是工程根，早期实现因此
    在未设置环境变量的全新 shell 中直接找不到原始数据（`python run_all.py`
    这条可复现命令实际不可用）。向上搜索使"在哪一层被拷入"都不影响定位。
    """
    env = os.environ.get("JILONG_ROOT")
    if env:
        return Path(env).resolve()
    here = Path(__file__).resolve()
    for cand in here.parents:
        if (cand / "求解结果").is_dir() and (cand / "临时产物").is_dir():
            return cand
    # 兜底：若目录结构尚未建立，退回三层（源码被放在 <root>/源码/ 的旧布局）
    return here.parents[3]


ROOT = project_root()
RAW = ROOT / "临时产物" / "原始数据"
RESULT = ROOT / "求解结果"
DATA_OUT = RESULT / "数据"
FIG_OUT = RESULT / "图表"
CACHE = DATA_OUT / "cache"

for _d in (DATA_OUT, FIG_OUT, CACHE):
    _d.mkdir(parents=True, exist_ok=True)

STATION_DIR = RAW / "（1）西藏吉隆高寒山区地气相互作用逐小时综合观测数据集（2021–2026）"
TEMPERATURE_DIR = RAW / "（2）2005-2025年吉隆县及周边县市逐日平均气温数据" / "2005-2025年吉隆县周边县市逐日平均气温数据"
GLACIER_DHDT_DIR = RAW / "（3）冰川高程变化" / "西藏日喀则市吉隆县特大洪水—200km周围内的冰川和冰川厚度变化数据集"
GLACIER_NC = RAW / "（4）吉隆口岸山洪灾害-蓝塘里壤峰冰崩前冰川地形、冰厚及表面速度数据集" / (
    "Glacier topographic and dynamic drivers (Glacier-collapse and debris-flow disasters at Langtang Lirung, Nepal).nc")
LANDCOVER_DIR = RAW / "（5）西藏日喀则市吉隆县泥石流灾害区土地覆盖数据（2020-2023）"
HAZARD_DIR = RAW / "（6）西藏日喀则市吉隆县泥石流灾害区历史灾点数据" / "西藏日喀则市吉隆县泥石流灾害区历史灾点数据"
DEM_DIR = RAW / "（7）2026年8月26日日喀则市吉隆县泥石流灾害区DEM数据"
CENSUS_DIR = RAW / "（8）西藏日喀则市吉隆县泥石流灾害区吉隆县第七次人口普查数据" / "西藏日喀则市吉隆县第七次人口普查数据"

# 站点元信息（坐标与海拔均为假定/推断值，非实测安装元数据，见 问题与数据说明.md）
# elev_m 取气压反演值 3978.85 m；触发计算中的 3980.0 是同一推断值的取整工作常数，
# 两者差 1.15 m，远小于反演不确定度，不构成两套口径。
# 晴空辐射模型（meteo.clear_sky_rsd）为 Haurwitz 形式，不含海拔项，故辐射结果不使用 3980.0；
# 该参数仅为接口保留，不参与计算。
STATION = {
    "name": "吉隆高寒山区综合观测站",
    "lon": 85.30,
    "lat": 28.40,
    "elev_m": 3978.85,
    "elev_m_deprecated": 3955.0,
    "elev_m_note": (
        "elev_m 取本站气压反演值 3978.85 m；elev_m_deprecated = 3955.0 为 v2 遗留配置值，"
        "旧版推断值仅留作审计痕迹，不作为站点海拔。elev_m 不参与任何下游计算。"
    ),
    "period": ("2021-04-23 11:00", "2026-08-27 08:00"),
    "index_first": "2021-01-01 00:00",
    "note": (
        "坐标为吉隆沟高寒山区假定代表点（原始 CSV 无站点元数据）；"
        "海拔为标准大气由本站气压反演值，非实测安装高度；"
        "逐时序列按 2021-01-01 00:00 起的完整小时轴重采样，故 rows 大于 period 跨度对应的小时数；"
        "反演细节见 求解结果/数据/00_站点元信息.json 的 elevation_inferred_m"
    ),
}

# 事件时刻（题面给定）与口岸位置（非题面给定，见 note）
EVENT = {
    "name": "2026-08-26 吉隆口岸冰崩-泥石流",
    "datetime_local": "2026-08-26 10:30",
    # 口岸工作坐标：研究区 DEM 图上量取的近似位置，题面未给坐标。
    # 公开地理坐标为 28.2797N/85.3777E，与本工作坐标相差约 571 m。
```

```
    # 该距离约合 19--21 个 DEM 像元（像元 27.2 x 30.7 m），并非小于像元尺度；
    # 它小于口岸集水区与模拟域尺度，故空间结果统一以此工作坐标为准，
    # 并在敏感性分析中把路径吸附偏移量级一并披露。
    "port_lon": 85.3826,
    "port_lat": 28.2826,
    "port_note": "量图近似位置，非题面给定、非实测；候选位置仍待现场核实；重演见求解结果/数据/06_坐标敏感性.json",
}

TAG = 9999.0 # 缺测标记
PREPROCESS_VERSION = "qc-center-v4"

# ---------------------------------------------------------------- TIFF

def _lzw_decode(data: bytes) -> bytes:
    table = [bytes([i]) for i in range(256)] + [b"", b""]
    out = bytearray()
    nbits, bitpos, total, prev = 9, 0, len(data) * 8, None
    while bitpos + nbits <= total:
        b = bitpos >> 3
        chunk = data[b:b + 4].ljust(4, b"\x00")
        shift = 32 - (bitpos & 7) - nbits
        if shift < 0:
            break
        code = (int.from_bytes(chunk, "big") >> shift) & ((1 << nbits) - 1)
        bitpos += nbits
        if code == 256:
            table, nbits, prev = table[:258], 9, None
            continue
        if code == 257:
            break
        if prev is None:
            entry = table[code]
            out += entry
            prev = entry
            continue
        if code < len(table):
            entry = table[code]
            table.append(prev + entry[:1])
        else:
            entry = prev + prev[:1]
            table.append(entry)
        out += entry
        prev = entry
        if len(table) == (1 << nbits) - 1 and nbits < 12:
            nbits += 1
    return bytes(out)


_DTYPE = {(3, 32): "f4", (3, 64): "f8", (1, 8): "u1", (1, 16): "u2",
          (1, 32): "u4", (2, 8): "i1", (2, 16): "i2", (2, 32): "i4"}


def read_geotiff(path: str | Path):
    """读取 GeoTIFF（支持 LZW/Deflate/无压缩，条带或瓦片），返回 (数组, 地理信息)。"""
    import tifffile
    path = str(path)
    with tifffile.TiffFile(path) as tf:
        page = tf.pages[0]
        tags = {t.code: t.value for t in page.tags.values()}
        width, height = int(tags[256]), int(tags[257])
        comp = int(tags.get(259, 1))
        spp = int(tags.get(277, 1))
        bits = tags.get(258, 32)
        bits = int(bits[0]) if isinstance(bits, (list, tuple)) else int(bits)
        sf = tags.get(339, 1)
        sf = int(sf[0]) if isinstance(sf, (list, tuple)) else int(sf)
        dtype = _DTYPE[(sf, bits)]
        bps = bits // 8
        fh = tf.filehandle
        tw = int(tags.get(322, 0) or 0)
        th = int(tags.get(323, 0) or 0)

        def unpack(blob: bytes) -> bytes:
            if comp == 1:
                return blob
            if comp == 5:
                return _lzw_decode(blob)
            if comp in (8, 32946):
                return zlib.decompress(blob)
            raise ValueError(f"不支持的压缩类型 {comp}")

        if tw and th:
            across = (width + tw - 1) // tw
            out = np.zeros(((height + th - 1) // th * th, across * tw), dtype=np.dtype(dtype))
            need = tw * th * spp * bps
            for idx, (off, cnt) in enumerate(zip(tags[324], tags[325])):
                fh.seek(off)
                blob = unpack(fh.read(cnt))
                if len(blob) < need:
                    blob += b"\x00" * (need - len(blob))
                arr = np.frombuffer(blob[:need], dtype=dtype).reshape(th, tw, spp) if spp > 1 else \
                    np.frombuffer(blob[:need], dtype=dtype).reshape(th, tw)
                r, c = divmod(idx, across)
                out[r * th:(r + 1) * th, c * tw:(c + 1) * tw] = arr
            arr = out[:height, :width]
        else:
```

```
        chunks = []
        for off, cnt in zip(tags[273], tags[279]):
            fh.seek(off)
            arr_c = np.frombuffer(unpack(fh.read(cnt)), dtype=dtype)
            chunks.append(arr_c)
        arr = np.concatenate(chunks)
        arr = arr.reshape(height, width, spp) if spp > 1 else arr[:height * width].reshape(height, width)
    geo = {}
    for key, code in (("pixel_scale", 33550), ("tiepoint", 33922), ("geokeys", 34735),
                      ("gcs_ascii", 34737), ("nodata", 42113")):
        if code in tags:
            geo[key] = tags[code]
    return np.asarray(arr), geo


def geotiff_grid(geo: dict) -> dict:
    """由 GeoTIFF 标签给出 (lon0, lat0, dlon, dlat) 栅格定位（左上角像元中心为锚点）。"""
    sx, sy = float(geo["pixel_scale"][0]), float(geo["pixel_scale"][1])
    tp = geo["tiepoint"]
    keys = list(geo.get("geokeys", []))
    raster_type = 1
    for i in range(4, len(keys) - 3, 4):
        if keys[i] == 1025:
            raster_type = int(keys[i + 3])
    half = 0.5 if raster_type == 1 else 0.0
    return {"lon0": float(tp[3]) + (half - float(tp[0])) * sx,
            "lat0": float(tp[4]) - (half - float(tp[1])) * sy,
            "dlon": sx, "dlat": -sy, "anchor": "pixel_center",
            "raster_type": raster_type, "preprocess_version": PREPROCESS_VERSION}


def lonlat_to_rowcol(grid: dict, lon, lat):
    return (np.asarray(lat) - grid["lat0"]) / grid["dlat"], (np.asarray(lon) - grid["lon0"]) / grid["dlon"]


# ---------------------------------------------------------------- 站点观测

_MET_FMT = "%Y-%m-%d %H:%M"
_FLUX_FMT = "%Y/%m/%d %H:%M"


def _load_family(tag: str, fmt: str) -> pd.DataFrame:
    files = sorted(STATION_DIR.glob(f"{tag}_Gyirong_*.csv"))
    if not files:
        raise FileNotFoundError(f"未找到 {tag} 观测文件：{STATION_DIR}")
    frames = []
    for f in files:
        df = pd.read_csv(f, encoding="utf-8")
        df["ts"] = pd.to_datetime(df["Timestamp"], format=fmt)
        frames.append(df.drop(columns=["Timestamp"]))
    data = pd.concat(frames, ignore_index=True).set_index("ts").sort_index()
    data = data[~data.index.duplicated(keep="last")]
    num = data.select_dtypes("number").columns
    data[num] = data[num].mask(data[num].abs() >= TAG)
    stems = {_clean_name(c).lower(): c for c in data if not c.startswith("QC_")}
    for qc in [c for c in data if c.startswith("QC_")]:
        target = stems.get(qc[3:].lower())
        if target:
            data[target] = data[target].where(data[qc].eq(0))
    return data


def _clean_name(c: str) -> str:
    return (c.replace("(degree C)", "").replace("(degree)", "").replace("(m/s)", "").replace("(%)", "")
            .replace("(hPa)", "").replace("(mm)", "").replace("(W/m2)", "").replace("(m3/m3)", "")
            .replace("(kPa)", "").replace("(g/m3)", "").replace("(mg/m2*s)", "").replace("(kg/m*s2)", "")
            .replace("(m2/s2)", "").replace("(m)", "").strip())


def load_station(use_cache: bool = True) -> pd.DataFrame:
    """合并 MET/RADM/SOIL/FLUX 四类逐小时观测，返回带统一列名的 DataFrame。"""
    cache = CACHE / "station_hourly.parquet"
    if use_cache and cache.exists():
        old = pd.read_parquet(cache)
        if old.attrs.get("preprocess_version") == PREPROCESS_VERSION:
            return old

    met = _load_family("MET", _MET_FMT)
    met = met.drop(columns=[c for c in met.columns if c.startswith("QC_")])
    met.columns = [_clean_name(c) for c in met.columns]

    radm = _load_family("RADM", _MET_FMT)
    radm = radm.drop(columns=[c for c in radm.columns if c.startswith("QC_")])
    radm.columns = [_clean_name(c) for c in radm.columns]

    soil = _load_family("SOIL", _MET_FMT)
    soil = soil.drop(columns=[c for c in soil.columns if c.startswith("QC_")])
    soil.columns = [_clean_name(c) for c in soil.columns]

    flux = _load_family("FLUX", _FLUX_FMT)
    flux.columns = [_clean_name(c) for c in flux.columns]

    df = met.join(radm, how="outer").join(soil, how="outer").join(flux, how="outer")
    df.index.name = "ts"
    df = df.loc["2026-08-27 08:00"]
    # 统一为浮点，剔除同一列名冲突（RADM 的 QC_rsu 等已在上游删除）
```

```
    df = df.loc[:, ~df.columns.duplicated()]
    for c in df.columns:
        df[c] = pd.to_numeric(df[c], errors="coerce")
    df = df.astype("float64")
    df = df.reindex(pd.date_range(df.index.min(), df.index.max(), freq="h", name="ts"))
    for c in df:
        if c.startswith("RH_"):
            df[c] = df[c].where(df[c].between(0, 100))
        elif c.startswith("WS_") or c == "Precip":
            df[c] = df[c].where(df[c] >= 0)
        elif c.startswith("Ta_"):
            df[c] = df[c].where(df[c].between(-60, 50))
    df.attrs["preprocess_version"] = PREPROCESS_VERSION
    df.to_parquet(cache)
    return df


def load_temperature(use_cache: bool = True) -> pd.DataFrame:
    """五个县 2005–2025 年逐日 2 m 平均气温。"""
    cache = CACHE / "temperature_daily.parquet"
    if use_cache and cache.exists():
        return pd.read_parquet(cache)
    frames = []
    for f in sorted(TEMPERATURE_DIR.glob("2米气温数据_*.xlsx")):
        df = pd.read_excel(f, sheet_name=0)
        df = df.rename(columns={"时间": "date"})
        df["date"] = pd.to_datetime(df["date"])
        frames.append(df)
    out = pd.concat(frames, ignore_index=True).drop_duplicates("date").set_index("date").sort_index()
    out.to_parquet(cache)
    return out


# ---------------------------------------------------------------------- 矢量

def _dbf_encoding(dbf_path: Path) -> str:
    cpg = dbf_path.with_suffix(".cpg")
    if cpg.exists():
        txt = cpg.read_text(encoding="ascii", errors="ignore").strip().lower()
        if "utf" in txt:
            return "utf-8"
        if "936" in txt or "gbk" in txt or "gb2312" in txt:
            return "gbk"
    raw = dbf_path.read_bytes()
    # 逐个字段名检验，不能只看文件头前 64 字节：首个字段常为 ASCII
    # （Name/flag/x_coord…），只看开头会把 GBK 表误判为 UTF-8，中文字段名
    # （河流/课题/沟比降…）在 utf-8+ignore 下塌缩为空串并在 dict 中互相
    # 覆盖——附件（6）正是这种情况（无 .cpg，首字段 ASCII，其余字段 GBK）。
    try:
        hlen = struct.unpack("<H", raw[8:10])[0]
        nf = (hlen - 33) // 32
        for i in range(nf):
            raw[32 + i * 32:32 + i * 32 + 11].split(b"\x00")[0].decode("utf-8")
        return "utf-8"
    except (UnicodeDecodeError, struct.error):
        return "gbk"


def read_dbf(path: str | Path) -> pd.DataFrame:
    path = Path(path)
    enc = _dbf_encoding(path)
    raw = path.read_bytes()
    nrec, hlen, rlen = struct.unpack("<IHH", raw[4:12])
    nf = (hlen - 33) // 32
    fields = []
    for i in range(nf):
        off = 32 + i * 32
        name = raw[off:off + 11].split(b"\x00")[0].decode(enc, "ignore")
        fields.append((name, chr(raw[off + 11]), raw[off + 16]))
    rows = []
    for r in range(nrec):
        base = hlen + r * rlen
        rec = raw[base:base + rlen]
        p, row = 1, {}
        for name, _typ, ln in fields:
            row[name] = rec[p:p + ln].decode(enc, "ignore").strip()
            p += ln
        rows.append(row)
    return pd.DataFrame(rows)


def read_shp_polygons(path: str | Path):
    """解析 polygon shapefile，返回每个要素的顶点数组与 bbox。"""
    raw = Path(path).read_bytes()
    i, out = 100, []
    while i < len(raw) - 8:
        _rec, = struct.unpack(">i", raw[i:i + 4])
        clen, = struct.unpack(">i", raw[i + 4:i + 8])
        stype, = struct.unpack("<i", raw[i + 8:i + 12])
        if stype in (5, 15):
            bbox = struct.unpack("<4d", raw[i + 12:i + 44])
            nparts, npoints = struct.unpack("<ii", raw[i + 44:i + 52])
            parts = struct.unpack("<%di" % nparts, raw[i + 52:i + 52 + 4 * nparts])
            p0 = i + 52 + 4 * nparts
            pts = np.frombuffer(raw[p0:p0 + 16 * npoints], dtype="<f8").reshape(npoints, 2).copy()
```

```
            out.append({"bbox": bbox, "parts": list(parts), "pts": pts})
        i += 8 + clen * 2
    return out


def load_hazard_points(use_cache: bool = True) -> pd.DataFrame:
    cache = CACHE / "hazard_points.parquet"
    if use_cache and cache.exists():
        return pd.read_parquet(cache)
    df = read_dbf(HAZARD_DIR / "西藏日喀则市吉隆县泥石流灾害区历史灾点数据.dbf")
    for c in ("x_coord", "y_coord", "height", "Area", "DisFrmFlt", "沟比降", "主沟长", "高差",
              "沟口H", "高程Max", "扇宽", "扇长"):
        if c in df.columns:
            df[c] = pd.to_numeric(df[c], errors="coerce")
    df = df.rename(columns={"x_coord": "lon", "y_coord": "lat", "Area": "area_km2",
                            "沟比降": "grad", "主沟长": "length_m", "高差": "relief_m",
                            "沟口H": "mouth_m", "高程Max": "zmax_m", "扇宽": "fan_w",
                            "扇长": "fan_l"})
    df.to_parquet(cache)
    return df


def load_census() -> dict:
    df = read_dbf(CENSUS_DIR / "西藏日喀则市吉隆县第七次人口普查数据.dbf")
    row = df.iloc[0].to_dict()
    out = {}
    for k, v in row.items():
        try:
            out[k] = float(v)
        except (TypeError, ValueError):
            out[k] = v
    return out


# ---------------------------------------------------------------- 栅格

def load_dem() -> tuple[np.ndarray, dict]:
    cache = CACHE / "dem.npy"
    if cache.exists():
        g = json.loads((CACHE / "dem_grid.json").read_text(encoding="utf-8"))
        if g.get("preprocess_version") == PREPROCESS_VERSION:
            return np.load(cache), g
    arr, geo = read_geotiff(DEM_DIR / "DEM.tif")
    grid = geotiff_grid(geo)
    arr = arr.astype("float32")
    arr[arr <= -1000] = np.nan
    np.save(cache, arr)
    (CACHE / "dem_grid.json").write_text(json.dumps(grid, ensure_ascii=False, indent=1), encoding="utf-8")
    return arr, grid


def load_landcover(year: int = 2023) -> tuple[np.ndarray, dict]:
    cache = CACHE / f"lc{year}.npy"
    if cache.exists():
        g = json.loads((CACHE / f"lc{year}_grid.json").read_text(encoding="utf-8"))
        if g.get("preprocess_version") == PREPROCESS_VERSION:
            return np.load(cache), g
    arr, geo = read_geotiff(LANDCOVER_DIR / f"jilong_LUCD_{year}.tif")
    grid = geotiff_grid(geo)
    np.save(cache, arr)
    (CACHE / f"lc{year}_grid.json").write_text(json.dumps(grid, ensure_ascii=False, indent=1), encoding="utf-8")
    return arr, grid


LC_NAMES = {
    k: f"覆被原码{k}" for k in range(11)
}


# ---------------------------------------------------------------- 冰川

def load_glacier_nc() -> dict:
    """读取蓝塘里壤峰源区冰川地形/厚度/速度 NetCDF。

    HDF5 的 C 接口在超长中文路径下会打开失败，故首次读取时先复制到短路径缓存。
    """
    import h5py
    import shutil
    short = CACHE / "glacier_source.nc"
    if not short.exists():
        shutil.copyfile(GLACIER_NC, short)
    out = {}
    with h5py.File(str(short), "r") as f:
        for key in ("dem", "slope", "thickness", "v", "vx", "vy", "v_amp", "glacier_mask"):
            if key in f:
                out[key] = np.asarray(f[key][()], dtype="float64")
        out["x"] = np.asarray(f["x"][()], dtype="float64")
        out["y"] = np.asarray(f["y"][()], dtype="float64")
        out["crs"] = str(f["spatial_ref"].attrs.get("crs_wkt", ""))
        gt = f["spatial_ref"].attrs.get("GeoTransform", [])
        try:
            out["geotransform"] = [float(v) for v in gt]
        except (TypeError, ValueError):
            out["geotransform"] = [float(v) for v in str(gt).split()]
    return out
```

```


def load_glacier_outlines(version: int = 6) -> pd.DataFrame:
    cache = CACHE / f"glacier_rgi{version}.parquet"
    if cache.exists():
        return pd.read_parquet(cache)
    dbf = GLACIER_DHDT_DIR / f"RGI_V{version}_JilongFlood_Buffer_200km.dbf"
    raw = dbf.read_bytes()
    nrec, hlen, rlen = struct.unpack("<IHH", raw[4:12])
    nf = (hlen - 33) // 32
    fields = []
    for i in range(nf):
        off = 32 + i * 32
        name = raw[off:off + 11].split(b"\x00")[0].decode("utf-8", "ignore")
        fields.append((name, raw[off + 16]))
    rows = []
    for r in range(nrec):
        base = hlen + r * rlen
        rec = raw[base:base + rlen]
        p, row = 1, {}
        for name, ln in fields:
            row[name] = rec[p:p + ln].decode("utf-8", "ignore").strip()
            p += ln
        rows.append(row)
    df = pd.DataFrame(rows)
    for c in ("area_km2", "cenlon", "cenlat", "zmin_m", "zmax_m", "zmed_m", "slope_deg", "lmax_m"):
        if c in df.columns:
            df[c] = pd.to_numeric(df[c], errors="coerce")
    df.to_parquet(cache)
    return df


def glacier_dhdt_stats() -> list[dict]:
    """统计四个时段、各瓦片冰川表面高程变化率，用于源区/区域冰川减薄背景。"""
    cache = CACHE / "dhdt_stats.json"
    if cache.exists():
        old = json.loads(cache.read_text(encoding="utf-8"))
        if old and all(x.get("aggregation") == "valid_cell_weighted_v2" for x in old):
            return old
    recs = []
    for period_dir in sorted(GLACIER_DHDT_DIR.glob("13_14_15_rgi60_*")):
        period = period_dir.name.replace("13_14_15_rgi60_", "")
        vals, errs, cells, err_cells = [], [], 0, 0
        value_sum, error_sum = 0.0, 0.0
        for f in sorted((period_dir / "dhdt_UTM").glob("*.tif")):
            arr, _ = read_geotiff(f)
            arr = arr.astype("float64")
            arr[~np.isfinite(arr) | (np.abs(arr) > 1e3)] = np.nan
            if np.isfinite(arr).any():
                vals.append(float(np.nanmean(arr)))
                cells += int(np.isfinite(arr).sum())
                value_sum += float(np.nansum(arr))
        for f in sorted((period_dir / "dhdt_err_UTM").glob("*.tif")):
            arr, _ = read_geotiff(f)
            arr = arr.astype("float64")
            arr[~np.isfinite(arr) | (np.abs(arr) > 1e3)] = np.nan
            if np.isfinite(arr).any():
                errs.append(float(np.nanmean(arr)))
                err_cells += int(np.isfinite(arr).sum())
                error_sum += float(np.nansum(arr))
        recs.append({"period": period, "n_tiles": len(vals),
                     "mean_dhdt_m_per_yr": value_sum / cells if cells else None,
                     "mean_err_m_per_yr": error_sum / err_cells if err_cells else None,
                     "valid_cells": cells, "error_valid_cells": err_cells,
                     "aggregation": "valid_cell_weighted_v2",
                     "interpretation": "regional static background; mean error is not standard error of regional mean"})
    CACHE.joinpath("dhdt_stats.json").write_text(json.dumps(recs, ensure_ascii=False, indent=1), encoding="utf-8")
    return recs


def write_json(path: str | Path, obj) -> None:
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    Path(path).write_text(json.dumps(obj, ensure_ascii=False, indent=2, default=float), encoding="utf-8")
```

`mmlib/meteo.py` 气象产品读取与时间对齐

```
"""气象与近地层特征工程，能量输出为融化潜势代理而非实测消融。

所有函数以 pandas DataFrame（逐小时索引）为输入/输出，列名遵循：
  Ta_<z>m / RH_<z>m / WS_<z>m / WD_<z>m / soilT_<d>m / VWC_<d>m / G_<d>m
"""
from __future__ import annotations

import numpy as np
import pandas as pd

G = 9.80665
LV = 2.501e6
CP = 1004.67
R_D = 287.058

LEVELS_T = [1, 2, 4, 10, 20]
LEVELS_W = [1, 2, 4, 10, 20]
DEPTHS = ["0.1", "0.2", "0.4", "0.8", "1.6"]
RI_SHEAR_EPS = 1e-6

```

```python

def hourly_regular(df: pd.DataFrame) -> pd.DataFrame:
    """显式补齐小时索引；只补缺测位置，不插值或把缺测当无雨。"""
    if not isinstance(df.index, pd.DatetimeIndex) or not df.index.is_unique:
        raise ValueError("观测必须有唯一的 DatetimeIndex")
    if not (df.index == df.index.floor("h")).all():
        raise ValueError("特征工程只接受整点小时记录")
    out = df.sort_index().reindex(pd.date_range(df.index.min(), df.index.max(), freq="h"))
    out.attrs = df.attrs.copy()
    return out


# ---------------------------------------------------------------- 热力学量
def es_hpa(t_celsius: pd.Series | np.ndarray) -> np.ndarray:
    """饱和水汽压（Bolton 1980），hPa。"""
    t = np.asarray(t_celsius, dtype="float64")
    return 6.112 * np.exp(17.67 * t / (t + 243.5))


def dewpoint(e_hpa: pd.Series | np.ndarray) -> np.ndarray:
    """由水汽压反算露点温度（°C）。"""
    e = np.clip(np.asarray(e_hpa, dtype="float64"), 1e-3, None)
    ln = np.log(e / 6.112)
    return 243.5 * ln / (17.67 - ln)


def specific_humidity(e_hpa: pd.Series | np.ndarray, p_hpa: pd.Series | np.ndarray) -> np.ndarray:
    """比湿（kg/kg）。"""
    e = np.asarray(e_hpa, dtype="float64")
    p = np.asarray(p_hpa, dtype="float64")
    return 0.622 * e / (p - 0.378 * e)


def potential_temperature(t_celsius: pd.Series | np.ndarray, p_hpa: pd.Series | np.ndarray) -> np.ndarray:
    """位温（K）。"""
    t = np.asarray(t_celsius, dtype="float64") + 273.15
    p = np.asarray(p_hpa, dtype="float64")
    return t * (1000.0 / p) ** 0.2857


def virtual_potential_temperature(t_celsius, q, p_hpa) -> np.ndarray:
    """虚位温（K）。"""
    return potential_temperature(t_celsius, p_hpa) * (1.0 + 0.61 * np.asarray(q, dtype="float64"))


def air_density(p_hpa, t_celsius) -> np.ndarray:
    """空气密度（kg/m³）。"""
    p = np.asarray(p_hpa, dtype="float64") * 100.0
    t = np.asarray(t_celsius, dtype="float64") + 273.15
    return p / (R_D * t)


def hydrostatic_profiles(df: pd.DataFrame, z_ref: int = 2,
                         corrected: bool = True) -> pd.DataFrame:
    """Five-layer common-sample thermodynamics; reference height is assumed, not observed."""
    if z_ref not in LEVELS_T:
        raise ValueError("reference height must be one of the observed levels")
    needed = [f"{family}_{z}m" for family in ("Ta", "RH") for z in LEVELS_T] + ["Pressure"]
    t = df.reindex(columns=[f"Ta_{z}m" for z in LEVELS_T]).to_numpy(float) + 273.15
    rh = df.reindex(columns=[f"RH_{z}m" for z in LEVELS_T]).to_numpy(float)
    pref = df.reindex(columns=["Pressure"]).to_numpy(float)
    valid = (np.isfinite(df.reindex(columns=needed).to_numpy(float)).all(axis=1)
           & (t > 0).all(axis=1) & ((rh >= 0) & (rh <= 100)).all(axis=1)
           & (pref[:, 0] > 0))
    e = es_hpa(t - 273.15) * rh / 100
    p = np.repeat(pref, len(LEVELS_T), axis=1)
    valid &= (p - .378 * e > 0).all(axis=1)
    p[~valid] = np.nan
    residual = np.zeros(len(df), dtype=float)
    ref_index = LEVELS_T.index(z_ref)
    if corrected:
        for _ in range(3):
            q = specific_humidity(e, p)
            tv = t * (1 + .61 * q)
            segments = np.diff(LEVELS_T)[None, :] * .5 * (1 / tv[:, :-1] + 1 / tv[:, 1:])
            integral = np.column_stack((np.zeros(len(df)), np.cumsum(segments, axis=1)))
            new_p = pref * np.exp(-G / R_D * (integral - integral[:, ref_index, None]))
            valid &= (np.isfinite(new_p).all(axis=1) & (tv > 0).all(axis=1)
                      & (new_p - .378 * e > 0).all(axis=1))
            new_p[~valid] = np.nan
            delta = np.abs(new_p - p)
            residual = np.max(np.where(np.isfinite(delta), delta, 0), axis=1)
            p = new_p
    residual[~valid] = np.nan
    q = specific_humidity(e, p)
    theta = t * (1000 / p) ** .2857
    theta_v = theta * (1 + .61 * q)
    out = pd.DataFrame(index=df.index)
    for k, z in enumerate(LEVELS_T):
        for name, values in (("p", p), ("q", q), ("theta", theta), ("thetav", theta_v)):
            out[f"{name}_{z}m"] = values[:, k]
    out["thermo_valid"] = valid
    out["pressure_iteration_residual_hPa"] = residual
    finite_residual = residual[np.isfinite(residual)]
    out.attrs.update({"reference_height_m_assumed": z_ref, "hydrostatic_corrected": corrected,
```

```
                "iterations": 3 if corrected else 0, "five_layer_common_hours": int(valid.sum()),
                "total_hours": len(df), "missing_fraction": float(1 - valid.mean()),
                "iteration_3_minus_2_max_hPa": float(finite_residual.max()) if finite_residual.size else None,
                "iteration_3_minus_2_median_hPa": float(np.median(finite_residual)) if finite_residual.size else
                    None,
                "pressure_source": "MET Pressure; reference height assumed; FLUX PA not substituted"})
    return out


def vector_shear_squared(df: pd.DataFrame, lo: int = 2, hi: int = 20) -> pd.Series:
    needed = [f"{family}_{z}m" for family in ("WS", "WD") for z in (lo, hi)]
    if not set(needed).issubset(df.columns):
        return pd.Series(np.nan, index=df.index)
    a, b = np.radians(df[f"WD_{lo}m"]), np.radians(df[f"WD_{hi}m"])
    du = -df[f"WS_{hi}m"] * np.sin(b) + df[f"WS_{lo}m"] * np.sin(a)
    dv = -df[f"WS_{hi}m"] * np.cos(b) + df[f"WS_{lo}m"] * np.cos(a)
    return (du ** 2 + dv ** 2).where(df[needed].notna().all(axis=1))


# ---------------------------------------------------------------- 边界层结构
def temperature_gradient(df: pd.DataFrame) -> pd.DataFrame:
    """各相邻层的温度梯度（K/100m，按层高差归一）。"""
    out = {}
    for lo, hi in zip(LEVELS_T[:-1], LEVELS_T[1:]):
        col_lo, col_hi = f"Ta_{lo}m", f"Ta_{hi}m"
        if col_lo in df and col_hi in df:
            out[f"dT_{lo}_{hi}m"] = (df[col_hi] - df[col_lo]) / (hi - lo) * 100.0
    return pd.DataFrame(out, index=df.index)


def inversion_metrics(df: pd.DataFrame) -> pd.DataFrame:
    """逆温强度（最大正温度梯度）与逆温层结高度。"""
    grad = temperature_gradient(df)
    if grad.empty:
        return pd.DataFrame(index=df.index)
    out = pd.DataFrame(index=df.index)
    out["inv_max"] = grad.clip(lower=0).max(axis=1) # K/100m
    out["inv_count"] = (grad > 0.05).sum(axis=1) # 逆温层数
    out["dT_total_1_20m"] = df["Ta_20m"] - df["Ta_1m"]
    return out


def wind_shear(df: pd.DataFrame) -> pd.DataFrame:
    """风切变与对数律拟合（粗糙度、摩擦速度）。"""
    out = pd.DataFrame(index=df.index)
    if {"WS_2m", "WS_10m"}.issubset(df.columns):
        out["shear_2_10m"] = (df["WS_10m"] - df["WS_2m"]) / 8.0
    if {"WS_1m", "WS_20m"}.issubset(df.columns):
        out["shear_1_20m"] = (df["WS_20m"] - df["WS_1m"]) / 19.0
    out["vector_shear_2_20m"] = np.sqrt(vector_shear_squared(df)) / 18.0
    out["WS_bulk"] = df[[f"WS_{z}m" for z in LEVELS_W if f"WS_{z}m" in df]].mean(axis=1)
    return out


def bulk_richardson(df: pd.DataFrame, profile: pd.DataFrame | None = None,
                    z_ref: int = 2, corrected: bool = True) -> pd.Series:
    """整体理查森数 Ri_b = g·Δθ_v·Δz / (θ_v·ΔU²)，取 2 m 与 20 m 之间。"""
    need = {"Ta_2m", "Ta_20m", "RH_2m", "RH_20m", "WS_2m", "WS_20m",
            "Pressure", "WD_2m", "WD_20m"}
    if not need.issubset(df.columns):
        return pd.Series(np.nan, index=df.index)
    profile = hydrostatic_profiles(df, z_ref, corrected) if profile is None else profile
    th2, th20 = profile["thetav_2m"], profile["thetav_20m"]
    dtheta = th20 - th2
    du2 = vector_shear_squared(df)
    dz = 18.0
    theta_mean = 0.5 * (th2 + th20)
    with np.errstate(divide="ignore", invalid="ignore"):
        ri = G / theta_mean * dtheta * dz / du2.where(du2 >= RI_SHEAR_EPS)
    ri = ri.replace([np.inf, -np.inf], np.nan)
    eligible = df[list(need)].notna().all(axis=1) & profile["thermo_valid"]
    ri.attrs["shear_mask_count"] = int((eligible & (du2 < RI_SHEAR_EPS)).sum())
    ri.attrs["eligible_count"] = int(eligible.sum())
    ri.attrs["shear_epsilon_m2_s2"] = RI_SHEAR_EPS
    return ri


def stability_class(ri: pd.Series | np.ndarray) -> np.ndarray:
    """按 Ri_b 分级：1 强不稳定 … 5 强稳定。"""
    ri = np.asarray(ri, dtype="float64")
    cls = np.full(ri.shape, np.nan)
    cls[ri < -0.2] = 1.0
    cls[(ri >= -0.2) & (ri < -0.01)] = 2.0
    cls[(ri >= -0.01) & (ri <= 0.01)] = 3.0
    cls[(ri > 0.01) & (ri <= 0.25)] = 4.0
    cls[ri > 0.25] = 5.0
    return cls


def pbl_diagnostics(df: pd.DataFrame, profile: pd.DataFrame | None = None,
                    z_ref: int = 2, corrected: bool = True) -> pd.DataFrame:
    """Keep observed threshold crossing separate from right censoring."""
    profile = hydrostatic_profiles(df, z_ref, corrected) if profile is None else profile
    out = pd.DataFrame({"pblh": np.nan, "pbl_crossing_height_m": np.nan,
                        "pbl_right_censored": np.nan}, index=df.index)
    pending = pd.Series(True, index=df.index)
```

```
    all_valid = pd.Series(True, index=df.index)
    for hi in (4, 10, 20):
        needed = [f"{fam}_{z}m" for fam in ("Ta", "RH", "WS", "WD") for z in (2, hi)] + ["Pressure"]
        if not set(needed).issubset(df.columns):
            return out
        thlo, thhi = profile["thetav_2m"], profile[f"thetav_{hi}m"]
        du2 = vector_shear_squared(df, 2, hi)
        valid = df[needed].notna().all(axis=1) & profile["thermo_valid"] & (du2 >= RI_SHEAR_EPS)
        all_valid &= valid
        ri = G * (thhi - thlo) * (hi - 2) / (0.5 * (thhi + thlo) * du2.where(valid))
        hit = pending & all_valid & (ri >= 0.25)
        out.loc[hit, ["pblh", "pbl_crossing_height_m"]] = float(hi)
        out.loc[hit, "pbl_right_censored"] = 0.
        pending &= ~hit
    out.loc[pending & all_valid, "pblh"] = 20.0
    out.loc[pending & all_valid, "pbl_right_censored"] = 1.
    out.attrs["right_censored_at_20m"] = int((pending & all_valid).sum())
    return out


def pbl_height(df: pd.DataFrame, profile: pd.DataFrame | None = None) -> pd.Series:
    diagnostics = pbl_diagnostics(df, profile)
    out = diagnostics["pblh"].copy()
    out.attrs.update(diagnostics.attrs)
    return out


# ---------------------------------------------------------------- 辐射收支
def clear_sky_rsd(index: pd.DatetimeIndex, lat: float = 28.4, elev: float = 3980.0) -> np.ndarray:
    """晴空水平面太阳辐射（Haurwitz 模型），W/m²。"""
    doy = index.dayofyear.values.astype(float)
    hour = index.hour.values.astype(float) + index.minute.values / 60.0
    decl = np.radians(23.45 * np.sin(2 * np.pi * (284 + doy) / 365.0))
    B = 2 * np.pi * (doy - 81) / 364.0
    eot = 9.87 * np.sin(2 * B) - 7.53 * np.cos(B) - 1.5 * np.sin(B) # 分钟
    # 时间按UTC+8假设，中央经线120E；站点经度不等于时区中央经线。
    solar_time = hour + (85.3 - 120.0) / 15.0 + eot / 60.0
    ha = np.radians(15.0 * (solar_time - 12.0))
    phi = np.radians(lat)
    cosz = np.sin(phi) * np.sin(decl) + np.cos(phi) * np.cos(decl) * np.cos(ha)
    cosz = np.clip(cosz, 0.0, None)
    r0 = 1.0 - 0.03344 * np.cos(2 * np.pi * doy / 365.0)
    s = 1361.0 * r0 * cosz
    clear = 1098.0 * cosz * np.exp(-0.057 / np.maximum(cosz, 0.02))
    return np.where(cosz > 0.01, np.minimum(s, clear) * r0, 0.0)


def radiation_budget(df: pd.DataFrame) -> pd.DataFrame:
    """辐射收支：净短波、净长波、净辐射、反照率、云量指数、晴空指数。"""
    out = pd.DataFrame(index=df.index)
    rsd = df["Rsd"]; rsu = df["Rsu"]; rld = df["Rld"]; rlu = df["Rlu"]
    out["Rn"] = rsd - rsu + rld - rlu
    out["Rns"] = rsd - rsu
    out["Rnl"] = rld - rlu
    with np.errstate(divide="ignore", invalid="ignore"):
        alb = rsu / rsd
    out["albedo"] = alb.where((rsd > 50) & (alb.between(0, 1.2)))
    cs = clear_sky_rsd(df.index, lat=float(df.attrs.get("lat", 28.4)))
    out["Rsd_clear"] = cs
    with np.errstate(divide="ignore", invalid="ignore"):
        ki = rsd / np.where(cs > 20, cs, np.nan)
    out["clearness"] = ki.clip(0, 1.4)
    out["cloud_idx"] = (1.0 - out["clearness"]).where(rsd > 20)
    return out


def energy_partition(df: pd.DataFrame, rn: pd.Series | None = None) -> pd.DataFrame:
    """地表能量配分：波文比、闭合比、消融有效能量、正积温。

    Parameters
    ----------
    df : 原始观测表（需含 H、LE、G_0.1m、Ta_2m）
    rn : 净辐射序列；缺省时用降级的 Rsd-Rsu+Rld-Rlu 现算，保证函数可独立调用
    """
    out = pd.DataFrame(index=df.index)
    if rn is None and {"Rsd", "Rsu", "Rld", "Rlu"}.issubset(df.columns):
        rn = df["Rsd"] - df["Rsu"] + df["Rld"] - df["Rlu"]
    h = df.get("H"); le = df.get("LE"); g = df.get("G_0.1m")
    if h is not None and le is not None:
        with np.errstate(divide="ignore", invalid="ignore"):
            beta = h / le.where(le.abs() > 5)
        out["bowen"] = beta.clip(-10, 10)
        out["available"] = h + le
        if g is not None and rn is not None:
            avail = rn - g
            out["closure"] = (h + le) / avail.where(avail.abs() > 20)
    if g is not None and rn is not None:
        # 可用于融雪/融冰的能量（净辐射扣除土壤热通量与湍流交换后的余量，限正）
        exchange = h + le if h is not None and le is not None else pd.Series(np.nan, index=df.index)
        resid = rn - g - exchange
        out["melt_energy"] = resid.clip(lower=0)
        out["cum_melt_energy"] = out["melt_energy"] * 3600.0 / 1e6 # MJ/m² per hour
    if "Ta_2m" in df:
        out["PDD_hour"] = df["Ta_2m"].clip(lower=0) / 24.0  # 正积温（°C·d）
    return out
```

```python


# ---------------------------------------------------------------------- 湍流
def turbulence_metrics(df: pd.DataFrame) -> pd.DataFrame:
    """湍流：摩擦速度、拖曳系数、湍流强度、湍流动能。"""
    out = pd.DataFrame(index=df.index)
    if "USTAR" in df:
        out["ustar"] = df["USTAR"].where(df["USTAR"] >= 0)
    if {"USTAR", "WS_10m"}.issubset(df.columns):
        with np.errstate(divide="ignore", invalid="ignore"):
            cd = (df["USTAR"] / df["WS_10m"].where(df["WS_10m"] > 0.5)) ** 2
        out["Cd"] = cd.clip(1e-4, 1.0)
    if "TKE" in df:
        out["TKE_obs"] = df["TKE"].where(df["TKE"] >= 0)
    if {"Ux", "Uy", "Uz"}.issubset(df.columns):
        out["sigma_u"] = df["Ux"].rolling(3, min_periods=3).std()
        out["sigma_w"] = df["Uz"].rolling(3, min_periods=3).std()
    if "FETCH_90" in df:
        out["fetch90"] = df["FETCH_90"]
    return out


# ---------------------------------------------------------------------- 降水特征
def precipitation_features(df: pd.DataFrame) -> pd.DataFrame:
    """降水：前累积、有效降雨指数、最大雨强、降水历时、干湿转折。"""
    out = pd.DataFrame(index=df.index)
    p = df["Precip"].where(df["Precip"] >= 0)
    out["P"] = p
    for hz in (1, 3, 6, 12, 24, 72, 168):
        out[f"P_{hz}h_coverage"] = p.notna().rolling(hz, min_periods=hz).mean()
        out[f"P_{hz}h"] = p.rolling(hz, min_periods=hz).sum()
    # 缺测打断递推，并屏蔽重启后的7日热身：不假定未知降雨为0。
    api = np.full(len(p), np.nan)
    acc = 0.0
    valid_run = 0
    decay = 0.85 ** (1.0 / 24.0)
    for i, v in enumerate(p.values):
        if not np.isfinite(v):
            acc, valid_run = 0.0, 0
            continue
        acc = acc * decay + v
        valid_run += 1
        if valid_run >= 168:
            api[i] = acc
    out["API"] = api
    out["wet"] = (p > 0.1).astype(float).where(p.notna())
    out["storm_len"] = out["wet"].groupby((out["wet"] != out["wet"].shift()).cumsum()).cumsum() * out["wet"]
    for hz in (1, 3):
        out[f"Pmax_{hz}h"] = p.rolling(hz, min_periods=hz).max()
    out["AP_7d"] = out["P_168h"]
    return out


# ---------------------------------------------------------------------- 综合特征表
def build_feature_table(df: pd.DataFrame) -> pd.DataFrame:
    """拼接全部派生特征，返回与原表同索引的特征表。"""
    df = hourly_regular(df)
    rad = radiation_budget(df)
    parts = [
        df[[c for c in df.columns if c.startswith(("Ta_", "RH_", "WS_", "WD_", "soilT_", "VWC_"))
            or c in ("Pressure", "Precip", "Rsd", "Rsu", "Rld", "Rlu", "H", "LE", "G_0.1m",
                     "USTAR", "TKE", "VPD", "PA")]],
        temperature_gradient(df),
        inversion_metrics(df),
        wind_shear(df),
        rad,
        energy_partition(df, rad["Rn"]),
        turbulence_metrics(df),
        precipitation_features(df),
    ]
    feat = pd.concat(parts, axis=1)
    profile = hydrostatic_profiles(df)
    feat = pd.concat((feat, profile), axis=1)
    ri = bulk_richardson(df, profile)
    feat["Ri_b"] = ri
    feat["stab"] = stability_class(feat["Ri_b"])
    feat["dtheta_1_20"] = feat["theta_20m"] - feat["theta_1m"]
    es = es_hpa(df["Ta_2m"])
    e = es * df["RH_2m"] / 100.0
    feat["e_s"] = es
    feat["e_a"] = e
    feat["VPD2"] = es - e
    feat["Td"] = dewpoint(e)
    feat["q"] = profile["q_2m"]
    feat["rho"] = air_density(profile["p_2m"], df["Ta_2m"])
    t_kelvin = df["Ta_2m"] + 273.15
    feat["theta_e"] = profile["theta_2m"] * np.exp(
        LV * feat["q"] / (CP * t_kelvin.where(t_kelvin > 0)))
    feat["PDD_3d"] = df["Ta_2m"].clip(lower=0).rolling(72, min_periods=72).sum()
    feat["PDD_7d"] = df["Ta_2m"].clip(lower=0).rolling(168, min_periods=168).sum()
    feat["rise_24h"] = df["Ta_2m"] - df["Ta_2m"].shift(24)
    feat["freeze_thaw"] = ((df["Ta_2m"] * df["Ta_2m"].shift() < 0).astype(float)
                           .where(df["Ta_2m"].notna() & df["Ta_2m"].shift().notna())
                           .rolling(24, min_periods=24).sum())
    pbl = pbl_diagnostics(df, profile)
    feat = pd.concat((feat, pbl), axis=1)
```

```
    feat.attrs["pbl_right_censored_count"] = pbl.attrs.get("right_censored_at_20m", 0)
    feat.attrs["hydrostatic"] = profile.attrs.copy()
    feat.attrs.update(ri.attrs)
    feat.attrs["api_missing_policy"] = "缺测重置：连续168小时有效后才提供API，不填0"
    feat.attrs["feature_version"] = "v2-hydrostatic-five-layer-3iter"
    feat.attrs["energy_units"] = {"melt_energy": "W/m2, unadjusted proxy",
                                  "cum_melt_energy": "MJ/m2 for one hour"}
    return feat


def closure_sensitivity(feat: pd.DataFrame, n_boot: int = 500, seed: int = 20260826) -> tuple[pd.DataFrame, dict]:
    """以训练年块直接自举均值比C；未调整能量保留为基准。"""
    mask = ((feat.index.year >= 2021) & (feat.index.year <= 2025))
    common = feat.loc[mask, ["Rn", "G_0.1m", "H", "LE"]].dropna()
    x = common["Rn"] - common["G_0.1m"]
    common = common.loc[x > 20].copy()
    common["x"] = common["Rn"] - common["G_0.1m"]
    common["y"] = common["H"] + common["LE"]
    if len(common) < 100:
        raise ValueError("训练期共同有效能量样本不足")
    c = float(common["y"].sum() / common["x"].sum())
    if not np.isfinite(c) or c <= 0:
        raise ValueError("训练期闭合比非正，不能据此调整")
    slope, intercept = np.polyfit(common["x"], common["y"], 1)
    blocks = common.groupby(common.index.year)[["x", "y"]].sum()
    rng = np.random.default_rng(seed)
    draws = []
    for _ in range(n_boot):
        b = blocks.iloc[rng.integers(0, len(blocks), len(blocks))].sum()
        ratio = float(b["y"] / b["x"]) if abs(b["x"]) > 1e-9 else np.nan
        draws.append(ratio if np.isfinite(ratio) and ratio > 0 else np.nan)
    draws = np.asarray(draws)
    finite = draws[np.isfinite(draws)]
    if len(finite) < 0.8 * n_boot:
        raise ValueError("闭合比自举无效成员过多")
    lo, hi = np.percentile(finite, [2.5, 97.5])
    out = feat.copy()
    residual = out["Rn"] - out["G_0.1m"]
    turbulent = out["H"] + out["LE"]
    out["melt_energy_adjusted"] = (residual - turbulent / c).clip(lower=0)
    out["cum_melt_energy_adjusted"] = out["melt_energy_adjusted"] * 0.0036
    summaries = {}
    ev = (out.index >= "2026-08-24") & (out.index < "2026-08-27")
    for tag, cc in (("unadjusted", 1.0), ("C_point", c), ("C_lower", lo), ("C_upper", hi)):
        energy = (residual - turbulent / cc).clip(lower=0) * 0.0036
        summaries[tag] = {"C": float(cc), "训练共同样本平均小时能量_MJm2": float(energy.loc[common.index].mean()),
                          "事件三日有效小时": int(energy.loc[ev].notna().sum()),
                          "事件三日已观测能量和_MJm2": float(energy.loc[ev].sum(min_count=1))}
    return out, {"训练期": [2021, 2025], "共同有效样本": int(len(common)),
                 "选择规则": "Rn-G>20 W/m²且Rn/G/H/LE共同有效；不按各情景另筛样本",
                 "闭合比估计量": "sum(H+LE)/sum(Rn-G)", "闭合比C": c,
                 "回归斜率_非C": float(slope), "回归截距": float(intercept),
                 "自举单位": "完整训练年块", "自举重复": n_boot, "随机种子": seed,
                 "闭合比C_95pct": [float(lo), float(hi)],
                 "自举无效比例": float(1 - len(finite) / n_boot),
                 "自举C样本": draws.tolist(), "能量敏感性": summaries,
                 "口径": "melt_energy为未调整基准；adjusted仅为冻结C的能量代理敏感性，非实测融化"}
```

`mmlib/ranking.py` 稳健异常与关联排序

```
"""同源气象资料的两套排序及有时间边界的敏感性检验。"""
from __future__ import annotations

import numpy as np
import pandas as pd
from scipy import stats
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.feature_selection import mutual_info_classif
from sklearn.inspection import permutation_importance
from sklearn.metrics import roc_auc_score

SEED = 20260826
EVENT = pd.Timestamp("2026-08-26")
WEIGHTS = {"PI_rank": 0.30, "MI_rank": 0.25, "AN_rank": 0.20, "MP_rank": 0.25}
SOURCE_FAMILY = {"P", "P_24h", "P_3h", "Pmax_1h", "API", "AP_7d", "PDD_3d", "PDD_7d"}
CONSTRUCTIVE = {"P", "P_24h", "P_3h", "Pmax_1h", "API", "AP_7d", "cum_melt_energy", "Rn", "Ta_2m"}
NEGATIVE_DIRECTION = {"RH_2m", "Ri_b", "Pressure", "albedo"}


def _tau(left: list[str], right: list[str]) -> float:
    common = [x for x in left if x in right]
    if len(common) < 2:
        return float("nan")
    return float(stats.kendalltau([left.index(x) for x in common],
                                  [right.index(x) for x in common]).statistic)


def compare(left: list[str], right: list[str]) -> dict:
    return {"前5重合数": len(set(left[:5]) & set(right[:5])),
            "全排序Kendall_tau": _tau(left, right), "共同特征数": len(set(left) & set(right))}


```

```python
def connected_components(frame: pd.DataFrame, cols: list[str], threshold: float = 0.8) -> tuple[list[list[str]], list[dict]]:
    corr = frame[cols].corr(min_periods=100).abs()
    edges = [{"变量1": a, "变量2": b, "绝对相关": float(corr.loc[a, b])}
             for i, a in enumerate(cols) for b in cols[i + 1:]
             if np.isfinite(corr.loc[a, b]) and corr.loc[a, b] > threshold]
    adjacent = {c: set() for c in cols}
    for edge in edges:
        a, b = edge["变量1"], edge["变量2"]
        adjacent[a].add(b)
        adjacent[b].add(a)
    components, seen = [], set()
    for c in cols:
        if c in seen:
            continue
        todo, component = [c], []
        while todo:
            v = todo.pop()
            if v in seen:
                continue
            seen.add(v)
            component.append(v)
            todo.extend(sorted(adjacent[v] - seen))
        components.append(sorted(component))
    return components, edges


def deduplicate(rows: pd.DataFrame, components: list[list[str]], score: str,
                ascending: bool = True) -> tuple[pd.DataFrame, list[dict]]:
    order = rows.sort_values([score, "变量"], ascending=[ascending, True])
    place = {v: i for i, v in enumerate(order["变量"])}
    keep, disclosure = [], []
    for i, component in enumerate(components, 1):
        members = [v for v in component if v in place]
        if not members:
            continue
        winner = min(members, key=place.get)
        keep.append(winner)
        disclosure.append({"簇": i, "成员": members, "保留变量": winner,
                           "剔除变量": [v for v in members if v != winner]})
    display = order[order["变量"].isin(keep)].copy()
    display["展示次序"] = np.arange(1, len(display) + 1)
    return display, disclosure


def _anomaly(feat: pd.DataFrame, cols: list[str], start: pd.Timestamp,
             end: pd.Timestamp, baseline: pd.DataFrame) -> pd.Series:
    result = {}
    for c in cols:
        s = baseline[c].dropna()
        event = feat.loc[start:end, c].dropna()
        if len(s) < 100 or len(event) < 12:
            result[c] = np.nan
            continue
        value = event.max() if abs(s.skew()) > 1 else event.mean()
        probability = max(2 * min((s >= value).mean(), (s <= value).mean()), 1 / (len(s) + 1))
        result[c] = -np.log10(min(probability, 1.0))
    return pd.Series(result)


def _fit_evidence(feat: pd.DataFrame, qw: pd.Series, cols: list[str],
                  train_mask: np.ndarray, test_mask: np.ndarray, quantile: float = 0.90,
                  repeats: int = 5, iterations: int = 160) -> tuple[pd.Series, pd.Series, dict]:
    from . import trigger
    api_mask = (train_mask & np.isin(feat.index.year, trigger.API_TRAIN_YEARS)
                & np.isin(feat.index.month, trigger.API_WARM_MONTHS))
    fold_api = pd.to_numeric(feat.loc[api_mask, "API"], errors="coerce")
    fold_api = fold_api.where(np.isfinite(fold_api) & fold_api.ge(0)).dropna()
    if len(fold_api) < 100:
        raise ValueError("该折2021--2025暖季有效API样本不足")
    api_thresholds = fold_api.quantile([1 / 3, 2 / 3]).to_numpy().tolist()
    fold_features = feat.copy(deep=False)
    fold_features.attrs = {**feat.attrs, "api_quantiles": api_thresholds}
    model = trigger.default_model(fold_features, calibrate=False)
    qw = model.discharge(fold_features)["Q_water"]
    valid_q = qw.notna() & np.isfinite(qw)
    train = feat.loc[train_mask & valid_q, cols]
    test = feat.loc[test_mask & valid_q, cols]
    threshold = float(qw.loc[train.index].quantile(quantile))
    train = train.iloc[::max(1, int(np.ceil(len(train) / 30000)))]
    test = test.iloc[::max(1, int(np.ceil(len(test) / 12000)))]
    if len(train) < 500 or len(test) < 100:
        raise ValueError("时间留出排序样本不足")
    medians = train.median().fillna(0)
    xtr, xte = train.fillna(medians), test.fillna(medians)
    ytr = (qw.loc[train.index] > threshold).astype(int)
    yte = (qw.loc[test.index] > threshold).astype(int)
    if ytr.nunique() < 2 or yte.nunique() < 2:
        raise ValueError("时间留出标签不足两类")
    clf = HistGradientBoostingClassifier(max_iter=iterations, learning_rate=0.07,
                                         max_depth=5, min_samples_leaf=40, random_state=SEED)
    clf.fit(xtr, ytr)
    pi = pd.Series(permutation_importance(clf, xte, yte, n_repeats=repeats,
                                          scoring="roc_auc", n_jobs=1, random_state=SEED).importances_mean,
                   index=cols)
```

```
    mi = pd.Series(mutual_info_classif(xtr, ytr, random_state=SEED), index=cols)
    constant = train.columns[train.nunique(dropna=True) <= 1]
    pi.loc[constant] = 0.0
    mi.loc[constant] = 0.0
    note = {"训练样本": len(train), "测试样本": len(test),
            "训练时间": [str(train.index.min()), str(train.index.max())],
            "测试时间": [str(test.index.min()), str(test.index.max())],
            "标签分位": quantile, "训练标签阈值_m3s": threshold,
            "训练正例率": float(ytr.mean()), "测试正例率": float(yte.mean()),
            "测试AUC": float(roc_auc_score(yte, clf.predict_proba(xte)[:, 1])),
            "训练插补中位数": medians.to_dict(),
            "训练全缺失列": train.columns[train.isna().all()].tolist(),
            "训练常量或全缺失列_重要性置零": constant.tolist(),
            "训练API分位": api_thresholds,
            "标签水文预处理": "各折2021--2025训练暖季API分位冻结后重算整序列Qw；不覆盖共享qwater缓存；未事件标定",
            "口径": "AUC只度量代理水文标签重建，不是灾害发生识别；插补与标签分位只用训练期"}
    return pi, mi, note


def _primary_table(flat: list[tuple[str, str, str]], pi: pd.Series, mi: pd.Series,
                   anomaly: pd.Series, prior: dict) -> pd.DataFrame:
    rows = pd.DataFrame([{"条件组": group, "特征": name, "变量": c,
                          "置换重要性": pi.get(c, np.nan), "互信息": mi.get(c, np.nan),
                          "事件异常度": anomaly.get(c, np.nan), "机理先验": prior.get(c, 0.3),
                          "特征性质": "构造性驱动项" if c in CONSTRUCTIVE else "独立于标签构造的调制项"}
                         for group, name, c in flat])
    for col, tag in (("置换重要性", "PI"), ("互信息", "MI"), ("事件异常度", "AN"), ("机理先验", "MP")):
        rows[f"{tag}_rank"] = rows[col].rank(ascending=False, method="average", na_option="bottom")
    rows["综合得分"] = sum(rows[k] * w for k, w in WEIGHTS.items())
    rows["无先验得分"] = sum(rows[k] * WEIGHTS[k] / 0.75 for k in ("PI_rank", "MI_rank", "AN_rank"))
    rows = rows.sort_values(["综合得分", "变量"]).reset_index(drop=True)
    rows["总次序"] = np.arange(1, len(rows) + 1)
    rows["组内次序"] = rows.groupby("条件组")["综合得分"].rank(method="first").astype(int)
    return rows


def _daily(feat: pd.DataFrame, cols: list[str]) -> pd.DataFrame:
    d = feat[cols].resample("D").mean()
    counts = feat[cols].resample("D").count()
    d = d.where(counts >= 20)
    for c in ("albedo", "clearness"):
        if c in cols:
            eligible = feat["Rsd"] > (50 if c == "albedo" else 20)
            denom = eligible.resample("D").sum()
            valid = feat[c].where(eligible).resample("D").count()
            d[c] = feat[c].where(eligible).resample("D").mean().where((denom >= 4) & (valid >= .8*denom))
    d["P"] = feat["P"].resample("D").sum(min_count=24)
    for c in ("AP_7d", "PDD_3d", "PDD_7d"):
        if c in cols:
            # 23:00缺测不得用更早的“最后有效值”悄悄缩短窗口。
            d[c] = feat[c].where(feat.index.hour == 23).resample("D").max()
    return d


def critic_ranking(feat: pd.DataFrame, flat: list[tuple[str, str, str]],
                   components: list[list[str]]) -> dict:
    cols = [x[2] for x in flat]
    daily = _daily(feat, cols)
    baseline = daily.loc[(daily.index.year <= 2025) & (daily.index.month.isin([6, 7, 8, 9]))]
    if "PDD_3d" not in daily or "AP_7d" not in daily:
        raise KeyError("CRITIC需要PDD_3d与AP_7d")
    rain95, pdd95 = baseline["P"].quantile(.95), baseline["PDD_3d"].quantile(.95)
    label_valid = baseline[["P", "PDD_3d"]].notna().all(axis=1)
    label = ((baseline["P"] >= rain95) | (baseline["PDD_3d"] >= pdd95)).where(label_valid)
    normalized = {}
    norm_params = {}
    for c in ("AP_7d", "PDD_3d"):
        lo, hi = baseline[c].min(), baseline[c].max()
        if not np.isfinite(lo + hi) or hi <= lo:
            raise ValueError(f"CRITIC触发核{c}无法归一化")
        normalized[c] = ((baseline[c] - lo) / (hi - lo)).clip(0, 1)
        norm_params[c] = [float(lo), float(hi)]
    kernel = pd.concat(normalized, axis=1).max(axis=1, skipna=False)
    doy = daily.index.dayofyear
    delta = np.abs(doy - EVENT.dayofyear)
    near = daily.loc[(daily.index.year <= 2025) & (np.minimum(delta, 366 - delta) <= 15)]
    event = daily.loc["2026-08-24":"2026-08-26"]
    evidence, detail = [], []
    for group, name, c in flat:
        direction = -1 if c in NEGATIVE_DIRECTION else 1
        clim, ev = near[c].dropna(), event[c].dropna()
        z = (ev.mean() - clim.mean()) / clim.std(ddof=1) if len(clim) > 10 and len(ev) > 0 and clim.std() > 0 else np
            .nan
        a = max(direction * z, 0) if np.isfinite(z) else np.nan
        quantile = float((clim <= ev.mean()).mean()) if len(clim) and len(ev) else np.nan
        b = np.nan
        valid = label.notna() & baseline[c].notna()
        if c not in {"P", "Pmax_1h", "PDD_3d"} and valid.sum() >= 30 and label[valid].nunique() == 2:
            b = float(roc_auc_score(label[valid].astype(int), direction * baseline.loc[valid, c]))
        cc = np.nan
        valid_c = kernel.notna() & baseline[c].notna()
        if c not in SOURCE_FAMILY and valid_c.sum() >= 30:
```

```
            cc = abs(float(stats.spearmanr(baseline.loc[valid_c, c], kernel[valid_c]).statistic))
        evidence.append([a, b, cc])
        detail.append({"条件组": group, "特征": name, "变量": c, "增益方向假设": direction,
                       "事件标准异常z": z, "事件分位秩": quantile,
                       "证据A": a, "证据B": b, "证据C": cc,
                       "B排除构造变量": c in {"P", "Pmax_1h", "PDD_3d"},
                       "C排除触发族": c in SOURCE_FAMILY,
                       "特征性质": "构造性驱动项" if c in CONSTRUCTIVE else "独立于标签构造的调制项"})
    matrix = pd.DataFrame(evidence, index=cols, columns=["A", "B", "C"])
    scaled = (matrix - matrix.min()) / (matrix.max() - matrix.min()).replace(0, np.nan)
    scaled = scaled.where(matrix.notna())
    # 权重估计采用各证据对的共同可用特征，不把排除项当零值。
    corr = scaled.corr(min_periods=3).fillna(0)
    for col in corr.columns:
        corr.loc[col, col] = 1.0
    information = scaled.std(ddof=0).fillna(0) * (1 - corr).sum(axis=1)
    if information.sum() <= 0:
        raise ValueError("CRITIC证据信息量为零")
    weights = information / information.sum()
    available_weight = scaled.notna().mul(weights, axis=1).sum(axis=1)
    scores = scaled.mul(weights, axis=1).sum(axis=1, min_count=1) / available_weight.replace(0, np.nan)
    table = pd.DataFrame(detail)
    table["CRITIC得分"] = table["变量"].map(scores)
    table["可用证据权重和"] = table["变量"].map(available_weight)
    table = table.sort_values(["CRITIC得分", "变量"], ascending=[False, True])
    table["总次序"] = np.arange(1, len(table) + 1)
    display, clusters = deduplicate(table, components, "CRITIC得分", ascending=False)
    return {"权重": weights.to_dict(), "信息承载量": information.to_dict(),
            "证据相关矩阵": corr.to_dict(), "原始证据矩阵": matrix.to_dict(orient="index"),
            "归一化证据矩阵": scaled.to_dict(orient="index"), "排序表": table.to_dict("records"),
            "去冗余排序表": display.to_dict("records"), "前5特征": display.head(5).to_dict("records"),
            "冗余簇": clusters, "训练阈值": {"日雨量p95_mm": rain95, "PDD3_p95_°Ch": pdd95},
            "触发核归一化范围": norm_params, "事件窗口": ["2026-08-24", "2026-08-26"],
            "气候事件基线": "2021–2025同年积日±15日", "样本覆盖规则": "日均>=20小时，日雨完整24小时；白昼反照率/晴空指数按物理可用时段>=80%且至少4小时；72/168h累积要求整窗有效且23时有效",
            "解释": "与主排序共享原始气象和事件资料，仅为不同证据权重方案的一致性核对；非独立灾害验证"}


def association(feat: pd.DataFrame, qw: pd.Series, groups: dict, prior: dict) -> dict:
    flat = [(g, name, c) for g, names in groups.items() for name, c in names.items() if c in feat]
    cols = [c for _, _, c in flat]
    train_mask = (feat.index.year >= 2021) & (feat.index.year <= 2024)
    test_mask = feat.index.year == 2025
    pi, mi, fit_note = _fit_evidence(feat, qw, cols, train_mask, test_mask)
    start, end = pd.Timestamp("2026-08-20"), pd.Timestamp("2026-08-26 23:00")
    baseline = feat.loc[train_mask]
    anomaly = _anomaly(feat, cols, start, end, baseline)
    table = _primary_table(flat, pi, mi, anomaly, prior)
    components, edges = connected_components(baseline, cols)
    display, clusters = deduplicate(table, components, "综合得分")
    no_prior, np_clusters = deduplicate(table, components, "无先验得分")
    np_full = table.sort_values(["无先验得分", "变量"])["变量"].tolist()
    primary_full = table["变量"].tolist()
    sensitivity = []
    for q in (.75, .80, .85, .95):
        p2, m2, note = _fit_evidence(feat, qw, cols, train_mask, test_mask, q, repeats=2, iterations=120)
        t2 = _primary_table(flat, p2, m2, anomaly, prior)
        d2, _ = deduplicate(t2, components, "综合得分")
        sensitivity.append({"标签分位": q, **note, **compare(primary_full, t2["变量"].tolist()),
                            "去冗余前5重合数": len(set(display.head(5)["变量"]) & set(d2.head(5)["变量"]))})
    loyo = {}
    for year in (2023, 2024, 2025):
        tr, te = (feat.index.year >= 2021) & (feat.index.year < year), feat.index.year == year
        p2, m2, note = _fit_evidence(feat, qw, cols, tr, te, repeats=2, iterations=120)
        # 事件异常只用于固定的回顾性排序部分，不能把整榜称作该年的独立预测重要性。
        t2 = _primary_table(flat, p2, m2, anomaly, prior)
        fold_components, _ = connected_components(feat.loc[tr], cols)
        d2, _ = deduplicate(t2, fold_components, "综合得分")
        loyo[str(year)] = {**note, **compare(primary_full, t2["变量"].tolist()),
                           "前5_去冗余": d2.head(5)["变量"].tolist(),
                           "排序表": t2.to_dict("records"),
                           "限制": "向前留一年仅PI/MI重估；事件异常与机理先验固定，整榜非独立灾前验证"}
    shifted = []
    for year in range(2021, 2026):
        # 控制年从其自身气候参照剔除，减少自身进入经验分布的影响。
        ref = feat.loc[(feat.index.year <= 2025) & (feat.index.year != year)]
        shifted.append(_anomaly(feat, cols, pd.Timestamp(f"{year}-08-20"),
                                pd.Timestamp(f"{year}-08-26 23:00"), ref))
    shifted_anomaly = pd.concat(shifted, axis=1).mean(axis=1)
    real_order = anomaly.sort_values(ascending=False).index.tolist()
    shift_order = shifted_anomaly.sort_values(ascending=False).index.tolist()
    critic = critic_ranking(feat, flat, components)
    cross = compare(primary_full, [r["变量"] for r in critic["排序表"]])
    cross["去冗余前5重合数"] = len(set(display.head(5)["变量"]) & {r["变量"] for r in critic["前5特征"]})
    taus = [x["全排序Kendall_tau"] for x in sensitivity]
    top5 = display.head(5).copy()
    top5["次序"] = np.arange(1, len(top5) + 1)
    return {"候选特征数": len(cols), "标签定义": "Qw超过2021–2024训练期90%分位；Qw缺测不生成负例",
            "标签正例率": fit_note["训练正例率"], **fit_note, "权重": WEIGHTS,
```

```
        "排序表": table.to_dict("records"), "去冗余排序表": display.to_dict("records"),
        "前5特征": top5.to_dict("records"), "冗余簇": clusters, "训练相关边": edges,
        "去先验对照": {"权重_无先验": {k: v / .75 for k, v in WEIGHTS.items() if k != "MP_rank"},
                     **compare(primary_full, np_full),
                     "前5_主排序": display.head(5)["特征"].tolist(),
                     "前5_无先验": no_prior.head(5)["特征"].tolist(),
                     "去冗余前5重合数": len(set(display.head(5)["变量"]) & set(no_prior.head(5)["变量"])),
                     "完整排序表": table.sort_values("无先验得分").to_dict("records"), "冗余簇": np_clusters},
        "留一年AUC": {y: r["测试AUC"] for y, r in loyo.items()}, "逐年前向留出": loyo,
        "排序稳健性": {"阈值扰动Kendall_tau": taus, "均值": float(np.nanmean(taus)),
                     "最小值": float(np.nanmin(taus)), "逐阈值": sensitivity},
        "控制组_日历平移": {"排序一致性Kendall_tau": _tau(real_order, shift_order),
                         "事件年异常度": anomaly.to_dict(), "平移年异常度": shifted_anomaly.to_dict(),
                         "说明": "同日历窗口平移至2021–2025，控制年不进入其自身参照；只检验季节混杂"},
        "CRITIC": critic, "两方案一致性": cross,
        "共同限制": "排序包含事件后描述资料；代理标签包含能量/降水输入；高AUC不证明真实泥石流因果或识别能力"}
```

`mmlib/terrain.py`　地形、D8 路由与断面

```python
"""DEM 地形水文分析：填洼、D8 流向、汇流累积、坡向/曲率、集水区与沟道剖面。"""
from __future__ import annotations

import heapq

import numpy as np

DIRS = {1: (-1, -1), 2: (-1, 0), 3: (-1, 1), 4: (0, -1), 5: (0, 1), 6: (1, -1), 7: (1, 0), 8: (1, 1)}


def fill_sinks(z: np.ndarray, eps: float = 1e-3) -> np.ndarray:
    """Priority-Flood 洼地填平（Barnes et al. 2014，堆实现，O(N log N)）。

    边界像元视为出口；填平后地形不存在闭合洼地，D8 可连续汇流至边界。
    """
    src = np.asarray(z, dtype="float64")
    h, w = src.shape
    nodata = ~np.isfinite(src)
    out = src.copy()
    closed = nodata.copy()
    heap: list[tuple[float, int, int]] = []
    border_cells = [(r, c) for r in range(h) for c in (0, w - 1)]
    border_cells += [(r, c) for c in range(w) for r in (0, h - 1)]
    for r, c in border_cells:
        if not closed[r, c]:
            closed[r, c] = True
            heapq.heappush(heap, (float(src[r, c]), r, c))
    while heap:
        level, r, c = heapq.heappop(heap)
        for dr, dc in DIRS.values():
            rr, cc = r + dr, c + dc
            if 0 <= rr < h and 0 <= cc < w and not closed[rr, cc]:
                closed[rr, cc] = True
                if src[rr, cc] <= level:
                    out[rr, cc] = level + eps
                    heapq.heappush(heap, (out[rr, cc], rr, cc))
                else:
                    heapq.heappush(heap, (src[rr, cc], rr, cc))
    out[nodata] = np.nan
    return out


def d8_slope(z: np.ndarray, cell: tuple[float, float] = (1.0, 1.0)) -> tuple[np.ndarray, np.ndarray]:
    """D8 流向编码（1–8 顺时针自西北起，0 表示无下游）与最陡坡降（无量纲）。"""
    h, w = z.shape
    pad = np.pad(z, 1, mode="edge")
    best = np.zeros((h, w), dtype=np.int8)
    steep = np.zeros((h, w))
    for k, (dr, dc) in DIRS.items():
        nb = pad[1 + dr:1 + dr + h, 1 + dc:1 + dc + w]
        drop = (z - nb) / float(np.hypot(dr * cell[0], dc * cell[1]))
        better = drop > steep
        steep = np.where(better, drop, steep)
        best = np.where(better, k, best)
    return best, steep


def d8(z: np.ndarray, cell: tuple[float, float] = (1.0, 1.0)) -> np.ndarray:
    """D8 流向编码。"""
    return d8_slope(z, cell)[0]


def flow_accumulation(direction: np.ndarray, weights: np.ndarray | None = None) -> np.ndarray:
    """按 D8 流向自上游向下游累加（入度拓扑序推进）。"""
    h, w = direction.shape
    acc = np.ones((h, w)) if weights is None else np.asarray(weights, dtype="float64").copy()
    indeg = np.zeros((h, w), dtype="int32")
    rows = np.arange(h)[:, None] * np.ones((1, w), dtype="int64")
    cols = np.ones((h, 1), dtype="int64") * np.arange(w)[None, :]
    for k, (dr, dc) in DIRS.items():
        rr = rows + dr
        cc = cols + dc
        ok = (rr >= 0) & (rr < h) & (cc >= 0) & (cc < w)
        tgt = np.zeros((h, w), dtype="int32")
```

```
      np.add.at(tgt, (rr[ok], cc[ok]), (direction == k)[ok].astype("int32"))
      indeg += tgt
   queue = [tuple(int(v) for v in p) for p in np.argwhere(indeg == 0)]
   while queue:
      r, c = queue.pop()
      k = int(direction[r, c])
      if k == 0:
         continue
      dr, dc = DIRS[k]
      rr, cc = r + dr, c + dc
      if 0 <= rr < h and 0 <= cc < w:
         acc[rr, cc] += acc[r, c]
         indeg[rr, cc] -= 1
         if indeg[rr, cc] == 0:
            queue.append((rr, cc))
   return acc


def upstream_mask(direction: np.ndarray, seed: tuple[int, int]) -> np.ndarray:
   """从出口像元沿 D8 反向遍历，返回集水区掩膜。"""
   h, w = direction.shape
   donor: dict[int, list[int]] = {}
   for k, (dr, dc) in DIRS.items():
      rs, cs = np.nonzero(direction == k)
      rr, cc = rs + dr, cs + dc
      ok = (rr >= 0) & (rr < h) & (cc >= 0) & (cc < w)
      for a, b in zip((rs * w + cs)[ok], (rr * w + cc)[ok]):
         donor.setdefault(int(b), []).append(int(a))
   mask = np.zeros(h * w, dtype=bool)
   stack = [int(seed[0] * w + seed[1])]
   while stack:
      cur = stack.pop()
      if mask[cur]:
         continue
      mask[cur] = True
      stack.extend(donor.get(cur, ()))
   return mask.reshape(h, w)


def terrain_geometry(z: np.ndarray, dy: float, dx: float) -> dict:
   gy, gx = np.gradient(z, dy, dx)
   slope = np.degrees(np.arctan(np.hypot(gx, gy)))
   aspect = np.degrees(np.arctan2(-gx, gy)) % 360.0
   curv = np.gradient(gx, dx, axis=1) + np.gradient(gy, dy, axis=0)
   return {"slope": slope, "aspect": aspect, "curv": curv, "gx": gx, "gy": gy}


def trace_downstream(direction: np.ndarray, start: tuple[int, int], max_steps: int = 20000) -> list[tuple[int, int
    ]]:
   """自 start 沿 D8 顺流追踪，返回像元序列。"""
   h, w = direction.shape
   path = [(int(start[0]), int(start[1]))]
   cur = path[0]
   for _ in range(max_steps):
      k = int(direction[cur[0], cur[1]])
      if k == 0:
         break
      dr, dc = DIRS[k]
      nxt = (cur[0] + dr, cur[1] + dc)
      if not (0 <= nxt[0] < h and 0 <= nxt[1] < w):
         break
      path.append(nxt)
      cur = nxt
   return path


def trace_upstream_mainstem(direction: np.ndarray, acc: np.ndarray, start: tuple[int, int],
                            min_acc: float = 50.0, max_steps: int = 20000) -> list[tuple[int, int]]:
   """自出口向上游追踪主河道（每次选择汇流面积最大的来水像元）。"""
   h, w = direction.shape
   path = [(int(start[0]), int(start[1]))]
   cur = path[0]
   for _ in range(max_steps):
      best = None
      best_acc = -np.inf
      for k, (dr, dc) in DIRS.items():
         rr, cc = cur[0] - dr, cur[1] - dc
         if 0 <= rr < h and 0 <= cc < w and direction[rr, cc] == k and acc[rr, cc] >= min_acc:
            if acc[rr, cc] > best_acc:
               best_acc = float(acc[rr, cc])
               best = (int(rr), int(cc))
      if best is None:
         break
      path.append(best)
      cur = best
   return path


def channel_profile(z: np.ndarray, direction: np.ndarray, acc: np.ndarray,
                    threshold: float) -> tuple[np.ndarray, np.ndarray]:
   """河道栅格（汇流累积 ≥ threshold）及其顺流距离场（以像元为单位）。"""
   river = acc >= threshold
   h, w = z.shape
   dist = np.where(river, 0.0, np.nan)
   queue = [(int(r), int(c)) for r, c in np.argwhere(river)]
   while queue:
```

```
        r, c = queue.pop()
        k = int(direction[r, c])
        if k == 0:
            continue
        dr, dc = DIRS[k]
        rr, cc = r + dr, c + dc
        if 0 <= rr < h and 0 <= cc < w:
            step = float(np.hypot(dr, dc))
            if np.isnan(dist[rr, cc]):
                dist[rr, cc] = dist[r, c] + step
                queue.append((rr, cc))
            elif dist[rr, cc] < dist[r, c] + step - 1e-9:
                dist[rr, cc] = dist[r, c] + step
                queue.append((rr, cc))
    return river, dist


def slope_horn(z: np.ndarray, cell: tuple[float, float]) -> np.ndarray:
    """Horn 法坡度（正切值，3×3 加权差分）。

    与二维引擎（B2 移植）的同名函数一致，供集水区坡度统计等使用；
    返回 tanβ，调用方自行 arctan 转角度。
    """
    dy, dx = cell
    a = np.where(np.isfinite(z), z, np.nan)
    zp = np.pad(a, 1, mode="edge")
    dzdx = ((zp[2:, 2:] + 2 * zp[1:-1, 2:] + zp[:-2, 2:]) -
            (zp[2:, :-2] + 2 * zp[1:-1, :-2] + zp[:-2, :-2])) / (8 * dx)
    dzdy = ((zp[2:, 2:] + 2 * zp[2:, 1:-1] + zp[2:, :-2]) -
            (zp[:-2, 2:] + 2 * zp[:-2, 1:-1] + zp[:-2, :-2])) / (8 * dy)
    return np.hypot(dzdx, dzdy)
```

`mmlib/terrain_prep.py` DEM 填洼与网格准备

```
"""DEM 地形派生量的一次性预计算与缓存。

产出（写入 io_data.CACHE）：
    filled.npy 填洼后高程（priority-flood，eps=1e-3 递增梯度）
    dir.npy    D8 流向编码 1-8（0=无下游；与 mmlib.terrain.DIRS 同约定）
    acc.npy    D8 汇流累积（像元数）
    recv.npy   展平的 D8 接收者索引（-1=流出域外，供二维引擎使用）
    slink.npy  接收者链接坡降 (z_i - z_recv)/dist（无量纲）
    distlink.npy 接收者链接距离（m）
    terrain_meta.json 网格与像元尺寸登记

设计说明：fill/d8/acc 与既有脚本（step04/trigger/explore_terrain）读取的
缓存文件名逐一同名兼容；recv/slink/distlink 是二维过程引擎
（step04b_sim2d.py）需要的派生量，与 B2 成果的栅格运动波引擎同构。
全部量由附件7 DEM 唯一确定，属确定性预计算，因此按缓存复用。
"""
from __future__ import annotations

import json
import hashlib

import numpy as np

from . import io_data as io
from . import terrain as tr

LAT_REF = 28.4 # 像元尺寸换算参考纬度（与 step04 场量面积口径一致）
TERRAIN_VERSION = "metric-d8-v3"


def cell_meters(grid: dict) -> tuple[float, float]:
    """经纬网格 → 米制像元尺寸 (dy_m, dx_m)。"""
    dy = abs(float(grid["dlat"])) * 110574.0
    dx = abs(float(grid["dlon"])) * 111320.0 * np.cos(np.radians(LAT_REF))
    return dy, dx


def dir_to_recv(direction: np.ndarray) -> np.ndarray:
    """D8 编码 → 展平接收者索引（流出域外/无下游记 -1）。"""
    ny, nx = direction.shape
    recv = np.full(ny * nx, -1, dtype=np.int64)
    for k, (dr, dc) in tr.DIRS.items():
        rs, cs = np.nonzero(direction == k)
        rr, cc = rs + dr, cs + dc
        ok = (rr >= 0) & (rr < ny) & (cc >= 0) & (cc < nx)
        recv[rs[ok] * nx + cs[ok]] = rr[ok] * nx + cc[ok]
    return recv


def link_geometry(filled: np.ndarray, recv: np.ndarray,
                  cell: tuple[float, float]) -> tuple[np.ndarray, np.ndarray]:
    """接收者链接的坡降与距离（米）。无下游处为 NaN。"""
    ny, nx = filled.shape
    dy, dx = cell
    flat = filled.ravel()
    idx = np.arange(ny * nx)
    has = recv >= 0
    srow, scol = np.divmod(idx[has], nx)
```

```
    rrow, rcol = np.divmod(recv[has], nx)
    dist = np.hypot((rrow - srow) * dy, (rcol - scol) * dx)
    slink = np.full(ny * nx, np.nan)
    slink[has] = (flat[idx[has]] - flat[recv[has]]) / dist
    return slink.reshape(ny, nx), dist


def prepare(dem: np.ndarray, grid: dict) -> dict:
    """对给定 DEM 网格完成 fill/d8/acc/recv/slink/distlink 计算（不落盘）。"""
    cell = cell_meters(grid)
    filled = tr.fill_sinks(dem.astype("float64"))
    direction = tr.d8(filled, cell)
    acc = tr.flow_accumulation(direction)
    recv = dir_to_recv(direction)
    slink, dist_flat = link_geometry(filled, recv, cell)
    dist = np.full(dem.size, np.nan)
    has = recv >= 0
    dist[has] = dist_flat
    return {"filled": filled, "dir": direction, "acc": acc,
            "recv": recv, "slink": slink, "dist": dist.reshape(dem.shape),
            "cell": cell, "grid": grid}


def ensure_terrain_cache(force: bool = False) -> dict:
    """加载（或首次计算并缓存）30 m 主网格的全部地形派生量。"""
    dem, grid = io.load_dem()
    meta_path = io.CACHE / "terrain_meta.json"
    meta = json.loads(meta_path.read_text(encoding="utf-8")) if meta_path.exists() else {}
    dem_hash = hashlib.sha256(dem.tobytes()).hexdigest()
    need = force or meta.get("algorithm_version") != TERRAIN_VERSION \
        or meta.get("dem_sha256") != dem_hash or meta.get("grid") != grid \
        or any(not (io.CACHE / name).exists() for name in
               ("filled.npy", "dir.npy", "acc.npy", "recv.npy", "slink.npy", "dist.npy"))
    if need:
        tp = prepare(dem, grid)
        np.save(io.CACHE / "filled.npy", tp["filled"])
        np.save(io.CACHE / "dir.npy", tp["dir"].astype("int8"))
        np.save(io.CACHE / "acc.npy", tp["acc"])
        np.save(io.CACHE / "recv.npy", tp["recv"])
        np.save(io.CACHE / "slink.npy", tp["slink"])
        np.save(io.CACHE / "dist.npy", tp["dist"])
        io.write_json(io.CACHE / "terrain_meta.json",
                      {"shape": list(dem.shape), "cell_dy_m": tp["cell"][0],
                       "cell_dx_m": tp["cell"][1],
                       "lat_ref": LAT_REF,
                       "fill": "priority-flood eps=1e-3（mmlib.terrain.fill_sinks）",
                       "d8": "metric maximum drop/link length; codes 1-8, 0=no receiver",
                       "algorithm_version": TERRAIN_VERSION,
                       "dem_sha256": dem_hash, "grid": grid})
        return tp
    return {"filled": np.load(io.CACHE / "filled.npy"),
            "dir": np.load(io.CACHE / "dir.npy"),
            "acc": np.load(io.CACHE / "acc.npy"),
            "recv": np.load(io.CACHE / "recv.npy"),
            "slink": np.load(io.CACHE / "slink.npy"),
            "dist": np.load(io.CACHE / "dist.npy"),
            "cell": cell_meters(grid), "grid": grid}


def prepare_coarse(k: int = 3) -> dict:
    """k×k 块均值聚合后的粗网格地形量（90 m 分辨率重演；每次现算不缓存）。"""
    dem, grid = io.load_dem()
    ny, nx = dem.shape
    ny2, nx2 = ny // k * k, nx // k * k
    b = np.where(np.isfinite(dem), dem, np.nan)[:ny2, :nx2]
    with np.errstate(all="ignore"):
        dem9 = np.nanmean(b.reshape(ny2 // k, k, nx2 // k, k), axis=(1, 3))
    grid9 = dict(grid)
    grid9["lon0"] = float(grid["lon0"]) + (k-1)/2*float(grid["dlon"])
    grid9["lat0"] = float(grid["lat0"]) + (k-1)/2*float(grid["dlat"])
    grid9["dlon"] = float(grid["dlon"]) * k
    grid9["dlat"] = float(grid["dlat"]) * k
    return prepare(dem9, grid9)
```

`mmlib/trigger.py` 床面力平衡与触发判别

```
"""泥石流触发判据：三条件耦合的水源—地形—物质链。

统一框架
--------
1. 水源供给 Q_w(t)：SCS-CN 产流（降雨）+ 能量平衡融水（冰/雪），均折算为沟口流量。
2. 地形起动门槛 Q_c(t)：Manning 流深 + 库仑床面抗力的渠道松散体起动判据，
   逐沟由矢量属性（面积、主沟长、高差、比降、扇宽/扇长）计算。
3. 触发指数 T(t) = Q_w / Q_c；仅在 water_controlled 且 Q_c>0 时定义。

标定
-----
旧版把 Q_c 中的集总系数 k 由 2026-08-26 事件窗口反标定，使窗口内 T 峰值为 1。
该 k 只作为 v4.3 旧方法审计对照，不得进入正式触发状态、问题三阈值或预测区间。

集水区比降口径
--------------
本模块提供三种口径，用途严格区分（详见 建模成果/问题2.md §2.4.2）：

```

```
1. `geometric`：√像元数×30 m×1.6 的几何近似，**只作面积—形状粗估，不进入 q_c**；
2. `traced`：自出口按最大汇流上游追迹的像元链，用于空间可靠性检验；
3. `centerline`：问题 4 中心线（源区冰厚高值像元 → 口岸），**q_c 的唯一计算口径**，
   并使问题 2 与问题 4 共用同一沟长/比降。

早期实现把口径 1 的比降再乘 3 后代入 q_c，该系数既无来源也未在论文披露，
会使 q_c 偏低约 4.2 倍；现已删除，改由口径 3 直接计算。

沟宽与 κ
--------
A_charge = q_c·B/κ 中，B 优先取扇宽/4，缺测时取 B = 9√A (km²)（百米量级经验式）；
κ = 6 为"有效过流宽度占全沟宽的比例"的倒数式集总系数。两者均为**显式假设**；
v5.0 不用事件窗 k 吸收其不确定度，而是分别报告参数情景与状态分支。
"""
from __future__ import annotations

import numpy as np
import pandas as pd

from . import io_data as io

G = 9.80665
RHO_W = 1000.0
LF = 334.0       # 融化潜热 MJ/m³（水当量），仅用于 MJ·m⁻² 口径的换算
LF_J = 334000.0 # 融化潜热 J/kg（用于 W·m⁻² 通量口径的换算）

# ---- 沟道起动判据中的显式假设常数（论文参数表须逐项列出，不得隐去）----
N_MAN_GULLY = 0.06 # 沟道曼宁糙率（砾石质沟道）
TAN_PHI = 0.55   # 松散体有效内摩擦角正切（约 28.8°）
ZB = 1.0         # 可蚀层厚度 m
DRHO = 1050.0    # 饱和松散体与水的密度差 kg/m³
KAPPA_Q = 6.0    # Q_c = q_c·B/κ 的集总系数（有效过流宽度比例的倒数式）
KAPPA_WIDTH = 9.0 # 缺测沟宽经验式 B = 9·√A(km²)（百米量级）
EPS_S = 1e-6     # 摩擦平衡状态的无量纲比降容差
CN0 = 85.0       # 独立声明的SCS-CN工程情景，不由土地覆盖类码推导
API_TRAIN_YEARS = (2021, 2022, 2023, 2024, 2025)
API_WARM_MONTHS = (6, 7, 8, 9)


def _density_difference(bed_solid_fraction: float) -> float:
    """Return rho_sat-rho_w for rho_sat=1000+1650*Cv."""
    return max(float(bed_solid_fraction) * 1650.0, 0.0)

# ---- 集水区比降口径 ----
SLOPE_DEFS = ("geometric", "traced", "centerline")
# ------------------------------------------------------------------- 沟道参数
def gully_table() -> pd.DataFrame:
    """84 条沟道的几何与地形参数（含派生的沟口流量门槛）。"""
    haz = io.load_hazard_points()
    df = haz.copy()
    df["面积_km2"] = pd.to_numeric(df["area_km2"], errors="coerce")
    df["主沟长_m"] = pd.to_numeric(df["length_m"], errors="coerce")
    df["高差_m"] = pd.to_numeric(df["relief_m"], errors="coerce")
    df["比降"] = pd.to_numeric(df["grad"], errors="coerce")
    df["沟口高程_m"] = pd.to_numeric(df["mouth_m"], errors="coerce")
    df["流域最高高程_m"] = pd.to_numeric(df["zmax_m"], errors="coerce")
    fan_w = pd.to_numeric(df["fan_w"], errors="coerce") if "fan_w" in df else pd.Series(np.nan, index=df.index)
    fan_l = pd.to_numeric(df["fan_l"], errors="coerce") if "fan_l" in df else pd.Series(np.nan, index=df.index)
    df["扇宽_m"] = fan_w
    df["扇长_m"] = fan_l
    # 沟道宽度：优先用扇宽/4（扇体远宽于沟槽），否则由面积经验式 B = 9·√A(km²) 估计。
    # 该式为显式经验假设，非推导结果，论文须按"经验式取定"表述（见模块 docstring）。
    width = (fan_w / 4.0).where(fan_w.between(20, 4000))
    width = width.fillna(KAPPA_WIDTH * np.sqrt(df["面积_km2"].clip(lower=0.01)))
    df["沟宽_m"] = width.clip(3.0, 300.0)
    mechanics = [_mechanics(s) for s in df["比降"]]
    df["mechanical_regime"] = [m["regime"] for m in mechanics]
    df["mechanical_reason"] = [m["reason"] for m in mechanics]
    df["h_crit_m"] = [m["h_crit_m"] for m in mechanics]
    df["tau_w_c_Pa"] = [m["tau_w_c_Pa"] for m in mechanics]
    df["tau_g_parallel_Pa"] = [m["tau_g_parallel_Pa"] for m in mechanics]
    df["tau_r_Pa"] = [m["tau_r_Pa"] for m in mechanics]
    df["q_crit_m2s"] = [m["q_crit_m2s"] for m in mechanics]
    df["Q_crit_m3s"] = df["q_crit_m2s"] * df["沟宽_m"] / KAPPA_Q
    # 旧式无床体沿坡自重项，只用于关闭/解释历史结果。
    df["q_crit_old_m2s"] = [_q_crit_old(s) for s in df["比降"]]
    df["Q_crit_old_m3s"] = df["q_crit_old_m2s"] * df["沟宽_m"] / KAPPA_Q
    return df


def _mechanics(slope: float, bed_solid_fraction: float = DRHO / 1650.0) -> dict:
    """完整无黏聚、无超孔压床层的力学分支与临界量。

    几何约定：z_b 为竖直床厚，应力和临界量均按单位坡面面积报告；
    h 为垂直床面的水深。令 S=tanθ，则
    τ_g,||=Δρ g z_b sinθ cosθ，
    τ_r=Δρ g z_b cos²θ tanφ，
    τ_w,c=Δρ g z_b cos²θ(tanφ−tanθ)。
    当 tanθ 超过 tanφ 时，声明模型下不需要外部水流即可自失稳；这不是现实安全性结论。
```

```
    """
    invalid = {
        "regime": "not_evaluable",
        "reason": "非有限或非正比降",
        "h_crit_m": None,
        "tau_w_c_Pa": None,
        "tau_g_parallel_Pa": None,
        "tau_r_Pa": None,
        "delta_rho_kg_m3": None,
        "q_crit_m2s": None,
    }
    if (
        not np.isfinite(slope)
        or slope <= 1e-4
        or not np.isfinite(bed_solid_fraction)
        or bed_solid_fraction <= 0.0
        or bed_solid_fraction > 1.0
    ):
        return invalid
    s = float(slope)
    cos2 = 1.0 / (1.0 + s * s)
    sin_theta = s / np.sqrt(1.0 + s * s)
    cos_theta = 1.0 / np.sqrt(1.0 + s * s)
    delta_rho = _density_difference(bed_solid_fraction)
    tau_g = delta_rho * G * ZB * sin_theta * cos_theta
    tau_r = delta_rho * G * ZB * cos2 * TAN_PHI
    net = TAN_PHI - s
    if net > EPS_S:
        regime = "water_controlled"
        reason = "tan(phi)-tan(theta)>epsilon_S"
        tau_wc = delta_rho * G * ZB * cos2 * net
        h = tau_wc / (RHO_W * G * sin_theta)
        q = h ** (5.0 / 3.0) * np.sqrt(s) / N_MAN_GULLY
    elif abs(net) <= EPS_S:
        regime = "friction_equilibrium"
        reason = "abs(tan(phi)-tan(theta))<=epsilon_S"
        tau_wc, h, q = 0.0, 0.0, 0.0
    else:
        regime = "gravity_unstable"
        reason = "tan(theta)-tan(phi)>epsilon_S"
        tau_wc, h, q = 0.0, 0.0, 0.0
    return {
        "regime": regime,
        "reason": reason,
        "h_crit_m": float(h),
        "tau_w_c_Pa": float(tau_wc),
        "tau_g_parallel_Pa": float(tau_g),
        "tau_r_Pa": float(tau_r),
        "delta_rho_kg_m3": float(delta_rho),
        "q_crit_m2s": float(q),
    }


def _q_crit_unit(slope: float, area_km2: float = 0.05,
                 bed_solid_fraction: float = DRHO / 1650.0) -> float:
    """单宽临界流量；零门槛分支返回 0，不能用于定义 T=Q/Q_c。"""
    del area_km2
    return float(_mechanics(slope, bed_solid_fraction)["q_crit_m2s"])


def _q_crit_old(slope: float, area_km2: float = 0.05,
                bed_solid_fraction: float = DRHO / 1650.0) -> float:
    """v4.3 旧式无床体沿坡自重项的单宽临界流量，仅供审计。"""
    del area_km2
    if not np.isfinite(slope) or slope <= 1e-4:
        return np.nan
    if not np.isfinite(bed_solid_fraction) or not (0.0 < bed_solid_fraction <= 1.0):
        return np.nan
    s = float(slope)
    delta_rho = _density_difference(bed_solid_fraction)
    tau_r = delta_rho * G * ZB * TAN_PHI / (1.0 + s * s)
    h = tau_r / (RHO_W * G * s)
    return float(h ** (5.0 / 3.0) * np.sqrt(s) / N_MAN_GULLY)


# ---------------------------------------------------------------------- 产流
def catchment_glacier_fraction(return_mask: bool = False):
    """口岸所在集水区的冰川覆盖率与高程特征（用于融水面积权重）。

    使用附件3的RGI6编目边界；高程带仅作审计对照，不进入融水面积。
    RGI6不是2026年实测冰川或积雪边界，编目年代差异另行披露。

    return_mask=True 时额外返回集水区掩膜（供静态特征统计复用，避免重算）。
    """
    from . import terrain as tr, geodesy as gd

    dem, grid = io.load_dem()
    z = dem.astype("float64")
    d = np.load(io.CACHE / "dir.npy")
    acc = np.load(io.CACHE / "acc.npy")
    r, c = io.lonlat_to_rowcol(grid, io.EVENT["port_lon"], io.EVENT["port_lat"])
    ri, ci = int(round(r)), int(round(c))
    # 取距口岸最近的主河道像元（汇流累积 ≥ 5×10⁵）作为集水区出口，
    # 避免沿程向下游搜索时误取更远处的干流像元
    cand = np.argwhere(acc >= 5e5)
```

```
    if len(cand):
        dist = np.hypot(cand[:, 0] - ri, cand[:, 1] - ci)
        best = tuple(int(v) for v in cand[int(np.argmin(dist))])
        best_acc = float(acc[best])
    else:
        best, best_acc = (ri, ci), float(acc[ri, ci])
    mask = tr.upstream_mask(d, best)
    from .terrain_prep import cell_meters
    lat_c = 28.4
    dy_cell, dx_cell = cell_meters(grid)
    cell_area = dy_cell * dx_cell
    area = float(mask.sum()) * cell_area / 1e6
    zc = z[mask]

    ice = z >= 4800.0
    # 附件（4）源区冰川像元
    yy, xx = np.mgrid[0:z.shape[0], 0:z.shape[1]]
    lon_g = float(grid["lon0"]) + xx * float(grid["dlon"])
    lat_g = float(grid["lat0"]) - yy * abs(float(grid["dlat"]))
    src = np.zeros_like(mask)
    try:
        g = io.load_glacier_nc()
        gt = g["geotransform"]
        valid = np.isfinite(g["thickness"]) & (g["thickness"] > 1.0)
        gy, gx = np.nonzero(valid)
        for row, col in zip(gy, gx):
            e = gt[0] + (col + 0.5) * gt[1]
            n = gt[3] + (row + 0.5) * gt[5]
            lo, la = gd.utm_to_lonlat(e, n, 45)
            rr = int(round((float(grid["lat0"]) - la) / abs(float(grid["dlat"]))))
            cc = int(round((lo - float(grid["lon0"])) / float(grid["dlon"])))
            if 0 <= rr < z.shape[0] and 0 <= cc < z.shape[1]:
                src[rr, cc] = 1
    except (KeyError, ValueError, OSError) as exc:
        raise RuntimeError("glacier coverage mapping failed; cannot silently omit attachment 4") from exc
    from .glacier_masks import rgi6_mask
    boundary_mask, boundary_meta = rgi6_mask(grid, z.shape)
    cover = boundary_mask & mask
    f_gl = float(cover.sum()) / max(int(mask.sum()), 1)

    # 三种沟长—比降口径并列（用途严格区分，见模块 docstring）。
    # 几何近似：√像元数×30 m×1.6，仅作面积—形状粗估。
    L_geo = float(np.sqrt(mask.sum()) * 30.0 * 1.6)
    z_range = float(np.nanmax(zc) - np.nanmin(zc))
    # 追迹口径：自出口按最大汇流上游追迹的像元链
    from . import terrain as _tr
    stem = _tr.trace_upstream_mainstem(d, acc, best, min_acc=2000.0)
    stem_z = np.array([z[p] for p in stem], dtype="float64")
    steps = np.diff(np.asarray(stem), axis=0)
    dy = abs(float(grid["dlat"])) * 110574.0
    dx = abs(float(grid["dlon"])) * 111320.0 * np.cos(np.radians(lat_c))
    L_trace = float(np.hypot(steps[:, 0] * dy, steps[:, 1] * dx).sum())
    S_trace = float((stem_z[-1] - stem_z[0]) / max(L_trace, 1.0)) if len(stem) > 5 else np.nan
    out = {
        "出口像元": [int(best[0]), int(best[1])],
        "出口汇流累积": int(best_acc),
        "集水区像元数": int(mask.sum()),
        "集水区面积_km2": round(area, 3),
        "像元面积_m2": round(cell_area, 1),
        # 高程带代理与冰厚场源区两个分量必须分别报告：二者量级差近两个数量级，
        # 并集值几乎完全由高程带决定，不能称为"冰川覆盖率"（论文失效点 9）。
        "冰川覆盖率": round(f_gl, 5),
        "覆盖率分解": {
            "高程带代理_ge4800m": round(float((ice & mask).sum() / mask.sum()), 5),
            "冰厚场源区": round(float(((src > 0) & mask).sum() / mask.sum()), 5),
            "旧高程代理并集_仅审计": round(float(((ice | (src > 0)) & mask).sum()/mask.sum()),5),
            "RGI6编目边界": round(f_gl, 5),
        },
        "冰川面积来源": boundary_meta,
        "冰川覆盖语义": "RGI6编目边界栅格化；非2026年消融区、积雪或实时冰川范围",
        "冰川区平均高程_m": round(float(np.nanmean(zc[cover[mask]])) if f_gl > 0 else float(np.nanmax(zc)), 1),
        "集水区平均高程_m": round(float(np.nanmean(zc)), 1),
        "集水区最高高程_m": round(float(np.nanmax(zc)), 1),
        "集水区最低高程_m": round(float(np.nanmin(zc)), 1),
        "集水区高差_m": round(z_range, 1),
        # 几何近似口径（仅粗估，不进入 q_c）
        "集水区主沟长_m": round(L_geo, 1),
        "集水区比降": round(z_range / max(L_geo, 1.0), 5),
        # 追迹口径（空间可靠性检验用）
        "追迹主沟长_m": round(L_trace, 1),
        "追迹比降": round(S_trace, 5) if np.isfinite(S_trace) else None,
    }
    return (out, mask) if return_mask else out


# 类码字典未核实，本版停用土地覆盖物理权重，仅保留原码统计。
def catchment_static_features(base: dict | None = None) -> dict:
    """集水区静态特征：坡度统计 + 土地覆盖构成 + 松散物质可蚀性指数。

    B1 成果直接携带 02_静态特征.json 而未随附生成代码，从原始数据冷启动
    即 KeyError；本函数补全其生产链。坡度用 Horn 法（与二维引擎同式，
```

```
    在填洼 DEM 上计算），土地覆盖（附件5, 2023）按 DEM 像元中心最近邻
    重采样，类码未核实，物理可蚀性指数返回 None。
    """
    from . import terrain as tr

    if base is None:
        base, mask = catchment_glacier_fraction(return_mask=True)
    else:
        _, mask = catchment_glacier_fraction(return_mask=True)
    dem, grid = io.load_dem()
    zf = np.load(io.CACHE / "filled.npy")
    # 与 terrain_prep.cell_meters 同式（纬度参考 28.4°）
    dy = abs(float(grid["dlat"])) * 110574.0
    dx = abs(float(grid["dlon"])) * 111320.0 * np.cos(np.radians(28.4))
    slp = np.degrees(np.arctan(tr.slope_horn(zf.astype("float64"), (dy, dx))))
    s = slp[mask & np.isfinite(slp)]

    # 土地覆盖 → DEM 网格（最近邻，按像元中心经纬度）
    lc, lg = io.load_landcover()
    yy, xx = np.nonzero(mask)
    lon_c = float(grid["lon0"]) + xx * float(grid["dlon"])
    lat_c = float(grid["lat0"]) + yy * float(grid["dlat"])
    rr_f = (lat_c - float(lg["lat0"])) / float(lg["dlat"])
    cc_f = (lon_c - float(lg["lon0"])) / float(lg["dlon"])
    rr = np.round(rr_f).astype(int)
    cc = np.round(cc_f).astype(int)
    ok = (rr >= 0) & (rr < lc.shape[0]) & (cc >= 0) & (cc < lc.shape[1])
    lcv = np.asarray(lc[rr[ok], cc[ok]]).astype(int)
    vals, cnts = np.unique(lcv, return_counts=True)
    comp = {str(int(v)): int(n) for v, n in zip(vals, cnts)}
    # No physical weights without the product's class dictionary.
    erod = None

    L_km = float(base["集水区主沟长_m"]) / 1000.0
    return {
        "集水区面积_km2": base["集水区面积_km2"],
        "平均坡度_deg": round(float(np.mean(s)), 4),
        "坡度p90_deg": round(float(np.percentile(s, 90)), 4),
        "陡坡面积比_gt30deg": round(float((s > 30.0).mean()), 5),
        "冰川面积_km2": round(float(base["集水区面积_km2"]) * float(base["冰川覆盖率"]), 3),
        "冰川覆盖率": base["冰川覆盖率"],
        "松散物质可蚀性指数": erod,
        "可蚀性状态": "unavailable: supplied class dictionary not verified; physical weighting disabled: class
            dictionary unverified",
        "可蚀性权重_先验": {},
        "土地覆盖构成": comp,
        "土地覆盖有效比例": float(ok.mean()),
        "土地覆盖名称": {str(k): v for k, v in io.LC_NAMES.items()},
        "土地覆盖重采样": "附件5(2023) 按 DEM 像元中心最近邻；越界像元不冒充0类；原码语义未确认",
        "平均高程_m": base["集水区平均高程_m"],
        "高差_m": base["集水区高差_m"],
        "比降": base["集水区比降"],
        "主沟长_m": base["集水区主沟长_m"],
        "流域形状系数": round(float(base["集水区面积_km2"]) / max(L_km ** 2, 1e-6), 5),
    }


def api_climate_thresholds(feat: pd.DataFrame) -> dict:
    """Return the frozen warm-season API terciles used for AMC classification."""
    api = pd.to_numeric(feat["API"], errors="coerce")
    preset = feat.attrs.get("api_quantiles")
    mask = np.isin(feat.index.year, API_TRAIN_YEARS) & np.isin(
        feat.index.month, API_WARM_MONTHS
    )
    training_api = api.loc[mask]
    baseline = training_api.where(np.isfinite(training_api) & training_api.ge(0)).dropna()
    if preset is not None:
        q1, q3 = (float(value) for value in preset)
        source = "frozen feat.attrs['api_quantiles']"
    else:
        if len(baseline) < 100:
            raise ValueError(
                "API warm-season training sample is too small; refusing an all-season fallback"
            )
        q1, q3 = (float(value) for value in baseline.quantile([1 / 3, 2 / 3]))
        source = "2021-2025 June-September observed training sample"
    if not (np.isfinite(q1) and np.isfinite(q3) and 0 <= q1 <= q3):
        raise ValueError(f"invalid API terciles: {(q1, q3)!r}")
    return {
        "q1": q1,
        "q3": q3,
        "source": source,
        "training_years": list(API_TRAIN_YEARS),
        "training_months": list(API_WARM_MONTHS),
        "training_valid_hours": int(len(baseline)),
        "index_start": str(feat.index.min()),
        "index_end": str(feat.index.max()),
    }


def event_rain_runoff(
    feat: pd.DataFrame, cn: float = CN0, dry_hours: int = 6,
) -> pd.DataFrame:
```

```
    """Event-cumulative SCS-CN with causal, observed-zero event boundaries.

    The index must be sorted, unique and on the hour. Gaps and invalid rain
    invalidate the event history; only ``dry_hours`` consecutive observed
    P=0 hours recover a known boundary. Positive rain before recovery has no
    event_id. An event freezes CN from API exactly one hour before its start;
    unavailable/invalid antecedent API makes all its positive rain unevaluable.
    Zero-rain runoff is always zero, including during unknown history.

    States: unknown_boundary, invalid_rain, dry_boundary, event, event_api_missing.
    ``Q`` is cumulative direct runoff in mm, not discharge. Unknown history has
    NaN cumulative_P/Q; a known boundary has zero cumulative_P/Q and no event_id.
    On the confirming dry hour the previous event is cleared; boundary_confirmed
    marks that hour only. Event IDs are local to this complete chronological call.
    No calendar reset or future event-closure requirement is imposed.
    """
    index = feat.index
    if not isinstance(index, pd.DatetimeIndex):
        raise ValueError("rainfall requires a DatetimeIndex")
    if index.hasnans or not index.is_unique or not index.is_monotonic_increasing:
        raise ValueError("rainfall index must be monotonic, unique and non-missing")
    if ((index.minute != 0) | (index.second != 0)
            | (index.microsecond != 0) | (index.nanosecond != 0)).any():
        raise ValueError("rainfall index must contain whole-hour timestamps")
    if (isinstance(dry_hours, (bool, np.bool_))
            or not isinstance(dry_hours, (int, np.integer)) or dry_hours < 1):
        raise ValueError("dry_hours must be a positive integer")
    cn = float(cn)
    if not np.isfinite(cn) or not 0 < cn <= 100:
        raise ValueError("base CN must be finite and in (0, 100]")
    p = pd.to_numeric(feat["P"], errors="coerce").to_numpy(dtype=float, na_value=np.nan)
    api = pd.to_numeric(feat["API"], errors="coerce").to_numpy(dtype=float, na_value=np.nan)
    meta = api_climate_thresholds(feat)
    q1, q3 = meta["q1"], meta["q3"]
    known_boundary = False
    dry_run = event_count = 0
    event_id = None
    event_cn = event_api = cumulative_p = cumulative_q = np.nan
    rows = []
    for i, value in enumerate(p):
        gap = i > 0 and index[i] - index[i - 1] != pd.Timedelta(hours=1)
        if gap:
            known_boundary, dry_run, event_id = False, 0, None
            event_cn = event_api = cumulative_p = cumulative_q = np.nan
        started = confirmed = False
        runoff = np.nan
        if not np.isfinite(value) or value < 0:
            known_boundary, dry_run, event_id = False, 0, None
            event_cn = event_api = cumulative_p = cumulative_q = np.nan
            state = "invalid_rain"
        elif value == 0:
            runoff = 0.0
            dry_run += 1
            if dry_run == dry_hours:
                known_boundary, confirmed, event_id = True, True, None
                event_cn = event_api = np.nan
                cumulative_p = cumulative_q = 0.0
            state = (
                "event" if np.isfinite(event_cn) else "event_api_missing"
            ) if event_id is not None else (
                "dry_boundary" if known_boundary else "unknown_boundary"
            )
        else:
            dry_run = 0
            if not known_boundary:
                state = "unknown_boundary"
            else:
                if event_id is None:
                    event_count += 1
                    event_id, started = event_count, True
                    event_api = api[i - 1]
                    cumulative_p = 0.0
                    if np.isfinite(event_api) and event_api >= 0:
                        factor = 0.75 if event_api < q1 else 1.15 if event_api > q3 else 1.0
                        event_cn = float(np.clip(cn * factor, 30, 98))
                        cumulative_q = 0.0
                    else:
                        event_cn = cumulative_q = np.nan
                cumulative_p += value
                if not np.isfinite(cumulative_p):
                    raise ValueError("event cumulative rainfall overflow")
                if np.isfinite(event_cn):
                    s_max = 25400.0 / event_cn - 254.0
                    excess = cumulative_p - 0.2 * s_max
                    new_q = excess / (1.0 + s_max / excess) if excess > 0 else 0.0
                    runoff = new_q - cumulative_q
                    cumulative_q = new_q
                    state = "event"
                else:
                    state = "event_api_missing"
        rows.append((runoff, state, event_id, event_cn, event_api,
                     cumulative_p, cumulative_q, dry_run, known_boundary,
                     started, confirmed, gap))
    out = pd.DataFrame(rows, index=index, columns=[
        "runoff_mm", "state", "event_id", "event_CN", "event_API",
        "cumulative_P", "Q", "dry_hours", "boundary_known",
        "event_start", "boundary_confirmed", "time_gap",
```

```
    ])
    out["event_id"] = pd.array(out["event_id"], dtype="Int64")
    out.attrs.update({
        "api_quantiles": (q1, q3), "api_quantiles_record": meta,
        "rainfall_event_protocol": {
            "dry_hours": int(dry_hours), "dry_rule": "finite P == 0",
            "CN_rule": "freeze at event start from API exactly one hour earlier",
            "cumulative_units": "mm", "calendar_reset": False,
        },
    })
    return out


def rain_runoff(
    feat: pd.DataFrame, cn: float = CN0, dry_hours: int = 6,
) -> pd.Series:
    """Hourly runoff increments (mm); event protocol/audit: event_rain_runoff."""
    return event_rain_runoff(feat, cn, dry_hours)["runoff_mm"].rename(None)


def melt_water(feat: pd.DataFrame, temp_offset: float = -7.5) -> pd.Series:
    """能量平衡融水（mm w.e./h）：有效融化能量 → 水当量，按源区气温调整有效时段。

    量纲：melt_energy 为辐射通量余量（W·m⁻²），故 1 h 的能量为 E×3600 J·m⁻²。
    化水当量深度 = E×3600 / (ρ_w·L_f)，ρ_w = 1000 kg·m⁻³、L_f = 334000 J·kg⁻¹，
    结果以 m 表示，乘 1000 得 mm。合并常数即 3600×1000 / (1000×334000) = 0.010778 mm 每 (W·m⁻²)。
    """
    energy = feat["melt_energy"]       # W·m⁻²; missing is not zero
    t_src = feat["Ta_2m"] + temp_offset
    # 源区气温低于 0 °C 时融化受抑制（指数过渡，避免硬截断）
    factor = 1.0 / (1.0 + np.exp(-(t_src - 0.0) / 1.5))
    mm_per_wm2 = 3600.0 / (RHO_W * LF_J) * 1000.0 # mm 每 (W·m⁻²)
    return energy * mm_per_wm2 * factor


# ---------------------------------------------------------------------- 触发指数
class TriggerModel:
    """正式水源—地形状态模型；旧事件 k 只能显式启用为审计模式。"""

    def __init__(self, area_km2: float, glacier_frac: float, q_crit_m2s: float,
                 width_m: float, cn: float = CN0, temp_offset: float = -7.5,
                 k: float = 1.0, mechanical_regime: str = "water_controlled",
                 use_legacy_k: bool = False):
        self.area = float(area_km2)
        self.f_gl = float(glacier_frac)
        self.q_crit = float(q_crit_m2s)
        self.width = float(width_m)
        self.cn = float(cn)
        self.temp_offset = float(temp_offset)
        self.k = float(k)
        self.mechanical_regime = str(mechanical_regime)
        self.use_legacy_k = bool(use_legacy_k)

    def discharge(self, feat: pd.DataFrame) -> pd.DataFrame:
        r = rain_runoff(feat, self.cn)
        m = melt_water(feat, self.temp_offset) * self.f_gl
        # Positional routing weights must not jump across missing clock hours.
        grid = pd.date_range(feat.index.min(), feat.index.max(), freq="h") if len(feat) else feat.index
        r, m = r.reindex(grid), m.reindex(grid)
        # 汇流：降雨按 1 h 滞后三角形权重，融水按 3 h 平滑
        w_rain = pd.Series([0.6, 0.3, 0.1], index=[0, 1, 2])
        q_rain = sum(r.shift(i) * w for i, w in w_rain.items())
        q_melt = m.rolling(3, min_periods=3).mean()
        water = (q_rain + q_melt) * self.area * 1000.0 / 3600.0 # m³/s
        out = pd.DataFrame({"Q_rain": q_rain * self.area * 1000 / 3600,
                            "Q_melt": q_melt * self.area * 1000 / 3600,
                            "Q_water": water}).reindex(feat.index)
        base_q = self.base_q_crit()
        out["Q_crit"] = base_q * (self.k if self.use_legacy_k else 1.0)
        if self.mechanical_regime == "water_controlled" and base_q > 0:
            out["T_index"] = out["Q_water"] / out["Q_crit"]
            hydraulic = pd.Series("not_evaluable", index=out.index, dtype=object)
            valid = (
                out["Q_water"].notna()
                & out["Q_crit"].notna()
                & out["Q_crit"].gt(0)
                & out["T_index"].notna()
            )
            hydraulic.loc[valid] = np.where(
                out.loc[valid, "T_index"] >= 1.0,
                "hydraulic_candidate",
                "below_threshold",
            )
            out["hydraulic_state"] = hydraulic
        else:
            out["T_index"] = np.nan
            hydraulic = pd.Series("inapplicable", index=out.index, dtype=object)
            hydraulic.loc[out["Q_water"].isna()] = "not_evaluable"
            out["hydraulic_state"] = hydraulic
        out["mechanical_regime"] = self.mechanical_regime
        out["legacy_k_used"] = self.use_legacy_k
        return out

    def base_q_crit(self) -> float:
        """未标定的起动流量 q_c·B/κ；零门槛分支返回 0。"""
```

```
        if self.mechanical_regime == "not_evaluable":
            return np.nan
        return float(self.q_crit * self.width / KAPPA_Q)

    def calibrate(self, feat: pd.DataFrame, window: tuple[str, str]) -> float:
        """旧方法审计：令事件窗口内触发指数峰值为 1，解出集总系数 k。

        此函数只在显式设置 ``use_legacy_k=True`` 后进入输出，不应被问题三调用。
        k = 窗口内 Q_w 峰值 / (q_c·B/κ)。零门槛或无效几何返回 NaN，不伪造有限标定。
        """
        d = self.discharge(feat)
        peak = float(d.loc[window[0]:window[1], "Q_water"].max())
        if not np.isfinite(peak) or peak <= 0:
            return 1.0
        base_q = self.base_q_crit()
        if not np.isfinite(base_q) or base_q <= 0:
            self.k = np.nan
            return self.k
        self.k = peak / base_q
        return self.k


def centerline_slope() -> dict:
    """问题 4 中心线的沟长与比降——q_c 的唯一计算口径。

    读 求解结果/数据/04_中心线校验.json（由 step04_simulation.py 先行产出）。
    该文件缺失时抛出明确错误，而不是静默回退到别的口径：口径混用正是
    "同一物理量出现多个互斥值"的根源，必须让依赖顺序的违约显式失败。
    """
    path = io.DATA_OUT / "04_中心线校验.json"
    if not path.exists():
        raise FileNotFoundError(
            "缺少 04_中心线校验.json：q_c 依赖问题 4 的中心线口径，"
            "请先运行 step04_simulation.py（run_all.py 已按此顺序排列）。"
        )
    import json
    d = json.loads(path.read_text(encoding="utf-8"))
    z = d["高程范围_m"]                # [沟口, 源头]
    L = float(d["中心线长度_km"]) * 1000.0
    drop = float(z[1]) - float(z[0])
    return {"沟长_m": L, "落差_m": drop, "比降": drop / max(L, 1.0),
            "来源": "问题4 中心线（04_中心线校验.json）"}


def default_model(feat: pd.DataFrame | None = None, calibrate: bool = False,
                  slope_def: str = "centerline") -> TriggerModel:
    """以口岸集水区为对象的默认触发模型。

    slope_def 只接受 "centerline"（默认，q_c 的正式口径）；其余两种口径
    由 `slope_variants()` 单独计算并用于敏感性/可靠性讨论，不进入正式模型。
    """
    if slope_def != "centerline":
        raise ValueError(
            f"q_c 只允许使用 centerline 口径（收到 {slope_def!r}）；"
            "三种口径并列见 slope_variants()，不得混用。"
        )
    info = catchment_glacier_fraction()
    sl = centerline_slope()
    # 集水区整体比降（中心线口径）→ 完整力学分支。不做任何未披露的放大。
    mechanics = _mechanics(sl["比降"])
    qc = float(mechanics["q_crit_m2s"]) if mechanics["q_crit_m2s"] is not None else np.nan
    w = KAPPA_WIDTH * np.sqrt(info["集水区面积_km2"])
    geometry = io.load_glacier_nc()
    valid = np.isfinite(geometry["thickness"]) & (geometry["thickness"] > 1.0)
    source_elevation = float(np.nanmean(geometry["dem"][valid]))
    # The source DEM meaning is uncertain; this is the declared unraised-DEM scenario.
    station_elevation = 3980.0
    offset = -(source_elevation - station_elevation) * 0.006
    m = TriggerModel(
        info["集水区面积_km2"], info["冰川覆盖率"], qc, min(w, 400.0),
        cn=CN0, temp_offset=offset, mechanical_regime=mechanics["regime"],
        use_legacy_k=bool(calibrate),
    )
    m.mechanics = mechanics
    m.q_crit_old = _q_crit_old(sl["比降"])
    m.temperature_geometry = {"source_elevation_m": source_elevation,
                              "station_elevation_m": station_elevation,
                              "lapse_K_per_m": 0.006, "offset_C": offset}
    m.cn0 = CN0
    m.slope_info = sl
    if calibrate and feat is not None:
        m.calibrate(feat, ("2026-08-26 08:00", "2026-08-26 13:00"))
    return m


def slope_variants() -> dict:
    """三种沟长—比降口径的并列结果，供论文与可靠性讨论逐项引用。"""
    info = catchment_glacier_fraction()
    cl = centerline_slope()
    out = {
        "geometric": {"沟长_m": info["集水区主沟长_m"], "比降": info["集水区比降"],
                      "用途": "几何近似，仅面积—形状粗估，不进入 q_c"},
```

```
        "traced": {"沟长_m": info.get("追迹主沟长_m"), "比降": info.get("追迹比降"),
                "用途": "D8 主河道追迹，用于分辨率敏感性与沟道迁移性"},
        "centerline": {"沟长_m": cl["沟长_m"], "比降": cl["比降"],
                    "用途": "问题 4 中心线，q_c 的唯一计算口径"},
    }
    for k, v in out.items():
        s = v["比降"]
        mechanics = _mechanics(s)
        v["mechanical_regime"] = mechanics["regime"]
        v["q_c_m2s"] = mechanics["q_crit_m2s"]
        v["q_c_old_m2s"] = _q_crit_old(s) if s and s > 0 else None
    return out
```

问题一

step01_boundary_layer.py 边界层结构与静力订正

```
"""问题 1：高寒山区大气边界层结构、天气过程、极端气象事件与泥石流关联性特征分析。

方法要点
--------
1. 边界层结构：5 层温/湿/风廓线 → 直减率、逆温强度与频率、整体理查森数、稳定度分类、
   近地层有效厚度、能量闭合与波文比、土壤温湿剖面。
2. 天气过程：廓线标准化后 SVD 提取主导模态（EOF），据降水—消融联合分位划分为四型。
3. 极端事件：POT-GPD 拟合各要素极值，给出 2/5 年再现期与复合极端事件统计。
4. 关联性排序：以水文代理响应状态为标签（训练期Qw的90%分位），
   对候选气象/边界层特征做置换重要性 + 互信息 + 事件异常度 + 机理先验的四准则
   Borda 融合，并按“水源—能量—物质”三条件分组排序。
   标签与降水/能量输入同源；构造性项显式披露，不作独立灾害验证。

输出：求解结果/数据/01_*.json，求解结果/图表/01_*.png
"""
from __future__ import annotations

import json
import os
import sys
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy import stats

sys.path.insert(0, str(Path(__file__).resolve().parent))
_PROJECT = Path(__file__).resolve().parents[2]
if os.environ.get("JILONG_ROOT") and Path(os.environ["JILONG_ROOT"]).resolve() != _PROJECT:
    raise RuntimeError("JILONG_ROOT must point to this project; refusing to write another version")
os.environ["JILONG_ROOT"] = str(_PROJECT)
from mmlib import io_data as io, meteo as mt, trigger as tg # noqa: E402

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "Noto Sans SC", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams.update({
    "font.size": 23,
    "axes.titlesize": 22,
    "axes.labelsize": 20,
    "xtick.labelsize": 17,
    "ytick.labelsize": 17,
    "legend.fontsize": 15,
})

FIG = io.FIG_OUT
OUT = io.DATA_OUT
EVENT_TS = pd.Timestamp("2026-08-26 10:30")
WIN = (pd.Timestamp("2026-08-20 00:00"), pd.Timestamp("2026-08-26 23:00"))


def _ser(x):
    if isinstance(x, dict):
        return {str(k): _ser(v) for k, v in x.items()}
    if isinstance(x, (list, tuple)):
        return [_ser(v) for v in x]
    if isinstance(x, (np.floating, float)):
        v = float(x)
        return None if not np.isfinite(v) else v
    if isinstance(x, (np.bool_, bool)):
        return bool(x)
    if isinstance(x, (np.integer, int)):
        return int(x)
    if isinstance(x, np.ndarray):
        return _ser(x.tolist())
    return x


def save_json(obj, name: str) -> Path:
    out = OUT / name
```

```
    out.parent.mkdir(parents=True, exist_ok=True)
    out.write_text(json.dumps(_ser(obj), ensure_ascii=False, indent=2), encoding="utf-8")
    return out


# ============================================================ 1. 边界层结构
def boundary_layer_structure(feat: pd.DataFrame) -> dict:
    res: dict = {}
    res["样本"] = {"起": str(feat.index.min()), "止": str(feat.index.max()),
              "小时数": int(len(feat)), "有效温度小时数": int(feat["Ta_2m"].notna().sum()),
              "有效降水小时数": int(feat["P"].notna().sum()),
              "有效辐射小时数": int(feat["Rsd"].notna().sum())}

    prof = {}
    for z in (1, 2, 4, 10, 20):
        for fam, key in (("Ta", "气温_°C"), ("WS", "风速_ms-1"), ("RH", "相对湿度_%")):
            col = f"{fam}_{z}m"
            if col in feat:
                prof.setdefault(key, {})[f"{z}m"] = {
                    "mean": feat[col].mean(), "p10": feat[col].quantile(0.1),
                    "p50": feat[col].median(), "p90": feat[col].quantile(0.9)}
    res["多层次廓线统计"] = prof

    grad_cols = [f"dT_{a}_{b}m" for a, b in [(1, 2), (2, 4), (4, 10), (10, 20)]]
    res["温度直减率_K每100m"] = {c: {"mean": feat[c].mean(), "p50": feat[c].median(),
                                 "p05": feat[c].quantile(0.05), "p95": feat[c].quantile(0.95)}
                          for c in grad_cols if c in feat}

    night = (feat.index.hour >= 20) | (feat.index.hour < 8)
    day = (feat.index.hour >= 9) & (feat.index.hour <= 17)
    def frequency(series, threshold):
        valid = series.dropna()
        return float((valid > threshold).mean()) if len(valid) else np.nan

    res["逆温"] = {
        "有效样本": int(feat["inv_max"].notna().sum()),
        "总频率": frequency(feat["inv_max"], 0.05), "操作阈值_K每100m":.05,
        "阈值说明": "预定操作阈值，不是由仪器精度确定的噪声界；逆温采用实际温度而非位温",
        "操作阈值敏感性": {str(t): {"总频率": frequency(feat["inv_max"], t),
                                 "夜间频率": frequency(feat.loc[night, "inv_max"], t),
                                 "白天频率": frequency(feat.loc[day, "inv_max"], t)}
                           for t in (0., .05, .5)},
        "强逆温频率": frequency(feat["inv_max"], 0.5),
        "夜间频率": frequency(feat.loc[night, "inv_max"], 0.05),
        "白天频率": frequency(feat.loc[day, "inv_max"], 0.05),
        "强度中位数_K每100m": feat["inv_max"].median(),
        "强度p95_K每100m": feat["inv_max"].quantile(0.95),
        "逆温状态下2m与20m温差中位数_K": feat.loc[feat["inv_max"] > 0.05, "dT_total_1_20m"].median(),
    }

    stab = feat["stab"].dropna()
    res["稳定度分类频率"] = {f"{int(k)}": float(v) for k, v in stab.value_counts(normalize=True).sort_index().items()}
    res["稳定度分类名称"] = {"1": "强不稳定", "2": "不稳定", "3": "中性", "4": "稳定", "5": "强稳定"}
    res["Ri_b"] = {"mean": feat["Ri_b"].mean(), "p05": feat["Ri_b"].quantile(0.05),
                "p50": feat["Ri_b"].median(), "p95": feat["Ri_b"].quantile(0.95),
                "切变平方屏蔽阈值_m2s2": mt.RI_SHEAR_EPS,
                "切变屏蔽样本": feat.attrs.get("shear_mask_count"),
                "共同有效原始样本": feat.attrs.get("eligible_count"),
                "切变屏蔽比例": (feat.attrs.get("shear_mask_count", 0) /
                             max(feat.attrs.get("eligible_count", 0), 1))}
    crossing = feat["pbl_crossing_height_m"]
    res["近地层有效厚度_m"] = {"mean": crossing.mean(), "p50": crossing.median(),
                      "夜间中位数": crossing.loc[night].median(),
                      "白天中位数": crossing.loc[day].median(),
                      "交点数": int(crossing.notna().sum()),
                      "交点层顶计数": {str(z): int(crossing.eq(z).sum()) for z in (4, 10, 20)},
                      "右删失数":feat.attrs.get("pbl_right_censored_count"),
                      "有效数":int(feat.pblh.notna().sum()),
                      "右删失比例": float(feat.pbl_right_censored.mean()),
                      "口径": "均值和分位只针对已观察到的离散交点，条件统计不是总体厚度；20m交点与20m右删失分开，不是CBL高度"}
    res["静力层压方法"] = feat.attrs.get("hydrostatic", {})

    diurnal = feat.groupby(feat.index.hour)[["dtheta_1_20", "shear_1_20m", "Rn", "melt_energy", "Ri_b"]].mean()
    res["日变化"] = {c: {int(h): diurnal.loc[h, c] for h in diurnal.index} for c in diurnal.columns}

    res["季节循环"] = {
        "气温月均_°C": {int(m): feat.loc[feat.index.month == m, "Ta_2m"].mean() for m in range(1, 13)},
        "降水月合计_mm": {int(m): feat.loc[feat.index.month == m, "P"].sum() for m in range(1, 13)},
        "融化能量月合计_MJm-2": {int(m): feat.loc[feat.index.month == m, "cum_melt_energy"].sum() for m in range(1, 13)
            },
        "净辐射月均_Wm-2": {int(m): feat.loc[feat.index.month == m, "Rn"].mean() for m in range(1, 13)},
    }

    month_threshold = feat.loc[feat.index.year <= 2025].groupby(feat.loc[feat.index.year <= 2025].index.month)["
         WS_20m"].quantile(.9)
    thr = pd.Series(feat.index.month, index=feat.index).map(month_threshold)
    llj = (feat["WS_20m"] > thr) & night
    res["近地层夜间强风代理_非低空急流识别"] = {"夜间频率": float(llj[night & feat["WS_20m"].notna()].mean()),
```

```
                "WS_20m月p90均值_ms-1": float(thr.mean()),
                "急流时WS_20m中位数_ms-1": feat.loc[llj, "WS_20m"].median()}

    ok = feat.loc[feat.index.year <= 2025, ["available", "Rn", "G_0.1m"]].dropna()
    ok = ok[(ok["Rn"] - ok["G_0.1m"]) > 20]
    if len(ok) > 100:
        eff = ok["Rn"] - ok["G_0.1m"]
        slope, intercept = np.polyfit(eff, ok["available"], 1)
        res["能量闭合"] = {"样本": int(len(ok)), "回归斜率": float(slope), "截距": float(intercept),
                    "相关系数": float(np.corrcoef(eff, ok["available"])[0, 1]),
                    "闭合比C_均值比": float(ok["available"].sum() / eff.sum()),
                    "逐时比值中位数_非C": float((ok["available"] / eff).median()),
                    "训练期": "2021–2025，共同有效且Rn-G>20 W/m²",
                    "有效能量中位数_Wm-2": float(eff.median())}
    res["波文比"] = {"中位数": feat["bowen"].median(), "p10": feat["bowen"].quantile(0.1),
                "p90": feat["bowen"].quantile(0.9)}
    res["土壤"] = {f"{d}m_温度均值_°C": feat[f"soilT_{d}m"].mean() for d in tg.__dict__.get("DEPTHS", ["0.1", "0.2", "
        0.4", "0.8", "1.6"])}
    res["土壤"].update({f"{d}m_含水率均值": feat[f"VWC_{d}m"].mean() for d in ["0.1", "0.2", "0.4", "0.8", "1.6"]})
    res["土壤"]["浅层含水率p95"] = feat["VWC_0.1m"].quantile(0.95)
    res["土壤"]["浅层含水率年均最大值"] = float(feat.groupby(feat.index.year)["VWC_0.1m"].max().mean())
    res["土壤"]["深层与浅层温差_°C"] = float((feat["soilT_1.6m"] - feat["soilT_0.1m"]).mean())
    res["累计窗口可用性"] = {c: {"有效小时": int(feat[c].notna().sum()),
                                "缺测比例": float(feat[c].isna().mean())}
                          for c in ("P_3h", "P_24h", "P_168h", "API", "PDD_3d")}
    res["季节循环"]["说明"] = "月合计为全观测期该日历月已观测量之和，不是单个完整年份月总量；缺测不填零"
    return res


def hydrostatic_sensitivity(raw: pd.DataFrame) -> tuple[dict, pd.DataFrame]:
    profiles = {"同压对照": mt.hydrostatic_profiles(raw, corrected=False)}
    profiles.update({f"参考{z}m": mt.hydrostatic_profiles(raw, z_ref=z) for z in (2, 1, 10, 20)})
    ris = {name: mt.bulk_richardson(raw, p) for name, p in profiles.items()}
    pbl = {name: mt.pbl_diagnostics(raw, p) for name, p in profiles.items()}
    common_ri = pd.concat(ris, axis=1).notna().all(axis=1)
    common_pbl = pd.concat({name: d.pblh for name, d in pbl.items()}, axis=1).notna().all(axis=1)
    common_thermo = pd.concat({name: p.thermo_valid for name, p in profiles.items()}, axis=1).all(axis=1)
    endpoint = raw[["Ta_2m", "Ta_20m", "RH_2m", "RH_20m", "Pressure"]].notna().all(axis=1)
    old_ri_possible = endpoint & mt.vector_shear_squared(raw).ge(mt.RI_SHEAR_EPS)
    base_class = pd.Series(mt.stability_class(ris["参考2m"]), index=raw.index)
    hourly = pd.DataFrame(index=raw.index)
    hourly["common_thermo"] = common_thermo
    hourly["common_Ri"] = common_ri
    hourly["common_pbl"] = common_pbl
    result = {"基准": "MET Pressure假定2m，五层Ta/RH+p共同有效，逆虚温分段梯形积分三次",
              "限制": "参考高度不是安装元数据；同压对照同样要求全五层有效；FLUX PA不替代；静力不处理非静力扰动",
              "样本": {"小时总数": len(raw), "原端点热力可用": int(endpoint.sum()),
                    "五层热力共同样本": int(common_thermo.sum()),
                    "五层要求新增热力缺失": int((endpoint & ~common_thermo).sum()),
                    "原端点可算Ri小时": int(old_ri_possible.sum()),
                    "各场景Ri共同小时": int(common_ri.sum()),
                    "Ri新增样本损失": int((old_ri_possible & ~common_ri).sum()),
                    "各场景交点或删失共同小时": int(common_pbl.sum())}, "场景": {}}
    for name, profile in profiles.items():
        ri = ris[name].loc[common_ri]
        cls = pd.Series(mt.stability_class(ri), index=ri.index)
        d = pbl[name].loc[common_pbl]
        transition = pd.crosstab(base_class.loc[common_ri], cls).reindex(index=range(1, 6), columns=range(1, 6),
            fill_value=0)
        hourly[f"{name}_Ri"] = ris[name]
        hourly[f"{name}_交点_m"] = pbl[name].pbl_crossing_height_m
        hourly[f"{name}_右删失"] = pbl[name].pbl_right_censored
        for field in ("p_1m", "p_2m", "p_20m", "q_2m", "theta_1m", "theta_20m", "thetav_2m", "thetav_20m"):
            hourly[f"{name}_{field}"] = profile[field]
        result["场景"][name] = {"层压审计": profile.attrs,
            "共同Ri中位": ri.median(), "共同Ri_p05_p95": ri.quantile([.05, .95]).tolist(),
            "共同稳定度频率": {str(i): float(cls.eq(i).mean()) for i in range(1, 6)},
            "与2m基准分类不同数": int(cls.ne(base_class.loc[common_ri]).sum()),
            "2m基准到本场景分类计数矩阵": transition.to_numpy().tolist(),
            "共同层压p2减p20中位_hPa": (profile.p_2m - profile.p_20m).loc[common_thermo].median(),
            "共同位温差1至20中位_K": (profile.theta_20m - profile.theta_1m).loc[common_thermo].median(),
            "共同交点计数": {str(z): int(d.pbl_crossing_height_m.eq(z).sum()) for z in (4, 10, 20)},
            "共同右删失数": int(d.pbl_right_censored.eq(1).sum()),
            "共同右删失比例": d.pbl_right_censored.mean()}
    hourly["参考2m_pressure_iteration_residual_hPa"] = profiles["参考2m"].pressure_iteration_residual_hPa
    return result, hourly


def plot_hydrostatic(result: dict) -> Path:
    scenarios = result["场景"]
    names = list(scenarios)
    fig, axes = plt.subplots(2, 2, figsize=(14, 10), constrained_layout=True)
    ax = axes[0, 0]
    x = np.arange(len(names)); bottom = np.zeros(len(names))
    for i, label in enumerate(("强不稳定", "不稳定", "中性", "稳定", "强稳定"), 1):
        values = np.array([scenarios[n]["共同稳定度频率"][str(i)] for n in names])
        ax.bar(x, values, bottom=bottom, label=label)
        bottom += values
```

```
    ax.set_xticks(x, names); ax.set_ylabel("共同样本频率"); ax.set_title("(a) 层间压降订正与参考高度分开比较")
    ax.legend(fontsize=14, ncol=3)
    ax = axes[0, 1]
    ax.bar(names, [scenarios[n]["共同Ri中位"] for n in names], color="#3e727b")
    ax.axhline(0, color="black", linewidth=.7); ax.set_ylabel("整体Ri中位数")
    ax.set_title("(b) 所有场景同一有效小时集合")
    ax = axes[1, 0]
    fields = [("4", "4米交点"), ("10", "10米交点"), ("20", "20米交点")]
    bottom = np.zeros(len(names))
    for key, label in fields:
        values = np.array([scenarios[n]["共同交点计数"][key] for n in names])
        ax.bar(x, values, bottom=bottom, label=label); bottom += values
    ax.bar(x, [scenarios[n]["共同右删失数"] for n in names], bottom=bottom,
           label="20米右删失（非交点）", color="#c7c5b6", hatch="//")
    ax.set_xticks(x, names); ax.set_ylabel("小时数"); ax.legend(fontsize=14)
    ax.set_title("(c) 交点与观测上限删失严格分开")
    ax = axes[1, 1]
    matrix = np.asarray(scenarios["同压对照"]["2m基准到本场景分类计数矩阵"])
    image = ax.imshow(np.log10(matrix + 1), cmap="YlGnBu")
    for i in range(5):
        for j in range(5):
            ax.text(j, i, str(matrix[i, j]), ha="center", va="center", fontsize=12)
    ax.set_xticks(range(5), range(1, 6)); ax.set_yticks(range(5), range(1, 6))
    ax.set_xlabel("同压对照类别"); ax.set_ylabel("静力2米基准类别")
    ax.set_title("(d) 分类转移计数；颜色为log10(n+1)"); fig.colorbar(image, ax=ax, shrink=.8)
    fig.suptitle("静力订正恢复热力学一致性，不消除传感器与非静力误差", fontsize=13)
    path = FIG / "01_静力订正敏感性.png"
    fig.savefig(path, dpi=160, bbox_inches="tight"); plt.close(fig)
    return path


# ============================================================ 2. 天气过程模态
def weather_regimes(feat: pd.DataFrame) -> dict:
    cols = [f"{family}_{z}m" for family in ("Ta", "RH", "WS") for z in (1, 2, 4, 10, 20)] + ["Pressure"]
    training = feat.loc[feat.index.year <= 2025, cols].dropna()
    if len(training) < 1000:
        raise ValueError("训练期完整廓线样本不足")
    means, scales = training.mean(), training.std().replace(0, np.nan)
    sub = training.iloc[::max(1, len(training) // 20000)]
    X = (sub - means) / scales
    U, S, Vt = np.linalg.svd(X.to_numpy(), full_matrices=False)
    var = S ** 2 / np.sum(S ** 2)
    pc = X.to_numpy() @ Vt.T
    sampled = feat.loc[X.index]
    def correlation(left, right):
        pair = pd.DataFrame({"pc": left, "x": right}, index=X.index).dropna()
        return float(pair.corr().iloc[0, 1])
    baseline = feat.loc[feat.index.year <= 2025]
    q_p = float(baseline["P_24h"].quantile(.9))
    q_m = float(baseline["cum_melt_energy"].quantile(.9))
    available = feat[["P_24h", "cum_melt_energy"]].notna().all(axis=1)
    wet = feat.loc[available, "P_24h"] > q_p
    melt = feat.loc[available, "cum_melt_energy"] > q_m
    return {"解释方差比": var[:8].tolist(), "主成分数_85pct": int(np.searchsorted(var.cumsum(), .85) + 1),
            "样本": len(X), "训练期": "2021–2025；标准化/SVD/分型阈值均冻结于该期",
            "标准化均值": means.to_dict(), "标准化标准差": scales.to_dict(),
            "载荷": {f"EOF{k+1}": dict(zip(cols, Vt[k].tolist())) for k in range(min(4, len(Vt)))},
            "主导模态": [{"模态": f"EOF{k+1}", "解释方差": float(var[k]),
                          "主导变量": pd.Series(Vt[k], index=cols).reindex(
                              pd.Series(Vt[k], index=cols).abs().nlargest(4).index).to_dict()}
                         for k in range(min(4, len(Vt)))],
            "PC1与24h降水相关": correlation(pc[:, 0], sampled["P_24h"]),
            "PC1与融化能量相关": correlation(pc[:, 0], sampled["cum_melt_energy"]),
            "PC2与融化能量相关": correlation(pc[:, 1], sampled["cum_melt_energy"]),
            "天气型频率": {"I_强降水主导": float((wet & ~melt).mean()),
                           "II_强消融主导": float((~wet & melt).mean()),
                           "III_降水消融复合": float((wet & melt).mean()),
                           "IV_平缓天气": float((~wet & ~melt).mean())},
            "分型有效小时": int(available.sum()), "分型缺测小时": int((~available).sum()),
            "分型阈值": {"P_24h_p90_mm": q_p, "cum_melt_p90_MJm-2": q_m}}


# ============================================================ 3. 极端事件
def extreme_events(feat: pd.DataFrame) -> dict:
    targets = {"小时降水_mm": "P", "24h降水_mm": "P_24h", "小时融化能量_MJm-2": "cum_melt_energy",
               "2m气温_°C": "Ta_2m", "10m风速_ms-1": "WS_10m", "饱和差_hPa": "VPD2",
               "逆温强度_K每100m": "inv_max", "净辐射_Wm-2": "Rn", "摩擦速度_ms-1": "ustar"}
    baseline = feat.loc[feat.index.year <= 2025]
    res = {"统计口径": "阈值与尾部分布仅训练期2021–2025；有效观测小时/8765.82为年暴露量"}
    for name, col in targets.items():
        if col not in baseline:
            continue
        s = baseline[col].dropna()
        if len(s) < 500 or s.std() < 1e-9:
            continue
        u = float(s.quantile(.97))
        exceed = s[s > u]
        if len(exceed) < 30:
            continue
        exposure = len(s) / (365.2425 * 24)
```

```
        # 同一过程内超阈间隔不足12h只留最大值：缺测不创造额外暴露时长。
        cluster_id = exceed.index.to_series().diff().ge(pd.Timedelta(hours=12)).cumsum()
        peaks = exceed.groupby(cluster_id.to_numpy()).max()
        def fit(values):
            if len(values) < 10:
                return {"状态": "去丛集后样本不足", "事件数": len(values)}
            c, _, scale = stats.genpareto.fit(values.to_numpy() - u, floc=0)
            rate = len(values) / exposure
            def level(years):
                probability = 1 / (years * rate)
                if probability >= 1:
                    return None
                value = u + scale / c * (probability ** (-c) - 1) if abs(c) > 1e-7 else u + scale * np.log(1 /
                    probability)
                if c < 0:
                    value = min(value, u - scale / c)
                return float(value)
            return {"状态": "已拟合", "事件数": len(values), "GPD_k": float(c), "GPD_sigma": float(scale),
                    "超阈频率_每年": float(rate), "2年再现期水平": level(2), "5年再现期水平": level(5),
                    "GPD有限上界": float(u - scale / c) if c < 0 else None}
        raw, declustered = fit(exceed), fit(peaks)
        chosen = declustered if declustered.get("状态") == "已拟合" else raw
        maxima = s.groupby(s.index.year).max()
        res[name] = {"阈值u_97pct": u, "超阈次数": len(exceed), "有效观测年暴露量": exposure,
                     "去丛集间隔_h": 12, "原始逐时拟合": raw, "去丛集拟合": declustered,
                     "主报告口径": "去丛集" if chosen is declustered else "逐时（去丛集样本不足）",
                     **chosen, "年最大均值": float(maxima.mean()), "年最大极值": float(maxima.max()),
                     "年最大极值出现年": int(maxima.idxmax())}
    qp, qt, qe = [float(baseline[c].quantile(.9)) for c in ("P_24h", "Ta_2m", "cum_melt_energy")]
    valid = feat[["P_24h", "Ta_2m", "cum_melt_energy"]].notna().all(axis=1)
    comp = ((feat["P_24h"] > qp) & (feat["Ta_2m"] > qt) & (feat["cum_melt_energy"] > qe)) & valid
    groups = comp.ne(comp.shift(fill_value=False)).cumsum()
    segments = comp.groupby(groups).sum()
    segments = segments[segments > 0]
    res["复合极端"] = {"判据": {"P_24h_mm": qp, "Ta_2m_°C": qt, "cum_melt_MJm-2": qe},
                   "阈值期": "2021–2025", "复合小时数": int(comp.sum()), "有效小时数": int(valid.sum()),
                   "事件段数": len(segments), "最长持续_h": int(segments.max()) if len(segments) else 0,
                   "年平均段数": len(segments) / max(valid.sum() / (365.2425 * 24), 1e-9),
                   "发生月份分布": {str(m): int((comp & (feat.index.month == m)).sum()) for m in range(1, 13)}}
    threshold = res.get("24h降水_mm", {}).get("5年再现期水平", np.inf)
    threshold = np.inf if threshold is None else threshold
    big = feat[feat["P_24h"] > threshold]
    res["5年一遇强降水超限小时数"] = len(big)
    res["5年一遇强降水时间"] = big.index[:20].astype(str).tolist()
    return res


# ============================================================ 4. 关联性排序
FEATURE_GROUPS = {
    "水源条件": {
        "24h累积降水": "P_24h", "1h降水强度": "Pmax_1h", "3h累积降水": "P_3h",
        "有效降水指数API": "API", "小时降水": "P", "7日累计降水": "AP_7d",
        "3日正积温": "PDD_3d", "7日正积温": "PDD_7d", "24h升温幅度": "rise_24h",
        "24h冻融交替次数": "freeze_thaw",
        "消融有效能量": "cum_melt_energy", "2m气温": "Ta_2m", "饱和水汽压差": "VPD2",
        "2m相对湿度": "RH_2m",
    },
    "能量与地形条件": {
        "整体理查森数": "Ri_b", "风切变": "shear_1_20m", "位温垂直梯度": "dtheta_1_20",
        "逆温强度": "inv_max", "10m风速": "WS_10m", "净辐射": "Rn", "气压": "Pressure",
    },
    "物质条件": {
        "浅层土壤含水率": "VWC_0.1m", "中层土壤含水率": "VWC_0.4m",
        "浅层土壤温度": "soilT_0.1m", "反照率": "albedo", "晴空指数": "clearness",
    },
}
MECH_PRIOR = { # 机理先验：是否直接进入三条件驱动项
    "AP_7d": 1.0, "PDD_3d": 0.9, "PDD_7d": 0.9, "rise_24h": 0.7, "freeze_thaw": 0.5,
    "P_24h": 1.0, "Pmax_1h": 1.0, "P_3h": 1.0, "API": 1.0, "P": 0.95,
    "cum_melt_energy": 1.0, "Ta_2m": 0.9, "VPD2": 0.7, "RH_2m": 0.65,
    "Ri_b": 0.6, "shear_1_20m": 0.6, "dtheta_1_20": 0.65, "inv_max": 0.5,
    "WS_10m": 0.45, "Rn": 0.75, "Pressure": 0.2,
    "VWC_0.1m": 0.95, "VWC_0.4m": 0.9, "soilT_0.1m": 0.6, "albedo": 0.35, "clearness": 0.3,
}


def association_ranking(feat: pd.DataFrame, qw: pd.Series) -> dict:
    """主排序及CRITIC同源一致性核对；分类器只用时间留出。"""
    from mmlib.ranking import association
    return association(feat, qw, FEATURE_GROUPS, MECH_PRIOR)


def stability_by_season(feat: pd.DataFrame) -> dict:
    cols = [c for c in ["Ta_2m", "RH_2m", "WS_10m", "dtheta_1_20", "Ri_b", "VPD2", "Pressure"]
            if c in feat.columns]
    width = int((WIN[1]-WIN[0]).total_seconds()/3600)+1
    common = feat[cols].dropna().reindex(feat.index)
    complete = common.notna().all(axis=1).astype(int)
    means = common.rolling(width,min_periods=int(np.ceil(.8*width))).mean()
    good = complete.rolling(width,min_periods=width).sum() >= .8*width
    eligible = ((feat.index.year<=2025)&feat.index.month.isin([6,7,8,9]) &
```

```
            (feat.index.hour == 23) & good.to_numpy())
    warm = means.loc[eligible].dropna()
    if len(warm)<100:return {"状态":"同长度窗口不足"}
    mu,cov=warm.mean().to_numpy(),np.cov(warm.to_numpy(),rowvar=False)
    inv=np.linalg.pinv(cov)
    def md(x):
        dd=np.atleast_2d(x)-mu
        return np.sqrt(np.maximum(0,np.einsum("ij,jk,ik->i",dd,inv,dd)))
    reference=md(warm.to_numpy())
    event=common.loc[WIN[0]:WIN[1]]
    if event.notna().all(axis=1).sum()<.8*width:return {"状态":"事件同长度窗口有效率不足"}
    value=float(md(event.mean().to_numpy())[0])
    return {"暖季样本":len(warm),"窗口小时":width,"有效小时门槛":int(np.ceil(.8*width)),
            "事件有效小时":int(event.notna().all(axis=1).sum()),"参照期":"2021–2025暖季同长窗口均值，每日23时取样；窗口重叠非独立",
            "事件窗口Mahalanobis距离":value,"暖季距离p50":float(np.median(reference)),
            "暖季距离p95":float(np.percentile(reference,95)),"事件超越暖季样本比例":float((reference<value).mean()),"参与变量":cols}


# ============================================================ 绘图
def plot_all(feat: pd.DataFrame, qw: pd.Series) -> list[Path]:
    paths = []
    fig, axes = plt.subplots(2, 2, figsize=(14, 10), constrained_layout=True)
    ax = axes[0, 0]
    for z, c in zip((1, 4, 20), ("#1f77b4", "#ff7f0e", "#d62728")):
        d = feat.groupby(feat.index.hour)[f"Ta_{z}m"].mean()
        ax.plot(d.index, d.values, "o-", ms=3, color=c, label=f"{z} m")
    ax.set_xlabel("地方时 (h)"); ax.set_ylabel("气温 (°C)")
    ax.set_title("(a) 多层气温日变化"); ax.legend(); ax.grid(alpha=0.3)

    ax = axes[0, 1]
    for z, c in zip((1, 4, 20), ("#1f77b4", "#ff7f0e", "#d62728")):
        d = feat.groupby(feat.index.hour)[f"WS_{z}m"].mean()
        ax.plot(d.index, d.values, "s-", ms=3, color=c, label=f"{z} m")
    ax.set_xlabel("地方时 (h)"); ax.set_ylabel("风速 (m/s)")
    ax.set_title("(b) 多层风速日变化"); ax.legend(); ax.grid(alpha=0.3)

    ax = axes[1, 0]
    mon = feat.groupby(feat.index.month)
    ax.bar(mon["P"].sum().index, mon["P"].sum().values, color="#4c72b0", label="月降水")
    ax.set_xlabel("月份"); ax.set_ylabel("降水 (mm)", color="#4c72b0")
    ax2 = ax.twinx()
    ax2.bar(mon["cum_melt_energy"].sum().index + 0.35,
            mon["cum_melt_energy"].sum().values / 1000.0, width=0.35, color="#dd8452")
    ax2.set_ylabel("未调整能量代理 (10³ MJ/m²)", color="#dd8452")
    ax.set_title("(c) 降水与能量代理的季节循环")
    ax.set_xticks(range(1, 13)); ax.grid(alpha=0.3, axis="y")

    ax = axes[1, 1]
    w = feat.loc["2026-08-18":"2026-08-27"]
    ax.plot(w.index, w["Ta_2m"], color="#d62728", label="2 m 气温")
    ax.plot(w.index, w["Ta_20m"], color="#ff9896", ls="--", label="20 m 气温")
    ax.axvline(EVENT_TS, color="k", ls=":", lw=2, label="口岸冲击锚点")
    ax.set_ylabel("气温 (°C)")
    ax3 = ax.twinx()
    ax3.bar(w.index, w["P"], width=0.03, color="#4c72b0", alpha=0.6, label="小时降水")
    ax3.set_ylabel("降水 (mm/h)")
    ax.set_title("(d) 2026-08-26 事件窗口气象过程")
    h1, l1 = ax.get_legend_handles_labels(); h2, l2 = ax3.get_legend_handles_labels()
    ax.legend(h1 + h2, l1 + l2, loc="upper left", fontsize=14)
    ax.grid(alpha=0.3)
    for lab in ax.get_xticklabels():
        lab.set_rotation(30)
    p = FIG / "01_边界层结构.png"; fig.savefig(p, dpi=160, bbox_inches="tight"); plt.close(fig); paths.append(p)

    fig, axes = plt.subplots(2, 2, figsize=(14, 10), constrained_layout=True)
    ax = axes[0, 0]
    st = feat["stab"].dropna()
    cnt = st.value_counts().sort_index()
    ax.bar(cnt.index, cnt.values / len(st),
           color=["#d62728", "#ff7f0e", "#bcbd22", "#2ca02c", "#1f77b4"])
    ax.set_xticks([1, 2, 3, 4, 5])
    ax.set_xticklabels(["强不稳定", "不稳定", "中性", "稳定", "强稳定"], fontsize=14)
    ax.set_ylabel("频率"); ax.set_title("(a) 近地层稳定度分类频率"); ax.grid(alpha=0.3, axis="y")

    ax = axes[0, 1]
    hh = feat.groupby(feat.index.hour)["inv_max"].apply(lambda s: (s.dropna() > 0.05).mean())
    ax.plot(hh.index, hh.values * 100, "o-", color="#6a3d9a")
    ax.set_xlabel("地方时 (h)"); ax.set_ylabel("逆温出现频率 (%)")
    ax.set_title("(b) 逆温频率日变化"); ax.grid(alpha=0.3)

    ax = axes[1, 0]
    ok = feat.loc[feat.index.year <= 2025, ["available", "Rn", "G_0.1m"]].dropna()
    ok = ok[(ok["Rn"] - ok["G_0.1m"]) > 20]
    eff = ok["Rn"] - ok["G_0.1m"]
    ax.scatter(eff, ok["available"], s=2, alpha=0.2, c="#2ca02c")
    lim = [0, float(eff.quantile(.995))]
    ax.plot(lim, lim, "k--", label="1:1 闭合线")
    slope, intercept = np.polyfit(eff, ok["available"], 1)
    ratio = float(ok["available"].sum() / eff.sum())
```

```
    ax.plot(lim, [slope * v + intercept for v in lim], "r-", label=f"回归斜率 {slope:.3f}；C={ratio:.3f}")
    ax.set_xlim(lim)
    ax.set_xlabel("可用能量 Rn−G (W/m²)"); ax.set_ylabel("湍流热通量 H+LE (W/m²)")
    ax.set_title("(c) 训练期共同有效样本的能量闭合"); ax.legend(fontsize=14); ax.grid(alpha=0.3)

    ax = axes[1, 1]
    cols = [f"Ta_{z}m" for z in (1, 2, 4, 10, 20)] + [f"RH_{z}m" for z in (1, 2, 4, 10, 20)] + \
           [f"WS_{z}m" for z in (1, 2, 4, 10, 20)] + ["Pressure"]
    modes=json.loads((OUT/"01_天气过程模态.json").read_text(encoding="utf8"))
    Vt=np.array([[modes["载荷"][f"EOF{k}"][c] for c in cols] for k in (1,2)])
    var=np.asarray(modes["解释方差比"])
    ax.plot(Vt[0], np.arange(len(cols)), "o-", color="#1f77b4", ms=4,
            label=f"EOF1 ({var[0] * 100:.0f}%)")
    ax.plot(Vt[1], np.arange(len(cols)), "s-", color="#ff7f0e", ms=3.5,
            label=f"EOF2 ({var[1] * 100:.0f}%)")
    ax.set_yticks(range(len(cols)))
    ax.set_yticklabels([c.replace("Ta_", "T").replace("RH_", "RH").replace("WS_", "WS")
                        .replace("Pressure", "气压").replace("m", " m") for c in cols], fontsize=12)
    ax.axvline(0, color="k", lw=0.8)
    ax.set_xlabel("载荷"); ax.set_title("(d) 廓线主导模态载荷")
    ax.legend(fontsize=14); ax.grid(alpha=0.3)
    p = FIG / "01_稳定度与能量.png"; fig.savefig(p, dpi=160, bbox_inches="tight"); plt.close(fig); paths.append(p)
    return paths


def plot_ranking(rank: dict) -> list[Path]:
    full = pd.DataFrame(rank["排序表"])
    display = pd.DataFrame(rank["去冗余排序表"])
    critic = pd.DataFrame(rank["CRITIC"]["去冗余排序表"])
    fig, axes = plt.subplots(2, 2, figsize=(15, 12), constrained_layout=True)
    colors = {"水源条件": "#4c72b0", "能量与地形条件": "#dd8452", "物质条件": "#55a868"}
    for ax, frame, value, title in (
            (axes[0, 0], display, "综合得分", "(a) 水源代理榜：训练相关连通簇去冗余"),
            (axes[0, 1], critic, "CRITIC得分", "(b) 三证据CRITIC：同源一致性核对")):
        d = frame.iloc[::-1]
        ax.barh(d["特征"], d[value], color=[colors[g] for g in d["条件组"]])
        ax.set_title(title, fontsize=11)
        ax.set_xlabel("融合得分（越小越强）" if value == "综合得分" else "关联得分（越大越强）")
        ax.tick_params(labelsize=8); ax.grid(alpha=.25, axis="x")
    ax = axes[1, 0]
    top = display.head(5)
    positions = np.arange(len(top))
    for j, (column, name) in enumerate((("PI_rank", "置换重要性秩"), ("MI_rank", "互信息秩"),
                                        ("AN_rank", "事件异常秩"), ("MP_rank", "机理先验秩"))):
        ax.bar(positions + (j - 1.5) * .2, top[column], .2, label=name)
    ax.set_xticks(positions); ax.set_xticklabels(top["特征"], rotation=20, fontsize=8)
    ax.set_ylabel("准则秩（不是不同量纲原值）"); ax.set_title("(c) 去冗余前五的分准则秩")
    ax.legend(fontsize=8); ax.grid(alpha=.25, axis="y")
    ax = axes[1, 1]
    crit_full = pd.DataFrame(rank["CRITIC"]["排序表"]).set_index("变量")
    x = full["总次序"].to_numpy()
    y = full["变量"].map(crit_full["总次序"]).to_numpy()
    ax.scatter(x, y, c="#4c72b0", s=30)
    ax.plot([1, len(full)], [1, len(full)], "k--", lw=1)
    for _, row in full[full["变量"].isin(top["变量"])].iterrows():
        ax.annotate(row["特征"], (row["总次序"], crit_full.loc[row["变量"], "总次序"]),
                    fontsize=7, xytext=(4, 4), textcoords="offset points")
    ax.set_xlabel("代理榜完整名次"); ax.set_ylabel("CRITIC完整名次")
    comparison = rank["两方案一致性"]
    ax.set_title(f"(d) 同源排序对照：τ={comparison['全排序Kendall_tau']:.2f}，去冗余前五重合{comparison['去冗余前5重合数']}")
    ax.grid(alpha=.25)
    path = FIG / "01_特征排序.png"
    fig.savefig(path, dpi=160, bbox_inches="tight"); plt.close(fig)
    return [path]


def main() -> int:
    FIG.mkdir(parents=True, exist_ok=True)
    # 必须从当前QC后的原始数据重建，不能消费旧版本派生缓存。
    print("构建特征表（当前QC原始小时观测）…")
    raw = io.load_station()
    feat = mt.build_feature_table(raw)
    hydro, hourly = hydrostatic_sensitivity(raw)
    save_json(hydro, "01_静力层压订正.json")
    hourly.to_csv(OUT / "01_静力层压逐时.csv", encoding="utf-8-sig", index_label="timestamp")
    print("图:", plot_hydrostatic(hydro))
    feat, closure = mt.closure_sensitivity(feat)
    feat.to_parquet(io.CACHE / "features.parquet")
    save_json(closure, "01_能量闭合敏感性.json")
    print(f"特征表 {feat.shape}，{feat.index.min()} ~ {feat.index.max()}")

    save_json(boundary_layer_structure(feat), "01_边界层结构.json")
    print("边界层结构 ok")
    save_json(weather_regimes(feat), "01_天气过程模态.json")
    print("天气过程模态 ok")
    save_json(extreme_events(feat), "01_极端事件.json")
    print("极端事件 ok")

    # 问题一的水源代理榜只消费未标定水源序列；事件窗回顾 k 不得在此重建。
```

```
    model = tg.default_model(feat, calibrate=False)
    dis = model.discharge(feat)
    dis.to_parquet(io.CACHE / "qwater.parquet")

    ar = association_ranking(feat, dis["Q_water"])
    ar["暖季稀有度"] = stability_by_season(feat)
    ar["能量基准与C敏感性"] = closure["能量敏感性"]
    save_json(ar, "01_特征关联排序.json")
    save_json(ar["CRITIC"], "01_CRITIC交叉核对.json")
    print("关联排序 ok；前5：")
    for r in ar["前5特征"]:
        print(f" {r['次序']}. {r['特征']}（{r['条件组']}，{r['变量']}）得分 {r['综合得分']:.2f}")
    print(f" 测试 AUC={ar['测试AUC']:.4f} 留一年AUC={ar['留一年AUC']}")
    print(f" 稳健性 tau 均值={ar['排序稳健性']['均值']}")

    for p in plot_all(feat, dis["Q_water"]) + plot_ranking(ar):
        print("图:", p)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

step01b_preimpact.py 冲击前窗口异常

```
"""Frozen pre-impact window-anomaly evidence; no water-source or disaster labels."""
from __future__ import annotations

import json
import os
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import kendalltau

PROJECT = Path(__file__).resolve().parents[2]
if "JILONG_ROOT" not in os.environ:
    os.environ["JILONG_ROOT"] = str(PROJECT)
from mmlib import io_data as io, meteo as mt # noqa: E402

TRAIN_YEARS = (2021, 2022, 2023, 2024)
TARGET = pd.Timestamp("2026-08-26")
LENGTHS = (168, 72)
SEED = 20260826
N_BOOT = 200
COVERAGE = .8
CALENDAR_RADIUS = 15
CANDIDATES = (
    ("WS_10m", "10米风速", "m/s", "未直接入水源式"),
    ("vector_shear_2_20m", "2—20米矢量风切变", "1/s", "未直接入水源式"),
    ("Pressure", "MET气压", "hPa", "未直接入水源式"),
    ("VWC_0.1m", "0.1米土壤含水率", "m3/m3", "未直接入水源式"),
    ("VWC_0.4m", "0.4米土壤含水率", "m3/m3", "未直接入水源式"),
    ("soilT_0.1m", "0.1米土温", "degC", "未直接入水源式"),
    ("ustar", "摩擦速度", "m/s", "未直接入水源式"),
    ("TKE_obs", "观测湍流动能", "m2/s2", "未直接入水源式"),
    ("Ri_b", "订正整体理查森数", "1", "共享温湿廓线"),
    ("dtheta_1_20", "订正1—20米位温差", "K", "共享温湿廓线"),
    ("inv_max", "实际温度逆温强度", "K/100m", "共享温湿廓线"),
    ("RH_2m", "2米相对湿度", "%", "共享温湿廓线"),
)
COLS = [v[0] for v in CANDIDATES]
NAMES = {v[0]: v[1] for v in CANDIDATES}
UNITS = {v[0]: v[2] for v in CANDIDATES}
ORDER = {c: i for i, c in enumerate(COLS)}
LIMITATION = (
    "异常是同季节已选历史窗口中的特殊性，不是灾害关联系数、发生概率或因果效应。"
    "小时标签起止和仪器延迟未知；仅称冲击锚点前可用，不保证早于源区失稳。"
    "每日窗口重叠、参照自参与尺度拟合；配置训练年4年，但2022年因ustar与TKE_obs缺测无共同有效窗，"
    "实际参照年为2021/2023/2024三年；尾秩非显著性p值，年块区间非独立灾害置信区间，2025伪事件不是真实非灾日。"
)


def serial(value):
    if isinstance(value, dict):
        return {str(k): serial(v) for k, v in value.items()}
    if isinstance(value, (list, tuple, np.ndarray)):
        return [serial(v) for v in value]
    if isinstance(value, (pd.Timestamp, np.datetime64)):
        return str(pd.Timestamp(value))
    if isinstance(value, (bool, np.bool_)):
        return bool(value)
    if isinstance(value, (int, np.integer)):
        return int(value)
    if isinstance(value, (float, np.floating)):
        return float(value) if np.isfinite(value) else None
    return value
```

```


def calendar_day(dates):
   """A common 365-day calendar; Feb 29 is never a release/query date."""
   dates = pd.DatetimeIndex(dates)
   if ((dates.month == 2) & (dates.day == 29)).any():
      raise ValueError("Feb 29 is not a query date")
   return np.array([pd.Timestamp(2001, d.month, d.day).dayofyear for d in dates])


def nearby(dates, query, radius=CALENDAR_RADIUS):
   delta = np.abs(calendar_day(dates) - calendar_day([query])[0])
   return np.minimum(delta, 365 - delta) <= radius


def all_release_dates():
   dates = pd.date_range("2021-01-01", "2025-12-31", freq="D")
   dates = dates[~((dates.month == 2) & (dates.day == 29))]
   return dates.append(pd.DatetimeIndex([TARGET]))


def candidate_features(features):
   f = features.copy()
   required = ["WS_2m", "WS_20m", "WD_2m", "WD_20m"]
   if not set(required).issubset(f):
      raise ValueError("vector wind shear needs measured wind speed/direction at 2 and 20 m")
   a2, a20 = np.deg2rad(f["WD_2m"]), np.deg2rad(f["WD_20m"])
   du = -f["WS_20m"] * np.sin(a20) + f["WS_2m"] * np.sin(a2)
   dv = -f["WS_20m"] * np.cos(a20) + f["WS_2m"] * np.cos(a2)
   f["vector_shear_2_20m"] = np.hypot(du, dv) / 18.
   missing = [c for c in COLS if c not in f]
   if missing:
      raise ValueError(f"frozen candidate fields missing: {missing}")
   return f[COLS].replace([np.inf, -np.inf], np.nan)


def window_table(features, length, releases):
   releases = pd.DatetimeIndex(releases)
   if (releases.hour != 0).any():
      raise ValueError("release time is fixed at midnight")
   end = releases - pd.Timedelta(hours=2)
   start = end - pd.Timedelta(hours=length - 1)
   grid = pd.date_range(min(start.min(), features.index.min()),
                        max(end.max(), features.index.max()), freq="h")
   f = features.reindex(grid)[COLS]
   means = f.rolling(length, min_periods=1).mean().reindex(end)
   counts = f.notna().rolling(length, min_periods=1).sum().reindex(end).fillna(0)
   means.index, counts.index = releases, releases
   threshold = int(np.ceil(length * COVERAGE))
   out = means.copy()
   out["common_valid"] = (counts >= threshold).all(axis=1)
   out["start"] = start
   out["end"] = end
   out["length_h"] = length
   for c in COLS:
      out[f"n__{c}"] = counts[c].astype(int)
   complete = f.notna().all(axis=1).rolling(length, min_periods=1).sum().reindex(end)
   out["all12_complete_hours"] = complete.to_numpy(dtype=int)
   out.index.name = "release"
   return out


def connected_clusters(features):
   warm = features.loc[features.index.year.isin(TRAIN_YEARS)
                       & features.index.month.isin([6, 7, 8, 9]), COLS].dropna()
   corr = warm.corr(method="spearman", min_periods=200)
   unknown = [(a, b) for i, a in enumerate(COLS) for b in COLS[i + 1:]
              if not np.isfinite(corr.loc[a, b])]
   edges = [(a, b, float(corr.loc[a, b])) for i, a in enumerate(COLS)
            for b in COLS[i + 1:]
            if np.isfinite(corr.loc[a, b]) and abs(corr.loc[a, b]) > .8]
   confirmed = len(warm) >= 200 and not unknown
   groups = components_from_edges(edges) if confirmed else []
   hours_by_year = {int(y): int(n) for y, n in warm.groupby(warm.index.year).size().items()}
   effective = tuple(sorted(hours_by_year))
   excluded = [int(y) for y in TRAIN_YEARS if int(y) not in effective]
   training = (f"相关聚类用2021–2024年6–9月十二项共同有效小时{len(warm)}小时"
               f"（分年{hours_by_year}）；窗口参照年及2022年缺测情况见training_year_audit"
               "（窗口要求80%共同覆盖，与逐时相关聚类不是同一资格）")
   if excluded:
      training += f"；聚类未参与年{','.join(map(str, excluded))}"
   return {"confirmed": confirmed, "common_hours": len(warm),
           "training": training, "hours_by_year": hours_by_year,
           "hourly_cluster_years": list(effective), "excluded_years": excluded,
           "method": "Spearman绝对相关>0.8完整连通簇；至少200共同小时",
           "unknown_pairs": unknown, "edges": edges, "clusters": groups,
           "correlation": corr.to_dict(),
           "missing_policy": "缺相关不填零；结构不可确认时不声称去冗余前五"}


def components_from_edges(edges):
   adjacent = {c: set() for c in COLS}
   for a, b, *_ in edges:
      adjacent[a].add(b)
      adjacent[b].add(a)
   seen, groups = set(), []
```

```
    for c in COLS:
        if c in seen:
            continue
        todo, component = [c], []
        while todo:
            v = todo.pop()
            if v not in seen:
                seen.add(v)
                component.append(v)
                todo.extend(adjacent[v] - seen)
        groups.append(sorted(component, key=ORDER.get))
    return groups


def robust_baseline(reference):
    center = reference.median()
    mad = reference.sub(center).abs().median() * 1.4826
    iqr = (reference.quantile(.75) - reference.quantile(.25)) / 1.349
    scale = mad.where(mad > 0, iqr)
    valid = np.isfinite(scale) & (scale > 0)
    method = pd.Series("MAD×1.4826", index=COLS)
    method.loc[~(mad > 0)] = "IQR/1.349回退"
    method.loc[~valid] = "不可排名：MAD及IQR均无有效正尺度"
    return center, scale.where(valid), method


def reference_rows(windows, query, years=TRAIN_YEARS):
    mask = windows.index.year.isin(years) & nearby(windows.index, query)
    return windows.loc[mask]


def effective_years(reference):
    """Training years that actually contribute a common-valid reference window."""
    valid = reference.loc[reference.common_valid]
    return tuple(sorted(int(y) for y in valid.index.year.unique()))


def year_window_audit(windows):
    """Per-length, per-year reference availability and the features that fail coverage."""
    audit = {}
    for length, table in windows.items():
        threshold = int(np.ceil(length * COVERAGE))
        ref = reference_rows(table, TARGET)
        by_year = {}
        for year in TRAIN_YEARS:
            sub = ref.loc[ref.index.year == int(year)]
            failed = sorted({c for _, row in sub.iterrows()
                             for c in COLS if int(row[f"n__{c}"]) < threshold})
            by_year[str(int(year))] = {
                "reference_rows": int(len(sub)),
                "common_valid_rows": int(sub.common_valid.sum()),
                "failed_features": failed,
            }
        audit[str(length)] = by_year
    return audit


def score_window(windows, query, clusters, reference=None):
    query = pd.Timestamp(query)
    row = windows.loc[query]
    rawref = reference_rows(windows, query) if reference is None else reference
    ref = rawref.loc[rawref.common_valid, COLS]
    summary = {"release": query, "start": row.start, "end": row.end,
            "length_h": int(row.length_h), "common_valid": bool(row.common_valid),
            "candidate_reference_windows": len(rawref), "reference_windows": len(ref),
            "reference_counts_by_year": ref.groupby(ref.index.year).size().to_dict(),
            "coverage": {c: int(row[f"n__{c}"]) / int(row.length_h) for c in COLS},
            "all12_complete_hours": int(row.all12_complete_hours)}
    if not row.common_valid or len(ref) < 2:
        summary.update(status="不可评估：目标共同覆盖不足或参照少于2窗", rows=[],
                       full_order=[], display=[], top5=[], maximum_A=np.nan)
        return summary
    center, scale, method = robust_baseline(ref)
    delta = row[COLS].astype(float) - center
    z, amplitude = delta / scale, (delta / scale).abs()
    refa = ref.sub(center).div(scale).abs()
    rows = []
    for c in COLS:
        ok = np.isfinite(amplitude[c])
        tail = (1 + int((refa[c] >= amplitude[c]).sum())) / (len(ref) + 1) if ok else np.nan
        rows.append({"feature": c, "name": NAMES[c], "unit": UNITS[c],
                     "mean": float(row[c]), "baseline_median": center[c],
                     "robust_scale": scale[c], "scale_method": method[c],
                     "delta": delta[c], "Z": z[c], "A": amplitude[c],
                     "tail_rank": tail, "rankable": ok})
    order = sorted([r for r in rows if r["rankable"]],
                   key=lambda r: (-r["A"], ORDER[r["feature"]]))
    place = {r["feature"]: i + 1 for i, r in enumerate(order)}
    for r in rows:
        r["full_rank"] = place.get(r["feature"])
    display = []
    for i, component in enumerate(clusters, 1):
        available = [r for r in order if r["feature"] in component]
        if available:
            winner = available[0]
            display.append({**winner, "cluster_id": i, "members": component,
```

```
                    "excluded": [c for c in component if c != winner["feature"]]})
    display.sort(key=lambda r: (-r["A"], ORDER[r["feature"]]))
    summary.update(status="可评估" if clusters else "完整得分可评估；相关簇不可确认",
                   rows=rows, full_order=[r["feature"] for r in order], display=display,
                   top5=[r["feature"] for r in display[:5]],
                   maximum_A=max((r["A"] for r in order), default=np.nan))
    return summary


def compare_scores(left, right):
    common = [c for c in left["full_order"] if c in right["full_order"]]
    tau = float(kendalltau([left["full_order"].index(c) for c in common],
                           [right["full_order"].index(c) for c in common]).statistic) if len(common) > 1 else np.nan
    return {"common_ranked_features": len(common), "full_rank_Kendall_tau": tau,
            "top5_overlap": len(set(left["top5"]) & set(right["top5"]))}


def score_records(result, method, **extra):
    return [{"method": method, "release": result["release"], "length_h": result["length_h"],
             "reference_windows": result["reference_windows"], **extra, **r}
            for r in result["rows"]]


def seven_day_reference(reference):
    selected = []
    for _, yearly in reference.groupby(reference.index.year):
        # Calendar scheduling precedes validity screening, so gaps never shift the schedule.
        selected.extend(pd.date_range(yearly.index.min(), yearly.index.max(), freq="7D"))
    return reference.loc[reference.index.isin(selected)]


def background_sensitivity(windows, target, clusters):
    reference = reference_rows(windows, TARGET)
    valid_reference = reference.loc[reference.common_valid]
    available_years = effective_years(reference)
    excluded_years = {str(int(y)): "该年无共同有效参照窗（ustar/TKE_obs缺测）"
                      for y in TRAIN_YEARS if int(y) not in available_years}
    rng = np.random.default_rng(SEED + int(target["length_h"]))
    replicates, records, choices = [], [], []
    for i in range(N_BOOT):
        if len(available_years) >= 2:
            selected = rng.choice(available_years, len(available_years), replace=True)
            sampled = pd.concat([valid_reference.loc[valid_reference.index.year == y] for y in selected])
            result = score_window(windows, TARGET, clusters, sampled)
            choices.append([int(y) for y in selected])
        else:
            result = {"release": TARGET, "length_h": int(target["length_h"]), "rows": [],
                      "full_order": [], "display": [], "top5": [], "maximum_A": np.nan,
                      "reference_windows": 0, "status": "不可评估：有效训练年少于2"}
            choices.append([])
        replicates.append(result)
        records.extend(score_records(result, "year_block_bootstrap", replicate=i))
    intervals = []
    for c in COLS:
        rows = [next((r for r in v["rows"] if r["feature"] == c), {}) for v in replicates]
        valid = [r for r in rows if r.get("rankable", False)]
        intervals.append({"feature": c, "valid_replicates": len(valid),
                          "intervals_025_50_975": {
                              key: np.quantile([r[key] for r in valid], [.025, .5, .975]).tolist()
                              if valid else [np.nan] * 3
                              for key in ("A", "Z", "delta", "baseline_median", "robust_scale", "tail_rank")},
                          "top5_selection_fraction": sum(c in v["top5"] for v in replicates) / N_BOOT})
    loyo = []
    for year in TRAIN_YEARS:
        year = int(year)
        if year not in available_years:
            loyo.append({"omitted_year": year, "status": "不可评估：该训练年无共同有效参照窗",
                         "reference_windows": 0, "top5": [], "common_ranked_features": 0,
                         "full_rank_Kendall_tau": None, "top5_overlap": None,
                         "reason": excluded_years[str(year)]})
            continue
        r = score_window(windows, TARGET, clusters, reference.loc[reference.index.year != year])
        records.extend(score_records(r, "leave_one_training_year_out", omitted_year=year))
        loyo.append({"omitted_year": year, "status": r["status"],
                     "reference_windows": r["reference_windows"], "top5": r["top5"],
                     **compare_scores(target, r)})
    seven = None
    if int(target["length_h"]) == 168:
        scheduled = seven_day_reference(reference)
        r = score_window(windows, TARGET, clusters, scheduled)
        records.extend(score_records(r, "fixed_seven_day_nonoverlap"))
        seven = {**r, "scheduled_dates_before_validity": scheduled.index.tolist(),
                 "comparison_with_daily": compare_scores(target, r)}
    return {"year_block_bootstrap": {"replicates": N_BOOT, "seed": SEED + int(target["length_h"]),
                                     "status": "可评估" if len(available_years) >= 2 else "不可评估：有效训练年少于2",
                                     "available_training_years": [int(y) for y in available_years],
                                     "excluded_training_years": excluded_years,
                                     "sampled_training_years": choices, "feature_intervals": intervals,
                                     "scope": "整年重抽样仅从有效训练年重估参照中位数与尺度；相关簇不重拟合；非灾害置信区间"},
            "leave_one_training_year_out": loyo, "fixed_seven_day_nonoverlap": seven}, records


def evaluate(features):
    f = candidate_features(features)
    dates = all_release_dates()
    windows = {length: window_table(f, length, dates) for length in LENGTHS}
```

```
effective_by_length = {str(length): [int(y) for y in effective_years(reference_rows(table, TARGET))]
                       for length, table in windows.items()}
cluster_info = connected_clusters(f)
window_year_text = ", ".join(f"{L}h=" + "/".join(map(str, years))
                             for L, years in effective_by_length.items())
cluster_info["window_reference_years_by_length"] = effective_by_length
cluster_info["training"] += (f"；窗口参照有效年{window_year_text}，"
                             "2022年因80%共同覆盖为0不进入窗口参照")
clusters = cluster_info["clusters"]
pseudo_dates = dates[(dates.year == 2025) & nearby(dates, TARGET)]
target_results, pseudo, sensitivities, records, audits, used_dates = {}, {}, {}, [], [], {}
for length, table in windows.items():
   target = score_window(table, TARGET, clusters)
   target_results[str(length)] = target
   pseudo_results = [score_window(table, d, clusters) for d in pseudo_dates]
   records.extend(score_records(target, "target"))
   rows = []
   for r in pseudo_results:
      comparison = compare_scores(target, r)
      records.extend(score_records(r, "pseudo_2025"))
      rows.append({k: r[k] for k in ("release", "status", "common_valid", "reference_windows", "top5", "
           maximum_A")}
                  | comparison)
   pseudo[str(length)] = {"candidate_days": len(pseudo_dates),
                          "common_valid_days": sum(r["common_valid"] for r in pseudo_results),
                          "evaluable_days": sum(bool(r["rows"]) for r in pseudo_results),
                          "days": rows,
                          "top5_selection_fraction": {c: sum(c in r["top5"] for r in pseudo_results)
                                                      / max(1, sum(bool(r["rows"]) for r in pseudo_results))
                                                     for c in COLS},
                          "scope": "2025只作冻结规则外检；不是有真值的非灾日，不计算伪警率"}
   sensitivity, more = background_sensitivity(table, target, clusters)
   sensitivities[str(length)] = sensitivity
   records.extend(more)
   used = set(pseudo_dates) | {TARGET}
   for d in list(pseudo_dates) + [TARGET]:
      used.update(reference_rows(table, d).index)
   used_dates[length] = used
   selected = table.loc[table.index.isin(used)]
   for release, r in selected.iterrows():
      audits.append({"release": release, "length_h": length, "start": r.start, "end": r.end,
                     "role": "target" if release == TARGET else "pseudo_2025" if release.year == 2025 else "
                          training_reference",
                     "common_valid": bool(r.common_valid), "all12_complete_hours": int(r.all12_complete_hours),
                     "failed_features": ";".join(c for c in COLS if r[f"n__{c}"] < np.ceil(length * COVERAGE)),
                     **{f"n__{c}": int(r[f"n__{c}"]) for c in COLS}})
common_dates_results = []
for d in [TARGET] + list(pseudo_dates):
   refs = [reference_rows(windows[length], d) for length in LENGTHS]
   shared = refs[0].index[refs[0].common_valid].intersection(refs[1].index[refs[1].common_valid])
   result = {length: score_window(windows[length], d, clusters, refs[i].loc[shared])
             for i, length in enumerate(LENGTHS)}
   both = all(r["common_valid"] and bool(r["rows"]) for r in result.values())
   for r in result.values():
      records.extend(score_records(r, "common_reference_dates", both_durations_evaluable=both))
   common_dates_results.append({"release": d, "both_durations_evaluable": both,
                                "shared_reference_windows": len(shared),
                                **compare_scores(result[168], result[72]),
                                "top5_168": result[168]["top5"], "top5_72": result[72]["top5"],
                                "target_scores" if d == TARGET else "pseudo_scores_omitted": result if d == TARGET else
                                     None})
effective_union = {y for years in effective_by_length.values() for y in years}
year_audit = year_window_audit(windows)
summary = {"schema_version": 1, "module": "step01b_preimpact", "seed": SEED,
           "frozen_protocol": {"target_release": TARGET, "last_usable_hour": TARGET - pd.Timedelta(hours=2),
                               "unused_buffer_label": TARGET - pd.Timedelta(hours=1),
                               "main_length_h": 168, "sensitivity_length_h": 72,
                               "training_years": TRAIN_YEARS, "pseudo_year": 2025,
                               "calendar": "共同非闰年日历±15日；2月29日不作为发布日",
                               "coverage_per_candidate": COVERAGE, "all_candidates_must_pass": True,
                               "imputation": "不插补；每项窗口均值仅用该项有效小时，十二项均达80%才接纳",
                               "scoring": "训练窗口中位数与MAD×1.4826；零MAD仅回退IQR/1.349；A=|Z|降序，固定候选顺序决胜",
                               "tail_rank": "(1+参照幅度>=查询幅度的窗数)/(Nref+1)，不参与加权",
                               "labels_or_priors_used": False,
                               "candidates": [{"feature": c, "name": name, "unit": unit, "relationship": relationship}
                                              for c, name, unit, relationship in CANDIDATES]},
           "clusters": cluster_info,
           "training_year_audit": {
              "configured_training_years": [int(y) for y in TRAIN_YEARS],
              "effective_training_years_by_length": effective_by_length,
              "excluded_training_years": {
                 str(int(y)): "该年无共同有效参照窗（ustar/TKE_obs缺测）"
                 for y in TRAIN_YEARS if int(y) not in effective_union},
              "by_length": year_audit,
              "policy": "留一年仅对有效年份执行；无窗年份标为不可评估，不用空操作结果冒充敏感性；年块自举仅从有效年份重抽样"},
           "target": target_results, "pseudo_2025": pseudo,
           "sensitivity": sensitivities,
           "duration_comparison": {"independently_screened": compare_scores(target_results["168"], target_results["
                72"]),
                                   "common_reference_dates": common_dates_results,
                                   "scope": "共同日期对照对每个查询分别取两长度均合格的训练发布日交集；2025查询也报告两长度共同资格"},
           "observed_flux_policy": "ustar与TKE_obs来自FLUX观测统计量；仅屏蔽负值，不插补、不作未经元数据支持的上界饱和截尾；测量
                高度未知，不补为10米或20米",
           "conditional_selection": "窗口审计CSV列出全部实际查询/参照小时起止与各项计数，含被共同覆盖规则排除的窗。FLUX缺测可使可
```

```
                用窗成为条件选样，数量不等于独立事件数。",
            "feature_metadata": serial(features.attrs), "limitations": LIMITATION}
    scores = pd.DataFrame(records)
    for c in ("method", "release", "length_h", "feature", "A", "omitted_year"):
        if c not in scores:
            scores[c] = pd.Series(index=scores.index, dtype="object" if c in ("method", "feature") else "float64")
    return summary, scores, pd.DataFrame(audits), windows


def plot_outputs(summary, records, directory):
    plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "Noto Sans SC", "DejaVu Sans"]
    plt.rcParams["axes.unicode_minus"] = False
    directory.mkdir(parents=True, exist_ok=True)
    labels = [NAMES[c] for c in COLS]
    y = np.arange(len(COLS))
    target = summary["target"]["168"]
    target_rows = {r["feature"]: r for r in target["rows"]}
    if not target_rows:
        paths = []
        for name in ("01b_冲击前窗口异常.png", "01b_伪事件与背景敏感性.png"):
            fig, ax = plt.subplots(figsize=(12, 6), constrained_layout=True)
            ax.axis("off")
            ax.text(.5, .65, "冻结冲击前168小时旁证不可评估", ha="center", fontsize=20)
            ax.text(.5, .45, target["status"], ha="center", fontsize=13)
            ax.text(.5, .25, "不以缺测代表平缓，不改候选或截止时刻；逐窗覆盖详见审计CSV", ha="center", fontsize=12)
            path = directory / name
            fig.savefig(path, dpi=170); plt.close(fig)
            paths.append(path)
        return paths
    a = np.array([target_rows.get(c, {}).get("A", np.nan) for c in COLS])
    fig, axes = plt.subplots(2, 2, figsize=(16, 12), constrained_layout=True)
    colors = ["#236a72" if c in target["top5"] else "#a9c8cc" for c in COLS]
    axes[0, 0].barh(y, a, color=colors)
    axes[0, 0].set_yticks(y, labels); axes[0, 0].invert_yaxis()
    axes[0, 0].set_xlabel("双侧异常幅度 A=|Z|（无量纲）")
    axes[0, 0].set_title("(a) 冲击前168小时完整十二候选；深色为去冗余前五")
    common = summary["duration_comparison"]["common_reference_dates"][0]["target_scores"]
    for length, offset, color in ((168, -.17, "#236a72"), (72, .17, "#c87243")):
        rows = {r["feature"]: r for r in common[length]["rows"]}
        axes[0, 1].barh(y + offset, [rows.get(c, {}).get("A", np.nan) for c in COLS],
                        height=.32, color=color, label=f"{length}小时")
    axes[0, 1].set_yticks(y, labels); axes[0, 1].invert_yaxis(); axes[0, 1].legend()
    axes[0, 1].set_xlabel("同一训练发布日交集上的 A")
    axes[0, 1].set_title("(b) 时长敏感性：同参照日期，不挑有利窗口")
    corr = pd.DataFrame(summary["clusters"]["correlation"]).reindex(index=COLS, columns=COLS)
    image = axes[1, 0].imshow(corr, vmin=-1, vmax=1, cmap="RdBu_r")
    axes[1, 0].set_xticks(y, labels, rotation=65, ha="right", fontsize=8)
    axes[1, 0].set_yticks(y, labels, fontsize=8)
    axes[1, 0].set_title(f"(c) 冻结暖季Spearman簇：共同小时 {summary['clusters']['common_hours']}")
    fig.colorbar(image, ax=axes[1, 0], shrink=.75, label="Spearman相关")
    for length, offset, color in ((168, -.17, "#236a72"), (72, .17, "#c87243")):
        coverage = summary["target"][str(length)]["coverage"]
        axes[1, 1].barh(y + offset, [coverage[c] for c in COLS], height=.32,
                        color=color, label=f"{length}小时")
    axes[1, 1].axvline(.8, color="#9b352d", ls="--", label="每项80%门槛")
    axes[1, 1].set_yticks(y, labels); axes[1, 1].invert_yaxis(); axes[1, 1].set_xlim(0, 1.04)
    axes[1, 1].set_xlabel("目标窗有效小时比例"); axes[1, 1].legend(fontsize=8)
    axes[1, 1].set_title("(d) 十二候选全部合格才接纳；FLUX缺测可能引入选样")
    fig.suptitle("冻结非循环窗口异常旁证 | 2026-08-26 00时发布，前日22时截止\n异常特殊性不是灾害关联或概率；共享观测不等于独立证据",
                 fontsize=13)
    p1 = directory / "01b_冲击前窗口异常.png"
    fig.savefig(p1, dpi=170); plt.close(fig)

    fig, axes = plt.subplots(2, 2, figsize=(16, 12), constrained_layout=True)
    pseudo = records[(records.method == "pseudo_2025") & (records.length_h == 168)]
    dates = pd.DatetimeIndex([v["release"] for v in summary["pseudo_2025"]["168"]["days"]])
    matrix = pseudo.pivot(index="feature", columns="release", values="A").reindex(index=COLS, columns=dates)
    image = axes[0, 0].imshow(matrix, aspect="auto", cmap="YlOrBr")
    ticks = np.arange(0, len(dates), 5)
    axes[0, 0].set_xticks(ticks, [dates[i].strftime("%m-%d") for i in ticks], rotation=35)
    axes[0, 0].set_yticks(y, labels, fontsize=8)
    axes[0, 0].set_title("(a) 2025同季节伪事件：冻结规则异常幅度")
    fig.colorbar(image, ax=axes[0, 0], shrink=.8, label="A（不是灾害概率）")
    intervals = summary["sensitivity"]["168"]["year_block_bootstrap"]["feature_intervals"]
    by_feature = {r["feature"]: r for r in intervals}
    for i, c in enumerate(COLS):
        lo, mid, hi = by_feature[c]["intervals_025_50_975"]["A"]
        axes[0, 1].hlines(i, lo, hi, color="#236a72", lw=3)
        axes[0, 1].plot(mid, i, "|", color="#236a72", ms=10)
    axes[0, 1].scatter(a, y, c="#bf613d", s=24, label="完整训练背景点值", zorder=3)
    axes[0, 1].set_yticks(y, labels); axes[0, 1].invert_yaxis(); axes[0, 1].legend()
    effective_168 = summary["training_year_audit"]["effective_training_years_by_length"]["168"]
    axes[0, 1].set_title(f"(b) 200次整年块背景敏感性：2.5%—97.5%端点（有效训练年{effective_168}）")
    axes[0, 1].set_xlabel("A：仅有效训练年，不是灾害置信区间")
    axes[1, 0].barh(y, [by_feature[c]["top5_selection_fraction"] for c in COLS],
                    color="#a9c8cc", label="年块重抽样")
    axes[1, 0].scatter([summary["pseudo_2025"]["168"]["top5_selection_fraction"][c] for c in COLS],
                       y, color="#bf613d", s=25, label="2025有效伪事件")
    axes[1, 0].set_yticks(y, labels); axes[1, 0].invert_yaxis(); axes[1, 0].set_xlim(0, 1.05)
    axes[1, 0].set_xlabel("进入去冗余前五的比例（非灾害频率）")
    axes[1, 0].set_title("(c) 前五稳定性而非灾害鉴别力"); axes[1, 0].legend(fontsize=8)
    leave = records[(records.method == "leave_one_training_year_out") & (records.length_h == 168)]
```

```
    palette = {2021: "#236a72", 2022: "#c87243", 2023: "#687c45", 2024: "#8e6685"}
    for year in effective_168:
        rows = leave[leave.omitted_year == year].set_index("feature").reindex(COLS)
        axes[1, 1].plot(rows.A, y, ".", color=palette.get(year, "#777777"), label=f"剔除{year}年")
    seven = summary["sensitivity"]["168"]["fixed_seven_day_nonoverlap"]
    rows = {r["feature"]: r for r in seven["rows"]}
    axes[1, 1].scatter([rows.get(c, {}).get("A", np.nan) for c in COLS], y,
                       marker="x", color="black", s=27, label="固定7日非重叠参照")
    axes[1, 1].set_yticks(y, labels); axes[1, 1].invert_yaxis(); axes[1, 1].legend(fontsize=8)
    axes[1, 1].set_title("(d) 逐训练年留一与非重叠参照（2022无共同有效窗，不可评估）")
    axes[1, 1].set_xlabel("A: 同一规则，不挑最有利榜单")
    fig.suptitle("伪事件外检与有限历史背景敏感性丨伪事件不是已证实非灾日\n尾秩不是显著性p值；每日窗口重叠，窗口数不是独立事件数",
                 fontsize=13)
    p2 = directory / "01b_伪事件与背景敏感性.png"
    fig.savefig(p2, dpi=170); plt.close(fig)
    return [p1, p2]


def main():
    if io.ROOT.resolve() != PROJECT:
        raise RuntimeError(f"JILONG_ROOT must point to this v2 project, not {io.ROOT}")
    features = mt.build_feature_table(io.load_station())
    summary, records, audit, _ = evaluate(features)
    paths = {
        "summary": io.DATA_OUT / "01b_冲击前窗口异常.json",
        "audit": io.DATA_OUT / "01b_窗口覆盖审计.csv",
        "query_scores": io.DATA_OUT / "01b_查询逐项异常.csv",
        "sensitivity_scores": io.DATA_OUT / "01b_背景敏感性逐项.csv",
    }
    audit.to_csv(paths["audit"], index=False, encoding="utf-8-sig")
    query = records.method.isin(["target", "pseudo_2025", "common_reference_dates"])
    records.loc[query].to_csv(paths["query_scores"], index=False, encoding="utf-8-sig")
    records.loc[~query].to_csv(paths["sensitivity_scores"], index=False, encoding="utf-8-sig")
    figures = plot_outputs(summary, records, io.FIG_OUT)
    summary["outputs"] = {**{k: str(v.relative_to(PROJECT)) for k, v in paths.items()},
                          "figures": [str(p.relative_to(PROJECT)) for p in figures]}
    paths["summary"].write_text(json.dumps(serial(summary), ensure_ascii=False, indent=2, allow_nan=False), encoding
        ="utf-8")
    for length in LENGTHS:
        target = summary["target"][str(length)]
        print(f"01b {length}h: {target['status']}; reference={target['reference_windows']}; top5={target['top5']}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

step01c_unified_association.py　31 项统一候选关联与去冗余

```
"""问题一正式候选池：冲击前环境关联的冻结单事件排名。

协议在查看本轮排名前冻结：

* 31 个唯一候选键，MET/RADM/SOIL/FLUX = 19/5/3/4；
* 目标窗为事件前 168 h，72 h 只作敏感性；
* 逐候选目标窗和参照窗均至少 80% 有效；
* 评分用参照中位数与 MAD×1.4826，零 MAD 回退 IQR/1.349；
* 相关去冗余用 2021--2024 暖季逐对 Spearman，
  每对至少 200 个共同有效小时，|rho|>0.8 连边后取连通簇。

输出是单事件环境特殊性关联，不是因果效应、发生概率或独立灾害验证。
"""
from __future__ import annotations

import json
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy import stats

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "Noto Sans SC", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams.update({
    "font.size": 23,
    "axes.titlesize": 22,
    "axes.labelsize": 20,
    "xtick.labelsize": 17,
    "ytick.labelsize": 17,
    "legend.fontsize": 15,
})

PROJECT = Path(__file__).resolve().parents[2]
OUT = PROJECT / "求解结果" / "数据"
FIG = PROJECT / "求解结果" / "图表"
CACHE = OUT / "cache"
EVENT_RELEASE = pd.Timestamp("2026-08-26")
REFERENCE_YEARS = (2021, 2023, 2024)
```

```
WINDOW_H = 168
SENSITIVITY_H = 72
MIN_COVERAGE = 0.80
CALENDAR_RADIUS = 15
MIN_PAIR_HOURS = 200
CLUSTER_ABS_RHO = 0.8

# The order is part of the frozen protocol and only resolves exact ties.
CANDIDATES = [
    # MET (19)
    ("Ta_2m", "2 m气温", "MET", "degC", "原始气象观测", ""),
    ("RH_2m", "2 m相对湿度", "MET", "%", "原始气象观测", ""),
    ("VPD2", "2 m水汽压差", "MET", "hPa", "温湿观测派生", "由2 m温度和相对湿度计算"),
    ("WS_10m", "10 m风速", "MET", "m/s", "原始气象观测", ""),
    ("vector_shear_2_20m", "2-20 m矢量风切变", "MET", "1/s", "风廓线派生", "由2/20 m风速和风向计算"),
    ("shear_1_20m", "1-20 m标量风切变", "MET", "1/s", "风廓线派生",
     "由WS_1m与WS_20m计算 (WS_20m-WS_1m)/19 m"),
    ("Pressure", "气压", "MET", "hPa", "原始气象观测", ""),
    ("API", "前期降水指数", "MET", "mm", "降水派生", "由历史小时降水递推"),
    ("P", "小时降水", "MET", "mm/h", "原始降水观测", ""),
    ("P_3h", "3 h累计降水", "MET", "mm", "降水派生", "3 h滚动累计"),
    ("P_24h", "24 h累计降水", "MET", "mm", "降水派生", "24 h滚动累计"),
    ("PDD_3d", "3 d正积温", "MET", "degC d", "温度派生", "3 d正积温"),
    ("PDD_7d", "7 d正积温", "MET", "degC d", "温度派生", "7 d正积温"),
    ("AP_7d", "7 d前期降水", "MET", "mm", "降水派生", "7 d滚动累计"),
    ("rise_24h", "24 h升温", "MET", "degC", "温度派生", "24 h温度变化"),
    ("freeze_thaw", "24 h冻融交替次数", "MET", "次/24h", "温度派生",
     "相邻小时温度乘积跨过0 degC的滚动24 h计数"),
    ("inv_max", "原始逆温强度", "MET", "K/100m", "温度廓线派生",
     "取未订正相邻温度层的原始正温度直减率最大值"),
    ("Ri_b", "整体理查森数", "MET", "1", "静力订正廓线派生",
     "由2 m与20 m静力订正虚位温、水平风速和层厚计算整体Ri_b"),
    ("dtheta_1_20", "1-20 m位温差", "MET", "K", "静力订正廓线派生",
     "由静力订正后的theta_20m减theta_1m计算"),
    # RADM (5)
    ("Rn", "净辐射", "RADM", "W/m2", "辐射观测派生", "四分量净辐射"),
    ("clearness", "晴空指数", "RADM", "1", "辐射派生", "仅白昼物理有效"),
    ("albedo", "反照率", "RADM", "1", "辐射派生", "仅白昼物理有效"),
    ("melt_energy_hourly", "小时消融有效能量", "RADM", "MJ m^-2 h^-1",
     "能量派生", "Rn-G-H-LE截正后乘3600/1e6的逐小时能量, 不是历史累计"),
    ("Rsd", "下行短波辐射", "RADM", "W/m2", "原始辐射观测", ""),
    # SOIL (3)
    ("soilT_0.1m", "0.1 m土温", "SOIL", "degC", "土壤观测", ""),
    ("VWC_0.1m", "0.1 m土壤含水率", "SOIL", "m3/m3", "土壤观测", ""),
    ("VWC_0.4m", "0.4 m土壤含水率", "SOIL", "m3/m3", "土壤观测", ""),
    # FLUX (4)
    ("USTAR", "摩擦速度", "FLUX", "m/s", "涡动相关通量观测", ""),
    ("TKE", "观测湍流动能", "FLUX", "m2/s2", "涡动相关通量观测", ""),
    ("H", "感热通量", "FLUX", "W/m2", "涡动相关通量观测", ""),
    ("LE", "潜热通量", "FLUX", "W/m2", "涡动相关通量观测", ""),
]

SOURCE_FIELDS = {
    "Ta_2m": "Ta_2m",
    "RH_2m": "RH_2m",
    "VPD2": "Ta_2m;RH_2m",
    "WS_10m": "WS_10m",
    "vector_shear_2_20m": "WS_2m;WD_2m;WS_20m;WD_20m",
    "shear_1_20m": "WS_1m;WS_20m",
    "Pressure": "Pressure",
    "API": "P",
    "P": "P",
    "P_3h": "P",
    "P_24h": "P",
    "PDD_3d": "Ta_2m",
    "PDD_7d": "Ta_2m",
    "AP_7d": "P",
    "rise_24h": "Ta_2m",
    "freeze_thaw": "Ta_2m",
    "inv_max": "Ta_1m;Ta_2m;Ta_4m;Ta_10m;Ta_20m",
    "Ri_b": "Ta_1m_to_20m;RH_1m_to_20m;WS_1m_to_20m;WD_1m_to_20m;Pressure",
    "dtheta_1_20": "Ta_1m_to_20m;RH_1m_to_20m;Pressure",
    "Rn": "Rsd;Rsu;Rld;Rlu",
    "clearness": "Rsd",
    "albedo": "Rsd;Rsu",
    "melt_energy_hourly": "Rn;G_0.1m;H;LE;Ta_2m",
    "Rsd": "Rsd",
    "soilT_0.1m": "SOIL temperature 0.1 m",
    "VWC_0.1m": "SOIL VWC 0.1 m",
    "VWC_0.4m": "SOIL VWC 0.4 m",
    "USTAR": "FLUX USTAR",
    "TKE": "FLUX TKE",
    "H": "FLUX H",
    "LE": "FLUX LE",
}

DOWNSTREAM_ROLES = {
    "Ta_2m": ["Q2_melt_temperature", "Q3_energy_fit"],
    "RH_2m": ["Q3_energy_fit"],
    "Rsd": ["Q3_energy_fit"],
```

```
    "Rn": ["Q2_melt_energy_component"],
    "H": ["Q2_melt_energy_component"],
    "LE": ["Q2_melt_energy_component"],
    "melt_energy_hourly": ["Q2_melt_flux_proxy"],
    "API": ["Q2_rain_proxy"],
    "P": ["Q2_rain_proxy"],
    "P_3h": ["Q2_rain_proxy"],
    "P_24h": ["Q2_rain_proxy"],
}

HOUR_AVAILABILITY_RULE = (
    "hourly label t denotes the observed/derived value available after t+1 h; "
    "the release at 2026-08-26 00:00 uses observations only through 2026-08-25 22:00"
)

COLUMNS = [row[0] for row in CANDIDATES]
COLUMN_ORDER = {column: i for i, column in enumerate(COLUMNS)}


def serial(value):
    if isinstance(value, dict):
        return {str(k): serial(v) for k, v in value.items()}
    if isinstance(value, (list, tuple, np.ndarray)):
        return [serial(v) for v in value]
    if isinstance(value, (bool, np.bool_)):
        return bool(value)
    if isinstance(value, (int, np.integer)):
        return int(value)
    if isinstance(value, (float, np.floating)):
        return float(value) if np.isfinite(value) else None
    if isinstance(value, (pd.Timestamp, np.datetime64)):
        return str(pd.Timestamp(value))
    return value


def calendar_day(values):
    return np.array([pd.Timestamp(2001, d.month, d.day).dayofyear for d in values])


def nearby(dates, query, radius=CALENDAR_RADIUS):
    delta = np.abs(calendar_day(dates) - calendar_day(pd.DatetimeIndex([query]))[0])
    return np.minimum(delta, 365 - delta) <= radius


def window_mean(feature: pd.DataFrame, release, length: int):
    end = pd.Timestamp(release) - pd.Timedelta(hours=2)
    start = end - pd.Timedelta(hours=length - 1)
    block = feature.loc[start:end]
    return block.mean(), block.count(), start, end


def robust_score(target_mean, reference: pd.Series):
    values = pd.to_numeric(reference, errors="coerce").dropna().to_numpy()
    if len(values) < 5:
        return None
    center = float(np.median(values))
    mad = float(np.median(np.abs(values - center)) * 1.4826)
    iqr = float((np.quantile(values, .75) - np.quantile(values, .25)) / 1.349)
    scale = mad if mad > 0 else iqr
    if not np.isfinite(scale) or scale <= 0:
        return None
    delta = float(target_mean - center)
    return {
        "delta": delta,
        "A": abs(delta / scale),
        "signed_z": delta / scale,
        "baseline_median": center,
        "robust_scale": scale,
        "n_reference": len(values),
        "scale_method": "MAD*1.4826" if mad > 0 else "IQR/1.349",
        "tail_rank": float((1 + np.sum(np.abs(values - center) >= abs(delta))) / (len(values) + 1)),
    }


def build_reference(features, columns, length, query, years=REFERENCE_YEARS):
    dates = pd.date_range(
        f"{min(years)}-01-01", f"{max(years)}-12-31", freq="D"
    )
    dates = dates[~((dates.month == 2) & (dates.day == 29))]
    dates = dates[np.isin(dates.year, years) & nearby(dates, query)]
    threshold = int(np.ceil(length * MIN_COVERAGE))
    rows = []
    for release in dates:
        mean, count, start, end = window_mean(features[columns], release, length)
        rows.append({
            "release": release,
            "start": start,
            "end": end,
            **{f"mean__{c}": mean[c] for c in columns},
            **{f"count__{c}": count[c] for c in columns},
        })
    return pd.DataFrame(rows), threshold


def pairwise_spearman(features):
    warm = features.loc[
        features.index.year.isin(range(2021, 2025))
```

```
      & features.index.month.isin([6, 7, 8, 9]),
      COLUMNS,
   ]
   correlation = pd.DataFrame(np.nan, index=COLUMNS, columns=COLUMNS)
   pair_n = pd.DataFrame(0, index=COLUMNS, columns=COLUMNS, dtype=int)
   edges = []
   for index, left in enumerate(COLUMNS):
      for right in COLUMNS[index + 1:]:
         common = warm[[left, right]].dropna()
         pair_n.loc[left, right] = pair_n.loc[right, left] = len(common)
         rho = np.nan
         if len(common) >= MIN_PAIR_HOURS:
            value = stats.spearmanr(common[left], common[right], nan_policy="omit")
            if np.isfinite(value.statistic):
               rho = float(value.statistic)
               correlation.loc[left, right] = correlation.loc[right, left] = rho
         edges.append({
            "left": left,
            "right": right,
            "common_valid_hours": int(len(common)),
            "spearman_rho": rho,
            "edge_abs_rho_gt_0.8": bool(np.isfinite(rho) and abs(rho) > CLUSTER_ABS_RHO),
         })
   return correlation, pair_n, pd.DataFrame(edges)


def connected_components(columns, correlation):
   adjacent = {c: set() for c in columns}
   for index, left in enumerate(columns):
      for right in columns[index + 1:]:
         value = correlation.loc[left, right]
         if np.isfinite(value) and abs(value) > CLUSTER_ABS_RHO:
            adjacent[left].add(right)
            adjacent[right].add(left)
   seen, groups = set(), []
   for column in columns:
      if column in seen:
         continue
      todo, component = [column], []
      while todo:
         current = todo.pop()
         if current in seen:
            continue
         seen.add(current)
         component.append(current)
         todo.extend(adjacent[current] - seen)
      groups.append(sorted(component, key=COLUMN_ORDER.get))
   return groups


def rank_once(features, clusters, length, query, years=REFERENCE_YEARS):
   target_mean, target_count, target_start, target_end = window_mean(
      features[COLUMNS], query, length
   )
   references, threshold = build_reference(features, COLUMNS, length, query, years)
   detail = {}
   for column in COLUMNS:
      valid_ref = references.loc[
         references[f"count__{column}"] >= threshold,
         f"mean__{column}",
      ]
      if int(target_count[column]) < threshold:
         detail[column] = {
            "rankable": False,
            "reason": "目标窗有效小时不足80%",
            "target_valid_hours": int(target_count[column]),
            "required_hours": threshold,
            "reference_windows": int(len(valid_ref)),
         }
         continue
      score = robust_score(float(target_mean[column]), valid_ref)
      if score is None:
         detail[column] = {
            "rankable": False,
            "reason": "参照窗不足或稳健尺度为零",
            "target_valid_hours": int(target_count[column]),
            "required_hours": threshold,
            "reference_windows": int(len(valid_ref)),
         }
         continue
      detail[column] = {
         "rankable": True,
         "target_mean": float(target_mean[column]),
         "target_valid_hours": int(target_count[column]),
         "required_hours": threshold,
         "reference_windows": int(len(valid_ref)),
         **score,
      }

   cluster_rows = []
   for component in clusters:
      eligible = [column for column in component if detail.get(column, {}).get("rankable")]
      if not eligible:
         cluster_rows.append({"members": component, "representative": None, "status": "不可排名"})
         continue
      representative = max(eligible, key=lambda column: detail[column]["A"])
```

```
        cluster_rows.append({
            "members": component,
            "representative": representative,
            "A_abs": detail[representative]["A"],
            "signed_z": detail[representative]["signed_z"],
            "excluded_members": [column for column in component if column != representative],
        })
    ranked = sorted(
        [row for row in cluster_rows if row.get("representative")],
        key=lambda row: row["A_abs"],
        reverse=True,
    )
    for position, row in enumerate(ranked, 1):
        row["rank"] = position
    return {
        "window_length_h": length,
        "target_window": [str(target_start), str(target_end)],
        "reference_window_count": len(references),
        "reference_years": list(years),
        "feature_details": detail,
        "cluster_ranking": cluster_rows,
        "ranked_clusters": ranked,
        "top5": [row["representative"] for row in ranked[:5]],
    }


def overlap(left, right):
    return len(set(left) & set(right))


def candidate_ranking_rows(result: dict, metadata: dict) -> list[dict]:
    """Return every candidate, including an explicit unrankable disposition."""
    rows = []
    for column in sorted(result["feature_details"], key=COLUMN_ORDER.get):
        detail = result["feature_details"][column]
        rows.append({
            "rank": None,
            "window_length_h": result["window_length_h"],
            "target_window_start": result["target_window"][0],
            "target_window_end": result["target_window"][1],
            **metadata[column],
            "rankable": bool(detail.get("rankable", False)),
            "target_valid_hours": detail.get("target_valid_hours"),
            "required_hours": detail.get("required_hours"),
            "reference_windows": detail.get("reference_windows"),
            "target_mean": detail.get("target_mean"),
            "A_abs": detail.get("A"),
            "signed_z": detail.get("signed_z"),
            "delta": detail.get("delta"),
            "baseline_median": detail.get("baseline_median"),
            "robust_scale": detail.get("robust_scale"),
            "scale_method": detail.get("scale_method"),
            "tail_rank": detail.get("tail_rank"),
            "exclusion_reason": detail.get("reason"),
        })
    ranked = sorted(
        [row for row in rows if row["rankable"]],
        key=lambda row: (-float(row["A_abs"]), COLUMN_ORDER[row["variable"]]),
    )
    unrankable = [row for row in rows if not row["rankable"]]
    for rank, row in enumerate(ranked, 1):
        row["rank"] = rank
    return ranked + unrankable


def cluster_ranking_rows(result: dict, metadata: dict) -> list[dict]:
    """Return every correlation component, including non-rankable clusters."""
    ranked_by_representative = {
        row["representative"]: row for row in result["ranked_clusters"]
    }
    rows = []
    for cluster in result["cluster_ranking"]:
        representative = cluster.get("representative")
        ranked = ranked_by_representative.get(representative)
        detail = result["feature_details"].get(representative, {})
        rows.append({
            "rank": ranked.get("rank") if ranked else None,
            "window_length_h": result["window_length_h"],
            "target_window_start": result["target_window"][0],
            "target_window_end": result["target_window"][1],
            "rankable": bool(ranked),
            "cluster_status": "ranked" if ranked else "not_rankable",
            "representative": representative,
            "name": metadata[representative]["name"] if representative else None,
            "family": metadata[representative]["family"] if representative else None,
            "direction": (
                "偏高" if detail.get("signed_z", 0) > 0 else "偏低"
            ) if ranked else None,
            "A_abs": detail.get("A") if ranked else None,
            "signed_z": detail.get("signed_z") if ranked else None,
            "delta": detail.get("delta") if ranked else None,
            "baseline_median": detail.get("baseline_median") if ranked else None,
            "tail_rank": detail.get("tail_rank") if ranked else None,
            "member_count": len(cluster["members"]),
            "cluster_members": "|".join(cluster["members"]),
            "excluded_members": "|".join(cluster.get("excluded_members", [])),
            "display_top5": bool(ranked and ranked["rank"] <= 5),
```

```
        })
    return sorted(
        rows,
        key=lambda row: (
            not row["rankable"],
            row["rank"] if row["rankable"] else 10**9,
            row["cluster_members"],
        ),
    )


def main():
    FIG.mkdir(parents=True, exist_ok=True)
    features = pd.read_parquet(CACHE / "features.parquet")
    # Keep the transport key stable at the source feature layer while the
    # association dictionary exposes its true hourly-energy meaning.
    if "melt_energy_hourly" not in features and "cum_melt_energy" in features:
        features["melt_energy_hourly"] = features["cum_melt_energy"]
    missing = [column for column in COLUMNS if column not in features]
    if missing:
        raise ValueError(f"冻结候选字段缺失: {missing}")
    if len(COLUMNS) != 31 or len(set(COLUMNS)) != 31:
        raise AssertionError("必须冻结31个唯一候选键")
    family_count = pd.Series([row[2] for row in CANDIDATES]).value_counts().to_dict()
    expected = {"MET": 19, "RADM": 5, "SOIL": 3, "FLUX": 4}
    assert family_count == expected, family_count

    correlation, pair_n, edges = pairwise_spearman(features)
    clusters = connected_components(COLUMNS, correlation)
    main_168 = rank_once(features, clusters, WINDOW_H, EVENT_RELEASE)
    sensitivity_72 = rank_once(features, clusters, SENSITIVITY_H, EVENT_RELEASE)
    leave_one_year = {}
    for year in REFERENCE_YEARS:
        kept = tuple(y for y in REFERENCE_YEARS if y != year)
        result = rank_once(features, clusters, WINDOW_H, EVENT_RELEASE, kept)
        leave_one_year[str(year)] = {
            "omitted_year": year,
            "top5": result["top5"],
            "top5_overlap": overlap(main_168["top5"], result["top5"]),
            "ranked_representatives": [
                row["representative"] for row in result["ranked_clusters"]
            ],
        }

    pseudo = pd.Timestamp("2025-08-26")
    pseudo_168 = rank_once(features, clusters, WINDOW_H, pseudo, (2021, 2022, 2023))

    direct_roles = {
        "原始气象观测", "原始降水观测", "原始辐射观测", "土壤观测",
        "涡动相关通量观测",
    }
    metadata = {}
    for row in CANDIDATES:
        if not row[5] and row[4] not in direct_roles:
            raise AssertionError(f"derived candidate lacks derivation: {row[0]}")
        metadata[row[0]] = {
            "variable": row[0],
            "name": row[1],
            "family": row[2],
            "unit": row[3],
            "provenance_role": row[4],
            "derivation": row[5] or "direct field; no additional transformation",
            "source_fields": SOURCE_FIELDS.get(row[0], row[0]),
            "available_time_rule": HOUR_AVAILABILITY_RULE,
            "downstream_model_roles": DOWNSTREAM_ROLES.get(row[0], ["Q1_only"]),
            "shared_with_downstream_model": bool(row[0] in DOWNSTREAM_ROLES),
        }
    top5 = []
    for rank, representative in enumerate(main_168["top5"], 1):
        detail = main_168["feature_details"][representative]
        cluster = next(
            row for row in main_168["ranked_clusters"]
            if row["representative"] == representative
        )
        top5.append({
            "rank": rank,
            "variable": representative,
            "name": metadata[representative]["name"],
            "family": metadata[representative]["family"],
            "direction": "事件窗偏高" if detail["delta"] > 0 else "事件窗偏低",
            "A_abs": detail["A"],
            "signed_z": detail["signed_z"],
            "delta": detail["delta"],
            "baseline_median": detail["baseline_median"],
            "tail_rank": detail["tail_rank"],
            "cluster_members": cluster["members"],
            "cluster_representative_reason": "同簇内绝对稳健异常最大；只在展示层去冗余",
        })

    candidate_dict = []
    for row in CANDIDATES:
        column = row[0]
        detail = main_168["feature_details"].get(column, {})
        candidate_dict.append({
            **metadata[column],
            "candidate_eligible": True,
```

```
        "target_valid_hours": detail.get("target_valid_hours"),
        "required_hours": detail.get("required_hours"),
        "reference_windows": detail.get("reference_windows"),
        "rankable": bool(detail.get("rankable", False)),
        "exclusion_reason": detail.get("reason"),
    })
audit_fields = (
    "variable",
    "name",
    "family",
    "unit",
    "provenance_role",
    "derivation",
    "source_fields",
    "available_time_rule",
    "downstream_model_roles",
    "shared_with_downstream_model",
    "target_valid_hours",
    "required_hours",
    "rankable",
)
if len(candidate_dict) != 31 or len({row["variable"] for row in candidate_dict}) != 31:
    raise AssertionError("candidate dictionary must contain 31 unique keys")
if any(not row.get(field) and row.get(field) is not False for row in candidate_dict for field in audit_fields):
    raise AssertionError("candidate dictionary audit field is empty")
pd.DataFrame(candidate_dict).to_csv(
    OUT / "01c_候选字典.csv", index=False, encoding="utf-8-sig"
)
edges.to_csv(OUT / "01c_相关边表.csv", index=False, encoding="utf-8-sig")

score_rows = []
for row in CANDIDATES:
    column = row[0]
    detail = main_168["feature_details"].get(column, {})
    score_rows.append({
        **metadata[column],
        "rankable": bool(detail.get("rankable", False)),
        "target_valid_hours": detail.get("target_valid_hours"),
        "reference_windows": detail.get("reference_windows"),
        "A_abs": detail.get("A"),
        "signed_z": detail.get("signed_z"),
        "delta": detail.get("delta"),
        "baseline_median": detail.get("baseline_median"),
        "tail_rank": detail.get("tail_rank"),
        "exclusion_reason": detail.get("reason"),
    })
pd.DataFrame(score_rows).to_csv(OUT / "01c_逐候选得分.csv", index=False, encoding="utf-8-sig")

candidate_168_rows = candidate_ranking_rows(main_168, metadata)
candidate_72_rows = candidate_ranking_rows(sensitivity_72, metadata)
cluster_168_rows = cluster_ranking_rows(main_168, metadata)
cluster_72_rows = cluster_ranking_rows(sensitivity_72, metadata)
if len(candidate_168_rows) != 31 or len(candidate_72_rows) != 31:
    raise AssertionError("both candidate rankings must contain all 31 candidates")
if len(cluster_168_rows) != len(main_168["cluster_ranking"]):
    raise AssertionError("168 h cluster ranking must retain every correlation cluster")
if len(cluster_72_rows) != len(sensitivity_72["cluster_ranking"]):
    raise AssertionError("72 h cluster ranking must retain every correlation cluster")
pd.DataFrame(candidate_168_rows).to_csv(
    OUT / "01c_168h候选完整排序.csv", index=False, encoding="utf-8-sig"
)
pd.DataFrame(candidate_72_rows).to_csv(
    OUT / "01c_72h候选完整排序.csv", index=False, encoding="utf-8-sig"
)
pd.DataFrame(cluster_168_rows).to_csv(
    OUT / "01c_完整关联排序.csv", index=False, encoding="utf-8-sig"
)
pd.DataFrame(cluster_168_rows).to_csv(
    OUT / "01c_168h簇完整排序.csv", index=False, encoding="utf-8-sig"
)
pd.DataFrame(cluster_72_rows).to_csv(
    OUT / "01c_72h簇完整排序.csv", index=False, encoding="utf-8-sig"
)

alias_audit = [
    {
        "excluded": "Pmax_1h",
        "included": "P",
        "reason": "1 h滑动最大值与同一小时P的NaN掩膜和数值逐行相同，是别名重复",
    },
    {
        "excluded": "ustar",
        "included": "USTAR",
        "reason": "ustar为USTAR的非负掩膜派生列，不另计物理候选",
    },
    {
        "excluded": "TKE_obs",
        "included": "TKE",
        "reason": "TKE_obs为TKE的非负掩膜派生列，不另计物理候选",
    },
]
pd.DataFrame(alias_audit).to_csv(
    OUT / "01c_别名审计.csv", index=False, encoding="utf-8-sig"
)
```

```
    summary = {
        "schema_version": 2,
        "status": "available",
        "target_definition": "2026-08-26冲击前168小时环境窗相对同积日±15日的2021/2023/2024参照窗的单事件关联描述",
        "event_release": str(EVENT_RELEASE),
        "last_usable_hour": str(EVENT_RELEASE - pd.Timedelta(hours=2)),
        "candidate_count": len(COLUMNS),
        "candidate_families": {family: family_count[family] for family in ("MET", "RADM", "SOIL", "FLUX")},
        "candidate_order": COLUMNS,
        "alias_audit": alias_audit,
        "coverage_rule": "逐候选目标窗与参照窗均至少80%有效；不插值、不把缺测当零",
        "score_rule": "事件窗均值相对参照窗中位数；尺度为MAD×1.4826，零MAD时回退IQR/1.349；A=|signed_z|",
        "cluster_rule": "2021–2024年6–9月逐对Spearman，绝对相关>0.8且共同有效小时>=200才连边；取完整连通簇",
        "cluster_common_hours_policy": {"minimum_pair_hours": MIN_PAIR_HOURS, "abs_rho_threshold": CLUSTER_ABS_RHO},
        "pairwise_valid_pairs": int((pair_n.values > 0).sum() // 2),
        "pairwise_valid_pair_hours_median": float(np.median([
            row.common_valid_hours for row in edges.itertuples() if row.common_valid_hours > 0
        ])) if len(edges) else None,
        "main_168h": main_168,
        "sensitivity_72h": sensitivity_72,
        "leave_one_reference_year_out": leave_one_year,
        "pseudo_2025_same_calendar": {
            "release": str(pseudo),
            "top5": pseudo_168["top5"],
            "max_A": (
                float(pseudo_168["ranked_clusters"][0]["A_abs"])
                if pseudo_168["ranked_clusters"] else None
            ),
            "ranked_representatives": [
                {"variable": row["representative"], "A_abs": row["A_abs"]}
                for row in pseudo_168["ranked_clusters"]
            ],
            "scope": "无灾害真值的同类过程外检，不是确证非灾日",
        },
        "top5": top5,
        "interpretation": "这是单次事件的环境关联证据，不是因果效应、发生概率或普适灾害排序。",
        "findings": [
            "候选池已断言覆盖MET、RADM、SOIL、FLUX四类，家族计数为19/5/3/4。",
            "Pmax_1h、ustar和TKE_obs作为重复/掩膜别名登记而不重复计数。",
            "晴空指数与反照率只在白昼有效；若覆盖不足，保留在字典并标为不可排名。",
        ],
        "outputs": {
            "candidate_dictionary": "01c_候选字典.csv",
            "pairwise_edges": "01c_相关边表.csv",
            "candidate_scores": "01c_逐候选得分.csv",
            "candidate_full_ranking_168h": "01c_168h候选完整排序.csv",
            "candidate_full_ranking_72h": "01c_72h候选完整排序.csv",
            "cluster_full_ranking_168h": "01c_168h簇完整排序.csv",
            "cluster_full_ranking_72h": "01c_72h簇完整排序.csv",
            "full_ranking": "01c_完整关联排序.csv",
            "figure": "01c_统一候选关联.png",
        },
    }

    fig, axes = plt.subplots(1, 2, figsize=(15, 6.5), constrained_layout=True)
    rank_rows = main_168["ranked_clusters"][:15]
    labels = [metadata[row["representative"]]["name"] for row in rank_rows]
    values = [row["A_abs"] for row in rank_rows]
    colors = ["#1f6f68" if i < 5 else "#a8c3bd" for i in range(len(rank_rows))]
    axes[0].barh(np.arange(len(rank_rows)), values, color=colors)
    axes[0].set_yticks(np.arange(len(rank_rows)), labels)
    axes[0].invert_yaxis()
    axes[0].set_xlabel("绝对稳健异常 A（无量纲）")
    axes[0].set_title("(a) 31项统一候选的相关簇代表")
    for index, value in enumerate(values):
        axes[0].text(value + .03, index, f"{value:.2f}", va="center", fontsize=14)
    axes[0].grid(axis="x", alpha=.2)

    left = {row["representative"]: row["A_abs"] for row in main_168["ranked_clusters"]}
    right = {row["representative"]: row["A_abs"] for row in sensitivity_72["ranked_clusters"]}
    common = [row["representative"] for row in main_168["ranked_clusters"][:12] if row["representative"] in right]
    x = np.arange(len(common))
    axes[1].plot(x, [left[c] for c in common], "o-", label="168小时主窗", color="#1f6f68")
    axes[1].plot(x, [right[c] for c in common], "s--", label="72小时敏感性", color="#b26745")
    axes[1].set_xticks(x, [metadata[c]["name"] for c in common], rotation=55, ha="right", fontsize=14)
    axes[1].set_ylabel("A")
    axes[1].set_title(f"(b) 前五重合 {overlap(main_168['top5'], sensitivity_72['top5'])}/5")
    axes[1].legend()
    axes[1].grid(axis="y", alpha=.2)
    fig.suptitle("单事件冲击前环境关联：31项冻结候选与逐对相关去冗余")
    fig.savefig(FIG / "01c_统一候选关联.png", dpi=180, bbox_inches="tight")
    plt.close(fig)

    (OUT / "01c_统一关联前五.json").write_text(
        json.dumps(serial(summary), ensure_ascii=False, indent=2) + "\n",
        encoding="utf-8",
    )
    print(
        "01c top5:",
        ", ".join(item["name"] for item in top5),
        f"| 168h refs={main_168['reference_window_count']}",
    )
```

```
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

问题二

step02_trigger_threshold.py 沟道临界条件与机械状态

```
"""问题2：训练期冻结的水源筛查与事件回顾评分，二者不混称预报。

降雨双包络共用缺测不补零、六小时干期切分的过程；各过程历时含内部干小时。
PDD、升温及气象基线只在2021--2025拟合，2026只评估已观测区间。
另报84灾点描述包络；像元空间对照由step02b_spatial.py单独实现。
"""
from __future__ import annotations

import json
import sys
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy import stats

sys.path.insert(0, str(Path(__file__).resolve().parent))
from mmlib import io_data as io, meteo as mt, trigger as tg, terrain as tr # noqa: E402

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "Noto Sans SC", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False

FIG = io.FIG_OUT
OUT = io.DATA_OUT
EVENT_TS = pd.Timestamp("2026-08-26 10:30")
FORMAL_EVENT_WIN = ("2026-08-26 08:00", "2026-08-26 13:00")
EVENT_WIN = FORMAL_EVENT_WIN
SEED = 20260826
CAL_YEARS = tuple(range(2021, 2026))
WARM_MONTHS = (6, 7, 8, 9)
BOOT = 200
STATE_TUPLE_FIELDS = ["G", "H", "Theta", "F_ice", "U_ice", "C_ice"]


def p_fields(value: float) -> dict:
    """Return a lossless p-value record; reject silent underflow to zero."""
    p = float(value)
    if not np.isfinite(p) or p < 0:
        raise ValueError(f"invalid p-value: {value!r}")
    if p == 0.0:
        raise FloatingPointError("p-value underflowed to zero; use higher precision or report a bound")
    return {"p值": p, "p值科学计数": f"{p:.6e}"}


def daily_valid(series: pd.Series, how="sum") -> pd.Series:
    """自然日有效小时至少20个；短截日与缺测日不伪装成零。"""
    grouped = series.resample("D")
    value = grouped.sum(min_count=20) if how == "sum" else grouped.mean()
    return value.where(grouped.count() >= 20)


def warm_training(index):
    return np.isin(index.year, CAL_YEARS) & np.isin(index.month, WARM_MONTHS)


def qsummary(values):
    x = pd.to_numeric(pd.Series(values), errors="coerce").dropna().to_numpy()
    x = x[np.isfinite(x)]
    return {"n": len(x), **{f"q{int(q*100):02d}": float(np.quantile(x, q)) if len(x) else None
                            for q in (.05, .25, .5, .75, .95)}}


def cn_scenario_sensitivity(feat: pd.DataFrame, model) -> dict:
    """Evaluate the declared CN=75/85/95 engineering scenarios."""
    scenarios = {}
    for cn in (75.0, 85.0, 95.0):
        scenario = tg.TriggerModel(
            area_km2=model.area,
            glacier_frac=model.f_gl,
            q_crit_m2s=model.q_crit,
            width_m=model.width,
            cn=cn,
            temp_offset=model.temp_offset,
            mechanical_regime=model.mechanical_regime,
            use_legacy_k=False,
        )
        discharge = scenario.discharge(feat)
        event = discharge.loc[EVENT_WIN[0]:EVENT_WIN[1]]
```

```
        q = event["Q_water"].dropna()
        t_index = event["T_index"].dropna()
        peak_time = str(t_index.idxmax()) if len(t_index) else None
        qrain = discharge["Q_rain"].dropna()
        rain_2026 = discharge.loc[discharge.index.year == 2026, "Q_rain"].dropna()
        runoff_2026 = tg.rain_runoff(feat, cn)
        runoff_2026 = runoff_2026.loc[runoff_2026.index.year == 2026].dropna()
        scenarios[f"{cn:.0f}"] = {
            "C_N0": cn,
            "is_formal_scenario": bool(np.isclose(cn, tg.CN0)),
            "formal_event_window_start": FORMAL_EVENT_WIN[0],
            "formal_event_window_end": FORMAL_EVENT_WIN[1],
            "event_Qwater_peak_m3s": float(q.max()) if len(q) else None,
            "event_Qrain_peak_m3s": (
                float(event["Q_rain"].dropna().max())
                if event["Q_rain"].notna().any() else None
            ),
            "event_Qmelt_peak_m3s": (
                float(event["Q_melt"].dropna().max())
                if event["Q_melt"].notna().any() else None
            ),
            "event_T_peak": float(t_index.max()) if len(t_index) else None,
            "event_T_peak_time": peak_time,
            "event_hours_T_ge_1": (
                int((t_index >= 1.0).sum()) if len(t_index) else None
            ),
            "event_valid_hours": int(len(q)),
            "series_Qrain_peak_m3s": float(qrain.max()) if len(qrain) else None,
            "rain_2026_Qrain_peak_m3s": (
                float(rain_2026.max()) if len(rain_2026) else None
            ),
            "rain_2026_runoff_sum_mm": (
                float(runoff_2026.sum()) if len(runoff_2026) else None
            ),
        }
    return {
        "formal_C_N0": float(tg.CN0),
        "formal_event_window": list(FORMAL_EVENT_WIN),
        "event_window_semantics": "inclusive hourly labels on the frozen 08:00--13:00 local event window",
        "scenario_values": [75.0, 85.0, 95.0],
        "AMC_adjustment": "API terciles from 2021-2025 training; factors 0.75/1.00/1.15 and clip [30,98]",
        "provenance": (
            "independent engineering scenario, not derived from unverified "
            "land-cover class codes and not calibrated to the 2026 event"
        ),
        "scenarios": scenarios,
    }


def _ser(x, key: str | None = None):
    if isinstance(x, dict):
        return {str(k): _ser(v, str(k)) for k, v in x.items()}
    if isinstance(x, (list, tuple)):
        return [_ser(v, key) for v in x]
    if isinstance(x, (np.floating, float)):
        v = float(x)
        if not np.isfinite(v):
            return None
        if key in {"p值", "p_value"}:
            return v
        if 0.0 < abs(v) < 1e-4:
            return round(v, 12)
        return round(v, 6)
    if isinstance(x, (np.bool_, bool)):
        return bool(x)
    if isinstance(x, (np.integer, int)):
        return int(x)
    if isinstance(x, np.ndarray):
        return _ser(x.tolist())
    return x


def save_json(obj, name: str) -> Path:
    out = OUT / name
    out.parent.mkdir(parents=True, exist_ok=True)
    out.write_text(json.dumps(_ser(obj), ensure_ascii=False, indent=2), encoding="utf-8")
    return out


def rank_association_summary(frame: pd.DataFrame, value_col: str) -> dict:
    """Report finite-sample rank tests with the exact flow quantity named."""
    scale_tau = stats.kendalltau(np.log10(frame[value_col]), frame["规模序"])
    frequency_tau = stats.kendalltau(np.log10(frame[value_col]), frame["频率序"])
    scale_rho = stats.spearmanr(frame[value_col], frame["规模序"], nan_policy="omit")
    return {
        "quantity": value_col,
        "units": "m3/s" if value_col == "Q_crit_m3s" else "m2/s",
        "sample_n": int(len(frame)),
        "kendall_vs_scale": {"tau": float(scale_tau.statistic), **p_fields(scale_tau.pvalue)},
        "kendall_vs_frequency": {
            "tau": float(frequency_tau.statistic),
            **p_fields(frequency_tau.pvalue),
        },
        "spearman_vs_scale": {"rho": float(scale_rho.statistic), **p_fields(scale_rho.pvalue)},
    }

```

```python

# ============================================================ S1 沟道尺度临界条件
def gully_critical_conditions() -> dict:
    """84 条沟道的起动临界流量及其与规模/频率等级的关系。"""
    g = tg.gully_table()
    positive = g.loc[
        (g["mechanical_regime"] == "water_controlled")
        & g["q_crit_m2s"].gt(0)
        & g["Q_crit_m3s"].notna()
    ].copy()
    order_scale = {"小": 1, "中": 2, "大": 3, "特大": 4}
    order_freq = {"低": 1, "中": 2, "高": 3}
    g["规模序"] = g["规模"].map(order_scale)
    g["频率序"] = g["频率"].map(order_freq)
    positive = g.loc[positive.index].copy()
    state_counts = g["mechanical_regime"].value_counts().to_dict()
    state_lists = {
        regime: g.loc[g["mechanical_regime"] == regime, ["Name", "类型", "规模", "比降"]]
        .to_dict(orient="records")
        for regime in ("water_controlled", "friction_equilibrium", "gravity_unstable", "not_evaluable")
    }

    out: dict = {
        "沟道数": int(len(g)),
        "机械状态计数": {str(k): int(v) for k, v in state_counts.items()},
        "机械状态清单": state_lists,
        "water_controlled_n": int(len(positive)),
        "zero_threshold_n": int(g["mechanical_regime"].isin(["friction_equilibrium", "gravity_unstable"]).sum()),
        "not_evaluable_n": int((g["mechanical_regime"] == "not_evaluable").sum()),
        "起动流量_m3s": {"min": float(positive["Q_crit_m3s"].min()), "p25": float(positive["Q_crit_m3s"].quantile(0.25)),
                 "p50": float(positive["Q_crit_m3s"].median()), "p75": float(positive["Q_crit_m3s"].quantile(0.75)),
                 "max": float(positive["Q_crit_m3s"].max())},
        "旧式起动流量_m3s": {"n": int(g["Q_crit_old_m3s"].notna().sum()),
                     "p50": float(g["Q_crit_old_m3s"].median()),
                     "note": "旧式缺少床体沿坡自重项，仅用于新旧对照"},
        "比降": {"min": float(g["比降"].min()), "p50": float(g["比降"].median()),
              "max": float(g["比降"].max())},
        "面积_km2": {"min": float(g["面积_km2"].min()), "p50": float(g["面积_km2"].median()),
                "max": float(g["面积_km2"].max())},
        "高差_m": {"min": float(g["高差_m"].min()), "p50": float(g["高差_m"].median()),
               "max": float(g["高差_m"].max())},
    }
    out["床面固相含量敏感性"] = {
        str(cb): {"bed_solid_fraction": cb,
                  "density_kg_m3": 1000.+1650.*cb,
                  "delta_rho_kg_m3": 1650.*cb,
                  "qcrit_m2s": qsummary([tg._q_crit_unit(s,bed_solid_fraction=cb) for s in positive["比降"]]),
                  "Qcrit_m3s": qsummary([tg._q_crit_unit(s,bed_solid_fraction=cb)*width/tg.KAPPA_Q
                                         for s,width in zip(positive["比降"],positive["沟宽_m"])])}
        for cb in (.15,1050./1650.,.8)}

    # 单宽起动流量与实际可能来水的量级对照：用面积×设计雨强估计一次洪水的峰值
    # 设计雨强取 I–D 阈值曲线的 1 h、3 h 值（见 S2），此处先用 p95 小时雨强
    out["按类型统计"] = {}
    for t, sub in g.groupby("类型"):
        positive_sub = positive.loc[positive["类型"] == t]
        out["按类型统计"][t] = {
            "n": int(len(sub)),
            "n_water_controlled": int(len(positive_sub)),
            "zero_or_not_evaluable_n": int(len(sub) - len(positive_sub)),
            "起动流量中位数_m3s": float(positive_sub["Q_crit_m3s"].median()) if len(positive_sub) else None,
            "比降中位数": float(sub["比降"].median()),
            "面积中位数_km2": float(sub["面积_km2"].median()),
            "高差中位数_m": float(sub["高差_m"].median()),
        }

    # 分级检验
    tests = {}
    for col, name in (("规模序", "规模"), ("频率序", "频率")):
        groups = [sub["Q_crit_m3s"].values for _, sub in positive.groupby(col) if len(sub) >= 2]
        labels = [k for k, sub in positive.groupby(col) if len(sub) >= 2]
        if len(groups) >= 2:
            h, p = stats.kruskal(*groups)
            tests[name] = {"等级": [int(v) for v in labels],
                           "各组n": [int(len(x)) for x in groups],
                           "各组起动流量中位数": [float(np.median(x)) for x in groups],
                           "Kruskal_H": float(h), **p_fields(p)}
    out["分级检验"] = tests

    # 序数 logistic：总起动流量能否预测规模等级
    try:
        from statsmodels.miscmodels.ordinal_model import OrderedModel
        Xo = np.log10(positive[["Q_crit_m3s"]].values)
        m = OrderedModel(positive["规模序"].values, Xo, distr="logit").fit(method="bfgs", disp=False, maxiter=500)
        out["序数logistic_规模"] = {
            "系数": float(m.params[0]), "标准误": float(m.bse[0]),
            **p_fields(m.pvalues[0]), "对数似然": float(m.llf),
            "解释": "系数符号为已发生灾点的条件规模关联，按实际方向判读，不预设递减",
```

```
    }
except Exception as e:
    out["序数logistic_规模"] = {"状态": f"拟合失败: {type(e).__name__}"}

def ascii_p_test(entry: dict, statistic: str) -> dict:
    return {
        statistic: entry[statistic],
        "p_value": entry["p值"],
        "p_display": entry["p值科学计数"],
    }

total_rank = rank_association_summary(positive, "Q_crit_m3s")
unit_rank = rank_association_summary(positive, "q_crit_m2s")
out["rank_test_statistics"] = {
    "sample_n": int(len(positive)),
    "zero_or_not_evaluable_n": int(len(g) - len(positive)),
    "total_flow_Q_crit_m3s": {
        "kendall_vs_scale": ascii_p_test(total_rank["kendall_vs_scale"], "tau"),
        "kendall_vs_frequency": ascii_p_test(
            total_rank["kendall_vs_frequency"], "tau"
        ),
        "spearman_vs_scale": ascii_p_test(total_rank["spearman_vs_scale"], "rho"),
    },
    "unit_width_q_crit_m2s": {
        "kendall_vs_scale": ascii_p_test(unit_rank["kendall_vs_scale"], "tau"),
        "kendall_vs_frequency": ascii_p_test(
            unit_rank["kendall_vs_frequency"], "tau"
        ),
        "spearman_vs_scale": ascii_p_test(unit_rank["spearman_vs_scale"], "rho"),
    },
    "note": (
        "总起动流量Q_crit= q_crit B/kappa与单宽起动流量q_crit的秩检验分别保存; "
        "p值以双精度结果和科学计数法并列保存, 零表示禁止静默下溢。"
    ),
}

# 易发性评分只在有正水流门槛的 water_controlled 分支上定义。
logq = np.log10(positive["Q_crit_m3s"])
qn = (logq - logq.min()) / max(logq.max() - logq.min(), 1e-9)
supply = np.log1p(positive["面积_km2"] * positive["比降"] * 1e3)
sn = (supply - supply.min()) / max(supply.max() - supply.min(), 1e-9)
g["易发性"] = np.nan
g.loc[positive.index, "易发性"] = (1.0 - qn) * (0.5 + 0.5 * sn)
out["易发性"] = {"min": float(g["易发性"].min()), "p50": float(g["易发性"].median()),
            "max": float(g["易发性"].max()), "n_defined": int(g["易发性"].notna().sum()),
            "zero_branch_state": "inapplicable; 不把零水流门槛填成最小易发分数",
            "公式": "S = (1 − Q̃_c) · (0.5 + 0.5 · S̃), Q̃_c 与 S̃ 为归一化起动流量与物质供给"}
top = g.nlargest(10, "易发性").copy()
top["q_crit_positive"] = top["Q_crit_m3s"].gt(0)
out["前10易发沟道"] = top[
    [
        "Name",
        "类型",
        "规模",
        "频率",
        "面积_km2",
        "比降",
        "mechanical_regime",
        "q_crit_positive",
        "Q_crit_m3s",
        "易发性",
    ]
].to_dict(orient="records")
g.to_parquet(io.CACHE / "gully_table.parquet")
return out


# ============================================================ S2 降雨型 I–D 阈值
def rainfall_events(series: pd.Series) -> pd.DataFrame:
    """Missing hours break a record; six *observed* dry hours separate events."""
    p = series.sort_index().asfreq("h")
    rows, start, last_wet, gap = [], None, None, 0
    dry_run, start_complete = 0, False

    def close(reason):
        if start is not None and last_wet is not None:
            segment = p.loc[start:last_wet]
            if segment.notna().all():
                duration = int((last_wet-start).total_seconds()/3600)+1
                assert len(segment) == duration
                rows.append({"start": start, "end": last_wet, "year": start.year,
                             "dur_h": duration, "total_mm": float(segment.sum()),
                             "mean_mmh": float(segment.sum()/duration),
                             "Imax_mmh": float(segment.max()), "curve": segment.to_numpy(),
                             "boundary": reason,
                             "complete": start_complete and reason == "six_observed_dry_hours"})
    for timestamp, value in p.items():
        if not np.isfinite(value):
            close("missing_boundary")
            start, last_wet, gap, dry_run = None, None, 0, 0
            continue
        if value > 0.1:
            if start is None:
```

```
            start = timestamp
            start_complete = dry_run >= 6
        last_wet, gap = timestamp, 0
        dry_run = 0
    elif start is not None:
        dry_run += 1
        gap += 1
        if gap >= 6:
            close("six_observed_dry_hours")
            start, last_wet, gap = None, None, 0
    else:
        dry_run += 1
 close("record_end")
 return pd.DataFrame(rows, columns=["start", "end", "year", "dur_h", "total_mm",
                            "mean_mmh", "Imax_mmh", "curve", "boundary", "complete"])


def _frequency_fit(events, durations):
 rows = []
 for duration in durations:
    values = []
    for curve in events["curve"]:
        if len(curve) >= duration:
            values.append(float(np.convolve(curve, np.ones(duration)/duration, "valid").max()))
    if len(values) >= 8:
        rows.append({"D_h": duration, "n": len(values), "I_p95": float(np.quantile(values, .95)),
                     "I_p50": float(np.median(values)), "I_p99": float(np.quantile(values,.99)),
                     "I_max": float(max(values))})
 if len(rows) < 2:
    return None, rows
 frame = pd.DataFrame(rows)
 slope, intercept = np.polyfit(np.log(frame.D_h), np.log(frame.I_p95), 1)
 return (float(np.exp(intercept)), float(-slope)), rows


def id_threshold(feat: pd.DataFrame) -> dict:
 from sklearn.linear_model import QuantileRegressor
 p = feat["P"].astype(float)
 all_events = rainfall_events(p)
 ev = all_events.loc[all_events.year.isin(CAL_YEARS) &
                     all_events.start.dt.month.isin(WARM_MONTHS) &
                     all_events.complete & (all_events.total_mm >= 1.)].copy()
 if len(ev) < 15:
    raise ValueError(f"insufficient complete training rainfall processes: {len(ev)}")
 durations = (1, 3, 6, 12, 24)
 fit, curve = _frequency_fit(ev, durations)
 if fit is None:
    raise ValueError("fewer than two adequately sampled rainfall durations")
 a, b = fit
 def lower_fit(frame):
    q = QuantileRegressor(quantile=.1, alpha=0, solver="highs")
    q.fit(np.log(frame[["dur_h"]].to_numpy()), np.log(frame.mean_mmh.to_numpy()))
    return float(np.exp(q.intercept_)), float(-q.coef_[0])
 lower = lower_fit(ev)
 rng = np.random.default_rng(SEED)
 freq_boot, lower_boot = [], []
 years = np.array(CAL_YEARS)
 for _ in range(BOOT):
    # Concatenation preserves bootstrap multiplicity (a set would be incorrect).
    pick = rng.choice(years, len(years), replace=True)
    sub = pd.concat([ev.loc[ev.year == year] for year in pick], ignore_index=True)
    pars, _ = _frequency_fit(sub, durations)
    if pars is not None:
        freq_boot.append(pars)
    if len(sub) >= 10 and sub.dur_h.nunique() >= 2:
        lower_boot.append(lower_fit(sub))
 if len(freq_boot) < BOOT*.8 or len(lower_boot) < BOOT*.8:
    raise ValueError("too few successful year-block bootstrap fits")
 fb, lb = np.asarray(freq_boot), np.asarray(lower_boot)
 dg = np.geomspace(1, max(24, ev.dur_h.max()), 80)
 def boot_band(samples):
    lines = samples[:, :1]*dg[None, :]**(-samples[:, 1:2])
    return {"D_h": dg.tolist(), "lo": np.quantile(lines,.025,axis=0).tolist(),
            "hi": np.quantile(lines,.975,axis=0).tolist()}
 def table(frame):
    return frame.drop(columns="curve").assign(start=frame.start.astype(str),
                                              end=frame.end.astype(str)).to_dict("records")
 save_json({"events": table(all_events), "missing_hours_not_filled": int(p.isna().sum()),
            "wet_threshold_mm_h": .1, "minimum_training_depth_mm": 1.,
            "duration": "inclusive start-to-last-wet span, internal observed dry hours included"},
           "02_降雨过程.json")
 return {"独立暴雨事件数": len(ev), "训练年份": list(CAL_YEARS),
         "训练月份": list(WARM_MONTHS), "分割干期_h": 6, "缺测处理": "缺测断开不补零；边界事件单列",
         "过程边界计数": all_events.boundary.value_counts().to_dict(),
         "历时定义": "起止跨度含内部干小时；强度=总量/跨度", "自助": "按年分块，保留重复抽样权重",
         "自助次数": BOOT, "强度历时样本": sum(r["n"] for r in curve),
         "幂律系数a": a, "幂指数b": b, "公式": "I=a*D^(-b)",
         "语义一": "训练降雨过程内各历时最大平均强度的95%频率包络，非起动临界",
         "分位水平": .95, "参数95pct置信区间_a": np.quantile(fb[:,0],[.025,.975]).tolist(),
         "参数95pct置信区间_b": np.quantile(fb[:,1],[.025,.975]).tolist(),
         "频率曲线自助带": boot_band(fb), "逐历时统计": curve,
         "实测历时阈值": {f'{r["D_h"]}h': r["I_p95"] for r in curve},
         "外推历时阈值": {f"{d}h": float(a*d**(-b)) for d in (24,72) if d > ev.dur_h.max()},
         "频率包络最大支持历时_h": max(r["D_h"] for r in curve),
```

```
        "频率外推说明":"超过频率拟合最大有效历时即为外推，即使小于最长过程历时",
        "事件最大历时_h": int(ev.dur_h.max()), "事件最大总量_mm": float(ev.total_mm.max()),
        "事件最大强度_mmh": float(ev.Imax_mmh.max()), "事件表": table(ev),
        "下包络": {"alpha": lower[0], "beta": lower[1], "quantile": .1,
                 "alpha_ci": np.quantile(lb[:,0],[.025,.975]).tolist(),
                 "beta_ci": np.quantile(lb[:,1],[.025,.975]).tolist(),
                 "band": boot_band(lb), "n": len(ev),
                 "解释": "非灾害标注降雨的低分位参考线，仅候选水源筛查，非充分条件"}}


# ============================================================ S3 消融型能量阈值
def melt_threshold(feat: pd.DataFrame, info: dict) -> dict:
    gn = io.load_glacier_nc()
    ok = np.isfinite(gn["thickness"]) & (gn["thickness"] > 1)
    z = float(np.nanmean(gn["dem"][ok]))
    offset = -(z-3980.)*.006
    weighted = feat["cum_melt_energy"]/(1+np.exp(-(feat["Ta_2m"]+offset)/1.5))
    daily = daily_valid(feat["cum_melt_energy"])
    src = daily_valid(weighted)
    train = src.loc[warm_training(src.index)].dropna()
    ec = float(train.quantile(.9))
    frac = float(info["冰川覆盖率"])
    day = pd.Timestamp("2026-08-26")
    src_event = float(src.get(day, np.nan))
    assert np.isclose((ec/.334)*.334, ec)
    ice_depth = ec*1e6/(900.*334000.)
    assert np.isclose(ice_depth*900.*334000./1e6, ec)
    return {"冰川区平均高程_m": z, "冰川平均冰厚_m": float(np.mean(gn["thickness"][ok])),
          "源区气温订正_°C": offset, "湿绝热直减率_K每100m": .6,
          "训练年份": list(CAL_YEARS), "日有效小时门槛": 20,
          "观测地日融化能量_p50": float(daily.loc[warm_training(daily.index)].median()),
          "观测地日融化能量_p90": float(daily.loc[warm_training(daily.index)].quantile(.9)),
          "源区订正日融化能量_p50": float(train.median()), "源区订正日融化能量_p90": ec,
          "事件窗口累计融化能量_MJm-2": src_event,
          "事件窗口定义": "2026-08-26自然日，仅与同日完整性门槛一致的训练日比较",
          "事件窗口化水当量_mm": src_event/.334,
          "源区事件日能量_MJm2": src_event, "集水区事件日化水当量_mm": frac*src_event/.334,
          "临界条件": {"判据": "源区日能量参考90%分位，非已证实物理阈值",
                    "E_s_p90_MJm-2": ec, "M_c_mm": ec/.334,
                    "集水区M_c_mm": frac*ec/.334, "对应冰厚_m": ice_depth},
          "直减率敏感性": {str(l*100): float(daily_valid(feat["cum_melt_energy"] /
              (1+np.exp(-(feat["Ta_2m"]-(z-3980.)*l)/1.5))).loc[warm_training(src.index)].quantile(.9))
              for l in (.005,.006,.007)},
          "说明": "能量与水当量同源，不作为两个独立命中证据；缺测不补零"}


def build_object_state_rows(
    gully: pd.DataFrame,
    q_event: float,
    theta_event: str,
    f_ice: int,
    u_ice: int,
) -> list[dict]:
    """Build one uniquely identified state tuple per source object."""
    if "Code" not in gully:
        raise ValueError("gully table must contain the source-unique Code field")
    object_ids = gully["Code"].astype(str).tolist()
    if len(object_ids) != len(set(object_ids)):
        raise ValueError("gully Code values must be unique")
    rows = []
    for object_id, (_, source_row) in zip(object_ids, gully.iterrows()):
        g_state = str(source_row["mechanical_regime"])
        qcrit = (
            float(source_row["Q_crit_m3s"])
            if np.isfinite(source_row["Q_crit_m3s"])
            else np.nan
        )
        if g_state == "not_evaluable" or not np.isfinite(q_event):
            h_state = "not_evaluable"
        elif g_state == "water_controlled":
            if not np.isfinite(qcrit) or qcrit <= 0:
                h_state = "not_evaluable"
            else:
                h_state = (
                    "hydraulic_candidate"
                    if q_event / qcrit >= 1.0
                    else "below_threshold"
                )
        else:
            h_state = "inapplicable"
        rows.append({
            "object_id": object_id,
            "display_name": str(source_row.get("Name", "")),
            "G": g_state,
            "H": h_state,
            "Theta": theta_event,
            "F_ice": f_ice,
            "U_ice": u_ice,
            "C_ice": int(theta_event == "thermal_precondition" and f_ice == 1),
            "ice_mechanics_unresolved": u_ice,
            "Q_water_m3s_at_event": q_event,
            "Q_crit_m3s": qcrit,
            "T_index_at_event": (
```

```
            q_event / qcrit
            if np.isfinite(q_event) and np.isfinite(qcrit) and qcrit > 0
            else np.nan
        ),
      })
   return rows


def build_state_tuple(
   feat: pd.DataFrame,
   dis: pd.DataFrame,
   model,
   melt_meta: dict,
   gully: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame, dict]:
   """Persist the theory's non-collapsed G/H/Theta/ice evidence tuple."""
   offset = float(melt_meta["源区气温订正_°C"])
   weighted_hourly = feat["cum_melt_energy"] / (
      1.0 + np.exp(-(feat["Ta_2m"] + offset) / 1.5)
   )
   daily_energy = daily_valid(weighted_hourly)
   threshold = float(melt_meta["临界条件"]["E_s_p90_MJm-2"])
   theta_daily = pd.Series("not_evaluable", index=daily_energy.index, dtype=object)
   valid_daily = daily_energy.notna()
   theta_daily.loc[valid_daily] = np.where(
      daily_energy.loc[valid_daily] >= threshold,
      "thermal_precondition",
      "below_thermal_reference",
   )
   theta = theta_daily.reindex(feat.index.normalize()).to_numpy()

   vol_path = OUT / "04_初始体积.json"
   h_thr, sl_thr = 50.0, 25.0
   if vol_path.exists():
      chain_a = json.loads(vol_path.read_text(encoding="utf-8")).get("链A", {})
      h_thr = float(chain_a.get("h_thr_m", h_thr))
      sl_thr = float(chain_a.get("sl_thr_deg", sl_thr))
   gn = io.load_glacier_nc()
   source_valid = (
      np.isfinite(gn["thickness"])
      & (gn["thickness"] >= h_thr)
      & np.isfinite(gn["slope"])
      & (gn["slope"] >= sl_thr)
   )
   source_n = int(source_valid.sum())
   f_ice = int(source_n > 0)
   u_ice = 1

   hourly = pd.DataFrame({
      "G": model.mechanical_regime,
      "H": dis["hydraulic_state"].to_numpy(),
      "Theta": theta,
      "F_ice": f_ice,
      "U_ice": u_ice,
      "C_ice": ((theta == "thermal_precondition") & (f_ice == 1)).astype(int),
      "ice_mechanics_unresolved": u_ice,
      "Q_water_m3s": dis["Q_water"].to_numpy(),
      "Q_crit_m3s": dis["Q_crit"].to_numpy(),
      "T_index": dis["T_index"].to_numpy(),
   }, index=feat.index)
   hourly.index.name = "timestamp"

   event_time = pd.Timestamp("2026-08-26 10:00")
   event_row = dis.loc[event_time] if event_time in dis.index else dis.iloc[-1]
   q_event = float(event_row["Q_water"]) if np.isfinite(event_row["Q_water"]) else np.nan
   theta_event = theta_daily.get(event_time.normalize(), "not_evaluable")
   gully_rows = build_object_state_rows(
      gully, q_event, theta_event, f_ice, u_ice
   )
   objects = pd.DataFrame(gully_rows)
   summary = {
      "fields": STATE_TUPLE_FIELDS,
      "ice_mechanics_unresolved": True,
      "F_ice_definition": "附件4存在冰厚和坡度同时达到源区阈值的编目像元",
      "source_thresholds": {"h_thr_m": h_thr, "sl_thr_deg": sl_thr},
      "source_pixel_count": source_n,
      "target_catchment_G": model.mechanical_regime,
      "target_catchment_H_counts": hourly["H"].value_counts().to_dict(),
      "target_catchment_Theta_counts": hourly["Theta"].value_counts().to_dict(),
      "gully_G_counts": objects["G"].value_counts().to_dict(),
      "gully_H_counts": objects["H"].value_counts().to_dict(),
      "event_day_Theta": theta_event,
      "hourly_csv": "02_状态元组_逐小时.csv",
      "objects_csv": "02_状态元组_逐沟道.csv",
   }
   return hourly, objects, summary


def block_quantile(series, q=.95):
   train = series.loc[warm_training(series.index)].dropna()
   rng = np.random.default_rng(SEED+1)
   boot = []
   for _ in range(BOOT):
      pick = rng.choice(CAL_YEARS, len(CAL_YEARS), replace=True)
      vals = np.concatenate([train.loc[train.index.year == y].to_numpy() for y in pick])
      if len(vals):
```

```
        boot.append(float(np.quantile(vals,q)))
    return {"threshold": float(train.quantile(q)), "q": q, "n_days": len(train),
            "ci": np.quantile(boot,[.025,.975]).tolist(), "bootstrap": "year-block"}


def business_screen(feat, idr):
    positive = feat["Ta_2m"].clip(lower=0).resample("D").sum(min_count=24)
    pdd = positive.rolling(3,min_periods=3).sum()
    temperature = daily_valid(feat["Ta_2m"], "mean")
    rise = temperature-temperature.shift(1)
    rain_day = daily_valid(feat["P"])
    pthr, rthr = block_quantile(pdd), block_quantile(rise)
    tab = pd.DataFrame({"PDD3_C_h": pdd, "rise24_C": rise, "rain_mm": rain_day})
    tab["rain_candidate"] = np.where(rain_day.notna(),0.,np.nan)
    lo = idr["下包络"]
    ev = rainfall_events(feat["P"])
    for _, row in ev.loc[~ev.complete].iterrows():
        dates=pd.date_range(row.start.normalize(),row.end.normalize(),freq="D")
        tab.loc[tab.index.intersection(dates),"rain_candidate"]=np.nan
    for _, row in ev.iterrows():
        if not row.complete or row.total_mm < 1. or row.mean_mmh < lo["alpha"]*row.dur_h**(-lo["beta"]):
            continue
        # Event-end evaluation: no claim of a causal intra-event alarm.
        dates = pd.date_range(row.start.normalize(),row.end.normalize(),freq="D")
        valid = tab.index.intersection(dates)
        tab.loc[valid,"rain_candidate"] = tab.loc[valid,"rain_candidate"].where(
            tab.loc[valid,"rain_mm"].isna(),1.)
    melt_valid = pdd.notna() & rise.notna()
    known_hit=(pdd>=pthr["threshold"]) | (rise>=rthr["threshold"])
    tab["melt_candidate"] = np.where(known_hit,1.,np.where(melt_valid,0.,np.nan))
    tab["PDD3_valid_hours"] = feat["Ta_2m"].resample("D").count().rolling(3,min_periods=3).sum()
    both = tab[["rain_candidate","melt_candidate"]]
    tab["Tw"] = np.where(both.eq(1).any(axis=1),1.,np.where(both.notna().all(axis=1),0.,np.nan))
    tab.to_csv(OUT/"02_逐日候选水源.csv",encoding="utf-8-sig",index_label="date")
    test = tab.loc[tab.index.year == 2026]
    warm = test.loc[test.index.month.isin(WARM_MONTHS)]
    def counts(frame):
        observed = frame.loc[frame.Tw.notna()]
        return {"observed_days": int(frame.Tw.notna().sum()), "candidate_days": int(frame.Tw.eq(1).sum()),
                "candidate_day_rate": float(frame.Tw.dropna().mean()),
                "first_date": str(observed.index.min()), "last_date": str(observed.index.max())}
    return {"training_years": list(CAL_YEARS), "PDD3": pthr, "rise24": rthr,
            "PDD_unit": "sum of positive hourly temperatures over 3 complete natural days, C*h",
            "rise_definition": "difference of adjacent natural-day mean temperature, 24h",
            "2026_observation_period": counts(test), "2026_warm_observed": counts(warm),
            "event_day": tab.loc["2026-08-26"].to_dict(),
            "daily_file": "02_逐日候选水源.csv", "rain_screen": "complete process span/intensity evaluated
                retrospectively",
            "interpretation": "candidate/alert-day rate is not a true false-alarm rate; unlabelled days are not
                confirmed negatives"}


def statistical_domains():
    g = tg.gully_table()
    cols = ["比降","主沟长_m","高差_m","面积_km2","沟口高程_m","流域最高高程_m","扇宽_m","扇长_m"]
    def describe(frame):
        return {c:qsummary(frame[c]) for c in cols}
    return {"n_historical_polygons": len(g), "all": describe(g),
            "by_type": {str(k):{"n":len(v),"quantiles":describe(v)} for k,v in g.groupby("类型")},
            "by_size": {str(k):{"n":len(v),"quantiles":describe(v)} for k,v in g.groupby("规模")},
            "interpretation": "case-only descriptive envelope, not occurrence probability or a validated critical
                boundary"}


def long_temperature_check(feat):
    ext = io.load_temperature()
    station = daily_valid(feat["Ta_2m"],"mean")
    station_pdd = station.clip(lower=0).rolling(3,min_periods=3).sum()
    result = {}
    for col in ext:
        values = pd.to_numeric(ext[col],errors="coerce").asfreq("D")
        values = values.where(values.between(-60,50))
        pdd = values.clip(lower=0).rolling(3,min_periods=3).sum()
        long = pdd.loc[(pdd.index.year <= 2025) & pdd.index.month.isin(WARM_MONTHS)].dropna()
        recent = pdd.loc[warm_training(pdd.index)].dropna()
        same = pd.concat([station.rename("station"),values.rename("era")],axis=1).dropna()
        same = same.loc[warm_training(same.index)]
        matched_pdd = pd.concat([station_pdd.rename("station"),pdd.rename("era")],axis=1).dropna()
        matched_pdd = matched_pdd.loc[warm_training(matched_pdd.index)]
        result[str(col)] = {"long_daily_PDD3_C_day":qsummary(long),
                            "recent_daily_PDD3_C_day":qsummary(recent),
                            "long_available_period":[str(long.index.min()),str(long.index.max())],
                            "overlap_days":len(same),
                            "common_date_PDD3_days":len(matched_pdd),
                            "common_date_station_PDD3":qsummary(matched_pdd.station),
                            "common_date_product_PDD3":qsummary(matched_pdd.era),
                            "common_date_PDD3_correlation":float(matched_pdd.corr().iloc[0,1]) if len(matched_pdd)>2 else
                                None,
                            "temperature_correlation":float(same.corr().iloc[0,1]) if len(same)>2 else None,
                            "station_minus_product_mean_C":float((same.station-same.era).mean())}
    return {"products":result,"station_daily_PDD3_C_day":qsummary(station_pdd.loc[warm_training(station_pdd.index)])
        ,
            "definition":"both products: max(daily mean temperature,0), sum over 3 natural days",
            "limitation":"daily-mean PDD is not hourly-positive PDD; no rainfall or I-D inference from temperature-
                only attachment"}
```

```python

def weather_baseline(feat):
    p, temp = feat["P"].asfreq("h"), feat["Ta_2m"].asfreq("h")
    trainmask = np.isin(p.index.year,CAL_YEARS)
    wet = p.gt(.1).astype(float).where(p.notna())
    frequency = float(wet.loc[trainmask].mean())
    pair = pd.concat([wet.shift(1).rename("prev"),wet.rename("now")],axis=1).loc[trainmask].dropna()
    corr = float(pair.prev.corr(pair.now))
    persistence = float(np.clip(corr,0,1)) if np.isfinite(corr) else 0.
    train_t = temp.loc[trainmask].dropna()
    climate = train_t.groupby([train_t.index.dayofyear,train_t.index.hour]).mean().unstack()
    climate = climate.reindex(range(1,367)).interpolate(limit_direction="both").rolling(15,center=True,min_periods
        =1).mean()
    climate = climate.fillna(train_t.groupby(train_t.index.hour).mean())
    diurnal = np.array([climate.loc[d,h] for d,h in zip(temp.index.dayofyear,temp.index.hour)])
    diurnal = pd.Series(diurnal,index=temp.index)
    table = pd.DataFrame(index=temp.index)
    table["wet_observed"] = wet
    table["wet_probability"] = frequency+persistence*(wet.shift(1)-frequency)
    table["climate_probability"] = frequency
    table["BS_hour"] = (table.wet_probability-wet)**2
    table["BS_climate_hour"] = (frequency-wet)**2
    table["BS_improvement_hour"] = table.BS_climate_hour-table.BS_hour
    table["T_observed_C"] = temp
    table["T_persistence_C"] = temp.shift(1)
    table["T_diurnal_persistence_C"] = temp.shift(1)+diurnal-diurnal.shift(1)
    table["rain_intensity_persistence_mm_h"] = p.shift(1)*persistence
    # Every target is at t, issued at t-1h; only 2026 is an external temporal test.
    table["issued_at"] = table.index-pd.Timedelta(hours=1)
    table["evaluation_2026"] = table.index.year == 2026
    table.to_csv(OUT/"02_逐小时气象基线.csv",encoding="utf-8-sig",index_label="target_time")
    test = table.loc[table.index.year == 2026]
    bsrows = test.dropna(subset=["wet_observed","wet_probability"])
    bs, ref = float(bsrows.BS_hour.mean()),float(bsrows.BS_climate_hour.mean())
    ts = test.dropna(subset=["T_observed_C","T_persistence_C","T_diurnal_persistence_C"])
    def error(col):
        e = ts[col]-ts.T_observed_C
        return {"RMSE_C":float(np.sqrt((e*e).mean())),"MAE_C":float(e.abs().mean()),"n":len(e)}
    return {"training_years":list(CAL_YEARS),"horizon_h":1,"wet_threshold_mm_h":.1,
            "wet_training_frequency":frequency,"persistence_weight":persistence,
            "BS":bs,"BS_climate":ref,"BSS":1-bs/ref if ref>0 else None,"n_test_hours":len(bsrows),
            "T_diurnal_persistence":error("T_diurnal_persistence_C"),"T_plain_persistence":error("T_persistence_C"),
            "hourly_file":"02_逐小时气象基线.csv","missing_not_filled":True,
            "BSS_interpretation":"positive=better than frozen climatology, nonpositive=no skill",
            "limitation":"retrospective one-hour baseline, not an external weather forecast; temperature and rain
                 intensity are not Brier probabilities"}


# ============================================================ S4 触发评估模型
def build_trigger_features(feat: pd.DataFrame, dis: pd.DataFrame, gully: pd.DataFrame) -> pd.DataFrame:
    """合成触发概率所需的物理量与集总参数。"""
    q = dis["Q_water"].astype(float)
    qc = dis["Q_crit"].astype(float)
    T = q / qc.replace(0, np.nan)
    X = pd.DataFrame({
        "lnT": np.log(T.clip(1e-6, 1e3)),
        "lnT_6h": np.log(T.clip(1e-6, 1e3)).rolling(6, min_periods=1).mean()),
        "P_3h": feat["P_3h"], "P_24h": feat["P_24h"], "API": feat["API"],
        "cum_melt_6h": feat["cum_melt_energy"].rolling(6, min_periods=1).sum(),
        "Ta_2m": feat["Ta_2m"], "VPD2": feat["VPD2"],
        "dtheta": feat["dtheta_1_20"], "VWC": feat["VWC_0.1m"],
    }, index=feat.index)
    return X


def trigger_probability(feat: pd.DataFrame, dis: pd.DataFrame, static: dict,
                        gully: pd.DataFrame) -> dict:
    """事件回顾指数的logistic相对评分，非发生概率；斜率不可单事件识别。"""
    from sklearn.metrics import roc_auc_score

    T = (dis["Q_water"] / dis["Q_crit"].replace(0, np.nan)).astype(float)
    ev_hour = pd.Timestamp("2026-08-26 10:00")
    T_ev = float(T.loc[ev_hour]) if ev_hour in T.index and np.isfinite(T.loc[ev_hour]) else float("nan")
    kT = 4.0
    P = 1.0 / (1.0 + np.exp(np.clip(-kT * (T - 1.0),-700,700)))
    P.name = "P_trigger"

    # 空间易发性（沟道尺度）与集水区尺度的合成
    erod = static.get("松散物质可蚀性指数")
    s_catch = None if erod is None else float(erod) * (1.0 - 0.5 * static.get("陡坡面积比_gt30deg",np.nan))
    S_gully = float(gully["易发性"].median())

    res = {
        "模型形式": "P(t) = σ[k_T·(T(t) − 1)], T(t) = Q_w(t)/Q_c",
        "状态变量": "触发指数 T",
        "T_事件时刻_10:00": T_ev,
        "显式先验斜率_kT": float(kT),
        "kT来源": "显式先验4，扫描1/2/4/8；没有用事件窗口拟合或标定",
        "kT敏感性": {str(k): {"score_gt05_hours":int((T>1).sum()),
            "score_gt08_hours":int((T>1+np.log(4)/k).sum())} for k in (1.,2.,4.,8.)},
        "使用边界": "事件窗口回顾系数k已退出正式模型；T/P仅作回顾相对强度评分，单事件不识别发生概率；空间S不与时间评分相乘",
        "集水区易发性": s_catch,
        "集水区评分状态": "unavailable: land-cover class semantics unverified" if s_catch is None else "conditional",
```

```
        "沟道易发性中位数": S_gully,
        "P最大值": float(np.nanmax(P)), "P最大时刻": str(P.idxmax()),
        "P超过0.5的小时数": int((P > 0.5).sum()),
        "P超过0.5的小时占比": float((P.dropna() > 0.5).mean()),
    }

    # 事件时刻复现：24 h 内 P 首次超过 0.5 的时刻（预警起点）
    around = P.loc[EVENT_WIN[0]:EVENT_WIN[1]]
    cross = around[around >= 0.5]
    if len(cross):
        res["事件窗口首次超过0.5"] = str(cross.index[0])
        res["相对事件时刻的超前_h"] = float((EVENT_TS - cross.index[0]).total_seconds() / 3600)
        res["事件窗口峰值时刻"] = str(around.idxmax())
        res["峰值相对事件滞后_h"] = float((around.idxmax() - EVENT_TS).total_seconds() / 3600)
    else:
        res["事件窗口首次超过0.5"] = None

    # 已发生灾点内规模排序检验，不是发生/未发生验证。
    # 判决规则在使用之前写定（建模成果/问题2.md §2.4.6）：
    #  AUC>0.5 显著 → 可作排序依据；≈0.5 → 无判别力；<0.5 → 方向错误，判定失效。
    # 任何已执行检验都必须报告，无论方向是否有利。
    g = gully.dropna(subset=["易发性", "规模序"]).copy()
    if len(g) >= 20:
        y = (g["规模序"] >= 3).astype(int).values
        if y.sum() >= 3 and (1 - y).sum() >= 3:
            auc_v = float(roc_auc_score(y, g["易发性"]))
            res["空间排序AUC_易发性预测大规模"] = auc_v
            # 置换检验：AUC 与 0.5 的偏离是否超出随机波动
            rng_s = np.random.default_rng(20260826)
            null = np.array([roc_auc_score(rng_s.permutation(y), g["易发性"])
                             for _ in range(2000)])
            p_two = float(np.mean(np.abs(null - 0.5) >= abs(auc_v - 0.5)))
            res["空间排序AUC_置换检验"] = {
                "AUC": auc_v, "置换次数": 2000, "双侧p值": p_two,
                "判读": ("可作排序依据" if auc_v > 0.5 and p_two < 0.05 else
                         "无判别力（与随机排序无异）" if p_two >= 0.05 else
                         "已发生灾点的大规模排序反向；不能推断发生概率方向"),
            }
        # 不拟合的样本内起动流量—规模序次关联。
        res["样本内规模Kendall_tau"] = float(stats.kendalltau(
            np.log10(g["Q_crit_m3s"]), g["规模序"]).statistic)
        res["规模关联_n"] = int(len(g))

    # 分级预警阈值
    res["分级阈值"] = {"蓝色": [0.2, 0.4], "黄色": [0.4, 0.6],
                  "橙色": [0.6, 0.8], "红色": [0.8, 1.0]}
    res["各级小时数"] = {
        "蓝色(0.2-0.4)": int(((P >= 0.2) & (P < 0.4)).sum()),
        "黄色(0.4-0.6)": int(((P >= 0.4) & (P < 0.6)).sum()),
        "橙色(0.6-0.8)": int(((P >= 0.6) & (P < 0.8)).sum()),
        "红色(≥0.8)": int((P >= 0.8).sum()),
    }
    pd.DataFrame({"P_trigger": P, "T_index": T}).to_parquet(io.CACHE / "ptrigger.parquet")
    return res


# ============================================================ S5 多尺度可靠性
def reliability(feat: pd.DataFrame, dis: pd.DataFrame) -> dict:
    T = (dis["Q_water"] / dis["Q_crit"].replace(0, np.nan)).astype(float)
    res: dict = {}

    # 时间尺度 1：聚合方式对检出率的影响
    res["时间聚合"] = {}
    for tag, rule in (("3h", "3h"), ("6h", "6h"), ("日", "D")):
        agg_max = T.resample(rule).max()
        agg_mean = T.resample(rule).mean()
        needed = 20 if rule=="D" else (3 if rule=="3h" else 6)
        available = T.resample(rule).count()>=needed
        agg_max=agg_max.where(available);agg_mean=agg_mean.where(available)
        res["时间聚合"][tag] = {
            "T≥1的窗口数": int((agg_max >= 1).sum()),
            "按峰值判据检出率": float((agg_max[available] >= 1).mean()),
            "按均值判据检出率": float((agg_mean[available] >= 1).mean()),
            "峰值保持率(T_agg/T_max)": float(agg_mean[available].sum() / max(agg_max[available].sum(), 1e-9)),
            "口径": "3h/6h须全窗有效，日>=20小时；同有效窗mean与max比", "有效窗口数":int(available.sum()),
        }

    # 时间尺度 2：事件回顾提前量与有效观测日报警频度。
    thr_grid = np.arange(0.2, 1.0, 0.1)
    rows = []
    for p0 in thr_grid:
        hit = bool((T.loc[EVENT_WIN[0]:EVENT_WIN[1]] >= p0).any())
        lead = None
        if hit:
            s = T.loc[EVENT_WIN[0]:EVENT_WIN[1]]
            first = s[s >= p0].index[0]
            lead = float((EVENT_TS - first).total_seconds() / 3600)
        day = daily_valid(T,"mean")
```

```
        peak = T.resample("D").max().where(day.notna())
        false_alarm = float((peak.dropna() >= p0).mean())
        rows.append({"阈值": float(p0), "命中事件": hit, "提前量_h": lead,
                     "全序列报警日率": false_alarm,"有效日数":int(peak.notna().sum())})
    res["提前量与报警日率"] = rows
    res["报警率说明"] = "仅阈值超越日频度，无确证非灾日不能报真实伪警率"

    # 时间尺度 3：事件回顾指数的年际峰值（不是留年拟合验证）。
    res["年际T峰值"] = {int(y): float(T[T.index.year == y].max()) for y in sorted(set(T.index.year))}

    # 空间尺度 1：DEM 分辨率对主沟比降与起动流量的影响
    # 注意：本项使用**追迹口径**，与 q_c 的正式口径（中心线）不同，两者只在本节
    # 并列出现（建模成果/问题2.md §2.4.2 的三口径表）。此处比降用于比较分辨率
    # 之间的相对偏差，不用于替换模型中的 q_c。
    from step02b_spatial import coarse_dem
    from mmlib.terrain_prep import cell_meters
    dem,grid = io.load_dem()
    sens,areas = {},{}
    for factor in (1,2,3,4,6):
        z,cgrid = coarse_dem(dem,grid,factor)
        dy,dx = cell_meters(cgrid)
        filled=tr.fill_sinks(z)
        padded=np.pad(filled,1,constant_values=np.nan)
        dd=np.zeros(z.shape,dtype=np.int8);steep=np.zeros(z.shape)
        for key,(dr,dc) in tr.DIRS.items():
            neighbour=padded[1+dr:1+dr+z.shape[0],1+dc:1+dc+z.shape[1]]
            slope=(filled-neighbour)/np.hypot(dr*dy,dc*dx)
            good=slope>steep
            dd[good],steep[good]=key,slope[good]
        aa=tr.flow_accumulation(dd)
        r,c=io.lonlat_to_rowcol(cgrid,io.EVENT["port_lon"],io.EVENT["port_lat"])
        candidates=np.argwhere((aa>=max(50.,5e5/factor**2))&np.isfinite(z))
        if not len(candidates):
            raise ValueError("no candidate outlet at coarse resolution")
        distances=np.hypot((candidates[:,0]-r)*dy,(candidates[:,1]-c)*dx)
        outlet=tuple(int(v) for v in candidates[np.argmin(distances)])
        stem=tr.trace_upstream_mainstem(dd,aa,outlet,min_acc=max(50.,2000./factor**2))
        coords=np.asarray(stem)
        length=float(np.hypot(np.diff(coords[:,0])*dy,np.diff(coords[:,1])*dx).sum())
        zz=np.array([z[v] for v in stem])
        ss=float((zz[-1]-zz[0])/length) if length>0 else np.nan
        sens[f"{factor}x"]={"像元_m":float(np.sqrt(dx*dy)),"dy_m":dy,"dx_m":dx,
                            "主沟像元数":len(stem),"主沟长度_km":length/1000,
                            "主沟比降":ss,"起动单宽流量_m2s":float(tg._q_crit_unit(ss)) if ss>0 else None,
                            "口岸投影偏移_m":float(distances.min()),"grid_center":cgrid}
        mask=tr.upstream_mask(dd,outlet)
        areas[f"{factor}x"]={"像元_m":float(np.sqrt(dx*dy)),"集水区像元数":int(mask.sum()),
                             "集水区面积_km2":float(mask.sum()*dx*dy/1e6)}
    base_slope=sens["1x"]["主沟比降"]
    res["空间分辨率敏感性"]=sens
    res["基准主沟比降"]=base_slope
    res["基准起动单宽流量_m2s"]=float(tg._q_crit_unit(base_slope))
    res["集水区面积敏感性"]=areas
    a1=areas["1x"]["集水区面积_km2"]
    res["集水区面积_相对偏差"]={k:(v["集水区面积_km2"]-a1)/a1 for k,v in areas.items() if k!="1x"}

    # 空间尺度 3：沟道属性外推的可迁移性（84 条沟道的比降分布 vs 口岸集水区）
    g = tg.gully_table()
    res["沟道迁移性"] = {
        "历史沟道比降分位": {q: float(g["比降"].quantile(q)) for q in (0.05, 0.25, 0.5, 0.75, 0.95)},
        "口岸集水区主沟比降": base_slope,
        "在历史分布中的分位": float((g["比降"] < base_slope).mean()),
        "历史沟道面积极值": [float(g["面积_km2"].min()), float(g["面积_km2"].max())],
    }
    return res


def main() -> int:
    FIG.mkdir(parents=True, exist_ok=True)
    feat = pd.read_parquet(io.CACHE / "features.parquet")
    api_meta = tg.api_climate_thresholds(feat)
    feat.attrs["api_quantiles"] = (api_meta["q1"], api_meta["q3"])
    save_json(api_meta, "02_API分位.json")
    # 集水区特征由 step04 先行产出（它同时定义中心线口径）；此处若缺失则自行计算。
    if (OUT / "02_集水区特征.json").exists():
        info = json.loads((OUT / "02_集水区特征.json").read_text(encoding="utf-8"))
    else:
        info = tg.catchment_glacier_fraction()
        save_json(info, "02_集水区特征.json")
    model = tg.default_model(feat, calibrate=False)
    dis = model.discharge(feat)
    _event = dis.loc[FORMAL_EVENT_WIN[0]:FORMAL_EVENT_WIN[1]]
    _peak = float(_event["T_index"].max()) if _event["T_index"].notna().any() else np.nan
    legacy_candidates = [
        io.ROOT / "临时产物" / "审稿记录" / "冻结父版v5.1记录" / "v43_旧触发标定审计.json",
        io.ROOT / "临时产物" / "审稿记录" / "v43_旧触发标定审计.json",
    ]
    legacy_path = next((p for p in legacy_candidates if p.exists()), legacy_candidates[0])
    legacy_doc = json.loads(legacy_path.read_text(encoding="utf-8")) if legacy_path.exists() else {}
    legacy = legacy_doc.get("value", {})
```

```
    save_json({
        "status": "audit_only",
        "legacy_record": legacy_path.relative_to(io.ROOT).as_posix() if legacy_path.exists() else None,
        "legacy_record_sha256": (
            __import__("hashlib").sha256(legacy_path.read_bytes()).hexdigest()
            if legacy_path.exists() else None
        ),
        "formal_model_uses_legacy_k": False,
        "formal_mechanical_regime": model.mechanical_regime,
        "formal_q_crit_m2s": model.q_crit,
        "formal_C_N0": model.cn0,
        "formal_CN_provenance": (
            "explicit engineering scenario independent of land-cover classes; "
            "75/85/95 sensitivity reported"
        ),
        "formal_temperature_geometry": model.temperature_geometry,
        "legacy_k": legacy.get("标定系数k"),
        "legacy_calibration_note": "v4.3 event-window k remains audit-only and must not be read downstream",
        "event_window_T_peak": _peak,
        "event_window_T_is_formal": bool(model.mechanical_regime == "water_controlled" and _peak == _peak),
    }, "02_旧方法审计.json")
    print(f"触发指数序列 ok（正式状态 {model.mechanical_regime}，事件窗 T 峰值 {_peak}，"
          f"q_c={model.q_crit:.2f} m2/s；旧 k 只审计）")

    gc = gully_critical_conditions()
    save_json(gc, "02_沟道临界条件.json")
    print(f"沟道临界条件 ok：{gc['沟道数']} 条，起动流量中位数 "
          f"{gc['起动流量_m3s']['p50']:.3g} m3/s")

    idr = id_threshold(feat)
    save_json(idr, "02_降雨阈值.json")
    print(f"降雨 I-D：I = {idr['幂律系数a']:.2f} D^(-{idr['幂指数b']:.2f})，"
          f"{idr['独立暴雨事件数']} 场独立暴雨")

    mtl = melt_threshold(feat, info)
    save_json(mtl, "02_消融阈值.json")
    print(f"消融阈值 ok：源区订正 {mtl['源区气温订正_°C']:.1f} °C，"
          f"源区日融化 p90 {mtl['临界条件']['E_s_p90_MJm-2']:.2f} MJ/m2")

    static = json.loads((OUT / "02_静态特征.json").read_text(encoding="utf-8")) \
        if (OUT / "02_静态特征.json").exists() else {}
    gully = pd.read_parquet(io.CACHE / "gully_table.parquet")

    hourly_tuple, object_tuple, tuple_summary = build_state_tuple(
        feat, dis, model, mtl, gully
    )
    hourly_tuple.to_csv(
        OUT / "02_状态元组_逐小时.csv", encoding="utf-8-sig", index=True
    )
    object_tuple.to_csv(
        OUT / "02_状态元组_逐沟道.csv", encoding="utf-8-sig", index=False
    )
    save_json(tuple_summary, "02_状态元组.json")

    tp = trigger_probability(feat, dis, static, gully)
    tp["曲线数情景"] = cn_scenario_sensitivity(feat, model)
    tp["API_q1_q3"] = api_meta
    tp["状态元组"] = tuple_summary
    save_json(tp, "02_触发评估模型.json")
    print(f"触发评估模型 ok：P 最大 {tp['P最大值']:.3f} @ {tp['P最大时刻']}，"
          f"事件窗口首次超 0.5 = {tp.get('事件窗口首次超过0.5')}")

    rel = reliability(feat, dis)
    save_json(rel, "02_多尺度可靠性.json")
    save_json(business_screen(feat,idr), "02_水源业务判据.json")
    save_json(statistical_domains(), "02_统计临界域.json")
    save_json(long_temperature_check(feat), "02_长期气温对比.json")
    save_json(weather_baseline(feat), "02_气象基线检验.json")
    print("多尺度可靠性 ok")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

step02b_spatial.py　空间像元对照

```
"""问题2像元采样环境对照：分组嵌套验证、尺度与负样本敏感性。

No rasterio/fiona. Group counts, not the sampled pixel count, delimit inference.
Every resolution rebuilds terrain and polygon labels. No occurrence probability is claimed.
"""
from __future__ import annotations

import json
from pathlib import Path
import sys

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
from matplotlib.path import Path as PolygonPath
import numpy as np
import pandas as pd
from scipy import ndimage
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, roc_curve
from sklearn.model_selection import StratifiedGroupKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

sys.path.insert(0,str(Path(__file__).resolve().parent))
from mmlib import io_data as io, terrain as tr, geodesy as gd
from mmlib.terrain_prep import cell_meters

SEED = 20260826
MAX_POS_PER_POLYGON = 200
NEG_RATIO = 2
BACKGROUND_BLOCK_M = 2500.
MIN_AREA_M2 = 60.*30.*30.
CS = (.01,.1,1.)
OUTER = 4
INNER = 3
NEG_REPEATS = 4
BOOT = 300
CONTINUOUS = ["Z_m","slope","log10_area_m2","TWI","relief_m","glacier_distance_m"]
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei","Noto Sans SC","DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False


def plain(value):
   if isinstance(value,dict):
      return {str(k):plain(v) for k,v in value.items()}
   if isinstance(value,(list,tuple,np.ndarray)):
      return [plain(v) for v in value]
   if isinstance(value,(np.bool_,bool)):
      return bool(value)
   if isinstance(value,(np.integer,int)):
      return int(value)
   if isinstance(value,(np.floating,float)):
      return float(value) if np.isfinite(value) else None
   return value


def save(obj,name):
   path = io.DATA_OUT/name
   path.parent.mkdir(parents=True,exist_ok=True)
   path.write_text(json.dumps(plain(obj),ensure_ascii=False,indent=2),encoding="utf-8")


def coarse_dem(dem,grid,k):
   if k == 1:
      return dem.astype(float),dict(grid)
   h,w = dem.shape[0]//k,dem.shape[1]//k
   blocks = dem[:h*k,:w*k].reshape(h,k,w,k)
   count = np.isfinite(blocks).sum(axis=(1,3))
   result = np.nansum(blocks,axis=(1,3))/np.maximum(count,1)
   result[count < .8*k*k] = np.nan
   g = dict(grid)
   # Grid coordinates are pixel CENTERS; coarse centers shift by (k-1)/2.
   g.update(lon0=grid["lon0"]+(k-1)*grid["dlon"]/2,
          lat0=grid["lat0"]+(k-1)*grid["dlat"]/2,
          dlon=k*grid["dlon"],dlat=k*grid["dlat"])
   return result,g


def rasterize(polygons,grid,shape):
   labels = np.zeros(shape,dtype=np.int32)
   ny,nx = shape
   xs = grid["lon0"]+np.arange(nx)*grid["dlon"]
   ys = grid["lat0"]+np.arange(ny)*grid["dlat"]
   overlaps = 0
   for pid,poly in enumerate(polygons,1):
      xmin,ymin,xmax,ymax = poly["bbox"]
      rows = np.flatnonzero((ys>=ymin)&(ys<=ymax))
      cols = np.flatnonzero((xs>=xmin)&(xs<=xmax))
      if not len(rows) or not len(cols):
         continue
      xx,yy = np.meshgrid(xs[cols],ys[rows])
      xy = np.column_stack([xx.ravel(),yy.ravel()])
      # Even-odd ring rule retains polygon holes and disjoint parts.
      inside = np.zeros(len(xy),dtype=bool)
      points = poly["pts"]
      parts = list(poly["parts"])+[len(points)]
      for a,b in zip(parts[:-1],parts[1:]):
         inside ^= PolygonPath(points[a:b]).contains_points(xy,radius=1e-12)
      local = inside.reshape(len(rows),len(cols))
      rr,cc = np.meshgrid(rows,cols,indexing="ij")
      overlaps += int(((labels[rr,cc]>0)&local).sum())
      labels[rr[local],cc[local]] = pid
   return labels,overlaps


def glacier_lonlat():
   gn = io.load_glacier_nc()
   thick = gn["thickness"]
   mask = np.isfinite(thick)&(thick>1)
   gt = gn["geotransform"]
```

```
    rr,cc = np.indices(thick.shape)
    xx = gt[0]+(cc+.5)*gt[1]+(rr+.5)*gt[2]
    yy = gt[3]+(cc+.5)*gt[4]+(rr+.5)*gt[5]
    coords = np.array([gd.utm_to_lonlat(float(x),float(y),45)
                   for x,y in zip(xx[mask],yy[mask])])
    return coords


def nearest_lc(grid,shape):
    lc,lg = io.load_landcover(2023)
    xs = grid["lon0"]+np.arange(shape[1])*grid["dlon"]
    ys = grid["lat0"]+np.arange(shape[0])*grid["dlat"]
    ci = np.rint((xs-lg["lon0"])/lg["dlon"]).astype(int)
    ri = np.rint((ys-lg["lat0"])/lg["dlat"]).astype(int)
    valid = (ri[:,None]>=0)&(ri[:,None]<lc.shape[0])&(ci[None,:]>=0)&(ci[None,:]<lc.shape[1])
    result = np.full(shape,-1,dtype=np.int16)
    selected = lc[np.clip(ri,0,lc.shape[0]-1)[:,None],np.clip(ci,0,lc.shape[1]-1)[None,:]]
    result[valid] = selected[valid]
    return result


def build_fields(dem,grid,polygons,glacier):
    dy,dx = cell_meters(grid)
    filled = tr.fill_sinks(dem)
    # Rebuild D8 using metric diagonal lengths, including anisotropic grid spacing.
    pad = np.pad(filled,1,constant_values=np.nan)
    direction = np.zeros(dem.shape,dtype=np.int8)
    steep = np.zeros(dem.shape)
    for key,(dr,dc) in tr.DIRS.items():
        nb = pad[1+dr:1+dr+dem.shape[0],1+dc:1+dc+dem.shape[1]]
        slope = (filled-nb)/np.hypot(dr*dy,dc*dx)
        good = slope>steep
        direction[good],steep[good] = key,slope[good]
    acc = tr.flow_accumulation(direction)
    fin = np.isfinite(dem)
    z = np.where(fin,dem,0.)
    gy,gx = np.gradient(z,dy,dx)
    slope = np.hypot(gx,gy)
    # A gradient touching nodata is unavailable, rather than an artificial cliff.
    valid_gradient = ndimage.binary_erosion(fin,structure=np.ones((3,3)),border_value=0)
    slope[~valid_gradient] = np.nan
    win = max(3,int(round(330/min(dx,dy))))
    if win%2 == 0:
        win += 1
    high = ndimage.maximum_filter(np.where(fin,dem,-np.inf),size=win)
    low = ndimage.minimum_filter(np.where(fin,dem,np.inf),size=win)
    relief = high-low
    area = acc*dx*dy
    twi = np.log(np.maximum(area/dx,1.)/np.maximum(slope,1e-4))
    gmask = np.zeros(dem.shape,dtype=bool)
    rr,cc = io.lonlat_to_rowcol(grid,glacier[:,0],glacier[:,1])
    rr,cc = np.rint(rr).astype(int),np.rint(cc).astype(int)
    ok = (rr>=0)&(rr<dem.shape[0])&(cc>=0)&(cc<dem.shape[1])
    gmask[rr[ok],cc[ok]] = True
    if not gmask.any():
        raise ValueError("attachment4 glacier has no cells inside DEM")
    distance = ndimage.distance_transform_edt(~gmask,sampling=(dy,dx))
    labels,overlaps = rasterize(polygons,grid,dem.shape)
    lc = nearest_lc(grid,dem.shape)
    valid = fin&np.isfinite(slope)&np.isfinite(twi)&np.isfinite(relief)&(lc>=0)
    pos = (labels>0)&valid
    buffer_distance = ndimage.distance_transform_edt(labels==0,sampling=(dy,dx))
    neg = valid&(labels==0)&(buffer_distance>500)&(area>=MIN_AREA_M2)
    fields = dict(Z_m=dem,slope=slope,log10_area_m2=np.log10(np.maximum(area,1)),
              TWI=twi,relief_m=relief,glacier_distance_m=distance,LC=lc)
    return fields,labels,pos,neg,dict(cell_dy_m=dy,cell_dx_m=dx,
          positive_full_pixels=int(pos.sum()),negative_pool_pixels=int(neg.sum()),
          full_valid_pixels_by_polygon={int(pid):int(((labels==pid)&pos).sum())
                                        for pid in np.unique(labels[pos])},
          polygons_with_valid_pixels=int(np.unique(labels[pos]).size),overlap_pixels_last_polygon_wins=overlaps,
          valid_dem_pixels=int(valid.sum()),relief_window_pixels=win,
          negative_min_area_m2=MIN_AREA_M2,buffer_m=500.)


def sample(fields,labels,pos,neg,meta,seed,positive_indices=None):
    rng = np.random.default_rng(seed)
    if positive_indices is None:
        selected = []
        for pid in np.unique(labels[pos]):
            pool = np.flatnonzero(pos&(labels==pid))
            selected.extend(rng.choice(pool,min(MAX_POS_PER_POLYGON,len(pool)),replace=False).tolist())
        positive_indices = np.asarray(selected,dtype=int)
    target = min(len(positive_indices),int(neg.sum())//NEG_RATIO)
    if target < len(positive_indices):
        positive_indices = rng.choice(positive_indices,target,replace=False)
    if target < 20:
        raise ValueError("insufficient matched sampling pool")
    negative_indices = rng.choice(np.flatnonzero(neg),NEG_RATIO*len(positive_indices),replace=False)
    mask = np.zeros(labels.size,dtype=bool)
    mask[positive_indices],mask[negative_indices] = True,True
    mask = mask.reshape(labels.shape)
    rr,cc = np.where(mask)
    # All arrays are extracted from exactly the same spatial-order mask.
    tab = pd.DataFrame({name:arr[mask] for name,arr in fields.items()})
    tab["row"],tab["col"] = rr,cc
    tab["y"] = pos[mask].astype(int)
    block_r = np.floor(rr*meta["cell_dy_m"]/BACKGROUND_BLOCK_M).astype(int)
```

```
   block_c = np.floor(cc*meta["cell_dx_m"]/BACKGROUND_BLOCK_M).astype(int)
   background = 1000000+block_r*10000+block_c
   tab["group"] = np.where(tab.y==1,labels[mask],background)
   assert np.array_equal(tab.y.to_numpy(),(labels[mask]>0).astype(int))
   assert set(tab.loc[tab.y==1,"group"]).isdisjoint(set(tab.loc[tab.y==0,"group"]))
   return tab,positive_indices


def matrix(tab,classes):
   x = tab[CONTINUOUS].to_numpy(dtype=float)
   cats = np.column_stack([(tab.LC.to_numpy()==c).astype(float) for c in classes])
   return np.column_stack([x,cats]),CONTINUOUS+[f"LC_original_code_{c}" for c in classes]


def folds(y,groups,n,seed):
   n = min(n,len(np.unique(groups[y==1])),len(np.unique(groups[y==0])))
   if n<2:
      raise ValueError("fewer than two independent groups per sample class")
   splitter = StratifiedGroupKFold(n_splits=n,shuffle=True,random_state=seed)
   return list(splitter.split(np.zeros(len(y)),y,groups))


def iv_fit(x,y):
   fitted = []
   for k in range(x.shape[1]):
      # Fits both cut points and class frequencies on TRAINING data only.
      bins = np.unique(np.quantile(x[:,k],[.2,.4,.6,.8]))
      edges = np.r_[-np.inf,bins,np.inf]
      idx = np.searchsorted(edges[1:-1],x[:,k],side="right")
      plus = np.bincount(idx[y==1],minlength=len(edges)-1)+.5
      minus = np.bincount(idx[y==0],minlength=len(edges)-1)+.5
      weights = np.log((plus/plus.sum())/(minus/minus.sum()))
      fitted.append((edges,weights))
   return fitted


def iv_predict(x,fitted):
   score = np.zeros(len(x))
   for k,(edges,weights) in enumerate(fitted):
      idx = np.searchsorted(edges[1:-1],x[:,k],side="right")
      score += weights[idx]
   return score


def fit_lr(x,y,c):
   clf = make_pipeline(StandardScaler(),LogisticRegression(C=c,max_iter=700,
                    class_weight="balanced",solver="lbfgs",random_state=SEED))
   clf.fit(x,y)
   return clf


def nested_cv(x,y,groups,seed=SEED):
   plr,piv = np.full(len(y),np.nan),np.full(len(y),np.nan)
   fold_id = np.full(len(y),-1)
   chosen,records = [],[]
   for fold,(train,test) in enumerate(folds(y,groups,OUTER,seed)):
      assert set(groups[train]).isdisjoint(set(groups[test]))
      inner = folds(y[train],groups[train],INNER,seed+fold+1)
      scores = {}
      for c in CS:
         aucs = []
         for a,b in inner:
            ia,ib = train[a],train[b]
            if len(np.unique(y[ia]))<2 or len(np.unique(y[ib]))<2:
               continue
            pred = fit_lr(x[ia],y[ia],c).predict_proba(x[ib])[:,1]
            aucs.append(roc_auc_score(y[ib],pred))
         scores[c] = float(np.mean(aucs)) if aucs else -np.inf
      c = max(CS,key=lambda v:scores[v])
      if not np.isfinite(scores[c]):
         raise ValueError("no valid class-balanced inner group fold")
      clf = fit_lr(x[train],y[train],c)
      plr[test] = clf.predict_proba(x[test])[:,1]
      piv[test] = iv_predict(x[test],iv_fit(x[train],y[train]))
      fold_id[test] = fold
      chosen.append(c)
      records.append({"fold":fold,"C":c,"train_groups":len(np.unique(groups[train])),
                      "test_groups":len(np.unique(groups[test])),"test_n":len(test),
                      "standardized_coefficients":clf.named_steps["logisticregression"].coef_[0].tolist()})
   assert np.isfinite(plr).all() and np.isfinite(piv).all() and (fold_id>=0).all()
   return plr,piv,fold_id,chosen,records


def group_bootstrap(y,p,groups,seed):
   rng = np.random.default_rng(seed)
   positive = np.unique(groups[y==1])
   background = np.unique(groups[y==0])
   indices = {g:np.flatnonzero(groups==g) for g in np.unique(groups)}
   aucs = []
   for _ in range(BOOT):
      pick = np.r_[rng.choice(positive,len(positive),replace=True),
                   rng.choice(background,len(background),replace=True)]
      ix = np.concatenate([indices[g] for g in pick])
      aucs.append(roc_auc_score(y[ix],p[ix]))
   return np.quantile(aucs,[.025,.975]).tolist()
```

```python
def evaluate(tab,classes,seed):
    x,names = matrix(tab,classes)
    y,groups = tab.y.to_numpy(),tab.group.to_numpy()
    plr,piv,fold,chosen,records = nested_cv(x,y,groups,seed)
    c = float(pd.Series(chosen).mode().iloc[0])
    final = fit_lr(x,y,c)
    coefs = final.named_steps["logisticregression"].coef_[0]
    direction = {}
    for i,name in enumerate(CONTINUOUS):
        difference = float(x[y==1,i].mean()-x[y==0,i].mean())
        direction[name] = {"positive_minus_background_mean":difference,
                           "standardized_coefficient":float(coefs[i]),
                           "sign_flip":bool(difference*coefs[i]<0),
                           "univariate_auc_positive_direction":float(roc_auc_score(y,x[:,i])),
                           "outer_fold_coefficient_signs":[int(np.sign(r["standardized_coefficients"][i])) for r in
                                records]}
    result = {"auc_lr_oof":roc_auc_score(y,plr),"auc_iv_oof":roc_auc_score(y,piv),
              "auc_lr_group_bootstrap_ci":group_bootstrap(y,plr,groups,seed+100),
              "auc_iv_group_bootstrap_ci":group_bootstrap(y,piv,groups,seed+101),
              "n_positive_pixels":int(y.sum()),"n_background_pixels":int((y==0).sum()),
              "n_polygon_groups":len(np.unique(groups[y==1])),
              "n_background_groups":len(np.unique(groups[y==0])),
              "sampled_pixels_by_group":tab.groupby("group").size().to_dict(),
              "outer_folds":records,"selected_C_by_fold":chosen,"final_descriptive_C":c,
              "continuous_correlation":pd.DataFrame(x[:,:len(CONTINUOUS)],columns=CONTINUOUS).corr().to_dict(),
              "feature_names":names,"original_landcover_codes":classes,
              "standardized_coefficients":dict(zip(names,coefs)),"direction_diagnostic":direction,
              "direction_note":"sign changes flag collinearity/sampling questions, not proof of bias",
              "group_bootstrap_note":"resamples polygon and background groups with replacement, fixed OOF scores, not a
                    full model-refitting CI"}
    tab = tab.copy()
    tab["lr_oof"],tab["iv_oof"],tab["outer_fold"] = plr,piv,fold
    return result,tab,final,names


def plot_results(base,scales,tab,mapgrid,dem,grid,polygons):
    fig,axes = plt.subplots(2,2,figsize=(13,10),constrained_layout=True)
    ax = axes[0,0]
    ext = [grid["lon0"],grid["lon0"]+grid["dlon"]*(dem.shape[1]-1),
         grid["lat0"]+grid["dlat"]*(dem.shape[0]-1),grid["lat0"]]
    image = ax.imshow(mapgrid,extent=ext,cmap="YlOrRd",vmin=0,vmax=1)
    for poly in polygons:
        ax.plot(poly["pts"][:,0],poly["pts"][:,1],lw=.35,c="navy",alpha=.5)
    ax.set_xlim(ext[:2]);ax.set_ylim(ext[2:])
    ax.set_xlabel("经度 (°E)");ax.set_ylabel("纬度 (°N)")
    ax.set_title("(a) 采样环境对比分数（非发生概率）")
    fig.colorbar(image,ax=ax,label="环境对比分数")
    ax = axes[0,1]
    for name,col in (("分组嵌套逻辑回归","lr_oof"),("训练折分箱信息量","iv_oof")):
        fpr,tpr,_ = roc_curve(tab.y,tab[col])
        ax.plot(fpr,tpr,label=f"{name} AUC={roc_auc_score(tab.y,tab[col]):.3f}")
    ax.plot([0,1],[0,1],"--",c="gray");ax.legend(fontsize=8)
    ax.set_xlabel("背景像元高评分率");ax.set_ylabel("灾点面像元高评分率")
    ax.set_title("(b) 外层组留出预测（非真实灾/非灾）")
    ax = axes[1,0]
    labels=list(scales); vals=[scales[k]["auc_lr_oof"] for k in labels]
    ci=np.array([scales[k]["auc_lr_group_bootstrap_ci"] for k in labels])
    ax.errorbar(np.arange(len(vals)),vals,yerr=np.maximum(0,np.vstack([np.array(vals)-ci[:,0],ci[:,1]-vals])),fmt="o
        -")
    ax.set_xticks(np.arange(len(vals)),["约30米","约90米","约270米"]);ax.set_ylim(0,1)
    ax.set_ylabel("组留出AUC及组自助95%带");ax.set_title("(c) 各尺度重新建模，不证明网格收敛")
    ax=axes[1,1]
    coef=base["standardized_coefficients"]
    names=CONTINUOUS
    ax.barh(np.arange(len(names)),[coef[k] for k in names])
    ax.set_yticks(np.arange(len(names)),["高程","坡度","汇流面积对数","湿润指数","局地高差","距附件4冰川"])
    ax.axvline(0,c="gray",lw=.8);ax.set_xlabel("全样本描述拟合标准化系数")
    ax.set_title("(d) 条件系数，不作因果方向")
    path=io.FIG_OUT/"02b_空间环境对照.png"
    fig.savefig(path,dpi=160);plt.close(fig)
    return str(path)


def main():
    io.DATA_OUT.mkdir(parents=True,exist_ok=True);io.FIG_OUT.mkdir(parents=True,exist_ok=True)
    dem0,grid0=io.load_dem()
    shp=io.HAZARD_DIR/"西藏日喀则市吉隆县泥石流灾害区历史灾点数据.shp"
    polygons=io.read_shp_polygons(shp)
    glacier=glacier_lonlat()
    scales,base_tab,base_map,base_fields,base_labels,base_pos,base_neg,base_meta={},None,None,None,None,None,None,
        None
    for k,name in ((1,"30m_nominal"),(3,"90m_nominal"),(9,"270m_nominal")):
        print("spatial rebuild",name,flush=True)
        dem,grid=coarse_dem(dem0,grid0,k)
        fields,labels,pos,neg,meta=build_fields(dem,grid,polygons,glacier)
        classes=sorted(int(c) for c in np.unique(fields["LC"][fields["LC"]>=0]))
        tab,pidx=sample(fields,labels,pos,neg,meta,SEED+k)
        result,oof,clf,names=evaluate(tab,classes,SEED+k)
        result["sampling_frame"]=meta
        result["landcover_by_sample_class"]={str(y):oof.loc[oof.y==y,"LC"].value_counts().to_dict() for y in (0,1)}
        oof.to_csv(io.DATA_OUT/f"02b_{name}_样本OOF.csv",index=False,encoding="utf-8-sig")
        scales[name]=result
        if k==1:
            base_tab=oof;base_fields=fields;base_labels=labels;base_pos=pos;base_neg=neg;base_meta=meta
```

```
        valid=np.isfinite(fields["slope"])&np.isfinite(fields["TWI"])&(fields["LC"]>=0)
        base_map=np.full(dem.shape,np.nan,dtype="float32")
        rr,cc=np.where(valid)
        for start in range(0,len(rr),50000):
            r,c=rr[start:start+50000],cc[start:start+50000]
            local=pd.DataFrame({n:arr[r,c] for n,arr in fields.items()})
            xx,_=matrix(local,classes)
            base_map[r,c]=clf.predict_proba(xx)[:,1]
        np.save(io.DATA_OUT/"02b_环境对比分数.npy",base_map)
        positives=pidx
        base_classes=classes
    print(name,result["auc_lr_oof"],"groups",result["n_polygon_groups"],flush=True)
repeats=[]
for i in range(NEG_REPEATS):
    tab,_=sample(base_fields,base_labels,base_pos,base_neg,base_meta,SEED+500+i,positives)
    result,_,_,_=evaluate(tab,base_classes,SEED)
    repeats.append({"repeat":i,"auc_lr_oof":result["auc_lr_oof"],"auc_iv_oof":result["auc_iv_oof"],
                    "n_polygon_groups":result["n_polygon_groups"],"n_background_groups":result["n_background_groups"
                        ]})
background=io.glacier_dhdt_stats()
fig=plot_results(scales["30m_nominal"],scales,base_tab,base_map,dem0,grid0,polygons)
save({"seed":SEED,"scales":scales,"negative_resampling":repeats,
      "n_historical_polygon_features":len(polygons),
      "base_grid":grid0,"base_shape":list(dem0.shape),
      "negative_resampling_auc_range":[min(r["auc_lr_oof"] for r in repeats),max(r["auc_lr_oof"] for r in repeats)
          ],
      "protocol":{"positive_cap_per_polygon":MAX_POS_PER_POLYGON,"negative_ratio":NEG_RATIO,
                  "background_block_m":BACKGROUND_BLOCK_M,"bootstrap_replicates":BOOT,
                  "negative_repeats":NEG_REPEATS,"CRS":"geographic grid, pixel-center sampling",
                  "independent_units":"polygons/background blocks, not pixels",
                  "scales_rebuild":"block-mean DEM, metric D8/accumulation/slope, polygon rasterization at new
                       centers"},
      "dhdt_static_background":background,"dhdt_in_coefficients":False,
      "dhdt_note":"regional background has no identified within-domain spatial variation; not interpolated or
           fitted",
      "interpretation":"sampling environment contrast only; background unlabelled, not confirmed non-disaster; not
            for zoning or trigger decisions",
      "figure":fig},"02b_空间对照.json")
return 0


if __name__=="__main__":
    raise SystemExit(main())
```

step02c_spatial_matched.py 共同域空间隔离

```
"""Common-channel sampling with shared physical groups and nested spatial purge."""
from __future__ import annotations

import json
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.spatial import cKDTree
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import StratifiedGroupKFold

import step02b_spatial as legacy
from mmlib import io_data as io

plt.rcParams.update({
    "font.family": "sans-serif",
    "font.sans-serif": ["Microsoft YaHei", "Noto Sans SC", "DejaVu Sans"],
    "font.size": 22,
    "axes.titlesize": 22,
    "axes.labelsize": 20,
    "xtick.labelsize": 16,
    "ytick.labelsize": 16,
    "legend.fontsize": 14,
})

SEED = 20260826
MIN_AREA_M2 = 50000.
BLOCK_M = 2500.
PURGE_M = 500.
BOOT = 300


class InsufficientDesign(ValueError):
    def __init__(self, message, diagnostics=None):
        super().__init__(message)
        self.diagnostics = diagnostics or {}


def shared_groups(labels, meta, polygons=None, grid=None):
    """Union all physical blocks touched by any single historical polygon."""
    rr, cc = np.indices(labels.shape)
    br = np.floor(rr * meta["cell_dy_m"] / BLOCK_M).astype(int)
    bc = np.floor(cc * meta["cell_dx_m"] / BLOCK_M).astype(int)
    ncols = int(bc.max()) + 1
    blocks = br * ncols + bc
    parent = np.arange(int(blocks.max()) + 1)
```

```

    def find(a):
        while parent[a] != a:
            parent[a] = parent[parent[a]]
            a = parent[a]
        return int(a)

    touched = {}
    identifiers = range(1,len(polygons)+1) if polygons is not None else np.unique(labels[labels > 0])
    for pid in identifiers:
        # Group from every original polygon before overlap priority can erase a cell.
        support = legacy.rasterize([polygons[pid-1]],grid,labels.shape)[0]>0 if polygons is not None else labels ==
            pid
        bs = np.unique(blocks[support])
        if len(bs)==0:
            continue
        touched[int(pid)] = bs.tolist()
        root = find(int(bs[0]))
        for b in bs[1:]:
            other = find(int(b))
            parent[max(root, other)] = min(root, other)
            root = find(root)
    lookup = np.array([find(i) for i in range(len(parent))])
    result = lookup[blocks]
    assert all(np.unique(lookup[np.asarray(bs)]).size == 1 for bs in touched.values())
    return result, {"physical_blocks": int(np.unique(blocks).size),
                    "connected_groups": int(np.unique(result).size),
                    "blocks_by_polygon": touched,
                    "union_rule": "all rasterized polygon cells before sampling, both sample classes share groups"}


def sample(fields, labels, pos, neg, meta, group_grid, seed, positive=None):
    rng = np.random.default_rng(seed)
    if positive is None:
        indices = []
        for pid in np.unique(labels[pos]):
            pool = np.flatnonzero(pos & (labels == pid))
            indices.extend(rng.choice(pool, min(200, len(pool)), replace=False).tolist())
        positive = np.asarray(indices, dtype=int)
    if len(positive) < 20 or neg.sum() < 2 * len(positive):
        raise InsufficientDesign("fewer than 20 positive cells or insufficient 2:1 background pool")
    negative = rng.choice(np.flatnonzero(neg), 2 * len(positive), replace=False)
    index = np.sort(np.r_[positive, negative])
    rr, cc = np.unravel_index(index, labels.shape)
    tab = pd.DataFrame({name: field.ravel()[index] for name, field in fields.items()})
    tab["row"], tab["col"] = rr, cc
    tab["x_m"], tab["y_m"] = cc * meta["cell_dx_m"], rr * meta["cell_dy_m"]
    tab["y"] = pos.ravel()[index].astype(int)
    tab["polygon"] = labels.ravel()[index]
    tab["group"] = group_grid.ravel()[index]
    assert np.isfinite(tab[legacy.CONTINUOUS].to_numpy()).all()
    assert (10 ** tab.log10_area_m2 >= MIN_AREA_M2 - 1e-6).all()
    assert (tab.groupby("polygon").group.nunique().drop(index=0, errors="ignore") <= 1).all()
    return tab, positive


def purge(train, test, xy):
    distance = cKDTree(xy[test]).query(xy[train], k=1)[0]
    keep = distance > PURGE_M
    retained = train[keep]
    return retained, {"train_before_purge": int(len(train)), "train_after_purge": int(keep.sum()),
                      "purged_n": int((~keep).sum()),
                      "minimum_retained_test_distance_m": float(distance[keep].min()) if keep.any() else None}


def grouped_folds(y, groups, xy, nmax, seed):
    attempts = []
    for n in range(min(nmax, len(np.unique(groups))), 1, -1):
        folds = []
        for train, test in StratifiedGroupKFold(n_splits=n, shuffle=True, random_state=seed).split(xy, y, groups):
            kept, record = purge(train, test, xy)
            if len(np.unique(y[kept])) < 2 or len(np.unique(y[test])) < 2:
                attempts.append({"n_folds":n,"failed_fold":len(folds),"train_classes":np.unique(y[kept]).tolist(),
                                 "test_classes":np.unique(y[test]).tolist(),**record})
                break
            assert set(groups[kept]).isdisjoint(set(groups[test]))
            assert record["minimum_retained_test_distance_m"] > PURGE_M
            record.update(train_positive=int(y[kept].sum()), test_positive=int(y[test].sum()),
                          train_background=int((y[kept] == 0).sum()), test_background=int((y[test] == 0).sum()),
                          train_groups=int(np.unique(groups[kept]).size), test_groups=int(np.unique(groups[test]).size),
                          train_group_ids=np.unique(groups[kept]).tolist(), test_group_ids=np.unique(groups[test]).tolist
                              ())
            folds.append((kept, test, record))
        if len(folds) == n:
            return folds
    raise InsufficientDesign("cannot form >=2 shared-group folds with both classes after 500 m purge",
                             {"n_positive_groups":int(np.unique(groups[y==1]).size),
                              "n_background_groups":int(np.unique(groups[y==0]).size),"attempts":attempts})


def group_bootstrap(y, score, groups, seed):
    rng = np.random.default_rng(seed)
    unique = np.unique(groups)
    indices = {g: np.flatnonzero(groups == g) for g in unique}
    auc = []
    for _ in range(BOOT):
        ix = np.concatenate([indices[g] for g in rng.choice(unique, len(unique), replace=True)])
```

```
      if len(np.unique(y[ix])) == 2:
         auc.append(roc_auc_score(y[ix], score[ix]))
   return {"interval": np.quantile(auc, [.025, .975]).tolist() if auc else None,
         "valid": len(auc), "requested": BOOT,
         "scope": "shared connected groups resampled once, both classes retained together; fixed OOF, not refitted-
              model CI"}


def evaluate(tab, classes, seed):
   x, names = legacy.matrix(tab, classes)
   y, groups = tab.y.to_numpy(), tab.group.to_numpy()
   xy = tab[["x_m", "y_m"]].to_numpy()
   lr, iv, fold_ids = np.full(len(y), np.nan), np.full(len(y), np.nan), np.full(len(y), -1)
   records = []
   outer = grouped_folds(y, groups, xy, 4, seed)
   for fold, (train, test, record) in enumerate(outer):
      try:
         inner = grouped_folds(y[train], groups[train], xy[train], 3, seed + fold + 1)
      except InsufficientDesign as exc:
         raise InsufficientDesign("inner spatial tuning unavailable: " + str(exc),
                           {"outer_fold":fold,"outer_record":record,"inner":exc.diagnostics}) from exc
      scores = {}
      for c in legacy.CS:
         scores[c] = float(np.mean([
            roc_auc_score(y[train[b]], legacy.fit_lr(x[train[a]], y[train[a]], c).predict_proba(x[train[b]])[:, 1])
            for a, b, _ in inner]))
      c = max(legacy.CS, key=lambda candidate: scores[candidate])
      clf = legacy.fit_lr(x[train], y[train], c)
      lr[test] = clf.predict_proba(x[test])[:, 1]
      iv[test] = legacy.iv_predict(x[test], legacy.iv_fit(x[train], y[train]))
      fold_ids[test] = fold
      record.update(fold=fold, selected_C=c, inner_scores={str(k): v for k, v in scores.items()},
                inner_folds=[rec for _, _, rec in inner],
                standardized_coefficients=dict(zip(names, clf.named_steps["logisticregression"].coef_[0])))
      records.append(record)
   assert np.isfinite(lr).all() and np.isfinite(iv).all() and (fold_ids >= 0).all()
   oof = tab.assign(lr_oof=lr, iv_oof=iv, outer_fold=fold_ids)
   result = {"status": "available", "n_positive": int(y.sum()), "n_background": int((y == 0).sum()),
          "n_polygons": int(tab.loc[tab.y == 1, "polygon"].nunique()),
          "n_shared_groups": int(np.unique(groups).size),
          "n_groups_with_both_classes": int((tab.groupby("group").y.nunique() == 2).sum()),
          "auc_lr_oof": float(roc_auc_score(y, lr)), "auc_iv_oof": float(roc_auc_score(y, iv)),
          "lr_group_bootstrap": group_bootstrap(y, lr, groups, seed + 100),
          "iv_group_bootstrap": group_bootstrap(y, iv, groups, seed + 101),
          "outer_folds": records, "feature_names": names,
          "effective_outer_folds": len(outer), "original_landcover_codes": classes}
   return result, oof


def plot_results(scales, old, base_tab):
   fig = plt.figure(figsize=(12.5, 8.0), constrained_layout=True)
   grid = fig.add_gridspec(2, 2, height_ratios=(0.85, 1.15))
   ax_a = fig.add_subplot(grid[0, :])
   ax_b = fig.add_subplot(grid[1, 0])
   ax_c = fig.add_subplot(grid[1, 1])
   keys = list(scales)
   xx = np.arange(len(keys))
   ax_a.plot(xx, [old["scales"][k]["auc_lr_oof"] for k in keys], "o--", label="继承采样与分组协议")
   available = [(i, scales[k]) for i, k in enumerate(keys) if scales[k]["status"] == "available"]
   if available:
      ax_a.plot([i for i, _ in available], [r["auc_lr_oof"] for _, r in available], "s-", label="共同沟谷域十空间清除")
   else:
      ax_a.text(.03,.09,"新协议三尺度均无可用嵌套验证\n不回退随机像元折，不填伪AUC",transform=ax_a.transAxes,fontsize=14)
   ax_a.set(xticks=xx, xticklabels=["约30米", "约90米", "约270米"], ylim=(0, 1), ylabel="外层组留出AUC", title="(a) 复合
        协议差异，非单项归因")
   ax_a.legend(fontsize=14)
   for i, result in available:
      bs = result["lr_group_bootstrap"]["interval"]
      if bs:
         ax_b.plot([i, i], bs, color="teal", lw=2)
         ax_b.plot(i, result["auc_lr_oof"], "o", color="teal")
   if not available:
      positive_groups = [scales[k]['sampling_frame'].get('n_sampled_positive_groups', 0) for k in keys]
      ax_b.bar(xx, positive_groups, color='teal', label="可用正类共享组")
      ax_b.axhline(8, color="crimson", ls="--", lw=1.5, label="4折最低需求（8组）")
      for i, value in enumerate(positive_groups):
         ax_b.text(i, value + 0.2, f"{value}", ha="center", fontsize=14)
      ax_b.set(xticks=xx, xticklabels=["约30米","约90米","约270米"], ylim=(0, 9),
            ylabel="含正类的共享连通组数", title="(b) 正类组仅2个，低于4折需求")
      ax_b.legend(fontsize=11, loc="upper right")
   else:
      ax_b.set(xticks=xx, xticklabels=["约30米","约90米","约270米"], ylim=(0, 1), ylabel="AUC与固定OOF组自助带", title="
           (b) 共享连通组重采样，非像元独立")
   if scales[keys[0]]["status"] == "available":
      records = scales[keys[0]]["outer_folds"]
      before = np.array([r["train_before_purge"] for r in records])
      kept = np.array([r["train_after_purge"] for r in records])
      ax_c.bar(np.arange(len(records)), kept, label="隔离后训练像元")
      ax_c.bar(np.arange(len(records)), before - kept, bottom=kept, label="500米内清除")
      ax_c.legend(fontsize=14)
      ax_c.set(xlabel="外层测试折序号", ylabel="像元数（非独立事件数）", title="(c) 外层清除量；内层同样隔离")
   else:
      ax_c.bar(xx,[scales[k]['sampling_frame']['positive_common_pixels'] for k in keys],label="共同域内历史面像元")
      ax_c.set(xticks=xx,xticklabels=["约30米","约90米","约270米"],ylabel="可用正类像元数",title="(c) 像元充足不等于有独立验
```

```
            证组")
        ax_c.legend(fontsize=14)
    path = io.FIG_OUT / "02c_共同域空间隔离.png"
    fig.savefig(path, dpi=160)
    plt.close(fig)
    return path.name


def main():
    dem0, grid0 = io.load_dem()
    polygons = io.read_shp_polygons(io.HAZARD_DIR / "西藏日喀则市吉隆县泥石流灾害区历史灾点数据.shp")
    glacier = legacy.glacier_lonlat()
    scales, base_tab, repeated = {}, None, []
    for k, name in [(1, "30m_nominal"), (3, "90m_nominal"), (9, "270m_nominal")]:
        print("matched spatial rebuild", name, flush=True)
        dem, grid = legacy.coarse_dem(dem0, grid0, k)
        fields, labels, valid_pos, old_neg, meta = legacy.build_fields(dem, grid, polygons, glacier)
        channel = 10 ** fields["log10_area_m2"] >= MIN_AREA_M2 - 1e-6
        # Reconstruct the exact common valid domain, not the older negative threshold.
        from scipy.ndimage import distance_transform_edt
        valid = np.isfinite(np.stack([fields[n] for n in legacy.CONTINUOUS])).all(axis=0) & (fields["LC"] >= 0)
        domain = valid & channel
        pos = domain & (labels > 0)
        buffer = distance_transform_edt(labels == 0, sampling=(meta["cell_dy_m"], meta["cell_dx_m"]))
        neg = domain & (labels == 0) & (buffer > PURGE_M)
        groups, grouping = shared_groups(labels, meta, polygons, grid)
        meta.update(common_domain_pixels=int(domain.sum()), positive_common_pixels=int(pos.sum()),
                background_common_pixels=int(neg.sum()), positive_lost_from_old_domain=int(valid_pos.sum() - pos.sum
                    ()),
                minimum_area_m2=MIN_AREA_M2, group_union=grouping,
                polygon_buffer_definition='distance to rasterized polygon cell centers, not exact vector boundary',
                polygon_buffer_half_cell_diagonal_m=float(np.hypot(meta['cell_dy_m'],meta['cell_dx_m'])/2),
                cv_purge_definition='distance between sampled metric cell centers; >500m',
                grouping_support='rasterized polygon support; subpixel slivers not guaranteed')
        # build_fields is shared with the legacy 02b design and reports its own 54000 m2
        # background threshold. The 02c background is drawn from the common domain above,
        # so the auditable negative threshold is MIN_AREA_M2; keep the old value explicit.
        legacy_negative = meta.get("negative_min_area_m2")
        meta["negative_min_area_m2"] = MIN_AREA_M2
        if legacy_negative is not None:
            meta["legacy_build_fields_negative_min_area_m2"] = legacy_negative
        meta["negative_domain_definition"] = (
            "common valid DEM/land-cover pixels with ground area >= minimum_area_m2, "
            "outside historical polygons by >500 m; legacy 54000 m2 metadata does not apply")
        classes = sorted(int(c) for c in np.unique(fields["LC"][domain]))
        try:
            tab, positive = sample(fields, labels, pos, neg, meta, groups, SEED + k)
            tab.to_csv(io.DATA_OUT / f"02c_{name}_共享组样本.csv", index=False, encoding="utf-8-sig")
            meta.update(n_sampled_positive_groups=int(tab.loc[tab.y==1,"group"].nunique()),
                    n_sampled_background_groups=int(tab.loc[tab.y==0,"group"].nunique()),
                    n_sampled_positive=int(tab.y.sum()), n_sampled_background=int((tab.y==0).sum()))
            if k == 1:
                base = (fields, labels, pos, neg, meta, groups, positive, classes)
            result, oof = evaluate(tab, classes, SEED + k)
            oof.to_csv(io.DATA_OUT / f"02c_{name}_共享组OOF.csv", index=False, encoding="utf-8-sig")
            if k == 1:
                base_tab = oof
        except InsufficientDesign as exc:
            result = {"status": "insufficient", "reason": str(exc), "diagnostics":exc.diagnostics,"fallback": "none"}
        result["sampling_frame"] = meta
        scales[name] = result
        print(name, result["status"], result.get("auc_lr_oof"), flush=True)
    if 'base' in locals():
        fields, labels, pos, neg, meta, groups, positive, classes = base
        for i in range(4):
            try:
                tab, _ = sample(fields, labels, pos, neg, meta, groups, SEED + 500 + i, positive)
                result, _ = evaluate(tab, classes, SEED + 1)
                repeated.append({"repeat": i, "status": "available", "auc_lr_oof": result["auc_lr_oof"],
                                 "auc_iv_oof": result["auc_iv_oof"], "n_shared_groups": result["n_shared_groups"]})
            except InsufficientDesign as exc:
                repeated.append({"repeat": i, "status": "insufficient", "reason": str(exc),"diagnostics":exc.
                    diagnostics})
    old = json.loads((io.DATA_OUT / "02b_空间对照.json").read_text(encoding="utf8"))
    figure = plot_results(scales, old, base_tab)
    legacy.save({"seed": SEED, "scales": scales, "background_redraws": repeated,
             "protocol": {"minimum_area_m2": MIN_AREA_M2, "shared_block_m": BLOCK_M, "purge_m": PURGE_M,
                      "outer_max": 4, "inner_max": 3, "positive_cap_per_polygon": 200, "background_ratio": 2,
                      "bootstrap_replicates": BOOT, "insufficient_action": "report unavailable; never random pixel
                          fallback"},
             "interpretation": "common sampling environmental contrast; unlabelled backgrounds, not occurrence
                 probability or hazard zoning",
             "comparison": "legacy versus new difference jointly changes candidate domain and validation protocol;
                 no single-factor causal attribution",
             "figure": figure}, "02c_共同域空间验证.json")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

`step02_figures.py` 问题二图件

```
"""问题 2 图表：临界条件、触发评估与多尺度可靠性。"""
from __future__ import annotations
```

```

import json
import sys
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

sys.path.insert(0, str(Path(__file__).resolve().parent))
from mmlib import io_data as io, trigger as tg # noqa: E402

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "Noto Sans SC", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["mathtext.fontset"] = "dejavusans"
plt.rcParams.update({
   "font.size": 23,
   "axes.titlesize": 22,
   "axes.labelsize": 20,
   "xtick.labelsize": 16,
   "ytick.labelsize": 16,
   "legend.fontsize": 14,
})

FIG = io.FIG_OUT
OUT = io.DATA_OUT
EVENT_TS = pd.Timestamp("2026-08-26 10:30")


def load(name):
   return json.loads((OUT / name).read_text(encoding="utf-8"))


def fig_thresholds(idr, mtl):
   fig, axes = plt.subplots(2, 2, figsize=(14, 10), constrained_layout=True)

   ax = axes[0, 0]
   dd = np.array([r["D_h"] for r in idr["逐历时统计"]], dtype=float)
   ii = np.array([r["I_p95"] for r in idr["逐历时统计"]], dtype=float)
   band = idr["频率曲线自助带"]
   Dg = np.asarray(band["D_h"])
   a,b = idr["幂律系数a"],idr["幂指数b"]
   ax.fill_between(Dg,band["lo"],band["hi"],alpha=.2,color="#b64b35",label="按年块自助95%带")
   support=idr["频率包络最大支持历时_h"]
   ax.plot(Dg[Dg<=support],a*Dg[Dg<=support]**(-b),color="#b64b35",lw=2,label=f"频率95%包络 b={b:.2f}")
   ax.plot(Dg[Dg>support],a*Dg[Dg>support]**(-b),color="#b64b35",ls="--",label="拟合历时支持外推")
   ax.scatter(dd,ii,c="black",label="各历时过程最大强度p95")
   ax.set_xscale("log");ax.set_yscale("log")
   ax.set_xlabel("连续起止跨度 / 滑动窗历时 D (h)");ax.set_ylabel("强度 I (mm/h)")
   ax.set_title("(a) 频率参考，非力学起动门槛")
   ax.legend(fontsize=14);ax.grid(alpha=.25,which="both")

   ax = axes[0,1]
   ev = pd.DataFrame(idr["事件表"])
   lower = idr["下包络"]
   band = lower["band"]
   Dg = np.asarray(band["D_h"])
   ax.scatter(ev.dur_h,ev.mean_mmh,s=12,alpha=.4,c="#405b69",label="完整训练降雨过程")
   ax.fill_between(Dg,band["lo"],band["hi"],alpha=.2,color="#2f766d",label="按年块自助95%带")
   ax.plot(Dg,lower["alpha"]*Dg**(-lower["beta"]),c="#2f766d",lw=2,
         label=f"低分位10%参考 β={lower['beta']:.2f}")
   ax.set_xscale("log");ax.set_yscale("log")
   ax.set_xlabel("过程起止跨度 D (h，含内部干小时)");ax.set_ylabel("总雨量 / 跨度 (mm/h)")
   ax.set_title("(b) 候选水源筛查，非水源充分条件")
   ax.legend(fontsize=14);ax.grid(alpha=.25,which="both")

   ax = axes[1, 0]
   g = pd.read_parquet(io.CACHE / "gully_table.parquet")
   water = g.loc[g["mechanical_regime"].eq("water_controlled")].copy()
   zero_n = int(g["mechanical_regime"].isin(
      ["friction_equilibrium", "gravity_unstable"]
   ).sum())
   invalid_n = int(g["mechanical_regime"].eq("not_evaluable").sum())
   order = ["小", "中", "大", "特大"]
   data = [water.loc[water["规模"] == k, "Q_crit_m3s"].dropna().values for k in order]
   data = [d for d in data if len(d)]
   lab = [k for k in order if (water["规模"] == k).sum() > 0]
   bp = ax.boxplot(data, tick_labels=lab, patch_artist=True, widths=0.55)
   for p_, c in zip(bp["boxes"], ("#4daf4a", "#ff7f00", "#e41a1c", "#6a3d9a")):
      p_.set_facecolor(c); p_.set_alpha(0.7)
   for i, d in enumerate(data):
      ax.scatter(np.full(len(d), i + 1) + np.random.default_rng(i).normal(0, 0.06, len(d)),
               d, s=16, c="k", alpha=0.6, zorder=3)
   ax.set_yscale("log")
   ax.set_ylabel("起动流量 Q$_c$ (m$^{3}$/s)"); ax.set_xlabel("灾害规模等级")
   ax.set_title("(c) 水流控制分支起动流量")
   ax.grid(alpha=0.3, axis="y")

   ax = axes[1, 1]
   typ = water.groupby("类型").agg(
```

```
        n=("Q_crit_m3s", "size"),
        q=("Q_crit_m3s", "median"),
        s=("比降", "median"),
        a=("面积_km2", "median"),
    ).sort_values("q")
    sc = ax.scatter(typ["s"], typ["q"], s=np.clip(typ["n"] * 16, 40, 400),
                    c=typ["n"], cmap="viridis", edgecolor="k", linewidth=0.8,
                    alpha=0.88, zorder=4)
    for name, r in typ.iterrows():
        ax.annotate(f"{name}\n(n={int(r['n'])}, A={r['a']:.0f} km$^{{2}}$)",
                    (r["s"], r["q"]), fontsize=12,
                    xytext=(8, 8), textcoords="offset points",
                    bbox=dict(boxstyle="round,pad=0.25", fc="white", alpha=0.75, lw=0))
    ax.set_xlabel("沟道比降中位数")
    ax.set_ylabel("起动流量中位数 $Q_c$ / (m$^{3}$/s)")
    ax.set_title("(d) 成因类型与起动条件")
    # 比降与起动流量的跨度都在一个数量级内，用对数轴会产生难读的
    # 2×10⁻¹、3×10⁻¹ 式刻度；改用线性轴并按数据自适应留白。
    xs = typ["s"].to_numpy(dtype=float)
    ys = typ["q"].to_numpy(dtype=float)
    xpad = max((xs.max() - xs.min()) * 0.45, 0.02)
    ypad = max((ys.max() - ys.min()) * 0.28, 40.0)
    ax.set_xlim(xs.min() - xpad, xs.max() + xpad)
    ax.set_ylim(max(ys.min() - ypad, 0), ys.max() + ypad)
    ax.grid(alpha=0.3)
    cb = fig.colorbar(sc, ax=ax, shrink=0.78, pad=0.02)
    cb.set_label("沟道条数", fontsize=12)
    del sc
    p = FIG / "02_临界条件.png"
    fig.savefig(p, dpi=160, bbox_inches="tight")
    plt.close(fig)
    return p


def fig_trigger(tp, rel):
    fig, axes = plt.subplots(2, 2, figsize=(14, 10), constrained_layout=True)
    pt = pd.read_parquet(io.CACHE / "ptrigger.parquet")

    ax = axes[0, 0]
    w = pt.loc["2026-08-23":"2026-08-28"]
    ax.plot(w.index, w["P_trigger"], color="#d62728", lw=1.8, label="事件回顾评分 P")
    ax.plot(w.index, w["T_index"].clip(0, 2) / 2, color="#1f77b4", lw=1.2, ls="--",
            label="触发指数 T/2")
    ax.axhline(0.5, color="gray", ls=":")
    ax.axvline(EVENT_TS, color="k", ls=":", lw=2, label="灾害发生时刻 10:30")
    ax.set_ylabel("评分 / 归一化指数"); ax.set_ylim(0, 1.05)
    ax.set_title("(a) 事件窗口回顾评分（非独立预报）")
    ax.legend(fontsize=14, loc="upper left"); ax.grid(alpha=0.3)
    for lab in ax.get_xticklabels():
        lab.set_rotation(30)

    ax = axes[0, 1]
    p = pt["P_trigger"].dropna()
    ax.hist(p, bins=60, color="#4c72b0", alpha=0.8)
    ax.set_yscale("log")
    ax.axvline(0.5, color="r", ls="--", label="预警阈值 0.5")
    ax.set_xlabel("相对强度评分 P"); ax.set_ylabel("小时数（对数）")
    ax.set_title("(b) 全序列回顾评分分布")
    ax.legend(fontsize=14); ax.grid(alpha=0.3, axis="y")

    ax = axes[1, 0]
    rows = pd.DataFrame(rel["提前量与报警日率"])
    ax.plot(rows["全序列报警日率"] * 100, rows["提前量_h"], "o-", color="#6a3d9a", ms=7)
    for _, r in rows.iterrows():
        if r["提前量_h"] is not None:
            ax.annotate(f'{r["阈值"]:.1f}', (r["全序列报警日率"] * 100, r["提前量_h"]),
                        fontsize=12, xytext=(4, 4), textcoords="offset points")
    if not rows["提前量_h"].notna().any():
        ax.text(
            0.5,
            0.54,
            "无命中阈值：提前量不可估计\n仅能报告完整观测日报警频度",
            transform=ax.transAxes,
            ha="center",
            va="center",
            fontsize=13,
            bbox={"facecolor": "white", "alpha": 0.9, "edgecolor": "#b0b0b0"},
        )
    ax.set_xlabel("完整观测日报警频度 (%)"); ax.set_ylabel("相对事件时刻的提前量 (h)")
    ax.set_title("(c) 阈值—提前量—报警频度（非伪警率）")
    ax.grid(alpha=0.3)

    ax = axes[1, 1]
    sens = rel["空间分辨率敏感性"]
    ks = list(sens.keys())
    slopes = [sens[k]["主沟比降"] for k in ks]
    qcrit = [sens[k]["起动单宽流量_m2s"] for k in ks]
    x = np.arange(len(ks))
    ax.bar(x - 0.2, slopes, 0.4, color="#4c72b0", label="主沟比降")
    ax.set_xticks(x); ax.set_xticklabels([f"{sens[k]['像元_m']:.0f} m" for k in ks])
    ax.set_ylabel("主沟比降", color="#4c72b0")
```

```
    ax.set_xlabel("DEM 分辨率")
    ax2 = ax.twinx()
    ax2.bar(x + 0.2, qcrit, 0.4, color="#dd8452", label="起动单宽流量")
    ax2.set_ylabel("起动单宽流量 (m$^{2}$/s)", color="#dd8452")
    ax.set_title("(d) 空间分辨率敏感性")
    h1, l1 = ax.get_legend_handles_labels(); h2, l2 = ax2.get_legend_handles_labels()
    ax.legend(h1 + h2, l1 + l2, fontsize=14)
    ax.grid(alpha=0.3, axis="y")
    p = FIG / "02_触发评估.png"
    fig.savefig(p, dpi=160, bbox_inches="tight"); plt.close(fig)
    return p


def fig_business():
    screen=load("02_水源业务判据.json")
    skill=load("02_气象基线检验.json")
    tab=pd.read_csv(OUT/"02_逐日候选水源.csv",index_col=0,parse_dates=True)
    test=tab.loc["2026-06-01":"2026-08-27"]
    fig,axes=plt.subplots(2,2,figsize=(13,9),constrained_layout=True)
    ax=axes[0,0]
    ax.plot(test.index,test.PDD3_C_h,label="3日逐小时正积温")
    ax.axhline(screen["PDD3"]["threshold"],ls="--",c="#a34533",label="2021—2025暖季95%分位")
    ax.axvline(EVENT_TS,ls=":",c="black")
    ax.set_ylabel("PDD3 (°C·h)");ax.set_title("(a) 冻结训练阈值的融水候选筛查")
    ax.legend(fontsize=8);ax.tick_params(axis="x",rotation=25)
    ax=axes[0,1]
    ax.plot(test.index,test.rise24_C,label="相邻完整日均温差")
    ax.axhline(screen["rise24"]["threshold"],ls="--",c="#a34533",label="训练95%分位")
    ax.set_ylabel("24 h升温 (°C)");ax.set_title("(b) 日尺度升温参考，缺测不补零")
    ax.legend(fontsize=8);ax.tick_params(axis="x",rotation=25)
    ax=axes[1,0]
    for i,(col,lab) in enumerate((("rain_candidate","降雨"),("melt_candidate","融水"),("Tw","任一通道"))):
        dates=test.index[test[col].eq(1)]
        ax.scatter(dates,np.full(len(dates),i),s=12,label=lab)
    ax.set_yticks([0,1,2],["降雨候选","融水候选","联合候选"])
    ax.axvline(EVENT_TS,ls=":",c="black")
    ax.set_title("(c) 仅统计可观测期报警日率，非真实伪警")
    ax.tick_params(axis="x",rotation=25)
    ax=axes[1,1]
    ax.bar([0,1],[skill["BS_climate"],skill["BS"]],color=["#a9b2af","#32766c"])
    ax.set_xticks([0,1],["冻结气候概率","湿小时持续性混合"])
    ax.set_ylabel("2026有效小时 Brier 分数（小者好）")
    ax.set_title(f"(d) 1小时基线 BSS={skill['BSS']:.3f}；不证明灾害预报技巧")
    fig.savefig(FIG/"02_冻结水源筛查与气象技巧.png",dpi=160)
    plt.close(fig)


def main() -> int:
    idr = load("02_降雨阈值.json")
    mtl = load("02_消融阈值.json")
    tp = load("02_触发评估模型.json")
    rel = load("02_多尺度可靠性.json")
    print(fig_thresholds(idr, mtl))
    print(fig_trigger(tp, rel))
    fig_business()
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

问题三

step03_forecast.py　3000 成员前向预报

```
"""v5.0 forward forecast: IFS melt-onset proxy, interval support, and D8 propagation.

The main product is deliberately narrower than "true debris-flow initiation":

* issue time: 2026-08-26 08:00 UTC+8 (00:00 UTC);
* target 1: the first new hourly support after issue with forecast melt energy,
  T>0 and SW>20, while the completed pre-issue hour is reported separately;
* target 2: the D8 port-proxy arrival obtained from that source support;
* every hourly source value is a half-open support [t,t+1), never a point;
* event-time quantiles use one fixed uniform draw within each member support
  while the complete support envelope is reported separately;
* finite quantiles are conditional on finite members;
* active-at-issue, no-event, not-evaluable, right-censored and no-arrival are
  distinct states.
"""
from __future__ import annotations

import hashlib
import json
import math
from pathlib import Path

```

```
import numpy as np
import pandas as pd

from mmlib import io_data as io

ROOT = Path(__file__).resolve().parents[2]
OUT = io.DATA_OUT
EXTERNAL = ROOT / "临时产物" / "外部数据"
ISSUE_UTC = pd.Timestamp("2026-08-26 00:00")
ISSUE_LOCAL = pd.Timestamp("2026-08-26 08:00")
PREISSUE_UTC = ISSUE_UTC - pd.Timedelta(hours=1)
WINDOW_END_UTC = ISSUE_UTC + pd.Timedelta(hours=24)
WINDOW_END_LOCAL = ISSUE_LOCAL + pd.Timedelta(hours=24)
HIT_LOCAL = pd.Timestamp("2026-08-26 10:30")
HIT_UTC = HIT_LOCAL - pd.Timedelta(hours=8)
N_PER_RUN = 1000
SEED = 20260826
MIN_FINITE = 100
SOURCE_STATES = (
    "finite",
    "no_event",
    "active_at_issue",
    "right_censored",
    "not_evaluable",
)
ARRIVAL_STATES = (
    "finite",
    "no_event",
    "active_at_issue_left_censored",
    "no_arrival",
    "right_censored",
    "not_evaluable",
)


def clean(x):
    if isinstance(x, dict):
        return {str(k): clean(v) for k, v in x.items()}
    if isinstance(x, (list, tuple, np.ndarray)):
        return [clean(v) for v in x]
    if isinstance(x, (bool, np.bool_)):
        return bool(x)
    if isinstance(x, (int, np.integer)):
        return int(x)
    if isinstance(x, (float, np.floating)):
        return float(x) if np.isfinite(x) else None
    if isinstance(x, (pd.Timestamp, np.datetime64)):
        return str(pd.Timestamp(x))
    return x


def write_json(name: str, value) -> None:
    (OUT / name).write_text(
        json.dumps(clean(value), ensure_ascii=False, indent=2, allow_nan=False) + "\n",
        encoding="utf-8",
    )


def iso_utc(value) -> str:
    stamp = pd.Timestamp(value)
    stamp = stamp.tz_localize("UTC") if stamp.tzinfo is None else stamp.tz_convert("UTC")
    return stamp.isoformat()


def iso_local(value) -> str:
    stamp = pd.Timestamp(value)
    stamp = stamp.tz_localize("UTC") if stamp.tzinfo is None else stamp.tz_convert("UTC")
    return stamp.tz_convert("Asia/Shanghai").isoformat()


def sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def load_json(name_or_path):
    path = Path(name_or_path)
    if not path.is_absolute():
        path = OUT / name_or_path
    return json.loads(path.read_text(encoding="utf-8"))


def load_ifs_manifest() -> dict:
    path = EXTERNAL / "ifs_single_run_manifest.json"
    manifest = json.loads(path.read_text(encoding="utf-8"))
    if manifest.get("actual_file_count") != 4:
        raise AssertionError("IFS manifest must describe four actual files")
    source_files = []
    port_files = []
    for item in manifest["files"]:
        raw = EXTERNAL / item["file"]
        if not raw.exists():
            raise FileNotFoundError(raw)
        actual = sha256(raw)
        if actual != item["sha256"]:
            raise AssertionError(f"IFS hash mismatch: {raw.name}")
        if item.get("role") == "source_primary":
            init = pd.Timestamp(item["init_time_utc"])
```

```
         if init + pd.Timedelta(hours=6) > ISSUE_UTC:
            raise AssertionError(f"IFS source run is unavailable at issue: {raw.name}")
         source_files.append(item)
      elif item.get("role") == "independent_port_context":
         port_files.append(item)
   if len(source_files) != 3 or len(port_files) != 1:
      raise AssertionError((len(source_files), len(port_files)))
   return {"path": path, "sha256": sha256(path), "manifest": manifest,
         "source_files": source_files, "port_files": port_files}


def align_ifs_hourly(raw: pd.DataFrame) -> pd.DataFrame:
   """Build [t,t+1h) supports from end-labelled fluxes and point states."""
   if raw.empty or not isinstance(raw.index, pd.DatetimeIndex):
      raise ValueError("IFS requires a nonempty datetime index")
   if raw.index.hasnans or not raw.index.is_unique or not raw.index.is_monotonic_increasing:
      raise ValueError("IFS times must be valid, sorted and unique")
   if not (raw.index == raw.index.floor("h")).all():
      raise ValueError("IFS labels must be exact hourly endpoints")
   # Reindex first: the next row must be t+1h, never a later surviving record.
   grid = pd.date_range(raw.index.min(), raw.index.max(), freq="h")
   raw = raw.reindex(grid).apply(pd.to_numeric, errors="coerce")
   raw = raw.where(np.isfinite(raw))
   end = raw.shift(-1)
   frame = pd.DataFrame({
      "T": (raw["T"] + end["T"]) / 2.0,
      "RH": (raw["RH"] + end["RH"]) / 2.0,
      "SW": end["SW"].where(end["SW"] >= 0.0),
      "P": end["P"].where(end["P"] >= 0.0),
   }, index=grid).iloc[:-1].copy()
   frame.attrs["time_support"] = {
      "model_support": "[t,t+1h)",
      "SW": "raw label t+1h; preceding-hour mean W/m2",
      "P": "raw label t+1h; preceding-hour accumulation mm",
      "T": "trapezoidal mean of instantaneous raw t and t+1h degC",
      "RH": "trapezoidal mean of instantaneous raw t and t+1h percent",
      "nonlinearity_order": "hourly average first, then positive part and gate",
      "missing_endpoint": "missing support; never a one-endpoint mean",
   }
   return frame


def load_if_hourly(item: dict) -> pd.DataFrame:
   raw = json.loads((EXTERNAL / item["file"]).read_text(encoding="utf-8"))
   h = raw["hourly"]
   idx = pd.to_datetime(h["time"], errors="coerce")
   frame = pd.DataFrame({
      "T": pd.to_numeric(h["temperature_2m"], errors="coerce"),
      "P": pd.to_numeric(h["precipitation"], errors="coerce"),
      "SW": pd.to_numeric(h["shortwave_radiation"], errors="coerce"),
      "RH": pd.to_numeric(h["relative_humidity_2m"], errors="coerce"),
   }, index=idx)
   return align_ifs_hourly(frame)


def historical_energy_support(features: pd.DataFrame, met="point",
                      rad_end=False, soil_end=False, flux_end=False) -> pd.DataFrame:
   """Rebuild both inputs and energy target for an explicit station-clock scenario."""
   if met not in {"point", "start_mean", "end_mean"}:
      raise ValueError(f"unknown MET support: {met}")
   if not features.index.is_unique or not features.index.is_monotonic_increasing:
      raise ValueError("station feature times must be sorted and unique")
   frame = features.reindex(pd.date_range(features.index.min(), features.index.max(), freq="h"))
   met_values = frame[["Ta_2m", "RH_2m"]]
   if met == "point":
      met_values = (met_values + met_values.shift(-1)) / 2.0
   elif met == "end_mean":
      met_values = met_values.shift(-1)
   rad = frame[["Rsd", "Rsu", "Rld", "Rlu"]].shift(-int(rad_end))
   soil = frame["G_0.1m"].shift(-int(soil_end))
   flux = frame[["H", "LE"]].shift(-int(flux_end))
   energy = (rad["Rsd"] - rad["Rsu"] + rad["Rld"] - rad["Rlu"]
           - soil - flux["H"] - flux["LE"]).clip(lower=0)
   result = met_values.copy()
   result["Rsd"] = rad["Rsd"]
   result["melt_energy"] = energy
   result = result.where(np.isfinite(result))
   result.attrs["station_support_assumption"] = {
      "MET": met, "RADM": "end_mean" if rad_end else "start_mean",
      "SOIL": "end_mean" if soil_end else "start_mean",
      "FLUX": "end_mean" if flux_end else "start_mean",
      "metadata_verified": False,
      "target_rebuilt_after_alignment": True,
   }
   return result


def mean_energy_proxy(
   beta: np.ndarray,
   t: float,
   sw: float,
   rh: float,
) -> float:
   """Mean-coefficient fitted energy proxy used for the pre-issue state."""
   if not (np.isfinite(t) and np.isfinite(sw) and np.isfinite(rh)):
      return float("nan")
   return float(
```

```
        beta[0]
        + beta[1] * max(t, 0.0)
        + beta[2] * max(sw, 0.0)
        + beta[3] * rh
    )


def estimate_ar1_residual(
    residual: np.ndarray,
    index: pd.DatetimeIndex,
) -> dict:
    """Estimate AR(1) only from consecutive observed hourly residuals."""
    residual = np.asarray(residual, dtype=float)
    index = pd.DatetimeIndex(index)
    if residual.shape != (len(index),):
        raise ValueError("residual and index lengths must match")
    valid = np.isfinite(residual)
    residual = residual[valid]
    times = index[valid]
    if len(times) != len(residual) or not times.is_monotonic_increasing:
        raise ValueError("residual time index must be sorted")
    adjacent = (
        times[1:].to_numpy() - times[:-1].to_numpy()
    ) == np.timedelta64(1, "h")
    previous = residual[:-1][adjacent]
    following = residual[1:][adjacent]
    if len(previous) < 2:
        raise ValueError("insufficient adjacent hourly pairs for AR(1)")
    phi = float(np.corrcoef(following, previous)[0, 1])
    if not np.isfinite(phi):
        phi = 0.0
    phi = float(np.clip(phi, -0.99, 0.99))
    marginal_sigma = float(np.std(residual, ddof=1))
    innovation = following - phi * previous
    sigma = float(np.std(innovation, ddof=1))
    if not np.isfinite(sigma) or sigma <= 0 or not np.isfinite(marginal_sigma):
        raise ValueError("invalid residual scale")
    return {
        "phi": phi,
        "innovation_sigma": sigma,
        "marginal_sigma": marginal_sigma,
        "adjacent_pair_n": int(len(previous)),
    }


def fit_energy_model(features: pd.DataFrame) -> dict:
    """Fit the declared historical W/m2 energy proxy and AR(1) residual scale."""
    train = features.loc[
        features.index.year.isin(range(2021, 2026))
        & features.index.month.isin([6, 7, 8, 9]),
        ["melt_energy", "Ta_2m", "Rsd", "RH_2m"],
    ].copy()
    train = train.apply(pd.to_numeric, errors="coerce").dropna()
    if len(train) < 500:
        raise ValueError("insufficient historical energy training rows")
    gn = io.load_glacier_nc()
    valid_ice = np.isfinite(gn["thickness"]) & (gn["thickness"] > 1.0)
    source_elevation = float(np.nanmean(gn["dem"][valid_ice]))
    station_elevation = 3980.0
    lapse = 0.006
    offset = -(source_elevation - station_elevation) * lapse
    t_src = train["Ta_2m"] + offset
    X = np.column_stack([
        np.ones(len(train)),
        np.maximum(t_src.to_numpy(), 0.0),
        np.maximum(train["Rsd"].to_numpy(), 0.0),
        train["RH_2m"].to_numpy(),
    ])
    y = train["melt_energy"].to_numpy(dtype=float)
    beta, *_ = np.linalg.lstsq(X, y, rcond=None)
    residual = y - X @ beta
    valid_residual = np.isfinite(residual)
    residual = residual[valid_residual]
    ar1 = estimate_ar1_residual(residual, train.index[valid_residual])
    phi = ar1["phi"]
    sigma = ar1["innovation_sigma"]
    marginal_sigma = ar1["marginal_sigma"]

    # Year-block coefficient resampling is an explicit model-design uncertainty.
    rng = np.random.default_rng(SEED)
    year_arrays = {}
    for year, sub in train.groupby(train.index.year):
        ts = sub["Ta_2m"] + offset
        x = np.column_stack([
            np.ones(len(sub)),
            np.maximum(ts.to_numpy(), 0.0),
            np.maximum(sub["Rsd"].to_numpy(), 0.0),
            sub["RH_2m"].to_numpy(),
        ])
        year_arrays[int(year)] = (x, sub["melt_energy"].to_numpy(dtype=float))
    years = sorted(year_arrays)
    beta_draws = []
    for _ in range(N_PER_RUN):
        picked = rng.choice(years, size=len(years), replace=True)
        xb = np.vstack([year_arrays[int(y)][0] for y in picked])
        yb = np.concatenate([year_arrays[int(y)][1] for y in picked])
        b, *_ = np.linalg.lstsq(xb, yb, rcond=None)
```

```
        beta_draws.append(b if np.all(np.isfinite(b)) else beta)
    beta_draws = np.asarray(beta_draws)
    return {
        "status": "fitted",
        "training_rows": int(len(train)),
        "training_years": years,
        "training_months": [6, 7, 8, 9],
        "station_support_assumption": features.attrs.get("station_support_assumption"),
        "source_elevation_m": source_elevation,
        "station_elevation_m": station_elevation,
        "temperature_offset_C": offset,
        "beta": beta.tolist(),
        "coef_names": ["intercept", "positive_source_temperature", "positive_shortwave", "relative_humidity"],
        "residual_sigma_W_m2": sigma,
        "residual_marginal_sigma_W_m2": marginal_sigma,
        "residual_ar1_phi": phi,
        "residual_adjacent_pair_n": ar1["adjacent_pair_n"],
        "residual_first_step": "zero-mean stationary AR(1) state with variance sigma^2/(1-phi^2)",
        "residual_innovation": "N(0, sigma^2), including the first innovation",
        "residual_mean_W_m2": float(np.mean(residual)),
        "residual_quantiles_W_m2": np.quantile(residual, [0.025, 0.5, 0.975]).tolist(),
        "coefficient_resampling": "5-year block bootstrap with replacement; 1000 draws, seed fixed",
        "coefficient_draws_n": int(len(beta_draws)),
        "beta_draws": beta_draws,
    }


def forecast_trace(item: dict) -> tuple[pd.DataFrame, dict]:
    frame = load_if_hourly(item)
    future_grid = pd.date_range(
        ISSUE_UTC, WINDOW_END_UTC - pd.Timedelta(hours=1), freq="h"
    )
    grid = pd.DatetimeIndex([PREISSUE_UTC, *future_grid])
    trace = frame.reindex(grid)
    available = int(trace[["T", "SW", "RH"]].notna().all(axis=1).sum())
    future_available = int(
        trace.reindex(future_grid)[["T", "SW", "RH"]].notna().all(axis=1).sum()
    )
    preissue_available = bool(
        trace.loc[[PREISSUE_UTC], ["T", "SW", "RH"]].notna().all(axis=1).iloc[0]
    )
    trajectory_end = pd.Timestamp(frame.index.max())
    trace.attrs.update({
        "trajectory_end_utc": trajectory_end,
        "preissue_hour_utc": PREISSUE_UTC,
    })
    return trace, {
        "file": item["file"],
        "sha256": item["sha256"],
        "init_time_utc": item["init_time_utc"],
        "available_at_utc": item["availability"]["available_at_utc"],
        "preissue_hour_utc": iso_utc(PREISSUE_UTC),
        "preissue_hour_available": preissue_available,
        "valid_available_hours_including_preissue": available,
        "required_hours_including_preissue": len(grid),
        "valid_future_hours": future_available,
        "required_future_hours": len(future_grid),
        "trajectory_end_utc": iso_utc(trajectory_end),
        "trajectory_end_semantics": "last complete model-support start; raw final endpoint minus one hour",
        "time_support": frame.attrs["time_support"],
        "all_future_hours_available": future_available == len(future_grid),
    }


def ar1_residual(rng, sigma: float, phi: float, n: int) -> np.ndarray:
    """Simulate the declared stationary AR(1) residual process."""
    out = np.empty(n, dtype=float)
    stationary_sigma = float(sigma) / max(math.sqrt(max(1.0 - float(phi) ** 2, 1e-12)), 1e-12)
    out[0] = float(rng.normal(0.0, stationary_sigma))
    for j in range(1, n):
        out[j] = float(phi) * out[j - 1] + float(sigma) * float(rng.normal())
    return out


def make_members(trace: pd.DataFrame, beta_draws: np.ndarray, model: dict,
                 run_tag: str) -> tuple[list[dict], np.ndarray]:
    # run_tag is "run06"/"run12"/"run18"; use the lead-hour suffix as a
    # deterministic stream offset while preserving the global seed.
    tag_suffix = int(str(run_tag).removeprefix("run"))
    rng = np.random.default_rng(SEED + tag_suffix)
    event_rng = np.random.default_rng(SEED + 10000 + tag_suffix)
    expected_index = pd.DatetimeIndex([
        PREISSUE_UTC,
        *pd.date_range(
            ISSUE_UTC, WINDOW_END_UTC - pd.Timedelta(hours=1), freq="h"
        ),
    ])
    if not trace.index.equals(expected_index):
        raise AssertionError(
            "forecast trace must contain the completed pre-issue hour and "
            "exactly 24 future hourly supports"
        )
    t = trace["T"].to_numpy(dtype=float)
    sw = trace["SW"].to_numpy(dtype=float)
    rh = trace["RH"].to_numpy(dtype=float)
    future_index = trace.index[1:]
    valid_input = np.isfinite(t) & np.isfinite(sw) & np.isfinite(rh)
```

```
    mean_beta = np.asarray(model.get("beta", np.mean(beta_draws, axis=0)), dtype=float)
    preissue_energy = mean_energy_proxy(
        mean_beta, float(t[0]), float(sw[0]), float(rh[0])
    )
    preissue_active = bool(
        valid_input[0]
        and t[0] > 0.0
        and sw[0] > 20.0
        and np.isfinite(preissue_energy)
        and preissue_energy > 0.0
    )
    trajectory_end = pd.Timestamp(
        trace.attrs.get("trajectory_end_utc", future_index[-1])
    )
    after_trajectory = future_index > trajectory_end
    records = []
    energy_members = []
    for i in range(N_PER_RUN):
        beta = beta_draws[i]
        eps = ar1_residual(
            rng,
            model["residual_sigma_W_m2"],
            model["residual_ar1_phi"],
            len(t),
        )
        raw = beta[0] + beta[1] * np.maximum(t, 0.0) + beta[2] * np.maximum(sw, 0.0) + beta[3] * rh + eps
        protected = np.where((t > 0.0) & (sw > 20.0), np.maximum(raw, 0.0), 0.0)
        protected = np.where(valid_input, protected, np.nan)
        future_protected = protected[1:]
        future_t = t[1:]
        future_sw = sw[1:]
        crossing = np.where(
            np.isfinite(future_protected)
            & (future_protected > 0.0)
            & (future_t > 0.0)
            & (future_sw > 20.0)
        )[0]
        missing_future = np.where(~np.isfinite(future_protected))[0]
        first_crossing = int(crossing[0]) if len(crossing) else None
        first_invalid = int(missing_future[0]) if len(missing_future) else None
        preissue_missing = not valid_input[0]
        if preissue_missing:
            state = "not_evaluable"
            state_reason = "preissue_input_missing"
            start = end = None
        elif preissue_active:
            state = "active_at_issue"
            state_reason = "completed_preissue_hour_already_active"
            start = end = None
        elif (
            first_crossing is None
            and missing_future.size
            and bool(after_trajectory[missing_future].all())
        ):
            state = "right_censored"
            state_reason = "forecast_trajectory_ends_before_window"
            start = end = None
        elif first_invalid is not None and (
            first_crossing is None or first_invalid < first_crossing
        ):
            # A missing input before the first crossing prevents a unique
            # first-event time; later observations cannot recover that hour.
            state = "not_evaluable"
            state_reason = "future_input_missing_before_first_crossing"
            start = end = None
        elif first_crossing is not None:
            j = int(crossing[0])
            state = "finite"
            state_reason = "future_first_crossing"
            start = ISSUE_UTC + pd.Timedelta(hours=j)
            end = start + pd.Timedelta(hours=1)
        elif missing_future.size:
            # If every unavailable future hour lies beyond the returned IFS
            # trajectory, the outcome is censored rather than no-event.
            state = "not_evaluable"
            state_reason = "future_input_missing_without_first_crossing"
            start = end = None
        else:
            state = "no_event"
            state_reason = "complete_future_trajectory_without_crossing"
            start = end = None
        event_time = None
        if state == "finite":
            # One representative point is sampled only for reporting event-time
            # quantiles. The complete [start,end) support remains the primary
            # object and is propagated unchanged.
            offset_s = 3600.0 * float(event_rng.random())
            event_time = start + pd.Timedelta(seconds=offset_s)
        records.append({
            "member_id": f"{run_tag}-{i:04d}",
            "run": run_tag,
            "state_tm": state,
            "state_tm_reason": state_reason,
            "tm_lower_utc": start,
            "tm_upper_utc": end,
            "tm_event_utc": event_time,
            "first_hour_index": int(crossing[0]) if len(crossing) else None,
```

```
        "preissue_T_C": float(t[0]) if np.isfinite(t[0]) else None,
        "preissue_SW_W_m2": float(sw[0]) if np.isfinite(sw[0]) else None,
        "preissue_mean_energy_W_m2": (
            float(preissue_energy) if np.isfinite(preissue_energy) else None
        ),
        "future_missing_hours": int(missing_future.size),
        "trajectory_end_utc": trajectory_end,
    })
    energy_members.append(protected)
  return records, np.asarray(energy_members)


def parse_geometry() -> dict:
  vol = load_json("04_初始体积.json")
  cl = load_json("04_中心线校验.json")
  g = vol["链A源几何"]
  sec = vol["共同断面"]
  lon1, lat1 = g["centroid_lonlat"]
  lon2, lat2 = sec["注入_lonlat"]
  horiz = float(np.hypot((lon1 - lon2) * 111320 * math.cos(math.radians(lat1)),
                         (lat1 - lat2) * 110574))
  options = []
  for tag, z in [
    ("DEM4_surface", g["z_dem_p90_m"]),
    ("DEM4_bed_plus_ice", g["z_dem_plus_ice_p90_m"]),
  ]:
    drop = float(z - sec["注入高程_raw_m"])
    if drop <= 0 or horiz <= 0:
      options.append({"name": tag, "valid": False, "reason": "nonpositive drop or distance"})
      continue
    length = float(np.hypot(drop, horiz))
    fall_s = float(np.sqrt(2.0 * length / (9.80665 * drop / length)))
    options.append({
      "name": tag,
      "valid": True,
      "source_centroid": [lon1, lat1],
      "injection": [lon2, lat2],
      "source_to_injection_horizontal_m": horiz,
      "source_to_injection_route_m": length,
      "drop_m": drop,
      "fall_s": fall_s,
    })
  port_lon, port_lat = sec["口岸_lonlat"]
  straight = float(np.hypot((lon1 - port_lon) * 111320 * math.cos(math.radians(lat1)),
                            (lat1 - port_lat) * 110574))
  route = float(cl["中心线长度_km"]) * 1000.0
  return {
    "source_centroid": [lon1, lat1],
    "injection": [lon2, lat2],
    "port_proxy": [port_lon, port_lat],
    "source_to_injection_horizontal_m": horiz,
    "centerline_injection_to_port_m": route,
    "combined_route_m": horiz + route,
    "source_to_port_straight_m": straight,
    "geometry_options": options,
    "source_semantics": "链A源区质心；不证明真实释放面或冰体失稳时刻",
    "port_semantics": "D8口岸代理断面；与题面地点存在模型吸附误差",
  }


def d8_configs() -> list[dict]:
  q4b = load_json("04b_二维引擎.json")
  rows = []
  for name, result in q4b["runs"].items():
    termination = str(result.get("termination_reason"))
    flags = terminal_flags(
      termination,
      result.get("domain_censor", result.get("runout_right_censored", False)),
      reached_port=bool(result.get("reached_port", False)),
    )
    rows.append({
      "id": name,
      "engine": "D8",
      "arrival_port_s": result.get("arrival_port_s"),
      "reached_port": bool(result.get("reached_port", False)),
      "termination_reason": termination,
      "final_runout_m": result.get("runout_L_m"),
      "params": result.get("params", {}),
      "volume_m3": result.get("V_df_m3"),
      "runout_right_censored": flags["right_censored"],
      "natural_stop": flags["natural_stop"],
      "time_limit": flags["time_limit"],
      "step_limit": flags["step_limit"],
      "domain_censor": flags["domain_censor"],
      "time_limit_nonarrival": flags["time_limit_nonarrival"],
      "step_limit_nonarrival": flags["step_limit_nonarrival"],
      "domain_censor_nonarrival": flags["domain_censor_nonarrival"],
      "weight": 1.0 / len(q4b["runs"]),
    })
  if len(rows) != 16:
    raise AssertionError(f"expected 16 unique D8 configurations, got {len(rows)}")
  signatures = {
    json.dumps(
      {"params": row["params"], "volume_m3": row["volume_m3"]},
      ensure_ascii=False,
```

```
            sort_keys=True,
        )
        for row in rows
    }
    if len(signatures) != 16:
        raise AssertionError(f"expected 16 distinct parameter sets, got {len(signatures)}")
    return rows


TERMINATION_REASONS = {
    "stationary_after_injection",
    "time_limit",
    "step_limit",
    "domain_censor",
}


def terminal_flags(
    termination_reason,
    domain_censor: bool = False,
    *,
    reached_port: bool = False,
) -> dict:
    """Map the explicit D8 terminal enum to separate, non-interchangeable flags."""
    reason = str(termination_reason)
    if reason not in TERMINATION_REASONS:
        raise AssertionError(f"unknown D8 termination reason: {reason!r}")
    domain_censor = bool(domain_censor or reason == "domain_censor")
    natural_stop = reason == "stationary_after_injection" and not domain_censor
    time_limit = reason == "time_limit"
    step_limit = reason == "step_limit"
    runout_censored = bool(domain_censor or time_limit or step_limit)
    arrival_censored = bool(not reached_port and runout_censored)
    return {
        "termination_reason": reason,
        "natural_stop": natural_stop,
        "time_limit": time_limit,
        "step_limit": step_limit,
        "domain_censor": domain_censor,
        "right_censored": runout_censored,
        "arrival_right_censored": arrival_censored,
        "time_limit_nonarrival": bool(not reached_port and time_limit),
        "step_limit_nonarrival": bool(not reached_port and step_limit),
        "domain_censor_nonarrival": bool(not reached_port and domain_censor),
    }


def build_legacy_audit(old: dict, source_relative: str, source_sha: str) -> dict:
    """Build a standalone, explicitly non-operative legacy inverse record."""
    return {
        "schema_version": 1,
        "status": "legacy_audit_only_not_used",
        "source_audit": {
            "path": source_relative,
            "sha256": source_sha,
        },
        "legacy_record": {
            "source_file": old.get("source_file"),
            "source_file_sha256": old.get("source_sha256"),
            "physical_source_inverse": old.get("physical_source_inverse"),
            "scope": old.get("scope"),
        },
        "downstream_use": {
            "forward_forecast": False,
            "coverage": False,
            "warning": False,
            "propagation": False,
        },
        "scope": (
            "v4.3 post-event conditional inverse retained only as an audit "
            "record; not an input to v5.0 prediction, coverage, warning or propagation"
        ),
    }


def runout_terminal_fields(config: dict) -> dict:
    """Read explicit terminal meanings from the D8 configuration record."""
    flags = terminal_flags(
        config.get("termination_reason"),
        config.get("domain_censor", config.get("runout_right_censored", False)),
        reached_port=bool(config.get("reached_port", False)),
    )
    return {
        key: bool(config.get(key, flags[key]))
        for key in (
            "natural_stop",
            "time_limit",
            "step_limit",
            "domain_censor",
            "right_censored",
            "time_limit_nonarrival",
            "step_limit_nonarrival",
            "domain_censor_nonarrival",
        )
    }
```

```python
def propagate(records: list[dict], configs: list[dict], geometry: dict) -> list[dict]:
    rng = np.random.default_rng(SEED + 17)
    out = []
    for i, record in enumerate(records):
        config = configs[int(rng.integers(0, len(configs)))]
        geo = geometry["geometry_options"][i % 2]
        item = dict(record)
        item["d8_configuration_id"] = config["id"]
        item["d8_design_weight"] = config["weight"]
        item["source_geometry"] = geo.get("name")
        item["source_fall_s"] = geo.get("fall_s")
        item["d8_travel_s"] = config.get("arrival_port_s")
        item["ta_censor_upper_utc"] = None
        if record["state_tm"] in {"no_event", "right_censored", "not_evaluable"}:
            item["state_ta"] = record["state_tm"]
            item["ta_lower_utc"] = None
            item["ta_upper_utc"] = None
        elif not geo.get("valid"):
            item["state_ta"] = "not_evaluable"
            item["ta_lower_utc"] = None
            item["ta_upper_utc"] = None
        elif not config["reached_port"]:
            terminal = runout_terminal_fields(config)
            item["state_ta"] = (
                "right_censored" if terminal["right_censored"] else "no_arrival"
            )
            item["ta_lower_utc"] = None
            item["ta_upper_utc"] = None
        elif config["arrival_port_s"] is None:
            item["state_ta"] = "no_arrival"
            item["ta_lower_utc"] = None
            item["ta_upper_utc"] = None
        elif record["state_tm"] == "active_at_issue":
            run = float(config["arrival_port_s"])
            upper = ISSUE_UTC + pd.Timedelta(seconds=geo["fall_s"] + run)
            item["state_ta"] = "active_at_issue_left_censored"
            item["ta_lower_utc"] = None
            item["ta_upper_utc"] = None
            item["ta_censor_upper_utc"] = upper
        elif record["state_tm"] == "finite":
            run = float(config["arrival_port_s"])
            item["state_ta"] = "finite"
            item["ta_lower_utc"] = record["tm_lower_utc"] + pd.Timedelta(seconds=geo["fall_s"] + run)
            item["ta_upper_utc"] = record["tm_upper_utc"] + pd.Timedelta(seconds=geo["fall_s"] + run)
            item["ta_event_utc"] = record["tm_event_utc"] + pd.Timedelta(seconds=geo["fall_s"] + run)
        else:
            raise AssertionError(f"unsupported source state: {record['state_tm']}")
        out.append(item)
    return out


def state_summary(records: list[dict], state_key: str) -> dict:
    if state_key == "state_tm":
        states = SOURCE_STATES
    elif state_key == "state_ta":
        states = ARRIVAL_STATES
    else:
        raise AssertionError(f"unknown state key: {state_key}")
    counts = {s: 0 for s in states}
    for record in records:
        state = record[state_key]
        if state not in counts:
            raise AssertionError(f"unknown state: {state}")
        counts[state] += 1
    total = len(records)
    if sum(counts.values()) != total:
        raise AssertionError((state_key, counts, total))
    return {
        "total_members": total,
        "counts": counts,
        "p_finite": counts["finite"] / total,
        "p_no_event": counts["no_event"] / total,
        "p_right_censored": counts["right_censored"] / total,
        "p_not_evaluable": counts["not_evaluable"] / total,
        **(
            {"p_active_at_issue": counts["active_at_issue"] / total}
            if state_key == "state_tm"
            else {
                "p_active_at_issue_left_censored": (
                    counts["active_at_issue_left_censored"] / total
                ),
                "p_no_arrival": counts["no_arrival"] / total,
            }
        ),
        "p_actionable": (
            (counts.get("finite", 0) + counts.get("active_at_issue", 0)) / total
            if state_key == "state_tm"
            else (
                counts.get("finite", 0)
                + counts.get("active_at_issue_left_censored", 0)
            ) / total
        ),
        "state_sum_check": sum(counts.values()) == total,
    }


def should_alarm(summary: dict) -> bool:
```

```
    """Apply the frozen source-risk rule using explicit actionable mass."""
    return bool(summary.get("p_actionable", 0.0) >= 0.5)


def interval_lead_bounds(interval: dict, target_time) -> tuple[float | None, float | None]:
    """Return [target - upper, target - lower] in hours for the arrival interval."""
    if interval.get("status") != "available":
        return None, None
    lower = pd.Timestamp(interval["lower_time"])
    upper = pd.Timestamp(interval["upper_time"])
    target = pd.Timestamp(target_time)
    if lower.tzinfo is None:
        lower = lower.tz_localize("UTC")
    if upper.tzinfo is None:
        upper = upper.tz_localize("UTC")
    if target.tzinfo is None:
        target = target.tz_localize("UTC")
    lower = lower.tz_convert("UTC")
    upper = upper.tz_convert("UTC")
    target = target.tz_convert("UTC")
    return (
        float((target - upper).total_seconds() / 3600.0),
        float((target - lower).total_seconds() / 3600.0),
    )


def finite_interval(records: list[dict], lower_key: str, upper_key: str,
                    event_key: str, state_key: str | None = None) -> dict:
    finite = [r for r in records if r.get("state_tm" if state_key is None else state_key) == "finite"]
    if len(finite) == 0:
        return {
            "status": "not_constructible_zero_finite",
            "finite_n": 0,
            "lower_time": None,
            "median_time": None,
            "upper_time": None,
            "width_hours": None,
            "support_envelope_lower_time": None,
            "support_envelope_upper_time": None,
            "support_envelope_width_hours": None,
            "support_endpoint_95_lower_time": None,
            "support_endpoint_95_upper_time": None,
            "support_endpoint_95_width_hours": None,
            "interval_semantics": (
                "no finite member exists; conditional interval, support "
                "envelope and endpoint diagnostics are not constructed"
            ),
        }
    if len(finite) < MIN_FINITE:
        return {
            "status": "insufficient_finite_members",
            "finite_n": len(finite),
            "lower_time": None,
            "median_time": None,
            "upper_time": None,
            "width_hours": None,
            "support_envelope_lower_time": None,
            "support_envelope_upper_time": None,
            "support_envelope_width_hours": None,
            "support_endpoint_95_lower_time": None,
            "support_endpoint_95_upper_time": None,
            "support_endpoint_95_width_hours": None,
            "interval_semantics": (
                f"only {len(finite)} finite members (<{MIN_FINITE}); conditional "
                "interval, support envelope and endpoint diagnostics are not constructed"
            ),
        }
    lo = pd.to_datetime([r[lower_key] for r in finite])
    hi = pd.to_datetime([r[upper_key] for r in finite])
    event = pd.to_datetime([r[event_key] for r in finite])
    if (
        lo.isna().any()
        or hi.isna().any()
        or event.isna().any()
    ):
        raise AssertionError("finite members must carry non-null supports and events")
    lo_ns = lo.astype("int64").to_numpy()
    hi_ns = hi.astype("int64").to_numpy()
    event_ns = event.astype("int64").to_numpy()
    lower_q = np.quantile(event_ns, 0.025)
    upper_q = np.quantile(event_ns, 0.975)
    median_q = np.quantile(event_ns, 0.5)
    endpoint_q_lower = np.quantile(lo_ns, 0.025)
    endpoint_q_upper = np.quantile(hi_ns, 0.975)
    complete_envelope_lower = int(np.min(lo_ns))
    complete_envelope_upper = int(np.max(hi_ns))
    return {
        "status": "available",
        "finite_n": len(finite),
        "lower_time": str(pd.Timestamp(int(lower_q), tz="UTC")),
        "median_time": str(pd.Timestamp(int(median_q), tz="UTC")),
        "upper_time": str(pd.Timestamp(int(upper_q), tz="UTC")),
        "width_hours": (upper_q - lower_q) / 3.6e12,
        "support_envelope_lower_time": str(pd.Timestamp(complete_envelope_lower, tz="UTC")),
        "support_envelope_upper_time": str(pd.Timestamp(complete_envelope_upper, tz="UTC")),
        "support_envelope_width_hours": (complete_envelope_upper - complete_envelope_lower) / 3.6e12,
        "support_endpoint_95_lower_time": str(pd.Timestamp(int(endpoint_q_lower), tz="UTC")),
```

```
        "support_endpoint_95_upper_time": str(pd.Timestamp(int(endpoint_q_upper), tz="UTC")),
        "support_endpoint_95_width_hours": (endpoint_q_upper - endpoint_q_lower) / 3.6e12,
        "interval_semantics": (
            "2.5% and 97.5% quantiles of one uniformly sampled event time within "
            "each paired half-open hourly support; conditional on finite members. "
            "The complete finite-member support envelope [min(L),max(U)] and the "
            "separately trimmed endpoint-quantile envelope are reported without "
            "being substituted for the event-time interval. The convex support "
            "envelope may contain gaps and is not the exact union of member supports."
        ),
    }


def main() -> int:
    features = pd.read_parquet(io.CACHE / "features.parquet")
    ifs = load_ifs_manifest()
    source = sorted(ifs["source_files"], key=lambda item: item["init_time_utc"])
    traces = []
    trace_meta = []
    for item in source:
        trace, info = forecast_trace(item)
        traces.append(trace)
        trace_meta.append(info)
    clock_rows = []
    for item, trace in zip(source, traces):
        for t, row in trace.iterrows():
            clock_rows.append({
                "file": item["file"], "file_sha256": item["sha256"],
                "init_time_utc": item["init_time_utc"],
                "support_start_utc": iso_utc(t),
                "support_end_utc": iso_utc(t + pd.Timedelta(hours=1)),
                "raw_SW_P_end_label_utc": iso_utc(t + pd.Timedelta(hours=1)),
                "raw_T_RH_left_time_utc": iso_utc(t),
                "raw_T_RH_right_time_utc": iso_utc(t + pd.Timedelta(hours=1)),
                "T_hour_mean_C": row["T"], "RH_hour_mean_percent": row["RH"],
                "SW_hour_mean_Wm2": row["SW"], "P_hour_total_mm": row["P"],
            })
    pd.DataFrame(clock_rows).to_csv(OUT / "03_IFS逐变量时间支持.csv", index=False, encoding="utf-8-sig")
    if any(not info["all_future_hours_available"] for info in trace_meta):
        # Keep missing values in member states rather than filling them.
        pass
    aligned_features = historical_energy_support(features)
    model = fit_energy_model(aligned_features)
    beta_draws = np.asarray(model.pop("beta_draws"))
    all_tm = []
    for item, trace in zip(source, traces):
        tag = "run" + pd.Timestamp(item["init_time_utc"]).strftime("%H")
        records, _ = make_members(trace, beta_draws, model, tag)
        all_tm.extend(records)
    geometry = parse_geometry()
    configs = d8_configs()
    all_ta = propagate(all_tm, configs, geometry)
    tm_summary = state_summary(all_tm, "state_tm")
    ta_summary = state_summary(all_ta, "state_ta")
    tm_interval = finite_interval(
        all_tm, "tm_lower_utc", "tm_upper_utc", "tm_event_utc", "state_tm"
    )
    ta_interval = finite_interval(
        all_ta, "ta_lower_utc", "ta_upper_utc", "ta_event_utc", "state_ta"
    )
    # A finite-member interval is unconditionally valid only when no member
    # mass is outside the finite state. A high but sub-unit finite mass is
    # not enough: a mixture distribution would require an explicit point-mass
    # construction, which this model does not make.
    unconditional_tm = (
        tm_interval
        if tm_interval["status"] == "available" and tm_summary["p_finite"] == 1.0
        else None
    )
    unconditional_ta = (
        ta_interval
        if ta_interval["status"] == "available" and ta_summary["p_finite"] == 1.0
        else None
    )
    unconditional_ok = unconditional_tm is not None and unconditional_ta is not None
    p_upper_tm = tm_summary["p_finite"] * 0.95
    p_upper_ta = ta_summary["p_finite"] * 0.95

    # A point port hit is only one event and does not become a frequency estimate.
    hit_utc = HIT_UTC
    port_cover = None
    if ta_interval["status"] == "available":
        port_cover = bool(pd.Timestamp(ta_interval["lower_time"]) <= hit_utc.tz_localize("UTC") < pd.Timestamp(
            ta_interval["upper_time"]))

    # Materialize the old post-event inverse as its own audit product. It is
    # never read by the forward-member, interval, coverage, warning or routing
    # calculations above; the explicit downstream-use flags make that boundary
    # machine-checkable rather than relying on prose.
    legacy_path = ROOT / "临时产物" / "审稿记录" / "冻结父版v5.1记录" / "v43_旧反演审计.json"
    legacy_meta = {
        "status": "legacy_audit_only_not_used",
        "source": None,
        "source_sha256": None,
        "output": None,
        "output_sha256": None,
        "downstream_use": {
```

```
        "forward_forecast": False,
        "coverage": False,
        "warning": False,
        "propagation": False,
    },
    "scope": None,
}
if legacy_path.exists():
    old = json.loads(legacy_path.read_text(encoding="utf-8"))
    legacy_relative = legacy_path.relative_to(ROOT).as_posix()
    legacy_sha = hashlib.sha256(legacy_path.read_bytes()).hexdigest()
    legacy_payload = build_legacy_audit(old, legacy_relative, legacy_sha)
    write_json("03_旧条件反演对照.json", legacy_payload)
    legacy_output = OUT / "03_旧条件反演对照.json"
    legacy_meta.update({
        "source": legacy_relative,
        "source_sha256": legacy_sha,
        "output": legacy_output.relative_to(ROOT).as_posix(),
        "output_sha256": sha256(legacy_output),
        "scope": legacy_payload["scope"],
    })

interval_rows = []
for target, interval, summary, unconditional in [
    ("source_melt_onset_proxy", tm_interval, tm_summary, unconditional_tm),
    ("instant_release_scenario_arrival", ta_interval, ta_summary, unconditional_ta),
]:
    unconditional_status = "available" if unconditional is not None else "not_constructible"
    row = {
        "event_or_issue_id": "issue_20260826T0800+0800",
        "issue_time": iso_local(ISSUE_UTC),
        "target_definition": target,
        "forecast_horizon": "24h",
        "model_version": "unreleased_repair_workspace",
        "product_role": "auxiliary_only_not_true_debris_flow_forecast",
        "calibration_version": "2021_2025_ols_ar1",
        "interval_method": "uniform_within_support_event_time_quantiles",
        "nominal_level": 0.95,
        "lower_time": interval["lower_time"],
        "median_time": interval["median_time"],
        "upper_time": interval["upper_time"],
        "support_envelope_lower_time": interval.get("support_envelope_lower_time"),
        "support_envelope_upper_time": interval.get("support_envelope_upper_time"),
        "support_envelope_width_hours": interval.get("support_envelope_width_hours"),
        "support_endpoint_95_lower_time": interval.get("support_endpoint_95_lower_time"),
        "support_endpoint_95_upper_time": interval.get("support_endpoint_95_upper_time"),
        "support_endpoint_95_width_hours": interval.get("support_endpoint_95_width_hours"),
        "finite_n": interval["finite_n"],
        "interval_status": interval["status"],
        "interval_condition": "finite_conditional",
        "unconditional_status": unconditional_status,
        "unconditional_lower_time": unconditional["lower_time"] if unconditional else None,
        "unconditional_upper_time": unconditional["upper_time"] if unconditional else None,
        "unconditional_probability_upper_bound": p_upper_tm if target.startswith("source") else p_upper_ta,
        "latest_input_available_time": iso_utc(ISSUE_UTC),
        "status": interval["status"],
    }
    row.update({
        key: value for key, value in summary.items()
        if key.startswith("p_")
    })
    if target.startswith("source"):
        row["p_active_at_issue_left_censored"] = None
        row["p_no_arrival"] = None
    else:
        row["p_active_at_issue"] = None
    interval_rows.append(row)
pd.DataFrame(interval_rows).to_csv(OUT / "forecast_intervals.csv", index=False, encoding="utf-8-sig")

true_trigger_row = {
    "event_id": "true_t_trig_20260826",
    "issue_time": iso_local(ISSUE_UTC),
    "target_definition": "true_debris_flow_formation",
    "truth_left": None,
    "truth_right": None,
    "truth_source": None,
    "truth_quality": "not_estimable: no independent source formation time/window",
    "lower_time": None,
    "upper_time": None,
    "definite_cover": None,
    "possible_cover": None,
    "prediction_missing": True,
    "width_hours": None,
    "interval_status": "not_estimable_release_model_unidentified",
    "evaluation_eligible": False,
    "exclusion_reason": "both release/formation forecast and independent truth unresolved; melt proxy is not this
        target",
}
true_trigger_row.update({
    key: value for key, value in tm_summary.items() if key.startswith("p_")
})
port_coverage_row = {
    "event_id": "port_hit_20260826_1030",
    "issue_time": iso_local(ISSUE_UTC),
    "target_definition": "instant_release_scenario_arrival",
    "product_role": "auxiliary_only_assumes_release_at_melt_onset",
    "truth_left": iso_local(HIT_UTC),
```

```
    "truth_right": iso_local(HIT_UTC),
    "truth_source": "题面单一口岸冲击锚点",
    "truth_quality": "single_point_event",
    "lower_time": ta_interval["lower_time"],
    "upper_time": ta_interval["upper_time"],
    "definite_cover": port_cover,
    "possible_cover": port_cover,
    "prediction_missing": ta_interval["status"] != "available",
    "width_hours": ta_interval.get("width_hours"),
    "interval_status": ta_interval["status"],
    "evaluation_eligible": ta_interval["status"] == "available",
    "exclusion_reason": (
        None if ta_interval["status"] == "available" else ta_interval["status"]
    ),
}
port_coverage_row.update({
    key: value for key, value in ta_summary.items() if key.startswith("p_")
})
actual_port_row = {**port_coverage_row,
    "target_definition": "actual_port_arrival", "product_role": "requested_real_target",
    "lower_time": None, "upper_time": None, "definite_cover": None, "possible_cover": None,
    "prediction_missing": True, "width_hours": None,
    "interval_status": "not_estimable_release_model_unidentified", "evaluation_eligible": False,
    "exclusion_reason": "no identifiable release state/time; conditional transport cannot substitute",
}
for row in (true_trigger_row, actual_port_row):
    for key in list(row):
        if key.startswith("p_"):
            row[key] = None
coverage_rows = [true_trigger_row, port_coverage_row, actual_port_row]
pd.DataFrame(coverage_rows).to_csv(OUT / "coverage_events.csv", index=False, encoding="utf-8-sig")

# The frozen operational rule uses only forecast-state mass and the
# predicted arrival window; the observed event anchor is evaluation-only.
alarm = should_alarm(tm_summary)
alarm_utc = ISSUE_UTC if alarm else None
lead_port = float((HIT_LOCAL - ISSUE_LOCAL).total_seconds() / 3600.0) if alarm else None
arrival_lead_lower, arrival_lead_upper = interval_lead_bounds(ta_interval, HIT_UTC)
latency_lead = {
    f"{minutes}min": (lead_port - minutes / 60.0
                      if lead_port is not None else None)
    for minutes in (0, 5, 10, 15, 30)
}
warning_rows = [{
    "event_id": "port_hit_20260826_1030",
    "rule_version": "v5.0_issue_if_p_tm_actionable_ge_0.5",
    "first_alarm_time": iso_local(alarm_utc) if alarm_utc is not None else None,
    "actionable_alarm_time": iso_local(alarm_utc) if alarm_utc is not None else None,
    "source_p_finite": tm_summary["p_finite"],
    "source_p_active_at_issue": tm_summary["p_active_at_issue"],
    "source_p_actionable": tm_summary["p_actionable"],
    "arrival_p_finite": ta_summary["p_finite"],
    "arrival_p_active_at_issue_left_censored": ta_summary[
        "p_active_at_issue_left_censored"
    ],
    "arrival_p_actionable": ta_summary["p_actionable"],
    "trigger_truth_left": None,
    "trigger_truth_right": None,
    "port_hit_time": iso_local(HIT_UTC),
    "lead_trigger_lower": None,
    "lead_trigger_upper": None,
    "model_start_lead_hours": (
        float(
            (
                pd.Timestamp(tm_interval["median_time"])
                - ISSUE_UTC.tz_localize("UTC")
            ).total_seconds()
            / 3600.0
        )
        if tm_interval["status"] == "available"
        else None
    ),
    "arrival_interval_lead_lower": arrival_lead_lower,
    "arrival_interval_lead_upper": arrival_lead_upper,
    "lead_port": lead_port,
    "missed_alarm": not alarm,
    "late_alarm": bool(alarm and lead_port is not None and lead_port < 0),
    "latency_assumptions": "no communication delay; 0/5/10/15/30 min treated as post-processing sensitivity in
        theory",
    **{f"lead_port_{key}": value for key, value in latency_lead.items()},
}]
warning_rows[0]["product_role"] = "melt_advisory_only_not_a_disaster_warning"
pd.DataFrame(warning_rows).to_csv(OUT / "melt_advisory_events.csv", index=False, encoding="utf-8-sig")
true_warning = {
    "event_id": "port_hit_20260826_1030", "target_definition": "true_debris_flow_warning",
    "status": "not_estimable_release_model_unidentified", "first_alarm_time": None,
    "actionable_alarm_time": None, "port_hit_time": iso_local(HIT_UTC),
    "lead_trigger_lower": None, "lead_trigger_upper": None, "lead_port": None,
    "missed_alarm": None, "late_alarm": None,
    "reason": "melt advisory and instant-release transport do not determine actual release",
}
pd.DataFrame([true_warning]).to_csv(OUT / "warning_events.csv", index=False, encoding="utf-8-sig")

# Keep the source trigger, source-to-injection fall, and D8 arrival in one
# member row. A configuration-only table would invite mixing the best
# source hour with the best travel configuration.
member_rows = []
```

```
config_by_id = {cfg["id"]: cfg for cfg in configs}
for record in all_ta:
    cfg = config_by_id[record["d8_configuration_id"]]
    member_rows.append({
        "member_id": record["member_id"],
        "run": record["run"],
        "state_tm": record["state_tm"],
        "state_tm_reason": record.get("state_tm_reason"),
        "tm_lower_utc": iso_utc(record["tm_lower_utc"]) if record["tm_lower_utc"] is not None else None,
        "tm_upper_utc": iso_utc(record["tm_upper_utc"]) if record["tm_upper_utc"] is not None else None,
        "tm_event_utc": iso_utc(record["tm_event_utc"]) if record.get("tm_event_utc") is not None else None,
        "preissue_T_C": record.get("preissue_T_C"),
        "preissue_SW_W_m2": record.get("preissue_SW_W_m2"),
        "preissue_mean_energy_W_m2": record.get("preissue_mean_energy_W_m2"),
        "future_missing_hours": record.get("future_missing_hours"),
        "trajectory_end_utc": (
            iso_utc(record["trajectory_end_utc"])
            if record.get("trajectory_end_utc") is not None else None
        ),
        "state_ta": record["state_ta"],
        "state_release": "not_identified",
        "release_time_utc": None,
        "state_actual_arrival": "not_evaluable_release_unidentified",
        "actual_arrival_time_utc": None,
        "propagation_hypothesis": "instant_release_at_melt_onset_with_prescribed_material",
        "ta_lower_utc": iso_utc(record["ta_lower_utc"]) if record["ta_lower_utc"] is not None else None,
        "ta_upper_utc": iso_utc(record["ta_upper_utc"]) if record["ta_upper_utc"] is not None else None,
        "ta_event_utc": iso_utc(record["ta_event_utc"]) if record.get("ta_event_utc") is not None else None,
        "ta_censor_upper_utc": (
            iso_utc(record["ta_censor_upper_utc"])
            if record.get("ta_censor_upper_utc") is not None else None
        ),
        "d8_configuration_id": record["d8_configuration_id"],
        "source_geometry": record.get("source_geometry"),
        "source_fall_s": record.get("source_fall_s"),
        "d8_travel_s": record.get("d8_travel_s"),
        "d8_design_weight": record["d8_design_weight"],
    })
pd.DataFrame(member_rows).to_csv(OUT / "03_前向预报成员.csv", index=False, encoding="utf-8-sig")

runout_rows = []
for record in all_ta:
    cfg = config_by_id[record["d8_configuration_id"]]
    runout_terminal = runout_terminal_fields(cfg)
    runout_rows.append({
        "member_id": record["member_id"],
        "trigger_member_id": record["member_id"],
        "d8_configuration_id": record["d8_configuration_id"],
        "state_tm": record["state_tm"],
        "state_ta": record["state_ta"],
        "source_definition": "chain_A_source_geometry",
        "target_section": "D8_port_proxy",
        "volume": cfg["volume_m3"],
        "parameter_set": json.dumps(cfg["params"], ensure_ascii=False, sort_keys=True),
        "engine": "D8",
        "distance_metric": "centerline_injection_to_port_proxy",
        "first_arrival_time": record["ta_lower_utc"] if record["state_ta"] == "finite" else None,
        "first_arrival_time_upper": record["ta_upper_utc"] if record["state_ta"] == "finite" else None,
        "first_arrival_censor_upper_time": record.get("ta_censor_upper_utc"),
        "first_arrival_time_s": (record.get("source_fall_s", 0.0) + record.get("d8_travel_s", 0.0)
                                 if record["state_ta"] == "finite" else None),
        "final_runout": cfg["final_runout_m"],
        "d8_configuration_reached_port": cfg["reached_port"],
        "reached_target": record["state_ta"] in {
            "finite", "active_at_issue_left_censored"
        },
        "evaluation_eligible_for_arrival": record["state_ta"] in {
            "finite", "active_at_issue_left_censored"
        },
        "termination_reason": cfg["termination_reason"],
        "natural_stop": runout_terminal["natural_stop"],
        "time_limit": runout_terminal["time_limit"],
        "step_limit": runout_terminal["step_limit"],
        "domain_censor": runout_terminal["domain_censor"],
        "time_limit_nonarrival": runout_terminal["time_limit_nonarrival"],
        "step_limit_nonarrival": runout_terminal["step_limit_nonarrival"],
        "domain_censor_nonarrival": runout_terminal["domain_censor_nonarrival"],
        "right_censored": runout_terminal["right_censored"],
        "weight_or_scenario_type": "equal_member_weight_over_config_design",
        "configuration_weight": record["d8_design_weight"],
        "weight": 1.0 / len(all_ta),
    })
runout_frame = pd.DataFrame(runout_rows)
if not np.isclose(float(runout_frame["weight"].sum()), 1.0, atol=1e-12):
    raise AssertionError("runout member weights must sum to one")
runout_frame.to_csv(OUT / "runout_ranges.csv", index=False, encoding="utf-8-sig")

d8_arrivals = [r["arrival_port_s"] for r in configs if r["arrival_port_s"] is not None and r["reached_port"]]
distance_payload = {
    "schema_version": 1,
    "status": "available",
    "source_definition": geometry["source_centroid"],
    "injection_definition": geometry["injection"],
    "port_proxy_definition": geometry["port_proxy"],
    "route_metrics": {
        "source_to_injection_horizontal_m": geometry["source_to_injection_horizontal_m"],
        "injection_to_port_centerline_m": geometry["centerline_injection_to_port_m"],
```

```
        "combined_route_m": geometry["combined_route_m"],
        "source_to_port_straight_m": geometry["source_to_port_straight_m"],
        "distances_not_added_across_metrics": True,
    },
    "d8_port_arrival_s_range": [min(d8_arrivals), max(d8_arrivals)] if d8_arrivals else None,
    "d8_final_runout_m_range": [
        min(r["final_runout_m"] for r in configs if r["final_runout_m"] is not None),
        max(r["final_runout_m"] for r in configs if r["final_runout_m"] is not None),
    ],
    "right_censored": all(cfg["runout_right_censored"] for cfg in configs),
    "scope": "D8 network route and model-advected port proxy; not observed hazard extent or natural stopping
        distance",
}
write_json("03_空间距离范围.json", distance_payload)

onset_support_counts = {}
for record in all_tm:
    if record["state_tm"] != "finite":
        continue
    offset_hours = int(
        (pd.Timestamp(record["tm_lower_utc"]) - ISSUE_UTC).total_seconds() // 3600
    )
    onset_support_counts[str(offset_hours)] = (
        onset_support_counts.get(str(offset_hours), 0) + 1
    )
if any(
    pd.Timestamp(record["tm_upper_utc"]) - pd.Timestamp(record["tm_lower_utc"])
    != pd.Timedelta(hours=1)
    for record in all_tm if record["state_tm"] == "finite"
):
    raise AssertionError("every finite source member must carry a one-hour support")
if any(
    pd.Timestamp(record["ta_upper_utc"]) - pd.Timestamp(record["ta_lower_utc"])
    != pd.Timedelta(hours=1)
    for record in all_ta if record["state_ta"] == "finite"
):
    raise AssertionError("paired arrival supports must preserve the source support width")
active_left = [
    record for record in all_ta
    if record["state_ta"] == "active_at_issue_left_censored"
]
if any(record.get("ta_censor_upper_utc") is None for record in active_left):
    raise AssertionError(
        "active-at-issue arrival must carry a left-censoring upper bound"
    )
if len(runout_frame) != len(all_ta):
    raise AssertionError("runout rows must preserve one-to-one member pairing")
validation = {
    "member_count": len(all_ta),
    "source_support_counts_by_forecast_hour": onset_support_counts,
    "all_source_supports_one_hour": True,
    "all_paired_arrival_supports_one_hour": True,
    "state_mass_checks": {
        "tm": tm_summary["state_sum_check"],
        "ta": ta_summary["state_sum_check"],
    },
    "source_state_sum": sum(tm_summary["counts"].values()),
    "arrival_state_sum": sum(ta_summary["counts"].values()),
    "p_actionable": {
        "source": tm_summary["p_actionable"],
        "arrival": ta_summary["p_actionable"],
    },
    "active_at_issue_n": tm_summary["counts"]["active_at_issue"],
    "active_at_issue_left_censored_n": len(active_left),
    "active_left_censored_bounds_present": bool(
        all(record.get("ta_censor_upper_utc") is not None for record in active_left)
    ),
    "interval_status": {
        "tm": tm_interval["status"],
        "ta": ta_interval["status"],
    },
    "runout_weight_sum": float(runout_frame["weight"].sum()),
    "runout_member_rows": int(len(runout_frame)),
    "port_single_event_covered": port_cover,
    "true_t_trig_evaluable": False,
}

payload = {
    "schema_version": 1,
    "status": "auxiliary_available_real_forecast_blocked",
    "target_definition": "source meltwater onset proxy and conditional D8 port arrival; not true t_trig",
    "release_model": {
        "status": "not_identified", "release_state": "unknown_not_no_release",
        "release_time_interval": None, "formation_time_interval": None,
        "actual_arrival_interval": None,
        "needed": ["causally available source deformation/strength/pressure constraints",
                "measurement errors and dependence", "release-to-flow transformation model"],
    },
    "task_fulfillment": {"q3_true_trigger_forecast": "not_completed", "release_allowed": False},
    "issue_time_local": iso_local(ISSUE_UTC),
    "issue_time_utc": iso_utc(ISSUE_UTC),
    "window_local": [iso_local(ISSUE_UTC), iso_local(WINDOW_END_UTC)],
    "window_semantics": "half-open prediction window [issue, issue+24h)",
    "ifs_manifest": {
        "path": ifs["path"].relative_to(ROOT).as_posix(),
        "sha256": ifs["sha256"],
        "actual_file_count": ifs["manifest"]["actual_file_count"],
```

```
        "source_runs": trace_meta,
        "port_context_not_used_in_main_chain": [x["file"] for x in ifs["port_files"]],
      },
      "fit": model,
      "state_summary_tm": tm_summary,
      "state_summary_ta": ta_summary,
      "validation": validation,
      "intervals": {
        "t_m": tm_interval,
        "t_a": ta_interval,
        "unconditional_interval": {
          "t_m": unconditional_tm,
          "t_a": unconditional_ta,
        } if unconditional_ok else None,
        "unconditional_status": "available" if unconditional_ok else "not_constructible",
        "unconditional_status_by_target": {
          "t_m": (
            "available" if unconditional_tm is not None
            else "not_constructible"
          ),
          "t_a": (
            "available" if unconditional_ta is not None
            else "not_constructible"
          ),
        },
        "unconditional_probability_upper_bound": {"t_m": p_upper_tm, "t_a": p_upper_ta},
      },
      "coverage": {
        "product_role": "auxiliary_instant_release_scenario_inclusion_only",
        "true_t_trig": "not_estimable",
        "actual_port_arrival": "not_estimable_release_model_unidentified",
        "actual_port_coverage": None,
        "port_hit_event": (
          "not_constructible" if port_cover is None
          else ("1/1" if port_cover else "0/1")
        ),
        "port_hit_covered": port_cover,
        "evaluation_denominator": 1,
        "interval_status": {
          "t_m": tm_interval["status"],
          "t_a": ta_interval["status"],
        },
      },
      "warning": true_warning,
      "melt_advisory": warning_rows[0],
      "geometry": geometry,
      "d8_configuration_ids": [r["id"] for r in configs],
      "legacy_inverse_audit": legacy_meta,
      "outputs": {
        "forecast_intervals": "forecast_intervals.csv",
        "coverage_events": "coverage_events.csv",
        "warning_events": "warning_events.csv",
        "melt_advisory_events": "melt_advisory_events.csv",
        "variable_time_support": "03_IFS逐变量时间支持.csv",
        "runout_ranges": "runout_ranges.csv",
        "forecast_members": "03_前向预报成员.csv",
        "distance": "03_空间距离范围.json",
        "legacy_inverse_audit": "03_旧条件反演对照.json",
      },
      "limitations": [
        "source meltwater onset is an operational proxy, not independent true t_trig",
        "active-at-issue members are left-censored at the port; their pre-issue arrival is not recoverable",
        "IFS station-to-source transfer is structural, not ice-surface validation",
        "D8 configuration sampling is a model-design distribution, not a real probability law",
        "one port hit cannot validate long-run 95% coverage",
      ],
    }
    write_json("03_前向预报.json", payload)
    write_json("03_前向预报成员摘要.json", {
      "tm": tm_summary,
      "ta": ta_summary,
      "tm_interval": tm_interval,
      "ta_interval": ta_interval,
    })
    print(
      "03 forward forecast:",
      f"tm finite={tm_summary['counts']['finite']}/{tm_summary['total_members']}",
      f"tm active={tm_summary['counts']['active_at_issue']}",
      f"p_actionable={tm_summary['p_actionable']:.3f}",
      f"ta finite={ta_summary['counts']['finite']}/{ta_summary['total_members']}",
      f"ta left_censored={ta_summary['counts']['active_at_issue_left_censored']}",
      f"port_cover={port_cover}",
    )
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

step03_time_support_sensitivity.py 时间支持敏感性

```
"""Refit all declared station-clock scenarios without selecting on event coverage."""
from itertools import product
import numpy as np
import pandas as pd
```

```
import step03_forecast as f
from mmlib import io_data as io


def main():
    features = pd.read_parquet(io.CACHE / "features.parquet")
    ifs = f.load_ifs_manifest()
    source = sorted(ifs["source_files"], key=lambda item: item["init_time_utc"])
    traces = [f.forecast_trace(item)[0] for item in source]
    geometry, configs = f.parse_geometry(), f.d8_configs()
    results = []
    for met, rad_end, soil_end, flux_end in product(
        ("point", "start_mean", "end_mean"), (False, True), (False, True), (False, True)
    ):
        aligned = f.historical_energy_support(features, met, rad_end, soil_end, flux_end)
        model = f.fit_energy_model(aligned)
        beta = np.asarray(model.pop("beta_draws"))
        members = []
        for item, trace in zip(source, traces):
            tag = "run" + pd.Timestamp(item["init_time_utc"]).strftime("%H")
            records, _ = f.make_members(trace, beta, model, tag)
            members.extend(records)
        arrivals = f.propagate(members, configs, geometry)
        results.append({
            "scenario": aligned.attrs["station_support_assumption"],
            "fit": model, "melt_state": f.state_summary(members, "state_tm"),
            "instant_release_arrival_state": f.state_summary(arrivals, "state_ta"),
            "melt_interval": f.finite_interval(members, "tm_lower_utc", "tm_upper_utc", "tm_event_utc", "state_tm"),
            "instant_release_arrival_interval": f.finite_interval(arrivals, "ta_lower_utc", "ta_upper_utc", "
                ta_event_utc", "state_ta"),
        })
        print("clock scenario", met, rad_end, soil_end, flux_end,
              "training", model["training_rows"], flush=True)
    assert len(results) == 24
    f.write_json("03_历史家族时间支持敏感性.json", {
        "status": "computed_conditional_sensitivity", "scenario_count": len(results),
        "scenario_selection": "all 24 specified combinations; no event truth used for fitting or selection",
        "main_scenario": {"MET": "point", "RADM": "start_mean", "SOIL": "start_mean", "FLUX": "start_mean"},
        "main_scenario_basis": "unverified working hypothesis, not a metadata finding",
        "release_forecast_status": "not_identified; these are only auxiliary melt and instant-release scenarios",
        "scenarios": results,
    })
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

step03_figures.py 问题三图件

```
"""Chinese figures for the v5.0 forward forecast and paired propagation products.

The figures use only current machine-readable outputs. They deliberately do
not draw a coverage rate for a true formation time that is not observed.
"""
from __future__ import annotations

import json
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from mmlib import io_data as io

ROOT = Path(__file__).resolve().parents[2]
OUT = io.DATA_OUT
FIG = io.FIG_OUT
LOCAL = "Asia/Shanghai"

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "Noto Sans SC", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams.update({
    "font.size": 20,
    "axes.titlesize": 20,
    "axes.labelsize": 18,
    "xtick.labelsize": 15,
    "ytick.labelsize": 15,
    "legend.fontsize": 13,
    # DateFormatter 默认按 UTC 解释时区感知坐标；不设此值会把当地时刻
    # 整体左移八小时，而轴标题仍写着“当地时间”。
    "timezone": LOCAL,
})


def load_json(name: str) -> dict:
    return json.loads((OUT / name).read_text(encoding="utf-8"))


def local_time(value: str | None):
    if value is None:
        return None
```

```
   try:
      if bool(pd.isna(value)):
         return None
   except (TypeError, ValueError):
      pass
   if isinstance(value, str) and value.strip().lower() in {
      "", "nan", "none", "null", "<na>", "not_estimable",
   }:
      return None
   stamp = pd.Timestamp(value)
   if stamp.tzinfo is None:
      stamp = stamp.tz_localize("UTC")
   return stamp.tz_convert(LOCAL)


def value_missing(value) -> bool:
   """Return an explicit missing-value predicate without bool(np.nan) traps."""
   if value is None:
      return True
   if isinstance(value, str):
      return value.strip().lower() in {
         "", "nan", "none", "null", "<na>", "not_estimable",
         "not_constructible", "not_constructible_zero_finite",
      }
   try:
      return bool(pd.isna(value))
   except (TypeError, ValueError):
      return False


def port_coverage_label(row: pd.Series) -> str:
   """Map a single-event coverage row to a conservative display label."""
   if bool(row.get("prediction_missing", False)):
      return "不可估计"
   status = str(row.get("interval_status", "")).strip().lower()
   if status and status != "available":
      return "不可估计"
   if value_missing(row.get("lower_time")) or value_missing(row.get("upper_time")):
      return "不可估计"
   value = row.get("definite_cover")
   if value_missing(value):
      return "不可估计"
   if isinstance(value, str):
      normalized = value.strip().lower()
      if normalized in {"1", "1.0", "true"}:
         return "1/1"
      if normalized in {"0", "0.0", "false"}:
         return "0/1"
      raise ValueError(f"invalid definite_cover label: {value!r}")
   if isinstance(value, (bool, np.bool_)):
      return "1/1" if bool(value) else "0/1"
   number = float(value)
   if not np.isfinite(number) or number not in (0.0, 1.0):
      raise ValueError(f"definite_cover must be 0, 1, or missing; got {value!r}")
   return "1/1" if number == 1.0 else "0/1"


def plot_alarm_marker(axes, alarm) -> bool:
   """Draw a valid alarm or an explicit no-alarm annotation without NaN access."""
   if value_missing(alarm):
      axes.text(
         0.02,
         0.96,
         "未构造报警/未报警",
         transform=axes.transAxes,
         ha="left",
         va="top",
         fontsize=14,
         bbox={"facecolor": "white", "alpha": .9, "edgecolor": "#d0d0d0"},
      )
      return False
   axes.plot([alarm, alarm], [0.25, 0.95], color="#444444", ls="--", lw=1.3)
   axes.text(alarm, 1.0, "08:00发布/报警", ha="center", va="bottom", fontsize=14)
   return True


def done(fig, name: str) -> None:
   FIG.mkdir(parents=True, exist_ok=True)
   fig.savefig(FIG / name, dpi=180, bbox_inches="tight")
   plt.close(fig)


def interval_and_states(payload: dict, intervals: pd.DataFrame) -> None:
   fig, axes = plt.subplots(2, 1, figsize=(11, 7.2), constrained_layout=True,
                            gridspec_kw={"height_ratios": [1.15, 1]})
   rows = [
      ("源区融化起始代理", intervals.iloc[0]),
      ("假定即时释放后的到达", intervals.iloc[1]),
   ]
   colors = ["#b36b3f", "#276b6a"]
   for y, (label, row) in enumerate(rows):
      lo = local_time(row["lower_time"])
      med = local_time(row["median_time"])
      hi = local_time(row["upper_time"])
      env_lo = local_time(row["support_envelope_lower_time"])
      env_hi = local_time(row["support_envelope_upper_time"])
```

```
        axes[0].plot([env_lo, env_hi], [y, y], color=colors[y], lw=18, alpha=.22,
                     solid_capstyle="butt")
        axes[0].plot([lo, hi], [y, y], color=colors[y], lw=9, solid_capstyle="butt",
                     label=label if y == 0 else None)
        axes[0].plot([med, med], [y - .22, y + .22], color="#17252b", lw=2)
        offset, vertical = (.22, "bottom") if y == 0 else (-.24, "top")
        axes[0].text(hi, y + offset, f"{lo:%H:%M}-{hi:%H:%M}", ha="left", va=vertical,
                     fontsize=14, color=colors[y])
    issue = local_time(payload["issue_time_utc"])
    hit = local_time("2026-08-26T02:30:00+00:00")
    axes[0].axvline(issue, color="#333333", ls="--", lw=1.2, label="08:00发布")
    axes[0].axvline(hit, color="#9b2c2c", ls=":", lw=1.5, label="10:30口岸冲击锚点")
    axes[0].set_yticks([0, 1], [row[0] for row in rows])
    axes[0].set_xlim(pd.Timestamp("2026-08-26 07:30", tz=LOCAL),
                     pd.Timestamp("2026-08-26 12:00", tz=LOCAL))
    axes[0].xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
    axes[0].set_xlabel("2026-08-26 当地时间")
    axes[0].set_title("(a) 代表时刻条件区间（深条）与完整支持包络（浅条）")
    axes[0].grid(axis="x", alpha=.22)
    axes[0].legend(loc="lower right", fontsize=14)

    states = [
        "finite", "no_event", "active_at_issue",
        "active_at_issue_left_censored", "no_arrival",
        "right_censored", "not_evaluable",
    ]
    labels = [
        "未来有限", "窗内未起始", "发布时已活动",
        "口岸到达左删失", "完整输入未到达",
        "轨迹右删失", "输入不可评价",
    ]
    colors_state = [
        "#276b6a", "#9a7a42", "#b36b3f", "#7d5b8c",
        "#4e5c76", "#9b5b45", "#777777",
    ]
    width = .34
    for offset, (target, summary) in enumerate([
        ("源区代理", payload["state_summary_tm"]),
        ("即时释放对照", payload["state_summary_ta"]),
    ]):
        bottom = 0.0
        for state, label, color in zip(states, labels, colors_state):
            value = summary["counts"].get(state, 0) / summary["total_members"]
            axes[1].bar(offset, value, width, bottom=bottom, color=color,
                        label=label if offset == 0 else None)
            if value > .02:
                axes[1].text(offset, bottom + value / 2, f"{value:.0%}",
                             ha="center", va="center", fontsize=14,
                             color="white" if value > .15 else "#202020")
            bottom += value
    axes[1].set_xticks([0, 1], ["源区融化起始代理", "即时释放条件到达"])
    axes[1].set_ylim(0, 1.02)
    axes[1].set_ylabel("辅助模型成员比例")
    axes[1].set_title("(b) 源区五类/口岸六类状态质量分别合计为1；缺测不并入无事件")
    axes[1].grid(axis="y", alpha=.2)
    axes[1].legend(ncol=4, fontsize=12, loc="upper center",
                   bbox_to_anchor=(.5, -.18))
    done(fig, "03_时间区间.png")


def coverage_and_warning(warning: pd.DataFrame, coverage: pd.DataFrame) -> None:
    fig, axes = plt.subplots(1, 2, figsize=(12, 4.7), constrained_layout=True)
    hit = local_time("2026-08-26T02:30:00+00:00")
    lo = local_time(coverage.iloc[1]["lower_time"])
    hi = local_time(coverage.iloc[1]["upper_time"])
    port_cover = port_coverage_label(coverage.iloc[1])
    alarm = local_time(warning.iloc[0]["first_alarm_time"])
    axes[0].errorbar(1, hit, yerr=None, fmt="D", ms=8, color="#9b2c2c",
                     label="题面口岸冲击10:30")
    axes[0].plot([lo, hi], [0.55, 0.55], color="#276b6a", lw=12,
                 solid_capstyle="butt", label="即时释放条件区间")
    plot_alarm_marker(axes[0], alarm)
    axes[0].text(0.01, 0.08, "真实释放、形成及到达预测：不可估计\n本图仅为即时释放假设的辅助对照",
                 transform=axes[0].transAxes, fontsize=14,
                 bbox={"facecolor": "white", "alpha": .88, "edgecolor": "#d0d0d0"})
    axes[0].set_ylim(0, 1.2)
    axes[0].set_xlim(pd.Timestamp("2026-08-26 08:00", tz=LOCAL),
                     pd.Timestamp("2026-08-26 12:00", tz=LOCAL))
    axes[0].xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
    axes[0].set_yticks([])
    axes[0].set_xlabel("2026-08-26 当地时间")
    axes[0].set_title(f"(a) 即时释放对照单点包含：{port_cover}")
    axes[0].grid(axis="x", alpha=.22)
    axes[0].legend(fontsize=14, loc="upper right")

    delays = [0, 5, 10, 15, 30]
    colnames = [f"lead_port_{d}min" for d in delays]
    values = [float(warning.iloc[0][c]) for c in colnames]
    bars = axes[1].bar([str(d) for d in delays], values,
                       color=["#276b6a", "#4f8581", "#789b96", "#a4b8b4", "#c6d1ce"])
    axes[1].axhline(0, color="black", lw=.8)
    for bar, value in zip(bars, values):
        axes[1].text(bar.get_x() + bar.get_width() / 2, value + .04,
                     f"{value:.2f} h", ha="center", va="bottom", fontsize=14)
```

```
    axes[1].set_ylim(0, max(values) + .45)
    axes[1].set_xlabel("发布后通信与组织延迟 / min")
    axes[1].set_ylabel("融水提示至冲击锚点的时间差 / h")
    axes[1].set_title("(b) 融水提示延迟对照：真实灾害预警不可估计")
    axes[1].grid(axis="y", alpha=.2)
    done(fig, "03_区间覆盖率.png")


def input_availability(payload: dict) -> None:
    runs = payload["ifs_manifest"]["source_runs"]
    fig, ax = plt.subplots(figsize=(10.5, 4.6), constrained_layout=True)
    issue = local_time(payload["issue_time_utc"])
    for index, run in enumerate(runs):
        init = local_time(run["init_time_utc"])
        available = local_time(run["available_at_utc"])
        ax.plot([init, available], [index, index], color="#7b8b87", lw=3)
        ax.scatter([init], [index], color="#9b5b45", s=55, label="起报" if index == 0 else None,
                   zorder=3)
        ax.scatter([available], [index], color="#276b6a", s=55,
                   label="可用上界" if index == 0 else None, zorder=3)
        ax.text(available, index + .18,
                f"{run['available_at_utc'][11:16]} UTC / {available:%H:%M} 当地",
                fontsize=12)
    ax.axvline(issue, color="#9b2c2c", ls="--", lw=1.3, label="08:00发布")
    ax.set_yticks(range(len(runs)),
                  [f"{run['init_time_utc'][11:16]} UTC 起报" for run in runs])
    ax.set_xlim(pd.Timestamp("2026-08-25 10:00", tz=LOCAL),
                pd.Timestamp("2026-08-26 09:00", tz=LOCAL))
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%m-%d %H:%M"))
    for label in ax.get_xticklabels():
        label.set_rotation(25)
        label.set_ha("right")
    ax.set_xlabel("当地时间")
    ax.set_title("三次源区IFS起报均在08:00发布前达到6 h可用上界")
    ax.grid(axis="x", alpha=.22)
    ax.legend(fontsize=14, loc="lower right")
    done(fig, "03_预警指标.png")


def paired_propagation(distance: dict) -> None:
    runout = pd.read_csv(OUT / "runout_ranges.csv")
    member = pd.read_csv(OUT / "03_前向预报成员.csv")
    merged = member.merge(
        runout[["member_id", "first_arrival_time_s", "configuration_weight"]],
        on="member_id",
        how="left",
        validate="one_to_one",
    )
    config_stats = merged.groupby("d8_configuration_id")["first_arrival_time_s"].agg(
        ["min", "median", "max", "count"]
    ).sort_values("median")

    fig, axes = plt.subplots(1, 2, figsize=(13, 5.1), constrained_layout=True,
                             gridspec_kw={"width_ratios": [1.25, 1]})
    y = np.arange(len(config_stats))
    axes[0].hlines(y, config_stats["min"] / 60, config_stats["max"] / 60,
                   color="#9ab2ae", lw=4)
    axes[0].plot(config_stats["median"] / 60, y, "o", color="#276b6a", ms=5)
    axes[0].set_yticks(y, config_stats.index, fontsize=12)
    axes[0].invert_yaxis()
    axes[0].set_xlabel("同一成员源区支持至口岸代理的传播时延 / min")
    axes[0].set_title("(a) 16个D8情景与成员源时钟配对")
    axes[0].grid(axis="x", alpha=.2)

    d = distance["route_metrics"]
    labels = ["源区质心→注入\n水平距离", "注入→口岸\n中心线", "源区→口岸\n平面直线"]
    values = [
        d["source_to_injection_horizontal_m"] / 1000,
        d["injection_to_port_centerline_m"] / 1000,
        d["source_to_port_straight_m"] / 1000,
    ]
    bars = axes[1].barh(labels, values, color=["#b36b3f", "#276b6a", "#7a7a7a"])
    for bar, value in zip(bars, values):
        axes[1].text(value + .3, bar.get_y() + bar.get_height() / 2,
                     f"{value:.2f} km", va="center", fontsize=14)
    axes[1].text(.02, .03,
                 f"D8终态{runout['final_runout'].median()/1000:.2f} km仅表示域末端右截断\n"
                 "不是自然停积距离，也不叠加不同路线度量",
                 transform=axes[1].transAxes, fontsize=12,
                 bbox={"facecolor": "white", "alpha": .9, "edgecolor": "#d0d0d0"})
    axes[1].set_xlim(0, max(values) * 1.22)
    axes[1].set_xlabel("距离 / km（分行报告，不相加）")
    axes[1].set_title("(b) 空间范围：路线度量与域末端截断")
    axes[1].grid(axis="x", alpha=.2)
    done(fig, "03_空间距离.png")


def main() -> int:
    payload = load_json("03_前向预报.json")
    intervals = pd.read_csv(OUT / "forecast_intervals.csv")
    warning = pd.read_csv(OUT / "melt_advisory_events.csv")
    coverage = pd.read_csv(OUT / "coverage_events.csv")
    distance = load_json("03_空间距离范围.json")
```

```
    interval_and_states(payload, intervals)
    coverage_and_warning(warning, coverage)
    input_availability(payload)
    paired_propagation(distance)
    print("problem 3 figures: 4 written")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

问题四

step04_simulation.py 一维纵剖面仿真

```
"""问题4一维场景诊断：同一注入断面、参数审计、经验停止判据。"""
from __future__ import annotations

import json
import sys
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.colors import LightSource

sys.path.insert(0, str(Path(__file__).resolve().parent))
from mmlib import io_data as io, terrain as tr, trigger as tg, hydro # noqa: E402

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "Noto Sans SC", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False

FIG = io.FIG_OUT
OUT = io.DATA_OUT
CACHE = io.CACHE

RHO_S = 2650.0     # 固相（岩石/冰碛）密度 kg/m³
RHO_W = 1000.0     # 水密度 kg/m³

PARAMS = {
    # V0 由"冰崩体量 × 可动比例"派生，在 main() 中按附件（4）覆写。
    # 这里不放与派生链无关的硬常数：若 V0 是独立常数，其标称来源便推不出该值。
    "V0_m3": None,
    "Cv": 0.45,       # 固体体积分数（题给 0.15–0.80，取偏洪水性值，非算术中点 0.475）
    # B0 与问题 2 的沟宽口径同源（B = 9√A(km²)），但**不做 κ 折算**。
    # 这不是笔误，而是两问的物理对象不同：
    #   问题 2 的 κ 描述"诱发水流只占全沟宽的一部分"（其余为滩地与堆积体），
    #   故起动流量用 q_c • B/κ；
    #   问题 4 描述的是已经起动的**泥石流浆体**，它充满整个沟道断面——
    #   这正是泥石流区别于洪水的定义性特征，故 Manning 公式用全沟宽 B。
    # Full-width transport is an explicit geometry assumption, not a debris-flow definition.
    "B0_m": 40.0,     # 占位，main() 中按 B=9√A 覆写
    "n_man": 0.10,    # 曼宁糙率（泥石流区间 0.08–0.12 的中值）
    "mu": 0.255,      # 等效摩擦系数（附件（6）反演中位数；main() 中按规模分组覆写）
    "T_pass_s": 900.0, # 过流历时
    "dx_m": 100.0,    # 中心线重采样间距
    "slope_window": 7, # 纵剖面平滑窗口（节点数）
    "E0_m2s2": 200.0,
    "start_index": 0,
    "beta_B": 0.8,
    "sustain_nodes": 5, # 停止判据的持续节点数
    "slope_floor": 0.01, # 曼宁公式的坡度下限
    "h_cap": 15.0,    # 断面流深上限
    "v_cap": 30.0,    # 断面流速上限
    "half_angle_deg": 25.0, # 扇体扩散半角
    "mobilizable_fraction": 0.15, # 冰崩体量中参与形成碎屑流的比例（链B）
    # ---- 链A（冰厚-坡度筛选链，基准链）预登记参数（§4.5.4）----
    "kappa_rel": 0.10, # 冰体释放比例（敏感性 ±50%；κ 符号保留给问题2 沟宽比）
    "gamma_enrich": 3.0, # 物源放大系数 γ（沟床松散物补给；先验定值并预登记，
                         #   定值先于与历史堆积规模的量级对照——对照仅校核不定标）
    "h_thr_m": 50.0,  # 链A 源区冰厚阈值（m）
    "sl_thr_deg": 25.0, # 链A 源区坡度阈值（°）
    "rho_ice": 900.0, # 冰密度（冰-泥密度换算 ρ_ice/ρ_df 用）
}

# 敏感性水平：V0 与 B0 在 main() 中按派生基准值展开（见 build_sens）。
# V0 的情景族 = 链A 基准 ±50%（κ_rel • γ 乘积水平的等效扫描）∪ 链B 上界，
# 不再使用与派生链无关的 {1e6, 2.5e6, 5e6} 硬常数。
SENS = {
    "n_man": [0.06, 0.10, 0.14],
    # μ 水平在 build_sens 中按采用值 μb（大+特大中位数）锚定展开；
```

```
    # 此处留空，不使用与基准值无关的孤立数值。
    "mu": [],
    "B0_m": [20.0, 40.0, 80.0],
    "T_pass_s": [300.0, 900.0, 2700.0],

    "Cv": [0.15, 0.45, 0.80],
    "E0_m2s2": [0.0, 200.0, 400.0],
}

# 强度分级阈值（N/m，即单位宽度冲击力）。口径与 I = ρ_m·h·v² 一致，
# 取 10 / 100 / 1000 kN·m⁻¹（ρ_m≈1742 kg·m⁻³）：
#  10 kN/m ≈ h=0.5 m、v=3.4 m/s —— 可推动沟床松散物质
#  100 kN/m ≈ h=1.5 m、v=6.2 m/s —— 可破坏浆砌石护岸
#  1000 kN/m ≈ h=3.0 m、v=13.8 m/s —— 可摧毁钢筋混凝土结构
I_CLASS_EDGES = (0.0, 1.0e4, 1.0e5, 1.0e6, np.inf)


def mixture_density(cv: float) -> float:
    """混合物密度 ρ_m = C_v ρ_s + (1−C_v) ρ_w（kg/m³）。"""
    return float(cv) * RHO_S + (1.0 - float(cv)) * RHO_W


def _tg_kappa_width() -> float:
    """问题2 沟宽经验式的系数（同一真源，避免两问各写一个数）。"""
    from mmlib import trigger as _tg
    return float(_tg.KAPPA_WIDTH)


def _tg_kappa_q() -> float:
    """问题2 起动流量中的 κ（全沟宽 → 有效过流宽度）。"""
    from mmlib import trigger as _tg
    return float(_tg.KAPPA_Q)


def build_sens(v0_base: float, b0_base: float, v_upper: float | None = None,
               mu_base: float | None = None) -> dict:
    """按派生基准值展开敏感性水平，并保证每个参数的扫描序列包含基准值。

    体积情景族（§4.5.4）：链A 基准 ±50%（κ_rel 与 γ 只以乘积 κ·γ 进入
    V_df，故体积水平扫描即覆盖二者联合效应）+ 链B 上界情景。
    μ 水平锚定在采用值 μb 上（0.7×/0.85×/1×/1.2×）：覆盖"更流动→运程
    触顶（前锋抵达中心线末端）"与"更阻滞→未到达口岸"两个方向，
    不再混入与基准无关的孤立数值（旧水平 0.147 并不等于采用值 μb，
    "扫描序列包含基准值"的约定即由此破坏）。
    """
    out = {k: list(v) for k, v in SENS.items()}
    if mu_base is not None:
        out["mu"] = sorted({round(0.7 * mu_base, 4), round(0.85 * mu_base, 4),
                            float(mu_base), round(1.2 * mu_base, 4)})
    lv = sorted({0.5 * v0_base, v0_base, 1.5 * v0_base})
    if v_upper is not None:
        lv = sorted(set(lv) | {float(v_upper)})
    out["V0_m3"] = lv
    out["B0_m"] = sorted({0.5 * b0_base, b0_base, 2.0 * b0_base})
    return out


def _ser(x):
    if isinstance(x, dict):
        return {str(k): _ser(v) for k, v in x.items()}
    if isinstance(x, (list, tuple)):
        return [_ser(v) for v in x]
    if isinstance(x, (np.floating, float)):
        v = float(x)
        return None if not np.isfinite(v) else round(v, 6)
    if isinstance(x, (np.integer, int)):
        return int(x)
    if isinstance(x, (np.bool_, bool)):
        return bool(x)
    if isinstance(x, np.ndarray):
        return _ser(x.tolist())
    return x


def save_json(obj, name: str):
    out = OUT / name
    out.parent.mkdir(parents=True, exist_ok=True)
    out.write_text(json.dumps(_ser(obj), ensure_ascii=False, indent=2), encoding="utf-8")
    return out


# ============================================================== 中心线
def channel_centerline(dx: float = 100.0) -> dict:
    """源区至口岸的主沟中心线，等距重采样。

    高程处理：D8 追迹路径逐像元取高程时会出现大量伪"上升步"
    （本数据集实测 139 步，随地形缓存复核并写入 04_中心线校验.json 的
    "原始纵剖面上升步数"），成因是 D8 在近等高线上横向摆动而非真正反坡。
    直接使用会使 Heim 判据的累计比能被虚假上升步不断扣减。
    故对纵剖面做**单调递减回归**（isotonic regression），在最小二乘
    意义下得到物理一致的单调纵剖面。
    """
    dem, grid = io.load_dem()
```

```
    z = dem.astype("float64")
    # 地形派生量由 terrain_prep 一次性预计算并缓存（fill/d8/acc 与二维引擎共用）
    from mmlib import terrain_prep as _tpp
    tpz = _tpp.ensure_terrain_cache()
    zf = tpz["filled"].astype("float64")
    d8 = tpz["dir"]
    acc = tpz["acc"]
    cell_x = abs(grid["dlon"]) * 111320.0 * np.cos(np.radians(28.4))
    cell_y = abs(grid["dlat"]) * 110574.0

    gn = io.load_glacier_nc()
    from mmlib import geodesy as gd
    gt = gn["geotransform"]
    mask = (np.isfinite(gn["thickness"]) &
           (gn["thickness"] >= PARAMS["h_thr_m"]) &
           np.isfinite(gn["slope"]) & (gn["slope"] >= PARAMS["sl_thr_deg"]))
    gy_, gx_ = np.nonzero(mask)
    pts = []
    for row, col in zip(gy_, gx_):
        lo, la = gd.utm_to_lonlat(gt[0] + (col + .5) * gt[1],
                                  gt[3] + (row + .5) * gt[5], 45)
        # Geographic metadata uses the upper-left pixel center.
        rr = int(np.floor((la - grid["lat0"]) / grid["dlat"] + .5))
        cc = int(np.floor((lo - grid["lon0"]) / grid["dlon"] + .5))
        if 0 <= rr < z.shape[0] and 0 <= cc < z.shape[1] and np.isfinite(z[rr, cc]):
            pts.append((rr, cc))
    if not pts:
        raise ValueError("chain A source does not intersect DEM")
    start = max(set(pts), key=lambda q: acc[q])
    path = tr.trace_downstream(d8, start)
    rp, cp = io.lonlat_to_rowcol(grid, io.EVENT["port_lon"], io.EVENT["port_lat"])
    distance = np.hypot((np.array([q[0] for q in path]) - rp) * cell_y,
                        (np.array([q[1] for q in path]) - cp) * cell_x)
    ip = int(np.argmin(distance))
    path = path[:ip + 1]
    if len(path) < 3:
        raise ValueError("common source-to-port path too short")
    z_path = np.array([z[p] for p in path])
    z_fill_path = np.array([zf[p] for p in path])
    from sklearn.isotonic import IsotonicRegression
    idx = np.arange(len(z_path), dtype="float64")
    z_iso = IsotonicRegression(increasing=False, out_of_bounds="clip").fit_transform(idx, z_path)

    xy = np.array([[p[1] * cell_x, p[0] * cell_y] for p in path], dtype="float64")
    seg = np.hypot(np.diff(xy[:, 0]), np.diff(xy[:, 1]))
    s = np.concatenate([[0.0], np.cumsum(seg)])
    n = max(int(s[-1] / dx) + 1, 20)
    s_new = np.linspace(0.0, s[-1], n)
    sx = np.interp(s_new, s, xy[:, 0])
    sy = np.interp(s_new, s, xy[:, 1])
    zz = np.interp(s_new, s, z_iso)
    zz_fill = np.interp(s_new, s, z_fill_path)
    return {
        "x_m": sx, "y_m": sy, "s_m": s_new, "z_m": zz,
        "z_filled_m": zz_fill, "z_raw_m": np.interp(s_new, s, z_path),
        "length_km": float(s[-1] / 1000), "n": n,
        "start_rc": [int(v) for v in path[0]], "end_rc": [int(v) for v in path[-1]],
        "start_lonlat": [float(grid["lon0"]) + path[0][1] * float(grid["dlon"]),
                         float(grid["lat0"]) + path[0][0] * float(grid["dlat"])],
        "end_lonlat": [float(grid["lon0"]) + path[-1][1] * float(grid["dlon"]),
                       float(grid["lat0"]) + path[-1][0] * float(grid["dlat"])],
        "port_offset_m": float(distance[ip]),
        "injection_z_raw_m": float(z_path[0]),
        "injection_z_filled_m": float(z_fill_path[0]),
        "source_mapping": {"native_cells": int(mask.sum()), "inside_instances": len(pts),
                           "unique_cells": len(set(pts))},
        "clock": "common_chain_A_injection_to_port",
        "path_rc": np.array(path, dtype=int),
        "原始纵剖面上升步数": int((np.diff(z_path) > 0).sum()),
        "单调回归调整量_m": {"mean": float(np.mean(z_iso - z_path)),
                          "max": float(np.max(np.abs(z_iso - z_path)))},
        "与填洼高程差_m": {"mean": float(np.mean(zz - zz_fill)),
                        "max": float(np.max(np.abs(zz - zz_fill)))},
        "高程范围_m": [float(zz[-1]), float(zz[0])],
    }


def source_geometry(gn, mask):
    from mmlib import geodesy as gd
    rr, cc = np.nonzero(mask)
    gt = gn["geotransform"]
    coords = [gd.utm_to_lonlat(gt[0] + (c+.5)*gt[1], gt[3] + (r+.5)*gt[5], 45)
              for r, c in zip(rr, cc)]
    z = np.asarray(gn["dem"])[mask]
    hi = np.asarray(gn["thickness"])[mask]
    good = np.isfinite(z) & np.isfinite(hi)
    return {"centroid_lonlat": np.mean(coords, axis=0).tolist(),
            "z_dem_p90_m": float(np.percentile(z[good], 90)),
            "z_dem_plus_ice_p90_m": float(np.percentile((z+hi)[good], 90)),
            "release_ice_m3": float(np.sum(hi) * abs(gt[1]*gt[5]) * PARAMS["kappa_rel"]),
            "height_semantics": "two hypotheses; equal weight only as declared engineering prior"}


def _jonckheere_terpstra(groups: list[np.ndarray]) -> dict:
    """Jonckheere-Terpstra 趋势检验（正态近似，含并列校正）。

```

```
    Kruskal–Wallis 只能回答"组间有差异"，不能区分趋势方向；JT 检验
    组间均值随**预设顺序**单调。J = Σ_{i<j}[#(x_i>x_j) + 0.5·#(x_i=x_j)]，
    E[J] = (N²−Σn²)/4，Var = [N²(2N+3) − Σn²(2n+3) − Σt(t²−1)]/72
    （t 为合并样本中并列组的大小）。J 偏大 ⇔ 后面的组系统性地更小。
    """
    from scipy import stats as _st
    n = np.array([len(g) for g in groups], dtype=float)
    x = np.concatenate(groups)
    N = float(x.size)
    J = 0.0
    for i in range(len(groups)):
        for j in range(i + 1, len(groups)):
            a = np.asarray(groups[i])[:, None]
            b = np.asarray(groups[j])[None, :]
            J += float((a > b).sum() + 0.5 * (a == b).sum())
    EJ = (N ** 2 - float(np.sum(n ** 2))) / 4.0
    _, cnt = np.unique(x, return_counts=True)
    t = cnt.astype(float)
    var = (N ** 2 * (2 * N + 3) - float(np.sum(n ** 2 * (2 * n + 3)))
           - float(np.sum(t ** 3 - t))) / 72.0
    se = float(np.sqrt(max(var, 0.0)))
    z = (J - EJ) / se if se > 0 else 0.0
    return {"J": float(J), "E_J": float(EJ), "SE": se, "z": float(z),
            "p_单侧_递减": float(_st.norm.sf(z))}


# ============================================================== μ 反演
def invert_mu() -> dict:
    """由附件（6）反演等效摩擦系数 μ = Δz / (L_channel + L_fan)。"""
    hp = io.load_hazard_points()
    hp = hp.copy()
    hp["mu"] = hp["relief_m"] / (hp["length_m"] + hp["fan_l"])
    hp["mu_ch"] = hp["relief_m"] / hp["length_m"]
    v = hp["mu"].replace([np.inf, -np.inf], np.nan).dropna()
    out = {
        "样本数": int(len(v)),
        "分位数": {f"p{q}": float(np.percentile(v, q)) for q in (5, 10, 25, 50, 75, 90, 95)},
        "均值": float(v.mean()), "标准差": float(v.std()),
        "按规模": {str(k): {"n": int(len(g)), "mu中位数": float(g["mu"].median())}
                   for k, g in hp.groupby("规模")},
        "按类型": {str(k): {"n": int(len(g)), "mu中位数": float(g["mu"].median())}
                   for k, g in hp.groupby("类型")},
        "统计检验_规模与mu": {},
    }
    # Kruskal–Wallis（组间差异）+ Jonckheere–Terpstra（有序趋势）：
    # 采用条件 μ（大+特大）的前提是"μ 随规模等级单调递减"，KW 检验不了
    # 方向，JT 补上。组顺序显式固定为 小→中→大→特大——不能用 groupby
    # 的字典序（其顺序由中文码点决定，与规模等级无关）。
    from scipy import stats
    scale_order = [k for k in ("小", "中", "大", "特大")
                   if (hp["规模"] == k).sum() >= 2]
    groups = [hp.loc[hp["规模"] == k, "mu"].dropna().values
              for k in scale_order]
    if len(groups) >= 2:
        H, p = stats.kruskal(*groups)
        out["统计检验_规模与mu"] = {"H": float(H), "p": float(p),
                                    "结论": "μ 随规模等级存在显著差异" if p < 0.05
                                    else "μ 与规模等级无显著差异"}
    if len(groups) >= 3:
        jt = _jonckheere_terpstra(groups)
        jt.update({"顺序": "小→中→大→特大",
                   "方向": "J 偏大表示 μ 随规模等级递减（条件 μ 的采用前提）",
                   "结论": ("μ 随规模等级显著递减（p<0.05），条件 μ（大+特大）"
                            "取低于全样本中位数的值有统计依据"
                            if jt["p_单侧_递减"] < 0.05 else
                            "未检出单调递减趋势，条件 μ 的采用依据减弱（登记）")})
        out["趋势检验_JonckheereTerpstra"] = jt
    shift = {str(k): float(g["mu"].median() - v.median()) for k, g in hp.groupby("规模")}
    out["规模相对总体中位数的偏移"] = shift
    return out


# ============================================================== 主仿真
def run_once(params: dict, cl: dict | None = None, verbose: bool = True) -> dict:
    if cl is None:
        cl = channel_centerline(params["dx_m"])
    if "状态" in cl:
        return cl
    r = hydro.solve_flow(cl["s_m"], cl["z_m"], params)
    dem, grid = io.load_dem()
    cell_area = (abs(grid["dlon"]) * 111320.0 * np.cos(np.radians(28.4))) * \
                (abs(grid["dlat"]) * 110574.0)
    r["中心线"] = cl
    r["input_parameters"] = dict(params)

    # 强度指标：单位宽度冲击力 I = ρ_m·h·v²（N·m⁻¹），ρ_m 由 C_v 给出。
    # hydro.solve_flow 内部只算几何量 h·v²；密度在这里进入，使 C_v 成为
    # 真正进入方程的参数而不是装饰性定义。
    cv = float(params["Cv"])
    rho = mixture_density(cv)
```

```
    wet_mask = r["h_m"] > 0.05
    # 单位宽度冲击力 I = ρ_m·h·v²（N/m = J/m²），是泥石流强度分级的常用量。
    # 量纲：kg·m⁻³ × m × m²·s⁻² = kg·s⁻² = N/m。注意它不是压强（Pa）。
    r["intensity"] = rho * r["h_m"] * r["v_m_s"] ** 2
    r["Cv"] = cv
    r["rho_m"] = rho
    r["I_max"] = float(np.max(r["intensity"]))
    r["I_median"] = float(np.median(r["intensity"][wet_mask])) if wet_mask.any() else 0.0

    # 强度场：沿中心线走廊带，每个像元取最近节点的强度。
    # 走廊半宽 = 该断面沟宽的一半（B/2），这样强度分布反映真实的
    # 沟道宽度变化，而不是人为设定的固定带宽。
    h, w = dem.shape
    inten_field = np.zeros((h, w), dtype="float32")
    for i in range(len(cl["s_m"])):
        if r["v_m_s"][i] <= 0:
            continue
        frac = i / max(len(cl["s_m"]) - 1, 1)
        rr = int(np.floor(cl["y_m"][i] / (abs(grid["dlat"]) * 110574.0)+.5))
        cc = int(np.floor(cl["x_m"][i] / (abs(grid["dlon"]) * 111320.0 * np.cos(np.radians(28.4)))+.5))
        # 半宽（像元数）= 沟宽一半 / 像元尺寸
        bw = max(1, int(round(0.5 * r["B_m"][i] / 30.0)))
        r0_, r1_ = max(0, rr - bw), min(h, rr + bw + 1)
        c0_, c1_ = max(0, cc - bw), min(w, cc + bw + 1)
        inten_field[r0_:r1_, c0_:c1_] = np.maximum(
            inten_field[r0_:r1_, c0_:c1_], float(r["intensity"][i]))
    r["强度场"] = inten_field
    r["强度分级_走廊"] = hydro.intensity_classes(
        inten_field, inten_field > 0.05, cell_area, edges=I_CLASS_EDGES)
    r["强度分级_中心线"] = hydro.intensity_classes(
        r["intensity"], wet_mask, cell_area, edges=I_CLASS_EDGES)
    r["走廊面积_km2"] = float((inten_field > 0).sum() * cell_area / 1e6)

    # 截断诊断：触顶节点数决定流深/流速峰值是否由模型输出还是人为上限决定
    r["触顶节点数"] = {"流深": int((r["h_m"] >= float(params.get("h_cap", 15.0))).sum()),
                "流速": int((r["v_m_s"] >= float(params.get("v_cap", 30.0))).sum())}

    if verbose:
        print(f" 起始 {r['s_km'][np.where(r['v_m_s'] > 0)[0][0]]:.2f} km, "
              f"停止 {r['runout_km']:.2f} km（总长 {r['total_km']:.2f} km）")
        print(f" 流速 中位 {r['v_median_m_s']:.1f} / 最大 {r['v_max_m_s']:.1f} m/s, "
              f"流深 中位 {r['h_median_m']:.2f} m, Fr 中位 {r['Fr_median']:.2f}")
        print(f" 到达口岸 {r['arrival_outlet_s']} s "
              f"到达秒={r['arrival_outlet_s']}, 到达={r['reached_outlet']}")
    return r


def main() -> int:
    FIG.mkdir(parents=True, exist_ok=True)
    CACHE.mkdir(parents=True, exist_ok=True)

    print("[1/6] 中心线提取…")
    cl = channel_centerline(PARAMS["dx_m"])
    print(f" 中心线 {cl['length_km']:.2f} km, {cl['n']} 节点, "
          f"高程 {cl['z_m'][0]:.0f}→{cl['z_m'][-1]:.0f} m")
    save_json({"中心线长度_km": cl["length_km"], "节点数": cl["n"],
               "起点": cl["start_lonlat"], "终点": cl["end_lonlat"],
               "原始纵剖面上升步数": cl["原始纵剖面上升步数"],
               "单调回归调整量_m": cl["单调回归调整量_m"],
               "与填洼高程差_m": cl["与填洼高程差_m"],
               "高程范围_m": cl["高程范围_m"]}, "04_中心线校验.json")

    print("[2/6] 等效摩擦系数反演（附件6, 84 条历史沟道）…")
    mu_inv = invert_mu()
    print(f" μ 中位数 {mu_inv['分位数']['p50']:.3f}（p25 {mu_inv['分位数']['p25']:.3f}, "
          f"p75 {mu_inv['分位数']['p75']:.3f}）")
    # 条件 μ：本沟为**大型链式事件**（冰崩—碎屑流—泥石流—山洪），
    # 故取"大 + 特大"规模沟道 μ 的中位数作为基准，而非全样本中位数。
    # 依据：μ 与规模等级显著负相关（事件越大、越持续、流动性越强），
    # 全样本中位数会把本事件按"中型"处理，导致运移距离被系统性低估。
    hp = io.load_hazard_points().copy()
    hp["mu"] = hp["relief_m"] / (hp["length_m"] + hp["fan_l"])
    big = hp[hp["规模"].isin(["大", "特大"])]["mu"].dropna()
    mu_cond = float(np.median(big))
    PARAMS["mu"] = mu_cond
    print(f" 条件 μ（大+特大, n={len(big)}）中位数 {mu_cond:.3f}")
    mu_inv["条件μ_大特大"] = {"n": int(len(big)), "median": mu_cond,
                        "p25": float(np.percentile(big, 25)),
                        "p75": float(np.percentile(big, 75)),
                        "采用值": mu_cond,
                        "理由": "大型链式事件的条件场景选择，非独立率定", "samples": big.to_list()}
    save_json(mu_inv, "04_摩擦系数反演.json")

    print("[3/6] 初始释放体积（附件4 冰厚场, 双链情景族 §4.5.4）…")
    gn = io.load_glacier_nc()
    ice = gn["thickness"]
    thick = np.where(np.isfinite(ice), ice, 0.0)
    cell_ice = abs(gn["geotransform"][1] * gn["geotransform"][5])
    # ---- 链A（冰厚-坡度筛选链，基准链）----
```

```
    # Chain A geometry on the native glacier grid; slope is mandatory.
    slope_g = gn["slope"]
    srcA = (thick >= PARAMS["h_thr_m"]) & np.isfinite(slope_g) & (slope_g >= PARAMS["sl_thr_deg"])
    V_src_ice = float(np.sum(thick[srcA]) * cell_ice)
    rho_df = mixture_density(PARAMS["Cv"]) # ρ_df = Cv·ρ_s+(1−Cv)·ρ_w
    V_ice_A = PARAMS["kappa_rel"] * V_src_ice
    V_df_A = PARAMS["gamma_enrich"] * (PARAMS["rho_ice"] / rho_df) * V_ice_A
    # ---- 链B（可动比例链，大体量上界情景）----
    thr = float(np.nanpercentile(thick[thick > 0], 90))
    # 冰崩释放区：冰厚 ≥90% 分位的像元（冰崩只发生在冰体最厚、坡度最陡处）
    rel_mask = thick >= thr
    A_rel = float(rel_mask.sum() * cell_ice) / 1e6
    h_ice = float(np.mean(thick[rel_mask])) if rel_mask.any() else 0.0
    V_ice = A_rel * 1e6 * h_ice           # = V_b（冰崩体量）
    # 进入沟道的**可动**体积：冰崩体并非全部转化为泥石流。按链式过程
    # 的实际观测，冰崩体量中约 10%–20% 参与形成碎屑流，其余在崩落区
    # 停积或融化。取 mobilizable_fraction 作为显式参数。
    frac_mob = float(PARAMS["mobilizable_fraction"])
    V0_B = V_ice * frac_mob
    # 基准取链A（§4.5.4 报告规则）：链B 作为上界情景进入系综。
    # V0 直接采用派生值，不引入任何与派生链无关的硬常数。
    PARAMS["V0_m3"] = V_df_A
    # 源区合理性旁证（D12）：崩前表面速度场与链A 源区的空间重叠。
    # 速度场不进入运动方程（崩前状态，不是崩后初值），仅作源区选取旁证。
    v_amp = gn.get("v_amp", gn.get("v"))
    d12 = {"可用": False}
    if v_amp is not None:
       vsrc = np.nan_to_num(np.where(np.isfinite(v_amp), v_amp, 0.0), nan=0.0)
       if vsrc.sum() > 0:
          d12 = {
             "可用": True,
             "速度加权占比": float(vsrc[srcA].sum() / vsrc.sum()),
             "口径_速度加权占比": "源区像元的崩前速度 / 全冰川速度总和（速度加权）",
             "P90高速区命中率": float(
                (srcA & (vsrc >= np.percentile(vsrc[vsrc > 0], 90))).sum()
                / max((vsrc >= np.percentile(vsrc[vsrc > 0], 90)).sum(), 1)),
             "口径_P90": "速度 P90 以上像元中落入链A 源区的比例",
             "说明": "崩前速度仅为对照；低重叠不支持既定源区，不能称通过验证",
             "源区面积占比": float(srcA.sum()/max(np.isfinite(ice).sum(),1)),
          }
    # 初始沟宽与问题2 起动判据同一口径（B = 9√A），避免同一沟道两处取不同宽度。
    # 集水区面积由本脚本直接算（不读 02_集水区特征.json：该文件由 step02 产出，
    # 而 step02 又依赖本脚本的中心线口径，读它会形成循环依赖）。
    from mmlib import trigger as _tg
    A_catch = _tg.catchment_glacier_fraction()
    save_json(A_catch, "02_集水区特征.json")
    # 静态特征（坡度/土地覆盖/可蚀性）在同一集水区掩膜上一并产出：
    # B1 成果携带 02_静态特征.json 却没有生成代码，step02 冷启动即 KeyError。
    save_json(_tg.catchment_static_features(A_catch), "02_静态特征.json")
    _W_full = float(_tg_kappa_width() * np.sqrt(A_catch["集水区面积_km2"]))
    PARAMS["B0_m"] = _W_full
    # 问题3 的路径 A 需要标定后的起动流量；此处把中心线比降口径一并写入，
    # 使"q_c 用哪条比降"这件事只有一处定义。
    print(f" 链A：源区 {srcA.sum()} 像元（冰厚≥{PARAMS['h_thr_m']:.0f} m 且坡度≥"
          f"{PARAMS['sl_thr_deg']:.0f}°），V_src={V_src_ice:.3e} m³，"
          f"ρ_df={rho_df:.0f} kg/m³ → V_df={V_df_A:.3e} m³（基准）")
    print(f" 链B：冰厚 p90={thr:.1f} m，释放区 {A_rel:.2f} km²，平均冰厚 {h_ice:.1f} m，"
          f"冰崩体量 {V_ice:.2e} m³ × 可动比例 {frac_mob:.2f} = {V0_B:.3e} m³（上界情景）")
    if d12["可用"]:
       print(f" D12 旁证：速度加权占比 {d12['速度加权占比']:.3f}，"
             f"P90 高速区命中率 {d12['P90高速区命中率']:.3f}")
    print(f" 初始沟宽 B0 = {PARAMS['B0_m']:.1f} m（=9√A；泥石流浆体充满断面，不除 κ）")
    save_json({"冰厚p90_m": thr, "释放区面积_km2": A_rel, "平均冰厚_m": h_ice,
               "冰崩体量_m3": V_ice,
               "可动比例": frac_mob,
               "链B_V0_m3": V0_B,
               "链A": {"h_thr_m": PARAMS["h_thr_m"], "sl_thr_deg": PARAMS["sl_thr_deg"],
                   "源区像元数": int(srcA.sum()),
                   "源区面积_km2": float(srcA.sum() * cell_ice) / 1e6,
                   "V_src_ice_m3": V_src_ice,
                   "kappa_rel": PARAMS["kappa_rel"],
                   "gamma": PARAMS["gamma_enrich"],
                   "rho_ice": PARAMS["rho_ice"], "rho_df": rho_df,
                   "V_ice_A_m3": V_ice_A, "V_df_A_m3": V_df_A},
               "链A_V_df_m3": V_df_A,
               "采用_V0_m3": PARAMS["V0_m3"],
               "体积情景水平_m3": sorted({0.5 * V_df_A, V_df_A, 1.5 * V_df_A, V0_B}),
               "双链差异比": float(V0_B / V_df_A),
               "源区旁证_D12": d12,
               "共同断面": {"注入_rc": cl["start_rc"], "口岸_rc": cl["end_rc"],
                         "注入_lonlat": cl["start_lonlat"], "口岸_lonlat": cl["end_lonlat"],
                         "注入高程_raw_m": cl["injection_z_raw_m"],
                         "注入高程_filled_m": cl["injection_z_filled_m"],
                         "口岸偏移_m": cl["port_offset_m"]},
               "链A源几何": source_geometry(gn, srcA),
               "截断事件": False,
               "初始沟宽_B0_m": PARAMS["B0_m"],
```

```
            "说明": ("基准 V0 取链A（冰厚-坡度筛选链）：V_df = γ·(ρ_ice/ρ_df)·κ_rel·"
                  "Σ_src(h_ice·a)，源区为附件4 原生网格上冰厚≥h_thr 且坡度≥sl_thr "
                  "的格网集合。链B（P90 释放区 × 可动比例）作为本情景族大体量对照"
                  "进入场景集合，不是现实严格上界；两链差异是初始条件假设差异。"
                  "κ_rel 与 γ 仅以乘积进入 V_df（框架性质，登记）。"
                  "本文件是一维诊断/D8网络两引擎体积的唯一真源。"
                  "初始沟宽 B0 与问题2 起动判据同口径 B=9√A(km²)，"
                  "使两问的沟道宽度不再是两个互不相干的数。")},
          "04_初始体积.json")

print("[4/6] 基线仿真…")
base = run_once(dict(PARAMS), cl=cl)
payload = {k: v for k, v in base.items() if k not in ("中心线", "强度场")}
payload["中心线"] = cl
payload["params"] = dict(PARAMS)
save_json(payload, "04_仿真基线.json")

sens = build_sens(PARAMS["V0_m3"], PARAMS["B0_m"], v_upper=V0_B,
                  mu_base=PARAMS["mu"])
print(f"[5/6] 单参数敏感性（{len(sens)} 参数，含 Cv 三水平）…")
rows = {}
n_dup_baseline = 0
for key, vals in sens.items():
    rows[key] = []
    for v in vals:
        p = dict(PARAMS)
        if key == "mu_prop":
            p["mu"] = PARAMS["mu"] * float(v)
        else:
            p[key] = v
        r = run_once(p, cl=cl, verbose=False)
        capped = int(r["触顶节点数"]["流深"] + r["触顶节点数"]["流速"])
        reached = bool(r["reached_outlet"])
        rows[key].append({
            "取值": v,
            "runout_km": r["runout_km"],
            "reached_outlet": reached,
            # 未到达情境记 None：不外推、不以 0 计入任何统计。
            # hydro.solve_flow 的 arrival_outlet_s=t[stop_idx] 在未到达时
            # 是运动学外推值（非到达时刻），下游统计（step03 系综、
            # 04_结果区间）一律按 reached_outlet 过滤。
            "到达口岸_min": (round(r["arrival_outlet_s"] / 60, 4)
                         if reached else None),
            "v_median_m_s": r["v_median_m_s"],
            "v_max_m_s": r["v_max_m_s"],
            "h_median_m": r["h_median_m"],
            "I_max": r["I_max"],
            "I_median": r["I_median"],
            "Fr_median": r["Fr_median"],
            "波列长度_m": r["波列长度_m"],
            "液型": r["液型"],
            "触顶节点数": capped,
            "params": dict(p),
            "termination_reason": r["termination_reason"],
            "continuity_relative_residual_max": r["continuity_relative_residual_max"],
            # 运程触顶：前锋抵达中心线末端（runout ≥ 0.999×total），
            # Heim 判据未在域内自然停止。与"触顶节点数"（h/v 触发人为
            # 上限的节点数）是两类事件，分开登记、分开解读。
            "运程触顶": bool(r["runout_km"] >= 0.999 * r["total_km"]),
        })
        tags = ((f"（触顶 {capped}）" if capped else "")
              + ("［运程触顶］" if r["runout_km"] >= 0.999 * r["total_km"] else "")
              + ("［未到达口岸］" if not reached else ""))
        print(f" {key}={v:g}: runout {r['runout_km']:.2f} km, "
              f"v_med {r['v_median_m_s']:.1f} m/s, "
              f"I_max {r['I_max']/1e3:.0f} kN/m{tags}")
# 独立情境数：1 + Σ(n_i − 1)，避免把同一基线重复计数（各序列均含基准值）
unique = {json.dumps(dict(PARAMS), sort_keys=True): {
    "id": "base", "params": dict(PARAMS), "reached_outlet": bool(base["reached_outlet"]),
    "arrival_outlet_s": base["arrival_outlet_s"], "runout_km": base["runout_km"],
    "termination_reason": base["termination_reason"],
    "到达口岸_min": base["arrival_outlet_s"]/60 if base["arrival_outlet_s"] is not None else None,
    **{k:base[k] for k in ("v_median_m_s","v_max_m_s","h_median_m","I_max","I_median")}}}
for key, group in rows.items():
    for j, record in enumerate(group):
        signature = json.dumps(record["params"], sort_keys=True)
        unique.setdefault(signature, {"id": f"{key}-{j}", **record,
                          "arrival_outlet_s": (record["到达口岸_min"] * 60
                                               if record["到达口岸_min"] is not None else None)})
n_scenarios = len(unique)
unique_runs = list(unique.values())
n_dup_baseline = sum(len(v) for v in sens.values()) - n_scenarios
save_json({"基准参数": PARAMS, "扰动设置": sens, "敏感性结果": rows,
           "unique_runs": unique_runs,
           "敏感性情境总数_独立": n_scenarios,
           "敏感性情境总数_含重复基线": sum(len(v) for v in sens.values()),
           "重复基线计数": n_dup_baseline,
           "情境计数口径": "按完整参数字典去重；扫描表可含重复基线",
           "μ水平锚定": (f"μ 水平 = {{0.7×, 0.85×, 1×, 1.2×}}·μb（μb="
```

```
                    f"{PARAMS['mu']:.6f}，采用值：大+特大事件的条件中位数）；"
                    "到达口径：未到达情境记 None"),
            "触顶情境": {k: [r["取值"] for r in rows[k] if r["运程触顶"]]
                      for k in rows if any(r["运程触顶"] for r in rows[k])},
            "触顶口径": ("运程触顶：runout_km ≥ 0.999×total_km（前锋抵达"
                    "中心线末端/口岸，Heim 判据未自然停止）；"
                    "与 h/v 截断的触顶节点数分开登记"),
            "未到达情境": {k: [r["取值"] for r in rows[k]
                      if not r["reached_outlet"]]
                    for k in rows
                    if any(not r["reached_outlet"] for r in rows[k])},
            "未到达口径": "reached_outlet=False 的情境：其到达时间不进入任何统计"},
          "04_敏感性.json")

    # 结果区间
    def spread(sub):
        vs = [r[sub] for r in unique_runs if r.get(sub) is not None]
        return {"min": float(np.min(vs)), "p50": float(np.median(vs)),
                "max": float(np.max(vs))} if vs else {}

    n_noreach = sum(not r["reached_outlet"] for r in unique_runs)
    n_reach_arr = sum(r["reached_outlet"] for r in unique_runs)
    save_json({
        "runout_km": spread("runout_km"),
        "到达口岸_min": spread("到达口岸_min"),
        "到达口岸_min_口径": (f"仅统计到达情境（reached_outlet=True，"
                    f"n={n_reach_arr}）；未到达情境（{n_noreach} 个）"
                    "记 None，不以 0 或运动学外推值计入"),
        "未到达情境数": n_noreach,
        "v_median_m_s": spread("v_median_m_s"),
        "v_max_m_s": spread("v_max_m_s"),
        "h_median_m": spread("h_median_m"),
        "I_max": spread("I_max"),
        "I_median": spread("I_median"),
        "基线": {"runout_km": base["runout_km"],
              "到达口岸_min": base["arrival_outlet_s"]/60 if base["arrival_outlet_s"] is not None else None,
              "v_median_m_s": base["v_median_m_s"],
              "v_max_m_s": base["v_max_m_s"],
              "h_median_m": base["h_median_m"],
              "I_max": base["I_max"],
              "I_median": base["I_median"]},
        "敏感性情境总数_独立": n_scenarios,
        "敏感性情境总数_含重复基线": sum(len(v) for v in sens.values()),
    }, "04_结果区间.json")

    print("[6/6] 场量导出与图件…")
    dem, grid = io.load_dem()
    cell_area = abs(grid["dlon"]) * 111320.0 * np.cos(np.radians(28.4)) * \
                abs(grid["dlat"]) * 110574.0
    inten_field = base["强度场"]
    np.savez_compressed(CACHE / "sim_fields.npz",
                        intensity=inten_field,
                        s_km=cl["s_m"] / 1000.0, z_m=cl["z_m"],
                        h_m=base["h_m"], v_m_s=base["v_m_s"],
                        t_s=np.nan_to_num(base["t_s"], nan=-1.0),
                        I=base["intensity"])
    save_json({"走廊像元数": int((inten_field > 0).sum()),
               "走廊面积_km2": base["走廊面积_km2"],
               "最大强度_N_m": float(np.max(inten_field)),
               "Cv": base["Cv"], "rho_m": base["rho_m"],
               "强度指标": "I = ρ_m·h·v²（单位宽度冲击力，N·m⁻¹）",
               "分级（走廊内）": base["强度分级_走廊"],
               "分级（中心线断面）": base["强度分级_中心线"]},
              "04_强度场.json")

    print("完成")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

step04b_sim2d.py　D8 网络路由与质量收支

```
"""D8 network routing on a geographic grid; conservative scenario simulation."""
from __future__ import annotations

import json
from pathlib import Path
import numpy as np
from mmlib import geodesy as gd, io_data as io, terrain as tr, terrain_prep as tp

OUT, CACHE, FIG = io.DATA_OUT, io.CACHE, io.FIG_OUT
P2D = dict(n_man=.10, tau_y=800., s_min=.02, tau_src=1800., Cv=.45,
           rho_s=2650., rho_w=1000., h_min=.10, T_end=21600., eps_mass=1e-8,
           cfl=.20, dt_max=1.0)
SNAP_TIMES = (600, 1200, 1800, 3600)
SENS2D = {'n_low': {'n_man': .08}, 'n_high': {'n_man': .12},
          'tauy_low': {'tau_y': 400.}, 'tauy_high': {'tau_y': 1600.},
          'Cv_low': {'Cv': .15}, 'Cv_high': {'Cv': .80},
```

```
        'smin_low': {'s_min': .01}, 'smin_high': {'s_min': .04},
        'duration_low': {'tau_src': 900.}, 'duration_high': {'tau_src': 2700.},
        'arrival_low': {'h_min': .05}, 'arrival_high': {'h_min': .20}}


def rho_df_of(cv):
    return cv * P2D['rho_s'] + (1-cv) * P2D['rho_w']


def load_volumes():
    return json.loads((OUT / '04_初始体积.json').read_text(encoding='utf-8'))


def glacier_src_region():
    gn = io.load_glacier_nc()
    v = load_volumes()['链A']
    mask = (np.isfinite(gn['thickness']) & (gn['thickness'] >= v['h_thr_m']) &
            np.isfinite(gn['slope']) & (gn['slope'] >= v['sl_thr_deg']))
    gt = gn['geotransform']
    return mask, abs(gt[1]*gt[5]), gt


def reproject_src_cells(mask, gt, grid, shape):
    rr, cc = np.nonzero(mask)
    cells = []
    for r, c in zip(rr, cc):
        lo, la = gd.utm_to_lonlat(gt[0]+(c+.5)*gt[1], gt[3]+(r+.5)*gt[5], 45)
        ri = int(np.floor((la-grid['lat0'])/grid['dlat']+.5))
        ci = int(np.floor((lo-grid['lon0'])/grid['dlon']+.5))
        if 0 <= ri < shape[0] and 0 <= ci < shape[1]:
            cells.append(ri*shape[1]+ci)
    return np.unique(cells).astype('int64'), {
        '源区像元数_冰川网格': int(mask.sum()), '落域像元次': len(cells),
        '落域去重像元数': len(set(cells)), '落域比例': len(cells)/max(mask.sum(), 1)}


def project_sections(terrain, source, target_lonlat=None):
    ny, nx = terrain['filled'].shape
    fin = np.isfinite(terrain['filled']).ravel()
    source = source[fin[source]]
    if not source.size:
        raise ValueError('source geometry unavailable')
    inj = int(source[np.argmax(terrain['acc'].ravel()[source])])
    path = tr.trace_downstream(terrain['dir'], np.unravel_index(inj, (ny, nx)))
    grid, (dy, dx) = terrain['grid'], terrain['cell']
    lon, lat = target_lonlat or (io.EVENT['port_lon'], io.EVENT['port_lat'])
    rp, cp = io.lonlat_to_rowcol(grid, lon, lat)
    offsets = np.hypot((np.array([r for r,c in path])-rp)*dy,
                       (np.array([c for r,c in path])-cp)*dx)
    port_rc = path[int(np.argmin(offsets))]
    border = np.zeros((ny,nx),bool)
    border[[0,-1],:] = True
    border[:,[0,-1]] = True
    out = int(np.argmax(np.where(border & fin.reshape(ny,nx), terrain['acc'], -1)))
    return inj, int(port_rc[0]*nx+port_rc[1]), out, float(offsets.min())


def flow_length_from(source, recv, dist_flat):
    length = np.full(recv.size, np.inf)
    for start in source:
        i, d, seen = int(start), 0., set()
        while i >= 0 and i not in seen:
            seen.add(i)
            length[i] = min(length[i], d)
            nxt = int(recv[i])
            if nxt < 0:
                break
            d += float(dist_flat[i])
            i = nxt
    return length


def cumulative_source(t, volume, duration):
    u = np.clip(t/duration, 0., 1.)
    return volume*(2*u-u*u)


def simulate2d(recv, slink, dist, cell, src_px, inj_px, V_df, over,
               port_idx, out_idx, FL=None, keep_fields=False):
    p = {**P2D, **over}
    dy, dx = cell
    area, width = dy*dx, dx
    N = recv.size
    slope = np.maximum(np.nan_to_num(slink.ravel(), nan=p['s_min']), p['s_min'])
    sqrtS = np.sqrt(slope)
    density = rho_df_of(p['Cv'])
    critical = p['tau_y']/(density*9.81)
    h = np.zeros(N)
    arrival = np.full(N, np.inf)
    hp = np.zeros(N, dtype='float32') if keep_fields else None
    vp = np.zeros(N, dtype='float32') if keep_fields else None
    Ip = np.zeros(N, dtype='float32') if keep_fields else None
    inj_px = np.unique(inj_px)
    active = inj_px.copy()
    t, step, vout, vdom = 0., 0, 0., 0.
    cfl_max, min_dt, min_h, closure_max = 0., np.inf, 0., 0.
```

```
   src_err, trunc_steps, volume_check_error = 0., 0, 0.
   front, peak_port, peak_global, peak_port_v, peak_port_I = 0.,0.,0.,0.,0.
   ledger, leader, port_curve, snapshots = [], [], [], {}
   termination = 'time_limit'

   def append_record():
      injected = cumulative_source(t, V_df, p['tau_src'])
      ledger.append(dict(t_s=t, V_inj_m3=float(injected), V_dom_m3=float(vdom),
                     V_out_m3=float(vout), V_loss_m3=float(injected-vdom-vout)))
      leader.append(dict(t_s=t, front_m=front))
      vel = (h[port_idx]**(2/3)*sqrtS[port_idx]/p['n_man']
           if h[port_idx]*slope[port_idx] >= critical else 0.)
      port_curve.append(dict(t_s=t, h_m=float(h[port_idx]), v_m_s=float(vel),
                        I_N_m=float(density*h[port_idx]*vel*vel)))

   while t < p['T_end']-1e-8 and step < 500000:
      active = np.unique(np.concatenate((active,inj_px)))
      remaining_snaps = [s for s in SNAP_TIMES if s > t+1e-8]
      dt = min(p['dt_max'], p['T_end']-t,
             (remaining_snaps[0]-t) if remaining_snaps else np.inf,
             (p['tau_src']-t) if t < p['tau_src']-1e-8 else np.inf)
      # Include analytic injection in the state used to select the CFL step.
      for attempt in range(60):
         dv = float(cumulative_source(t+dt,V_df,p['tau_src'])-
                 cumulative_source(t,V_df,p['tau_src']))
         hh = h[active].copy()
         if dv:
            hh[np.searchsorted(active,inj_px)] += dv/(inj_px.size*area)
         vv = hh**(2/3)*sqrtS[active]/p['n_man']
         co = (5/3)*vv*width*dt/area
         maxco = float(co.max()) if co.size else 0.
         if maxco <= p['cfl']*(1+1e-10):
            break
         dt *= max(.1, .90*p['cfl']/maxco)
      else:
         raise RuntimeError('source-aware CFL failed to find a step')
      if dt <= 1e-10:
         raise RuntimeError('CFL step too small')
      min_dt, cfl_max = min(min_dt,dt), max(cfl_max,maxco)
      h[inj_px] += dv/(inj_px.size*area)
      vdom += dv
      hw = h[active]
      canflow = (hw*slope[active] >= critical) & (hw > 0.)
      vel = np.where(canflow,hw**(2/3)*sqrtS[active]/p['n_man'],0.)
      raw = vel*hw*width
      cap = hw*area/dt
      trunc_steps += int(np.any(raw > cap*(1+1e-10)))
      flow = np.minimum(raw,cap)
      inside = recv[active] >= 0
      rout = float(flow[~inside].sum())*dt
      targets = recv[active[inside]]
      touched = np.unique(np.concatenate((active,targets)))
      gain = np.bincount(np.searchsorted(touched,targets), weights=flow[inside],
                     minlength=touched.size)
      loss = np.bincount(np.searchsorted(touched,active), weights=flow,
                     minlength=touched.size)
      if keep_fields:
         hp[active] = np.maximum(hp[active],hw)
         vp[active] = np.maximum(vp[active],vel)
         Ip[active] = np.maximum(Ip[active],density*hw*vel*vel)
      peak_global = max(peak_global,float(hw.max()))
      j = np.searchsorted(active,port_idx)
      if j < active.size and active[j] == port_idx:
         peak_port = max(peak_port,float(hw[j]))
         peak_port_v = max(peak_port_v,float(vel[j]))
         peak_port_I = max(peak_port_I,float(density*hw[j]*vel[j]**2))
      h[touched] += (gain-loss)*dt/area
      vout += rout
      vdom -= rout
      min_h = min(min_h,float(h[touched].min()))
      if min_h < -1e-9:
         raise RuntimeError('negative depth violates positive update')
      t += dt
      step += 1
      newly = touched[(h[touched] >= p['h_min']) & np.isinf(arrival[touched])]
      arrival[newly] = t
      if newly.size and FL is not None:
         front = max(front,float(np.max(FL[newly][np.isfinite(FL[newly])])
                   if np.isfinite(FL[newly]).any() else 0.)
      active = touched[h[touched] > 1e-12]
      if keep_fields:
         vv = np.where(h[touched]*slope[touched] >= critical,
                    h[touched]**(2/3)*sqrtS[touched]/p['n_man'],0.)
         hp[touched] = np.maximum(hp[touched],h[touched])
         vp[touched] = np.maximum(vp[touched],vv)
         Ip[touched] = np.maximum(Ip[touched],density*h[touched]*vv*vv)
      peak_global=max(peak_global,float(h[touched].max()))
      peak_port=max(peak_port,float(h[port_idx]))
      postv=(h[port_idx]**(2/3)*sqrtS[port_idx]/p['n_man']
           if h[port_idx]*slope[port_idx]>=critical else 0.)
      peak_port_v=max(peak_port_v,float(postv))
      peak_port_I=max(peak_port_I,float(density*h[port_idx]*postv**2))
      at_snap=any(abs(t-st)<1e-6 for st in SNAP_TIMES)
      if step % 250 == 0 or at_snap:
         actual_dom=float(h.sum())*area
         volume_check_error=max(volume_check_error,abs(actual_dom-vdom)/V_df)
         vdom=actual_dom
```

```
      injected=float(cumulative_source(t,V_df,p['tau_src']))
      closure_max=max(closure_max,abs(injected-vdom-vout)/max(injected,1.))
      if step % 25 == 0 or at_snap:
         append_record()
      if at_snap and keep_fields:
         snapshots[f't{int(round(t))}']=h.copy()
      if step % 250 == 0 and t >= max(SNAP_TIMES) and t > p['tau_src']:
         mobile=h[active][h[active]*slope[active]>=critical].sum()*area
         if mobile <= 1e-7*V_df:
            termination='stationary_after_injection'
            break
   if step >= 500000:
      termination='step_limit'
   actual_dom=float(h.sum())*area
   volume_check_error=max(volume_check_error,abs(actual_dom-vdom)/V_df)
   vdom=actual_dom
   append_record()
   injected=float(cumulative_source(t,V_df,p['tau_src']))
   closure_max=max(closure_max,abs(injected-vdom-vout)/max(injected,1.))
   src_err=abs(injected-V_df)/V_df if t >= p['tau_src'] else None
   if closure_max>p['eps_mass'] or min_h < -1e-9 or cfl_max > p['cfl']*(1+1e-8):
      raise RuntimeError('numerical acceptance failed')
   if src_err is not None and src_err > 1e-10:
      raise RuntimeError('analytic source volume acceptance failed')
   reached=np.isfinite(arrival[port_idx])
   domain_front_max = (
      float(np.nanmax(FL[np.isfinite(FL)]))
      if FL is not None and np.isfinite(FL).any()
      else None
   )
   domain_censor = bool(
      domain_front_max is not None
      and np.isfinite(front)
      and front >= 0.999 * domain_front_max
   )
   loop_termination = termination
   if loop_termination not in {"stationary_after_injection", "time_limit", "step_limit"}:
      raise RuntimeError(f"unknown terminal reason: {loop_termination}")
   # A domain-boundary endpoint is right-censored regardless of what the
   # mobile-volume check did on the last iteration; it is never a natural stop.
   if domain_censor:
      termination = "domain_censor"
   natural_stop = bool(loop_termination == "stationary_after_injection" and not domain_censor)
   time_limit = loop_termination == "time_limit"
   step_limit = loop_termination == "step_limit"
   runout_right_censored = bool(domain_censor or time_limit or step_limit)
   critical_depth_port = critical / max(float(slope[port_idx]), float(p['s_min']))
   result=dict(arrival_port_s=float(arrival[port_idx]) if reached else None,
            arrival_outlet_s=float(arrival[out_idx]) if np.isfinite(arrival[out_idx]) else None,
            peak_depth_port_m=peak_port,final_depth_port_m=float(h[port_idx]),
            peak_velocity_port_m_s=peak_port_v,peak_intensity_port_N_m=peak_port_I,
            max_depth_m=peak_global,final_max_depth_m=float(h.max()),
            wet_area_km2=float((h>.05).sum())*area/1e6,
            deposit_area_km2=float(((h>.05)&(h*slope<critical)).sum())*area/1e6,
            affected_area_km2=float(np.isfinite(arrival).sum())*area/1e6,
            n_steps=step,t_end_sim_s=t,termination_reason=termination,
            loop_termination_reason=loop_termination,
            natural_stop=natural_stop,time_limit=time_limit,
            step_limit=step_limit,domain_censor=domain_censor,
            reached_port=bool(reached),rho_df=density,
            critical_depth_slope_product_m=float(critical),
            critical_depth_port_m=float(critical_depth_port),
            params=p,V_df_m3=V_df,ledger=ledger,leader=leader,
            port_hydrograph=port_curve,closure_max=closure_max,
            source_relative_error=src_err,actual_cfl_max=cfl_max,min_dt_s=min_dt,
            nonnegative_min=min_h,volume_check_relative_error=volume_check_error,
            flux_trunc_step_frac=trunc_steps/max(step,1),runout_L_m=front,
            runout_right_censored=runout_right_censored,
            domain_front_max_m=domain_front_max,
            runout_semantics=(
               "historical first-wetting front; at/near domain maximum is a right-censored "
               "analysis extent, not a natural stopping distance"
            ),
            smin_path_fraction=float(np.mean(np.nan_to_num(slink.ravel()[np.isfinite(FL)],nan=0)<p['s_min']))
            if FL is not None else None)
   if keep_fields:
      shape=slink.shape
      result.update(arrival=arrival.reshape(shape),final_h=h.reshape(shape),
                  peak_h=hp.reshape(shape),peak_v=vp.reshape(shape),peak_I=Ip.reshape(shape),
                  snapshots={k:a.reshape(shape) for k,a in snapshots.items()})
   return result


def main():
   terrain=tp.ensure_terrain_cache()
   dem,grid=io.load_dem()
   terrain['grid']=grid
   vol=load_volumes()
   mask,_,gt=glacier_src_region()
   source,stats=reproject_src_cells(mask,gt,grid,dem.shape)
   inj,port,out,offset=project_sections(terrain,source)
   expected=vol['共同断面']
   if list(np.unravel_index(inj,dem.shape)) != expected['注入_rc'] or list(np.unravel_index(port,dem.shape)) !=
        expected['口岸_rc']:
      raise ValueError('one/two dimensional common sections mismatch')
   distance=flow_length_from(np.array([inj]),terrain['recv'],terrain['dist'].ravel())
```

```
    configs={'base': ({},vol['链A_V_df_m3']),
           'V_low': ({},.5*vol['链A_V_df_m3']),
           'V_high': ({},1.5*vol['链A_V_df_m3']),
           'V_chainB': ({},vol['链B_V0_m3'])}
    configs.update({k:(v,vol['链A_V_df_m3']) for k,v in SENS2D.items()})
    runs,seen={},{}
    basefull=None
    for name,(over,V) in configs.items():
        signature=json.dumps({'params':{**P2D,**over},'V_df_m3':V},sort_keys=True)
        if signature in seen:
            continue
        seen[signature]=name
        result=simulate2d(terrain['recv'],terrain['slink'],terrain['dist'].ravel(),terrain['cell'],
                          source,np.array([inj]),V,over,port,out,FL=distance,keep_fields=name=='base')
        if name=='base':
            basefull=result
        runs[name]={k:v for k,v in result.items() if k not in ('arrival','final_h','peak_h','peak_v','peak_I','
             snapshots')}
        print(name,result['arrival_port_s'],result['closure_max'],flush=True)
    for field in ('arrival','final_h','peak_h','peak_v','peak_I'):
        np.save(CACHE/f'q4b_{field}.npy',basefull[field].astype('float32'))
    for key,value in basefull['snapshots'].items():
        np.save(CACHE/f'q4b_snap_{key}.npy',value.astype('float32'))
    np.save(CACHE/'q4b_cellmeta.npy',np.array([*terrain['cell'],*dem.shape]))
    (CACHE/'q4b_leader.json').write_text(json.dumps(basefull['leader']),encoding='utf-8')
    coarse=tp.prepare_coarse(k=3)
    src9,_=reproject_src_cells(mask,gt,coarse['grid'],coarse['filled'].shape)
    i9,p9,o9,offset9=project_sections(coarse,src9)
    r9=simulate2d(coarse['recv'],coarse['slink'],coarse['dist'].ravel(),coarse['cell'],src9,np.array([i9]),
                  vol['链A_V_df_m3'],{},p9,o9)
    rt=simulate2d(terrain['recv'],terrain['slink'],terrain['dist'].ravel(),terrain['cell'],source,np.array([inj]),
                  vol['链A_V_df_m3'],{'cfl':P2D['cfl']/2},port,out,FL=distance)
    result={'model':'D8 network routing, not lateral 2D dynamics','grid':grid,
            '基准参数':P2D,'源区落域统计':stats,'runs':runs,'scale90_rerun':r9,'time_refinement':rt,
            '断面':{'注入_rc':list(np.unravel_index(inj,dem.shape)),
                    '口岸投影_rc':list(np.unravel_index(port,dem.shape)),
                    '投影偏移_m':offset,'90m口岸偏移_m':offset9,
                    '全域出口_rc':list(np.unravel_index(out,dem.shape))},
            '敏感性情境总数_独立':len(runs),'到达口岸样本_s':[v['arrival_port_s'] for v in runs.values() if v['reached_port'
                 ]],
            'not_reached':[k for k,v in runs.items() if not v['reached_port']],
            '账本验收':{'eps_mass':P2D['eps_mass'],'全部情景闭合比':{k:v['closure_max'] for k,v in runs.items()}},
            'scale_interpretation':'grid-dependent width and route; not a convergence claim'}
    (OUT/'04b_二维引擎.json').write_text(json.dumps(result,ensure_ascii=False,indent=2,default=lambda x:x.item()),
         encoding='utf-8')
    return 0


if __name__=='__main__':
    raise SystemExit(main())
```

step04b_map.py　到达时间与停积制图

```
"""Arrival-time field and terminal depth map for the D8 routing baseline."""
from __future__ import annotations
import json
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.colors import LightSource, Normalize
from matplotlib.collections import LineCollection
from matplotlib.ticker import FormatStrFormatter
from mmlib import io_data as io

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams['mathtext.fontset'] = 'dejavusans'
plt.rcParams.update({
    'font.size': 12.5,
    'axes.titlesize': 12,
    'axes.labelsize': 11,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'legend.fontsize': 10,
})


def main():
    q = json.loads((io.DATA_OUT / '04b_二维引擎.json').read_text(encoding='utf-8'))
    dem, g = io.load_dem()
    cache, fig = io.CACHE, io.FIG_OUT
    arrival = np.load(cache / 'q4b_arrival.npy')
    final = np.load(cache / 'q4b_final_h.npy')
    recv = np.load(cache / 'recv.npy')
    wet = np.isfinite(arrival)
    rr, cc = np.nonzero(wet)
    pad = 25
    r0, r1 = max(0, rr.min() - pad), min(dem.shape[0], rr.max() + pad)
    c0, c1 = max(0, cc.min() - pad), min(dem.shape[1], cc.max() + pad)
    sl = np.s_[r0:r1, c0:c1]
    extent = [g['lon0'] + c0 * g['dlon'], g['lon0'] + c1 * g['dlon'],
              g['lat0'] + r1 * g['dlat'], g['lat0'] + r0 * g['dlat']]
```

```

    inj = tuple(q['断面']['注入_rc'])
    port = tuple(q['断面']['口岸投影_rc'])
    ncol = dem.shape[1]
    path = [inj]
    cur = inj
    reached = False
    for _ in range(20000):
        nxt = int(recv[cur[0] * ncol + cur[1]])
        if nxt < 0:
            break
        cur = (nxt // ncol, nxt % ncol)
        path.append(cur)
        if cur == port:
            reached = True
            break
    lat = [g['lat0'] + r * g['dlat'] for r, _ in path]
    lon = [g['lon0'] + c * g['dlon'] for _, c in path]
    tmin = np.array([arrival[r, c] / 60.0 if wet[r, c] else np.nan for r, c in path])
    segs = [((lon[i], lat[i]), (lon[i + 1], lat[i + 1])) for i in range(len(lon) - 1)]
    seg_t = np.array([np.nanmean(tmin[i:i + 2]) for i in range(len(lon) - 1)])

    ls = LightSource(azdeg=315, altdeg=45)
    shade = ls.hillshade(dem[sl], vert_exag=3.0, dx=g['dlon'], dy=g['dlat'])
    norm = Normalize(vmin=0.0, vmax=float(np.nanmax(tmin)) if np.isfinite(tmin).any() else 1.0)

    f = plt.figure(figsize=(7.6, 3.05), constrained_layout=True)
    gs = f.add_gridspec(1, 4, width_ratios=[1.0, 0.030, 1.0, 0.030])
    ax0 = f.add_subplot(gs[0, 0])
    cax0 = f.add_subplot(gs[0, 1])
    ax1 = f.add_subplot(gs[0, 2])
    cax1 = f.add_subplot(gs[0, 3])
    for ax in (ax0, ax1):
        ax.imshow(shade, extent=extent, cmap='gray', alpha=.35, origin='upper')

    im0 = ax0.imshow(np.ma.masked_less_equal(np.where(wet, arrival / 60.0, 0.0)[sl], 0),
                     extent=extent, cmap='viridis', norm=norm, alpha=.95)
    lc = LineCollection(segs, cmap='viridis', norm=norm, linewidths=1.1)
    lc.set_array(seg_t)
    ax0.add_collection(lc)
    cb0 = f.colorbar(im0, cax=cax0)
    cb0.set_label('到达时间 / min', fontsize=11)
    cb0.ax.tick_params(labelsize=8)
    ax0.set_title('首达时间场与仿真流路', fontsize=13)

    vmax = float(np.nanmax(final))
    lon_w = g['lon0'] + cc * g['dlon']
    lat_w = g['lat0'] + rr * g['dlat']
    im1 = ax1.scatter(lon_w, lat_w, c=final[rr, cc], cmap='inferno', vmin=0, vmax=vmax,
                      s=9, marker='s', linewidths=0, alpha=.95)
    cb1 = f.colorbar(im1, cax=cax1)
    cb1.set_label('终态泥深 / m', fontsize=11)
    cb1.ax.tick_params(labelsize=8)
    ax1.set_title('分析域末端终态泥深（右截断）', fontsize=13)

    for ax in (ax0, ax1):
        ax.plot(g['lon0'] + inj[1] * g['dlon'], g['lat0'] + inj[0] * g['dlat'],
                'o', color='royalblue', ms=5.5, mec='white', mew=.8, label='共同注入断面')
        ax.plot(g['lon0'] + port[1] * g['dlon'], g['lat0'] + port[0] * g['dlat'],
                '*', color='crimson', ms=10, mec='white', mew=.8, label='口岸代理断面')
        ax.set_xlabel('经度 / °E', fontsize=11)
        ax.set_ylabel('纬度 / °N', fontsize=11)
        ax.set_xticks([85.38, 85.42, 85.46, 85.50])
        ax.set_yticks([28.26, 28.28, 28.30, 28.32, 28.34])
        ax.xaxis.set_major_formatter(FormatStrFormatter('%.2f'))
        ax.yaxis.set_major_formatter(FormatStrFormatter('%.2f'))
        ax.tick_params(labelsize=8)
        ax.legend(fontsize=11, loc='upper left', framealpha=.85)
    f.savefig(fig / '04b_到达与停积.png', dpi=220)
    plt.close(f)
    print(json.dumps({'reached_port_proxy': reached, 'path_cells': len(path),
                      'arrival_max_min': float(np.nanmax(tmin)),
                      'final_max_m': vmax}, ensure_ascii=False))
    return 0


if __name__ == '__main__':
    raise SystemExit(main())
```

`step04c_dynamics.py`　有限库存与分段动力

```
"""问题4：有限库存分段动力对照。只读旧04输入，独立写04c结果。"""
from __future__ import annotations

import hashlib
import itertools
import json
import math
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
```

```
from sklearn.isotonic import IsotonicRegression

from mmlib import finite_inventory as fi
from mmlib import io_data as io
from test04c_dynamics import run_report

plt.rcParams.update({"font.sans-serif": ["Microsoft YaHei", "DejaVu Sans"],
                     "axes.unicode_minus": False, "mathtext.fontset": "dejavusans",
                     "font.size": 20, "axes.titlesize": 20, "axes.labelsize": 18,
                     "xtick.labelsize": 15, "ytick.labelsize": 15,
                     "legend.fontsize": 13, "savefig.dpi": 180})

DOMAIN = {"mu_dyn": [.05, .1, .15, .2], "kappa_m_inv": [0., 1e-5, 5e-5, 1e-4],
          "v0_m_s": [math.sqrt(200), 30., 60.], "q_r": [0., .5, 1.], "q_w": [0., .5, 1.],
          "baseline": {"mu": .1, "kappa": 1e-5, "v0": 30.},
          "node_horizontal_fractions": [1/3, 2/3], "node_weights": [.5, .5],
          "M2_mu_multiplier": .6, "target_vreq_m_s": 1.,
          "inverse_relative_perturbation": 1e-6}
LABELS = {"raw": "原始剖面", "filled": "填洼剖面", "isotonic": "单调剖面"}
COLORS = {"raw": "#a14332", "filled": "#27767d", "isotonic": "#283d65"}


def sha(path):
    h = hashlib.sha256()
    with Path(path).open("rb") as stream:
        for block in iter(lambda: stream.read(1024*1024), b""):
            h.update(block)
    return h.hexdigest()


def save(obj, name):
    def convert(value):
        if isinstance(value, np.ndarray):
            return value.tolist()
        if isinstance(value, np.generic):
            return value.item()
        raise TypeError(type(value).__name__)
    p = io.DATA_OUT/name
    p.write_text(json.dumps(obj, ensure_ascii=False, indent=2, default=convert,
                            allow_nan=False), encoding="utf-8")
    return p


def summary(result):
    return {k: v for k, v in result.items()
            if k not in ("trajectory", "segment_ledger", "node_ledger")}


def load_geometry():
    required = [io.DATA_OUT/"04_仿真基线.json", io.DATA_OUT/"04_初始体积.json",
                io.CACHE/"dem.npy", io.CACHE/"filled.npy", io.CACHE/"terrain_meta.json"]
    for p in required:
        if not p.is_file():
            raise FileNotFoundError(f"先运行原04与地形缓存，缺少 {p}")
    old = json.loads(required[0].read_text(encoding="utf-8"))
    volume = json.loads(required[1].read_text(encoding="utf-8"))
    meta = json.loads(required[-1].read_text(encoding="utf-8"))
    path = np.asarray(old["中心线"]["path_rc"], int)
    dem, filled = np.load(required[2], mmap_mode="r"), np.load(required[3], mmap_mode="r")
    delta = np.diff(path, axis=0)
    if np.any(np.max(np.abs(delta), axis=1) > 1) or np.any(np.all(delta == 0, axis=1)):
        raise ValueError("原生D8路线不是相邻且非重复的像元序列")
    dh = np.hypot(delta[:, 0]*meta["cell_dy_m"], delta[:, 1]*meta["cell_dx_m"])
    sh = np.r_[0., dh.cumsum()]
    if not math.isclose(sh[-1]/1000, old["total_km"], abs_tol=1e-6):
        raise ValueError("原生水平里程与旧共同路线不一致")
    zr, zf = np.asarray(dem[path[:, 0], path[:, 1]], float), np.asarray(filled[path[:, 0], path[:, 1]], float)
    zi = IsotonicRegression(increasing=False).fit_transform(np.arange(len(zr)), zr)
    profiles = {"raw": zr, "filled": zf, "isotonic": zi}
    if not all(np.all(np.isfinite(z)) for z in profiles.values()):
        raise ValueError("路线存在缺测高程")
    cv, vdf = float(old["Cv"]), float(volume["采用_V0_m3"])
    initial = np.array([cv*vdf, 0., (1-cv)*vdf])
    if not math.isclose(fi.mass(initial)/vdf, old["rho_m"], abs_tol=1e-6):
        raise ValueError("等效岩水初值与旧密度不一致")
    details = {"native_node_count": len(path), "horizontal_length_m": sh[-1],
               "native_path_rc": path.tolist(), "route_s_h_m": sh.tolist(),
               "profile_z_m": {k: z.tolist() for k, z in profiles.items()},
               "horizontal_length_source": "native D8 row/column differences with terrain_meta metric cells",
               "profiles": {k: {"three_dimensional_length_m": float(np.hypot(dh, np.diff(z)).sum()),
                                "uphill_steps": int(np.sum(np.diff(z) > 0)),
                                "drop_m": float(z[0]-z[-1])} for k, z in profiles.items()},
               "start_lonlat": old["中心线"]["start_lonlat"],
               "port_proxy_lonlat": old["中心线"]["end_lonlat"],
               "port_offset_m": old["中心线"]["port_offset_m"],
               "original_isotonic_max_adjustment_m": float(np.max(abs(zi-zr))),
               "old_diagnostic": {k: old[k] for k in ["runout_km", "arrival_outlet_s", "stop_time_s", "mu"]},
               "initial_rock_ice_water_m3": initial.tolist(), "initial_Cv": cv,
               "initial_mass_kg": fi.mass(initial),
               "Vi_zero_semantics": "equivalent rock-water representation; not evidence all real ice melted",
               "gamma_account": "Vdf already includes inlet enrichment; synthetic stocks are separate hypothetical
                    downstream accounts",
               "input_sha256": {str(p.relative_to(io.ROOT)): sha(p) for p in required}}
    return sh, profiles, initial, details

```

```


def artificial_nodes(sh, initial, qr, qw):
    if qr == 0 and qw == 0:
        return []
    return [{"id": f"artificial_{j+1}", "s_h_m": sh[-1]*fraction,
             "rock_m3": qr*initial[0]*.5, "water_m3": qw*initial[2]*.5}
            for j, fraction in enumerate(DOMAIN["node_horizontal_fractions"])]


def checked_inverse(sh, z, initial, **kwargs):
    inv = fi.inverse_threshold(sh, z, initial, vreq=1., **kwargs)
    v = inv["required_v0_m_s"]
    if v is None or v <= 0:
        inv["forward_check"] = {"status": "zero_or_unrepresentable_threshold_needs_start_rule"}
        return inv
    eps = DOMAIN["inverse_relative_perturbation"]
    checks = {}
    for tag, vv in [("below", v*(1-eps)), ("above", v*(1+eps))]:
        r = fi.simulate(sh, z, initial, v0=vv, **kwargs)
        success = r["reached_target_positive"] and r["final_velocity_m_s"] >= 1.
        checks[tag] = {"v0_m_s": vv, "status": r["status"],
                       "velocity_target_or_stop_m_s": r["final_velocity_m_s"],
                       "meets_vreq": bool(success)}
    inv["forward_check"] = checks
    inv["forward_check_pass"] = not checks["below"]["meets_vreq"] and checks["above"]["meets_vreq"]
    if not inv["forward_check_pass"]:
        raise AssertionError(f"逆向阈值未通过前向核验: {checks}")
    inv["required_v0_within_declared_scan_range"] = DOMAIN["v0_m_s"][0] <= v <= DOMAIN["v0_m_s"][-1]
    return inv


def water_competition(result):
    records = []
    for node in result["node_ledger"]:
        downstream = [s for s in result["segment_ledger"] if s["s0_m"] >= node["s_h_m"]-1e-9]
        if not downstream:
            continue
        seg = downstream[0]
        if seg["a_m_s2"] >= 0:
            records.append({"node_id": node["id"], "status": "immediate_segment_not_decelerating"})
            continue
        b, k, length = -seg["a_m_s2"], seg["kappa_m_inv"], seg["length_m"]
        crit2 = b/k*math.expm1(2*k*length) if k else 2*b*length
        vpre = node["velocity_after_m_s"]*(1+node["lambda"])
        dv = node["consumed_volumes_m3"]
        other = fi.RHO_R*dv[0]+fi.RHO_I*dv[1]
        upper = (node["mass_before_kg"]*(vpre/math.sqrt(crit2)-1)-other)/fi.RHO_W
        records.append({"node_id": node["id"], "status": "fixed_actual_segment_local_necessary_bound",
                        "segment_s1_m": seg["s1_m"], "vcrit_m_s": math.sqrt(crit2),
                        "strict_added_water_upper_m3": upper, "actual_added_water_m3": dv[2],
                        "R_local": vpre*vpre/((1+node["lambda"])**2*crit2),
                        "not_global_or_true_material_boundary": True})
    return records


def real_route_refinement(sh, profiles, initial):
    checks = []
    for name, z in profiles.items():
        for mode in ["M0", "M1", "M2"]:
            nodes = artificial_nodes(sh, initial, .5, .5)
            a = fi.simulate(sh, z, initial, nodes=nodes, mode=mode, **DOMAIN["baseline"])
            b = fi.simulate(sh, z, initial, nodes=nodes, mode=mode,
                            subdivisions=5, **DOMAIN["baseline"])
            entry = {"profile": name, "mode": mode, "refinement_factor": 5,
                     "horizontal_distance_difference_m": b["final_sh_m"]-a["final_sh_m"],
                     "time_difference_s": b["final_time_s"]-a["final_time_s"],
                     "velocity_difference_m_s": b["final_velocity_m_s"]-a["final_velocity_m_s"],
                     "same_event": a["status"] == b["status"],
                     "same_consumed_node_count": len(a["node_ledger"]) == len(b["node_ledger"]),
                     "same_consumed_inventory_mass": a["consumed_inventory_mass_kg"] == b["consumed_inventory_mass_kg"]}
            entry["pass"] = (entry["same_event"] and entry["same_consumed_node_count"] and
                             entry["same_consumed_inventory_mass"] and
                             abs(entry["horizontal_distance_difference_m"]) < 1e-6 and
                             abs(entry["time_difference_s"]) < 1e-6 and
                             abs(entry["velocity_difference_m_s"]) < 1e-6)
            if not entry["pass"]:
                raise AssertionError(entry)
            checks.append(entry)
    return checks


def figures(sh, profiles, baseline, scan, pairs, mechanisms, inverses):
    fig, axes = plt.subplots(3, 1, figsize=(10, 10), sharex=True)
    for name, z in profiles.items():
        axes[0].plot(sh/1000, z, color=COLORS[name], label=LABELS[name], lw=1.25)
        tr = baseline[name]["trajectory"]
        axes[1].plot([p["s_h_m"]/1000 for p in tr], [p["velocity_m_s"] for p in tr],
                     color=COLORS[name], label=LABELS[name])
        axes[2].plot([p["s_h_m"]/1000 for p in tr], [p["time_s"]/60 for p in tr], color=COLORS[name])
    axes[0].set_ylabel("高程 / m")
    axes[1].set_ylabel("代表团块速度 / (m·s$^{-1}$)")
    axes[2].set_ylabel("注入后时间 / min")
    axes[2].set_xlabel("固定路线水平里程 / km")
    axes[0].set_title("无新增节点：三种高程结构的条件动力对照", loc="left")
    for ax in axes:
        ax.grid(alpha=.2)
```

```
      ax.axvline(sh[-1]/1000, color=".3", ls=":", lw=.8)
   axes[0].legend(ncol=3, fontsize=9)
   fig.text(.5, .015, "工程参数非率定；曲线终止表示停止或口岸代理事件，不代表真实前锋观测", ha="center", fontsize=9)
   fig.tight_layout(rect=[0, .035, 1, 1])
   fig.savefig(io.FIG_OUT/"04c_三剖面动力.png")
   plt.close(fig)

   fig, axes = plt.subplots(1, 3, figsize=(12, 4.8), sharey=True)
   kappas = DOMAIN["kappa_m_inv"]
   for ax, name in zip(axes, profiles):
      for iv, v in enumerate(DOMAIN["v0_m_s"]):
         points = [r for r in scan if r["profile"] == name and r["parameters"]["v0_m_s"] == v]
         xx = [kappas.index(r["parameters"]["kappa_m_inv"])+(.22*(iv-1)) for r in points]
         yy = [r["parameters"]["mu_dyn"] for r in points]
         cc = [r["final_sh_m"]/sh[-1] for r in points]
         im = ax.scatter(xx, yy, c=cc, cmap="YlGnBu", vmin=0, vmax=1, s=68,
                         marker=["o", "s", "^"][iv], edgecolors=".3", linewidths=.4)
      ax.set_xticks(range(4), ["0", "$10^{-5}$", "$5×10^{-5}$", "$10^{-4}$"])
      ax.set_xlabel("二次阻力 κ / m$^{-1}$")
      ax.set_title(LABELS[name])
      ax.grid(alpha=.15)
   axes[0].set_ylabel("工程动摩擦参数 μ")
   fig.colorbar(im, ax=axes, label="终止/目标水平里程占比", fraction=.025, pad=.02)
   fig.suptitle("预声明无节点工程域：每剖面48配置；符号依次表示入口速度√200、30、60 m/s", fontsize=10)
   fig.subplots_adjust(left=.07, right=.87, bottom=.16, top=.84, wspace=.18)
   fig.savefig(io.FIG_OUT/"04c_无节点工程域.png")
   plt.close(fig)

   fig, axes = plt.subplots(1, 3, figsize=(12, 4.6), sharey=True)
   for ax, name in zip(axes, profiles):
      for mode, marker, color in [("M0", "o", "#4d5967"), ("M1", "s", "#2e7c87"), ("M2", "^", "#b25c33")]:
         subset = [p for p in pairs if p["profile"] == name and p["q_r"] == .5 and p["mode"] == mode]
         ax.plot([p["q_w"] for p in subset], [mechanisms[p["run_id"]]["final_sh_m"]/1000 for p in subset],
                 marker=marker, color=color, label=mode)
      ax.axhline(sh[-1]/1000, color=".4", ls=":", lw=1)
      ax.set_title(LABELS[name])
      ax.set_xlabel("合成新增水幅度 $q_w$（非实测）")
      ax.grid(alpha=.2)
      ax.set_xticks([0, .5, 1])
   axes[0].set_ylabel("首次停止或目标水平里程 / km")
   axes[-1].legend(fontsize=13)
   fig.suptitle("人工1/3、2/3节点，固定$q_r=0.5$：加水混合与唯一假设减阻分离", fontsize=14)
   fig.text(.5, .02, "M2仅假设μ乘0.6；三组均不证明实际补水、实际减阻或真实灾害到达", ha="center", fontsize=12)
   fig.tight_layout(rect=[0, .05, 1, .93])
   fig.savefig(io.FIG_OUT/"04c_合成补水机制.png")
   plt.close(fig)

   fig, axes = plt.subplots(1, 2, figsize=(10, 4.5))
   for name in profiles:
      inv = inverses[f"no_nodes_{name}_0.1_1e-05"]
      rec = inv["records"]
      axes[0].plot([r["s_h_m"]/1000 for r in rec],
                   [math.sqrt(r["required_pre_w_m2_s2"]) for r in rec], color=COLORS[name], label=LABELS[name])
      r = baseline[name]
      terms = [r["coulomb_loss_J"], r["drag_loss_J"], r["mix_loss_J"]]
      axes[1].plot([0, 1, 2], np.asarray(terms)/1e12, marker="o", color=COLORS[name], label=LABELS[name])
   axes[0].set_xlabel("固定路线水平里程 / km")
   axes[0].set_ylabel("逆推局部入口速度需求 / (m·s$^{-1}$)")
   axes[0].set_title("固定阻力瓶颈，目标约定1 m/s")
   axes[1].set_xticks([0, 1, 2], ["摩擦耗散", "二次阻力耗散", "混合耗散"])
   axes[1].set_ylabel("耗散能量 / $10^{12}$ J")
   axes[1].set_title("无节点基线机械能账户")
   for ax in axes:
      ax.grid(alpha=.2)
   axes[0].legend(fontsize=10)
   fig.tight_layout()
   fig.savefig(io.FIG_OUT/"04c_瓶颈与能量.png")
   plt.close(fig)


def main():
   tests = run_report()
   save(tests, "04c_合成测试.json")
   if not tests["passed"]:
      raise AssertionError(tests)
   sh, profiles, initial, geometry = load_geometry()
   source = Path(__file__).parent
   sources = [source/"mmlib/finite_inventory.py", source/"step04c_dynamics.py",
              source/"test04c_dynamics.py", io.ROOT/"建模成果/问题4.md"]
   declaration = {"domain": DOMAIN, "geometry": geometry,
                  "density_kg_m3": {"rock": fi.RHO_R, "water": fi.RHO_W, "ice_inactive": fi.RHO_I},
                  "density_basis": "engineering rock-water constants identical to old step04 RHO_S=2650, RHO_W=1000;
                       not site measurements; ice unused",
                  "source_sha256": {str(p.relative_to(io.ROOT)): sha(p) for p in sources},
                  "random_seed": None, "randomness": "none", "ice_initialization": "not executed",
                  "time_limit_s": None, "numerical_zero_speed_threshold": None,
                  "positive_target_arrival": "positive-speed passage; zero-speed contact separate",
                  "boundaries": ["fixed-route representative parcel, not observed front",
                                 "no lateral momentum, no depth, intensity, conversion time or event probability",
                                 "two artificial nodes, not measured geomorphic inventories",
                                 "geometric corners assume constrained contact; no resolved impact or detachment",
                                 "historical Heim mu not pure dynamic friction calibration",
                                 "no restarting after first zero speed"]}
```

```
save(declaration, "04c_参数与路径.json")
baseline, scan, inverses = {}, [], {}
for name, z in profiles.items():
    baseline[name] = fi.simulate(sh, z, initial, **DOMAIN["baseline"])
    for mu, k in itertools.product(DOMAIN["mu_dyn"], DOMAIN["kappa_m_inv"]):
        key = f"no_nodes_{name}_{mu}_{k}"
        inverses[key] = checked_inverse(sh, z, initial, mu=mu, kappa=k)
        for v in DOMAIN["v0_m_s"]:
            r = fi.simulate(sh, z, initial, mu=mu, kappa=k, v0=v)
            scan.append({"profile": name, **summary(r)})
    print(name, "48 no-node configurations complete", flush=True)
save({"baseline": baseline, "scan": scan, "count": len(scan),
      "per_profile": 48, "not_probabilities": True}, "04c_无节点动力.json")
mechanisms, pairs, details, cache = {}, [], {}, {}
for name, z in profiles.items():
    for qr, qw, mode in itertools.product(DOMAIN["q_r"], DOMAIN["q_w"], ["M0", "M1", "M2"]):
        water, effective_mode = fi.resolve_mode(mode, qw)
        identity = (name, qr, water, effective_mode)
        if identity not in cache:
            rid = f"{name}_qr{qr}_qw{water}_{effective_mode}"
            cache[identity] = rid
            nodes = artificial_nodes(sh, initial, qr, water)
            r = fi.simulate(sh, z, initial, nodes=nodes, mode=effective_mode, **DOMAIN["baseline"])
            mechanisms[rid] = {"profile": name, "q_r": qr, "q_w": water,
                               "effective_mode": effective_mode, **summary(r),
                               "node_ledger": r["node_ledger"], "water_competition": water_competition(r)}
            inverses[rid] = checked_inverse(sh, z, initial, nodes=nodes, mode=effective_mode,
                                            mu=.1, kappa=1e-5)
            if qr == .5 and water in [0., .5]:
                details[rid] = r
        if mode == "M0":
            alias_reason = "M0关闭新增水，q_w强制为0；与M1在q_w=0时为同一运行"
        elif mode == "M2" and water == 0:
            alias_reason = "M2要求新增水，但q_w=0不触发减阻映射；与M1在q_w=0时为同一运行"
        elif mode == "M2":
            alias_reason = "M2新增水严格大于0，使用预登记的下游mu=0.6mu0映射"
        else:
            alias_reason = "M1直接运行"
        pairs.append({"profile": name, "q_r": qr, "q_w": qw, "mode": mode,
                      "effective_mode": effective_mode, "run_id": cache[identity],
                      "alias_reason": alias_reason})
requested = {}
for p in pairs:
    requested.setdefault(p["run_id"], []).append(p["mode"])
for rid, modes in requested.items():
    mechanisms[rid]["requested_modes"] = sorted(set(modes), key=["M0", "M1", "M2"].index)
comparisons = []
for name, qr, qw in itertools.product(profiles, DOMAIN["q_r"], DOMAIN["q_w"]):
    ids = {p["mode"]: p["run_id"] for p in pairs if p["profile"] == name and p["q_r"] == qr and p["q_w"] == qw}
    entry = {"profile": name, "q_r": qr, "q_w": qw, "run_ids": ids}
    for first, second in [("M1", "M0"), ("M2", "M1")]:
        a, b = mechanisms[ids[first]], mechanisms[ids[second]]
        entry[f"{first}_minus_{second}"] = {
            "final_horizontal_distance_difference_m": a["final_sh_m"]-b["final_sh_m"],
            "arrival_difference_s": a["arrival_s"]-b["arrival_s"] if a["arrival_s"] is not None and b["arrival_s"]
                is not None else None,
            "both_reached": a["reached_target_positive"] and b["reached_target_positive"]}
    comparisons.append(entry)
mode_resolution = {
    "deduplication_identity": "(profile, q_r, water, effective_mode)",
    "M0": {"water_rule": "每个请求q_w均强制为0", "effective_run_mode": "M1",
           "note": "与M1在q_w=0时数值相同，不另设独立results行"},
    "M1": {"water_rule": "按请求q_w加水", "effective_run_mode": "M1"},
    "M2": {"water_rule": "按请求q_w加水", "effective_run_mode": "M2 if q_w>0 else M1",
           "note": "q_w=0不触发预登记减阻映射，回落到M1"},
}
save({"unique_configurations": len(mechanisms),
      "requested_configuration_count": len(pairs),
      "mode_resolution": mode_resolution,
      "deduplication_rule": "M0恒为q_w=0并别名到M1(q_w=0)；M2在q_w=0时也别名到该运行；每个结果的requested_modes列出请求模式
          ",
      "contrast_references": pairs,
      "results": mechanisms, "comparisons": comparisons, "selected_full_ledgers": details,
      "additional_no_node_baseline_duplicates": 3,
      "physical_inventory_evidence": "absent; all nodes and stocks artificial"}, "04c_合成库存机制.json")
save({"vreq_m_s": 1., "thresholds": inverses,
      "all_positive_threshold_forward_checks_pass": all(v.get("forward_check_pass", True) for v in inverses.values
          ()),
      "outside_speed_domain": [k for k, v in inverses.items() if not v.get("required_v0_within_declared_scan_range
          ", False)],
      "interpretation": "conditional analytic infima, not fitted speeds or event probabilities"}, "04c_逆向瓶颈.
          json")
figures(sh, profiles, baseline, scan, pairs, mechanisms, inverses)
all_results = scan+list(mechanisms.values())
checks = {"synthetic_tests_pass": tests["passed"], "no_node_scan_count": len(scan),
          "synthetic_unique_count": len(mechanisms), "contrast_reference_count": len(pairs),
          "inverse_count": len(inverses),
          "mass_relative_residual_max": max(abs(r["mass_residual_kg"])/r["initial_mass_kg"] for r in all_results),
          **{k: max(r[k] for r in all_results) for k in ["energy_relative_residual_max",
              "segment_energy_relative_residual_max", "node_momentum_relative_residual_max", "
                  node_energy_relative_residual_max"]},
          "null_arrival_for_nonpositive_passage": all(r["arrival_s"] is None for r in all_results if not r["
              reached_target_positive"]),
          "baseline_summary": {n: summary(r) for n, r in baseline.items()},
          "status_counts": {name: {s: sum(r["status"] == s for r in scan if r["profile"] == name)
```

```
                            for s in sorted({r["status"] for r in scan})} for name in profiles},
          "figures": [p.name for p in sorted(io.FIG_OUT.glob("04c_*.png"))],
          "figure_review": "not inspected here; pending central four-up visual QA"}
   checks["real_route_fixed_node_refinement"] = real_route_refinement(sh, profiles, initial)
   checks["acceptance_tolerances"] = {"relative_mass": 1e-10, "relative_energy": 1e-9,
                                      "relative_node_momentum": 1e-10}
   checks["numerical_acceptance_pass"] = (
      checks["mass_relative_residual_max"] < 1e-10 and
      checks["energy_relative_residual_max"] < 1e-9 and
      checks["segment_energy_relative_residual_max"] < 1e-9 and
      checks["node_energy_relative_residual_max"] < 1e-9 and
      checks["node_momentum_relative_residual_max"] < 1e-10 and
      checks["null_arrival_for_nonpositive_passage"])
   save(checks, "04c_数值核验.json")
   if not checks["numerical_acceptance_pass"]:
      raise AssertionError("04c mass/momentum/energy numerical acceptance failed")
   print(json.dumps(checks, ensure_ascii=False, indent=2))


if __name__ == "__main__":
   main()
```

step04_figures.py　问题四图件

```
"""问题 4 的图件：仿真环境、过程、结果与敏感性。"""
from __future__ import annotations

import json
import sys
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.colors import BoundaryNorm, LightSource, ListedColormap

sys.path.insert(0, str(Path(__file__).resolve().parent))
from mmlib import io_data as io, hydro # noqa: E402

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "Noto Sans SC", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
# SimHei 缺少上标数字（U+00B2/U+00B3）与上标减号（U+207B），若沿用默认
# mathtext 字体集，$hv^2$ 一类公式会被渲染成方框。改用 DejaVu Sans 渲染
# 数学、中文仍走 SimHei，二者混排正常。
plt.rcParams["mathtext.fontset"] = "dejavusans"
plt.rcParams["figure.dpi"] = 130
plt.rcParams["savefig.dpi"] = 200
plt.rcParams["font.size"] = 20
plt.rcParams["axes.titlesize"] = 19
plt.rcParams["axes.labelsize"] = 17
plt.rcParams["xtick.labelsize"] = 15
plt.rcParams["ytick.labelsize"] = 15
plt.rcParams["legend.fontsize"] = 13

FIG = io.FIG_OUT
OUT = io.DATA_OUT
CACHE = io.CACHE
C_MAIN = "#1f4e79"
C_WARN = "#c00000"
C_FILL = "#2e75b6"
C_OK = "#2f8f46"  # 原代码在液型比 ≥0.6 的分支引用了未定义的 C_OK，
                  # 因该分支长期未被执行而潜伏；基准体量改为派生值后暴露。


def _load(name):
   return json.loads((OUT / name).read_text(encoding="utf-8"))


def fig_profile(base):
   """中心线纵剖面 + 流速/流深/强度包络。"""
   s = np.asarray(base["s_km"])
   z = np.asarray(base["z_m"])
   v = np.asarray(base["v_m_s"])
   h = np.asarray(base["h_m"])
   I = np.asarray(base["intensity"]) / 1000.0
   wet = v > 0
   z0 = base["z_m"][0]
   mu = base["mu"]

   fig, ax = plt.subplots(4, 1, figsize=(9.2, 9.8), sharex=True,
                          gridspec_kw={"height_ratios": [1.15, 1, 1, 1], "hspace": 0.16})

   ax[0].plot(s, z, color="#333333", lw=1.6, label="纵剖面高程")
   ax[0].fill_between(s, z, z0, color="#e8eef5", zorder=0)
   if wet.any():
      i0 = int(np.where(wet)[0][0])
      e_line = z[i0] + base.get("params", {}).get("E0_m2s2", 0.0)/ (2*9.80665) - mu * (s - s[i0])*1000
      ax[0].plot(s, e_line, "--", color=C_WARN, lw=1.4,
               label=f"Heim 能量线（μ={mu:.3f}, {base['等效摩擦角_deg']:.1f}°）")
      ax[0].scatter([s[i0]], [z[i0]], s=46, color=C_WARN, zorder=5,
                  label=f"起始点 {s[i0]:.2f} km")
      ax[0].axvline(base["runout_km"], color="#555555", ls=":", lw=1.2,
                  label=f"末端/停止位置 {base['runout_km']:.2f} km")
```

```
    ax[0].set_ylabel("高程 / m")
    ax[0].set_ylim(z.min() - 150, z.max() + 200)
    ax[0].legend(loc="upper right", fontsize=14, framealpha=0.93)
    ax[0].set_title("吉隆主沟纵剖面与 Heim 能量线（μ 由附件6 反演）", loc="left")

    ax[1].plot(s[wet], v[wet], color=C_MAIN, lw=1.8)
    ax[1].fill_between(s[wet], 0, v[wet], color=C_MAIN, alpha=0.18)
    ax[1].set_ylabel("流速 / (m·s$^{-1}$)")
    ax[1].axhline(base["v_median_m_s"], color=C_WARN, ls=":", lw=1.1)
    ax[1].text(s[wet][-1] * 0.02, base["v_median_m_s"] * 1.07,
            f"中位 {base['v_median_m_s']:.1f} m/s", color=C_WARN, fontsize=12,
            bbox=dict(facecolor="white", edgecolor="none", alpha=0.8, pad=0.5))
    ax[1].set_title(f"断面平均流速（峰值 {base['v_max_m_s']:.1f} m/s）", loc="left")

    ax[2].plot(s[wet], h[wet], color="#548235", lw=1.8)
    ax[2].fill_between(s[wet], 0, h[wet], color="#548235", alpha=0.18)
    ax[2].set_ylabel("流深 / m")
    ax[2].set_title(f"断面平均流深（中位 {base['h_median_m']:.2f} m）", loc="left")

    ax[3].semilogy(s[wet], np.maximum(I[wet], 1e-2), color="#7030a0", lw=1.8)
    for thr, lab in [(10.0, "低/中"), (100.0, "中/高"), (1000.0, "高/极高")]:
        ax[3].axhline(thr, color="#999999", ls="--", lw=0.8)
        ax[3].text(s[wet][-1] * 0.02, thr * 1.2, lab, fontsize=12, color="#666666")
    ax[3].set_ylabel("冲击力强度 $I$ / (kN·m$^{-1}$)")
    ax[3].set_xlabel("沿程里程 / km")
    ax[3].set_title(rf"强度指标 $I=\rho_m h v^2$（峰值 {base['I_max']/1e3:.0f} kN/m）", loc="left")
    for a in ax:
        a.grid(alpha=0.25)
        a.set_xlim(0, s[-1])
    fig.savefig(FIG / "04_沿程演进.png", bbox_inches="tight")
    plt.close(fig)


def fig_arrival(base):
    """到达时间—里程曲线。"""
    s = np.asarray(base["s_km"])
    t = np.asarray([x if x is not None else np.nan for x in base["t_s"]])
    wet = np.isfinite(t)
    fig, ax = plt.subplots(figsize=(7.6, 4.3))
    ax.plot(s[wet], t[wet] / 60, color=C_MAIN, lw=2.0, marker="o", ms=2.2)
    ax.fill_between(s[wet], 0, t[wet] / 60, color=C_MAIN, alpha=0.15)
    t_arr = base["arrival_outlet_s"] or base.get("stop_time_s", float(np.nanmax(t)))
    ax.axhline(t_arr / 60, color=C_WARN, ls="--", lw=1.2)
    ax.annotate(f"{'抵达口岸' if base['reached_outlet'] else '自然停止（未到口岸）'} {t_arr / 60:.1f} min",
            xy=(s[wet][-1], t_arr / 60), xytext=(-130, -22),
            textcoords="offset points", color=C_WARN, fontsize=9,
            arrowprops=dict(arrowstyle="->", color=C_WARN, lw=1.1))
    ax.set_ylim(-1.5, t_arr / 60 * 1.14)
    ax.set_ylabel("自起动起算时间 / min")
    ax.set_title("泥石流前锋到达时间曲线（Manning 均匀流积分）", loc="left")
    ax.grid(alpha=0.25)
    fig.savefig(FIG / "04_到达时间.png", bbox_inches="tight")
    plt.close(fig)


def fig_intensity_map(base):
    """一维走廊强度图：只有抵达口岸后才允许附加非动力学扇形包络。"""
    dem, grid = io.load_dem()
    inten = np.load(CACHE / "sim_fields.npz")["intensity"]
    half_angle = 25.0
    reach = base["runout_km"]
    I0 = base["I_max"]
    cl = base["中心线"]

    reached = bool(base.get("reached_outlet", False))
    if reached:
        spread = hydro.lateral_spread(
            {"x_m": np.array(cl["x_m"]), "y_m": np.array(cl["y_m"]),
             "s_m": np.array(cl["s_m"]), "end_rc": cl["end_rc"]},
            reach_km=min(6.0, reach * 0.28), half_angle_deg=half_angle,
            I0=I0, dem=dem, grid=grid)
        spread.update(status="geometric_envelope",
                      reason="一维情景抵达口岸：扇形仅为非动力学几何包络，不表示守恒输运或实际堆积场",
                      reached_outlet=True)
    else:
        # 口岸是中心线终点，不是未到达情景的实际扇顶；禁止在此凭空生成扇体。
        spread = {"status": "not_applicable",
                  "reason": "一维基线未抵达口岸，不能以中心线口岸终点生成扇体扩散区",
                  "reached_outlet": False,
                  "intensity": np.zeros_like(dem, dtype="float32"),
                  "affected_cells": 0, "affected_area_km2": 0.0,
                  "max_intensity": 0.0, "I0": 0.0,
                  "apex_rc": None, "main_direction_deg": None,
                  "half_angle_deg": None, "reach_km": 0.0}
    fan = spread["intensity"]

    ext = [grid["lon0"], grid["lon0"] + grid["dlon"] * dem.shape[1],
           grid["lat0"] + grid["dlat"] * dem.shape[0], grid["lat0"]]
    ls = LightSource(azdeg=315, altdeg=45)
    hs = ls.hillshade(dem.astype("float64"), vert_exag=2.0, dx=27.2, dy=30.7)

    fig, ax = plt.subplots(figsize=(10.2, 6.5))
    ax.imshow(hs, extent=ext, cmap="gray", alpha=0.9, origin="upper")
```

```
    lv = [0, 1e4, 1e5, 1e6, 1e9]
    cmap = ListedColormap(["#7fb3d5", "#f4d03f", "#e67e22", "#a93226"])
    norm = BoundaryNorm(np.arange(5), cmap.N)
    im = ax.imshow(np.ma.masked_where(inten<=0.05, np.digitize(inten,[1e4,1e5,1e6])+.5), extent=ext,
                   cmap=cmap, norm=norm, alpha=0.9, origin="upper")
    if reached and np.any(fan > 1e4):
        ax.contour(np.linspace(ext[0], ext[1], fan.shape[1]),
                   np.linspace(ext[3], ext[2], fan.shape[0]),
                   np.ma.masked_less_equal(fan, 1e4), levels=4,
                   colors="#00e5ff", linewidths=1.1, alpha=0.9)
    cb = fig.colorbar(im, ax=ax, orientation="vertical", shrink=0.74, pad=0.015)
    cb.set_ticks([.5,1.5,2.5,3.5]); cb.set_ticklabels(["低 <10", "中 10-100", "高 100-1000", "极高 ≥1000"])
    cb.set_label(r"单位宽动量通量 $I=\rho_m h v^2$ / (kN/m)", fontsize=9)

    unit_x = abs(grid["dlon"]) * 111320.0 * np.cos(np.radians(28.4))
    unit_y = abs(grid["dlat"]) * 110574.0
    clon = grid["lon0"] + (np.array(cl["x_m"]) / unit_x) * grid["dlon"]
    clat = grid["lat0"] - (np.array(cl["y_m"]) / unit_y) * abs(grid["dlat"])
    ax.plot(clon, clat, color="#00ffff", lw=1.3, label="主沟中心线")
    ax.plot(io.EVENT["port_lon"], io.EVENT["port_lat"], "*", ms=18,
            color="#ff0000", mec="white", mew=1.2, label="吉隆口岸", zorder=10)
    ax.set_xlim(clon.min() - 0.03, clon.max() + 0.03)
    ax.set_ylim(clat.min() - 0.02, clat.max() + 0.02)
    ax.set_xlabel("经度 / °E")
    ax.set_ylabel("纬度 / °N")
    title = (f"一维走廊强度与非动力学扇形包络（青色等值线，半角 {half_angle:.0f}°）"
             if reached else "一维走廊强度分布（基线未达口岸，不生成扇体）")
    ax.set_title(title, loc="left")
    ax.legend(loc="lower left", fontsize=8, framealpha=0.92)
    fig.savefig(FIG / "04_强度分级平面.png", bbox_inches="tight")
    plt.close(fig)
    return spread


def fig_tornado(sens, base):
    """单参数敏感性龙卷风图。"""
    names = {"E0_m2s2": "初始激发尺度 $E_0$", "V0_m3": "初始体积 $V_0$", "n_man": "糙率 $n$", "mu": "摩擦系数 $\\mu$",
             "B0_m": "沟宽 $B_0$", "T_pass_s": "过流历时 $T$",
             "mu_prop": "μ 相对扰动", "Cv": "固体体积分数 $C_v$"}
    fig, axes = plt.subplots(1, 3, figsize=(12.8, 4.4))
    targets = [("v_median_m_s", "中位流速 / (m·s$^{-1}$)"),
               ("h_median_m", "中位流深 / m"),
               ("I_max", "峰值强度 / (kN·m$^{-1}$)")]
    for ax, (key, lab) in zip(axes, targets):
        rows = []
        for k, lst in sens["敏感性结果"].items():
            if k == "mu_prop":
                continue
            vals = [r[key] for r in lst if r.get(key) is not None]
            if not vals:
                continue
            if key == "I_max":            # 强度用 kPa 显示
                vals = [v / 1e3 for v in vals]
            rows.append((names.get(k, k), min(vals), max(vals)))
        rows.sort(key=lambda r: r[2] - r[1])
        for i, (nm, lo, hi) in enumerate(rows):
            ax.barh(i, max(hi - lo, 1e-9), left=lo, height=0.55,
                    color=C_FILL, alpha=0.78)
            ax.plot([lo, hi], [i, i], color=C_MAIN, lw=0.8)
        ax.set_yticks(np.arange(len(rows)))
        ax.set_yticklabels([r[0] for r in rows], fontsize=8.5)
        ax.set_xlabel(lab, fontsize=9)
        ax.grid(alpha=0.25, axis="x")
    fig.suptitle("问题4 关键参数的单参数敏感性（各参数在其取值范围内扫描）",
                 fontsize=11.5, y=1.005)
    fig.tight_layout()
    fig.savefig(FIG / "04_参数敏感性.png", bbox_inches="tight")
    plt.close(fig)


def fig_mu_curve(base):
    """μ—运程响应曲线 + 历史沟道 μ 的规模分组分布。"""
    from step04_simulation import channel_centerline
    cl = channel_centerline(100.0)
    mu_used = float(base["mu"])
    # 用敏感性结果里记录的基准参数，而不是模块默认值——模块默认中 V0 为 None
    # （由 main() 按附件派生），直接取它会把 None 传进求解器而崩溃。
    sens = _load("04_敏感性.json")
    p = dict(sens["基准参数"])
    p["mu"] = mu_used
    mus = np.linspace(0.09, 0.34, 26)
    r = hydro.runout_curve(cl["s_m"], cl["z_m"], mus, p)
    hp = io.load_hazard_points().copy()
    hp["muv"] = hp["relief_m"] / (hp["length_m"] + hp["fan_l"])

    fig, ax = plt.subplots(1, 2, figsize=(11.8, 4.4))
    rr = r["曲线"]
    ax[0].plot([c["mu"] for c in rr], [c["runout_km"] for c in rr],
               color=C_MAIN, lw=2.0, marker="o", ms=3)
    ax[0].axvline(mu_used, color=C_WARN, ls="--", lw=1.3,
                  label=f"采用 $\\mu$={mu_used:.3f}")
    ax[0].axvline((cl["z_m"][0] - cl["z_m"][-1]) / cl["s_m"][-1], color="#7f7f7f",
```

```
            ls="-.", lw=1.1, label="临界 $\\mu_{crit}$=Δz/L")
    ax[0].axhline(cl["length_km"], color="#555555", ls=":", lw=1.2,
                  label=f"沟道全长 {cl['length_km']:.2f} km")
    ax[0].set_xlabel("等效摩擦系数 $\\mu$")
    ax[0].set_ylabel("运移距离 / km")
    ax[0].set_title("$\\mu$ 对运移距离的控制（Heim 能量线）", loc="left")
    ax[0].legend(fontsize=8.0, loc="lower left")
    ax[0].grid(alpha=0.25)

    for k, c in zip(["小", "中", "大", "特大"],
                    ["#7fb3d5", "#f4d03f", "#e67e22", "#a93226"]):
        v = hp[hp["规模"] == k]["muv"].dropna()
        if len(v):
            ax[1].hist(v, bins=12, alpha=0.7, label=f"{k}（n={len(v)}）", color=c)
    ax[1].axvline(mu_used, color=C_WARN, ls="--", lw=1.5,
                  label=f"采用 $\\mu$={mu_used:.3f}")
    ax[1].set_xlabel("附件6 反演 $\\mu$ = Δz /(L沟+L扇)")
    ax[1].set_ylabel("沟道条数")
    ax[1].set_title("历史沟道 $\\mu$ 的规模分组分布", loc="left")
    ax[1].legend(fontsize=8, loc="upper right")
    ax[1].grid(alpha=0.22)
    fig.tight_layout()
    fig.savefig(FIG / "04_运移距离标定.png", bbox_inches="tight")
    plt.close(fig)


def fig_budget(base, inv):
    """体积诊断（波列 vs 满槽）+ 质量构成 + 初始体积估算链。"""
    fig, ax = plt.subplots(1, 3, figsize=(13.4, 4.3))

    # (a) 波列长度 vs 运移路程：区分脉冲型与满槽型流动
    Ls = float(base.get("波列长度_m", np.nan)) / 1000.0
    run = float(base["runout_km"])
    S = float(base["沟槽储水体积_m3"]) / 1e6
    V0 = float(base["V0"]) / 1e6
    ratio = float(base.get("波列占路程比", np.nan))
    kind = base.get("液型", "")

    ax[0].barh([1], [run], height=0.34, color="#d9d9d9", edgecolor="#7f7f7f",
            label=f"运移路程 {run:.1f} km")
    ax[0].barh([0], [Ls], height=0.34,
            color=C_WARN if ratio < 0.6 else C_OK,
            label=f"诊断波长 {Ls:.1f} km（比值 {Ls/run*100:.0f}%）")
    ax[0].set_yticks([1, 0])
    ax[0].set_yticklabels(["运移路程", "过流波列"], fontsize=9.5)
    ax[0].set_xlabel("沿程距离 / km")
    ax[0].set_xlim(0, max(run, Ls) * 1.18)
    ax[0].set_ylim(-0.7, 1.7)
    ax[0].set_title(f"(a) {kind}: $V_0$ 决定波列长度，$\\mu$ 决定停止位置",
                    loc="left", fontsize=9.6)
    ax[0].legend(fontsize=8.2, loc="lower right")
    ax[0].grid(alpha=0.22, axis="x")
    ax[0].text(0.5, 0.90,
            f"满槽储水 {S:.2f}×10$^6$ m$^3$ / $V_0$ {V0:.2f}×10$^6$ m$^3$\n"
            f"储量/入口体积 = {S / V0:.2f}；波列/过流段 = {Ls/run*100:.0f}%（非守恒）",
            transform=ax[0].transAxes, ha="center", va="top", fontsize=8.2,
            bbox=dict(boxstyle="round,pad=0.3", fc="#f2f2f2", ec="#999999"))

    # (b) 质量构成
    cv = float(base["Cv"])
    rho_s, rho_w = 2650.0, 1000.0
    rho_m = cv * rho_s + (1 - cv) * rho_w
    m_s = base["V0"] * cv * rho_s / 1e9
    m_w = base["V0"] * (1 - cv) * rho_w / 1e9
    bars = ax[1].bar(["固相质量", "液相质量"], [m_s, m_w],
                     color=["#a93226", "#2e75b6"], alpha=0.88, width=0.48)
    for b, v in zip(bars, [m_s, m_w]):
        ax[1].text(b.get_x() + b.get_width() / 2, v * 1.015, f"{v:.2f}",
                   ha="center", fontsize=9.5, fontweight="bold")
    ax[1].set_ylim(0, max(m_s, m_w) * 1.18)
    ax[1].set_ylabel("质量 / $10^{9}$ kg")
    ax[1].set_title(f"(b) 质量构成（$C_v$={cv}，$\\rho_m$={rho_m:.0f} kg·m$^{{-3}}$）",
                    loc="left")
    ax[1].grid(alpha=0.22, axis="y")

    # (c) V0 估算链（双链：链A 冰厚-坡度筛选为基准，链B P90×可动比例为上界）
    chA = inv["链A"]
    labels_c = ["链A\n源区冰体积", "链A\n×$\\kappa_{rel}$", "链A\n×$\\gamma$·$\\rho$比",
                "链B\n厚冰掩膜总体积", "链B\n×可动比例"]
    vals_c = [chA["V_src_ice_m3"] / 1e6, chA["V_ice_A_m3"] / 1e6,
              chA["V_df_A_m3"] / 1e6, inv["冰崩体量_m3"] / 1e6,
              inv["链B_V0_m3"] / 1e6]
    notes_c = [None,
               f"×{chA['kappa_rel']:.2f}",
               f"×{chA['gamma']:.0f}·{chA['rho_ice']:.0f}/{chA['rho_df']:.0f}",
               None, f"×{inv['可动比例']:.2f}"]
    colors_c = ["#c55a11", "#c55a11", "#2e75b6", "#7f7f7f", "#7f7f7f"]
    xs = list(range(5))
    for x, hv, nt, cc in zip(xs, vals_c, notes_c, colors_c):
        adopted = (x == 2)
```

```
      ax[2].bar(x, hv, color=cc, alpha=0.9 if adopted else 0.6, width=0.58,
             edgecolor="#1a1a1a" if adopted else "none",
             linewidth=1.4 if adopted else 0)
      ax[2].text(x, hv * 1.03, f"{hv:.1f}", ha="center", fontsize=9,
              fontweight="bold")
      if nt:
         ax[2].text(x, hv * 0.55, nt, ha="center", fontsize=7,
                 color="white", fontweight="bold")
   ax[2].annotate("采用 $V_0$（链A 基准）", xy=(2, vals_c[2]),
              xytext=(2.6, max(vals_c) * 0.86), fontsize=8.5,
              arrowprops=dict(arrowstyle="->", lw=1.0), color="#1a1a1a")
   ax[2].set_xticks(xs)
   ax[2].set_xticklabels(labels_c, fontsize=8)
   ax[2].set_ylabel("体积 / $10^{6}$ m$^{3}$")
   ax[2].set_title("(c) 初始释放体积的双链估算（链A 基准 / 链B 大体量对照）",
              loc="left")
   ax[2].grid(alpha=0.22, axis="y")
   ax[2].set_ylim(0, max(vals_c) * 1.22)

   fig.tight_layout()
   fig.savefig(FIG / "04_收支校验.png", bbox_inches="tight")
   plt.close(fig)


def fig_process_stages(base):
   """仿真过程：4 个时刻的前锋位置与过流段。"""
   s = np.asarray(base["s_km"])
   t = np.asarray([x if x is not None else np.nan for x in base["t_s"]])
   z = np.asarray(base["z_m"])
   wet = np.isfinite(t)
   tmax = np.nanmax(t)
   stages = [0.15, 0.35, 0.6, 0.85]
   fig, axes = plt.subplots(2, 2, figsize=(11.2, 7.5))
   for ax, frac in zip(axes.ravel(), stages):
      tend = tmax * frac
      front = s[wet][np.searchsorted(t[wet], tend, side="right") - 1]
      ax.plot(s, z, color="#444444", lw=1.4)
      m = s <= front
      ax.plot(s[m], z[m], color=C_FILL, lw=2.8)
      ax.axvline(front, color=C_WARN, ls="--", lw=1.2)
      ax.scatter([front], [np.interp(front, s, z)], s=74, color=C_WARN, zorder=6)
      ax.set_xlim(0, s[-1])
      ax.set_ylim(z.min() - 200, z.max() + 200)
      ax.set_title(f"$t$ = {tend / 60:.1f} min 前锋 {front:.1f} km", fontsize=10)
      ax.set_xlabel("里程 / km")
      ax.set_ylabel("高程 / m")
      ax.grid(alpha=0.22)
   fig.suptitle("一维准恒定传播时间诊断（非守恒时间推进）", fontsize=11.5, y=0.995)
   fig.tight_layout()
   fig.savefig(FIG / "04_演进过程.png", bbox_inches="tight")
   plt.close(fig)


def main() -> int:
   FIG.mkdir(parents=True, exist_ok=True)
   base = _load("04_仿真基线.json")
   sens = _load("04_敏感性.json")
   inv = _load("04_初始体积.json")

   fig_profile(base)
   print(" 04_沿程演进.png")
   fig_arrival(base)
   print(" 04_到达时间.png")
   spread = fig_intensity_map(base)
   print(f" 04_强度分级平面.png（扇体影响区 {spread['affected_area_km2']:.2f} km²）")
   fig_tornado(sens, base)
   print(" 04_参数敏感性.png")
   fig_mu_curve(base)
   print(" 04_运移距离标定.png")
   fig_budget(base, inv)
   print(" 04_收支校验.png")
   fig_process_stages(base)
   print(" 04_演进过程.png")

   (OUT / "04_扇体扩散.json").write_text(
      json.dumps({k: (v if not isinstance(v, np.ndarray) else None)
                  for k, v in spread.items() if k != "intensity"},
                 ensure_ascii=False, indent=2, default=float), encoding="utf-8")
   return 0


if __name__ == "__main__":
   raise SystemExit(main())
```

`step04b_figures.py`　D8 图件

```
"""Figures for current D8 routing runs, with distinct peak and final fields."""
from __future__ import annotations
import json
import numpy as np
import matplotlib
matplotlib.use('Agg')
```

```
import matplotlib.pyplot as plt
from mmlib import io_data as io
plt.rcParams['font.sans-serif']=['Microsoft YaHei','DejaVu Sans']
plt.rcParams['axes.unicode_minus']=False
plt.rcParams['mathtext.fontset']='dejavusans'
plt.rcParams.update({
   'font.size': 19,
   'axes.titlesize': 19,
   'axes.labelsize': 17,
   'xtick.labelsize': 15,
   'ytick.labelsize': 15,
   'legend.fontsize': 13,
})


def main():
   q=json.loads((io.DATA_OUT/'04b_二维引擎.json').read_text(encoding='utf-8'))
   b=q['runs']['base']
   dem,g=io.load_dem()
   cache,fig=io.CACHE,io.FIG_OUT
   arrival=np.load(cache/'q4b_arrival.npy')
   wet=np.isfinite(arrival)
   rr,cc=np.nonzero(wet)
   pad=25
   r0,r1=max(0,rr.min()-pad),min(dem.shape[0],rr.max()+pad)
   c0,c1=max(0,cc.min()-pad),min(dem.shape[1],cc.max()+pad)
   sl=np.s_[r0:r1,c0:c1]
   extent=[g['lon0']+c0*g['dlon'],g['lon0']+c1*g['dlon'],
         g['lat0']+r1*g['dlat'],g['lat0']+r0*g['dlat']]
   def map_field(ax,a,title,vmax=None):
      ax.imshow(dem[sl],extent=extent,cmap='Greys',alpha=.3)
      im=ax.imshow(np.ma.masked_less_equal(a[sl],0),extent=extent,cmap='YlOrRd',
                vmin=0,vmax=vmax)
      for name,rc,marker,col in [('注入',q['断面']['注入_rc'],'o','royalblue'),
                           ('口岸代理',q['断面']['口岸投影_rc'],'*','cyan')]:
         ax.plot(g['lon0']+rc[1]*g['dlon'],g['lat0']+rc[0]*g['dlat'],
                marker,color=col,ms=7,label=name)
      ax.set_title(title)
      ax.set_xlabel('经度 / °E');ax.set_ylabel('纬度 / °N')
      return im
   snapshots=[np.load(cache/f'q4b_snap_t{t}.npy') for t in (600,1200,1800,3600)]
   maximum=max(float(a.max()) for a in snapshots)
   f,axes=plt.subplots(2,2,figsize=(11,8),constrained_layout=True)
   for ax,t,a in zip(axes.ravel(),(600,1200,1800,3600),snapshots):
      im=map_field(ax,a,f't = {t/60:.0f} min',maximum)
   axes[0,0].legend(fontsize=13)
   f.colorbar(im,ax=axes.ravel().tolist(),label='等效泥深 / m',shrink=.75)
   f.suptitle('D8 单流向路由演进快照（不含横向二维动力）')
   f.savefig(fig/'04b_过程快照.png',dpi=180);plt.close(f)
   final=np.load(cache/'q4b_final_h.npy');peak=np.load(cache/'q4b_peak_h.npy')
   f,axes=plt.subplots(1,2,figsize=(11,5),constrained_layout=True)
   for ax,a,title in zip(axes,(peak,final),('历史最大泥深','终态残留泥深（分析域末端右截断）')):
      im=map_field(ax,a,title,float(peak.max()))
   f.colorbar(im,ax=axes,label='等效泥深 / m',shrink=.75)
   f.savefig(fig/'04b_峰值与终态.png',dpi=180);plt.close(f)
   f,axes=plt.subplots(1,2,figsize=(11,4.5),constrained_layout=True)
   aa=np.where(wet,arrival/60,0.)
   im=map_field(axes[0],aa,'首次达到分析泥深阈值的到达时间')
   f.colorbar(im,ax=axes[0],label='到达时间 / min')
   curve=b['leader']
   axes[1].plot([x['t_s']/60 for x in curve],[x['front_m']/1000 for x in curve])
   axes[1].set(xlabel='自共同注入起算 / min',ylabel='历史到达前锋里程 / km',title='前锋按首次 h≥阈值定义')
   axes[1].grid(alpha=.25)
   f.savefig(fig/'04b_到达与前锋.png',dpi=180);plt.close(f)
   f,axes=plt.subplots(2,2,figsize=(11,7),constrained_layout=True)
   l=b['ledger'];tt=[x['t_s']/60 for x in l]
   for key,label in [('V_inj_m3','累计注入'),('V_dom_m3','域内'),('V_out_m3','累计出域')]:
      axes[0,0].plot(tt,[x[key]/1e6 for x in l],label=label)
   axes[0,0].axhline(b['V_df_m3']/1e6,ls=':',color='black',label='源项目标')
   axes[0,0].legend(fontsize=14);axes[0,0].set(ylabel=r'体积 / ($10^6$ m$^3$)',xlabel='时间 / min',title='源项目标与质
      量账本')
   axes[0,1].plot(tt,[x['V_loss_m3']/max(x['V_inj_m3'],1) for x in l])
   axes[0,1].set(
      xlabel='时间 / min',
      ylabel='相对收支残差',
      title=f"max Co={b['actual_cfl_max']:.3g}\n源项注入差={b['source_relative_error']:.2e}（构造）",
   )
   p=b['port_hydrograph'];pt=[x['t_s']/60 for x in p]
   axes[1,0].plot(pt,[x['h_m'] for x in p],label='泥深')
   axes[1,0].axhline(b['peak_depth_port_m'],ls=':',label='历史峰深')
   axes[1,0].legend(fontsize=14);axes[1,0].set(xlabel='时间 / min',ylabel='口岸代理泥深 / m',title='口岸过程线：峰值不等于
      终态')
   labels=['主网格','粗网格','半 CFL']
   rs=[b,q['scale90_rerun'],q['time_refinement']]
   axes[1,1].bar(labels,[x['arrival_port_s']/60 if x['arrival_port_s'] is not None else np.nan for x in rs])
   axes[1,1].set(ylabel='到达口岸代理 / min',title='尺度及时间步对照（不是空间收敛证明）')
   for ax in axes.ravel():ax.grid(alpha=.2)
   f.savefig(fig/'04b_质量收支.png',dpi=180);plt.close(f)
   names=list(q['runs']);vals=[q['runs'][n]['arrival_port_s'] for n in names]
   f,ax=plt.subplots(figsize=(11,4),constrained_layout=True)
   cn={'base':'基准','V_low':'体积低','V_high':'体积高','V_chainB':'体积链B',
```

```
      'n_low':'糙率低','n_high':'糙率高','tauy_low':'屈服低','tauy_high':'屈服高',
      'Cv_low':'含固低','Cv_high':'含固高','smin_low':'坡限低','smin_high':'坡限高',
      'duration_low':'源历时短','duration_high':'源历时长','arrival_low':'到达阈值低','arrival_high':'到达阈值高'}
   ax.bar(range(len(names)),[x/60 if x is not None else np.nan for x in vals])
   ax.set_xticks(range(len(names)),[cn.get(n,n) for n in names],rotation=45,ha='right')
   ax.set(ylabel='到达口岸代理 / min',title='唯一定义配置的单因素情景（非概率样本）')
   ax.grid(axis='y',alpha=.2)
   f.savefig(fig/'04b_情景敏感性.png',dpi=180);plt.close(f)
   return 0

if __name__=='__main__':
   raise SystemExit(main())
```

数据整理与校验

step00_data_inventory.py 附件清单与元数据

```
"""第 0 步：数据装载、质量评估与空间要素核验，产出统一中间数据。

输出：
  求解结果/数据/cache/*.parquet|npy 清洗后的中间数据
  求解结果/数据/00_数据清单.json       数据本体、范围、缺测与站点元信息
  求解结果/数据/00_栅格分区统计.csv     灾害区 DEM/土地覆盖分区统计
  求解结果/数据/00_冰川dhdt统计.json   四期冰川高程变化率均值与误差
"""
from __future__ import annotations

import sys
from pathlib import Path

import numpy as np
import pandas as pd

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from mmlib import io_data as io # noqa: E402


def station_meta(df: pd.DataFrame) -> dict:
   """由本站气压反演站点海拔，并统计要素缺测率与观测有效期。"""
   p = df["Pressure"].dropna()
   p0, t0 = 1013.25, 288.15
   lapse, rd, g = 0.0065, 287.05, 9.80665
   elev = t0 / lapse * (1.0 - (p.median() / p0) ** (rd * lapse / g))
   qc_cols = {
      "风速": ["WS_1m", "WS_2m", "WS_4m", "WS_10m", "WS_20m"],
      "气温": ["Ta_1m", "Ta_2m", "Ta_4m", "Ta_10m", "Ta_20m"],
      "相对湿度": ["RH_1m", "RH_2m", "RH_4m", "RH_10m", "RH_20m"],
      "降水": ["Precip"],
      "辐射": ["Rsd", "Rsu", "Rld", "Rlu"],
      "土壤温湿": [c for c in df.columns if c.startswith(("soilT_", "VWC_"))],
      "湍流": ["H", "LE", "USTAR", "TKE", "TAU"],
   }
   miss = {}
   for k, cols in qc_cols.items():
      cols = [c for c in cols if c in df.columns]
      miss[k] = float(df[cols].isna().mean().mean()) if cols else None
   valid = df.notna().mean(axis=1)
   ok = df.index[valid > 0.5]
   return {
      "rows": int(len(df)),
      "valid_first": str(ok[0]) if len(ok) else None,
      "valid_last": str(ok[-1]) if len(ok) else None,
      "valid_hours": int(len(ok)),
      "missing_fraction_by_family": miss,
      "pressure_median_hPa": float(p.median()),
      "elevation_inferred_m": float(elev),
      "precip_annual_sum_mm": {str(k.year): float(v) for k, v in df["Precip"].resample("YS").sum().items()},
      "precip_max_hourly_mm": float(df["Precip"].max()),
      "ta_2m_mean": float(df["Ta_2m"].mean()),
      "ws_10m_mean": float(df["WS_10m"].mean()),
   }


def energy_balance_closure(df: pd.DataFrame) -> dict:
   rn = df["Rsd"] - df["Rsu"] + df["Rld"] - df["Rlu"]
   g0 = df["G_0.1m"] if "G_0.1m" in df else df["G_0.2m"]
   sub = pd.DataFrame({"Rn": rn, "H": df["H"], "LE": df["LE"], "G": g0}).dropna()
   hours = sub.index.hour
   sub = sub[(sub["Rn"] > 0) & (hours >= 9) & (hours <= 17)]
   avail = (sub["Rn"] - sub["G"]).to_numpy()
   turb = (sub["H"] + sub["LE"]).to_numpy()
   slope, icept = np.polyfit(avail, turb, 1)
   r2 = float(np.corrcoef(avail, turb)[0, 1] ** 2)
   return {"n": int(len(sub)), "slope": float(slope), "intercept": float(icept),
         "r2": float(r2), "ratio_sum": float(turb.sum() / avail.sum())}


def terrain_stats(dem: np.ndarray, lc: np.ndarray, grid: dict, lc_grid: dict,
             hazards: pd.DataFrame) -> pd.DataFrame:
```

```
    """对 DEM 覆盖范围内每个历史灾点沟道，统计高程、坡度、土地覆盖构成。"""
    from scipy import ndimage

    z = dem.astype("float64")
    dy = abs(grid["dlat"]) * 110574.0
    dx = grid["dlon"] * 111320.0 * np.cos(np.radians(grid["lat0"]))
    gy, gx = np.gradient(z, dy, dx)
    slope = np.degrees(np.arctan(np.hypot(gx, gy)))
    aspect = np.degrees(np.arctan2(-gx, gy)) % 360.0
    # 地形湿度指数：基于 D8 汇流累积的近似（多流向简化）
    filled = z.copy()
    filled[np.isnan(filled)] = np.nanmax(filled)
    acc = np.ones_like(filled)
    for _ in range(60):
        acc = np.minimum(1.0, ndimage.uniform_filter(acc, 3)) + 1.0
        acc = ndimage.gaussian_filter(acc, 0.6)
    tpi = z - ndimage.uniform_filter(z, 15)
    rows = []
    for _, r in hazards.iterrows():
        rr, cc = io.lonlat_to_rowcol(grid, r["lon"], r["lat"])
        ri, ci = int(round(rr)), int(round(cc))
        if not (0 <= ri < dem.shape[0] and 0 <= ci < dem.shape[1]):
            continue
        win = (slice(max(ri - 12, 0), ri + 13), slice(max(ci - 12, 0), ci + 13))
        lr, lcn = io.lonlat_to_rowcol(lc_grid, r["lon"], r["lat"])
        lri, lci = int(round(lr)), int(round(lcn))
        lwin = (slice(max(lri - 40, 0), lri + 41), slice(max(lci - 40, 0), lci + 41))
        sub_lc = lc[lwin]
        vals, counts = np.unique(sub_lc, return_counts=True)
        share = {io.LC_NAMES.get(int(v), str(v)): float(c / sub_lc.size) for v, c in zip(vals, counts)}
        rows.append({
            "code": r.get("课题"), "lon": r["lon"], "lat": r["lat"],
            "elev_m": float(z[ri, ci]), "slope_deg": float(np.nanmean(slope[win])),
            "aspect_deg": float(np.nanmean(aspect[win])), "tpi_m": float(np.nanmean(tpi[win])),
            "relief_1km_m": float(np.nanpercentile(z[win], 95) - np.nanpercentile(z[win], 5)),
            "lc_dominant": max(share, key=share.get) if share else None,
            "lc_veg_share": None,
            "lc_bare_share": None,
            "lc_water_ice_share": None,
            "lc_semantics_status": "unavailable: supplied class dictionary not verified; raw codes retained",
        })
    stats = pd.DataFrame(rows)
    stats["grad_obs"] = hazards.set_index("课题").reindex(stats["code"])["grad"].to_numpy()
    return stats


def main() -> int:
    print("[1/6] 装载站点观测 ...")
    station = io.load_station()
    meta = station_meta(station)
    meta["energy_balance_closure"] = energy_balance_closure(station)
    io.write_json(io.DATA_OUT / "00_站点元信息.json", meta)

    print("[2/6] 装载气温序列 ...")
    temp = io.load_temperature(use_cache=False)

    print("[3/6] 装载灾点与人口 ...")
    hazards = io.load_hazard_points(use_cache=False)
    census = io.load_census()

    print("[4/6] 装载 DEM 与土地覆盖 ...")
    dem, dgrid = io.load_dem()
    lc, lgrid = io.load_landcover(2023)

    print("[5/6] 灾点地形/覆被统计 ...")
    stats = terrain_stats(dem, lc, dgrid, lgrid, hazards)
    stats.to_csv(io.DATA_OUT / "00_栅格分区统计.csv", index=False, encoding="utf-8-sig")

    print("[6/6] 冰川高程变化统计 ...")
    dhdt = io.glacier_dhdt_stats()
    io.write_json(io.DATA_OUT / "00_冰川dhdt统计.json", dhdt)

    inventory = {
        "station": {**io.STATION, **meta},
        "temperature": {"county_count": 5, "first": str(temp.index.min().date()),
                        "last": str(temp.index.max().date()), "rows": int(len(temp)),
                        "columns": list(temp.columns)},
        "hazards": {"count": int(len(hazards)),
                    "types": hazards["类型"].value_counts().to_dict(),
                    "scales": hazards["规模"].value_counts().to_dict(),
                    "frequencies": hazards["频率"].value_counts().to_dict(),
                    "lon_range": [float(hazards["lon"].min()), float(hazards["lon"].max())],
                    "lat_range": [float(hazards["lat"].min()), float(hazards["lat"].max())],
                    "grad_range": [float(hazards["grad"].min()), float(hazards["grad"].max())],
                    "relief_range": [float(hazards["relief_m"].min()), float(hazards["relief_m"].max())]},
        "dem": {"shape": list(dem.shape), "grid": dgrid,
                "elev_range": [float(np.nanmin(dem)), float(np.nanmax(dem))],
                "cell_m": [float(dgrid["dlon"] * 111320 * np.cos(np.radians(dgrid["lat0"]))),
                           float(abs(dgrid["dlat"]) * 110574)]},
        "landcover": {"shape": list(lc.shape), "grid": lgrid,
                      "classes": {io.LC_NAMES.get(int(v), str(v)): float(c / lc.size)
                                  for v, c in zip(*np.unique(lc, return_counts=True))}},
        "glacier_dhdt": dhdt,
        "census": census,
```

```
        "event": io.EVENT,
    }
    io.write_json(io.DATA_OUT / "00_数据清单.json", inventory)
    print("完成: ", io.DATA_OUT / "00_数据清单.json")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

step06_data_audit.py 数据可用性核查

```
"""Offline repeatable source/time alignment and target-location sensitivity audit."""
from pathlib import Path
import hashlib,json
import numpy as np
import pandas as pd
from mmlib import io_data as io, terrain as tr, terrain_prep as tp
import step04b_sim2d as routing
ROOT=io.ROOT
RAW=ROOT/'临时产物/外部数据'
(io.DATA_OUT/'外部').mkdir(parents=True,exist_ok=True)
def read(name): return json.loads((RAW/name).read_text(encoding='utf-8'))
def meteo_hourly(name):
 d=read(name);h=d['hourly'];return d,pd.DataFrame({'T':h['temperature_2m'],'P':h['precipitation']},index=pd.to_datetime(h['time'],utc=True)).astype(float)
def nasa(name,hourly=True):
 d=read(name);p=d['properties']['parameter'];f=pd.DataFrame({'T':p['T2M'],'P':p['PRECTOTCORR']}).astype(float)
 f.index=pd.to_datetime(f.index,format='%Y%m%d%H' if hourly else '%Y%m%d',utc=True)
 return d,f.replace(d['header']['fill_value'],np.nan)
def day_summary(frame):
 out=[]
 for day,g in frame.tz_convert('Asia/Shanghai').groupby(lambda t:t.date()):
  if len(g)!=24 or not g.notna().all().all():continue
  out.append({'date':str(day),'n_hours':24,'mean_T_C':float(g['T'].mean()),'max_T_C':float(g['T'].max()),'min_T_C':float(g['T'].min()),'precip_mm':float(g['P'].sum())})
 return out
def external():
 om,a=meteo_hourly('open_meteo_era5_port_utc.json');pw,b=nasa('nasa_power_port_hourly_utc.json');hf,c=meteo_hourly('historical_forecast_ifs.json')
 station=io.load_station()[['Ta_2m','Precip']].rename(columns={'Ta_2m':'T','Precip':'P'}).copy()
 station.index=station.index.tz_localize('Asia/Shanghai').tz_convert('UTC')
 nd,n=nasa('nasa_power_station_daily_utc.json',False)
 count=station.resample('D').count();sd=station.resample('D').agg({'T':'mean','P':'sum'}).where(count==24)
 pair=sd.join(n,lsuffix='_station',rsuffix='_nasa').dropna()
 annual=[]
 for year,g in pair.groupby(pair.index.year):
  annual.append({'year':int(year),'common_complete_UTC_days':len(g),'first':str(g.index.min()),'last':str(g.index.max()),'station_precip_mm':float(g.P_station.sum()),'nasa_precip_mm':float(g.P_nasa.sum()),'ratio_nasa_to_station':float(g.P_nasa.sum()/g.P_station.sum()) if g.P_station.sum()>0 else None,'temperature_bias_nasa_minus_station_C':float((g.T_nasa-g.T_station).mean())})
 pair.to_csv(io.DATA_OUT/'外部/06_同日界逐日站点对照.csv',encoding='utf-8-sig',index_label='UTC_day')
 result={'version':'v5.0','event_day_timezone':'Asia/Shanghai','hour_stamp_assumption':'hour labels treated as hour-start; actual instrument label convention unknown','open_meteo_era5':{'request_model':'era5','elevation_downscaling':'disabled: elevation=nan','returned_grid':{k:om[k] for k in ('latitude','longitude','elevation','timezone')},'days':day_summary(a)},'nasa_power_merra2':{'header':pw['header'],'geometry':pw['geometry'],'days':day_summary(b)},'station_comparison':{'coordinate_status':'assumed representative point, not verified real station','station_timezone_assumption':'UTC+8 converted to UTC','daily_policy':'exactly 24 valid observed hours for BOTH temperature and precipitation; paired common UTC dates only','annual_matched_subsets':annual},'forecast_archive':{'model':'ecmwf_ifs04','returned_grid':{k:hf[k] for k in ('latitude','longitude','elevation','timezone')},'days':day_summary(c),'initialization':None,'lead_time':None,'status':'valid-time archive retrieved; not proof of pre-event issuance or forecast skill'},'interpretation':'Inter-product comparison at coarse grids; no independence guarantee or same-site validation. Old default-product and 18 K altitude-only explanation withdrawn.'}
 io.write_json(io.DATA_OUT/'外部/06_气象同口径核验.json',result)
 return result
def location_sensitivity():
 terrain=tp.ensure_terrain_cache();dem,grid=io.load_dem();terrain['grid']=grid
 mask,_,gt=routing.glacier_src_region();source,_=routing.reproject_src_cells(mask,gt,grid,dem.shape)
 volume=routing.load_volumes()['链A_V_df_m3'];shape=dem.shape;rows=[]
 targets=[('working',(io.EVENT['port_lon'],io.EVENT['port_lat'])),('alternative_reference',(85.3777,28.2797))]
 for name,target in targets:
  inj,port,out,offset=routing.project_sections(terrain,source,target)
  r=routing.simulate2d(terrain['recv'],terrain['slink'],terrain['dist'].ravel(),terrain['cell'],source,np.array([inj]),volume,{},port,out)
  rc=tuple(int(x) for x in np.unravel_index(port,shape));cm=tr.upstream_mask(terrain['dir'],rc)
  rr,cc=io.lonlat_to_rowcol(grid,*target);cand=np.argwhere(terrain['acc']>=5e5);near=tuple(int(x) for x in cand[np.argmin(np.hypot(cand[:,0]-rr,cand[:,1]-cc))]);hydro_mask=tr.upstream_mask(terrain['dir'],near)
  path=tr.trace_downstream(terrain['dir'],np.unravel_index(inj,shape));pi=path.index(rc);p=np.asarray(path[:pi+1]);dist=float(np.hypot(np.diff(p[:,0])*terrain['cell'][0],np.diff(p[:,1])*terrain['cell'][1]).sum())
  rows.append({'target':name,'lonlat':target,'route_snap_rc':rc,'route_snap_offset_m':offset,'route_length_m':dist,'route_section_upstream_area_km2':float(cm.sum()*np.prod(terrain['cell'])/1e6),'water_model_outlet_rc':near,'water_model_upstream_area_km2':float(hydro_mask.sum()*np.prod(terrain['cell'])/1e6),'arrival_s':r['arrival_port_s'],'peak_depth_m':r['peak_depth_port_m'],'peak_velocity_m_s':r['peak_velocity_port_m_s'],'peak_intensity_N_m':r['peak_intensity_port_N_m'],'mass_closure':r['closure_max']})
 result={'version':'v5.0','targets':rows,'alternative_coordinate_status':'inherited public-reference candidate; live Wikidata retrieval failed, exact affected section unresolved','method':'same source, grid, volume and physical parameters; only target section changed. Route and trigger-outlet snapping reported separately.','scope':'location sensitivity, not observed affected-section verification; peak intensity is not observed peak discharge'}
 io.write_json(io.DATA_OUT/'06_坐标敏感性.json',result);return result
def main():
 external();location_sensitivity()
 io.write_json(io.DATA_OUT/'06_数据可用性.json',{'version':'v5.0','actual_disaster_trigger_95CI':None,'actual_disaster_coverage':None,'issued_forecast_validation':None,'landcover_physical_weights':None,'
```

```
        station_metadata':'unverified coordinates, altitude, timezone and hour-label semantics','resolved':['inclusive
         floating endpoints','training-only constant baseline','RGI6 inventory boundary supply area','unknown
        landcover physical weights disabled','common UTC complete-day external comparison','target-coordinate
        sensitivity','source response archive','IFS single-run pre-issue availability at frozen 08:00 issue'],'
        unresolved':['real multi-event start times and confirmed non-events','real station installation metadata','
        2026 glacier and snow extent','verified landcover code dictionary','actual affected target section','
        verification of actual operational distribution latency','real source release and along-route inventory']})
 print('external alignment and target-section sensitivity complete')
 return 0
if __name__=='__main__':raise SystemExit(main())
```

archive_external.py 外部响应归档

```
"""Offline verification of archived external responses.

The competition run is offline. Raw API responses retrieved during the single
network session are archived under 临时产物/外部数据/; this script re-computes
their SHA-256 and writes the provenance record without any network access.
The optional online refresh helper is not included in this offline package
and is not part of run_all.py.
"""
from __future__ import annotations

import hashlib
import json
from pathlib import Path

ROOT = Path(__file__).resolve().parents[2]
RAW = ROOT / "临时产物" / "外部数据"
OUT = ROOT / "求解结果" / "数据" / "外部"

SOURCES = [
    {
        "tag": "open_meteo_era5_port_utc",
        "raw_file": "open_meteo_era5_port_utc.json",
        "product": "Open-Meteo Archive API, model=era5",
        "request": {
            "latitude": 28.2797,
            "longitude": 85.3777,
            "start_date": "2026-08-20",
            "end_date": "2026-08-27",
            "hourly": ["temperature_2m", "precipitation", "wind_speed_10m"],
            "timezone": "UTC",
            "elevation": "nan (automatic downscaling disabled)",
        },
        "scope": "event-window external comparison only; not a model input",
    },
    {
        "tag": "nasa_power_port_hourly_utc",
        "raw_file": "nasa_power_port_hourly_utc.json",
        "product": "NASA POWER hourly point, MERRA-2 based",
        "request": {
            "parameters": ["T2M", "PRECTOTCORR"],
            "longitude": 85.3777,
            "latitude": 28.2797,
            "start": "20260819",
            "end": "20260828",
            "time-standard": "UTC",
        },
        "scope": "event-window external comparison only; not a model input",
    },
    {
        "tag": "nasa_power_station_daily_utc",
        "raw_file": "nasa_power_station_daily_utc.json",
        "product": "NASA POWER daily point, MERRA-2 based",
        "request": {
            "parameters": ["T2M", "PRECTOTCORR"],
            "longitude": 85.30,
            "latitude": 28.40,
            "start": "20210101",
            "end": "20250802",
            "time-standard": "UTC",
        },
        "scope": "assumed representative station point; not verified installation metadata",
    },
    {
        "tag": "historical_forecast_ifs",
        "raw_file": "historical_forecast_ifs.json",
        "product": "Open-Meteo Historical Forecast API, model=ecmwf_ifs04",
        "request": {
            "latitude": 28.2797,
            "longitude": 85.3777,
            "start_date": "2026-08-20",
            "end_date": "2026-08-27",
            "hourly": ["temperature_2m", "precipitation"],
            "timezone": "UTC",
            "elevation": "nan",
        },
        "scope": "valid-time archive only; no stored initialization or lead time, so not an issued-forecast
             validation",
    },
]

DOCUMENTATION = [
    ("open_meteo_product_docs", "open_meteo_docs.html", "Open-Meteo historical weather API documentation"),
    ("nasa_daily_time_standard", "nasa_power_daily_docs.html", "NASA POWER daily temporal API documentation"),
```

```
]


def sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def main() -> int:
    OUT.mkdir(parents=True, exist_ok=True)
    sources, missing = [], []
    for item in SOURCES:
        path = RAW / item["raw_file"]
        if not path.is_file():
            missing.append(item["raw_file"])
            continue
        raw = path.read_bytes()
        try:
            keys = list(json.loads(raw))[:15]
        except (ValueError, UnicodeError):
            keys = None
        sources.append({
            "tag": item["tag"],
            "product": item["product"],
            "request": item["request"],
            "raw_file": path.relative_to(ROOT).as_posix(),
            "bytes": len(raw),
            "sha256": sha256(path),
            "json_keys": keys,
            "retrieval": "archived during the single online session; this record is recomputed offline",
            "status": "archived" if keys is not None else "archived_unparsed",
            "scope": item["scope"],
        })
    documentation = []
    for tag, name, product in DOCUMENTATION:
        path = RAW / name
        documentation.append({
            "tag": tag,
            "product": product,
            "raw_file": path.relative_to(ROOT).as_posix() if path.is_file() else None,
            "sha256": sha256(path) if path.is_file() else None,
            "status": "archived" if path.is_file() else "missing",
        })
    result = {
        "schema_version": 2,
        "version": "v5.2",
        "network_access": "none during this run; hashes recomputed from archived responses",
        "sources": sources,
        "documentation": documentation,
        "missing_raw_files": missing,
        "withdrawn_claims": {
            "open_meteo_default_product": "unqualified default is Best Match (IFS HRES + ERA5 + ERA5-Land); an
                unqualified ERA5 claim is not supportable",
            "nasa_default_day_boundary": "default is local standard time, so a UTC+8 versus default-LST daily
                comparison is not a valid same-day comparison",
            "event_parameters_from_media": "reported runout, travel time, front speed and peak discharge have no
                verifiable original text or observation definition for this event and are withdrawn from validation",
            "station_metadata": "public station metadata endpoint returned no usable record; assumed coordinates,
                inferred altitude, UTC+8 and hour-label convention remain hypotheses",
            "issued_forecast": "archived forecast response carries valid times only; without initialization and lead
                time it cannot demonstrate pre-event availability or skill",
        },
    }
    (OUT / "06_外部来源核实.json").write_text(
        json.dumps(result, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    print(f"external provenance archived: {len(sources)} sources, {len(documentation)} documents, {len(missing)}
        missing")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

select_paper_values.py　论文取值汇总

```
"""Build the ASCII-key paper selection from current v5.0 result files."""
from __future__ import annotations

import json
from pathlib import Path


ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "求解结果" / "数据"
OUT = DATA / "00_paper_values_v50_selected.json"


def load(name: str):
    return json.loads((DATA / name).read_text(encoding="utf-8"))


def main() -> int:
    q1 = load("01c_统一关联前五.json")
    gully = load("02_沟道临界条件.json")
    old = load("02_旧方法审计.json")
    trigger = load("02_触发评估模型.json")
```

```
    api_quantiles = load("02_API分位.json")
    state_tuple = load("02_状态元组.json")
    reliability = load("02_多尺度可靠性.json")
    q3 = load("03_前向预报.json")
    distance = load("03_空间距离范围.json")
    q4 = load("04b_二维引擎.json")
    baseline = load("04_仿真基线.json")

    top5 = [
        {
            "rank": row["rank"],
            "variable": row["variable"],
            "name": row["name"],
            "family": row["family"],
            "direction": row["direction"],
            "A_abs": row["A_abs"],
            "signed_z": row["signed_z"],
            "delta": row["delta"],
            "baseline_median": row["baseline_median"],
            "tail_rank": row["tail_rank"],
            "cluster_members": row["cluster_members"],
            "cluster_representative_reason": row["cluster_representative_reason"],
        }
        for row in q1["top5"]
    ]
    sensitivity72 = q1["sensitivity_72h"]["top5"]
    leave = {
        year: info["top5_overlap"]
        for year, info in q1["leave_one_reference_year_out"].items()
    }
    pseudo = q1["pseudo_2025_same_calendar"]
    rank = gully["rank_test_statistics"]
    runs = list(q4["runs"].values())
    base = q4["runs"]["base"]

    payload = {
        "q1": {
            "top5": top5,
            "refs": q1["main_168h"]["reference_window_count"],
            "sensitivity72": sensitivity72,
            "leave": leave,
            "pseudo": pseudo["top5"],
            "pseudo_max_A": pseudo["max_A"],
        },
        "q2": {
            "states": gully["机械状态计数"],
            "qcrit_q": gully["起动流量_m3s"],
            "old_q": gully["旧式起动流量_m3s"],
            "rank": rank,
            "formal_qcrit_m2s": old["formal_q_crit_m2s"],
            "formal_C_N0": old["formal_C_N0"],
            "CN_provenance": old["formal_CN_provenance"],
            "CN_sensitivity": trigger["曲线数情景"],
            "legacy_k": old["legacy_k"],
            "event_peak": old["event_window_T_peak"],
            "event_T": trigger["T_事件时刻_10:00"],
            "event_first": trigger["事件窗口首次超过0.5"],
            "api_quantiles": api_quantiles,
            "state_tuple": state_tuple,
            "auc": trigger.get("空间排序AUC_易发性预测大规模"),
            "reliability": reliability,
        },
        "q3": {
            "issue": q3["issue_time_local"],
            "tm": q3["intervals"]["t_m"],
            "ta": q3["intervals"]["t_a"],
            "unconditional_status": q3["intervals"]["unconditional_status"],
            "unconditional_status_by_target": q3["intervals"][
                "unconditional_status_by_target"
            ],
            "tm_state": q3["state_summary_tm"]["counts"],
            "tm_state_summary": q3["state_summary_tm"],
            "ta_state": q3["state_summary_ta"]["counts"],
            "ta_state_summary": q3["state_summary_ta"],
            "warning": q3["warning"],
            "coverage": q3["coverage"],
            "distance": distance,
            "ifs": q3["ifs_manifest"],
            "fit": q3["fit"],
        },
        "q4": {
            "base_arrival_s": base["arrival_port_s"],
            "base_arrival_min": base["arrival_port_s"] / 60.0,
            "runout_L_m": base["runout_L_m"],
            "runout_right": bool(base["runout_right_censored"]),
            "termination_reason": base["termination_reason"],
            "loop_termination_reason": base["loop_termination_reason"],
            "natural_stop": bool(base["natural_stop"]),
            "domain_censor": bool(base["domain_censor"]),
            "critical_depth_slope_product_m": base["critical_depth_slope_product_m"],
            "critical_depth_port_m": base["critical_depth_port_m"],
            "domain_front_max": base["domain_front_max_m"],
            "configs": len(runs),
            "config_arrival_min": [row["arrival_port_s"] / 60.0 for row in runs],
            "max_depth": base["max_depth_m"],
```

```
            "final_max_depth": base["final_max_depth_m"],
            "peak_port": base["peak_depth_port_m"],
            "final_port": base["final_depth_port_m"],
            "peak_v": base["peak_velocity_port_m_s"],
            "closure": base["closure_max"],
            "cfl": base["actual_cfl_max"],
            "one_d_base_arrival_s": baseline["arrival_outlet_s"],
            "one_d_runout_km": baseline["runout_km"],
            "one_d_reached_outlet": baseline["reached_outlet"],
        },
    }
    OUT.write_text(
        json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False) + "\n",
        encoding="utf-8",
    )
    print(f"paper selection written: {OUT.relative_to(ROOT).as_posix()}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

validate_v50.py 确定性结果验收

```
"""Deterministic acceptance checks for the complete v5.0 result chain."""
from __future__ import annotations

import json
import hashlib
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "求解结果" / "数据"
EXTERNAL = ROOT / "临时产物" / "外部数据"
OUT = DATA / "00_v50验收报告.json"


def load(name: str):
    return json.loads((DATA / name).read_text(encoding="utf-8"))


def check(condition: bool, message: str, errors: list[str]) -> None:
    if not condition:
        errors.append(message)


def main() -> int:
    errors: list[str] = []
    checks: dict[str, object] = {}

    q1 = load("01c_统一关联前五.json")
    check(q1.get("candidate_count") == 31, "q1_candidate_count_not_31", errors)
    check(
        q1.get("candidate_families") == {"MET": 19, "RADM": 5, "SOIL": 3, "FLUX": 4},
        "q1_family_counts_mismatch",
        errors,
    )
    check(len(q1.get("top5", [])) == 5, "q1_top5_length_not_5", errors)
    check(
        q1.get("main_168h", {}).get("reference_window_count", 0) >= 80,
        "q1_reference_windows_insufficient",
        errors,
    )
    candidate_168 = pd.read_csv(DATA / "01c_168h候选完整排序.csv")
    candidate_72 = pd.read_csv(DATA / "01c_72h候选完整排序.csv")
    cluster_168 = pd.read_csv(DATA / "01c_168h簇完整排序.csv")
    cluster_72 = pd.read_csv(DATA / "01c_72h簇完整排序.csv")
    for tag, frame in (("168h", candidate_168), ("72h", candidate_72)):
        check(len(frame) == 31, f"q1_{tag}_candidate_rows_not_31", errors)
        check(frame["variable"].nunique() == 31, f"q1_{tag}_candidate_keys_not_unique", errors)
        rankable = frame.loc[frame["rankable"].astype(bool)]
        check(
            rankable["rank"].notna().all()
            and frame.loc[~frame["rankable"].astype(bool), "rank"].isna().all(),
            f"q1_{tag}_candidate_rank_contract_wrong",
            errors,
        )
        check(
            rankable["A_abs"].notna().all(),
            f"q1_{tag}_rankable_candidate_missing_score",
            errors,
        )
    expected_clusters_168 = len(q1["main_168h"]["cluster_ranking"])
    expected_clusters_72 = len(q1["sensitivity_72h"]["cluster_ranking"])
    check(
        len(cluster_168) == expected_clusters_168,
        "q1_168h_cluster_rows_incomplete",
        errors,
    )
    check(
        len(cluster_72) == expected_clusters_72,
        "q1_72h_cluster_rows_incomplete",
```

```
    errors,
)
checks["q1"] = {
    "candidate_count": q1.get("candidate_count"),
    "top5": [row["variable"] for row in q1.get("top5", [])],
    "reference_windows": q1.get("main_168h", {}).get("reference_window_count"),
    "candidate_full_ranking_rows": {
        "168h": len(candidate_168),
        "72h": len(candidate_72),
    },
    "cluster_full_ranking_rows": {
        "168h": len(cluster_168),
        "72h": len(cluster_72),
    },
}

q2 = load("02_沟道临界条件.json")
states = q2.get("机械状态计数", {})
allowed = {"water_controlled", "friction_equilibrium", "gravity_unstable", "not_evaluable"}
check(q2.get("沟道数") == 84, "q2_gully_count_not_84", errors)
check(sum(states.values()) == 84, "q2_state_counts_do_not_sum_84", errors)
check(set(states) <= allowed, "q2_unknown_mechanical_state", errors)
check(
    q2.get("water_controlled_n") + q2.get("zero_threshold_n") + q2.get("not_evaluable_n") == 84,
    "q2_substantive_branch_counts_do_not_sum_84",
    errors,
)
rank_tests = q2.get("rank_test_statistics", {})
check(rank_tests.get("sample_n") == 78, "q2_rank_sample_not_78", errors)
expected_rank_values = {
    ("total_flow_Q_crit_m3s", "kendall_vs_scale"): (
        "tau", 0.408386, 5.471436410144295e-06,
    ),
    ("total_flow_Q_crit_m3s", "spearman_vs_scale"): (
        "rho", 0.498676, 3.3745699520687622e-06,
    ),
    ("unit_width_q_crit_m2s", "kendall_vs_scale"): (
        "tau", 0.318602187, 0.000390472,
    ),
    ("unit_width_q_crit_m2s", "spearman_vs_scale"): (
        "rho", 0.400619987, 0.000278478,
    ),
}
for (quantity, test_name), (statistic_key, expected_stat, expected_p) in (
    expected_rank_values.items()
):
    entry = rank_tests.get(quantity, {}).get(test_name, {})
    observed_stat = entry.get(statistic_key)
    observed_p = entry.get("p_value")
    check(
        observed_stat is not None
        and np.isclose(float(observed_stat), expected_stat, rtol=2e-6, atol=1e-9),
        f"q2_rank_stat_mismatch:{quantity}:{test_name}",
        errors,
    )
    check(
        observed_p is not None
        and np.isfinite(float(observed_p))
        and float(observed_p) > 0.0
        and np.isclose(float(observed_p), expected_p, rtol=2e-5, atol=0.0),
        f"q2_rank_p_mismatch:{quantity}:{test_name}",
        errors,
    )
old = load("02_旧方法审计.json")
trigger_model = load("02_触发评估模型.json")
api_quantiles = load("02_API分位.json")
state_tuple = load("02_状态元组.json")
hourly_state = pd.read_csv(DATA / "02_状态元组_逐小时.csv")
object_state = pd.read_csv(DATA / "02_状态元组_逐沟道.csv")
check(
    api_quantiles.get("training_years") == [2021, 2022, 2023, 2024, 2025],
    "q2_api_training_years_wrong",
    errors,
)
check(
    api_quantiles.get("training_months") == [6, 7, 8, 9],
    "q2_api_training_months_wrong",
    errors,
)
check(
    api_quantiles.get("training_valid_hours", 0) >= 100
    and api_quantiles.get("q1") <= api_quantiles.get("q3"),
    "q2_api_quantiles_invalid",
    errors,
)
tuple_fields = ["G", "H", "Theta", "F_ice", "U_ice", "C_ice"]
check(
    state_tuple.get("fields") == tuple_fields,
    "q2_state_tuple_fields_wrong",
    errors,
)
check(
    state_tuple.get("ice_mechanics_unresolved") is True,
    "q2_ice_mechanics_unresolved_flag_wrong",
    errors,
```

```
    )
    check(
        set(tuple_fields + ["ice_mechanics_unresolved"]) <= set(hourly_state.columns),
        "q2_hourly_state_tuple_columns_missing",
        errors,
    )
    check(
        set(tuple_fields + ["ice_mechanics_unresolved"]) <= set(object_state.columns),
        "q2_object_state_tuple_columns_missing",
        errors,
    )
    check(len(object_state) == 84, "q2_object_state_rows_not_84", errors)
    check(
        object_state["object_id"].nunique() == 84,
        "q2_object_state_ids_not_unique",
        errors,
    )
    gully_cache = DATA / "cache" / "gully_table.parquet"
    if gully_cache.is_file():
        source_codes = set(
            pd.read_parquet(gully_cache)["Code"].astype(str)
        )
        check(
            set(object_state["object_id"].astype(str)) == source_codes,
            "q2_object_state_ids_do_not_match_source_codes",
            errors,
        )
    if "Q_water_m3s" in hourly_state and "H" in hourly_state:
        missing_water = hourly_state["Q_water_m3s"].isna()
        check(
            hourly_state.loc[missing_water, "H"].eq("not_evaluable").all(),
            "q2_missing_water_not_marked_not_evaluable",
            errors,
        )
    check(
        trigger_model.get("状态元组", {}).get("fields") == tuple_fields,
        "q2_trigger_model_state_tuple_missing",
        errors,
    )
    density_15 = q2.get("床面固相含量敏感性", {}).get("0.15", {})
    check(
        np.isclose(density_15.get("density_kg_m3", np.nan), 1247.5, atol=1e-12),
        "q2_bed_density_cv015_wrong",
        errors,
    )
    check(
        np.isclose(density_15.get("delta_rho_kg_m3", np.nan), 247.5, atol=1e-12),
        "q2_delta_rho_cv015_wrong",
        errors,
    )
    check(
        density_15.get("qcrit_m2s", {}).get("q50", 0) > 0,
        "q2_qcrit_cv015_not_positive",
        errors,
    )
    check(old.get("formal_model_uses_legacy_k") is False, "q2_formal_model_uses_legacy_k", errors)
    check(old.get("formal_q_crit_m2s", 0) > 0, "q2_formal_qcrit_nonpositive", errors)
    check(old.get("formal_C_N0") == 85.0, "q2_formal_cn0_not_85", errors)
    cn = trigger_model.get("曲线数情景", {})
    check(cn.get("formal_C_N0") == 85.0, "q2_cn_metadata_missing", errors)
    check(
        cn.get("formal_event_window") == ["2026-08-26 08:00", "2026-08-26 13:00"],
        "q2_cn_event_window_wrong",
        errors,
    )
    check(set(cn.get("scenarios", {})) == {"75", "85", "95"}, "q2_cn_scenarios_not_75_85_95", errors)
    check(
        all(
            row.get("event_Qwater_peak_m3s") is not None
            for row in cn.get("scenarios", {}).values()
        ),
        "q2_cn_scenario_result_missing",
        errors,
    )
    check(
        all(
            row.get("formal_event_window_start") == "2026-08-26 08:00"
            and row.get("formal_event_window_end") == "2026-08-26 13:00"
            and row.get("event_T_peak_time") is not None
            for row in cn.get("scenarios", {}).values()
        ),
        "q2_cn_window_or_peak_time_missing",
        errors,
    )
    top_gullies = q2.get("前10易发沟道", [])
    name_256 = next((row for row in top_gullies if str(row.get("Name")) == "256"), None)
    check(name_256 is not None, "q2_name_256_missing_from_top10", errors)
    if name_256 is not None:
        check(
            name_256.get("Q_crit_m3s", 0) > 0,
            "q2_name_256_positive_threshold_rounded_to_zero",
            errors,
        )
        check(
            name_256.get("mechanical_regime") == "water_controlled",
            "q2_name_256_mechanical_regime_missing",
```

```
        errors,
    )
    check(
        name_256.get("q_crit_positive") is True,
        "q2_name_256_positive_flag_missing",
        errors,
    )
checks["q2"] = {
    "mechanical_states": states,
    "rank_tests": rank_tests,
    "formal_qcrit_m2s": old.get("formal_q_crit_m2s"),
    "event_window_T_peak": old.get("event_window_T_peak"),
    "formal_C_N0": old.get("formal_C_N0"),
    "cn_scenarios": cn.get("scenarios"),
    "legacy_k_audit_only": old.get("formal_model_uses_legacy_k") is False,
}

q3 = load("03_前向预报.json")
intervals = pd.read_csv(DATA / "forecast_intervals.csv")
coverage = pd.read_csv(DATA / "coverage_events.csv")
warning = pd.read_csv(DATA / "warning_events.csv")
advisory = pd.read_csv(DATA / "melt_advisory_events.csv")
members = pd.read_csv(DATA / "03_前向预报成员.csv")
runout = pd.read_csv(DATA / "runout_ranges.csv")
legacy = load("03_旧条件反演对照.json")
check(
    legacy.get("status") == "legacy_audit_only_not_used",
    "q3_legacy_status_wrong",
    errors,
)
check(
    legacy.get("downstream_use") == {
        "forward_forecast": False,
        "coverage": False,
        "warning": False,
        "propagation": False,
    },
    "q3_legacy_downstream_use_not_disabled",
    errors,
)
source_audit = legacy.get("source_audit", {})
source_path = ROOT / str(source_audit.get("path", ""))
check(source_path.is_file(), "q3_legacy_source_missing", errors)
if source_path.is_file():
    source_hash = hashlib.sha256(source_path.read_bytes()).hexdigest()
    check(
        source_audit.get("sha256") == source_hash,
        "q3_legacy_source_hash_mismatch",
        errors,
    )
    check(
        legacy.get("legacy_record", {}).get("source_file_sha256")
        == json.loads(source_path.read_text(encoding="utf-8")).get("source_sha256"),
        "q3_legacy_original_source_hash_mismatch",
        errors,
    )
check(
    q3.get("legacy_inverse_audit", {}).get("output")
    == "求解结果/数据/03_旧条件反演对照.json",
    "q3_legacy_output_pointer_missing",
    errors,
)
check(q3.get("issue_time_local") == "2026-08-26T08:00:00+08:00", "q3_issue_time_wrong", errors)
check(len(q3.get("ifs_manifest", {}).get("source_runs", [])) == 3, "q3_source_run_count_not_3", errors)
check(
    all(row.get("all_future_hours_available") for row in q3["ifs_manifest"]["source_runs"]),
    "q3_future_hours_missing",
    errors,
)
check(len(intervals) == 2, "q3_interval_rows_not_2", errors)
check(
    intervals["interval_method"].eq("uniform_within_support_event_time_quantiles").all(),
    "q3_event_time_interval_method_wrong",
    errors,
)
check(
    intervals["interval_status"].isin([
        "available", "not_constructible_zero_finite",
        "insufficient_finite_members",
    ]).all(),
    "q3_interval_status_unknown",
    errors,
)
endpoint_columns = [
    "lower_time", "median_time", "upper_time", "width_hours",
    "support_envelope_lower_time", "support_envelope_upper_time",
    "support_envelope_width_hours",
    "support_endpoint_95_lower_time", "support_endpoint_95_upper_time",
    "support_endpoint_95_width_hours",
]
unavailable = intervals.loc[intervals["interval_status"].ne("available")]
if len(unavailable):
    check(
        unavailable[endpoint_columns].isna().all().all(),
        "q3_nonconstructible_interval_has_endpoint_values",
        errors,
```

```
    )
available_intervals = intervals.loc[intervals["interval_status"].eq("available")]
if len(available_intervals):
    check(
        available_intervals[
            ["support_envelope_lower_time", "support_envelope_upper_time"]
        ].notna().all().all(),
        "q3_support_envelope_missing",
        errors,
    )
check(len(coverage) == 3, "q3_coverage_rows_not_3", errors)
check(set(coverage["target_definition"]) == {
    "true_debris_flow_formation", "instant_release_scenario_arrival",
    "actual_port_arrival",
}, "q3_coverage_target_contract_wrong", errors)
check(len(members) == 3000, "q3_member_count_not_3000", errors)
check(len(runout) == 3000, "q3_runout_rows_not_3000", errors)
check(
    np.isclose(float(runout["weight"].sum()), 1.0, atol=1e-12),
    "q3_runout_weights_do_not_sum_one",
    errors,
)
q4_configs = load("04b_二维引擎.json").get("runs", {})
check(
    set(runout["d8_configuration_id"].astype(str)) == set(q4_configs),
    "q3_runout_configuration_ids_mismatch",
    errors,
)
for config_id, group in runout.groupby("d8_configuration_id", sort=True):
    config = q4_configs[str(config_id)]
    expected_right = bool(config.get("runout_right_censored"))
    expected_natural = bool(config.get("natural_stop"))
    expected_time_limit = bool(config.get("time_limit", False))
    expected_step_limit = bool(config.get("step_limit", False))
    expected_domain_censor = bool(config.get("domain_censor", False))
    check(
        group["right_censored"].astype(bool).eq(expected_right).all(),
        f"q3_runout_right_censored_mismatch:{config_id}",
        errors,
    )
    check(
        group["natural_stop"].astype(bool).eq(expected_natural).all(),
        f"q3_runout_natural_stop_mismatch:{config_id}",
        errors,
    )
    check(
        group["time_limit"].astype(bool).eq(expected_time_limit).all(),
        f"q3_runout_time_limit_flag_mismatch:{config_id}",
        errors,
    )
    check(
        group["step_limit"].astype(bool).eq(expected_step_limit).all(),
        f"q3_runout_step_limit_flag_mismatch:{config_id}",
        errors,
    )
    check(
        group["domain_censor"].astype(bool).eq(expected_domain_censor).all(),
        f"q3_runout_domain_censor_flag_mismatch:{config_id}",
        errors,
    )
    expected_time_nonarrival = bool(
        expected_time_limit
        and not bool(config.get("reached_port", False))
    )
    expected_step_nonarrival = bool(
        expected_step_limit
        and not bool(config.get("reached_port", False))
    )
    expected_domain_nonarrival = bool(
        expected_domain_censor
        and not bool(config.get("reached_port", False))
    )
    check(
        group["time_limit_nonarrival"].astype(bool).eq(expected_time_nonarrival).all(),
        f"q3_runout_time_limit_mismatch:{config_id}",
        errors,
    )
    check(
        group["step_limit_nonarrival"].astype(bool).eq(expected_step_nonarrival).all(),
        f"q3_runout_step_limit_nonarrival_mismatch:{config_id}",
        errors,
    )
    check(
        group["domain_censor_nonarrival"].astype(bool).eq(expected_domain_nonarrival).all(),
        f"q3_runout_domain_censor_nonarrival_mismatch:{config_id}",
        errors,
    )
expected_tm = {"finite", "no_event", "active_at_issue", "right_censored", "not_evaluable"}
expected_ta = {
    "finite", "no_event", "active_at_issue_left_censored",
    "no_arrival", "right_censored", "not_evaluable",
}
for summary_key, expected_states in (
    ("state_summary_tm", expected_tm),
    ("state_summary_ta", expected_ta),
):
    summary = q3[summary_key]
```

```
    check(set(summary["counts"]) == expected_states, f"{summary_key}_keys_wrong", errors)
    check(sum(summary["counts"].values()) == summary["total_members"], f"{summary_key}_sum_failed", errors)
    check(
        all(
            np.isclose(
                summary[f"p_{state}"],
                summary["counts"][state] / summary["total_members"],
                atol=1e-12,
            )
            for state in expected_states
        ),
        f"{summary_key}_masses_wrong",
        errors,
    )
tm_summary = q3["state_summary_tm"]
ta_summary = q3["state_summary_ta"]
check(
    np.isclose(
        tm_summary["p_actionable"],
        tm_summary["p_finite"] + tm_summary["p_active_at_issue"],
        atol=1e-12,
    ),
    "q3_source_p_actionable_wrong",
    errors,
)
check(
    np.isclose(
        ta_summary["p_actionable"],
        ta_summary["p_finite"] + ta_summary["p_active_at_issue_left_censored"],
        atol=1e-12,
    ),
    "q3_arrival_p_actionable_wrong",
    errors,
)
finite_members = members.loc[members["state_tm"].eq("finite")]
finite_runout = members.loc[members["state_ta"].eq("finite")]
source_lower = pd.to_datetime(finite_members["tm_lower_utc"], utc=True)
source_upper = pd.to_datetime(finite_members["tm_upper_utc"], utc=True)
arrival_lower = pd.to_datetime(finite_runout["ta_lower_utc"], utc=True)
arrival_upper = pd.to_datetime(finite_runout["ta_upper_utc"], utc=True)
check(
    bool(((source_upper - source_lower) == pd.Timedelta(hours=1)).all()),
    "q3_source_support_not_one_hour",
    errors,
)
check(
    bool(((arrival_upper - arrival_lower) == pd.Timedelta(hours=1)).all()),
    "q3_arrival_support_not_one_hour",
    errors,
)
true_row = coverage.loc[coverage["target_definition"].eq("true_debris_flow_formation")].iloc[0]
port_row = coverage.loc[coverage["target_definition"].eq("instant_release_scenario_arrival")].iloc[0]
actual_row = coverage.loc[coverage["target_definition"].eq("actual_port_arrival")].iloc[0]
check(bool(true_row["evaluation_eligible"]) is False, "q3_true_trigger_marked_evaluable", errors)
for row in (true_row, actual_row):
    check(row["interval_status"] == "not_estimable_release_model_unidentified",
          "q3_unidentified_release_status_wrong", errors)
    check(not bool(row["evaluation_eligible"]), "q3_actual_target_marked_evaluable", errors)
    fields = ["lower_time", "upper_time", "definite_cover", "possible_cover", "width_hours"]
    fields += [key for key in row.index if key.startswith("p_")]
    check(row[fields].isna().all(), "q3_actual_target_contains_auxiliary_prediction", errors)
check(members["state_release"].eq("not_identified").all(),
      "q3_release_falsely_identified", errors)
check(members[["release_time_utc", "actual_arrival_time_utc"]].isna().all().all(),
      "q3_actual_times_filled_from_auxiliary_scenario", errors)
if port_row["interval_status"] == "available":
    check(isinstance(bool(port_row["definite_cover"]), bool), "q3_port_cover_not_boolean", errors)
else:
    check(pd.isna(port_row["definite_cover"]), "q3_nonconstructible_port_cover_not_null", errors)
check(warning.iloc[0]["port_hit_time"] == "2026-08-26T10:30:00+08:00", "q3_port_hit_time_wrong", errors)
check(warning.iloc[0]["status"] == "not_estimable_release_model_unidentified",
      "q3_actual_warning_status_wrong", errors)
check(warning[["first_alarm_time", "actionable_alarm_time", "lead_trigger_lower",
               "lead_trigger_upper", "lead_port", "missed_alarm", "late_alarm"]].isna().all().all(),
      "q3_actual_warning_contains_auxiliary_alarm", errors)
check(advisory["product_role"].eq("melt_advisory_only_not_a_disaster_warning").all(),
      "q3_advisory_role_wrong", errors)
alarm = bool(float(advisory.iloc[0]["source_p_actionable"]) >= 0.5)
if alarm:
    check(float(advisory.iloc[0]["lead_port"]) == 2.5, "q3_advisory_information_lead_not_2_5h", errors)
else:
    check(pd.isna(advisory.iloc[0]["lead_port"]), "q3_missed_advisory_has_lead", errors)
active_left = members.loc[members["state_ta"].eq("active_at_issue_left_censored")]
check(
    active_left.empty or active_left["ta_censor_upper_utc"].notna().all(),
    "q3_active_left_without_upper_bound",
    errors,
)
check(
    set(members["state_tm"]).issubset(expected_tm),
    "q3_member_source_state_unknown",
    errors,
)
check(
    set(members["state_ta"]).issubset(expected_ta),
    "q3_member_arrival_state_unknown",
    errors,
```

```
    )
    checks["q3"] = {
        "issue_time_local": q3["issue_time_local"],
        "legacy_audit_output": q3["legacy_inverse_audit"]["output"],
        "legacy_audit_sha256": q3["legacy_inverse_audit"]["output_sha256"],
        "source_state_counts": q3["state_summary_tm"]["counts"],
        "arrival_state_counts": q3["state_summary_ta"]["counts"],
        "source_interval_utc": q3["intervals"]["t_m"],
        "instant_release_scenario_interval_utc": q3["intervals"]["t_a"],
        "instant_release_scenario_single_event_inclusion": (
            bool(port_row["definite_cover"])
            if port_row["interval_status"] == "available" else None
        ),
        "runout_terminal_flag_counts": {
            column: int(runout[column].astype(bool).sum())
            for column in ("natural_stop", "time_limit_nonarrival", "right_censored")
        },
        "true_trigger_coverage": "not_estimable",
        "actual_port_coverage": None,
        "actual_disaster_warning_lead_h": None,
        "melt_advisory_information_lead_h": (
            float(advisory.iloc[0]["lead_port"]) if alarm else None
        ),
    }

    q4 = load("04b_二维引擎.json")
    runs = q4.get("runs", {})
    check(len(runs) == 16, "q4_d8_config_count_not_16", errors)
    required_terminal = {
        "termination_reason",
        "natural_stop",
        "time_limit",
        "step_limit",
        "domain_censor",
        "runout_right_censored",
        "reached_port",
    }
    check(
        all(required_terminal <= set(run) for run in runs.values()),
        "q4_terminal_flags_missing",
        errors,
    )
    check(
        all(
            "critical_depth_slope_product_m" in run
            and "critical_depth_port_m" in run
            and "h_stop_m" not in run
            for run in runs.values()
        ),
        "q4_critical_depth_fields_mislabeled",
        errors,
    )
    check(
        all(bool(run.get("domain_censor")) for run in runs.values()),
        "q4_domain_censor_not_recorded",
        errors,
    )
    check(
        all(
            not (bool(run.get("domain_censor")) and bool(run.get("natural_stop")))
            for run in runs.values()
        ),
        "q4_domain_censor_conflicts_with_natural_stop",
        errors,
    )
    check(
        all(bool(run.get("runout_right_censored")) for run in runs.values()),
        "q4_runout_not_marked_censored",
        errors,
    )
    baseline = runs["base"]
    check(abs(float(baseline["arrival_port_s"]) - 1965.996174) < 1e-5, "q4_baseline_arrival_changed", errors)
    checks["q4"] = {
        "d8_config_count": len(runs),
        "baseline_arrival_port_s": baseline["arrival_port_s"],
        "runout_L_m": baseline["runout_L_m"],
        "runout_right_censored": baseline["runout_right_censored"],
        "domain_front_max_m": baseline.get("domain_front_max_m"),
        "terminal_flags": {
            key: baseline.get(key)
            for key in ("termination_reason", "natural_stop", "time_limit",
                        "step_limit", "domain_censor")
        },
    }

    external_manifest = EXTERNAL / "ifs_single_run_manifest.json"
    check(external_manifest.is_file(), "ifs_manifest_missing", errors)
    report = {
        "schema_version": 1,
        "status": "pass" if not errors else "blocked",
        "errors": errors,
        "checks": checks,
        "scope": "deterministic result-chain integrity only; not physical validation or task completion",
        "task_completion": {
            "status": "blocked", "release_eligible": False,
            "unmet_requirements": [
                "Q3 identifiable real release/formation-time forecast and its 95% interval",
```

```
            "Q3 real disaster warning and independently evaluable coverage",
        ],
        "reason": "melt onset and instant-release transport are auxiliary scenarios, not the requested target",
    },
}
OUT.write_text(json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
print(json.dumps({"status": report["status"], "errors": errors, "output": OUT.as_posix()},
                 ensure_ascii=False))
return 0 if not errors else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

`export_summary.py` 汇总结果导出

```
"""Export evidence-bound paper values with ASCII-only JSON keys.

Only the two 05 JSON artifacts are written. Original result files are read-only.
"""
from __future__ import annotations

import csv
import hashlib
import json
import math
import re
from pathlib import Path

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / '求解结果' / '数据'
VALUES_PATH = DATA / '05_paper_values.json'
PROVENANCE_PATH = DATA / '05_value_provenance.json'


def sha(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def clean_number(value):
    if isinstance(value, float) and not math.isfinite(value):
        return None
    return value


def main():
    sources, inventory, provenance = {}, [], {}
    for path in sorted(DATA.rglob('*')):
        if 'cache' in path.parts or not path.is_file():
            continue
        if path.parent == DATA and not path.name.startswith(('00', '01', '02', '03', '04', '06')):
            continue
        if path.suffix.lower() not in ('.json', '.csv'):
            continue
        rel = path.relative_to(ROOT).as_posix()
        record = {'source_file': rel, 'sha256': sha(path), 'bytes': path.stat().st_size}
        if path.suffix.lower() == '.json':
            sources[path.relative_to(DATA).as_posix()] = json.loads(path.read_text(encoding='utf-8-sig'))
            record['format'] = 'json'
        else:
            with path.open(encoding='utf-8-sig', newline='') as handle:
                reader = csv.DictReader(handle)
                headers = reader.fieldnames or []
                n = sum(1 for _ in reader)
            record.update(format='csv', rows=n, columns=headers)
        inventory.append(record)
    hashes = {r['source_file']: r['sha256'] for r in inventory}

    def get(source, keys):
        node = sources[source]
        for key in keys:
            node = node[key]
        return node

    def pointer(keys):
        return '/' + '/'.join(str(k).replace('~', '~0').replace('/', '~1') for k in keys) if keys else ''

    def original_locator(keys):
        result = '$'
        for key in keys:
            if isinstance(key, int):
                result += f'[{key}]'
            elif re.fullmatch(r'[A-Za-z_][A-Za-z0-9_]*', str(key)):
                result += '.' + str(key)
            else:
                result += '[' + json.dumps(str(key), ensure_ascii=False) + ']'
        return result

    def ascii_key(key, index):
        text = str(key)
        if re.fullmatch(r'[A-Za-z_][A-Za-z0-9_]*', text):
            return text
        if re.fullmatch(r'[0-9]+', text):
            return 'n_' + text
        return f'k{index:03d}'

```

```
    def copy(node, source, original, output):
        rel = (Path('求解结果') / '数据' / source).as_posix()
        if isinstance(node, dict):
            result = {}
            for i, (key, val) in enumerate(node.items()):
                if source.startswith('04c_') and key in ('trajectory','segments','segment_ledger','records','selected_full_ledgers',
                                                        'native_path_rc','sh_m','native_profiles','path_profiles',
                                                        'route_s_h_m','profile_z_m'):
                    continue
                if source == '04b_二维引擎.json' and key in ('ledger','leader','port_hydrograph'):
                    continue
                if source == '04_敏感性.json' and key in ('s_km','z_m','h_m','v_m_s'):
                    continue
                target = ascii_key(key, i)
                if target in result:
                    target += f'_{i}'
                result[target] = copy(val, source, original+[key], output+'.'+target)
            return result
        if isinstance(node, list):
            return [copy(v, source, original+[i], output+f'[{i}]') for i, v in enumerate(node)]
        value = clean_number(node)
        provenance[output] = {'source_file': rel, 'source_locator': original_locator(original),
                              'source_json_pointer': pointer(original), 'source_sha256': hashes[rel],
                              'operation': 'direct copy', 'value': value}
        return value

    values = {'schema_version': 1, 'scope': 'Current computed results; interval targets and limitations remain distinct.',
              'q0': {}, 'q1': {}, 'q2': {}, 'q3': {}, 'q4': {}, 'external': {}, 'csv_inventory': {}}

    def add(section, name, source, *keys):
        values[section][name] = copy(get(source, list(keys)), source, list(keys), f'$.{section}.{name}')

    add('q0','station','00_站点元信息.json')
    for key in ('temperature','hazards','dem','landcover','census','event'):
        add('q0',key,'00_数据清单.json',key)
    add('q0','glacier_change','00_冰川dhdt统计.json')
    ranking='01_特征关联排序.json'
    for name,key in [('top5','前5特征'),('prior_removal','去先验对照'),('ranking_agreement','两方案一致性'),
                     ('robustness','排序稳健性'),('seasonal_shift_control','控制组_日历平移'),
                     ('warm_season_rarity','暖季稀有度'),('leave_year_auc','留一年AUC'),
                     ('classifier_auc','测试AUC'),('candidate_count','候选特征数')]:
        add('q1',name,ranking,key)
    for name,key in [('critic_top5','前5特征'),('critic_weights','权重')]:
        add('q1',name,'01_CRITIC交叉核对.json',key)
    add('q1','boundary_layer','01_边界层结构.json')
    add('q1','hydrostatic_correction','01_静力层压订正.json')
    add('q1','preimpact_anomaly','01b_冲击前窗口异常.json')
    add('q1','unified_association','01c_统一关联前五.json')
    add('q1','weather_modes','01_天气过程模态.json')
    # Raw bootstrap samples are unnecessary for the reader-facing summary.
    energy=sources['01_能量闭合敏感性.json']
    for key in energy:
        if key != '自举C样本':
            add('q1','energy_'+ascii_key(key,list(energy).index(key)),'01_能量闭合敏感性.json',key)
    add('q1','extremes','01_极端事件.json')
    rainfall='02_降雨阈值.json'
    for name,key in [('rain_event_count','独立暴雨事件数'),('rain_upper_a','幂律系数a'),
                     ('rain_upper_b','幂指数b'),('rain_upper_a_ci','参数95pct置信区间_a'),
                     ('rain_upper_b_ci','参数95pct置信区间_b'),('rain_lower','下包络'),
                     ('rain_duration_support_h','频率包络最大支持历时_h')]:
        add('q2',name,rainfall,key)
    for name,source in [('business_screen','02_水源业务判据.json'),('meteorology','02_气象基线检验.json'),
                        ('melt_threshold','02_消融阈值.json'),('historical_domain','02_统计临界域.json'),
                        ('reliability','02_多尺度可靠性.json'),('gully_thresholds','02_沟道临界条件.json'),
                        ('trigger_score','02_触发评估模型.json'),('spatial_contrast','02b_空间对照.json'),
                        ('long_term_temperature','02_长期气温对比.json'),('catchment','02_集水区特征.json'),
                        ('legacy_method_audit','02_旧方法审计.json')]:
        add('q2',name,source)
    add('q2','matched_spatial','02c_共同域空间验证.json')
    for name,source in [('forecast','03_前向预报.json'),
                        ('forecast_member_summary','03_前向预报成员摘要.json'),
                        ('distance','03_空间距离范围.json')]:
        add('q3',name,source)
    for name,source in [('initial_volume','04_初始体积.json'),('centerline_checks','04_中心线校验.json'),
                        ('friction','04_摩擦系数反演.json'),('scenario_summary','04_结果区间.json'),
                        ('intensity','04_强度场.json'),('fan_envelope','04_扇体扩散.json')]:
        add('q4',name,source)
    baseline=sources['04_仿真基线.json']
    for key in ('params','arrival_outlet_s','stop_time_s','termination_reason','runout_km',
                'total_km','reached_outlet','continuity_relative_residual_max','runout_right_censored',
                'v_median_m_s','v_max_m_s','h_median_m','I_max','Cv','rho_m','drop_m','stopped_drop_m'):
        if key in baseline: add('q4','one_d_'+key,'04_仿真基线.json',key)
    add('q4','one_d_unique_configs','04_敏感性.json','unique_runs')
    for name,source in [('finite_inventory_protocol','04c_参数与路径.json'),
                        ('finite_inventory_no_nodes','04c_无节点动力.json'),
                        ('finite_inventory_synthetic','04c_合成库存机制.json'),
```

```
                ('finite_inventory_inverse','04c_逆向瓶颈.json'),
                ('finite_inventory_checks','04c_数值核验.json'),
                ('finite_inventory_tests','04c_合成测试.json')]:
      add('q4',name,source)
   routing='04b_二维引擎.json'
   for name,key in [('routing_sections','断面'),('routing_source_mapping','源区落域统计'),
                ('routing_scenarios','runs'),('routing_coarse','scale90_rerun'),
                ('routing_time_refinement','time_refinement'),('routing_acceptance','账本验收'),
                ('routing_unique_count','敏感性情境总数_独立'),('routing_nonarrival','not_reached')]:
      add('q4',name,routing,key)
   for i,item in enumerate(r for r in inventory if r['format']=='csv'):
      name='table_'+str(i)
      values['csv_inventory'][name]={'rows':item['rows'],'columns':item['columns'],'source_file':item['source_file']}
      provenance[f'$.csv_inventory.{name}.rows']={'source_file':item['source_file'],
         'source_sha256':item['sha256'],'operation':'derived','calculation':'Count data records with csv.DictReader, excluding header',
         'value':item['rows']}
   for index, source in enumerate(sorted(sources)):
      if source.startswith(('外部/', '06_')):
         add('external', 'audit_'+str(index), source)
   VALUES_PATH.write_text(json.dumps(values,ensure_ascii=False,indent=2,allow_nan=False)+'\n',encoding='utf-8')
   package={'schema_version':1,'summary_file':'求解结果/数据/05_paper_values.json',
         'summary_sha256':sha(VALUES_PATH),'sources':inventory,'entries':provenance,
         'key_policy':'Semantic top-level ASCII aliases; non-ASCII nested source keys become kNNN in original insertion order. Every copied leaf has original locator and JSON Pointer.',
         'refresh':'Rerun after any upstream result changes; no source result is modified.'}
   PROVENANCE_PATH.write_text(json.dumps(package,ensure_ascii=False,indent=2,allow_nan=False)+'\n',encoding='utf-8')
   print(f'Exported {len(provenance)} evidence-bound leaves from {len(inventory)} source files.')
   print(f'Summary SHA256: {package["summary_sha256"]}')
   return 0


if __name__=='__main__':
   raise SystemExit(main())
```

测试

test01b_hydrostatic.py 静力层压测试

```
"""Synthetic checks for the frozen five-layer hydrostatic protocol."""
from __future__ import annotations

import unittest

import numpy as np
import pandas as pd

from mmlib import meteo as mt


def observations(n=4):
   frame = pd.DataFrame(index=pd.date_range("2024-08-01", periods=n, freq="h"))
   for z in mt.LEVELS_T:
      frame[f"Ta_{z}m"] = 5.
      frame[f"RH_{z}m"] = 30.
      frame[f"WS_{z}m"] = 1. + .1 * z
      frame[f"WD_{z}m"] = 0.
   frame["Pressure"] = 600.
   return frame


class HydrostaticTests(unittest.TestCase):
   def test_pressure_anchor_and_monotonicity(self):
      df = observations()
      for ref in (1, 2, 10, 20):
         profile = mt.hydrostatic_profiles(df, ref)
         np.testing.assert_allclose(profile[f"p_{ref}m"], df.Pressure)
         self.assertTrue((np.diff(profile[[f"p_{z}m" for z in mt.LEVELS_T]], axis=1) < 0).all())
         self.assertLess(profile.attrs["iteration_3_minus_2_max_hPa"], 1e-7)
         self.assertEqual(profile.attrs["iterations"], 3)

   def test_isothermal_analytic_dry_column(self):
      df = observations(); df[[f"RH_{z}m" for z in mt.LEVELS_T]] = 0.
      profile = mt.hydrostatic_profiles(df)
      for z in mt.LEVELS_T:
         expected = 600 * np.exp(-mt.G * (z - 2) / (mt.R_D * 278.15))
         np.testing.assert_allclose(profile[f"p_{z}m"], expected, rtol=1e-12)

   def test_neutral_dry_adiabatic_column(self):
      df = observations()
      for z in mt.LEVELS_T:
         df[f"RH_{z}m"] = 0.
         df[f"Ta_{z}m"] = 5 - mt.G / mt.CP * (z - 2)
      profile = mt.hydrostatic_profiles(df)
      self.assertLess(float((profile.theta_20m - profile.theta_2m).abs().max()), 2e-5)
      self.assertLess(float(mt.bulk_richardson(df, profile).abs().max()), 1e-4)

   def test_missing_any_layer_masks_thermodynamics(self):
```

```
      for family in ("Ta", "RH"):
         for z in mt.LEVELS_T:
            df = observations(); df.loc[df.index[1], f"{family}_{z}m"] = np.nan
            corrected = mt.hydrostatic_profiles(df)
            old = mt.hydrostatic_profiles(df, corrected=False)
            self.assertFalse(corrected.thermo_valid.iloc[1])
            self.assertTrue(corrected.filter(regex=r"^(p|q|theta|thetav)_").iloc[1].isna().all())
            self.assertTrue(mt.bulk_richardson(df, corrected).isna().iloc[1])
            np.testing.assert_array_equal(corrected.thermo_valid, old.thermo_valid)

   def test_vector_shear_uses_direction(self):
      df = observations(); df[[f"WS_{z}m" for z in mt.LEVELS_T]] = 2.
      df["WD_20m"] = 180.
      np.testing.assert_allclose(mt.vector_shear_squared(df), 16.)
      np.testing.assert_allclose(mt.wind_shear(df).vector_shear_2_20m, 4 / 18)
      df["WD_20m"] = 0.
      self.assertTrue(mt.bulk_richardson(df).isna().all())

   def test_crossing_at_20_is_not_right_censoring(self):
      df = observations()
      profile = mt.hydrostatic_profiles(df)
      for z in mt.LEVELS_T:
         profile[f"thetav_{z}m"] = 300.
      profile.loc[df.index[0], "thetav_20m"] = 302.
      result = mt.pbl_diagnostics(df, profile)
      self.assertEqual(result.pbl_crossing_height_m.iloc[0], 20.)
      self.assertEqual(result.pbl_right_censored.iloc[0], 0.)
      self.assertTrue(pd.isna(result.pbl_crossing_height_m.iloc[1]))
      self.assertEqual(result.pbl_right_censored.iloc[1], 1.)
      self.assertEqual(result.pblh.iloc[0], result.pblh.iloc[1])

   def test_missing_lower_wind_pair_cannot_be_skipped(self):
      df = observations(); df.loc[df.index[0], "WD_4m"] = np.nan
      profile = mt.hydrostatic_profiles(df)
      profile["thetav_20m"] += 10.
      result = mt.pbl_diagnostics(df, profile)
      self.assertTrue(result.iloc[0].isna().all())

   def test_pressure_is_shared_by_all_derived_quantities(self):
      df = observations()
      for name in ("Precip", "Rsd", "Rsu", "Rld", "Rlu", "H", "LE", "G_0.1m"):
         df[name] = 0.
      feat = mt.build_feature_table(df)
      np.testing.assert_allclose(feat.q, feat.q_2m)
      np.testing.assert_allclose(feat.dtheta_1_20, feat.theta_20m - feat.theta_1m)
      np.testing.assert_allclose(feat.Ri_b, mt.bulk_richardson(df))
      self.assertEqual(feat.attrs["feature_version"], "v2-hydrostatic-five-layer-3iter")


if __name__ == "__main__":
   unittest.main(verbosity=2)
```

test01b_preimpact.py　冲击前窗口测试

```
"""Small deterministic checks for the frozen label-free window protocol."""
import unittest

import numpy as np
import pandas as pd

import step01b_preimpact as p


class PreimpactTests(unittest.TestCase):
   def test_nonleap_calendar(self):
      self.assertEqual(p.calendar_day(["2024-08-26"])[0], p.calendar_day(["2025-08-26"])[0])
      self.assertTrue(p.nearby(pd.DatetimeIndex(["2024-08-11", "2023-09-10"]), p.TARGET).all())
      self.assertFalse(p.nearby(pd.DatetimeIndex(["2024-08-10"]), p.TARGET)[0])
      with self.assertRaises(ValueError):
         p.calendar_day(["2024-02-29"])

   def test_clock_and_shared_qualification(self):
      release = pd.Timestamp("2026-08-26")
      grid = pd.date_range("2026-08-18", "2026-08-26", freq="h")
      f = pd.DataFrame(1., index=grid, columns=p.COLS)
      table = p.window_table(f, 168, [release])
      self.assertEqual(table.loc[release, "end"], pd.Timestamp("2026-08-25 22:00"))
      self.assertEqual(table.loc[release, "start"], pd.Timestamp("2026-08-18 23:00"))
      self.assertEqual(table.loc[release, "n__WS_10m"], 168)
      start = table.loc[release, "start"]
      f.loc[pd.date_range(start, periods=33, freq="h"), "TKE_obs"] = np.nan
      self.assertTrue(p.window_table(f, 168, [release]).loc[release, "common_valid"])
      f.loc[start + pd.Timedelta(hours=33), "TKE_obs"] = np.nan
      self.assertFalse(p.window_table(f, 168, [release]).loc[release, "common_valid"])

   def test_scale_fallback_and_no_epsilon(self):
      f = pd.DataFrame({c: [0., 0., 0., 1.] for c in p.COLS})
      f[p.COLS[-1]] = 0.
      center, scale, method = p.robust_baseline(f)
      self.assertEqual(center.iloc[0], 0.)
      self.assertAlmostEqual(scale.iloc[0], .25 / 1.349)
      self.assertEqual(method.iloc[0], "IQR/1.349回退")
      self.assertTrue(np.isnan(scale.iloc[-1]))

   def test_complete_connected_components(self):
      a, b, c = p.COLS[:3]
```

```
        groups = p.components_from_edges([(a, b, .9), (b, c, -.9)])
        self.assertEqual(groups[0], [a, b, c])
        self.assertEqual(sum(map(len, groups)), 12)
        short = pd.DataFrame(np.arange(12 * 199).reshape(199, 12),
                             index=pd.date_range("2021-06-01", periods=199, freq="h"), columns=p.COLS)
        result = p.connected_clusters(short)
        self.assertFalse(result["confirmed"])
        self.assertEqual(result["clusters"], [])

    def test_seven_day_schedule_precedes_missing_screen(self):
        dates = pd.date_range("2021-08-11", "2021-09-10", freq="D")
        table = pd.DataFrame({"common_valid": True}, index=dates)
        table.loc[dates[0], "common_valid"] = False
        selected = p.seven_day_reference(table)
        self.assertEqual(selected.index[0], dates[0])
        self.assertEqual(selected.index[1], dates[7])
        self.assertFalse(selected.common_valid.iloc[0])

    def test_score_uses_window_distribution_and_frozen_years(self):
        dates = pd.DatetimeIndex(["2021-08-26", "2022-08-26", "2023-08-26", "2024-08-26", "2025-08-26", "2026-08-26"
            ])
        table = pd.DataFrame({c: [1., 2., 3., 4., 999., 5.] for c in p.COLS}, index=dates)
        table["common_valid"] = True
        table["start"] = dates - pd.Timedelta(hours=169)
        table["end"] = dates - pd.Timedelta(hours=2)
        table["length_h"] = 168
        table["all12_complete_hours"] = 168
        for c in p.COLS:
            table[f"n__{c}"] = 168
        result = p.score_window(table, p.TARGET, [[c] for c in p.COLS])
        row = result["rows"][0]
        self.assertEqual(result["reference_windows"], 4)
        self.assertEqual(row["baseline_median"], 2.5)
        self.assertEqual(row["tail_rank"], .2)
        self.assertEqual(result["top5"], p.COLS[:5])
        self.assertNotIn(2025, result["reference_counts_by_year"])

    def test_missing_training_year_is_not_a_noop_sensitivity(self):
        dates = pd.DatetimeIndex(["2021-08-26", "2022-08-26", "2023-08-26",
                                  "2024-08-26", p.TARGET])
        table = pd.DataFrame({c: [1., 2., 3., 4., 2.5] for c in p.COLS}, index=dates)
        table["common_valid"] = [True, False, True, True, True]
        table["start"] = dates - pd.Timedelta(hours=169)
        table["end"] = dates - pd.Timedelta(hours=2)
        table["length_h"] = 168
        table["all12_complete_hours"] = 168
        for c in p.COLS:
            table[f"n__{c}"] = 168
        clusters = [[c] for c in p.COLS]
        target = p.score_window(table, p.TARGET, clusters)
        result, _ = p.background_sensitivity(table, target, clusters)
        bootstrap = result["year_block_bootstrap"]
        self.assertEqual(bootstrap["available_training_years"], [2021, 2023, 2024])
        self.assertEqual(set(bootstrap["excluded_training_years"]), {"2022"})
        self.assertTrue(all(2022 not in draw for draw in bootstrap["sampled_training_years"]))
        loyo = {row["omitted_year"]: row for row in result["leave_one_training_year_out"]}
        self.assertEqual(loyo[2022]["status"], "不可评估：该训练年无共同有效参照窗")
        self.assertEqual(loyo[2022]["reference_windows"], 0)
        self.assertIsNone(loyo[2022]["full_rank_Kendall_tau"])


if __name__ == "__main__":
    unittest.main()
```

test02c03_protocols.py　空间与预报协议测试

```
"""Synthetic and protocol checks for the v5.0 trigger and forecast chain."""
from __future__ import annotations

import unittest

import numpy as np
import pandas as pd

import step01c_unified_association as association
import step02_trigger_threshold as threshold
import step02c_spatial_matched as spatial
import step03_figures as figures
import step03_forecast as forecast
from mmlib import trigger as tg


class SpatialProtocolTests(unittest.TestCase):
    def test_polygon_block_union_and_shared_background(self):
        labels = np.zeros((8, 15), dtype=int)
        labels[1, 1:10] = 1
        labels[2, 8:14] = 2
        grid, _ = spatial.shared_groups(labels, {"cell_dy_m": 1000., "cell_dx_m": 1000.})
        self.assertEqual(np.unique(grid[labels > 0]).size, 1)
        self.assertEqual(grid[0, 0], grid[1, 1])
        self.assertNotEqual(grid[-1, -1], grid[1, 1])

    def test_purge_includes_exact_boundary(self):
        xy = np.array([[0., 0.], [300., 400.], [501., 0.], [900., 900.]])
        kept, record = spatial.purge(np.array([1, 2, 3]), np.array([0]), xy)
        self.assertEqual(kept.tolist(), [2, 3])
```

```
        self.assertEqual(record["purged_n"], 1)

    def test_fully_overwritten_polygon_grouping(self):
        polygon = {
            "bbox": (0., 0., 4., 4.),
            "pts": np.array([[0, 0], [4, 0], [4, 4], [0, 4], [0, 0]], dtype=float),
            "parts": [0],
        }
        grid = {"lon0": 0., "lat0": 4., "dlon": 1., "dlat": -1.}
        labels, _ = spatial.legacy.rasterize([polygon, polygon], grid, (5, 5))
        self.assertNotIn(1, np.unique(labels))
        groups, meta = spatial.shared_groups(
            labels, {"cell_dy_m": 1000., "cell_dx_m": 1000.},
            [polygon, polygon], grid,
        )
        self.assertEqual(len(meta["blocks_by_polygon"]), 2)
        self.assertEqual(np.unique(groups[labels > 0]).size, 1)

    def test_shared_group_bootstrap_not_class_split(self):
        y = np.tile([0, 1], 10)
        g = np.repeat(np.arange(10), 2)
        score = y * .8 + .1
        r = spatial.group_bootstrap(y, score, g, 11)
        self.assertEqual(r["valid"], spatial.BOOT)
        np.testing.assert_allclose(r["interval"], [1., 1.])

    def test_insufficient_does_not_random_pixel_fallback(self):
        y = np.array([0, 0, 1, 1])
        g = np.array([0, 0, 1, 1])
        xy = np.array([[0, 0], [1, 0], [2000, 0], [2001, 0]])
        with self.assertRaises(spatial.InsufficientDesign):
            spatial.grouped_folds(y, g, xy, 4, 1)


class AssociationProtocolTests(unittest.TestCase):
    def test_frozen_candidate_dictionary_has_31_unique_keys(self):
        self.assertEqual(len(association.CANDIDATES), 31)
        self.assertEqual(len(association.COLUMNS), 31)
        self.assertEqual(len(set(association.COLUMNS)), 31)
        counts = pd.Series([row[2] for row in association.CANDIDATES]).value_counts().to_dict()
        self.assertEqual(counts, {"MET": 19, "RADM": 5, "SOIL": 3, "FLUX": 4})

    def test_aliases_are_not_independent_candidates(self):
        self.assertNotIn("Pmax_1h", association.COLUMNS)
        self.assertIn("P", association.COLUMNS)
        self.assertIn("H", association.COLUMNS)
        self.assertIn("LE", association.COLUMNS)
        self.assertIn("Rsd", association.COLUMNS)

    def test_candidate_ranking_retains_unrankable_candidates(self):
        metadata = {
            column: {"variable": column, "name": column, "family": "TEST"}
            for column in association.COLUMNS[:3]
        }
        result = {
            "window_length_h": 168,
            "target_window": ["start", "end"],
            "feature_details": {
                association.COLUMNS[0]: {
                    "rankable": True, "A": 1.0, "signed_z": 1.0,
                    "target_valid_hours": 168, "required_hours": 135,
                    "reference_windows": 90,
                },
                association.COLUMNS[1]: {
                    "rankable": False, "reason": "insufficient coverage",
                    "target_valid_hours": 10, "required_hours": 135,
                    "reference_windows": 90,
                },
                association.COLUMNS[2]: {
                    "rankable": True, "A": 2.0, "signed_z": -2.0,
                    "target_valid_hours": 168, "required_hours": 135,
                    "reference_windows": 90,
                },
            },
        }
        rows = association.candidate_ranking_rows(result, metadata)
        self.assertEqual(len(rows), 3)
        self.assertEqual([row["rank"] for row in rows], [1, 2, None])
        self.assertEqual(rows[-1]["exclusion_reason"], "insufficient coverage")

    def test_full_cluster_ranking_retains_unrankable_cluster(self):
        metadata = {
            column: {"variable": column, "name": column, "family": "TEST"}
            for column in association.COLUMNS[:2]
        }
        result = {
            "window_length_h": 168,
            "target_window": ["start", "end"],
            "feature_details": {
                association.COLUMNS[0]: {"signed_z": 1.0, "A": 1.0},
                association.COLUMNS[1]: {"rankable": False},
            },
            "ranked_clusters": [{
                "rank": 1,
                "representative": association.COLUMNS[0],
                "members": [association.COLUMNS[0]],
            }],
```

```
        "cluster_ranking": [
            {
                "representative": association.COLUMNS[0],
                "members": [association.COLUMNS[0]],
            },
            {
                "representative": None,
                "members": [association.COLUMNS[1]],
                "status": "不可排名",
            },
        ],
    }
    rows = association.cluster_ranking_rows(result, metadata)
    self.assertEqual(len(rows), 2)
    self.assertEqual(rows[0]["rank"], 1)
    self.assertIsNone(rows[1]["rank"])
    self.assertEqual(rows[1]["cluster_status"], "not_rankable")


class TriggerMechanicsTests(unittest.TestCase):
    def test_formal_event_window_is_frozen_and_positive_threshold_retains_precision(self):
        self.assertEqual(
            threshold.FORMAL_EVENT_WIN,
            ("2026-08-26 08:00", "2026-08-26 13:00"),
        )
        self.assertEqual(
            threshold._ser({"Q_crit_m3s": 7.180069e-05})["Q_crit_m3s"],
            7.180069e-05,
        )

    def test_water_controlled_positive_threshold(self):
        result = tg._mechanics(0.30)
        self.assertEqual(result["regime"], "water_controlled")
        self.assertGreater(result["tau_w_c_Pa"], 0)
        self.assertGreater(result["q_crit_m2s"], 0)
        self.assertGreater(result["tau_r_Pa"], result["tau_g_parallel_Pa"])

    def test_friction_equilibrium_has_zero_water_threshold(self):
        result = tg._mechanics(tg.TAN_PHI)
        self.assertEqual(result["regime"], "friction_equilibrium")
        self.assertEqual(result["q_crit_m2s"], 0.0)
        self.assertEqual(result["h_crit_m"], 0.0)

    def test_gravity_unstable_has_zero_water_threshold(self):
        result = tg._mechanics(0.72)
        self.assertEqual(result["regime"], "gravity_unstable")
        self.assertEqual(result["q_crit_m2s"], 0.0)
        self.assertGreater(result["tau_g_parallel_Pa"], result["tau_r_Pa"])

    def test_invalid_geometry_is_not_evaluable(self):
        for slope in (0.0, -0.1, np.nan):
            self.assertEqual(tg._mechanics(slope)["regime"], "not_evaluable")

    def test_bed_density_sensitivity_is_monotone(self):
        low = tg._mechanics(0.30, bed_solid_fraction=0.15)["q_crit_m2s"]
        high = tg._mechanics(0.30, bed_solid_fraction=0.80)["q_crit_m2s"]
        self.assertGreater(high, low)

    def test_density_difference_matches_declared_density_law(self):
        self.assertAlmostEqual(tg._density_difference(1050.0 / 1650.0), 1050.0, places=12)
        self.assertAlmostEqual(tg._density_difference(0.15), 247.5, places=12)
        result = tg._mechanics(0.30, bed_solid_fraction=0.15)
        self.assertAlmostEqual(result["delta_rho_kg_m3"], 247.5, places=12)
        self.assertGreater(result["q_crit_m2s"], 0.0)

    def test_cn0_is_frozen_and_runoff_scenarios_are_ordered(self):
        self.assertEqual(tg.CN0, 85.0)
        index = pd.date_range("2024-07-01", periods=10, freq="h")
        feat = pd.DataFrame({
            "P": [0.0] * 6 + [12.0, 12.0, 0.0, 0.0],
            "API": [10.0] * 6 + [5.0, 10.0, 20.0, 25.0],
        }, index=index)
        feat.attrs["api_quantiles"] = (8.0, 18.0)
        low = tg.rain_runoff(feat, 75.0)
        base = tg.rain_runoff(feat, 85.0)
        high = tg.rain_runoff(feat, 95.0)
        self.assertTrue((low <= base).all())
        self.assertTrue((base <= high).all())
        self.assertGreater(float(high.sum()), float(low.sum()))

    def test_api_terciles_use_only_2021_2025_warm_season(self):
        warm = pd.date_range("2024-06-01", periods=120, freq="h")
        winter = pd.date_range("2024-01-01", periods=120, freq="h")
        feat = pd.DataFrame(
            {"P": np.zeros(240), "API": np.r_[np.linspace(0.0, 119.0, 120),
                                              np.full(120, 1e6)]},
            index=warm.append(winter),
        )
        meta = tg.api_climate_thresholds(feat)
        expected_q1, expected_q3 = np.quantile(np.arange(120.0), [1 / 3, 2 / 3])
        self.assertAlmostEqual(meta["q1"], expected_q1)
        self.assertAlmostEqual(meta["q3"], expected_q3)
        self.assertEqual(meta["training_valid_hours"], 120)
        self.assertEqual(meta["training_months"], [6, 7, 8, 9])

    def test_zero_or_invalid_solid_fraction_is_not_evaluable(self):
        for fraction in (0.0, -0.1, 1.01, np.nan):
```

```
        with self.subTest(fraction=fraction):
            result = tg._mechanics(0.30, bed_solid_fraction=fraction)
            self.assertEqual(result["regime"], "not_evaluable")
            self.assertIsNone(result["q_crit_m2s"])

    def test_trigger_constructor_uses_zero_branch_without_division(self):
        feat = pd.DataFrame(
            {"P": [0.0], "API": [10.0], "Ta_2m": [2.0], "melt_energy": [10.0]},
            index=pd.date_range("2024-01-01", periods=1, freq="h"),
        )
        feat.attrs["api_quantiles"] = (8.0, 18.0)
        model = tg.TriggerModel(
            area_km2=1.0,
            glacier_frac=0.2,
            q_crit_m2s=0.0,
            width_m=10.0,
            mechanical_regime="gravity_unstable",
        )
        discharge = model.discharge(feat)
        self.assertTrue(np.isnan(discharge.loc[feat.index[0], "T_index"]))

    def test_missing_water_is_not_a_below_threshold_observation(self):
        feat = pd.DataFrame(
            {"P": [np.nan], "API": [10.0], "Ta_2m": [2.0], "melt_energy": [10.0]},
            index=pd.date_range("2024-01-01", periods=1, freq="h"),
        )
        feat.attrs["api_quantiles"] = (8.0, 18.0)
        model = tg.TriggerModel(
            area_km2=1.0,
            glacier_frac=0.2,
            q_crit_m2s=0.0,
            width_m=10.0,
            mechanical_regime="gravity_unstable",
        )
        discharge = model.discharge(feat)
        self.assertEqual(discharge.iloc[0]["hydraulic_state"], "not_evaluable")

    def test_object_state_tuple_has_unique_ids_and_missing_water_semantics(self):
        count = 84
        gully = pd.DataFrame({
            "Code": [f"g{i:03d}" for i in range(count)],
            "Name": ["same display name"] * count,
            "mechanical_regime": ["water_controlled"] * 78
            + ["gravity_unstable"] * 6,
            "Q_crit_m3s": [10.0] * count,
        })
        missing = pd.DataFrame(threshold.build_object_state_rows(
            gully, np.nan, "below_thermal_reference", 1, 1
        ))
        self.assertEqual(len(missing), count)
        self.assertEqual(missing["object_id"].nunique(), count)
        self.assertEqual(set(missing["object_id"]), set(gully["Code"]))
        self.assertTrue(missing["H"].eq("not_evaluable").all())
        self.assertTrue(missing["ice_mechanics_unresolved"].eq(1).all())
        self.assertTrue(set(threshold.STATE_TUPLE_FIELDS).issubset(missing.columns))

        present = pd.DataFrame(threshold.build_object_state_rows(
            gully, 20.0, "below_thermal_reference", 1, 1
        ))
        self.assertTrue(
            present.loc[:77, "H"].eq("hydraulic_candidate").all()
        )
        self.assertTrue(
            present.loc[78:, "H"].eq("inapplicable").all()
        )

    def test_rank_tests_name_total_and_unit_width_quantities(self):
        frame = pd.DataFrame({
            "Q_crit_m3s": [1., 2., 4., 8., 16., 32., 64., 128.],
            "q_crit_m2s": [1., 4., 2., 8., 4., 16., 8., 32.],
            "规模序": [1., 1., 2., 2., 3., 3., 4., 4.],
            "频率序": [1., 2., 1., 2., 1., 2., 1., 2.],
        })
        total = threshold.rank_association_summary(frame, "Q_crit_m3s")
        unit = threshold.rank_association_summary(frame, "q_crit_m2s")
        self.assertEqual(total["quantity"], "Q_crit_m3s")
        self.assertEqual(unit["quantity"], "q_crit_m2s")
        for result in (total, unit):
            self.assertGreater(result["kendall_vs_scale"]["p值"], 0)
            self.assertGreater(result["spearman_vs_scale"]["p值"], 0)


class ForecastProtocolTests(unittest.TestCase):
    def make_trace(self) -> pd.DataFrame:
        index = pd.date_range(
            forecast.PREISSUE_UTC, periods=25, freq="h"
        )
        frame = pd.DataFrame(
            {"T": 2.0, "SW": 100.0, "RH": 80.0, "P": 0.0},
            index=index,
        )
        frame.loc[forecast.PREISSUE_UTC, ["T", "SW"]] = [-2.0, 0.0]
        return frame

    def make_model(self) -> dict:
        return {
```

```
        "residual_sigma_W_m2": 0.0,
        "residual_ar1_phi": 0.0,
        "residual_quantiles_W_m2": [0.0, 0.0, 0.0],
        "beta": np.array([0.0, 1.0, 0.01, 0.0]),
    }

def test_ar1_uses_only_consecutive_hour_pairs(self):
    index = pd.DatetimeIndex([
        "2024-06-01 00:00",
        "2024-06-01 01:00",
        "2024-06-01 02:00",
        "2024-07-01 00:00",
        "2024-07-01 01:00",
        "2024-07-01 02:00",
    ])
    residual = np.array([1.0, 2.0, 4.0, 100.0, 200.0, 400.0])
    result = forecast.estimate_ar1_residual(residual, index)
    self.assertEqual(result["adjacent_pair_n"], 4)
    self.assertAlmostEqual(result["phi"], 0.99, places=12)
    innovation = np.array([
        2.0 - 0.99 * 1.0,
        4.0 - 0.99 * 2.0,
        200.0 - 0.99 * 100.0,
        400.0 - 0.99 * 200.0,
    ])
    self.assertAlmostEqual(
        result["innovation_sigma"], float(np.std(innovation, ddof=1)), places=12
    )
    self.assertGreater(result["marginal_sigma"], result["innovation_sigma"])

def test_ar1_first_sample_uses_stationary_variance(self):
    class FixedRNG:
        def __init__(self, draws):
            self.draws = iter(draws)

        def normal(self, loc=0.0, scale=1.0):
            return loc + scale * next(self.draws)

    sigma = 2.0
    phi = 0.5
    out = forecast.ar1_residual(FixedRNG([1.0, 2.0, 3.0]), sigma, phi, 3)
    stationary = sigma / np.sqrt(1.0 - phi**2)
    expected = np.array([
        stationary,
        phi * stationary + sigma * 2.0,
        phi * (phi * stationary + sigma * 2.0) + sigma * 3.0,
    ])
    np.testing.assert_allclose(out, expected, rtol=0, atol=1e-14)

def test_source_states_are_mutually_exclusive_and_sum(self):
    records = [
        {"state_tm": "finite"},
        {"state_tm": "no_event"},
        {"state_tm": "active_at_issue"},
        {"state_tm": "not_evaluable"},
        {"state_tm": "right_censored"},
    ]
    summary = forecast.state_summary(records, "state_tm")
    self.assertEqual(summary["total_members"], 5)
    self.assertTrue(summary["state_sum_check"])
    self.assertEqual(sum(summary["counts"].values()), 5)
    self.assertEqual(summary["p_active_at_issue"], 0.2)
    self.assertEqual(summary["p_actionable"], 0.4)

def test_arrival_states_are_mutually_exclusive_and_sum(self):
    records = [
        {"state_ta": "finite"},
        {"state_ta": "no_event"},
        {"state_ta": "active_at_issue_left_censored"},
        {"state_ta": "no_arrival"},
        {"state_ta": "not_evaluable"},
        {"state_ta": "right_censored"},
    ]
    summary = forecast.state_summary(records, "state_ta")
    self.assertEqual(summary["total_members"], 6)
    self.assertTrue(summary["state_sum_check"])
    self.assertEqual(sum(summary["counts"].values()), 6)
    self.assertEqual(summary["p_active_at_issue_left_censored"], 1 / 6)
    self.assertEqual(summary["p_actionable"], 2 / 6)

def test_hour_support_is_exactly_one_hour(self):
    old_n = forecast.N_PER_RUN
    forecast.N_PER_RUN = 2
    try:
        beta = np.array([[0., 1., 0.01, 0.], [0., 1., 0.01, 0.]])
        records, _ = forecast.make_members(
            self.make_trace(), beta, self.make_model(), "run06"
        )
    finally:
        forecast.N_PER_RUN = old_n
    self.assertEqual(records[0]["state_tm"], "finite")
    lower = pd.Timestamp(records[0]["tm_lower_utc"])
    upper = pd.Timestamp(records[0]["tm_upper_utc"])
    self.assertEqual(upper - lower, pd.Timedelta(hours=1))
    self.assertEqual(lower, forecast.ISSUE_UTC)

def test_active_preissue_hour_is_not_reported_as_future_finite(self):
```

```
        old_n = forecast.N_PER_RUN
        forecast.N_PER_RUN = 2
        trace = self.make_trace()
        trace.loc[forecast.PREISSUE_UTC, ["T", "SW"]] = [2.0, 100.0]
        trace.loc[trace.index[1:], ["T", "SW"]] = [-2.0, 0.0]
        try:
            beta = np.array([[0., 1., 0.01, 0.], [0., 1., 0.01, 0.]])
            model = self.make_model()
            model["beta"] = beta.mean(axis=0)
            records, _ = forecast.make_members(trace, beta, model, "run18")
        finally:
            forecast.N_PER_RUN = old_n
        self.assertEqual(records[0]["state_tm"], "active_at_issue")
        self.assertIsNone(records[0]["tm_lower_utc"])

    def test_truncated_future_trajectory_is_right_censored(self):
        old_n = forecast.N_PER_RUN
        forecast.N_PER_RUN = 2
        trace = self.make_trace()
        trace.loc[forecast.PREISSUE_UTC, ["T", "SW"]] = [-2.0, 0.0]
        trace.loc[trace.index[1:], ["T", "SW"]] = [-2.0, 0.0]
        trace.loc[trace.index[10:], ["T", "SW", "RH"]] = np.nan
        trace.attrs["trajectory_end_utc"] = trace.index[9]
        try:
            beta = np.array([[0., 1., 0.01, 0.], [0., 1., 0.01, 0.]])
            records, _ = forecast.make_members(
                trace, beta, self.make_model(), "run18"
            )
        finally:
            forecast.N_PER_RUN = old_n
        self.assertEqual(records[0]["state_tm"], "right_censored")

    def test_missing_before_first_crossing_is_not_evaluable(self):
        old_n = forecast.N_PER_RUN
        forecast.N_PER_RUN = 2
        trace = self.make_trace()
        trace.loc[forecast.PREISSUE_UTC, ["T", "SW"]] = [-2.0, 0.0]
        trace.loc[trace.index[1], "T"] = np.nan
        trace.loc[trace.index[1:3], "SW"] = 0.0
        try:
            beta = np.array([[0., 1., 0.01, 0.], [0., 1., 0.01, 0.]])
            records, _ = forecast.make_members(
                trace, beta, self.make_model(), "run12"
            )
        finally:
            forecast.N_PER_RUN = old_n
        self.assertEqual(records[0]["state_tm"], "not_evaluable")
        self.assertIsNone(records[0]["tm_lower_utc"])

    def test_finite_interval_uses_only_finite_members(self):
        start = forecast.ISSUE_UTC + pd.Timedelta(hours=2)
        records = [
            {
                "state": "finite",
                "lower": start,
                "upper": start + pd.Timedelta(hours=1),
                "event": start + pd.Timedelta(minutes=15),
            }
            for i in range(100)
        ] + [{"state": "no_event", "lower": None, "upper": None, "event": None}]
        result = forecast.finite_interval(records, "lower", "upper", "event", "state")
        self.assertEqual(result["status"], "available")
        self.assertEqual(result["finite_n"], 100)
        self.assertEqual(result["width_hours"], 0.0)

    def test_zero_finite_members_return_all_null_interval_fields(self):
        records = [{"state": "no_event", "lower": None, "upper": None, "event": None}]
        result = forecast.finite_interval(records, "lower", "upper", "event", "state")
        self.assertEqual(result["status"], "not_constructible_zero_finite")
        self.assertEqual(result["finite_n"], 0)
        for field in (
            "lower_time", "median_time", "upper_time", "width_hours",
            "support_envelope_lower_time", "support_envelope_upper_time",
            "support_envelope_width_hours",
            "support_endpoint_95_lower_time", "support_endpoint_95_upper_time",
            "support_endpoint_95_width_hours",
        ):
            self.assertIsNone(result[field])

    def test_one_and_ninety_nine_finite_members_are_insufficient(self):
        start = forecast.ISSUE_UTC + pd.Timedelta(hours=2)
        for count in (1, 99):
            with self.subTest(count=count):
                records = [
                    {
                        "state": "finite",
                        "lower": start,
                        "upper": start + pd.Timedelta(hours=1),
                        "event": start + pd.Timedelta(minutes=30),
                    }
                    for _ in range(count)
                ]
                result = forecast.finite_interval(
                    records, "lower", "upper", "event", "state"
                )
                self.assertEqual(result["status"], "insufficient_finite_members")
                self.assertEqual(result["finite_n"], count)
```

```
            self.assertIsNone(result["lower_time"])
            self.assertIsNone(result["support_envelope_lower_time"])
            self.assertIsNone(result["width_hours"])

    def test_event_time_quantiles_are_not_support_endpoint_envelope(self):
        start = forecast.ISSUE_UTC + pd.Timedelta(hours=2)
        records = [
            {
                "state": "finite",
                "lower": start,
                "upper": start + pd.Timedelta(hours=1),
                "event": start + pd.Timedelta(hours=i / 100.0),
            }
            for i in range(100)
        ]
        result = forecast.finite_interval(records, "lower", "upper", "event", "state")
        self.assertGreater(result["width_hours"], 0.9)
        self.assertLess(result["width_hours"], 1.0)
        self.assertEqual(result["support_envelope_width_hours"], 1.0)

    def test_complete_support_envelope_contains_endpoint_outliers(self):
        start = forecast.ISSUE_UTC + pd.Timedelta(hours=2)
        records = [
            {
                "state": "finite",
                "lower": start,
                "upper": start + pd.Timedelta(hours=1),
                "event": start + pd.Timedelta(minutes=30),
            }
            for _ in range(100)
        ]
        records[0].update(
            lower=start - pd.Timedelta(hours=10),
            upper=start + pd.Timedelta(hours=20),
            event=start + pd.Timedelta(hours=5),
        )
        result = forecast.finite_interval(records, "lower", "upper", "event", "state")
        self.assertEqual(
            pd.Timestamp(result["support_envelope_lower_time"]),
            (start - pd.Timedelta(hours=10)).tz_localize("UTC"),
        )
        self.assertEqual(
            pd.Timestamp(result["support_envelope_upper_time"]),
            (start + pd.Timedelta(hours=20)).tz_localize("UTC"),
        )
        self.assertLess(
            pd.Timestamp(result["support_endpoint_95_upper_time"]),
            (start + pd.Timedelta(hours=2)).tz_localize("UTC"),
        )

    def test_support_envelope_covers_gap_without_calling_it_exact_union(self):
        first = forecast.ISSUE_UTC + pd.Timedelta(hours=2)
        second = forecast.ISSUE_UTC + pd.Timedelta(hours=4)
        records = []
        for start in (first, second):
            records.extend([
                {
                    "state": "finite",
                    "lower": start,
                    "upper": start + pd.Timedelta(hours=1),
                    "event": start + pd.Timedelta(minutes=30),
                }
                for _ in range(50)
            ])
        result = forecast.finite_interval(records, "lower", "upper", "event", "state")
        self.assertEqual(
            pd.Timestamp(result["support_envelope_lower_time"]),
            first.tz_localize("UTC"),
        )
        self.assertEqual(
            pd.Timestamp(result["support_envelope_upper_time"]),
            (second + pd.Timedelta(hours=1)).tz_localize("UTC"),
        )
        self.assertEqual(result["support_envelope_width_hours"], 3.0)
        self.assertIn("not", result["interval_semantics"].lower())

    def test_active_at_issue_arrival_is_left_censored_with_upper_bound(self):
        record = {
            "member_id": "run18-0000",
            "run": "run18",
            "state_tm": "active_at_issue",
            "tm_lower_utc": None,
            "tm_upper_utc": None,
            "tm_event_utc": None,
        }
        config = {
            "id": "reached",
            "arrival_port_s": 600.0,
            "reached_port": True,
            "termination_reason": "stationary_after_injection",
            "final_runout_m": 1000.0,
            "params": {},
            "volume_m3": 1.0,
            "runout_right_censored": False,
            "natural_stop": False,
            "weight": 1.0,
        }
```

```
    geometry = {
        "geometry_options": [
            {"name": "g", "valid": True, "fall_s": 120.0}
        ]
    }
    result = forecast.propagate([record], [config], geometry)[0]
    self.assertEqual(result["state_ta"], "active_at_issue_left_censored")
    self.assertIsNone(result["ta_lower_utc"])
    self.assertEqual(
        pd.Timestamp(result["ta_censor_upper_utc"]),
        forecast.ISSUE_UTC + pd.Timedelta(seconds=720),
    )

def test_arrival_state_precedence_no_arrival_before_left_censoring(self):
    record = {
        "member_id": "run18-0000",
        "run": "run18",
        "state_tm": "active_at_issue",
        "tm_lower_utc": None,
        "tm_upper_utc": None,
        "tm_event_utc": None,
    }
    config = {
        "id": "stop",
        "arrival_port_s": None,
        "reached_port": False,
        "termination_reason": "stationary_after_injection",
        "final_runout_m": 1000.0,
        "params": {},
        "volume_m3": 1.0,
        "runout_right_censored": False,
        "natural_stop": True,
        "weight": 1.0,
    }
    geometry = {
        "geometry_options": [
            {"name": "g", "valid": True, "fall_s": 120.0}
        ]
    }
    result = forecast.propagate([record], [config], geometry)[0]
    self.assertEqual(result["state_ta"], "no_arrival")
    self.assertIsNone(result["ta_censor_upper_utc"])

def test_terminal_enum_separates_six_arrival_cases(self):
    base_geometry = {
        "geometry_options": [
            {"name": "g", "valid": True, "fall_s": 120.0}
        ]
    }
    cases = [
        ("stationary_after_injection", False, False, "no_arrival"),
        ("time_limit", False, False, "right_censored"),
        ("step_limit", False, False, "right_censored"),
        ("stationary_after_injection", False, True, "right_censored"),
        ("time_limit", True, True, "finite"),
        ("step_limit", True, True, "finite"),
    ]
    for reason, reached, domain, expected in cases:
        with self.subTest(reason=reason, reached=reached, domain=domain):
            record = {
                "member_id": "m",
                "state_tm": "finite",
                "tm_lower_utc": pd.Timestamp("2026-08-26T01:00:00+00:00"),
                "tm_upper_utc": pd.Timestamp("2026-08-26T02:00:00+00:00"),
                "tm_event_utc": pd.Timestamp("2026-08-26T01:30:00+00:00"),
            }
            config = {
                "id": "case",
                "arrival_port_s": 600.0 if reached else None,
                "reached_port": reached,
                "termination_reason": reason,
                "domain_censor": domain,
                "runout_right_censored": domain or reason in {
                    "time_limit", "step_limit"
                },
                "natural_stop": reason == "stationary_after_injection",
                "time_limit": reason == "time_limit",
                "step_limit": reason == "step_limit",
                "time_limit_nonarrival": (
                    reason == "time_limit" and not reached
                ),
                "step_limit_nonarrival": (
                    reason == "step_limit" and not reached
                ),
                "domain_censor_nonarrival": (
                    domain and not reached
                ),
                "params": {},
                "volume_m3": 1.0,
                "weight": 1.0,
            }
            result = forecast.propagate([record], [config], base_geometry)[0]
            self.assertEqual(result["state_ta"], expected)

def test_legacy_audit_payload_is_standalone_and_nonoperative(self):
    payload = forecast.build_legacy_audit(
        {
```

```
            "source_file": "求解结果/数据/old.json",
            "source_sha256": "abc",
            "physical_source_inverse": {"conditional": [1, 2]},
            "scope": "old",
        },
        "临时产物/审计.json",
        "def",
    )
    self.assertEqual(payload["status"], "legacy_audit_only_not_used")
    self.assertEqual(payload["source_audit"]["sha256"], "def")
    self.assertEqual(payload["source_audit"]["path"], "临时产物/审计.json")
    self.assertFalse(any(payload["downstream_use"].values()))

def test_local_time_of_port_hit_is_not_double_shifted(self):
    self.assertEqual(
        forecast.iso_local(forecast.HIT_UTC),
        "2026-08-26T10:30:00+08:00",
    )

def test_arrival_interval_lead_bounds_preserve_negative_lead(self):
    interval = {
        "status": "available",
        "lower_time": "2026-08-26T02:30:36.821000+00:00",
        "upper_time": "2026-08-26T03:33:09.275000+00:00",
    }
    lower, upper = forecast.interval_lead_bounds(interval, forecast.HIT_UTC)
    self.assertAlmostEqual(lower, -(1 + 3 / 60 + 9.275 / 3600), places=9)
    self.assertAlmostEqual(upper, -(36.821 / 3600), places=9)
    self.assertLessEqual(lower, upper)

def test_arrival_interval_lead_is_unavailable_without_interval(self):
    lower, upper = forecast.interval_lead_bounds(
        {"status": "not_constructible_zero_finite"},
        forecast.HIT_UTC,
    )
    self.assertIsNone(lower)
    self.assertIsNone(upper)

def test_d8_terminal_flags_separate_natural_and_censored_states(self):
    natural = forecast.terminal_flags("stationary_after_injection", False)
    limited = forecast.terminal_flags("time_limit", False)
    self.assertTrue(natural["natural_stop"])
    self.assertFalse(natural["right_censored"])
    self.assertFalse(limited["natural_stop"])
    self.assertTrue(limited["right_censored"])
    self.assertTrue(limited["time_limit_nonarrival"])
    reached_after_time_limit = forecast.terminal_flags(
        "time_limit", True, reached_port=True
    )
    self.assertTrue(reached_after_time_limit["right_censored"])
    self.assertFalse(reached_after_time_limit["arrival_right_censored"])

def test_runout_terminal_fields_come_from_configuration(self):
    natural = forecast.runout_terminal_fields({
        "termination_reason": "stationary_after_injection",
        "runout_right_censored": False,
        "natural_stop": True,
        "time_limit_nonarrival": False,
    })
    censored = forecast.runout_terminal_fields({
        "termination_reason": "stationary_after_injection",
        "runout_right_censored": True,
        "natural_stop": False,
        "time_limit_nonarrival": False,
    })
    limited = forecast.runout_terminal_fields({
        "termination_reason": "time_limit",
        "runout_right_censored": True,
        "natural_stop": False,
        "time_limit_nonarrival": True,
    })
    self.assertEqual(natural, {
        "natural_stop": True,
        "time_limit": False,
        "step_limit": False,
        "domain_censor": False,
        "time_limit_nonarrival": False,
        "step_limit_nonarrival": False,
        "domain_censor_nonarrival": False,
        "right_censored": False,
    })
    self.assertTrue(censored["right_censored"])
    self.assertFalse(censored["natural_stop"])
    self.assertTrue(limited["time_limit_nonarrival"])

def test_port_coverage_label_does_not_coerce_nan_to_one(self):
    common = {
        "interval_status": "available",
        "lower_time": "2026-08-26T02:30:00+00:00",
        "upper_time": "2026-08-26T03:30:00+00:00",
        "prediction_missing": False,
    }
    self.assertEqual(
        figures.port_coverage_label(pd.Series({
            **common, "definite_cover": np.nan,
        })),
```

```
            "不可估计",
        )
        self.assertEqual(
            figures.port_coverage_label(pd.Series({
                **common, "definite_cover": None,
            })),
            "不可估计",
        )
        self.assertEqual(
            figures.port_coverage_label(pd.Series({
                **common, "definite_cover": False,
            })),
            "0/1",
        )
        self.assertEqual(
            figures.port_coverage_label(pd.Series({
                **common, "definite_cover": True,
            })),
            "1/1",
        )

    def test_no_alarm_branch_is_annotated_without_plotting_nan(self):
        fig, axes = figures.plt.subplots()
        try:
            self.assertFalse(figures.plot_alarm_marker(axes, None))
            self.assertTrue(
                any("未报警" in text.get_text() for text in axes.texts)
            )
        finally:
            figures.plt.close(fig)

    def test_alarm_rule_uses_actionable_mass(self):
        self.assertTrue(forecast.should_alarm({
            "p_finite": 0.3, "p_active_at_issue": 0.2, "p_actionable": 0.5,
        }))
        self.assertFalse(forecast.should_alarm({
            "p_finite": 0.49, "p_active_at_issue": 0.0, "p_actionable": 0.49,
        }))
        self.assertFalse(forecast.should_alarm({
            "p_finite": 0.0, "p_active_at_issue": 0.0,
            "p_right_censored": 1.0, "p_actionable": 0.0,
        }))

    def test_ifs_runs_are_available_before_issue(self):
        manifest = forecast.load_ifs_manifest()
        self.assertEqual(len(manifest["source_files"]), 3)
        self.assertEqual(len(manifest["port_files"]), 1)
        for item in manifest["source_files"]:
            init = pd.Timestamp(item["init_time_utc"])
            self.assertLessEqual(init + pd.Timedelta(hours=6), forecast.ISSUE_UTC)


if __name__ == "__main__":
    unittest.main()
```

test04c_dynamics.py 有限库存动力测试

```
"""Deterministic synthetic checks; no GIS writes or observed-event calibration."""
from __future__ import annotations

import math
import unittest

import numpy as np
from scipy.integrate import solve_ivp

from mmlib import finite_inventory as fi


class DynamicsTests(unittest.TestCase):
    def assertClose(self, left, right, rtol=1e-9, atol=1e-9):
        self.assertTrue(math.isclose(left, right, rel_tol=rtol, abs_tol=atol),
                        f"{left!r} != {right!r}")

    def test_requested_mode_resolution(self):
        self.assertEqual(fi.resolve_mode("M0", 0.7), (0.0, "M1"))
        self.assertEqual(fi.resolve_mode("M1", 0.5), (0.5, "M1"))
        self.assertEqual(fi.resolve_mode("M2", 0.5), (0.5, "M2"))
        self.assertEqual(fi.resolve_mode("M2", 0.0), (0.0, "M1"))
        with self.assertRaises(ValueError):
            fi.resolve_mode("M3", 0.1)
        with self.assertRaises(ValueError):
            fi.resolve_mode("M1", -0.1)

    def test_kappa_zero_energy_line(self):
        sh = np.array([0., 30., 80., 200.])
        z = np.array([100., 95., 89., 70.])
        r = fi.simulate(sh, z, [4., 0., 6.], v0=20, mu=.05, kappa=0)
        for p in r["trajectory"]:
            expected = 400+2*fi.G*(100-p["z_m"]-.05*p["s_h_m"])
            self.assertClose(p["velocity_m_s"]**2, expected)

    def test_ode_all_signs_and_terminal_sides(self):
        for a, k, v, length in [(2., .01, 4., 30.), (2., .01, 30., 30.),
                                (-2., .01, 20., 30.), (0., .01, 20., 30.),
                                (2., 0., 4., 30.), (-2., 0., 20., 30.),
```

```
                        (0., 0., 20., 30.), (1e-12, .01, 20., 30.)]:
        with self.subTest(a=a, k=k, v=v):
            ref = fi.segment(v*v, a, k, length)
            def rhs(t, y):
                return [y[1], a-k*y[1]**2]
            def target(t, y):
                return y[0]-ref["distance_m"]
            target.terminal = True
            target.direction = 1
            ode = solve_ivp(rhs, [0., 2*ref["time_s"]+1], [0., v],
                            events=target, rtol=2e-11, atol=2e-12, max_step=.02)
            self.assertTrue(len(ode.t_events[0]) == 1)
            self.assertClose(ode.t_events[0][0], ref["time_s"], 2e-9)
            self.assertClose(ode.y_events[0][0, 1]**2, ref["w_out_m2_s2"], 2e-9)

    def test_stop_ode_and_finite_time(self):
        a, k, v = -2., .01, 10.
        ref = fi.segment(v*v, a, k, 1000.)
        def rhs(t, y):
            return [y[1], a-k*y[1]**2]
        def stop(t, y):
            return y[1]
        stop.terminal, stop.direction = True, -1
        ode = solve_ivp(rhs, [0, 100.], [0., v], events=stop,
                        rtol=1e-11, atol=1e-12, max_step=.02)
        self.assertTrue(ref["stopped"])
        self.assertClose(ode.t_events[0][0], ref["time_s"])
        self.assertClose(ode.y_events[0][0, 0], ref["distance_m"])

    def test_terminal_exact(self):
        a, k, w = 2., .01, 200.
        r = fi.segment(w, a, k, 50.)
        self.assertClose(r["w_out_m2_s2"], w)
        self.assertClose(r["time_s"], 50/math.sqrt(w))

    def test_degenerate_limits(self):
        base = fi.segment(400., .2, 0., 20.)
        tiny = fi.segment(400., .2, 1e-14, 20.)
        self.assertClose(base["w_out_m2_s2"], tiny["w_out_m2_s2"])
        self.assertClose(base["time_s"], tiny["time_s"])
        zero = fi.segment(400., 0., .01, 20.)
        for a in [-1e-12, 1e-12]:
            r = fi.segment(400., a, .01, 20.)
            self.assertClose(r["w_out_m2_s2"], zero["w_out_m2_s2"])
            self.assertClose(r["time_s"], zero["time_s"])

    def test_near_terminal(self):
        for eps in [-1e-11, 1e-11]:
            r = fi.segment(200*(1+eps), 2., .01, 50.)
            self.assertClose(r["time_s"], 50/math.sqrt(200), 1e-9)

    def test_flat_never_finite_stop(self):
        r = fi.segment(400., 0., .01, 200.)
        self.assertFalse(r["stopped"])
        self.assertGreater(r["w_out_m2_s2"], 0)

    def test_signed_uphill_and_first_stop(self):
        r = fi.simulate([0., 100., 200.], [0., 10., -100.], [4., 0., 6.],
                        v0=10., mu=.1, kappa=0)
        self.assertEqual(r["status"], "first_stop_before_target")
        self.assertLess(r["final_sh_m"], 100)
        self.assertIsNone(r["arrival_s"])
        self.assertClose(r["final_velocity_m_s"], 0)

    def test_zero_speed_target_contact(self):
        L, mu, v = 100., .1, 10.
        sh = v*v/(2*fi.G*mu)
        r = fi.simulate([0., L], [0., 0.], [4., 0., 6.], v0=v, mu=mu,
                        kappa=0, target_sh=sh)
        self.assertEqual(r["status"], "zero_speed_target_contact")
        self.assertIsNone(r["arrival_s"])

    def test_target_inside_segment(self):
        r = fi.simulate([0., 100.], [0., 0.], [4., 0., 6.],
                        v0=10., mu=0., kappa=0., target_sh=37.)
        self.assertClose(r["arrival_s"], 3.7)
        self.assertClose(r["final_sh_m"], 37.)

    def test_stationary_mixing(self):
        r = fi.stationary_mix([4., 2., 6.], 10., [1., 0., 3.])
        self.assertClose(r["mass_before_kg"]*10,
                         r["mass_after_kg"]*r["velocity_after_m_s"])
        self.assertGreater(r["mix_loss_J"], 0)
        self.assertLess(r["velocity_after_m_s"], 10)
        self.assertClose(r["kinetic_residual_J"], 0, atol=1e-8)

    def test_zero_stock_identity(self):
        n = [{"id": "zero", "s_h_m": 50., "rock_m3": 0., "water_m3": 0.}]
        args = ([0., 100.], [20., 0.], [4., 0., 6.])
        a = fi.simulate(*args, mu=.05, kappa=.001)
        b = fi.simulate(*args, nodes=n, mu=.05, kappa=.001)
        self.assertClose(a["arrival_s"], b["arrival_s"])
        self.assertClose(a["final_velocity_m_s"], b["final_velocity_m_s"])

    def test_inventory_once_and_energy(self):
        n = [{"id": "a", "s_h_m": 40., "rock_m3": 2., "water_m3": 3.},
             {"id": "b", "s_h_m": 70., "rock_m3": 1., "water_m3": 2.}]
```

```
        r = fi.simulate([0., 100.], [40., 0.], [4., 0., 6.], nodes=n,
                        mu=.05, kappa=.001)
        self.assertEqual(len(r["node_ledger"]), 2)
        self.assertClose(r["remaining_inventory_mass_kg"], 0)
        self.assertClose(r["mass_residual_kg"], 0)
        self.assertLess(r["energy_relative_residual_max"], 1e-12)
        self.assertLess(r["segment_energy_relative_residual_max"], 1e-12)

    def test_subdivision_fixed_nodes(self):
        n = [{"id": "a", "s_h_m": 33.333333333, "rock_m3": 2., "water_m3": 3.},
             {"id": "b", "s_h_m": 66.666666666, "rock_m3": 1., "water_m3": 2.}]
        args = ([0., 50., 100.], [40., 20., 0.], [4., 0., 6.])
        for mode in ["M0", "M1", "M2"]:
            a = fi.simulate(*args, nodes=n, mode=mode, mu=.05, kappa=.001)
            b = fi.simulate(*args, nodes=n, mode=mode, mu=.05, kappa=.001, subdivisions=7)
            self.assertClose(a["arrival_s"], b["arrival_s"])
            self.assertClose(a["final_velocity_m_s"], b["final_velocity_m_s"])
            self.assertEqual(len(b["node_ledger"]), 2)
            self.assertClose(a["consumed_inventory_mass_kg"], b["consumed_inventory_mass_kg"])

    def test_subdivision_stop(self):
        args = ([0., 100.], [0., 0.], [4., 0., 6.])
        a = fi.simulate(*args, v0=10., mu=.1, kappa=.001)
        b = fi.simulate(*args, v0=10., mu=.1, kappa=.001, subdivisions=13)
        self.assertClose(a["final_sh_m"], b["final_sh_m"])
        self.assertClose(a["stop_time_s"], b["stop_time_s"])

    def test_inverse_terminal_forward(self):
        args = ([0., 40., 100.], [0., 0., 0.], [4., 0., 6.])
        for k in [0., .001]:
            inv = fi.inverse_threshold(*args, mu=.1, kappa=k, vreq=1.)
            v = inv["required_v0_m_s"]
            high = fi.simulate(*args, v0=v*(1+1e-6), mu=.1, kappa=k)
            low = fi.simulate(*args, v0=v*(1-1e-6), mu=.1, kappa=k)
            self.assertTrue(high["reached_target_positive"])
            self.assertGreater(high["final_velocity_m_s"], 1.)
            self.assertTrue(not low["reached_target_positive"] or low["final_velocity_m_s"] < 1.)

    def test_inverse_internal_bottleneck(self):
        args = ([0., 50., 100.], [0., 10., -100.], [4., 0., 6.])
        inv = fi.inverse_threshold(*args, mu=.1, kappa=.001)
        self.assertClose(inv["controlling_boundary_sh_m"], 50.)
        v = inv["required_v0_m_s"]
        a = fi.simulate(*args, v0=v*(1-1e-6), mu=.1, kappa=.001)
        b = fi.simulate(*args, v0=v*(1+1e-6), mu=.1, kappa=.001)
        self.assertFalse(a["reached_target_positive"])
        self.assertTrue(b["reached_target_positive"])

    def test_inverse_with_nodes_and_M2(self):
        args = ([0., 100.], [0., 0.], [4., 0., 6.])
        n = [{"id": "a", "s_h_m": 40., "rock_m3": 2., "water_m3": 3.}]
        for mode in ["M0", "M1", "M2"]:
            inv = fi.inverse_threshold(*args, nodes=n, mu=.1, kappa=.001, mode=mode)
            v = inv["required_v0_m_s"]
            r = fi.simulate(*args, nodes=n, mu=.1, kappa=.001, mode=mode, v0=v)
            self.assertClose(r["final_velocity_m_s"], 1., atol=2e-10)

    def test_timeout_unknown(self):
        r = fi.simulate([0., 100.], [0., 0.], [4., 0., 6.], v0=10,
                        mu=0, kappa=0, time_limit_s=2)
        self.assertEqual(r["status"], "time_limit_unknown")
        self.assertIsNone(r["arrival_s"])
        self.assertClose(r["final_sh_m"], 20.)
        self.assertClose(r["final_time_s"], 2.)

    def test_M2_unique_mapping(self):
        args = ([0., 100.], [20., 0.], [4., 0., 6.])
        n = [{"id": "a", "s_h_m": 50., "water_m3": 3.}]
        p = fi.prepare(*args, nodes=n, mode="M2", mu=.1)
        self.assertClose(p["segments"][0]["mu"], .1)
        self.assertClose(p["segments"][1]["mu"], .06)
        n[0]["water_m3"] = 0.
        p = fi.prepare(*args, nodes=n, mode="M2", mu=.1)
        self.assertTrue(all(s["mu"] == .1 for s in p["segments"]))

    def test_unvisited_inventory_remains(self):
        n = [{"id": "a", "s_h_m": 90., "rock_m3": 2.}]
        r = fi.simulate([0., 100.], [0., 0.], [4., 0., 6.], nodes=n,
                        v0=2, mu=.1, kappa=0)
        self.assertEqual(r["unvisited_node_ids"], ["a"])
        self.assertClose(r["remaining_inventory_mass_kg"], 5300.)
        self.assertClose(r["mass_residual_kg"], 0.)

    def test_no_zero_velocity_restart(self):
        with self.assertRaises(ValueError):
            fi.simulate([0., 100.], [100., 0.], [4., 0., 6.], v0=0)


def run_report():
    suite = unittest.defaultTestLoader.loadTestsFromTestCase(DynamicsTests)
    result = unittest.TestResult()
    suite.run(result)
    return {"tests_run": result.testsRun, "passed": result.wasSuccessful(),
            "failures": [{"test": str(t), "trace": msg} for t, msg in result.failures],
            "errors": [{"test": str(t), "trace": msg} for t, msg in result.errors],
            "test_names": sorted(n for n in dir(DynamicsTests) if n.startswith("test_")),
            "randomness": "none; deterministic synthetic cases"}
```

```

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

`test_scs_event_runoff.py` SCS-CN 事件径流测试

```
"""Independent numeric and boundary checks for event-cumulative SCS-CN."""
from __future__ import annotations

import unittest
from unittest import mock

import numpy as np
import pandas as pd

from mmlib import ranking, trigger as tg


Q20_CN85 = 2.1800981218098032


def features(rain, api=10.0, start="2024-07-01"):
    frame = pd.DataFrame({"P": rain, "API": api},
                         index=pd.date_range(start, periods=len(rain), freq="h"))
    frame.attrs["api_quantiles"] = (8.0, 18.0)
    return frame


def known_event(rain, **kwargs):
    return features([0.0] * 6 + list(rain), **kwargs)


class EventRunoffTests(unittest.TestCase):
    def test_independent_twenty_mm_reference_and_hourly_increments(self):
        out = tg.event_rain_runoff(known_event([5.0] * 4))
        np.testing.assert_allclose(out.runoff_mm.iloc[6:],
                                   [0.0, 0.02337246853590084,
                                    0.6928213511154855, 1.4639043021584168],
                                   rtol=0, atol=1e-12)
        self.assertAlmostEqual(out.Q.iloc[-1], Q20_CN85, places=12)
        self.assertEqual(out.cumulative_P.iloc[-1], 20.0)

    def test_same_event_total_is_invariant_to_rainfall_partition(self):
        for rain in ([20.0], [5.0] * 4, [1.0] * 20):
            with self.subTest(rain=rain):
                out = tg.event_rain_runoff(known_event(rain))
                self.assertAlmostEqual(out.runoff_mm.sum(), Q20_CN85, places=12)
                self.assertTrue(out.runoff_mm.ge(0).all())
                self.assertTrue(out.runoff_mm.le(out.cumulative_P.diff().fillna(0)).all())

    def test_five_dry_hours_keep_event_six_start_new_event(self):
        for dry in (5, 6):
            with self.subTest(dry=dry):
                out = tg.event_rain_runoff(known_event([20.0] + [0.0] * dry + [20.0]))
                first, last = out.iloc[6], out.iloc[-1]
                if dry == 5:
                    self.assertEqual(first.event_id, last.event_id)
                    merged = tg.rain_runoff(known_event([40.0])).sum()
                    self.assertAlmostEqual(out.runoff_mm.sum(), merged, places=12)
                    self.assertEqual(last.cumulative_P, 40.0)
                else:
                    self.assertNotEqual(first.event_id, last.event_id)
                    self.assertAlmostEqual(out.runoff_mm.sum(), 2 * Q20_CN85, places=12)
                    self.assertTrue(out.boundary_confirmed.iloc[-2])
                    self.assertTrue(pd.isna(out.event_id.iloc[-2]))

    def test_only_strict_zero_counts_towards_a_boundary(self):
        for drizzle in (0.1, 1e-12):
            with self.subTest(drizzle=drizzle):
                out = tg.event_rain_runoff(features([0.] * 3 + [drizzle] + [0.] * 3 + [20.]))
                self.assertTrue(np.isnan(out.runoff_mm.iloc[-1]))
                self.assertFalse(out.boundary_confirmed.any())
                self.assertTrue(out.event_id.isna().all())
        out = tg.event_rain_runoff(known_event([0.1] + [0.] * 5 + [20.]))
        self.assertEqual(out.event_id.iloc[6], out.event_id.iloc[-1])
        self.assertAlmostEqual(out.cumulative_P.iloc[-1], 20.1)
        self.assertGreater(out.runoff_mm.sum(), Q20_CN85)

    def test_initial_unknown_is_not_mistaken_for_a_new_event(self):
        out = tg.event_rain_runoff(features([20.] + [0.] * 5 + [20.] + [0.] * 6 + [20.]))
        self.assertTrue(out.runoff_mm.iloc[[0, 6]].isna().all())
        self.assertTrue(out.event_id.iloc[:13].isna().all())
        self.assertTrue(out.state.iloc[:12].eq("unknown_boundary").all())
        self.assertTrue(out.runoff_mm.iloc[1:6].eq(0).all())
        self.assertFalse(out.boundary_known.iloc[5])
        self.assertAlmostEqual(out.runoff_mm.iloc[-1], Q20_CN85, places=12)

    def test_invalid_rain_breaks_even_an_almost_complete_dry_period(self):
        for invalid in (np.nan, -0.1, np.inf, -np.inf):
            with self.subTest(invalid=invalid):
                rain = [0.] * 5 + [invalid] + [0.] * 5 + [20.] + [0.] * 6 + [20.]
                out = tg.event_rain_runoff(features(rain))
                self.assertEqual(out.state.iloc[5], "invalid_rain")
                self.assertTrue(np.isnan(out.runoff_mm.iloc[5]))
                self.assertTrue(np.isnan(out.runoff_mm.iloc[11]))
```

```
            self.assertTrue(pd.isna(out.event_id.iloc[11]))
            self.assertAlmostEqual(out.runoff_mm.iloc[-1], Q20_CN85, places=12)

    def test_missing_rain_invalidates_continuation_without_erasing_known_prefix(self):
        out = tg.event_rain_runoff(known_event([20., np.nan, 20., 0., 0.]))
        self.assertAlmostEqual(out.runoff_mm.iloc[6], Q20_CN85, places=12)
        self.assertTrue(out.runoff_mm.iloc[7:9].isna().all())
        self.assertTrue(out.event_id.iloc[7:].isna().all())
        self.assertTrue(out.cumulative_P.iloc[7:].isna().all())
        self.assertTrue(out.runoff_mm.iloc[9:].eq(0).all())

    def test_cn_uses_previous_hour_api_and_is_frozen_within_event(self):
        for prior_api, expected_cn in ((1., 63.75), (10., 85.), (25., 97.75)):
            with self.subTest(prior_api=prior_api):
                frame = known_event([5., 5., 0., 5., 5.])
                frame.loc[frame.index[5], "API"] = prior_api
                frame.loc[frame.index[6:], "API"] = [100., np.nan, -1., np.inf, 0.]
                out = tg.event_rain_runoff(frame)
                np.testing.assert_allclose(out.event_CN.iloc[6:], expected_cn, rtol=0, atol=1e-12)
                self.assertTrue(out.event_API.iloc[6:].eq(prior_api).all())
                constant = known_event([5., 5., 0., 5., 5.], api=prior_api)
                np.testing.assert_allclose(out.runoff_mm, tg.rain_runoff(constant))

    def test_api_class_boundaries_are_inclusive_and_cn_is_clipped(self):
        for api, cn, expected in ((8., 85., 85.), (18., 85., 85.),
                                  (25., 95., 98.), (1., 20., 30.)):
            with self.subTest(api=api, cn=cn):
                out = tg.event_rain_runoff(known_event([20.], api=api), cn)
                self.assertEqual(out.event_CN.iloc[-1], expected)

    def test_missing_antecedent_api_invalidates_whole_event_then_recovers(self):
        for invalid in (np.nan, np.inf, -np.inf, -1.):
            with self.subTest(invalid=invalid):
                frame = known_event([5., 0., 15.] + [0.] * 6 + [20.])
                frame.loc[frame.index[5], "API"] = invalid
                out = tg.event_rain_runoff(frame)
                self.assertTrue(out.runoff_mm.iloc[[6, 8]].isna().all())
                self.assertTrue(out.state.iloc[6:9].eq("event_api_missing").all())
                self.assertTrue(out.event_CN.iloc[6:9].isna().all())
                self.assertEqual(out.runoff_mm.iloc[7], 0.)
                self.assertEqual(out.cumulative_P.iloc[8], 20.)
                self.assertTrue(np.isnan(out.Q.iloc[8]))
                self.assertAlmostEqual(out.runoff_mm.iloc[-1], Q20_CN85, places=12)
                self.assertNotEqual(out.event_id.iloc[6], out.event_id.iloc[-1])

    def test_year_boundary_and_unclosed_record_do_not_reset_event(self):
        frame = known_event([20., 0., 20.], start="2024-12-31 17:00")
        out = tg.event_rain_runoff(frame)
        self.assertEqual(out.index[6].year, 2024)
        self.assertEqual(out.index[-1].year, 2025)
        self.assertEqual(out.event_id.iloc[6], out.event_id.iloc[-1])
        self.assertEqual(out.cumulative_P.iloc[-1], 40.)
        reference = tg.rain_runoff(known_event([40.])).sum()
        self.assertAlmostEqual(out.runoff_mm.sum(), reference, places=12)
        closed = tg.event_rain_runoff(pd.concat([frame, features([0.] * 6, start="2025-01-01 02:00")]))
        np.testing.assert_allclose(out.runoff_mm, closed.runoff_mm.iloc[:len(out)])

    def test_time_gap_during_dry_hours_restarts_confirmation(self):
        frame = features([0.] * 6 + [20.] + [0.] * 6 + [20.])
        frame.index = frame.index[:3].append(frame.index[3:] + pd.Timedelta(hours=1))
        out = tg.event_rain_runoff(frame)
        self.assertTrue(out.time_gap.iloc[3])
        self.assertTrue(np.isnan(out.runoff_mm.iloc[6]))
        self.assertTrue(pd.isna(out.event_id.iloc[6]))
        self.assertAlmostEqual(out.runoff_mm.iloc[-1], Q20_CN85, places=12)
        pd.testing.assert_index_equal(out.index, frame.index)

    def test_time_gap_during_rain_invalidates_event_and_antecedent(self):
        frame = known_event([20., 20.])
        frame.index = frame.index[:-1].append(pd.DatetimeIndex([frame.index[-1] + pd.Timedelta(hours=1)]))
        out = tg.event_rain_runoff(frame)
        self.assertAlmostEqual(out.runoff_mm.iloc[6], Q20_CN85, places=12)
        self.assertEqual(out.state.iloc[-1], "unknown_boundary")
        self.assertTrue(np.isnan(out.runoff_mm.iloc[-1]))
        self.assertTrue(np.isnan(out.Q.iloc[-1]))

    def test_boundary_sensitivity_three_six_twelve_hours(self):
        frame = features([0.] * 12 + [20.] + [0.] * 6 + [20.])
        for dry in (3, 6, 12):
            with self.subTest(dry=dry):
                out = tg.event_rain_runoff(frame, dry_hours=dry)
                expected = 2 * Q20_CN85 if dry <= 6 else tg.rain_runoff(known_event([40.])).sum()
                self.assertAlmostEqual(out.runoff_mm.sum(), expected, places=12)
                self.assertEqual(out.attrs["rainfall_event_protocol"]["dry_hours"], dry)

    def test_wrapper_diagnostics_and_no_input_mutation(self):
        frame = known_event([20., 0.])
        before = frame.copy(deep=True)
        out = tg.event_rain_runoff(frame)
        pd.testing.assert_series_equal(tg.rain_runoff(frame), out.runoff_mm.rename(None))
        pd.testing.assert_frame_equal(frame, before)
        self.assertEqual(frame.attrs, before.attrs)
        self.assertEqual(str(out.event_id.dtype), "Int64")
        self.assertEqual(out.event_start.sum(), 1)
        self.assertEqual(out.boundary_confirmed.sum(), 1)
        self.assertTrue(out.runoff_mm.iloc[:6].eq(0).all())
        self.assertTrue(out.Q.iloc[:5].isna().all())
```

```
      self.assertEqual(out.Q.iloc[5], 0.)

   def test_invalid_index_and_parameters_are_rejected(self):
      base = known_event([20.])
      indexes = [base.index[::-1], pd.DatetimeIndex([base.index[0]] * len(base)),
              pd.RangeIndex(len(base)), base.index + pd.Timedelta(minutes=30),
              pd.DatetimeIndex([pd.NaT, *base.index[1:]])]
      for index in indexes:
         with self.subTest(index=index):
            frame = base.copy()
            frame.index = index
            with self.assertRaises(ValueError):
               tg.event_rain_runoff(frame)
      for cn in (0., -1., 101., np.inf, np.nan):
         with self.subTest(cn=cn), self.assertRaises(ValueError):
            tg.event_rain_runoff(base, cn=cn)
      for dry in (0, -1, 1.5, True):
         with self.subTest(dry=dry), self.assertRaises(ValueError):
            tg.event_rain_runoff(base, dry_hours=dry)

   def test_timezone_aware_index_and_empty_input(self):
      frame = known_event([20.])
      frame.index = frame.index.tz_localize("Asia/Shanghai")
      out = tg.event_rain_runoff(frame)
      pd.testing.assert_index_equal(frame.index, out.index)
      self.assertAlmostEqual(out.runoff_mm.iloc[-1], Q20_CN85, places=12)
      empty = tg.event_rain_runoff(frame.iloc[:0])
      self.assertTrue(empty.empty)
      self.assertIn("boundary_confirmed", empty.columns)

   def test_routing_keeps_missing_water_and_preserves_known_runoff_volume(self):
      model = tg.TriggerModel(1., 0., 1., 1.)
      frame = known_event([20., np.nan, 0., 0., 0.])
      frame["melt_energy"], frame["Ta_2m"] = 0., 10.
      discharge = model.discharge(frame)
      self.assertTrue(discharge.Q_rain.iloc[7:10].isna().all())
      self.assertTrue(discharge.Q_water.iloc[7:10].isna().all())
      self.assertTrue(discharge.hydraulic_state.iloc[7:10].eq("not_evaluable").all())
      self.assertEqual(discharge.Q_rain.iloc[10], 0.)
      frame = known_event([20., 0., 0., 0.])
      frame["melt_energy"], frame["Ta_2m"] = 0., 10.
      discharge = model.discharge(frame)
      self.assertAlmostEqual(discharge.Q_rain.sum() * 3600 / 1000, Q20_CN85, places=12)

   def test_routing_uses_clock_hours_across_a_missing_index_row(self):
      frame = known_event([20., 0., 0., 0.])
      frame["melt_energy"], frame["Ta_2m"] = 10., 10.
      frame.index = frame.index[:7].append(frame.index[7:] + pd.Timedelta(hours=1))
      discharge = tg.TriggerModel(1., 0.2, 1., 1.).discharge(frame)
      pd.testing.assert_index_equal(discharge.index, frame.index)
      self.assertTrue(discharge.Q_rain.iloc[7:9].isna().all())
      self.assertTrue(discharge.Q_melt.iloc[7:9].isna().all())
      self.assertEqual(discharge.Q_rain.iloc[9], 0.)
      self.assertTrue(np.isfinite(discharge.Q_melt.iloc[9]))


class FoldAPIThresholdTests(unittest.TestCase):
   def test_only_this_folds_2021_2025_warm_season_enters_terciles(self):
      groups = [("2020-07-01", np.full(120, 1e9)),
             ("2024-01-01", np.full(120, 1e6)),
             ("2024-07-01", np.arange(120.)),
             ("2025-07-01", np.full(120, 1e7)),
             ("2026-07-01", np.full(120, 1e8))]
      frame = pd.concat([pd.DataFrame({"API": values}, index=pd.date_range(start, periods=120, freq="h"))
                     for start, values in groups])
      frame.attrs["api_quantiles"] = (1e12, 1e13)
      train = frame.index.year != 2025
      test = ~train
      class ThresholdsCaptured(Exception):
         pass
      with mock.patch.object(tg, "default_model", side_effect=ThresholdsCaptured) as model:
         with self.assertRaises(ThresholdsCaptured):
            ranking._fit_evidence(frame, pd.Series(dtype=float), [], train, test)
      frozen = model.call_args.args[0]
      np.testing.assert_allclose(frozen.attrs["api_quantiles"], [119 / 3, 238 / 3])
      self.assertEqual(frame.attrs["api_quantiles"], (1e12, 1e13))
      pd.testing.assert_index_equal(frozen.index, frame.index)

   def test_insufficient_valid_warm_sample_cannot_fall_back_to_winter(self):
      frame = features([0.] * 120)
      frame["API"] = np.r_[np.arange(99.), np.full(21, np.inf)]
      winter = features([0.] * 120, api=1e6, start="2024-01-01")
      frame = pd.concat([winter, frame])
      train = np.ones(len(frame), dtype=bool)
      with mock.patch.object(tg, "default_model") as model:
         with self.assertRaisesRegex(ValueError, "暖季"):
            ranking._fit_evidence(frame, pd.Series(dtype=float), [], train, ~train)
      model.assert_not_called()


if __name__ == "__main__":
   unittest.main()
```

`test_time_support.py` 时间支持测试

```
"""Checks at the raw-clock boundary, including independent energy reconstruction."""
```

```
import unittest
import numpy as np
import pandas as pd
import step03_forecast as f


class ClockTests(unittest.TestCase):
   def raw(self):
      return pd.DataFrame({"T": [2., 6., 10.], "RH": [40., 60., 80.],
                           "SW": [999., 100., 200.], "P": [99., 1., 2.]},
                          index=pd.date_range("2026-08-26", periods=3, freq="h"))

   def test_variable_sentinels_and_final_support(self):
      raw = self.raw()
      result = f.align_ifs_hourly(raw)
      np.testing.assert_allclose(result[["T", "RH", "SW", "P"]],
                                 [[4., 50., 100., 1.], [8., 70., 200., 2.]])
      self.assertEqual(result.index[-1] + pd.Timedelta(hours=1), raw.index[-1])
      self.assertIn("preceding-hour", result.attrs["time_support"]["SW"])

   def test_missing_hour_does_not_jump_to_next_surviving_row(self):
      result = f.align_ifs_hourly(self.raw().iloc[[0, 2]])
      self.assertTrue(result["T"].isna().all())
      self.assertTrue(pd.isna(result["SW"].iloc[0]))
      self.assertEqual(result["SW"].iloc[1], 200.)

   def test_one_missing_endpoint_is_not_an_hourly_average(self):
      raw = self.raw()
      raw.iloc[1, raw.columns.get_loc("RH")] = np.nan
      self.assertTrue(f.align_ifs_hourly(raw)["RH"].isna().all())

   def test_preissue_hour_uses_issue_end_flux_and_two_states(self):
      raw = self.raw()
      raw.index = pd.date_range(f.PREISSUE_UTC, periods=3, freq="h")
      result = f.align_ifs_hourly(raw)
      self.assertEqual(result.loc[f.PREISSUE_UTC, "SW"], 100.)
      self.assertEqual(result.loc[f.PREISSUE_UTC, "T"], 4.)

   def test_duplicate_and_unsorted_times_rejected(self):
      raw = self.raw()
      for broken in (raw.iloc[[0, 0, 1]], raw.iloc[::-1]):
         with self.assertRaises(ValueError):
            f.align_ifs_hourly(broken)

   def test_invalid_flux_is_missing_not_zero(self):
      raw = self.raw()
      raw.loc[raw.index[1], ["SW", "P"]] = [-1., np.inf]
      result = f.align_ifs_hourly(raw)
      self.assertTrue(result.iloc[0][["SW", "P"]].isna().all())


class HistoricalClockTests(unittest.TestCase):
   def features(self):
      return pd.DataFrame({"Ta_2m": [2., 6., 10.], "RH_2m": [40., 60., 80.],
                           "Rsd": [100., 300., 500.], "Rsu": [0., 0., 0.],
                           "Rld": [0., 0., 0.], "Rlu": [0., 0., 0.],
                           "G_0.1m": [10., 20., 30.], "H": [30., 40., 50.],
                           "LE": [5., 10., 15.], "melt_energy": [9999.]*3},
                          index=pd.date_range("2024-07-01", periods=3, freq="h"))

   def test_energy_target_rebuilt_after_independent_family_alignment(self):
      raw = self.features()
      mixed = f.historical_energy_support(raw, rad_end=True)
      self.assertEqual(mixed.iloc[0]["melt_energy"], 255.)
      all_end = f.historical_energy_support(raw, rad_end=True, soil_end=True, flux_end=True)
      self.assertEqual(all_end.iloc[0]["melt_energy"], 230.)
      self.assertEqual(mixed.iloc[0]["Ta_2m"], 4.)
      self.assertNotEqual(mixed.iloc[0]["melt_energy"], raw.iloc[0]["melt_energy"])

   def test_met_mean_and_point_are_different_declared_scenarios(self):
      raw = self.features()
      self.assertEqual(f.historical_energy_support(raw, met="start_mean").iloc[0]["Ta_2m"], 2.)
      self.assertEqual(f.historical_energy_support(raw, met="end_mean").iloc[0]["Ta_2m"], 6.)
      self.assertEqual(f.historical_energy_support(raw).iloc[0]["Ta_2m"], 4.)
      self.assertFalse(f.historical_energy_support(raw).attrs["station_support_assumption"]["metadata_verified"])

   def test_family_endpoint_missing_propagates_to_energy_target(self):
      raw = self.features()
      raw.iloc[1, raw.columns.get_loc("H")] = np.nan
      result = f.historical_energy_support(raw, flux_end=True)
      self.assertTrue(pd.isna(result.iloc[0]["melt_energy"]))


if __name__ == "__main__":
   unittest.main()
```

`test_v32_data_fixes.py`　数据修复回归测试

```
import unittest
import numpy as np
from mmlib.interval_metrics import canonical_endpoint,covered,interval_score

class IntervalRegressionTests(unittest.TestCase):
 def test_integer_boundary_roundoff_is_covered(self):
  self.assertTrue(bool(covered(13,13.000000000000002,16)))
  self.assertFalse(bool(covered(12.99,13.000000000000002,16)))
```

```
 def test_canonicalization_does_not_round_meaningful_fraction(self):
  np.testing.assert_allclose(canonical_endpoint([13+2e-15,12.3912]),[13,12.3912],rtol=0,atol=0)
 def test_interval_score_penalizes_misses_and_width(self):
  self.assertAlmostEqual(float(interval_score(14,13,16)),3)
  self.assertAlmostEqual(float(interval_score(17,13,16)),43,places=6)
 def test_labels_outside_day_remain_outside(self):
  self.assertFalse(bool(covered(-1,0,24)))
  self.assertFalse(bool(covered(25,0,24)))
if __name__=='__main__':unittest.main()
```